

Code Export

Source Code PDF Report

Generated 2026-05-13 09:37

Root directory C:\Users\LENOVO\Documents\Java_run\Smart_EmployeeManagementSystem\SmartEmployeeManagementSys

Files found 222

Total lines 28,387

Table of Contents

1. Parent: README.md (3 lines)
2. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\conf\logging\simplelogger.properties (31 lines)
3. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\conf\settings.xml (266 lines)
4. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\conf\toolchains.xml (103 lines)
5. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\lib\ext\hazelcast\README.txt (7 lines)
6. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\lib\ext\README.txt (3 lines)
7. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\lib\ext\redisson\README.txt (7 lines)
8. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\lib\jansi-native\README.txt (9 lines)
9. Current: .m2repo\wrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\README.txt (42 lines)
10. Current: .mvn\local-settings.xml (6 lines)
11. Current: .mvn\wrapper\maven-wrapper.properties (4 lines)
12. Current: pom.xml (138 lines)
13. Current: src\main\java\com\example\EmployeeManagementSystem\code\CodeToPdf.java (347 lines)
14. Current: src\main\java\com\example\EmployeeManagementSystem\config\BasicAuthConfig.java (52 lines)
15. Current: src\main\java\com\example\EmployeeManagementSystem\config\JWTauth\JWTAuthConfig.java (202 lines)
16. Current: src\main\java\com\example\EmployeeManagementSystem\config\LeaveTypeDataInitializer.java (44 lines)
17. Current: src\main\java\com\example\EmployeeManagementSystem\config\Seassion\SessionAuthConfig.java (32 lines)
18. Current: src\main\java\com\example\EmployeeManagementSystem\config\swaggerConfig.java (38 lines)
19. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AdminController.java (183 lines)
20. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ApiKeyController.java (51 lines)
21. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AuthController.java (212 lines)
22. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\BillController.java (48 lines)

23. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeliveryController.java (33 lines)

24. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeviceController.java (134 lines)

25. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\EmployeeController.java (59 lines)

26. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java (159 lines)

27. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\HolidayController.java (40 lines)

28. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\InvoiceController.java (43 lines)

29. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java (119 lines)

30. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java (102 lines)

31. Current: src\main\java\com\example\EmployeeManagementSystem\Controller>LoginController.java (17 lines)

32. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ManagerController.java (72 lines)

33. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\NotificationController.java (60 lines)

34. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\OAuthController.java (52 lines)

35. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\RestaurantController.java (124 lines)

36. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerController.java (103 lines)

37. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerTestController.java (63 lines)

38. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ServiceRequestController.java (89 lines)

39. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SessionAuthController.java (183 lines)

40. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SubscriptionController.java (73 lines)

41. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\TotpController.java (124 lines)

42. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorController.java (59 lines)

43. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorNegotiationController.java (64 lines)

44. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AccessTokenRequest.java (18 lines)

45. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ActionDTO.java (34 lines)

46. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminActionDTO.java (34 lines)

47. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminEmployeeDTO.java (61 lines)

48. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ApprovalActionDTO.java (44 lines)

49. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AssignDeviceRequest.java (14 lines)

50. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthRequest.java (28 lines)

51. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthResponse.java (28 lines)

52. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceActivityDTO.java (61 lines)

53. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.java (91 lines)

54. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceDTO.java (55 lines)

55. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceResponseDTO.java (123 lines)

56. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.java (43 lines)

57. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDetails.java (52 lines)

58. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDTO.java (61 lines)

59. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeStatusDTO.java (71 lines)

60. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\HolidayDTO.java (50 lines)

61. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\InvoiceDTO.java (75 lines)

62. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveDashboardResponse.java (122 lines)

63. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveRequestDTO.java (48 lines)

64. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveResponseDTO.java (82 lines)

65. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerDTO.java (43 lines)

66. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerEmployeeDetailsDTO.java (72 lines)

67. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerLeaveResponseDTO.java (73 lines)

68. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\MaternityLeaveRequestDTO.java (25 lines)

69. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\PromoteRequest.java (23 lines)

70. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RentalBillDTO.java (63 lines)

71. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RepairBillDTO.java (72 lines)

72. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RepairDTO.java (60 lines)

73. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RepairLogResponseDTO.java (125 lines)

74. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantDTO.java (42 lines)

75. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantRequest.java (46 lines)

76. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestDTO.java (44 lines)

77. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestResponseDTO.java (116 lines)

78. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionDTO.java (103 lines)

79. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionRequest.java (72 lines)

80. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorDTO.java (33 lines)

81. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationMessageRequest.java (9 lines)

82. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationMessageResponse.java (29 lines)

83. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationRequest.java (31 lines)

84. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationResponse.java (59 lines)

85. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationStatusRequest.java (11 lines)

86. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorRequest.java (47 lines)

87. Current:
src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication.java (16 lines)

88. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ApiKey.java (79 lines)

89. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Delivery.java (93 lines)

90. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Device.java (119 lines)

91. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\DeviceAssignment.java (68 lines)

92. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Employee.java (197 lines)

93. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\FixedHoliday.java (61 lines)

94. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Invoice.java (114 lines)

95. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveEntitlement.java (104 lines)

96. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveRequest.java (105 lines)

97. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveType.java (49 lines)

98. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveWarning.java (58 lines)

99. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Notification.java (42 lines)

100. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RefreshToken.java (62 lines)

101. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RentalBill.java (76 lines)

102. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RepairBill.java (77 lines)

103. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RepairLog.java (116 lines)

104. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Restaurant.java (58 lines)

105. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ServiceRequest.java (141 lines)

106. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Subscription.java (105 lines)

107. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\TotpUser.java (10 lines)

108. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Vendor.java (133 lines)

109. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\VendorNegotiation.java (86 lines)

110. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\VendorNegotiationMessage.java (56 lines)

111. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\YearEndCarryForwardLog.java (62 lines)

112. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\AssignmentStatus.java (8 lines)

113. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java (9 lines)

114. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java (12 lines)

115. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceType.java (15 lines)

116. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Gender.java (5 lines)

117. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\InvoiceStatus.java (8 lines)

118. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\InvoiceType.java (7 lines)

119. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java (9 lines)

120. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java (9 lines)

121. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java (8 lines)

122. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\MessageSenderType.java (7 lines)

123. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\NegotiationStatus.java (11 lines)

124. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\PaymentStatus.java (10 lines)

125. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Permission.java (21 lines)

126. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Role.java (61 lines)

127. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java (7 lines)

128. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestStatus.java (13 lines)

129. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestType.java (8 lines)

130. Current: src\main\java\com\example\EmployeeManagementSystem\Enum>Status.java (8 lines)

131. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java (8 lines)

132. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java (8 lines)

133. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\EmployeeNotFound.java (10 lines)

134. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java

(8 lines)

135. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java
(8 lines)

136. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java
(8 lines)

137. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundException.java (8
lines)

138. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.java (8 lines)

139. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\RentalBillNotFoundException.java (8 lines)

140. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\RepairBillNotFoundException.java (8 lines)

141. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException.java (8 lines)

142. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java
(8 lines)

143. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException.java (7 lines)

144. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java
(7 lines)

145. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\ApiKeyFilter.java (67 lines)

146. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\JwtAuthFilter.java (99 lines)

147. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ApiKeyRepository.java (34
lines)

148. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeliveryRepository.java (23
lines)

149. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java
(22 lines)

150. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceRepository.java (24
lines)

151. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\EmployeeRepo.java (27 lines)

152. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\FixedHolidayRepository.java
(13 lines)

153. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\InvoiceRepository.java (20
lines)

154. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveEntitlementRepository.java (31 lines)

155. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveRequestRepo.java (151

lines)

156. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveTypeRepository.java
(13 lines)

157. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.java
(21 lines)

158. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\NotificationRepository.java
(12 lines)

159. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java
(19 lines)

160. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RentalBillRepository.java (13
lines)

161. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RepairBillRepository.java (18
lines)

162. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RepairLogRepository.java (16
lines)

163. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java
(31 lines)

164. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\ServiceRequestRepository.java (32 lines)

165. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java
(32 lines)

166. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\VendorNegotiationMessageRepository.java
(11 lines)

167. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\VendorNegotiationRepository.java (13 lines)

168. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\VendorRepo.java (16 lines)

169. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\YearEndCarryForwardLogRepository.java
(18 lines)

170. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\DeliveryScheduler.java (30
lines)

171. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveAccrualScheduler.java
(71 lines)

172. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveStatusScheduler.java
(107 lines)

173. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\MonthlyBillScheduler.java (54
lines)

174. Current:
src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationProvider.java (86 lines)

175. Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java

(84 lines)

176. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ApiKeyService.java (52 lines)

177. Current: src\main\java\com\example\EmployeeManagementSystem\Service\BillService.java (134 lines)

178. Current:

src\main\java\com\example\EmployeeManagementSystem\Service\CarryForwardWarningService.java (126 lines)

179. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java (41 lines)

180. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CustomOAuth2UserService.java (87 lines)

181. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryService.java (179 lines)

182. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryTimeService.java (74 lines)

183. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeviceService.java (479 lines)

184. Current: src\main\java\com\example\EmployeeManagementSystem\Service\EmployeeService.java (223 lines)

185. Current: src\main\java\com\example\EmployeeManagementSystem\Service\HolidayService.java (39 lines)

186. Current: src\main\java\com\example\EmployeeManagementSystem\Service\InvoiceService.java (97 lines)

187. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveAccrualService.java (159 lines)

188. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveDashboardService.java (215 lines)

189. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveRequestService.java (594 lines)

190. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ManagerService.java (231 lines)

191. Current: src\main\java\com\example\EmployeeManagementSystem\Service\MaternityLeaveService.java (129 lines)

192. Current: src\main\java\com\example\EmployeeManagementSystem\Service\MealSlotService.java (22 lines)

193. Current: src\main\java\com\example\EmployeeManagementSystem\Service\NotificationService.java (57 lines)

194. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RefreshTokenService.java (41 lines)

195. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RestaurantService.java (162 lines)

196. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ServiceRequestService.java (346 lines)

197. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SickLeaveResetService.java (67 lines)

198. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SubscriptionService.java (228 lines)

199. Current: src\main\java\com\example\EmployeeManagementSystem\Service\TotpService.java (45 lines)

200. Current: src\main\java\com\example\EmployeeManagementSystem\Service\VendorNegotiationService.java (254 lines)

201. Current:
src\main\java\com\example\EmployeeManagementSystem\Service\YearEndCarryForwardService.java (224 lines)

202. Current: src\main\java\com\example\EmployeeManagementSystem\Util\AuthUtil.java (21 lines)

203. Current: src\main\java\com\example\EmployeeManagementSystem\Util\JWTUtil.java (76 lines)

204. Current: src\main\java\com\example\EmployeeManagementSystem\Util\OAuth2SuccessHandler.java (85 lines)

205. Current: src\main\resources\application.yaml (71 lines)

206. Current: src\main\resources\static\dashboard.html (2,503 lines)

207. Current: src\main\resources\static\device_management.html (1,257 lines)

208. Current: src\main\resources\static\employee-dashboard.html (1,776 lines)

209. Current: src\main\resources\static\google.html (669 lines)

210. Current: src\main\resources\static\leave_management.html (812 lines)

211. Current: src\main\resources\static\manager-dashboard.html (2,399 lines)

212. Current: src\main\resources\static\pending.html (85 lines)

213. Current: src\main\resources\static\tech-vendor-dashboard.html (1,902 lines)

214. Current: src\main\resources\static\otp-setup.html (442 lines)

215. Current: src\main\resources\static\vendor-dashboard.html (828 lines)

216. Current: src\main\resources\static\vendor-login.html (725 lines)

217. Current:
src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplicationTests.java (14 lines)

218. Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveAccrualServiceTest.java (190 lines)

219. Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveRequestServiceTest.java (243 lines)

220. Current: src\test\java\com\example\EmployeeManagementSystem\service\ManagerServiceTest.java (76 lines)

221. Current: src\test\java\com\example\EmployeeManagementSystem\service\SickLeaveResetServiceTest.java (135 lines)

222. Current:
src\test\java\com\example\EmployeeManagementSystem\service\YearEndCarryForwardServiceTest.java (137 lines)

1 Parent: README.md

3 lines

```
1 # SmartEmployeeManagementSystem
2 A secure and scalable Spring Boot backend for employee management, leave tracking, and meal subscription
  services with JWT, OAuth2, API key authentication, and role-based access control.
3
```

2 Current: .m2repolwrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\conf\logging\simplele

```
1 # Licensed to the Apache Software Foundation (ASF) under one
2 # or more contributor license agreements. See the NOTICE file
3 # distributed with this work for additional information
4 # regarding copyright ownership. The ASF licenses this file
5 # to you under the Apache License, Version 2.0 (the
6 # "License"); you may not use this file except in compliance
7 # with the License. You may obtain a copy of the License at
8 #
9 # http://www.apache.org/licenses/LICENSE-2.0
10 #
11 # Unless required by applicable law or agreed to in writing,
12 # software distributed under the License is distributed on an
13 # "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
14 # KIND, either express or implied. See the License for the
15 # specific language governing permissions and limitations
16 # under the License.
17
18 org.slf4j.simpleLogger.defaultLogLevel=info
19 org.slf4j.simpleLogger.showDateTime=false
20 org.slf4j.simpleLogger.showThreadName=false
21 org.slf4j.simpleLogger.showLogName=false
22 org.slf4j.simpleLogger.logFile=System.out
23 org.slf4j.simpleLogger.cacheOutputStream=true
24 org.slf4j.simpleLogger.levelInBrackets=true
25 org.slf4j.simpleLogger.log.Sisu=info
26 org.slf4j.simpleLogger.warnLevelString=WARNING
27
28 # MNG-6181: mvn -X also prints all debug logging from HttpClient
29 org.slf4j.simpleLogger.log.org.apache.http=off
30 org.slf4j.simpleLogger.log.org.apache.http.wire=off
31
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <!--
4 Licensed to the Apache Software Foundation (ASF) under one
5 or more contributor license agreements. See the NOTICE file
6 distributed with this work for additional information
7 regarding copyright ownership. The ASF licenses this file
8 to you under the Apache License, Version 2.0 (the
9 "License"); you may not use this file except in compliance
10 with the License. You may obtain a copy of the License at
11
12     http://www.apache.org/licenses/LICENSE-2.0
13
14 Unless required by applicable law or agreed to in writing,
15 software distributed under the License is distributed on an
16 "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 KIND, either express or implied. See the License for the
18 specific language governing permissions and limitations
19 under the License.
20 -->
21
22 <!--
23 | This is the configuration file for Maven. It can be specified at two levels:
24 |
25 | 1. User Level. This settings.xml file provides configuration for a single user,
26 |    and is normally provided in ${user.home}/.m2/settings.xml.
27 |
28 |    NOTE: This location can be overridden with the CLI option:
29 |
30 |        -s /path/to/user/settings.xml
31 |
32 | 2. Global Level. This settings.xml file provides configuration for all Maven
33 |    users on a machine (assuming they're all using the same Maven
34 |    installation). It's normally provided in
35 |    ${maven.conf}/settings.xml.
36 |
37 |    NOTE: This location can be overridden with the CLI option:
38 |
39 |        -gs /path/to/global/settings.xml
40 |
41 | The sections in this sample file are intended to give you a running start at
42 | getting the most out of your Maven installation. Where appropriate, the default
43 | values (values used when the setting is not specified) are provided.
44 |
45 |-->
46 <settings xmlns="http://maven.apache.org/SETTINGS/1.2.0"
47     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
48     xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.2.0 https://maven.apache.org/xsd/settings-
49 1.2.0.xsd">
50     <!-- localRepository
51     | The path to the local repository maven will use to store artifacts.
52     |
53     | Default: ${user.home}/.m2/repository
54     <localRepository>/path/to/local/repo</localRepository>
55     -->
56
57     <!-- interactiveMode
58     | This will determine whether maven prompts you when it needs input. If set to false,
59     | maven will use a sensible default value, perhaps based on some other setting, for
60     | the parameter in question.
61     |
62     | Default: true
63     <interactiveMode>true</interactiveMode>
64     -->
65
66     <!-- offline
67     | Determines whether maven should attempt to connect to the network when executing a build.
68     | This will have an effect on artifact downloads, artifact deployment, and others.
69     |
70     | Default: false
71     <offline>>false</offline>

```

```

71 -->
72
73 <!-- pluginGroups
74 | This is a list of additional group identifiers that will be searched when resolving plugins by their
75 | prefix, i.e.
76 | when invoking a command line like "mvn prefix:goal". Maven will automatically add the group identifiers
77 | "org.apache.maven.plugins" and "org.codehaus.mojo" if these are not already contained in the list.
78 |-->
79 <pluginGroups>
80 <!-- pluginGroup
81 | Specifies a further group identifier to use for plugin lookup.
82 <pluginGroup>com.your.plugins</pluginGroup>
83 -->
84 </pluginGroups>
85
86 <!-- TODO Since when can proxies be selected as depicted? -->
87 <!-- proxies
88 | This is a list of proxies which can be used on this machine to connect to the network.
89 | Unless otherwise specified (by system property or command-line switch), the first proxy
90 | specification in this list marked as active will be used.
91 |-->
92 <proxies>
93 <!-- proxy
94 | Specification for one proxy, to be used in connecting to the network.
95 |
96 <proxy>
97 <id>optional</id>
98 <active>true</active>
99 <protocol>http</protocol>
100 <username>proxyuser</username>
101 <password>proxypass</password>
102 <host>proxy.host.net</host>
103 <port>80</port>
104 <nonProxyHosts>local.net|some.host.com</nonProxyHosts>
105 </proxy>
106 -->
107 </proxies>
108
109 <!-- servers
110 | This is a list of authentication profiles, keyed by the server-id used within the system.
111 | Authentication profiles can be used whenever maven must make a connection to a remote server.
112 |-->
113 <servers>
114 <!-- server
115 | Specifies the authentication information to use when connecting to a particular server, identified
116 | by
117 | a unique name within the system (referred to by the 'id' attribute below).
118 |
119 | NOTE: You should either specify username/password OR privateKey/passphrase, since these pairings are
120 | used together.
121 |
122 <server>
123 <id>deploymentRepo</id>
124 <username>repouser</username>
125 <password>repopwd</password>
126 </server>
127 -->
128
129 <!-- Another sample, using keys to authenticate.
130 <server>
131 <id>siteServer</id>
132 <privateKey>/path/to/private/key</privateKey>
133 <passphrase>optional; leave empty if not used.</passphrase>
134 </server>
135 -->
136 </servers>
137
138 <!-- mirrors
139 | This is a list of mirrors to be used in downloading artifacts from remote repositories.
140 |
141 | It works like this: a POM may declare a repository to use in resolving certain artifacts.
142 | However, this repository may have problems with heavy traffic at times, so people have mirrored
143 | it to several places.
144 |
145 | That repository definition will have a unique id, so we can create a mirror reference for that
146 | repository, to be used as an alternate download site. The mirror site will be the preferred

```

```

145 | server for that repository.
146 |-->
147 <mirrors>
148   <!-- mirror
149   | Specifies a repository mirror site to use instead of a given repository. The repository that
150   | this mirror serves has an ID that matches the mirrorOf element of this mirror. IDs are used
151   | for inheritance and direct lookup purposes, and must be unique across the set of mirrors.
152   |
153   <mirror>
154     <id>mirrorId</id>
155     <mirrorOf>repositoryId</mirrorOf>
156     <name>Human Readable Name for this Mirror.</name>
157     <url>http://my.repository.com/repo/path</url>
158   </mirror>
159   -->
160   <mirror>
161     <id>maven-default-http-blocker</id>
162     <mirrorOf>external:http:*</mirrorOf>
163     <name>Pseudo repository to mirror external repositories initially using HTTP.</name>
164     <url>http://0.0.0.0/</url>
165     <blocked>true</blocked>
166   </mirror>
167 </mirrors>
168
169 <!-- profiles
170 | This is a list of profiles which can be activated in a variety of ways, and which can modify
171 | the build process. Profiles provided in the settings.xml are intended to provide local machine-
172 | specific paths and repository locations which allow the build to work in the local environment.
173 |
174 | For example, if you have an integration testing plugin - like cactus - that needs to know where
175 | your Tomcat instance is installed, you can provide a variable here such that the variable is
176 | dereferenced during the build process to configure the cactus plugin.
177 |
178 | As noted above, profiles can be activated in a variety of ways. One way - the activeProfiles
179 | section of this document (settings.xml) - will be discussed later. Another way essentially
180 | relies on the detection of a property, either matching a particular value for the property,
181 | or merely testing its existence. Profiles can also be activated by JDK version prefix, where a
182 | value of '1.4' might activate a profile when the build is executed on a JDK version of '1.4.2_07'.
183 | Finally, the list of active profiles can be specified directly from the command line.
184 |
185 | NOTE: For profiles defined in the settings.xml, you are restricted to specifying only artifact
186 |       repositories, plugin repositories, and free-form properties to be used as configuration
187 |       variables for plugins in the POM.
188 |
189 |-->
190 <profiles>
191   <!-- profile
192   | Specifies a set of introductions to the build process, to be activated using one or more of the
193   | mechanisms described above. For inheritance purposes, and to activate profiles via
194   <activatedProfiles/>
195   | or the command line, profiles have to have an ID that is unique.
196   |
197   | An encouraged best practice for profile identification is to use a consistent naming convention
198   | for profiles, such as 'env-dev', 'env-test', 'env-production', 'user-jdcasey', 'user-brett', etc.
199   | This will make it more intuitive to understand what the set of introduced profiles is attempting
200   | to accomplish, particularly when you only have a list of profile id's for debug.
201   |
202   | This profile example uses the JDK version to trigger activation, and provides a JDK-specific repo.
203   <profile>
204     <id>jdk-1.4</id>
205
206     <activation>
207       <jdk>1.4</jdk>
208     </activation>
209
210     <repositories>
211       <repository>
212         <id>jdk14</id>
213         <name>Repository for JDK 1.4 builds</name>
214         <url>http://www.myhost.com/maven/jdk14</url>
215         <layout>default</layout>
216         <snapshotPolicy>always</snapshotPolicy>
217       </repository>
218     </repositories>
219   </profile>
220   -->

```

```
220
221 <!--
222 | Here is another profile, activated by the property 'target-env' with a value of 'dev', which
223 | provides a specific path to the Tomcat instance. To use this, your plugin configuration might
224 | hypothetically look like:
225 |
226 | ...
227 | <plugin>
228 |   <groupId>org.myco.myplugins</groupId>
229 |   <artifactId>myplugin</artifactId>
230 |
231 |   <configuration>
232 |     <tomcatLocation>${tomcatPath}</tomcatLocation>
233 |   </configuration>
234 | </plugin>
235 | ...
236 |
237 | NOTE: If you just wanted to inject this configuration whenever someone set 'target-env' to
238 |       anything, you could just leave off the <value/> inside the activation-property.
239 |
240 <profile>
241   <id>env-dev</id>
242
243   <activation>
244     <property>
245       <name>target-env</name>
246       <value>dev</value>
247     </property>
248   </activation>
249
250   <properties>
251     <tomcatPath>/path/to/tomcat/instance</tomcatPath>
252   </properties>
253 </profile>
254 -->
255 </profiles>
256
257 <!-- activeProfiles
258 | List of profiles that are active for all builds.
259 |
260 <activeProfiles>
261   <activeProfile>alwaysActiveProfile</activeProfile>
262   <activeProfile>anotherAlwaysActiveProfile</activeProfile>
263 </activeProfiles>
264 -->
265 </settings>
266
```


4 Current: .m2repolwrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\conf\toolchains.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <!--
4 Licensed to the Apache Software Foundation (ASF) under one
5 or more contributor license agreements. See the NOTICE file
6 distributed with this work for additional information
7 regarding copyright ownership. The ASF licenses this file
8 to you under the Apache License, Version 2.0 (the
9 "License"); you may not use this file except in compliance
10 with the License. You may obtain a copy of the License at
11
12     http://www.apache.org/licenses/LICENSE-2.0
13
14 Unless required by applicable law or agreed to in writing,
15 software distributed under the License is distributed on an
16 "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 KIND, either express or implied. See the License for the
18 specific language governing permissions and limitations
19 under the License.
20 -->
21
22 <!--
23 | This is the toolchains file for Maven. It can be specified at two levels:
24 |
25 | 1. User Level. This toolchains.xml file provides configuration for a single user,
26 |    and is normally provided in ${user.home}/.m2/toolchains.xml.
27 |
28 |    NOTE: This location can be overridden with the CLI option:
29 |
30 |        -t /path/to/user/toolchains.xml
31 |
32 | 2. Global Level. This toolchains.xml file provides configuration for all Maven
33 |    users on a machine (assuming they're all using the same Maven
34 |    installation). It's normally provided in
35 |    ${maven.conf}/toolchains.xml.
36 |
37 |    NOTE: This location can be overridden with the CLI option:
38 |
39 |        -gt /path/to/global/toolchains.xml
40 |
41 | The sections in this sample file are intended to give you a running start at
42 | getting the most out of your Maven installation.
43 |-->
44 <toolchains xmlns="http://maven.apache.org/TOOLCHAINS/1.1.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
45 instance"
46     xsi:schemaLocation="http://maven.apache.org/TOOLCHAINS/1.1.0 http://maven.apache.org/xsd/toolchains-
47 1.1.0.xsd">
48     <!--
49     | With toolchains you can refer to installations on your system. This
50     | way you don't have to hardcode paths in your pom.xml.
51     |
52     | Every toolchain consist of 3 elements:
53     | * type: the type of tool. An often used value is 'jdk'. Toolchains-aware
54     |    plugins should document which type you must use.
55     |
56     | * provides: A list of key/value-pairs.
57     |    Based on the toolchain-configuration in the pom.xml Maven will search for
58     |    matching <provides/> configuration. You can decide for yourself which key-value
59     |    pairs to use. Often used keys are 'version', 'vendor' and 'arch'. By default
60     |    the version has a special meaning. If you configured in the pom.xml '1.5'
61     |    Maven will search for 1.5 and above.
62     |
63     | * configuration: Additional configuration for this tool.
64     |    Look for documentation of the toolchains-aware plugin which configuration elements
65     |    can be used.
66     |
67     | See also https://maven.apache.org/guides/mini/guide-using-toolchains.html
68     |
69     | General example
```

```
70 <toolchain>
71 <type/>
72 <provides>
73 <version>1.0</version>
74 </provides>
75 <configuration/>
76 </toolchain>
77
78 | JDK examples
79
80 <toolchain>
81 <type>jdk</type>
82 <provides>
83 <version>1.5</version>
84 <vendor>sun</vendor>
85 </provides>
86 <configuration>
87 <jdkHome>/path/to/jdk/1.5</jdkHome>
88 </configuration>
89 </toolchain>
90 <toolchain>
91 <type>jdk</type>
92 <provides>
93 <version>1.6</version>
94 <vendor>sun</vendor>
95 </provides>
96 <configuration>
97 <jdkHome>/path/to/jdk/1.6</jdkHome>
98 </configuration>
99 </toolchain>
100
101 -->
102
103 </toolchains>
```

5 Current: .m2repolwrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\lib\ext\hazelcast\REA

1 This directory is intended to contain Hazelcast [1] JARs for Maven Resolver Named Locks using Hazelcast.
2
3 See here [2] on how to add necessary JARs.
4
5 [1] <https://github.com/hazelcast/hazelcast>
6 [2] <https://maven.apache.org/resolver/maven-resolver-named-locks-hazelcast/index.html#installation-testing>
7

6 Current: .m2repolwrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\lib\ext\README.txt

- 1 Use this directory to add third party extensions to Maven Core. These extensions can either extend or override
- 2 Maven's default implementation.
- 3

7 Current: .m2repolwrapper\dists\apache-maven-3.9.14\ed7edd442f634ac1c1ef5ba2b61b6d690b5221091f1a8e1123f5fadcc967520d\lib\ext\redisson\README

1 This directory is intended to contain Redisson [1] JARs for Maven Resolver Named Locks using Redisson.

2

3 See here [2] on how to add necessary JARs.

4

5 [1] <https://github.com/redisson/redisson>

6 [2] <https://maven.apache.org/resolver/maven-resolver-named-locks-redisson/index.html#installation-testing>

7

```
1 This directory contains Jansi native libraries extracted from Jansi JAR.
2
3 You can add your own build for platforms not natively supported by Jansi.
4 See here [1] on how to compile for your platform and and here [2] how libraries
5 follow Jansi's directory and filename conventions.
6
7 [1] https://github.com/fusesource/jansi/tree/master/src/main/native
8 [2]
9 https://github.com/fusesource/jansi/blob/321a8ff71c731e10f4ea05c607860180276b2215/src/main/java/org/fusesource/jansi/internal/OSInfo.java
```

```
1
2           Apache Maven
3
4  What is it?
5  -----
6
7  Maven is a software project management and comprehension tool. Based on
8  the concept of a Project Object Model (POM), Maven can manage a project's
9  build, reporting and documentation from a central piece of information.
10
11  Documentation
12  -----
13
14  The most up-to-date documentation can be found at https://maven.apache.org/.
15
16  Release Notes
17  -----
18
19  The full list of changes, system requirements and related can be found at
20  https://maven.apache.org/docs/history.html.
21
22  Installing Maven
23  -----
24
25  For complete documentation see https://maven.apache.org/download.html#Installation
26
27  Licensing
28  -----
29
30  Please see the file called LICENSE.
31
32  Maven URLs
33  -----
34
35  Home Page:      https://maven.apache.org/
36  Downloads:      https://maven.apache.org/download.html
37  Release Notes:  https://maven.apache.org/docs/history.html
38  Mailing Lists:  https://maven.apache.org/mailling-lists.html
39  Source Code:    https://gitbox.apache.org/repos/asf/maven.git
40  Issue Tracking: https://issues.apache.org/jira/browse/MNG
41  Wiki:           https://cwiki.apache.org/confluence/display/MAVEN/
42  Available Plugins: https://maven.apache.org/plugins/
```

```
1 <settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0 https://maven.apache.org/xsd/settings-
1.0.0.xsd">
4     <localRepository>${user.dir}/.m2repo/repository</localRepository>
5 </settings>
6
```


11 Current: .mvn\wrapper\maven-wrapper.properties

4 lines

```
1 wrapperVersion=3.3.4
2 distributionType=only-script
3 distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.14/apache-maven-
3.9.14-bin.zip
4
```

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4      <modelVersion>4.0.0</modelVersion>
5      <parent>
6          <groupId>org.springframework.boot</groupId>
7          <artifactId>spring-boot-starter-parent</artifactId>
8          <version>4.0.5</version>
9          <relativePath/> <!-- lookup parent from repository -->
10     </parent>
11     <groupId>com.example</groupId>
12     <artifactId>EmployeeLeaveManagementSystem</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14     <name/>
15     <description/>
16     <url/>
17     <licenses>
18         <license/>
19     </licenses>
20     <developers>
21         <developer/>
22     </developers>
23     <scm>
24         <connection/>
25         <developerConnection/>
26         <tag/>
27         <url/>
28     </scm>
29     <properties>
30         <java.version>21</java.version>
31     </properties>
32     <dependencies>
33         <dependency>
34             <groupId>org.springframework.boot</groupId>
35             <artifactId>spring-boot-starter-data-jpa</artifactId>
36         </dependency>
37         <dependency>
38             <groupId>org.springframework.boot</groupId>
39             <artifactId>spring-boot-starter-webmvc</artifactId>
40         </dependency>
41         <dependency>
42             <groupId>org.springdoc</groupId>
43             <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
44             <version>3.0.2</version>
45         </dependency>
46         <dependency>
47             <groupId>dev.samstevens.totp</groupId>
48             <artifactId>totp-spring-boot-starter</artifactId>
49             <version>1.7.1</version>
50         </dependency>
51         <dependency>
52             <groupId>org.springframework.session</groupId>
53             <artifactId>spring-session-core</artifactId>
54         </dependency>
55         <!-- TOTP -->
56         <dependency>
57             <groupId>dev.samstevens.totp</groupId>
58             <artifactId>totp-spring-boot-starter</artifactId>
59             <version>1.7.1</version>
60         </dependency>
61         <!-- -->
62         <dependency>
63             <groupId>org.springframework.boot</groupId>
64             <artifactId>spring-boot-devtools</artifactId>
65             <scope>runtime</scope>
66             <optional>true</optional>
67         </dependency>
68         <dependency>
69             <groupId>com.mysql</groupId>
70             <artifactId>mysql-connector-j</artifactId>
71             <scope>runtime</scope>
72     </dependencies>

```

```

73     <dependency>
74         <groupId>org.springframework.boot</groupId>
75         <artifactId>spring-boot-starter-data-jpa-test</artifactId>
76         <scope>test</scope>
77     </dependency>
78     <dependency>
79         <groupId>com.itextpdf</groupId>
80         <artifactId>layout</artifactId>
81         <version>7.2.5</version>
82     </dependency>
83     <dependency>
84         <groupId>org.springframework.boot</groupId>
85         <artifactId>spring-boot-starter-security</artifactId>
86     </dependency>
87
88     <dependency>
89         <groupId>org.springframework.boot</groupId>
90         <artifactId>spring-boot-starter-security-test</artifactId>
91         <scope>test</scope>
92     </dependency>
93     <dependency>
94         <groupId>org.springframework.boot</groupId>
95         <artifactId>spring-boot-starter-webmvc-test</artifactId>
96         <scope>test</scope>
97     </dependency>
98     <dependency>
99         <groupId>io.jsonwebtoken</groupId>
100        <artifactId>jjwt-api</artifactId>
101        <version>0.13.0</version>
102        <scope>compile</scope>
103    </dependency>
104    <dependency>
105        <groupId>io.jsonwebtoken</groupId>
106        <artifactId>jjwt-impl</artifactId>
107        <version>0.13.0</version>
108        <scope>runtime</scope>
109    </dependency>
110    <dependency>
111        <groupId>io.jsonwebtoken</groupId>
112        <artifactId>jjwt-jackson</artifactId>
113        <version>0.13.0</version>
114        <scope>runtime</scope>
115    </dependency>
116    <dependency>
117        <groupId>org.springframework.boot</groupId>
118        <artifactId>spring-boot-starter-security-oauth2-client</artifactId>
119    </dependency>
120
121    <dependency>
122        <groupId>org.springframework.boot</groupId>
123        <artifactId>spring-boot-starter-security-oauth2-client-test</artifactId>
124        <scope>test</scope>
125    </dependency>
126 </dependencies>
127
128 <build>
129     <plugins>
130         <plugin>
131             <groupId>org.springframework.boot</groupId>
132             <artifactId>spring-boot-maven-plugin</artifactId>
133         </plugin>
134     </plugins>
135 </build>
136
137 </project>
138

```

```

1 package com.example.EmployeeManagementSystem.code;
2
3 import com.itextpdf.kernel.colors.ColorConstants;
4 import com.itextpdf.kernel.colors.DeviceRgb;
5 import com.itextpdf.kernel.font.PdfFont;
6 import com.itextpdf.kernel.font.PdfFontFactory;
7 import com.itextpdf.kernel.geom.PageSize;
8 import com.itextpdf.kernel.pdf.PdfDocument;
9 import com.itextpdf.kernel.pdf.PdfWriter;
10 import com.itextpdf.layout.Document;
11 import com.itextpdf.layout.element.AreaBreak;
12 import com.itextpdf.layout.element.Cell;
13 import com.itextpdf.layout.element.Paragraph;
14 import com.itextpdf.layout.element.Table;
15 import com.itextpdf.layout.element.Text;
16 import com.itextpdf.layout.properties.AreaBreakType;
17 import com.itextpdf.layout.properties.UnitValue;
18 import com.itextpdf.io.font.constants.StandardFonts;
19
20 import java.io.File;
21 import java.io.IOException;
22 import java.nio.file.*;
23 import java.nio.file.attribute.BasicFileAttributes;
24 import java.time.LocalDateTime;
25 import java.time.format.DateTimeFormatter;
26 import java.util.*;
27
28 /**
29  * CodeToPdf - Extracts all source code files from the current, parent,
30  * and immediate child directories into a formatted PDF document.
31  *
32  * Dependencies (add to your pom.xml or build.gradle):
33  *   com.itextpdf:itext7-core:7.2.5 (or itext7 bundle)
34  *
35  * Compile:
36  *   javac -cp itext7-core-7.2.5.jar:. CodeToPdf.java
37  *
38  * Run:
39  *   java -cp itext7-core-7.2.5.jar:. CodeToPdf
40  *
41  * Or with Maven:
42  *   mvn compile exec:java -Dexec.mainClass=CodeToPdf
43  */
44 public class CodeToPdf {
45
46     // Configuration
47
48     /** Output PDF filename */
49     private static final String OUTPUT_FILE = "code_export.pdf";
50
51     /** Maximum file size to include (skip very large generated files) */
52     private static final long MAX_FILE_SIZE_BYTES = 500_000; // 500 KB
53
54     /** Source code file extensions to include */
55     private static final Set<String> CODE_EXTENSIONS = new LinkedHashSet<>(Arrays.asList(
56         // JVM languages
57         ".java", ".kt", ".groovy", ".scala", ".clj",
58         // Web
59         ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".scss", ".less",
60         // Python / Ruby / PHP
61         ".py", ".rb", ".php",
62         // Systems / low-level
63         ".c", ".cpp", ".cc", ".h", ".hpp", ".rs", ".go", ".swift",
64         // Data / config
65         ".xml", ".json", ".yaml", ".yml", ".toml", ".properties", ".env",
66         // Shell / scripts
67         ".sh", ".bash", ".zsh", ".bat", ".ps1",
68         // Build
69         ".gradle", ".cmake", "Makefile", "Dockerfile",
70         // Other
71         ".sql", ".md", ".txt"

```

```

72     });
73
74     /** Directories to skip entirely */
75     private static final Set<String> SKIP_DIRS = new HashSet<>(Arrays.asList(
76         ".git", ".svn", ".hg", "node_modules", "target", "build",
77         "out", ".idea", ".vscode", "__pycache__", ".gradle", "dist", "vendor"
78     ));
79
80     // Colors
81     private static final DeviceRgb COLOR_HEADER_BG = new DeviceRgb(0x18, 0x5F, 0xA5); // blue
82     private static final DeviceRgb COLOR_HEADER_TEXT = new DeviceRgb(0xFF, 0xFF, 0xFF);
83     private static final DeviceRgb COLOR_FILE_TITLE = new DeviceRgb(0x18, 0x5F, 0xA5);
84     private static final DeviceRgb COLOR_CODE_BG = new DeviceRgb(0xF7, 0xF7, 0xF7);
85     private static final DeviceRgb COLOR_LINE_NUM = new DeviceRgb(0x99, 0x99, 0x99);
86     private static final DeviceRgb COLOR_TOC_HEADER = new DeviceRgb(0x18, 0x5F, 0xA5);
87
88     // Font sizes
89     private static final float FONT_TITLE = 20f;
90     private static final float FONT_SECTION = 13f;
91     private static final float FONT_FILE = 11f;
92     private static final float FONT_CODE = 7.5f;
93     private static final float FONT_TOC = 10f;
94
95     // State
96     private final List<FileEntry> fileEntries = new ArrayList<>();
97     private int totalLines = 0;
98
99     //
100
101     public static void main(String[] args) throws Exception {
102         new CodeToPdf().run();
103     }
104
105     private void run() throws Exception {
106         Path currentDir = Paths.get("").toAbsolutePath();
107         Path parentDir = currentDir.getParent();
108
109         System.out.println("=== Code to PDF Extractor ===");
110         System.out.println("Current directory : " + currentDir);
111         System.out.println("Parent directory : " + (parentDir != null ? parentDir : "(none)"));
112         System.out.println();
113
114         // 1. Collect files
115
116         // Parent directory (top-level files only, not recursive)
117         if (parentDir != null && Files.exists(parentDir)) {
118             collectFiles(parentDir, false, "Parent");
119         }
120
121         // Current directory (recursive – includes all child subdirectories)
122         collectFiles(currentDir, true, "Current");
123
124         System.out.printf("%nFound %d code file(s) with %,d total lines.%n",
125             fileEntries.size(), totalLines);
126
127         if (fileEntries.isEmpty()) {
128             System.out.println("No code files found. Exiting.");
129             return;
130         }
131
132         // 2. Generate PDF
133         generatePdf(currentDir);
134
135         System.out.println("PDF saved to: " + Paths.get(OUTPUT_FILE).toAbsolutePath());
136     }
137
138     // File Collection
139
140     private void collectFiles(Path root, boolean recursive, String scope) throws IOException {
141         if (!Files.exists(root)) return;
142
143         Files.walkFileTree(root, new SimpleFileVisitor<>() {
144             @Override
145             public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
146                 String name = dir.getFileName() == null ? "" : dir.getFileName().toString();
147                 // Never recurse into skip-dirs; always allow the root itself

```

```

148         if (!dir.equals(root) && SKIP_DIRS.contains(name)) {
149             return FileVisitResult.SKIP_SUBTREE;
150         }
151         // If not recursive, skip any subdirectory of root
152         if (!recursive && !dir.equals(root)) {
153             return FileVisitResult.SKIP_SUBTREE;
154         }
155         return FileVisitResult.CONTINUE;
156     }
157
158     @Override
159     public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
160         if (isCodeFile(file) && attrs.size() <= MAX_FILE_SIZE_BYTES) {
161             String content = Files.readString(file);
162             int lines = content.split("\n", -1).length;
163             String relativePath = scope + ": " + root.relativize(file);
164             fileEntries.add(new FileEntry(relativePath, file, content, lines));
165             totalLines += lines;
166             System.out.println("  " + relativePath + " (" + lines + " lines)");
167         }
168         return FileVisitResult.CONTINUE;
169     }
170 });
171 }
172
173 private boolean isCodeFile(Path file) {
174     String name = file.getFileName().toString();
175     // Check for exact filename match (e.g. "Makefile", "Dockerfile")
176     if (CODE_EXTENSIONS.contains(name)) return true;
177     // Check extension
178     int dot = name.lastIndexOf('.');
179     if (dot >= 0) {
180         return CODE_EXTENSIONS.contains(name.substring(dot));
181     }
182     return false;
183 }
184
185 // PDF Generation
186
187 private void generatePdf(Path currentDir) throws IOException {
188     PdfWriter writer = new PdfWriter(OUTPUT_FILE);
189     PdfDocument pdfDoc = new PdfDocument(writer);
190     Document doc = new Document(pdfDoc, PageSize.A4);
191     doc.setMargins(40, 40, 40, 40);
192
193     PdfFont sansFont = PdfFontFactory.createFont(StandardFonts.HELVETICA);
194     PdfFont boldFont = PdfFontFactory.createFont(StandardFonts.HELVETICA_BOLD);
195     PdfFont monoFont = PdfFontFactory.createFont(StandardFonts.COURIER);
196
197     // Cover page
198     addCoverPage(doc, sansFont, boldFont, currentDir);
199     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
200
201     // Table of contents
202     addTableOfContents(doc, sansFont, boldFont);
203     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
204
205     // Code sections
206     for (int i = 0; i < fileEntries.size(); i++) {
207         FileEntry entry = fileEntries.get(i);
208         addCodeSection(doc, entry, i + 1, sansFont, boldFont, monoFont);
209         if (i < fileEntries.size() - 1) {
210             doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
211         }
212     }
213
214     doc.close();
215 }
216
217 private void addCoverPage(Document doc, PdfFont sansFont, PdfFont boldFont, Path dir)
218     throws IOException {
219
220     doc.add(new Paragraph("\n\n\n"));
221
222     // Big title
223     doc.add(new Paragraph("Code Export"))

```

```

224         .setFont(boldFont).setFontSize(FONT_TITLE + 10)
225         .setFontColor(COLOR_FILE_TITLE)
226         .setMarginBottom(6));
227
228     doc.add(new Paragraph("Source Code PDF Report")
229         .setFont(sansFont).setFontSize(FONT_SECTION)
230         .setFontColor(ColorConstants.DARK_GRAY)
231         .setMarginBottom(24));
232
233     // Stats table
234     Table stats = new Table(UnitValue.createPercentArray(new float[]{40, 60}))
235         .setWidth(UnitValue.createPercentValue(80));
236     addStatRow(stats, "Generated",      LocalDateTime.now()
237         .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), sansFont, boldFont);
238     addStatRow(stats, "Root directory", dir.toString(), sansFont, boldFont);
239     addStatRow(stats, "Files found",   String.valueOf(fileEntries.size()), sansFont, boldFont);
240     addStatRow(stats, "Total lines",   String.format("%,d", totalLines), sansFont, boldFont);
241     doc.add(stats);
242 }
243
244 private void addStatRow(Table t, String label, String value,
245     PdfFont sans, PdfFont bold) {
246     t.addCell(new Cell().add(new Paragraph(label)
247         .setFont(bold).setFontSize(10)
248         .setFontColor(ColorConstants.DARK_GRAY))
249         .setBorder(null).setPaddingBottom(6));
250     t.addCell(new Cell().add(new Paragraph(value)
251         .setFont(sans).setFontSize(10))
252         .setBorder(null).setPaddingBottom(6));
253 }
254
255 private void addTableOfContents(Document doc, PdfFont sansFont, PdfFont boldFont)
256     throws IOException {
257     doc.add(new Paragraph("Table of Contents")
258         .setFont(boldFont).setFontSize(FONT_SECTION + 2)
259         .setFontColor(COLOR_TOC_HEADER)
260         .setMarginBottom(12));
261
262     for (int i = 0; i < fileEntries.size(); i++) {
263         FileEntry entry = fileEntries.get(i);
264         String label = String.format("%d.  %s  (%,d lines)",
265             i + 1, entry.relativePath, entry.lineCount);
266         doc.add(new Paragraph(label)
267             .setFont(sansFont).setFontSize(FONT_TOC)
268             .setMarginBottom(3));
269     }
270 }
271
272 private void addCodeSection(Document doc, FileEntry entry, int index,
273     PdfFont sansFont, PdfFont boldFont, PdfFont monoFont)
274     throws IOException {
275
276     // Section header bar
277     Table header = new Table(UnitValue.createPercentArray(new float[]{5, 75, 20}))
278         .setWidth(UnitValue.createPercentValue(100))
279         .setMarginBottom(8);
280
281     header.addCell(new Cell().add(new Paragraph(String.valueOf(index))
282         .setFont(boldFont).setFontSize(FONT_FILE)
283         .setFontColor(COLOR_HEADER_TEXT))
284         .setBackgroundColor(COLOR_HEADER_BG)
285         .setBorder(null).setPadding(6));
286
287     header.addCell(new Cell().add(new Paragraph(entry.relativePath)
288         .setFont(boldFont).setFontSize(FONT_FILE)
289         .setFontColor(COLOR_HEADER_TEXT))
290         .setBackgroundColor(COLOR_HEADER_BG)
291         .setBorder(null).setPadding(6));
292
293     header.addCell(new Cell().add(new Paragraph(entry.lineCount + " lines")
294         .setFont(sansFont).setFontSize(FONT_FILE - 1)
295         .setFontColor(COLOR_HEADER_TEXT))
296         .setBackgroundColor(COLOR_HEADER_BG)
297         .setBorder(null).setPadding(6));
298
299     doc.add(header);

```

```

300
301     // Code block with line numbers
302     String[] lines = entry.content.split("\n", -1);
303     int padWidth = String.valueOf(lines.length).length();
304
305     for (int ln = 0; ln < lines.length; ln++) {
306         String lineNum = String.format("%" + padWidth + "d", ln + 1);
307         String codeLine = lines[ln]
308             .replace("\t", "    ") // expand tabs
309             .replace("\r", "");    // strip CR
310
311         // Truncate very long lines
312         if (codeLine.length() > 200) {
313             codeLine = codeLine.substring(0, 197) + "...";
314         }
315
316         Text numText = new Text(lineNum + " ").setFont(monoFont)
317             .setFontSize(FONT_CODE).setFontColors(COLOR_LINE_NUM);
318         Text codeText = new Text(codeLine).setFont(monoFont)
319             .setFontSize(FONT_CODE).setFontColors(ColorConstants.BLACK);
320
321         Paragraph p = new Paragraph().add(numText).add(codeText)
322             .setBackgroundColor(ln % 2 == 0 ? COLOR_CODE_BG : ColorConstants.WHITE)
323             .setMargin(0).setPaddingLeft(4).setPaddingRight(4)
324             .setMultipliedLeading(1.3f)
325             .setFixedLeading(FONT_CODE + 2.5f);
326
327         doc.add(p);
328     }
329 }
330
331 // Data class
332
333 private static class FileEntry {
334     final String relativePath;
335     final Path absolutePath;
336     final String content;
337     final int lineCount;
338
339     FileEntry(String relativePath, Path absolutePath, String content, int lineCount) {
340         this.relativePath = relativePath;
341         this.absolutePath = absolutePath;
342         this.content = content;
343         this.lineCount = lineCount;
344     }
345 }
346 }
347

```



```

1 //package com.example.EmployeeManagementSystem.config;
2 //
3 //import org.springframework.beans.factory.annotation.Qualifier;
4 //import org.springframework.context.annotation.Bean;
5 //import org.springframework.context.annotation.Configuration;
6 //import org.springframework.core.annotation.Order;
7 //import org.springframework.http.HttpMethod;
8 //import org.springframework.security.authentication.AuthenticationManager;
9 //import org.springframework.security.authentication.ProviderManager;
10 //import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
11 //import org.springframework.security.config.Customizer;
12 //import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
13 //import org.springframework.security.config.annotation.web.builders.HttpSecurity;
14 //import org.springframework.security.core.userdetails.UserDetailsService;
15 //import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
16 //import org.springframework.security.crypto.password.PasswordEncoder;
17 //import org.springframework.security.web.SecurityFilterChain;
18 //
19 //
20 // @Configuration
21 // @EnableMethodSecurity
22 // public class BasicAuthConfig {
23 //     @Bean
24 //     @Order(3)
25 //     public SecurityFilterChain basicAuth(HttpSecurity security){
26 //         security
27 //             .csrf(csrf-> csrf.disable())
28 //             .authorizeHttpRequests(
29 //                 auth->auth
30 //                     .requestMatchers("/employee/register","/employee/register/manager").permitAll()
31 //                     .requestMatchers(HttpMethod.POST,"/vendors").permitAll()
32 //                     .anyRequest().authenticated()
33 //             )
34 //             .httpBasic(Customizer.withDefaults());
35 //         return security.build();
36 //     }
37 //
38 //     @Bean
39 //     public PasswordEncoder passwordEncoder(){
40 //         return new BCryptPasswordEncoder();
41 //     }
42 //
43 //     @Bean
44 //     public AuthenticationManager authenticationManager(@Qualifier("combinedUserDetailService")
45 // UserDetailsService userDetailsService,
46 // PasswordEncoder passwordEncoder){
47 //         DaoAuthenticationProvider employeeAuthenticationProvider=new
48 // DaoAuthenticationProvider(userDetailsService);
49 //         employeeAuthenticationProvider.setPasswordEncoder(passwordEncoder);
50 //         return new ProviderManager(employeeAuthenticationProvider);
51 //     }
52 // }

```

```

1 // src/main/java/com/example/EmployeeManagementSystem/config/JWTAuth/JWTAuthConfig.java
2 package com.example.EmployeeManagementSystem.config.JWTAuth;
3
4 import com.example.EmployeeManagementSystem.Filter.ApiKeyFilter;
5 import com.example.EmployeeManagementSystem.Filter.JwtAuthFilter;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
8 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
9 import com.example.EmployeeManagementSystem.Service.CustomOAuth2UserService;
10 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
11 import com.example.EmployeeManagementSystem.Util.JWTUtil;
12 import com.example.EmployeeManagementSystem.Util.OAuth2SuccessHandler;
13 import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationProvider;
14 import org.springframework.beans.factory.annotation.Qualifier;
15 import org.springframework.beans.factory.annotation.Value;
16 import org.springframework.context.annotation.Bean;
17 import org.springframework.context.annotation.Configuration;
18 import org.springframework.context.annotation.Primary;
19 import org.springframework.http.HttpMethod;
20 import org.springframework.security.authentication.AuthenticationManager;
21 import org.springframework.security.authentication.ProviderManager;
22 import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
23 import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
24 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
25 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
26 import org.springframework.security.config.http.SessionCreationPolicy;
27 import org.springframework.security.core.userdetails.UserDetailsService;
28 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
29 import org.springframework.security.crypto.password.PasswordEncoder;
30 import org.springframework.security.web.SecurityFilterChain;
31 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
32 import org.springframework.security.web.context.DelegatingSecurityContextRepository;
33 import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
34 import org.springframework.security.web.context.RequestAttributeSecurityContextRepository;
35 import org.springframework.web.client.RestTemplate;
36 import org.springframework.web.cors.CorsConfiguration;
37 import org.springframework.web.cors.CorsConfigurationSource;
38 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
39
40 import java.util.Arrays;
41 import java.util.List;
42
43 @Configuration
44 @EnableWebSecurity
45 @EnableMethodSecurity
46 public class JWTAuthConfig {
47
48     private final JwtAuthFilter jwtAuthFilter;
49     private final ApiKeyAuthenticationProvider apiKeyAuthProvider;
50     private final EmployeeRepo employeeRepo;
51     private final VendorRepo vendorRepo;
52     private final RefreshTokenService refreshTokenService;
53     private final RefreshTokenRepository refreshTokenRepository;
54     private final JWTUtil jwtUtil;
55
56     public JWTAuthConfig(JwtAuthFilter jwtAuthFilter,
57                         ApiKeyAuthenticationProvider apiKeyAuthProvider,
58                         EmployeeRepo employeeRepo,
59                         VendorRepo vendorRepo,
60                         RefreshTokenService refreshTokenService,
61                         RefreshTokenRepository refreshTokenRepository,
62                         JWTUtil jwtUtil) {
63         this.jwtAuthFilter = jwtAuthFilter;
64         this.apiKeyAuthProvider = apiKeyAuthProvider;
65         this.employeeRepo = employeeRepo;
66         this.vendorRepo = vendorRepo;
67         this.refreshTokenService = refreshTokenService;
68         this.refreshTokenRepository = refreshTokenRepository;
69         this.jwtUtil = jwtUtil;
70     }
71

```

```

72     @Bean
73     public RestTemplate getRestTemplate() {
74         return new RestTemplate();
75     }
76
77     @Bean
78     public ApiKeyFilter apiKeyFilter(
79         @Qualifier("apiKeyAuthenticationManager") AuthenticationManager apiKeyAuthManager) {
80         return new ApiKeyFilter(apiKeyAuthManager);
81     }
82
83     @Bean
84     public PasswordEncoder passwordEncoder() {
85         return new BCryptPasswordEncoder();
86     }
87
88     @Bean
89     public AuthenticationManager authenticationManager(
90         @Qualifier("combinedUserDetailsService") UserDetailsService userDetailsService,
91         PasswordEncoder passwordEncoder) {
92         DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
93         provider.setPasswordEncoder(passwordEncoder);
94         return new ProviderManager(provider);
95     }
96
97     @Bean
98     public CustomOAuth2UserService customOAuth2UserService() {
99         return new CustomOAuth2UserService(employeeRepo, vendorRepo, passwordEncoder());
100     }
101
102     @Bean
103     public OAuth2SuccessHandler oAuth2SuccessHandler() {
104         return new OAuth2SuccessHandler(jwtUtil, employeeRepo, vendorRepo,
105             refreshTokenService, refreshTokenRepository);
106     }
107
108     @Bean
109     @Qualifier("apiKeyAuthenticationManager")
110     public AuthenticationManager apiKeyAuthenticationManager() {
111         return new ProviderManager(apiKeyAuthProvider);
112     }
113
114     @Bean
115     public SecurityFilterChain securityFilterChain(HttpSecurity http, ApiKeyFilter apiKeyFilter) throws
Exception {
116         http
117             .csrf(csrf -> csrf.disable())
118             .cors(cors -> cors.configurationSource(corsConfigurationSource()))
119             .securityContext(ctx -> ctx
120                 .securityContextRepository(new DelegatingSecurityContextRepository(
121                     new HttpSessionSecurityContextRepository(),
122                     new RequestAttributeSecurityContextRepository()
123                 ))
124             )
125             // .sessionManagement(session -> session
126             //     .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
127             //     .maximumSessions(1)
128             //     .maxSessionsPreventsLogin(false)
129             // )
130             .authorizeHttpRequests(auth -> auth
131                 // Auth endpoints
132                 .requestMatchers("/Authenticate",
133                     "/Authenticate/totp", "/Authenticate/refresh").permitAll()
134                 .requestMatchers("/session/login", "/session/logout").permitAll()
135                 // OAuth2 endpoints
136                 .requestMatchers("/oauth2/**", "/login/oauth2/**").permitAll()
137                 .requestMatchers("/auth/google/init").permitAll()
138                 // API docs
139                 .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
140                 // Shared login page
141                 .requestMatchers("/login.html", "/google.html").permitAll()
142                 .requestMatchers("/error").permitAll()
143
144
145

```

```

146         // Dashboards – served as static HTML, actual data is protected by JWT
147         .requestMatchers("/login.html",
148             "/google.html",
149             "/vendor-login.html",
150             "/vendor-dashboard.html",
151             "/tech-vendor-dashboard.html",
152             "/employee-dashboard.html",
153             "/manager-dashboard.html",
154             "/dashboard.html",
155             "/pending.html",
156             "/leave_management.html",
157             "/totp-setup.html",
158             "/device_management.html").permitAll()
159
160         // Vendor pages
161         .requestMatchers("/vendor-login.html").permitAll()
162         .requestMatchers("/vendor-dashboard.html").permitAll()
163
164         // API key generation requires authentication
165         .requestMatchers("/api-keys/**").authenticated()
166         .requestMatchers("/manager/createAdmin").permitAll()
167         //totp
168         .requestMatchers("/totp/**").authenticated()
169         // notification
170         .requestMatchers(HttpMethod.GET, "/notifications").authenticated()
171         .requestMatchers(HttpMethod.GET, "/notifications/unread-count").authenticated()
172         .requestMatchers(HttpMethod.POST, "/notifications/mark-read").authenticated()
173         // Everything else requires a valid token
174         .anyRequest().authenticated()
175     )
176     .oauth2Login(oauth -> oauth
177         .userInfoEndpoint(userInfo -> userInfo
178             .userService(customOAuth2UserService()))
179     )
180     .successHandler(oAuth2SuccessHandler())
181 )
182 .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
183 // .addFilterBefore(apiKeyFilter, UsernamePasswordAuthenticationFilter.class);
184
185     return http.build();
186 }
187 @Value("${app.cors.allowed-origins}")
188 private String[] allowedOrigins;
189
190 @Bean
191 public CorsConfigurationSource corsConfigurationSource() {
192     CorsConfiguration config = new CorsConfiguration();
193     config.setAllowedOrigins(Arrays.asList(allowedOrigins));
194     config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
195     config.setAllowedHeaders(List.of("*"));
196     config.setAllowCredentials(true);
197     UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
198     source.registerCorsConfiguration("/**", config);
199     return source;
200 }
201 }
202

```

```
1 package com.example.EmployeeManagementSystem.config;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveType;
4 import com.example.EmployeeManagementSystem.Enum.Gender;
5 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 public class LeaveTypeDataInitializer implements CommandLineRunner {
11
12     private final LeaveTypeRepository leaveTypeRepository;
13
14     public LeaveTypeDataInitializer(LeaveTypeRepository leaveTypeRepository) {
15         this.leaveTypeRepository = leaveTypeRepository;
16     }
17
18     @Override
19     public void run(String... args) {
20         ensureLeaveType("SICK", false, true, 0, false, null);
21         ensureLeaveType("CASUAL", true, false, 30, false, null);
22         ensureLeaveType("MATERNITY", false, false, 0, true, Gender.F);
23         ensureLeaveType("UNPAID", false, false, 0, false, null);
24     }
25
26     private void ensureLeaveType(String name,
27                                boolean carriesForward,
28                                boolean monthlyReset,
29                                int maxCarryForward,
30                                boolean genderRestricted,
31                                Gender genderRestriction) {
32         leaveTypeRepository.findByName(name).orElseGet(() -> {
33             LeaveType leaveType = new LeaveType();
34             leaveType.setName(name);
35             leaveType.setCarriesForward(carriesForward);
36             leaveType.setMonthlyReset(monthlyReset);
37             leaveType.setMaxCarryForward(maxCarryForward);
38             leaveType.setGenderRestricted(genderRestricted);
39             leaveType.setGenderRestriction(genderRestriction);
40             return leaveTypeRepository.save(leaveType);
41         });
42     }
43 }
44
```

17 Current:

src\main\java\com\example\EmployeeManagementSystem\config\Session\SessionAuthConfig.java

```
1 //package com.example.EmployeeManagementSystem.config.Session;
2 //
3 //import org.springframework.context.annotation.Bean;
4 //import org.springframework.context.annotation.Configuration;
5 //import org.springframework.core.annotation.Order;
6 //import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 //import org.springframework.security.web.SecurityFilterChain;
8 //import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
9 //
10 //@Configuration
11 //public class SessionAuthConfig {
12 //
13 //    @Bean
14 //    @Order(1)
15 //    public SecurityFilterChain sessionFilterChain(HttpSecurity http) throws Exception {
16 //        http
17 //            .securityMatcher("/session/**", "/web/**")
18 //            .csrf(csrf -> csrf.disable())
19 //            .authorizeHttpRequests(auth -> auth
20 //                .requestMatchers("/session/login", "/session/logout").permitAll()
21 //                .anyRequest().authenticated()
22 //            )
23 //            .sessionManagement(session -> session
24 //                .maximumSessions(1) // one session per user
25 //                .maxSessionsPreventsLogin(false) // new login kicks old session
26 //            )
27 //            .securityContext(ctx -> ctx
28 //                .securityContextRepository(new HttpSessionSecurityContextRepository())
29 //            );
30 //        return http.build();
31 //    }
32 //}
```

```
1 package com.example.EmployeeManagementSystem.config;
2
3 import io.swagger.v3.oas.models.Components;
4 import io.swagger.v3.oas.models.OpenAPI;
5 import io.swagger.v3.oas.models.security.SecurityRequirement;
6 import io.swagger.v3.oas.models.security.SecurityScheme;
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 public class swaggerConfig {
12     @Bean
13     public OpenAPI openAPI() {
14         return new OpenAPI()
15             .components(new Components()
16                 .addSecuritySchemes("bearerAuth",
17                     new SecurityScheme()
18                         .type(SecurityScheme.Type.HTTP)
19                         .scheme("bearer")
20                         .bearerFormat("JWT")
21                 )
22             )
23             .addSecurityItem(new SecurityRequirement().addList("bearerAuth"));
24     }
25     // @Bean
26     // public OpenAPI openAPI() {
27     //     return new OpenAPI()
28     //         .components(new Components()
29     //             .addSecuritySchemes("basicAuth",
30     //                 new SecurityScheme()
31     //                     .type(SecurityScheme.Type.HTTP)
32     //                     .scheme("basic")
33     //             )
34     //         )
35     //         .addSecurityItem(new SecurityRequirement().addList("basicAuth"));
36     // }
37 }
38
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.*;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Service.*;
6 import org.springframework.format.annotation.DateTimeFormat;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.time.LocalDate;
14 import java.util.List;
15
16 @RestController
17 @RequestMapping("/admin")
18 @PreAuthorize("hasRole('ADMIN')")
19 public class AdminController {
20
21     private final EmployeeService employeeService;
22     private final DeliveryService.VendorService vendorService;
23     private final LeaveRequestService leaveRequestService;
24     private final RestaurantService restaurantService;
25     private final SubscriptionService subscriptionService;
26     private final LeaveAccrualService accrualService;
27     private final SickLeaveResetService resetService;
28
29     public AdminController(
30         EmployeeService employeeService,
31         DeliveryService.VendorService vendorService,
32         LeaveRequestService leaveRequestService,
33         RestaurantService restaurantService,
34         SubscriptionService subscriptionService,
35         LeaveAccrualService accrualService, SickLeaveResetService resetService) {
36         this.employeeService = employeeService;
37         this.vendorService = vendorService;
38         this.leaveRequestService = leaveRequestService;
39         this.restaurantService = restaurantService;
40         this.subscriptionService = subscriptionService;
41         this.accrualService = accrualService;
42         this.resetService = resetService;
43     }
44
45     //
46     // EMPLOYEE BRANCH
47     //
48
49     /** List every employee in the system. */
50     @GetMapping("/employees")
51     public ResponseEntity<?> getAllEmployees() {
52         return ResponseEntity.ok(employeeService.getAllEmployees());
53     }
54
55     /** Create a regular employee. */
56     @PostMapping("/employees")
57     public ResponseEntity<Employee> createEmployee(@RequestBody AdminEmployeeDTO dto) {
58         return ResponseEntity.status(HttpStatus.CREATED)
59             .body(employeeService.createEmployee(dto));
60     }
61
62     @PreAuthorize("hasRole('ADMIN')")
63     @GetMapping("/managers")
64     public ResponseEntity<List<ManagerDTO>> getAllManagers(){
65         return employeeService.getAllManagers();
66     }
67
68
69     /** Promote / create a manager. */
70     @PostMapping("/employees/manager")
71     public ResponseEntity<Employee> createManager(@RequestBody EmployeeDTO dto) {

```



```

72         return ResponseEntity.status(HttpStatus.CREATED)
73             .body(employeeService.createManager(dto));
74     }
75
76     /** Update any employee record. */
77     @PutMapping("/employees/{id}")
78     public ResponseEntity<Employee> updateEmployee(
79         @PathVariable long id,
80         @RequestBody EmployeeDTO dto) {
81         return ResponseEntity.ok(employeeService.updateEmployee(id, dto));
82     }
83
84     /** Hard-delete an employee. */
85     @DeleteMapping("/employees/{id}")
86     public ResponseEntity<String> deleteEmployee(@PathVariable long id) {
87         employeeService.deleteEmployee(id);
88         return ResponseEntity.ok("Employee with id: " + id + " has been deleted.");
89     }
90
91     /** Soft-deactivate an employee (sets status to INACTIVE). */
92     @PutMapping("/employees/{id}/inactive")
93     public ResponseEntity<String> deactivateEmployee(@PathVariable long id) {
94         return employeeService.inactivateUser(id);
95     }
96
97     //
98     // VENDOR BRANCH (only admin can register vendors)
99     //
100
101     /** List every vendor. */
102     @GetMapping("/vendors")
103     public ResponseEntity<List<VendorDTO>> getAllVendors() {
104         return ResponseEntity.ok(vendorService.getAllVendors());
105     }
106
107     @PostMapping("/vendors")
108     public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
109         return ResponseEntity.status(HttpStatus.CREATED)
110             .body(vendorService.registerVendor(request));
111     }
112
113     /** Remove a vendor by ID. */
114     @DeleteMapping("/vendors/{vendorId}")
115     public ResponseEntity<String> deleteVendor(@PathVariable Long vendorId) {
116         vendorService.deleteVendorById(vendorId);
117         return ResponseEntity.ok("Vendor with id: " + vendorId + " has been removed.");
118     }
119
120     //
121     // LEAVE MANAGEMENT BRANCH
122     //
123
124     /** Fetch all pending leave requests across the organisation. */
125     @GetMapping("/leaves/pending")
126     public ResponseEntity<?> getPendingLeaves(Authentication authentication) {
127         return ResponseEntity.ok(leaveRequestService.getPendingLeaves(authentication));
128     }
129
130     /** Approve a leave request. */
131     @PutMapping("/leaves/{id}/approve")
132     public ResponseEntity<?> approveLeave(@PathVariable Long id) {
133         return ResponseEntity.ok(leaveRequestService.approveLeave(id));
134     }
135
136     /** Reject a leave request. */
137     @PutMapping("/leaves/{id}/reject")
138     public ResponseEntity<?> rejectLeave(@PathVariable Long id) {
139         return ResponseEntity.ok(leaveRequestService.rejectLeave(id));
140     }
141
142     //
143     // RESTAURANT MANAGEMENT BRANCH
144     //
145
146     /** View all restaurants. */
147     @GetMapping("/restaurants")

```

```

148     public ResponseEntity<?> getAllRestaurants() {
149         return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
150     }
151
152     //
153     // SUBSCRIPTION MANAGEMENT BRANCH
154     //
155
156     /** View all active subscriptions. */
157     @GetMapping("/subscriptions")
158     public ResponseEntity<?> getAllSubscriptions() {
159         return ResponseEntity.ok(subscriptionService.getAllSubscriptions());
160     }
161
162     @PostMapping("/leave/accrue")
163     @PreAuthorize("hasRole('ADMIN')")
164     public ResponseEntity<String> triggerAccrual(
165         @RequestParam(required = false)
166         @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
167     ) {
168         LocalDate accrualDate = (date != null) ? date : LocalDate.now();
169         accrualService.runMonthlyAccrual(accrualDate);
170         return ResponseEntity.ok("Accrual complete for " + accrualDate);
171     }
172     @PostMapping("/reset-sick")
173     @PreAuthorize("hasRole('ADMIN')")
174     public ResponseEntity<String> triggerSickReset(
175         @RequestParam(required = false)
176         @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
177     ) {
178         LocalDate resetDate = (date != null) ? date : LocalDate.now();
179         resetService.runMonthlyReset(resetDate);
180         return ResponseEntity.ok("Sick leave reset complete for " + resetDate);
181     }
182
183 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.ApiKey;
4 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
5 import com.example.EmployeeManagementSystem.Service.ApiKeyService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.access.prepost.PreAuthorize;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api-keys")
14 public class ApiKeyController {
15
16     private final ApiKeyService apiKeyService;
17     private final ApiKeyRepository apiKeyRepository; // instance, lowercase
18
19     public ApiKeyController(ApiKeyService apiKeyService,
20                             ApiKeyRepository apiKeyRepository) {
21         this.apiKeyService = apiKeyService;
22         this.apiKeyRepository = apiKeyRepository;
23     }
24
25     @PostMapping("/vendor/{vendorId}/generate")
26     @PreAuthorize("hasRole('ADMIN')")
27     public ResponseEntity<ApiKey> generateVendorKey(
28         @PathVariable Long vendorId,
29         @RequestParam String name,
30         @RequestParam(defaultValue = "VENDOR_WRITE") String permissions,
31         @RequestParam(defaultValue = "365") int validDays) {
32         return ResponseEntity.ok(
33             apiKeyService.generateForVendor(vendorId, name, permissions, validDays));
34     }
35
36     @GetMapping("/vendor/{vendorId}")
37     @PreAuthorize("hasRole('ADMIN')")
38     public ResponseEntity<List<ApiKey>> getVendorKeys(@PathVariable Long vendorId) {
39         return ResponseEntity.ok(apiKeyRepository.findByVendor_IdAndActiveTrue(vendorId));
40     }
41
42     @DeleteMapping("/{keyId}/revoke")
43     @PreAuthorize("hasRole('ADMIN')")
44     public ResponseEntity<String> revokeKey(@PathVariable Long keyId) {
45         apiKeyRepository.findById(keyId).ifPresent(key -> {
46             key.setActive(false);
47             apiKeyRepository.save(key);
48         });
49         return ResponseEntity.ok("Key revoked.");
50     }
51 }
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AccessTokenRequest;
4 import com.example.EmployeeManagementSystem.DTO.AuthRequest;
5 import com.example.EmployeeManagementSystem.DTO.AuthResponse;
6 import com.example.EmployeeManagementSystem.Entity.Employee;
7 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
8 import com.example.EmployeeManagementSystem.Entity.Vendor;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
11 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
12 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
13 import com.example.EmployeeManagementSystem.Service.TotpService;
14 import com.example.EmployeeManagementSystem.Util.JWTUtil;
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.security.authentication.AuthenticationManager;
17 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
18 import org.springframework.security.core.Authentication;
19 import org.springframework.security.core.userdetails.UserDetails;
20 import org.springframework.web.bind.annotation.*;
21
22 import java.time.Instant;
23 import java.util.Map;
24 import java.util.Optional;
25 import java.util.UUID;
26 import java.util.concurrent.ConcurrentHashMap;
27
28 @RestController
29 @RequestMapping("/Authenticate")
30 public class AuthController {
31
32     private final AuthenticationManager authenticationManager;
33     private final JWTUtil jwtUtil;
34     private final RefreshTokenService refreshTokenService;
35     private final RefreshTokenRepository refreshTokenRepository;
36     private final TotpService totpService;
37     private final EmployeeRepo employeeRepo;
38     private final VendorRepo vendorRepo;
39
40     // Temporary store for pre-auth tokens (username -> pre-auth token)
41     // In production, use Redis or DB-backed store with TTL
42     private static final ConcurrentHashMap<String, PreAuthEntry> PRE_AUTH_STORE = new ConcurrentHashMap<>();
43
44     private record PreAuthEntry(String username, String role, long expiresAt) {}
45
46     public AuthController(AuthenticationManager authenticationManager,
47                           JWTUtil jwtUtil,
48                           RefreshTokenService refreshTokenService,
49                           RefreshTokenRepository refreshTokenRepository,
50                           TotpService totpService,
51                           EmployeeRepo employeeRepo,
52                           VendorRepo vendorRepo) {
53         this.authenticationManager = authenticationManager;
54         this.jwtUtil = jwtUtil;
55         this.refreshTokenService = refreshTokenService;
56         this.refreshTokenRepository = refreshTokenRepository;
57         this.totpService = totpService;
58         this.employeeRepo = employeeRepo;
59         this.vendorRepo = vendorRepo;
60     }
61
62     /**
63      * Step 1 of login. If TOTP is enabled, returns a pre-auth token instead of a JWT.
64      * The client must then call /Authenticate/totp with that pre-auth token + 6-digit code.
65      */
66     @PostMapping
67     public ResponseEntity<?> authenticateToken(@RequestBody AuthRequest authRequest) {
68         try {
69             Authentication auth = authenticationManager.authenticate(
70                 new UsernamePasswordAuthenticationToken(
71                     authRequest.getUsername(),

```

```

72         authRequest.getPassword()
73     )
74 );
75
76 UserDetails user = (UserDetails) auth.getPrincipal();
77
78 String role = user.getAuthorities().stream()
79     .map(a -> a.getAuthority())
80     .filter(a -> a.startsWith("ROLE_"))
81     .findFirst()
82     .map(a -> a.substring("ROLE_".length()))
83     .orElseThrow(() -> new RuntimeException("Role not found"));
84
85 // Handle Vendor login – with TOTP support
86 if (user instanceof Vendor vendor) {
87     if (vendor.isTotpEnabled()) {
88         String preAuthToken = UUID.randomUUID().toString();
89         PRE_AUTH_STORE.put(preAuthToken, new PreAuthEntry(
90             vendor.getEmail(),
91             role,
92             System.currentTimeMillis() + 5 * 60 * 1000L
93         ));
94         return ResponseEntity.ok(Map.of(
95             "requiresTotp", true,
96             "preAuthToken", preAuthToken
97         ));
98     }
99     return ResponseEntity.ok(issueTokens(vendor, role));
100 }
101
102 // Handle Employee login
103 Employee employee = employeeRepo.findByEmail(authRequest.getUsername())
104     .orElseThrow(() -> new RuntimeException("Employee not found"));
105
106 if (employee.isTotpEnabled()) {
107     String preAuthToken = UUID.randomUUID().toString();
108     PRE_AUTH_STORE.put(preAuthToken, new PreAuthEntry(
109         employee.getEmail(),
110         role,
111         System.currentTimeMillis() + 5 * 60 * 1000L
112     ));
113     return ResponseEntity.ok(Map.of(
114         "requiresTotp", true,
115         "preAuthToken", preAuthToken
116     ));
117 }
118
119 return ResponseEntity.ok(issueTokens(user, role));
120
121 } catch (Exception e) {
122     throw new RuntimeException(e);
123 }
124 }
125
126 /**
127  * Step 2 of login when TOTP is enabled.
128  * Submit the pre-auth token + 6-digit TOTP code to get the real JWT.
129  */
130 @PostMapping("/totp")
131 public ResponseEntity<?> completeTotpLogin(@RequestBody Map<String, String> body) {
132     String preAuthToken = body.get("preAuthToken");
133     String code = body.get("code");
134
135     if (preAuthToken == null || code == null) {
136         return ResponseEntity.badRequest().body(Map.of("error", "preAuthToken and code are required"));
137     }
138
139     // In completeTotpLogin(), after the existing employee lookup:
140     PreAuthEntry entry = PRE_AUTH_STORE.get(preAuthToken);
141     if (entry == null || entry.expiresAt() < System.currentTimeMillis()) {
142         PRE_AUTH_STORE.remove(preAuthToken);
143         return ResponseEntity.status(401).body(Map.of("error", "token expired"));
144     }
145
146 // Try employee first, then vendor
147 UserDetails subject;

```

```

148     String secret;
149     Optional<Employee> empOpt = employeeRepo.findByEmail(entry.username());
150     if (empOpt.isPresent()) {
151         Employee employee = empOpt.get();
152         secret = employee.getTotpSecret();
153         subject = employee;
154     } else {
155         Vendor vendor = vendorRepo.findByEmail(entry.username())
156             .orElseThrow(() -> new RuntimeException("User not found"));
157         secret = vendor.getTotpSecret();
158         subject = vendor;
159     }
160
161     if (!totpService.verifyCode(secret, code)) {
162         return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
163     }
164     PRE_AUTH_STORE.remove(preAuthToken);
165     return ResponseEntity.ok(issueTokens(subject, entry.role()));
166 }
167
168 /**
169  * Refresh token endpoint – unchanged from original.
170  */
171 @PostMapping("/refresh")
172 public AuthResponse refresh(@RequestBody AccessTokenRequest request) {
173     RefreshToken storedToken = refreshTokenRepository.findByToken(request.getRefreshToken())
174         .orElseThrow(() -> new RuntimeException("Invalid refresh token"));
175
176     if (storedToken.getExpiryTime().isBefore(Instant.now())) {
177         refreshTokenRepository.delete(storedToken);
178         throw new RuntimeException("Refresh token expired");
179     }
180
181     String username;
182     String role;
183     if (storedToken.getEmployee() != null) {
184         username = storedToken.getEmployee().getUsername();
185         role = storedToken.getEmployee().getRole().name();
186     } else if (storedToken.getVendor() != null) {
187         username = storedToken.getVendor().getUsername();
188         role = storedToken.getVendor().getRole().name();
189     } else {
190         throw new RuntimeException("Refresh token has no associated user");
191     }
192
193     String newAccessToken = jwtUtil.generateToken(username, role);
194     return new AuthResponse(newAccessToken, storedToken.getToken());
195 }
196
197 // ---- helper ----
198
199 private AuthResponse issueTokens(UserDetails user, String role) {
200     String accessToken = jwtUtil.generateToken(user.getUsername(), role);
201     RefreshToken refreshToken;
202     if (user instanceof Employee employee) {
203         refreshToken = refreshTokenService.createForEmployee(employee);
204     } else if (user instanceof Vendor vendor) {
205         refreshToken = refreshTokenService.createForVendor(vendor);
206     } else {
207         throw new RuntimeException("Unknown user type");
208     }
209     refreshTokenRepository.save(refreshToken);
210     return new AuthResponse(accessToken, refreshToken.getToken());
211 }
212 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3
4 import com.example.EmployeeManagementSystem.DTO.InvoiceDTO;
5 import com.example.EmployeeManagementSystem.DTO.RentalBillDTO;
6 import com.example.EmployeeManagementSystem.DTO.RepairBillDTO;
7 import com.example.EmployeeManagementSystem.Entity.RepairBill;
8 import com.example.EmployeeManagementSystem.Service.BillService;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/bills")
16 public class BillController {
17
18     private final BillService billService;
19
20     public BillController(BillService billService) {
21         this.billService = billService;
22     }
23
24     @GetMapping("/repairBill")
25     @PreAuthorize("hasRole('ADMIN')")
26     public List<RepairBillDTO> getAllRepairBill(){
27         return billService.getRepairBills();
28     }
29
30     @GetMapping("/rentalBill")
31     @PreAuthorize("hasRole('ADMIN')")
32     public List<RentalBillDTO> getAllRentalBill(){
33         return billService.getRentalBill();
34     }
35
36     @PostMapping("/repairBill/{id}/pay")
37     @PreAuthorize("hasRole('ADMIN')")
38     public InvoiceDTO payRepairBill(@PathVariable Long id) {
39         return billService.generateRepairInvoice(id);
40     }
41
42     @PostMapping("/rentalBill/{id}/pay")
43     @PreAuthorize("hasRole('ADMIN')")
44     public InvoiceDTO payRentalBill(@PathVariable Long id) {
45         return billService.generateRentalInvoice(id);
46     }
47 }
48
```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Delivery;
4 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
5 import com.example.EmployeeManagementSystem.Service.DeliveryService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/delivery")
13 public class DeliveryController {
14     private final DeliveryService deliveryService;
15
16     public DeliveryController(DeliveryService deliveryService) {
17         this.deliveryService = deliveryService;
18     }
19
20     @PutMapping("/{id}/status")
21     public ResponseEntity<Delivery> updateDeliveryStatus(
22         @PathVariable Long id,
23         @RequestParam DeliveryStatus status) {
24         return ResponseEntity.ok(deliveryService.updateDeliveryStatus(id, status));
25     }
26
27     @GetMapping("/subscription/{subscriptionId}")
28     public ResponseEntity<List<Delivery>> getDeliveriesBySubscription(
29         @PathVariable Long subscriptionId) {
30         return ResponseEntity.ok(deliveryService.getDeliveriesBySubscription(subscriptionId));
31     }
32 }
33
```



```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
4 import com.example.EmployeeManagementSystem.DTO.DeviceActivityDTO;
5 import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
6 import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
7 import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
8 import com.example.EmployeeManagementSystem.DTO.RepairLogResponseDTO;
9 import com.example.EmployeeManagementSystem.Entity.Device;
10 import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
11 import com.example.EmployeeManagementSystem.Service.DeviceService;
12 import org.springframework.http.HttpStatus;
13 import org.springframework.http.ResponseEntity;
14 import org.springframework.security.access.prepost.PreAuthorize;
15 import org.springframework.security.core.Authentication;
16 import org.springframework.web.bind.annotation.*;
17
18 import java.util.List;
19
20 @RestController
21 @RequestMapping("/devices")
22 public class DeviceController {
23     private final DeviceService deviceService;
24
25     public DeviceController(DeviceService deviceService) {
26         this.deviceService = deviceService;
27     }
28
29     @PostMapping
30     @PreAuthorize("hasRole('TECH_VENDOR')")
31     public ResponseEntity<Device> addDevice(@RequestBody DeviceDTO deviceDTO,
32                                             Authentication authentication) {
33         return ResponseEntity.status(HttpStatus.CREATED)
34             .body(deviceService.addDevice(deviceDTO, authentication));
35     }
36
37     @GetMapping("/vendor/me")
38     @PreAuthorize("hasRole('TECH_VENDOR')")
39     public ResponseEntity<List<DeviceResponseDTO>> getMyVendorDevices(Authentication authentication) {
40         return ResponseEntity.ok(deviceService.getDevicesForLoggedInVendor(authentication));
41     }
42
43     @GetMapping
44     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
45     public ResponseEntity<List<DeviceResponseDTO>> getMyDevices(Authentication authentication) {
46         return ResponseEntity.ok(deviceService.getDevicesForEmployee(authentication));
47     }
48
49     @GetMapping("/my")
50     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
51     public ResponseEntity<List<DeviceResponseDTO>> getMyAssignedDevices(Authentication authentication) {
52         return ResponseEntity.ok(deviceService.getDevicesForLoggedInEmployee(authentication));
53     }
54
55     @GetMapping("/unassigned")
56     @PreAuthorize("hasRole('ADMIN')")
57     public ResponseEntity<List<DeviceResponseDTO>> getUnassignedDevices() {
58         return ResponseEntity.ok(deviceService.getUnassignedDevices());
59     }
60
61     @GetMapping("/all")
62     @PreAuthorize("hasRole('ADMIN')")
63     public ResponseEntity<List<DeviceResponseDTO>> getAllDevices() {
64         return ResponseEntity.ok(deviceService.getAllDevices());
65     }
66
67     @GetMapping("/{deviceId}")
68     @PreAuthorize("hasAnyRole('ADMIN', 'TECH_VENDOR', 'EMPLOYEE', 'MANAGER')")
69     public ResponseEntity<DeviceResponseDTO> getDeviceDetails(@PathVariable Long deviceId,
70                                                             Authentication authentication) {
71         return ResponseEntity.ok(deviceService.getDeviceDetails(deviceId, authentication));

```

```

72     }
73
74     @GetMapping("/{deviceId}/activity")
75     @PreAuthorize("hasAnyRole('ADMIN', 'TECH_VENDOR', 'EMPLOYEE', 'MANAGER')")
76     public ResponseEntity<List<DeviceActivityDTO>> getDeviceActivity(@PathVariable Long deviceId,
77                                                                    Authentication authentication) {
78         return ResponseEntity.ok(deviceService.getDeviceActivity(deviceId, authentication));
79     }
80
81     @PostMapping("/{deviceId}/assign")
82     @PreAuthorize("hasRole('ADMIN')")
83     public ResponseEntity<String> assignDevice(@PathVariable Long deviceId,
84                                                @RequestBody AssignDeviceRequest request) {
85         DeviceAssignment assignment = deviceService.assignDevice(deviceId, request);
86         return ResponseEntity.status(HttpStatus.CREATED)
87             .body("Device " + assignment.getDevice().getId()
88                 + " assigned to employee " + assignment.getAssignedTo().getEmployeeId());
89     }
90
91     @GetMapping("/assignments")
92     @PreAuthorize("hasRole('TECH_VENDOR')")
93     public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfVendor(Authentication authentication){
94         return deviceService.getDeviceAssignmentOfVendor(authentication);
95     }
96 }
97
98 @GetMapping("/repaired_devices")
99 @PreAuthorize("hasRole('ADMIN')")
100 public List<DeviceResponseDTO> getRepairedDevices(){
101     return deviceService.getRepairedDevices();
102 }
103
104 @PutMapping("/{deviceId}/mark-assigned")
105 @PreAuthorize("hasRole('ADMIN')")
106 public DeviceResponseDTO markDeviceAssignedAfterRepair(@PathVariable Long deviceId) {
107     return deviceService.markDeviceAssignedAfterRepair(deviceId);
108 }
109
110 @GetMapping("/vendor/{deviceId}/repair-logs")
111 @PreAuthorize("hasRole('TECH_VENDOR')")
112 public List<RepairLogResponseDTO> getVendorRepairLogs(@PathVariable Long deviceId, Authentication
authentication) {
113     return deviceService.getRepairLogsForLoggedInVendorDevice(deviceId, authentication);
114 }
115
116 @PutMapping("/condemned/remove/{deviceId}")
117 @PreAuthorize("hasRole('ADMIN')")
118 public ResponseEntity<String> returnCondemned(@PathVariable Long deviceId, Authentication
authentication) {
119     return deviceService.returnCondemned(deviceId, authentication);
120 }
121
122 @DeleteMapping("/remove/{deviceId}")
123 @PreAuthorize("hasRole('TECH_VENDOR')")
124 public ResponseEntity<String> removeReturned(@PathVariable Long deviceId, Authentication
authentication) {
125     return deviceService.removeDevice(deviceId, authentication);
126 }
127
128 @GetMapping("/repair")
129 @PreAuthorize("hasRole('TECH_VENDOR')")
130 public List<DeviceResponseDTO> getDevicesUnderRepair(Authentication authentication) {
131     return deviceService.getDevicesUnderRepair(authentication);
132 }
133 }
134

```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.EmployeeAttendanceResponseDTO;
4 import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Service.EmployeeService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12 @RestController
13 @RequestMapping("/employee")
14 public class EmployeeController {
15     private final EmployeeService employeeService;
16
17     public EmployeeController(EmployeeService employeeService) {
18         this.employeeService = employeeService;
19     }
20
21     @GetMapping
22     @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN', 'EMPLOYEE', 'VENDOR')")
23     public ResponseEntity<?> getAllEmployees(){
24         return ResponseEntity.ok(employeeService.getAllEmployees());
25     }
26
27     @GetMapping("/me")
28     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE')")
29     public ResponseEntity<Employee> myAccount(Authentication authentication){
30         return ResponseEntity.ok(employeeService.getAccount(authentication));
31     }
32
33     @GetMapping("/attendance")
34     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE', 'VENDOR')")
35     public ResponseEntity<EmployeeAttendanceResponseDTO> getAttendanceOverview() {
36         return ResponseEntity.ok(employeeService.getEmployeeAttendanceOverview());
37     }
38
39     @PutMapping("/{id}")
40     @PreAuthorize("hasRole('ADMIN')")
41     public ResponseEntity<Employee> updateEmployee(@PathVariable long id, @RequestBody EmployeeDTO
employeeDTO){
42         return ResponseEntity.ok(employeeService.updateEmployee(id, employeeDTO));
43     }
44
45     @DeleteMapping("/{id}")
46     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
47     public ResponseEntity<?> deleteEmployee(@PathVariable long id){
48         employeeService.deleteEmployee(id);
49         return ResponseEntity.ok("Employee with id: "+id+" is deleted");
50     }
51
52     //should be done by manager
53     @PutMapping("/inactive")
54     @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
55     public ResponseEntity<String> inactivateEmployee(@RequestParam long employeeId) {
56         return employeeService.inactivateUser(employeeId);
57     }
58 }
59

```

26 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Exception.*;
4 import org.springframework.dao.DataIntegrityViolationException;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.ExceptionHandler;
8 import org.springframework.web.bind.annotation.RestControllerAdvice;
9 import org.springframework.web.context.request.WebRequest;
10 import org.springframework.security.access.AccessDeniedException;
11
12 import java.time.Instant;
13 import java.time.LocalDateTime;
14 import java.util.LinkedHashMap;
15 import java.util.Map;
16
17 @RestControllerAdvice
18 public class GlobalExceptionHandler {
19
20     private Map<String, Object> buildError(HttpStatus status, String message, WebRequest request) {
21         Map<String, Object> body = new LinkedHashMap<>();
22         body.put("timestamp", Instant.now());
23         body.put("status", status.value());
24         body.put("error", status.getReasonPhrase());
25         body.put("message", message);
26         body.put("path", request.getDescription(false).replace("uri=", ""));
27         return body;
28     }
29
30     @ExceptionHandler(EmployeeNotFoundException.class)
31     public ResponseEntity<Map<String, Object>> handleEmployeeNotFound(
32         EmployeeNotFoundException ex, WebRequest request) {
33         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
34             HttpStatus.NOT_FOUND);
35     }
36
37     @ExceptionHandler(InvalidStartDateException.class)
38     public ResponseEntity<Map<String, Object>> handleInvalidStartDate(
39         InvalidStartDateException ex, WebRequest request) {
40         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
41             HttpStatus.BAD_REQUEST);
42     }
43
44     @ExceptionHandler(InvalidEndDateException.class)
45     public ResponseEntity<Map<String, Object>> handleInvalidEndDate(
46         InvalidEndDateException ex, WebRequest request) {
47         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
48             HttpStatus.BAD_REQUEST);
49     }
50
51     @ExceptionHandler(OverlappingLeaveException.class)
52     public ResponseEntity<Map<String, Object>> handleOverlappingLeave(
53         OverlappingLeaveException ex, WebRequest request) {
54         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
55             HttpStatus.BAD_REQUEST);
56     }
57
58     @ExceptionHandler(DuplicateRequestException.class)
59     public ResponseEntity<Map<String, Object>> handleDuplicateRequest(
60         DuplicateRequestException ex, WebRequest request) {
61         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
62             HttpStatus.BAD_REQUEST);
63     }
64
65     @ExceptionHandler(LeaveRequestNotFoundException.class)
66     public ResponseEntity<Map<String, Object>> handleLeaveNotFound(
67         LeaveRequestNotFoundException ex, WebRequest request) {
68         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
69             HttpStatus.NOT_FOUND);
70     }
71 }
```

```

72     @ExceptionHandler(InvalidManagerException.class)
73     public ResponseEntity<Map<String, Object>> handleInvalidManager(
74         InvalidManagerException ex, WebRequest request) {
75         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
76             HttpStatus.FORBIDDEN);
77     }
78
79     @ExceptionHandler(SubscriptionAlreadyExists.class)
80     public ResponseEntity<Map<String, Object>> handleSubscriptionExists(
81         SubscriptionAlreadyExists ex, WebRequest request) {
82         return new ResponseEntity<>(buildError(HttpStatus.CONFLICT, ex.getMessage(), request),
83             HttpStatus.CONFLICT);
84     }
85
86
87     @ExceptionHandler(VendorNotFoundException.class)
88     public ResponseEntity<Map<String, Object>> handleVendorNotFound(
89         VendorNotFoundException ex, WebRequest request) {
90         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
91             HttpStatus.NOT_FOUND);
92     }
93
94     @ExceptionHandler(RestaurantNotFoundException.class)
95     public ResponseEntity<Map<String, Object>> handleRestaurantNotFound(
96         RestaurantNotFoundException ex, WebRequest request) {
97         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
98             HttpStatus.NOT_FOUND);
99     }
100
101     @ExceptionHandler(UnauthorizedAccessException.class)
102     public ResponseEntity<Map<String, Object>> handleUnauthorized(
103         UnauthorizedAccessException ex, WebRequest request) {
104         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
105             HttpStatus.FORBIDDEN);
106     }
107
108     @ExceptionHandler(IllegalArgumentException.class)
109     public ResponseEntity<Map<String, Object>> handleIllegalArgument(
110         IllegalArgumentException ex, WebRequest request) {
111         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
112             HttpStatus.BAD_REQUEST);
113     }
114
115     @ExceptionHandler(DataIntegrityViolationException.class)
116     public ResponseEntity<Map<String, Object>> handleDataIntegrity(
117         DataIntegrityViolationException ex, WebRequest request) {
118         return new ResponseEntity<>(
119             buildError(HttpStatus.CONFLICT, "Cannot delete: related records exist", request),
120             HttpStatus.CONFLICT);
121     }
122
123     @ExceptionHandler(AccessDeniedException.class)
124     public ResponseEntity<Map<String, Object>> handleAccessDenied(AccessDeniedException ex) {
125         return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of(
126             "error", "Access denied",
127             "message", ex.getMessage(),
128             "timestamp", LocalDateTime.now().toString()
129         ));
130     }
131
132     @ExceptionHandler(IllegalStateException.class)
133     public ResponseEntity<Map<String, Object>> handleIllegalState(IllegalStateException ex) {
134         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of(
135             "error", ex.getMessage(),
136             "message", ex.getMessage(),
137             "timestamp", LocalDateTime.now().toString()
138         ));
139     }
140
141     @ExceptionHandler(RepairBillNotFoundException.class)
142     public ResponseEntity<Map<String, Object>>
143     handleRepairBillNotFoundException(RepairBillNotFoundException ex) {
144         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of(
145             "error", "Repair Bill Not Found",
146             "message", ex.getMessage(),
147             "timestamp", LocalDateTime.now().toString()

```

```
147         ));
148     }
149
150     @ExceptionHandler(RentalBillNotFoundException.class)
151     public ResponseEntity<Map<String, Object>>
152     handleRentalBillNotFoundException(RentalBillNotFoundException ex) {
153         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of(
154             "error", "Rental Bill Not Found",
155             "message", ex.getMessage(),
156             "timestamp", LocalDateTime.now().toString()
157         ));
158     }
159 }
```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.HolidayDTO;
4 import com.example.EmployeeManagementSystem.Service.HolidayService;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.security.access.prepost.PreAuthorize;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.LocalDate;
10
11 @RestController
12 @RequestMapping("/holidays")
13 public class HolidayController {
14     private final HolidayService holidayService;
15
16     public HolidayController(HolidayService holidayService) {
17         this.holidayService = holidayService;
18     }
19
20     @PostMapping
21     @PreAuthorize("hasRole('ADMIN')")
22     public ResponseEntity<?> addHoliday(@RequestBody HolidayDTO dto) {
23         return ResponseEntity.ok(holidayService.addHoliday(dto));
24     }
25
26     @GetMapping
27     public ResponseEntity<?> getHolidays(
28         @RequestParam(defaultValue = "0") int year
29     ) {
30         int targetYear = (year == 0) ? LocalDate.now().getYear() : year;
31         return ResponseEntity.ok(holidayService.getHolidaysByYear(targetYear));
32     }
33
34     @DeleteMapping("/{id}")
35     @PreAuthorize("hasRole('ADMIN')")
36     public ResponseEntity<?> deleteHoliday(@PathVariable Long id) {
37         holidayService.deleteHoliday(id);
38         return ResponseEntity.ok("Holiday deleted");
39     }
40 }
```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.InvoiceDTO;
4 import com.example.EmployeeManagementSystem.Service.InvoiceService;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.security.access.prepost.PreAuthorize;
7 import org.springframework.security.core.Authentication;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/invoices")
14 public class InvoiceController {
15
16     private final InvoiceService invoiceService;
17
18     public InvoiceController(InvoiceService invoiceService) {
19         this.invoiceService = invoiceService;
20     }
21
22     // Vendor fetches their pending invoices (SENT only)
23     @GetMapping("/vendor/me")
24     @PreAuthorize("hasRole('TECH_VENDOR')")
25     public List<InvoiceDTO> getMyInvoices(Authentication authentication) {
26         return invoiceService.getPendingInvoicesForCurrentVendor(authentication);
27     }
28
29     // Vendor clicks "Mark as Received" – removes from their view
30     @PutMapping("/{id}/acknowledge")
31     @PreAuthorize("hasRole('TECH_VENDOR')")
32     public ResponseEntity<String> acknowledgeInvoice(@PathVariable Long id, Authentication authentication) {
33         return invoiceService.acknowledgeInvoice(id, authentication);
34         // This sets status = ACKNOWLEDGED, marks bill PAID, sets seenAt
35     }
36
37     // Admin can see all invoices
38     @GetMapping
39     @PreAuthorize("hasRole('ADMIN')")
40     public List<InvoiceDTO> getAllInvoices() {
41         return invoiceService.getAllInvoices();
42     }
43 }
```


29 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Service.LeaveDashboardService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.core.Authentication;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.security.oauth2.core.user.OAuth2User;
10 import org.springframework.web.server.ResponseStatusException;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14
15 import static org.springframework.http.HttpStatus.UNAUTHORIZED;
16
17 @RestController
18 @RequestMapping("/api")
19 public class LeaveDashboardController {
20
21     private final LeaveDashboardService dashboardService;
22     private final EmployeeRepo employeeRepository;
23
24     public LeaveDashboardController(LeaveDashboardService dashboardService,
25                                     EmployeeRepo employeeRepository) {
26         this.dashboardService = dashboardService;
27         this.employeeRepository = employeeRepository;
28     }
29
30     // GET own dashboard (any authenticated employee)
31     // GET /api/employees/me/leave-dashboard
32     @GetMapping("/employees/me/leave-dashboard")
33     public ResponseEntity<LeaveDashboardResponse> getMyDashboard(
34         Authentication authentication) {
35
36         Long callerId = resolveEmployeeId(authentication);
37         return ResponseEntity.ok(
38             dashboardService.getDashboard(callerId, callerId, resolveRole(authentication)));
39     }
40
41     // GET any employee's dashboard (manager/admin only)
42     // GET /api/employees/5/leave-dashboard
43     @GetMapping("/employees/{id}/leave-dashboard")
44     public ResponseEntity<LeaveDashboardResponse> getEmployeeDashboard(
45         @PathVariable Long id,
46         Authentication authentication) {
47
48         Long callerId = resolveEmployeeId(authentication);
49         String callerRole = resolveRole(authentication);
50         return ResponseEntity.ok(
51             dashboardService.getDashboard(id, callerId, callerRole));
52     }
53
54     // GET team dashboard (manager/admin only)
55     // GET /api/employees/me/team-dashboard
56     @GetMapping("/employees/me/team-dashboard")
57     public ResponseEntity<List<LeaveDashboardResponse>> getTeamDashboard(
58         Authentication authentication) {
59
60         Long callerId = resolveEmployeeId(authentication);
61         String callerRole = resolveRole(authentication);
62         return ResponseEntity.ok(
63             dashboardService.getTeamDashboard(callerId, callerRole));
64     }
65
66     // PATCH mark warning read
67     // PATCH /api/warnings/42/read
68     @PatchMapping("/warnings/{warningId}/read")
69     public ResponseEntity<Void> markWarningRead(
70         @PathVariable Long warningId,
71         Authentication authentication) {
```

```

72
73     Long callerId = resolveEmployeeId(authentication);
74     String callerRole = resolveRole(authentication);
75     dashboardService.markWarningRead(warningId, callerId, callerRole);
76     return ResponseEntity.noContent().build();
77 }
78
79 // Supports both JWT/session UserDetails and Google OAuth2 principals.
80 private Long resolveEmployeeId(Authentication authentication) {
81     String email = resolveEmail(authentication);
82     return employeeRepository
83         .findByEmail(email)
84         .orElseThrow(() -> new IllegalStateException("Authenticated user not found"))
85         .getEmployeeId();
86 }
87
88 private String resolveRole(Authentication authentication) {
89     if (authentication == null || !authentication.isAuthenticated()) {
90         throw new ResponseStatusException(UNAUTHORIZED, "Authentication required");
91     }
92     return authentication.getAuthorities().stream()
93         .map(a -> a.getAuthority())
94         .filter(authority -> authority.startsWith("ROLE_"))
95         .findFirst()
96         .map(authority -> authority.replace("ROLE_", ""))
97         .orElse("EMPLOYEE");
98 }
99
100 private String resolveEmail(Authentication authentication) {
101     if (authentication == null || !authentication.isAuthenticated()) {
102         throw new ResponseStatusException(UNAUTHORIZED, "Authentication required");
103     }
104
105     Object principal = authentication.getPrincipal();
106     if (principal instanceof UserDetails userDetails) {
107         return userDetails.getUsername();
108     }
109     if (principal instanceof OAuth2User oAuth2User) {
110         String email = oAuth2User.getAttribute("email");
111         if (email != null && !email.isBlank()) {
112             return email;
113         }
114     }
115
116     throw new ResponseStatusException(UNAUTHORIZED, "Authenticated email not found");
117 }
118 }
119

```

30 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.ActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
5 import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
6 import com.example.EmployeeManagementSystem.DTO.MaternityLeaveRequestDTO;
7 import com.example.EmployeeManagementSystem.Entity.Employee;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
10 import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
11 import com.example.EmployeeManagementSystem.Service.MaternityLeaveService;
12 import com.example.EmployeeManagementSystem.Util.AuthUtil;
13 import org.springframework.http.ResponseEntity;
14 import org.springframework.security.access.prepost.PreAuthorize;
15 import org.springframework.security.core.Authentication;
16 import org.springframework.web.bind.annotation.*;
17
18 import java.time.LocalDate;
19 import java.util.List;
20 import java.util.Map;
21
22 @RestController
23 @RequestMapping("leave_requests")
24 public class LeaveRequestController {
25     private final LeaveRequestService leaveRequestService;
26     private final MaternityLeaveService maternityLeaveService;
27     private final EmployeeRepo employeeRepo;
28     private final LeaveRequestRepo leaveRequestRepo;
29
30     public LeaveRequestController(LeaveRequestService leaveRequestService, MaternityLeaveService
31 maternityLeaveService, EmployeeRepo employeeRepo, LeaveRequestRepo leaveRequestRepo) {
32         this.leaveRequestService = leaveRequestService;
33         this.maternityLeaveService = maternityLeaveService;
34         this.employeeRepo = employeeRepo;
35         this.leaveRequestRepo = leaveRequestRepo;
36     }
37
38     @GetMapping
39     @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
40     public ResponseEntity<List<LeaveResponseDTO>> getAllLeaveRequest() {
41         return ResponseEntity.ok(leaveRequestService.getAllTheLeaveRequest());
42     }
43
44     @PostMapping
45     @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
46     public ResponseEntity<?> addLeaveRequest(Authentication authentication,@RequestBody LeaveRequestDTO
47 dto){
48         return leaveRequestService.createRequest(authentication,dto);
49     }
50
51     @GetMapping("/pending")
52     @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
53     public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests() {
54         return ResponseEntity.ok(leaveRequestService.getAllThePendingLeaveRequests());
55     }
56
57     @PutMapping("/approval")
58     @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
59     public ResponseEntity<?> updateLeaveRequestStatus(@RequestBody ActionDTO actionDTO,Authentication
60 authentication){
61         return leaveRequestService.updateLeaveRequestStatus(actionDTO,authentication);
62     }
63
64     @PutMapping("/cancel/{leaveId}")
65     @PreAuthorize("hasAuthority('LEAVE_CANCEL')")
66     public ResponseEntity<?> cancelLeaveRequest(Authentication authentication,@PathVariable long leaveId){
67         return leaveRequestService.cancelLeaveRequest(authentication, leaveId);
68     }
69
70     @GetMapping("/employee")
71     @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
```

```
69     public ResponseEntity<?> getEmployeeLeaves(Authentication authentication) {
70         return leaveRequestService.getLeaveRequestsByEmployee(authentication);
71     }
72
73     @PostMapping("/maternity/apply")
74     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE')")
75     public ResponseEntity<?> applyMaternityLeave(
76         Authentication authentication,
77         @RequestBody MaternityLeaveRequestDTO dto) {
78         return maternityLeaveService.applyMaternityLeave(authentication, dto);
79     }
80
81
82     // In LeaveRequestController (or wherever your leave endpoints live)
83     @GetMapping("/sick/used-this-month")
84     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE')")
85     public ResponseEntity<Map<String, Object>> sickLeaveUsedThisMonth(
86         Authentication authentication) {
87         String email= AuthUtil.extractEmail(authentication);
88         Employee emp = employeeRepo.findByEmail(email)
89             .orElseThrow();
90
91         LocalDate now = LocalDate.now();
92         boolean used = leaveRequestRepo.existsApprovedSickLeaveForMonth(
93             emp.getEmployeeId(), now.getYear(), now.getMonthValue());
94
95         return ResponseEntity.ok(Map.of(
96             "usedThisMonth", used,
97             "month", now.getMonth().toString()
98         ));
99     }
100
101 }
102
```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import org.springframework.http.ResponseEntity;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 import java.util.Map;
8
9 @RestController
10 public class LoginController {
11
12     @GetMapping("/login")
13     public ResponseEntity<?> login() {
14         return ResponseEntity.ok().body(Map.of("message", "Please use POST to /api/auth/login with
15         credentials"));
16     }
17 }
```

```

1  package com.example.EmployeeManagementSystem.Controller;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Service.ManagerService;
6  import org.springframework.http.ResponseEntity;
7  import org.springframework.security.access.prepost.PreAuthorize;
8  import org.springframework.security.core.Authentication;
9  import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/manager")
15 public class ManagerController {
16     private final ManagerService managerService;
17
18     public ManagerController(ManagerService managerService) {
19         this.managerService = managerService;
20     }
21
22     @PostMapping("/createAdmin")
23     public Employee createAdmin(@RequestBody EmployeeDetails employeeDetails){
24         return managerService.createAdmin(employeeDetails);
25     }
26
27     @PostMapping("/employees")
28     @PreAuthorize("hasAnyRole('MANAGER')")
29     public Employee createEmployee(@RequestBody EmployeeDetails employeeDetails, Authentication
authentication){
30         return managerService.createEmp(employeeDetails, authentication);
31     }
32
33     @GetMapping("/employees")
34     @PreAuthorize("hasRole('MANAGER')")
35     public List<Employee> getAllEmployeeByManager(Authentication authentication){
36         return managerService.getAllEmployeeByManager(authentication);
37     }
38
39     @GetMapping("/requests")
40     @PreAuthorize("hasRole('MANAGER')")
41     public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication){
42         return managerService.getAllLeaveRequestByManager(authentication);
43     }
44
45     @GetMapping("/Users")
46     @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
47     public List<EmployeeDTO> getUsers(){
48         return managerService.getUsers();
49     }
50
51     @PutMapping("/promote")
52     @PreAuthorize("hasRole('MANAGER')")
53     public ResponseEntity<?> promoteUser(Authentication authentication, @RequestBody PromoteRequest
promoteRequest){
54         return managerService.promoteUser(authentication, promoteRequest);
55     }
56
57     @GetMapping("/pending")
58     @PreAuthorize("hasAnyRole('MANAGER')")
59     public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests(Authentication authentication) {
60         return ResponseEntity.ok(managerService.getAllThePendingLeaveRequests(authentication));
61     }
62
63     @GetMapping("/employees/search")
64     @PreAuthorize("hasAnyRole('MANAGER')")
65     public ResponseEntity<ManagerEmployeeDetailsDTO> getEmployeeDetails( @RequestParam String name){
66         return ResponseEntity.ok(managerService.getEmployeeDetailsByName(name));
67     }
68
69

```

70

71 }

72

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Notification;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Service.NotificationService;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.core.Authentication;
9 import org.springframework.security.oauth2.core.user.OAuth2User;
10 import org.springframework.security.oauth2.client.authentication.OAuth2AuthenticationToken;
11 import org.springframework.web.bind.annotation.*;
12 import java.util.List;
13 import java.util.Map;
14
15 @RestController
16 @RequestMapping("/notifications")
17 public class NotificationController {
18
19     private final NotificationService notificationService;
20     private final EmployeeRepo employeeRepo;
21
22     public NotificationController(NotificationService notificationService,
23                                 EmployeeRepo employeeRepo) {
24         this.notificationService = notificationService;
25         this.employeeRepo = employeeRepo;
26     }
27
28     /** GET /notifications – all notifications for logged-in user */
29     @GetMapping
30     public ResponseEntity<List<Notification>> getAll(Authentication authentication) {
31         Employee employee = getEmployee(authentication);
32         return ResponseEntity.ok(notificationService.getForEmployee(employee));
33     }
34
35     /** GET /notifications/unread-count – for the badge number */
36     @GetMapping("/unread-count")
37     public ResponseEntity<?> unreadCount(Authentication authentication) {
38         Employee employee = getEmployee(authentication);
39         return ResponseEntity.ok(Map.of("count",
40                                         notificationService.getUnreadCount(employee)));
41     }
42
43     /** POST /notifications/mark-read – mark all as read when bell is opened */
44     @PostMapping("/mark-read")
45     public ResponseEntity<?> markRead(Authentication authentication) {
46         Employee employee = getEmployee(authentication);
47         notificationService.markAllRead(employee);
48         return ResponseEntity.ok(Map.of("message", "All notifications marked as read"));
49     }
50
51     private Employee getEmployee(Authentication authentication) {
52         final String email = (authentication instanceof OAuth2AuthenticationToken oauth)
53             ? oauth.getPrincipal().<String>getAttribute("email")
54             : authentication.getName(); // JWT path – getName() returns email
55
56         return employeeRepo.findByEmail(email)
57             .orElseThrow(() -> new RuntimeException("Employee not found: " + email));
58     }
59 }
60

```



```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AuthResponse;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.Role;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
10 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
11 import com.example.EmployeeManagementSystem.Service.CombinedUserDetailsService;
12 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
13 import com.example.EmployeeManagementSystem.Util.JWTUtil;
14 import jakarta.servlet.http.HttpServletRequest;
15 import jakarta.servlet.http.HttpServletResponse;
16 import jakarta.servlet.http.HttpSession;
17 import org.springframework.beans.factory.annotation.Autowired;
18 import org.springframework.beans.factory.annotation.Value;
19 import org.springframework.http.*;
20 import org.springframework.security.core.userdetails.UserDetails;
21 import org.springframework.security.core.userdetails.UsernameNotFoundException;
22 import org.springframework.security.crypto.password.PasswordEncoder;
23 import org.springframework.util.LinkedMultiValueMap;
24 import org.springframework.util.MultiValueMap;
25 import org.springframework.web.bind.annotation.GetMapping;
26 import org.springframework.web.bind.annotation.RequestMapping;
27 import org.springframework.web.bind.annotation.RequestParam;
28 import org.springframework.web.bind.annotation.RestController;
29 import org.springframework.web.client.RestTemplate;
30
31 import java.io.IOException;
32 import java.util.Collections;
33 import java.util.Map;
34 import java.util.Optional;
35 import java.util.UUID;
36
37 @RestController
38 @RequestMapping("/auth/google")
39 public class OAuthController {
40
41     @GetMapping("/init")
42     public void initOAuth(@RequestParam(defaultValue = "EMPLOYEE") String role,
43                          @RequestParam(defaultValue = "Asia/Kolkata") String timezone,
44                          HttpServletRequest request,
45                          HttpServletResponse response) throws IOException {
46
47         // Now hand off to Spring's built-in OAuth2 flow
48         System.out.println("Now hand off to Spring's built-in OAuth2 flow");
49         response.sendRedirect("/oauth2/authorization/google");
50     }
51 }
52
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
4 import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
5 import com.example.EmployeeManagementSystem.Enum.MealSlot;
6 import com.example.EmployeeManagementSystem.Service.RestaurantService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14 import java.util.stream.Collectors;
15
16 /**
17  * Restaurant management endpoints.
18  *
19  * VENDOR-ONLY (write operations)
20  * POST /restaurants - Create a restaurant (vendor supplies vendorId in body)
21  * PUT /restaurants/{id} - Update restaurant details (vendor ownership verified)
22  * PUT /restaurants/{id}/deactivate - Soft-delete a restaurant (vendor ownership verified)
23  * GET /restaurants/vendor/{vendorId} - List MY restaurants (vendor views their own)
24  *
25  * EMPLOYEE / MANAGER (read-only browse)
26  * GET /restaurants - Browse all active restaurants
27  * GET /restaurants/{id} - Get a specific restaurant
28  * GET /restaurants/by-slot?slot=LUNCH - Find restaurants that serve a given meal slot
29  *
30  * Employees use the browse endpoints to pick a restaurant when creating a subscription.
31  * Employees and managers cannot create, update, or deactivate restaurants.
32  */
33 @RestController
34 @RequestMapping("/restaurants")
35 public class RestaurantController {
36
37     private final RestaurantService restaurantService;
38
39     public RestaurantController(RestaurantService restaurantService) {
40         this.restaurantService = restaurantService;
41     }
42
43     // VENDOR-ONLY write operations
44
45     /**
46      * Create a new restaurant.
47      * Request body must include vendorId – the service validates the vendor exists.
48      */
49     @PostMapping
50     @PreAuthorize("hasRole('FOOD_VENDOR')")
51     public ResponseEntity<RestaurantDTO> createRestaurant(@RequestBody RestaurantRequest
52 request, Authentication authentication) {
53         return ResponseEntity.status(HttpStatus.CREATED)
54             .body(restaurantService.createRestaurant(request, authentication));
55     }
56
57     /**
58      * Update a restaurant's name, address, description, or supported meal slots.
59      * vendorId in the request body is used to verify ownership before any change is made.
60      */
61     @PutMapping("/{id}")
62     @PreAuthorize("hasRole('FOOD_VENDOR')")
63     public ResponseEntity<RestaurantDTO> updateRestaurant(@PathVariable Long id,
64 @RequestBody RestaurantRequest request,
65 Authentication authentication) {
66         return ResponseEntity.ok(restaurantService.updateRestaurant(id, request, authentication));
67     }
68
69     /**
70      * Soft-deactivate a restaurant. Active subscriptions remain in the DB but
71      * new deliveries will stop since the restaurant is inactive.

```

```

71     * Pass vendorId as a query param so ownership can be verified.
72     */
73     @PutMapping("/{id}/deactivate")
74     @PreAuthorize("hasRole('FOOD_VENDOR')")
75     public ResponseEntity<String> deactivateRestaurant(@PathVariable Long id,
76                                                         Authentication authentication) {
77         restaurantService.deactivateRestaurant(id, authentication);
78         return ResponseEntity.ok("Restaurant " + id + " has been deactivated.");
79     }
80
81     /**
82     * List all active restaurants owned by a specific vendor.
83     * Only the vendor themselves should call this to see their portfolio.
84     */
85     @GetMapping("/vendor")
86     @PreAuthorize("hasRole('FOOD_VENDOR')")
87     public ResponseEntity<List<RestaurantDTO>> getRestaurantsByVendor(Authentication authentication) {
88         return ResponseEntity.ok(restaurantService.getRestaurantsByVendor(authentication));
89     }
90
91     // EMPLOYEE / MANAGER read-only browse
92
93     /**
94     * Browse all active restaurants.
95     * Employees use this to discover available restaurants before subscribing.
96     */
97     @GetMapping
98     @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
99     public ResponseEntity<List<RestaurantDTO>> getAllActiveRestaurants() {
100         return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
101     }
102
103     /**
104     * Get a specific restaurant by id.
105     */
106     @GetMapping("/{id}")
107     @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
108     public ResponseEntity<RestaurantDTO> getRestaurant(@PathVariable Long id) {
109         return ResponseEntity.ok(restaurantService.getRestaurant(id));
110     }
111
112     /**
113     * Find all active restaurants that serve a specific meal slot.
114     * Example: GET /restaurants/by-slot?slot=LUNCH
115     * Employees call this when building their subscription to see valid options for each slot.
116     */
117     @GetMapping("/by-slot")
118     @PreAuthorize("hasAnyRole('ADMIN', 'VENDOR', 'EMPLOYEE', 'MANAGER')")
119     public ResponseEntity<List<RestaurantDTO>> getRestaurantsByMealSlot(@RequestParam MealSlot slot) {
120         return ResponseEntity.ok(restaurantService.getRestaurantsByMealSlot(slot));
121     }
122
123 }
124

```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Scheduler.LeaveStatusScheduler;
4 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
5 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
6 import org.slf4j.Logger;
7 import org.slf4j.LoggerFactory;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.time.LocalDate;
13 import java.time.YearMonth;
14 import java.util.Map;
15
16 /**
17  * Controller for Scheduler Operations
18  */
19 @RestController
20 @RequestMapping("/scheduler")
21 public class SchedulerController {
22
23     private static final Logger log = LoggerFactory.getLogger(SchedulerController.class);
24
25     private final LeaveStatusScheduler leaveStatusScheduler;
26     private final SickLeaveResetService sickLeaveResetService; // FIX: was missing
27     private final LeaveAccrualService leaveAccrualService; // FIX: was missing
28
29     public SchedulerController(LeaveStatusScheduler leaveStatusScheduler,
30                               SickLeaveResetService sickLeaveResetService,
31                               LeaveAccrualService leaveAccrualService) {
32         this.leaveStatusScheduler = leaveStatusScheduler;
33         this.sickLeaveResetService = sickLeaveResetService;
34         this.leaveAccrualService = leaveAccrualService;
35     }
36
37     /**
38      * Manual trigger for leave status scheduler
39      */
40     @PostMapping("/run")
41     @PreAuthorize("hasRole('ADMIN') or hasAuthority('SCHEDULER_TRIGGER')")
42     public ResponseEntity<?> runScheduler() {
43         try {
44             leaveStatusScheduler.runManually();
45             return ResponseEntity.ok(Map.of(
46                 "status", "success",
47                 "message", "Leave status scheduler executed successfully"
48             ));
49         } catch (Exception e) {
50             log.error("Leave status scheduler failed", e);
51             return ResponseEntity.internalServerError().body(Map.of(
52                 "status", "error",
53                 "message", e.getMessage()
54             ));
55         }
56     }
57
58     /**
59      * FIX: Manual trigger for sick leave reset (missing from original).
60      * Useful for recovery and testing without waiting for the end-of-month cron.
61      */
62     @PostMapping("/sick-leave-reset")
63     @PreAuthorize("hasRole('ADMIN')")
64     public ResponseEntity<?> runSickLeaveReset() {
65         try {
66             LocalDate lastDayOfMonth = YearMonth.now().atEndOfMonth();
67             sickLeaveResetService.runMonthlyReset(lastDayOfMonth);
68             return ResponseEntity.ok(Map.of(
69                 "status", "success",
70                 "message", "Sick leave reset executed for " + lastDayOfMonth
71             ));

```

```

72         } catch (Exception e) {
73             log.error("Sick leave reset failed", e);
74             return ResponseEntity.internalServerError().body(Map.of(
75                 "status", "error",
76                 "message", e.getMessage()
77             ));
78         }
79     }
80
81     /**
82      * FIX: Manual trigger for monthly leave accrual (missing from original).
83      * Useful for onboarding new employees or recovering from a missed cron run.
84      */
85     @PostMapping("/accrue")
86     @PreAuthorize("hasRole('ADMIN')")
87     public ResponseEntity<?> runMonthlyAccrual() {
88         try {
89             LocalDate today = LocalDate.now();
90             leaveAccrualService.runMonthlyAccrual(today);
91             return ResponseEntity.ok(Map.of(
92                 "status", "success",
93                 "message", "Monthly leave accrual executed for " + today
94             ));
95         } catch (Exception e) {
96             log.error("Monthly accrual failed", e);
97             return ResponseEntity.internalServerError().body(Map.of(
98                 "status", "error",
99                 "message", e.getMessage()
100             ));
101         }
102     }
103 }

```

37 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerTestController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Service.CarryForwardWarningService;
4 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
5 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
6 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.web.bind.annotation.PostMapping;
10 import org.springframework.web.bind.annotation.RequestMapping;
11 import org.springframework.web.bind.annotation.RequestParam;
12 import org.springframework.web.bind.annotation.RestController;
13
14 import java.time.LocalDate;
15
16 @RestController
17 @RequestMapping("/admin/scheduler")
18 @PreAuthorize("hasRole('ADMIN')")
19 public class SchedulerTestController {
20
21     private final LeaveAccrualService accrualService;
22     private final SickLeaveResetService resetService;
23     private final CarryForwardWarningService warningService;
24     private final YearEndCarryForwardService yearEndService;
25
26     public SchedulerTestController(LeaveAccrualService accrualService,
27                                   SickLeaveResetService resetService,
28                                   CarryForwardWarningService warningService,
29                                   YearEndCarryForwardService yearEndService) {
30         this.accrualService = accrualService;
31         this.resetService = resetService;
32         this.warningService = warningService;
33         this.yearEndService = yearEndService;
34     }
35
36     // Trigger monthly accrual as if today were the 1st of the month
37     @PostMapping("/run-accrual")
38     public ResponseEntity<String> runAccrual(
39         @RequestParam(required = false) String date) {
40         LocalDate target = (date != null) ? LocalDate.parse(date) : LocalDate.now();
41         accrualService.runMonthlyAccrual(target);
42         warningService.runWarningCheck(target);
43         return ResponseEntity.ok("Accrual ran for date: " + target);
44     }
45
46     // Trigger sick leave reset as if today were end of month
47     @PostMapping("/run-sick-reset")
48     public ResponseEntity<String> runSickReset(
49         @RequestParam(required = false) String date) {
50         LocalDate target = (date != null) ? LocalDate.parse(date) : LocalDate.now();
51         resetService.runMonthlyReset(target);
52         return ResponseEntity.ok("Sick leave reset ran for date: " + target);
53     }
54
55     // Trigger year-end carry forward
56     @PostMapping("/run-year-end")
57     public ResponseEntity<String> runYearEnd(
58         @RequestParam(required = false) Integer year) {
59         int targetYear = (year != null) ? year : LocalDate.now().getYear();
60         yearEndService.runYearEnd(targetYear);
61         return ResponseEntity.ok("Year-end ran for year: " + targetYear);
62     }
63 }
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.*;
4 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
5 import com.example.EmployeeManagementSystem.Service.ServiceRequestService;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.security.core.Authentication;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/service-requests")
16 public class ServiceRequestController {
17     private final ServiceRequestService serviceRequestService;
18
19     public ServiceRequestController(ServiceRequestService serviceRequestService) {
20         this.serviceRequestService = serviceRequestService;
21     }
22
23     @PostMapping
24     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
25     public ResponseEntity<ServiceRequest> createServiceRequest(
26         @RequestBody ServiceRequestDTO serviceRequestDTO,
27         Authentication authentication) {
28         return ResponseEntity.status(HttpStatus.CREATED)
29             .body(serviceRequestService.createServiceRequest(serviceRequestDTO, authentication));
30     }
31
32     @GetMapping("/my")
33     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
34     public ResponseEntity<List<ServiceRequestResponseDTO>> getMyServiceRequests(Authentication
35 authentication) {
36         return ResponseEntity.ok(serviceRequestService.getMyServiceRequests(authentication));
37     }
38
39     @GetMapping("/open")
40     @PreAuthorize("hasRole('ADMIN')")
41     public ResponseEntity<List<ServiceRequestResponseDTO>> getAllOpenServiceRequests() {
42         return ResponseEntity.ok(serviceRequestService.getAllOpenServiceRequests());
43     }
44
45     @PutMapping("/admin/repair")
46     @PreAuthorize("hasRole('ADMIN')")
47     public ResponseEntity<ServiceRequestResponseDTO> updateServiceRequestForRepair(
48         Authentication authentication,
49         @RequestBody AdminActionDTO actionDTO) {
50         return ResponseEntity.ok(serviceRequestService.updateServiceRequestForRepairByAdmin(authentication,
51 actionDTO));
52     }
53
54     @PutMapping("/admin/other")
55     @PreAuthorize("hasRole('ADMIN')")
56     public ResponseEntity<ServiceRequestResponseDTO> updateServiceRequestForOtherServices(
57         Authentication authentication,
58         @RequestBody ApprovalActionDTO actionDTO) {
59         return
60         ResponseEntity.ok(serviceRequestService.updateServiceRequestForOtherServicesByAdmin(authentication, actionDTO));
61     }
62
63     @GetMapping("/vendor/me")
64     @PreAuthorize("hasRole('TECH_VENDOR')")
65     public ResponseEntity<List<ServiceRequestResponseDTO>> getAllServiceRequestByVendor(Authentication
66 authentication) {
67         return ResponseEntity.ok(serviceRequestService.getAllServiceRequestByVendor(authentication));
68     }
69
70     @PutMapping("/vendor/finalUpdate")

```

```
68     @PreAuthorize("hasRole('TECH_VENDOR')")
69     public ResponseEntity<ServiceRequestResponseDTO> updateServiceRequestByVendor(
70         Authentication authentication,
71         @RequestBody RepairDTO repairDTO) {
72         return ResponseEntity.ok(serviceRequestService.updateServiceRequestByVendor(authentication,
73             repairDTO));
74     }
75     @PutMapping("/vendor/{serviceRequestId}")
76     @PreAuthorize("hasRole('TECH_VENDOR')")
77     public ResponseEntity<String> updateServiceRequest(Authentication authentication,
78         @PathVariable long serviceRequestId
79     ){
80         return serviceRequestService.updateServiceRequestToUnderRepair(authentication, serviceRequestId);
81     }
82
83     @GetMapping("/devices/repair")
84     @PreAuthorize("hasRole('TECH_VENDOR')")
85     public ResponseEntity<List<ServiceRequestResponseDTO>> getServiceRequestRecievedByVendor(Authentication
86 authentication){
87         return ResponseEntity.ok(serviceRequestService.getServiceRequestRecievedByVendor(authentication));
88     }
89 }
```



```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import com.example.EmployeeManagementSystem.Service.TotpService;
8 import jakarta.servlet.http.HttpServletRequest;
9 import jakarta.servlet.http.HttpServletResponse;
10 import jakarta.servlet.http.HttpSession;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.security.authentication.AuthenticationManager;
13 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
14 import org.springframework.security.core.Authentication;
15 import org.springframework.security.core.context.SecurityContext;
16 import org.springframework.security.core.context.SecurityContextHolder;
17 import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
18 import org.springframework.security.web.context.SecurityContextRepository;
19 import org.springframework.web.bind.annotation.*;
20 import jakarta.servlet.http.Cookie;
21
22 import java.util.Map;
23 import java.util.UUID;
24 import java.util.concurrent.ConcurrentHashMap;
25
26 @RestController
27 @RequestMapping("/session")
28 public class SessionAuthController {
29
30     private final AuthenticationManager authenticationManager;
31     private final TotpService totpService;
32     private final EmployeeRepo employeeRepo;
33     private final VendorRepo vendorRepo;
34     private final SecurityContextRepository securityContextRepository = new
HttpSessionSecurityContextRepository();
35     private static final ConcurrentHashMap<String, PreAuthSession> PRE_AUTH_STORE = new
ConcurrentHashMap<>();
36
37     private record PreAuthSession(Authentication authentication, long expiresAt) {}
38
39     public SessionAuthController(AuthenticationManager authenticationManager,
40                                 TotpService totpService,
41                                 EmployeeRepo employeeRepo,
42                                 VendorRepo vendorRepo) {
43         this.authenticationManager = authenticationManager;
44         this.totpService = totpService;
45         this.employeeRepo = employeeRepo;
46         this.vendorRepo = vendorRepo;
47     }
48
49     @PostMapping("/login")
50     public ResponseEntity<?> login(@RequestBody Map<String, String> body,
51                                    HttpServletRequest request,
52                                    HttpServletResponse response) {
53         String username = body.get("username");
54         String password = body.get("password");
55
56         try {
57             Authentication auth = authenticationManager.authenticate(
58                 new UsernamePasswordAuthenticationToken(username, password)
59             );
60
61             String secret = getTotpSecret(auth);
62             if (secret != null) {
63                 String preAuthToken = UUID.randomUUID().toString();
64                 PRE_AUTH_STORE.put(preAuthToken, new PreAuthSession(
65                     auth,
66                     System.currentTimeMillis() + 5 * 60 * 1000L
67                 ));
68                 return ResponseEntity.ok(Map.of(
69                     "requiresTotp", true,

```

```

70         "preAuthToken", preAuthToken
71     ));
72 }
73
74     saveAuthentication(auth, request, response);
75     HttpSession session = request.getSession(false);
76     return ResponseEntity.ok(Map.of(
77         "message", "Login successful",
78         "username", auth.getName(),
79         "sessionId", session != null ? session.getId() : "",
80         "roles", auth.getAuthorities().toString()
81     ));
82
83     } catch (Exception e) {
84         return ResponseEntity.status(401).body(Map.of("error", "Invalid credentials: " +
85 e.getMessage()));
86     }
87
88     @PostMapping("/totp")
89     public ResponseEntity<?> completeTotpLogin(@RequestBody Map<String, String> body,
90         HttpServletRequest request,
91         HttpServletResponse response) {
92         String preAuthToken = body.get("preAuthToken");
93         String code = body.get("code");
94
95         if (preAuthToken == null || code == null) {
96             return ResponseEntity.badRequest().body(Map.of("error", "preAuthToken and code are required"));
97         }
98
99         PreAuthSession entry = PRE_AUTH_STORE.get(preAuthToken);
100         if (entry == null || entry.expiresAt() < System.currentTimeMillis()) {
101             PRE_AUTH_STORE.remove(preAuthToken);
102             return ResponseEntity.status(401).body(Map.of("error", "token expired"));
103         }
104
105         String secret = getTotpSecret(entry.authentication());
106         if (secret == null || !totpService.verifyCode(secret, code)) {
107             return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
108         }
109
110         PRE_AUTH_STORE.remove(preAuthToken);
111         saveAuthentication(entry.authentication(), request, response);
112         HttpSession session = request.getSession(false);
113         return ResponseEntity.ok(Map.of(
114             "message", "Login successful",
115             "username", entry.authentication().getName(),
116             "sessionId", session != null ? session.getId() : "",
117             "roles", entry.authentication().getAuthorities().toString()
118         ));
119     }
120
121     private void saveAuthentication(Authentication auth,
122         HttpServletRequest request,
123         HttpServletResponse response) {
124         // Create and set security context
125         SecurityContext context = SecurityContextHolder.createEmptyContext();
126         context.setAuthentication(auth);
127         SecurityContextHolder.setContext(context);
128
129         // CRITICAL: Save the context to the HTTP session
130         HttpSession session = request.getSession(true);
131         securityContextRepository.saveContext(context, request, response);
132
133         // Alternative: Manual session attribute (redundant but safe)
134         session.setAttribute(HttpSessionSecurityContextRepository.SPRING_SECURITY_CONTEXT_KEY,
135 context);
136     }
137
138     private String getTotpSecret(Authentication authentication) {
139         Object principal = authentication.getPrincipal();
140         if (principal instanceof Employee employee && employee.isTotpEnabled()) {
141             return employeeRepo.findByEmail(employee.getEmail())
142                 .map(Employee::getTotpSecret)
143                 .orElse(employee.getTotpSecret());
144         }
145     }

```

```

144         if (principal instanceof Vendor vendor && vendor.isTotpEnabled()) {
145             return vendorRepo.findByEmail(vendor.getEmail())
146                 .map(Vendor::getTotpSecret)
147                 .orElse(vendor.getTotpSecret());
148         }
149         return null;
150     }
151
152     @PostMapping("/logout")
153     public ResponseEntity<?> logout(HttpSession session,
154                                     HttpServletRequest request,
155                                     HttpServletResponse response) {
156         // Invalidate Spring Security context
157         SecurityContextHolder.clearContext();
158
159         // Invalidate HTTP session (kills OAuth2 session too)
160         session.invalidate();
161
162         // Clear the session cookie
163         Cookie cookie = new Cookie("JSESSIONID", null);
164         cookie.setMaxAge(0);
165         cookie.setPath("/");
166         response.addCookie(cookie);
167
168         return ResponseEntity.ok("Logged out");
169     }
170
171     @GetMapping("/me")
172     public ResponseEntity<?> me(Authentication authentication) {
173         if (authentication == null || !authentication.isAuthenticated()) {
174             return ResponseEntity.status(401).body(Map.of("error", "Not logged in"));
175         }
176         return ResponseEntity.ok(Map.of(
177             "username", authentication.getName(),
178             "roles", authentication.getAuthorities().toString(),
179             "authenticated", true
180         ));
181     }
182 }
183

```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
4 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
5 import com.example.EmployeeManagementSystem.Entity.Subscription;
6 import com.example.EmployeeManagementSystem.Service.SubscriptionService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14
15 @RestController
16 @RequestMapping("/subscription")
17 public class SubscriptionController {
18     private final SubscriptionService subscriptionService;
19
20     public SubscriptionController(SubscriptionService subscriptionService) {
21         this.subscriptionService = subscriptionService;
22     }
23
24     //Authentication makes sure that whoever is logged can add subscription and cannot add other
25     //subscription
26     @PostMapping
27     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN','VENDOR')")
28     public ResponseEntity<?> addSubscription(@RequestBody SubscriptionRequest request, Authentication
29     authentication) {
30         subscriptionService.addSubscription(request, authentication);
31         return ResponseEntity.status(HttpStatus.CREATED)
32             .body("Added new subscription for user: " + authentication.getName());
33     }
34
35     @GetMapping
36     @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
37     public List<SubscriptionDTO> getAllSubscriptions() {
38         return subscriptionService.getAllSubscriptions();
39     }
40
41     @PutMapping("/{id}")
42     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
43     public ResponseEntity<Subscription> updateSubscription(
44         @PathVariable Long id,
45         @RequestBody SubscriptionRequest request) {
46         return ResponseEntity.ok(subscriptionService.updateSubscription(id, request));
47     }
48
49     @PutMapping("/{id}/pause")
50     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
51     public ResponseEntity<Subscription> pauseSubscription(@PathVariable Long id) {
52         return ResponseEntity.ok(subscriptionService.pauseSubscription(id));
53     }
54
55     @PutMapping("/{id}/resume")
56     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
57     public ResponseEntity<Subscription> resumeSubscription(@PathVariable Long id) {
58         return ResponseEntity.ok(subscriptionService.resumeSubscription(id));
59     }
60
61     @PutMapping("/{id}/expire")
62     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
63     public ResponseEntity<Subscription> expireSubscription(@PathVariable Long id) {
64         return ResponseEntity.ok(subscriptionService.expireSubscription(id));
65     }
66
67     @GetMapping("/user")
68     @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
69     public ResponseEntity<List<SubscriptionDTO>> getSubscriptionOfUser(Authentication authentication){
70         return ResponseEntity.ok(subscriptionService.getSubscriptionOfUser(authentication));
71     }

```

```
70     }
```

```
71
```

```
72 }
```

```
73
```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.TotpUser;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
8 import com.example.EmployeeManagementSystem.Service.TotpService;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.core.userdetails.UserDetails;
12 import org.springframework.web.bind.annotation.*;
13
14 import java.util.Map;
15 import java.util.Optional;
16
17 @RestController
18 @RequestMapping("/totp")
19 public class TotpController {
20
21     private final TotpService totpService;
22     private final EmployeeRepo employeeRepo;
23     private final VendorRepo vendorRepo;
24
25     public TotpController(TotpService totpService, EmployeeRepo employeeRepo, VendorRepo vendorRepo) {
26         this.totpService = totpService;
27         this.employeeRepo = employeeRepo;
28         this.vendorRepo = vendorRepo;
29     }
30
31     @GetMapping("/status")
32     public ResponseEntity<?> status(Authentication authentication) {
33         TotpUser user = getSubject(authentication);
34         return ResponseEntity.ok(Map.of("totpEnabled", user.isTotpEnabled()));
35     }
36
37     @PostMapping("/setup")
38     public ResponseEntity<?> setup(Authentication authentication) {
39         TotpUser user = getSubject(authentication);
40
41         if (user.isTotpEnabled()) {
42             return ResponseEntity.badRequest()
43                 .body(Map.of("error", "TOTP already enabled. Disable first."));
44         }
45
46         String secret = totpService.generateSecret();
47         user.setTotpSecret(secret);
48         user.setTotpEnabled(false);
49         save(user);
50
51         String qrUri = totpService.getQrCodeUri(secret, user.getName());
52         return ResponseEntity.ok(Map.of("secret", secret, "qrCodeUri", qrUri));
53     }
54
55     @PostMapping("/verify-setup")
56     public ResponseEntity<?> verifySetup(@RequestBody Map<String, String> body,
57                                         Authentication authentication) {
58         TotpUser user = getSubject(authentication);
59
60         if (user.getTotpSecret() == null) {
61             return ResponseEntity.badRequest()
62                 .body(Map.of("error", "No TOTP secret found. Call /totp/setup first."));
63         }
64
65         String code = body.get("code");
66         if (code == null || code.isBlank()) {
67             return ResponseEntity.badRequest().body(Map.of("error", "code is required"));
68         }
69
70         if (!totpService.verifyCode(user.getTotpSecret(), code)) {
71             return ResponseEntity.badRequest().body(Map.of("error", "Invalid code. Try again."));
```

```

72     }
73
74     user.setTotpEnabled(true);
75     save(user);
76     return ResponseEntity.ok(Map.of("message", "TOTP 2FA enabled successfully"));
77 }
78
79 @PostMapping("/validate")
80 public ResponseEntity<?> validate(@RequestBody Map<String, String> body,
81                                 Authentication authentication) {
82     TotpUser user = getSubject(authentication);
83
84     if (!user.isTotpEnabled()) {
85         return ResponseEntity.ok(Map.of("message", "TOTP not enabled for this user"));
86     }
87
88     if (user.getTotpSecret() == null) {
89         return ResponseEntity.status(500).body(Map.of("error", "TOTP secret missing"));
90     }
91
92     String code = body.get("code");
93     if (!totpService.verifyCode(user.getTotpSecret(), code)) {
94         return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
95     }
96
97     return ResponseEntity.ok(Map.of("message", "TOTP verified", "authenticated", true));
98 }
99
100 @DeleteMapping("/disable")
101 public ResponseEntity<?> disable(Authentication authentication) {
102     TotpUser user = getSubject(authentication);
103     user.setTotpSecret(null);
104     user.setTotpEnabled(false);
105     save(user);
106     return ResponseEntity.ok(Map.of("message", "TOTP disabled"));
107 }
108
109 // ---- helpers ----
110
111 private TotpUser getSubject(Authentication authentication) {
112     String email = authentication.getName();
113     return employeeRepo.findByEmail(email)
114         .map(e -> (TotpUser) e)
115         .orElseGet(() -> vendorRepo.findByEmail(email)
116             .map(v -> (TotpUser) v)
117             .orElseThrow(() -> new RuntimeException("User not found")));
118 }
119
120 private void save(TotpUser user) {
121     if (user instanceof Employee e) employeeRepo.save(e);
122     else if (user instanceof Vendor v) vendorRepo.save(v);
123 }
124 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.VendorDTO;
4 import com.example.EmployeeManagementSystem.DTO.VendorRequest;
5 import com.example.EmployeeManagementSystem.Service.DeliveryService;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.security.core.Authentication;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/vendors")
16 public class VendorController {
17
18     private final DeliveryService.VendorService vendorService;
19
20     public VendorController(DeliveryService.VendorService vendorService) {
21         this.vendorService = vendorService;
22     }
23
24     @PostMapping("/register")
25     @PreAuthorize("hasRole('ADMIN')")
26     public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
27         return ResponseEntity.status(HttpStatus.CREATED)
28             .body(vendorService.registerVendor(request));
29     }
30
31     @GetMapping
32     @PreAuthorize("hasAnyRole('FOOD_VENDOR', 'ADMIN', 'TECH_VENDOR')")
33     public ResponseEntity<List<VendorDTO>> getAllVendors() {
34         return ResponseEntity.ok(vendorService.getAllVendors());
35     }
36
37     @GetMapping("/me")
38     @PreAuthorize("hasAuthority('VENDOR_WRITE')")
39     public ResponseEntity<VendorDTO> getVendor(Authentication authentication) {
40         return ResponseEntity.ok(vendorService.getVendor(authentication));
41     }
42
43
44     @PutMapping("/update")
45     @PreAuthorize("hasAuthority('VENDOR_WRITE')")
46     public ResponseEntity<VendorDTO> updateVendor(
47         @RequestBody VendorRequest request,
48         Authentication authentication) {
49         return ResponseEntity.ok(vendorService.updateVendor(request, authentication));
50     }
51
52     @DeleteMapping
53     @PreAuthorize("hasRole('ADMIN')")
54     public ResponseEntity<String> deleteVendor(Authentication authentication) {
55         vendorService.deleteVendor(authentication);
56         return ResponseEntity.ok(
57             "Vendor account for " + authentication.getName() + " has been closed.");
58     }
59 }
```


43 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\VendorNegotiationController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.VendorNegotiationMessageRequest;
4 import com.example.EmployeeManagementSystem.DTO.VendorNegotiationRequest;
5 import com.example.EmployeeManagementSystem.DTO.VendorNegotiationResponse;
6 import com.example.EmployeeManagementSystem.DTO.VendorNegotiationStatusRequest;
7 import com.example.EmployeeManagementSystem.Service.VendorNegotiationService;
8 import org.springframework.http.HttpStatus;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.security.access.prepost.PreAuthorize;
11 import org.springframework.security.core.Authentication;
12 import org.springframework.web.bind.annotation.*;
13
14 import java.util.List;
15
16 @RestController
17 @RequestMapping("/vendor-negotiations")
18 @PreAuthorize("hasAnyRole('ADMIN', 'TECH_VENDOR', 'FOOD_VENDOR')")
19 public class VendorNegotiationController {
20
21     private final VendorNegotiationService negotiationService;
22
23     public VendorNegotiationController(VendorNegotiationService negotiationService) {
24         this.negotiationService = negotiationService;
25     }
26
27     @PostMapping
28     public ResponseEntity<VendorNegotiationResponse> createNegotiation(
29         @RequestBody VendorNegotiationRequest request,
30         Authentication authentication) {
31         return ResponseEntity.status(HttpStatus.CREATED)
32             .body(negotiationService.createNegotiation(request, authentication));
33     }
34
35     @GetMapping
36     public ResponseEntity<List<VendorNegotiationResponse>> listNegotiations(Authentication authentication) {
37         return ResponseEntity.ok(negotiationService.listNegotiations(authentication));
38     }
39
40     @GetMapping("/{negotiationId}")
41     public ResponseEntity<VendorNegotiationResponse> getNegotiation(
42         @PathVariable Long negotiationId,
43         Authentication authentication) {
44         return ResponseEntity.ok(negotiationService.getNegotiation(negotiationId, authentication));
45     }
46
47     @PostMapping("/{negotiationId}/messages")
48     public ResponseEntity<VendorNegotiationResponse> addMessage(
49         @PathVariable Long negotiationId,
50         @RequestBody VendorNegotiationMessageRequest request,
51         Authentication authentication) {
52         return ResponseEntity.status(HttpStatus.CREATED)
53             .body(negotiationService.addMessage(negotiationId, request, authentication));
54     }
55
56     @PutMapping("/{negotiationId}/status")
57     public ResponseEntity<VendorNegotiationResponse> updateStatus(
58         @PathVariable Long negotiationId,
59         @RequestBody VendorNegotiationStatusRequest request,
60         Authentication authentication) {
61         return ResponseEntity.ok(negotiationService.updateStatus(negotiationId, request, authentication));
62     }
63 }
64
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class AccessTokenRequest {
4     private String refreshToken;
5
6     public AccessTokenRequest(String refreshToken) {
7         this.refreshToken = refreshToken;
8     }
9
10    public void setRefreshToken(String refreshToken) {
11        this.refreshToken = refreshToken;
12    }
13
14    public String getRefreshToken() {
15        return refreshToken;
16    }
17 }
18
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 public class ActionDTO {
5     private long leaveRequestId;
6     private String action;
7     private String remarks;
8
9     public long getLeaveRequestId() {
10         return leaveRequestId;
11     }
12
13     public void setLeaveRequestId(long leaveRequestId) {
14         this.leaveRequestId = leaveRequestId;
15     }
16
17
18     public String getAction() {
19         return action;
20     }
21
22     public void setAction(String action) {
23         this.action = action;
24     }
25
26     public String getRemarks() {
27         return remarks;
28     }
29
30     public void setRemarks(String remarks) {
31         this.remarks = remarks;
32     }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4
5 public class AdminActionDTO {
6     private Long serviceRequestId;
7     private ServiceRequestStatus status;
8     private String adminRemarks;
9
10    public Long getServiceRequestId() {
11        return serviceRequestId;
12    }
13
14    public void setServiceRequestId(Long serviceRequestId) {
15        this.serviceRequestId = serviceRequestId;
16    }
17
18    public String getAdminRemarks() {
19        return adminRemarks;
20    }
21
22    public void setAdminRemarks(String adminRemarks) {
23        this.adminRemarks = adminRemarks;
24    }
25
26    public ServiceRequestStatus getStatus() {
27        return status;
28    }
29
30    public void setStatus(ServiceRequestStatus status) {
31        this.status = status;
32    }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4
5 public class AdminEmployeeDTO {
6     private String name;
7     private String email;
8     private String password;
9     private Gender gender;
10    private String timezone;
11    private long managerId;
12
13    public String getName() {
14        return name;
15    }
16
17    public String getPassword() {
18        return password;
19    }
20
21    public String getEmail() {
22        return email;
23    }
24
25    public String getTimezone() {
26        return timezone;
27    }
28
29    public Gender getGender() {
30        return gender;
31    }
32
33    public long getManagerId() {
34        return managerId;
35    }
36
37    public void setEmail(String email) {
38        this.email = email;
39    }
40
41    public void setPassword(String password) {
42        this.password = password;
43    }
44
45    public void setManagerId(long managerId) {
46        this.managerId = managerId;
47    }
48
49    public void setTimezone(String timezone) {
50        this.timezone = timezone;
51    }
52
53    public void setGender(Gender gender) {
54        this.gender = gender;
55    }
56
57    public void setName(String name) {
58        this.name = name;
59    }
60 }
61
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4
5
6 public class ApprovalActionDTO {
7     private Long serviceRequestId;
8     private ServiceRequestStatus status; // APPROVED or REJECTED
9     private String adminRemarks;
10    private Long replacementDeviceId; // which device to assign (for REPLACEMENT/UPGRADE)
11
12    public Long getServiceRequestId() {
13        return serviceRequestId;
14    }
15
16    public void setServiceRequestId(Long serviceRequestId) {
17        this.serviceRequestId = serviceRequestId;
18    }
19
20    public ServiceRequestStatus getStatus() {
21        return status;
22    }
23
24    public void setStatus(ServiceRequestStatus status) {
25        this.status = status;
26    }
27
28    public String getAdminRemarks() {
29        return adminRemarks;
30    }
31
32    public void setAdminRemarks(String adminRemarks) {
33        this.adminRemarks = adminRemarks;
34    }
35
36    public Long getReplacementDeviceId() {
37        return replacementDeviceId;
38    }
39
40    public void setReplacementDeviceId(Long replacementDeviceId) {
41        this.replacementDeviceId = replacementDeviceId;
42    }
43 }
44
```

```
1 package com.example.EmployeeManagementSystem.DTO;  
2  
3 public class AssignDeviceRequest {  
4     private Long employeeId;  
5  
6     public Long getEmployeeId() {  
7         return employeeId;  
8     }  
9  
10    public void setEmployeeId(Long employeeId) {  
11        this.employeeId = employeeId;  
12    }  
13 }  
14
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import com.fasterxml.jackson.annotation.JsonProperty;
5
6 public class AuthRequest {
7     @JsonProperty("username")
8     private String Username;
9     @JsonProperty("password")
10    private String Password;
11
12    public String getPassword() {
13        return Password;
14    }
15
16    public void setPassword(String password) {
17        Password = password;
18    }
19
20    public String getUsername() {
21        return Username;
22    }
23
24    public void setUsername(String username) {
25        Username = username;
26    }
27 }
28
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class AuthResponse {
4     private String accessToken;
5     private String refreshToken;
6
7     public AuthResponse(String accessToken, String token) {
8         this.accessToken=accessToken;
9         this.refreshToken=token;
10    }
11
12    public String getAccessToken() {
13        return accessToken;
14    }
15
16    public void setAccessToken(String accessToken) {
17        this.accessToken = accessToken;
18    }
19
20    public String getRefreshToken() {
21        return refreshToken;
22    }
23
24    public void setRefreshToken(String refreshToken) {
25        this.refreshToken = refreshToken;
26    }
27 }
28
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDateTime;
4
5 public class DeviceActivityDTO {
6     private LocalDateTime occurredAt;
7     private String type;
8     private String title;
9     private String description;
10    private String actorName;
11    private String status;
12
13    public LocalDateTime getOccurredAt() {
14        return occurredAt;
15    }
16
17    public void setOccurredAt(LocalDateTime occurredAt) {
18        this.occurredAt = occurredAt;
19    }
20
21    public String getType() {
22        return type;
23    }
24
25    public void setType(String type) {
26        this.type = type;
27    }
28
29    public String getTitle() {
30        return title;
31    }
32
33    public void setTitle(String title) {
34        this.title = title;
35    }
36
37    public String getDescription() {
38        return description;
39    }
40
41    public void setDescription(String description) {
42        this.description = description;
43    }
44
45    public String getActorName() {
46        return actorName;
47    }
48
49    public void setActorName(String actorName) {
50        this.actorName = actorName;
51    }
52
53    public String getStatus() {
54        return status;
55    }
56
57    public void setStatus(String status) {
58        this.status = status;
59    }
60 }
61
```

53 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
4 import com.example.EmployeeManagementSystem.Enum.DeviceType;
5
6 import java.time.LocalDate;
7
8 public class DeviceAssignmentResponseDTO {
9     private Long id;
10    private Long deviceId;
11    private String deviceName;
12    private String deviceBrand;
13    private DeviceType deviceType;
14    private Long employeeId;
15    private String employeeName;
16    private AssignmentStatus status;
17    private LocalDate assignedDate;
18
19    public Long getId() {
20        return id;
21    }
22
23    public void setId(Long id) {
24        this.id = id;
25    }
26
27    public String getDeviceName() {
28        return deviceName;
29    }
30
31    public void setDeviceName(String deviceName) {
32        this.deviceName = deviceName;
33    }
34
35    public Long getDeviceId() {
36        return deviceId;
37    }
38
39    public void setDeviceId(Long deviceId) {
40        this.deviceId = deviceId;
41    }
42
43    public DeviceType getDeviceType() {
44        return deviceType;
45    }
46
47    public void setDeviceType(DeviceType deviceType) {
48        this.deviceType = deviceType;
49    }
50
51    public String getDeviceBrand() {
52        return deviceBrand;
53    }
54
55    public void setDeviceBrand(String deviceBrand) {
56        this.deviceBrand = deviceBrand;
57    }
58
59    public String getEmployeeName() {
60        return employeeName;
61    }
62
63    public void setEmployeeName(String employeeName) {
64        this.employeeName = employeeName;
65    }
66
67    public Long getEmployeeId() {
68        return employeeId;
69    }
70
71    public void setEmployeeId(Long employeeId) {
```

```
72         this.employeeId = employeeId;
73     }
74
75     public AssignmentStatus getStatus() {
76         return status;
77     }
78
79     public void setStatus(AssignmentStatus status) {
80         this.status = status;
81     }
82
83     public LocalDate getAssignedDate() {
84         return assignedDate;
85     }
86
87     public void setAssignedDate(LocalDate assignedDate) {
88         this.assignedDate = assignedDate;
89     }
90 }
91
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.DeviceType;
4
5 import java.math.BigDecimal;
6 import java.time.LocalDate;
7
8 public class DeviceDTO {
9     private String deviceName;        // e.g. "Dell XPS 15"
10    private String brand;
11    private DeviceType deviceType;
12    private LocalDate warrantyExpiryDate;
13    private BigDecimal rentPerMonth;
14
15    public String getDeviceName() {
16        return deviceName;
17    }
18
19    public void setDeviceName(String deviceName) {
20        this.deviceName = deviceName;
21    }
22
23    public String getBrand() {
24        return brand;
25    }
26
27    public void setBrand(String brand) {
28        this.brand = brand;
29    }
30
31    public DeviceType getDeviceType() {
32        return deviceType;
33    }
34
35    public void setDeviceType(DeviceType deviceType) {
36        this.deviceType = deviceType;
37    }
38
39    public LocalDate getWarrantyExpiryDate() {
40        return warrantyExpiryDate;
41    }
42
43    public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
44        this.warrantyExpiryDate = warrantyExpiryDate;
45    }
46
47    public BigDecimal getRentPerMonth() {
48        return rentPerMonth;
49    }
50
51    public void setRentPerMonth(BigDecimal rentPerMonth) {
52        this.rentPerMonth = rentPerMonth;
53    }
54 }
55
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
6 import com.example.EmployeeManagementSystem.Enum.DeviceType;
7
8 import java.math.BigDecimal;
9 import java.time.LocalDate;
10 import java.time.LocalDateTime;
11
12 public class DeviceResponseDTO {
13     private Long id;
14     private String deviceName;
15     private String brand;
16     private DeviceType deviceType;
17     private DeviceStatus deviceStatus;
18     private String vendorName;
19     private long assignedEmployeeId;
20     private String assignedEmployeeName;
21     private LocalDate assignedDate;
22
23     private LocalDate warrantyExpiryDate;
24     private BigDecimal rentPerMonth;
25     private LocalDateTime createdAt;
26
27     public Long getId() {
28         return id;
29     }
30
31     public void setId(Long id) {
32         this.id = id;
33     }
34
35     public String getDeviceName() {
36         return deviceName;
37     }
38
39     public void setDeviceName(String deviceName) {
40         this.deviceName = deviceName;
41     }
42
43     public String getBrand() {
44         return brand;
45     }
46
47     public void setBrand(String brand) {
48         this.brand = brand;
49     }
50
51     public DeviceType getDeviceType() {
52         return deviceType;
53     }
54
55     public void setDeviceType(DeviceType deviceType) {
56         this.deviceType = deviceType;
57     }
58
59     public DeviceStatus getDeviceStatus() {
60         return deviceStatus;
61     }
62
63     public void setDeviceStatus(DeviceStatus deviceStatus) {
64         this.deviceStatus = deviceStatus;
65     }
66
67     public String getVendorName() {
68         return vendorName;
69     }
70
71     public void setVendorName(String vendorName) {
```

```
72         this.vendorName = vendorName;
73     }
74
75     public long getAssignedEmployeeId() {
76         return assignedEmployeeId;
77     }
78
79     public void setAssignedEmployeeId(long assignedEmployeeId) {
80         this.assignedEmployeeId = assignedEmployeeId;
81     }
82
83     public String getAssignedEmployeeName() {
84         return assignedEmployeeName;
85     }
86
87     public void setAssignedEmployeeName(String assignedEmployeeName) {
88         this.assignedEmployeeName = assignedEmployeeName;
89     }
90
91     public LocalDate getAssignedDate() {
92         return assignedDate;
93     }
94
95     public void setAssignedDate(LocalDate assignedDate) {
96         this.assignedDate = assignedDate;
97     }
98
99     public LocalDate getWarrantyExpiryDate() {
100         return warrantyExpiryDate;
101     }
102
103     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
104         this.warrantyExpiryDate = warrantyExpiryDate;
105     }
106
107     public BigDecimal getRentPerMonth() {
108         return rentPerMonth;
109     }
110
111     public void setRentPerMonth(BigDecimal rentPerMonth) {
112         this.rentPerMonth = rentPerMonth;
113     }
114
115     public LocalDateTime getCreatedAt() {
116         return createdAt;
117     }
118
119     public void setCreatedAt(LocalDateTime createdAt) {
120         this.createdAt = createdAt;
121     }
122 }
123
```

56 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.util.List;
4
5 public class EmployeeAttendanceResponseDTO {
6     private long workingCount;
7     private long onLeaveCount;
8     private List<EmployeeStatusDTO> workingEmployees;
9     private List<EmployeeStatusDTO> onLeaveEmployees;
10
11     public long getWorkingCount() {
12         return workingCount;
13     }
14
15     public void setWorkingCount(long workingCount) {
16         this.workingCount = workingCount;
17     }
18
19     public long getOnLeaveCount() {
20         return onLeaveCount;
21     }
22
23     public void setOnLeaveCount(long onLeaveCount) {
24         this.onLeaveCount = onLeaveCount;
25     }
26
27     public List<EmployeeStatusDTO> getWorkingEmployees() {
28         return workingEmployees;
29     }
30
31     public void setWorkingEmployees(List<EmployeeStatusDTO> workingEmployees) {
32         this.workingEmployees = workingEmployees;
33     }
34
35     public List<EmployeeStatusDTO> getOnLeaveEmployees() {
36         return onLeaveEmployees;
37     }
38
39     public void setOnLeaveEmployees(List<EmployeeStatusDTO> onLeaveEmployees) {
40         this.onLeaveEmployees = onLeaveEmployees;
41     }
42 }
43
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4
5 public class EmployeeDetails {
6     private String name;
7     private String email;
8     private String password;
9     private String timezone;
10    private Gender gender;
11
12    public String getName() {
13        return name;
14    }
15
16    public void setName(String name) {
17        this.name = name;
18    }
19
20    public String getPassword() {
21        return password;
22    }
23
24    public void setPassword(String password) {
25        this.password = password;
26    }
27
28    public String getEmail() {
29        return email;
30    }
31
32    public void setEmail(String email) {
33        this.email = email;
34    }
35
36    public String getTimezone() {
37        return timezone;
38    }
39
40    public void setTimezone(String timezone) {
41        this.timezone = timezone;
42    }
43
44    public Gender getGender() {
45        return gender;
46    }
47
48    public void setGender(Gender gender) {
49        this.gender = gender;
50    }
51 }
52
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import com.example.EmployeeManagementSystem.Enum.Gender;
5
6 public class EmployeeDTO {
7     private String name;
8     private String email;
9     private String dept;
10    private String password;
11    private Gender gender;
12    private String timezone;
13
14    public String getTimezone() {
15        return timezone;
16    }
17
18    public void setTimezone(String timezone) {
19        this.timezone = timezone;
20    }
21
22    public String getName() {
23        return name;
24    }
25
26    public void setName(String name) {
27        this.name = name;
28    }
29
30    public String getEmail() {
31        return email;
32    }
33
34    public void setEmail(String email) {
35        this.email = email;
36    }
37
38    public String getDept() {
39        return dept;
40    }
41
42    public void setDept(String dept) {
43        this.dept = dept;
44    }
45
46    public String getPassword() {
47        return password;
48    }
49
50    public void setPassword(String password) {
51        this.password = password;
52    }
53
54    public Gender getGender() {
55        return gender;
56    }
57
58    public void setGender(Gender gender) {
59        this.gender = gender;
60    }
61 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import com.example.EmployeeManagementSystem.Enum.Status;
5
6 public class EmployeeStatusDTO {
7     private long employeeId;
8     private String name;
9     private String email;
10    private String dept;
11    private Role role;
12    private Status status;
13    private String timezone;
14
15    public long getEmployeeId() {
16        return employeeId;
17    }
18
19    public void setEmployeeId(long employeeId) {
20        this.employeeId = employeeId;
21    }
22
23    public String getName() {
24        return name;
25    }
26
27    public void setName(String name) {
28        this.name = name;
29    }
30
31    public String getEmail() {
32        return email;
33    }
34
35    public void setEmail(String email) {
36        this.email = email;
37    }
38
39    public String getDept() {
40        return dept;
41    }
42
43    public void setDept(String dept) {
44        this.dept = dept;
45    }
46
47    public Role getRole() {
48        return role;
49    }
50
51    public void setRole(Role role) {
52        this.role = role;
53    }
54
55    public Status getStatus() {
56        return status;
57    }
58
59    public void setStatus(Status status) {
60        this.status = status;
61    }
62
63    public String getTimezone() {
64        return timezone;
65    }
66
67    public void setTimezone(String timezone) {
68        this.timezone = timezone;
69    }
70 }
71
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class HolidayDTO {
4     private Long id;
5     private String name;
6     private String date;
7     private String description;
8     private int year;
9
10    public Long getId() {
11        return id;
12    }
13
14    public void setId(Long id) {
15        this.id = id;
16    }
17
18    public String getName() {
19        return name;
20    }
21
22    public void setName(String name) {
23        this.name = name;
24    }
25
26    public String getDate() {
27        return date;
28    }
29
30    public void setDate(String date) {
31        this.date = date;
32    }
33
34    public String getDescription() {
35        return description;
36    }
37
38    public void setDescription(String description) {
39        this.description = description;
40    }
41
42    public int getYear() {
43        return year;
44    }
45
46    public void setYear(int year) {
47        this.year = year;
48    }
49 }
50
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.math.BigDecimal;
4 import java.time.LocalDate;
5
6 public class InvoiceDTO {
7
8     private Long id;
9     private String invoiceType;
10    private String vendorName;
11
12    private BigDecimal amount;
13
14    private LocalDate issuedDate;
15    private String status;    // SENT, SEEN, ACKNOWLEDGED
16
17    private LocalDate seenAt;
18
19    public Long getId() {
20        return id;
21    }
22
23    public void setId(Long id) {
24        this.id = id;
25    }
26
27    public BigDecimal getAmount() {
28        return amount;
29    }
30
31    public void setAmount(BigDecimal amount) {
32        this.amount = amount;
33    }
34
35    public String getVendorName() {
36        return vendorName;
37    }
38
39    public void setVendorName(String vendorName) {
40        this.vendorName = vendorName;
41    }
42
43    public String getInvoiceType() {
44        return invoiceType;
45    }
46
47    public void setInvoiceType(String invoiceType) {
48        this.invoiceType = invoiceType;
49    }
50
51    public String getStatus() {
52        return status;
53    }
54
55    public void setStatus(String status) {
56        this.status = status;
57    }
58
59    public LocalDate getIssuedDate() {
60        return issuedDate;
61    }
62
63    public void setIssuedDate(LocalDate issuedDate) {
64        this.issuedDate = issuedDate;
65    }
66
67    public LocalDate getSeenAt() {
68        return seenAt;
69    }
70
71    public void setSeenAt(LocalDate seenAt) {
```

```
72         this.seenAt = seenAt;
73     }
74 }
75
```

```

1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.math.BigDecimal;
4 import java.util.List;
5 import java.util.Map;
6
7 public class LeaveDashboardResponse {
8
9     private Long    employeeId;
10    private String  employeeName;
11    private int     year;
12
13    // Individual leave balances
14    private LeaveBalanceDto sick;
15    private LeaveBalanceDto casual;
16    private LeaveBalanceDto maternity;    // null for non-female employees
17
18    // Carry-forward meter (casual + maternity combined)
19    private CarryForwardMeterDto carryForwardMeter;
20
21    // Generic balances keyed by leave type name, e.g. SICK, CASUAL, MATERNITY.
22    private Map<String, LeaveBalanceDto> leaveBalances;
23
24    // Active warnings (unread)
25    private List<WarningDto> warnings;
26
27    // nested DTOs
28
29    public static class LeaveBalanceDto {
30        private String    leaveType;
31        private BigDecimal opening;
32        private BigDecimal accrued;
33        private BigDecimal used;
34        private BigDecimal available;
35        private boolean    carriesForward;
36
37        public LeaveBalanceDto(String leaveType, BigDecimal opening,
38                                BigDecimal accrued, BigDecimal used,
39                                BigDecimal available, boolean carriesForward) {
40            this.leaveType    = leaveType;
41            this.opening      = opening;
42            this.accrued      = accrued;
43            this.used         = used;
44            this.available    = available;
45            this.carriesForward = carriesForward;
46        }
47
48        public String    getLeaveType()    { return leaveType; }
49        public BigDecimal getOpening()     { return opening; }
50        public BigDecimal getAccrued()     { return accrued; }
51        public BigDecimal getUsed()        { return used; }
52        public BigDecimal getAvailable()   { return available; }
53        public boolean    isCarriesForward() { return carriesForward; }
54    }
55
56    public static class CarryForwardMeterDto {
57        private BigDecimal current;    // e.g. 28.00
58        private int        cap;        // always 30
59        private String     label;      // "28/30"
60        private String     status;     // SAFE | WARNING | CRITICAL | CAPPED
61
62        public CarryForwardMeterDto(BigDecimal current, int cap) {
63            this.current = current;
64            this.cap     = cap;
65            this.label   = current.toPlainString() + "/" + cap;
66            this.status  = resolveStatus(current, cap);
67        }
68
69        private String resolveStatus(BigDecimal current, int cap) {
70            int cmp = current.compareTo(BigDecimal.valueOf(cap));
71            if (cmp >= 0) return "CAPPED";

```

```

72         if (current.compareTo(BigDecimal.valueOf(cap - 1)) >= 0) return "CRITICAL";
73         if (current.compareTo(BigDecimal.valueOf(cap - 2)) >= 0) return "WARNING";
74         return "SAFE";
75     }
76
77     public BigDecimal getCurrent() { return current; }
78     public int         getCap()      { return cap; }
79     public String      getLabel()    { return label; }
80     public String      getStatus()   { return status; }
81 }
82
83 public static class WarningDto {
84     private Long id;
85     private String type;
86     private String message;
87     private String date;
88
89     public WarningDto(Long id, String type, String message, String date) {
90         this.id = id; this.type = type;
91         this.message = message; this.date = date;
92     }
93
94     public Long getId()      { return id; }
95     public String getType()  { return type; }
96     public String getMessage() { return message; }
97     public String getDate()  { return date; }
98 }
99
100 // root getters & setters
101
102 public Long         getEmployeeId()      { return employeeId; }
103 public String       getEmployeeName()    { return employeeName; }
104 public int          getYear()             { return year; }
105 public LeaveBalanceDto getSick()          { return sick; }
106 public LeaveBalanceDto getCasual()        { return casual; }
107 public LeaveBalanceDto getMaternity()     { return maternity; }
108 public CarryForwardMeterDto getCarryForwardMeter(){ return carryForwardMeter; }
109 public Map<String, LeaveBalanceDto> getLeaveBalances() { return leaveBalances; }
110 public List<WarningDto> getWarnings()     { return warnings; }
111
112 public void setEmployeeId(Long id)        { this.employeeId = id; }
113 public void setEmployeeName(String n)     { this.employeeName = n; }
114 public void setYear(int y)                { this.year = y; }
115 public void setSick(LeaveBalanceDto b)     { this.sick = b; }
116 public void setCasual(LeaveBalanceDto b)   { this.casual = b; }
117 public void setMaternity(LeaveBalanceDto b) { this.maternity = b; }
118 public void setCarryForwardMeter(CarryForwardMeterDto m) { this.carryForwardMeter = m; }
119 public void setLeaveBalances(Map<String, LeaveBalanceDto> b) { this.leaveBalances = b; }
120 public void setWarnings(List<WarningDto> w) { this.warnings = w; }
121 }
122

```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveType;
4
5 import java.time.LocalDate;
6
7 // DTO used when employee applies for leave.
8 // Contains only input fields required from employee.
9 public class LeaveRequestDTO {
10     private LeaveType leaveType;
11     private LocalDate startDate;
12     private LocalDate endDate;
13     private String reason;
14
15
16     public LeaveType getLeaveType() {
17         return leaveType;
18     }
19
20     public void setLeaveType(LeaveType leaveType) {
21         this.leaveType = leaveType;
22     }
23
24     public LocalDate getStartDate() {
25         return startDate;
26     }
27
28     public void setStartDate(LocalDate startDate) {
29         this.startDate = startDate;
30     }
31
32     public LocalDate getEndDate() {
33         return endDate;
34     }
35
36     public void setEndDate(LocalDate endDate) {
37         this.endDate = endDate;
38     }
39
40     public String getReason() {
41         return reason;
42     }
43
44     public void setReason(String reason) {
45         this.reason = reason;
46     }
47 }
48
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5
6 import java.time.LocalDate;
7
8 public class LeaveResponseDTO {
9     private long leaveRequestId;
10    private String employeeName;
11    private LeaveType leaveType;
12    private LocalDate startDate;
13    private LocalDate endDate;
14    private long numberOfDays;
15    private String reason;
16    private LeaveStatus status;
17
18    public long getLeaveRequestId() {
19        return leaveRequestId;
20    }
21
22    public void setLeaveRequestId(long leaveRequestId) {
23        this.leaveRequestId = leaveRequestId;
24    }
25
26    public String getEmployeeName() {
27        return employeeName;
28    }
29
30    public void setEmployeeName(String employeeName) {
31        this.employeeName = employeeName;
32    }
33
34    public LocalDate getStartDate() {
35        return startDate;
36    }
37
38    public void setStartDate(LocalDate startDate) {
39        this.startDate = startDate;
40    }
41
42    public LeaveType getLeaveType() {
43        return leaveType;
44    }
45
46    public void setLeaveType(LeaveType leaveType) {
47        this.leaveType = leaveType;
48    }
49
50    public LocalDate getEndDate() {
51        return endDate;
52    }
53
54    public void setEndDate(LocalDate endDate) {
55        this.endDate = endDate;
56    }
57
58    public long getNumberOfDays() {
59        return numberOfDays;
60    }
61
62    public void setNumberOfDays(long numberOfDays) {
63        this.numberOfDays = numberOfDays;
64    }
65
66    public String getReason() {
67        return reason;
68    }
69
70    public void setReason(String reason) {
71        this.reason = reason;
```

```
72     }
73
74     public LeaveStatus getStatus() {
75         return status;
76     }
77
78     public void setStatus(L leaveStatus status) {
79         this.status = status;
80     }
81 }
82
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import org.bouncycastle.crypto.signers.ISOTrailers;
4
5 public class ManagerDTO {
6     private long managerId;
7     private String name;
8     private String email;
9     private String dept;
10
11     public long getManagerId() {
12         return managerId;
13     }
14
15     public void setManagerId(long managerId) {
16         this.managerId = managerId;
17     }
18
19     public String getName() {
20         return name;
21     }
22
23     public void setName(String name) {
24         this.name = name;
25     }
26
27     public String getEmail() {
28         return email;
29     }
30
31     public void setEmail(String email) {
32         this.email = email;
33     }
34
35     public String getDept() {
36         return dept;
37     }
38
39     public void setDept(String dept) {
40         this.dept = dept;
41     }
42 }
43
```

66 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerEmployeeDetailsDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Status;
4
5 import java.util.List;
6
7 public class ManagerEmployeeDetailsDTO {
8     private Long employeeId;
9     private String name;
10    private String email;
11    private String dept;
12    private Status status;
13    private String attendance;
14    private List<DeviceResponseDTO> devices;
15
16    public Long getEmployeeId() {
17        return employeeId;
18    }
19
20    public void setEmployeeId(Long employeeId) {
21        this.employeeId = employeeId;
22    }
23
24    public String getName() {
25        return name;
26    }
27
28    public void setName(String name) {
29        this.name = name;
30    }
31
32    public String getEmail() {
33        return email;
34    }
35
36    public void setEmail(String email) {
37        this.email = email;
38    }
39
40    public String getDept() {
41        return dept;
42    }
43
44    public void setDept(String dept) {
45        this.dept = dept;
46    }
47
48    public Status getStatus() {
49        return status;
50    }
51
52    public void setStatus(Status status) {
53        this.status = status;
54    }
55
56    public String getAttendance() {
57        return attendance;
58    }
59
60    public void setAttendance(String attendance) {
61        this.attendance = attendance;
62    }
63
64    public List<DeviceResponseDTO> getDevices() {
65        return devices;
66    }
67
68    public void setDevices(List<DeviceResponseDTO> devices) {
69        this.devices = devices;
70    }
71 }
```


67 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerLeaveResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5
6 import java.time.LocalDate;
7
8 public class ManagerLeaveResponseDTO {
9     private String employeeName;
10    private LeaveType leaveType;
11    private LocalDate startDate;
12    private LocalDate endDate;
13    private long numberOfDays;
14    private String reason;
15    private LeaveStatus status;
16
17    public String getEmployeeName() {
18        return employeeName;
19    }
20
21    public void setEmployeeName(String employeeName) {
22        this.employeeName = employeeName;
23    }
24
25    public LeaveType getLeaveType() {
26        return leaveType;
27    }
28
29    public void setLeaveType(LeaveType leaveType) {
30        this.leaveType = leaveType;
31    }
32
33    public LocalDate getStartDate() {
34        return startDate;
35    }
36
37    public void setStartDate(LocalDate startDate) {
38        this.startDate = startDate;
39    }
40
41    public LocalDate getEndDate() {
42        return endDate;
43    }
44
45    public void setEndDate(LocalDate endDate) {
46        this.endDate = endDate;
47    }
48
49    public long getNumberOfDays() {
50        return numberOfDays;
51    }
52
53    public void setNumberOfDays(long numberOfDays) {
54        this.numberOfDays = numberOfDays;
55    }
56
57    public String getReason() {
58        return reason;
59    }
60
61    public void setReason(String reason) {
62        this.reason = reason;
63    }
64
65    public LeaveStatus getStatus() {
66        return status;
67    }
68
69    public void setStatus(LeaveStatus status) {
70        this.status = status;
71    }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDate;
4
5 public class MaternityLeaveRequestDTO {
6     private LocalDate startDate; // only start date – end is always fixed
7     private String reason;
8
9     public LocalDate getStartDate() {
10         return startDate;
11     }
12
13     public void setStartDate(LocalDate startDate) {
14         this.startDate = startDate;
15     }
16
17     public String getReason() {
18         return reason;
19     }
20
21     public void setReason(String reason) {
22         this.reason = reason;
23     }
24 }
25
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class PromoteRequest {
4     private String email;
5     private String password;
6
7     public String getEmail() {
8         return email;
9     }
10
11     public void setEmail(String email) {
12         this.email = email;
13     }
14
15     public String getPassword() {
16         return password;
17     }
18
19     public void setPassword(String password) {
20         this.password = password;
21     }
22 }
23
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import java.math.BigDecimal;
5 import java.time.LocalDate;
6
7 public class RentalBillDTO {
8     private Long id;
9     private BigDecimal amount;
10    private LocalDate billDate;
11    private LocalDate dueDate;
12    private String vendorName;
13    private String paymentStatus;
14
15    public Long getId() {
16        return id;
17    }
18
19    public void setId(Long id) {
20        this.id = id;
21    }
22
23    public BigDecimal getAmount() {
24        return amount;
25    }
26
27    public void setAmount(BigDecimal amount) {
28        this.amount = amount;
29    }
30
31    public LocalDate getBillDate() {
32        return billDate;
33    }
34
35    public void setBillDate(LocalDate billDate) {
36        this.billDate = billDate;
37    }
38
39    public LocalDate getDueDate() {
40        return dueDate;
41    }
42
43    public void setDueDate(LocalDate dueDate) {
44        this.dueDate = dueDate;
45    }
46
47    public String getVendorName() {
48        return vendorName;
49    }
50
51    public void setVendorName(String vendorName) {
52        this.vendorName = vendorName;
53    }
54
55    public String getPaymentStatus() {
56        return paymentStatus;
57    }
58
59    public void setPaymentStatus(String paymentStatus) {
60        this.paymentStatus = paymentStatus;
61    }
62 }
63
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import java.math.BigDecimal;
5 import java.time.LocalDate;
6
7 public class RepairBillDTO {
8     private Long billId;
9     private String deviceName;
10    private String repairedBy;
11    private LocalDate repairedDate;
12    private BigDecimal repairCost;
13    private LocalDate geneartedOn;
14    private String status;
15
16    public Long getBillId() {
17        return billId;
18    }
19
20    public void setBillId(Long billId) {
21        this.billId = billId;
22    }
23
24    public String getDeviceName() {
25        return deviceName;
26    }
27
28    public void setDeviceName(String deviceName) {
29        this.deviceName = deviceName;
30    }
31
32    public String getRepairedBy() {
33        return repairedBy;
34    }
35
36    public void setRepairedBy(String repairedBy) {
37        this.repairedBy = repairedBy;
38    }
39
40    public LocalDate getRepairedDate() {
41        return repairedDate;
42    }
43
44    public void setRepairedDate(LocalDate repairedDate) {
45        this.repairedDate = repairedDate;
46    }
47
48    public BigDecimal getRepairCost() {
49        return repairCost;
50    }
51
52    public void setRepairCost(BigDecimal repairCost) {
53        this.repairCost = repairCost;
54    }
55
56    public LocalDate getGeneartedOn() {
57        return geneartedOn;
58    }
59
60    public void setGeneartedOn(LocalDate geneartedOn) {
61        this.geneartedOn = geneartedOn;
62    }
63
64    public String getStatus() {
65        return status;
66    }
67
68    public void setStatus(String status) {
69        this.status = status;
70    }
71 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4
5 import java.math.BigDecimal;
6
7 public class RepairDTO {
8     private Long requestId;
9     private String resolution;
10    private ServiceRequestStatus status;
11    private String damagedComponent;
12    private String replacedComponent;
13    private BigDecimal repairCost;
14
15    public Long getRequestId() {
16        return requestId;
17    }
18
19    public void setRequestId(Long requestId) {
20        this.requestId = requestId;
21    }
22
23    public String getResolution() {
24        return resolution;
25    }
26
27    public void setResolution(String resolution) {
28        this.resolution = resolution;
29    }
30
31    public ServiceRequestStatus getStatus() {
32        return status;
33    }
34
35    public void setStatus(ServiceRequestStatus status) {
36        this.status = status;
37    }
38
39    public String getDamagedComponent() {
40        return damagedComponent;
41    }
42
43    public void setDamagedComponent(String damagedComponent) {
44        this.damagedComponent = damagedComponent;
45    }
46
47    public String getReplacedComponent() {
48        return replacedComponent;
49    }
50
51    public void setReplacedComponent(String replacedComponent) {
52        this.replacedComponent = replacedComponent;
53    }
54
55    public BigDecimal getRepairCost() {
56        return repairCost;
57    }
58
59 }
60
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.math.BigDecimal;
4 import java.time.LocalDate;
5
6 public class RepairLogResponseDTO {
7     private Long id;
8     private Long deviceId;
9     private String deviceName;
10    private Long serviceRequestId;
11    private String serviceRequestStatus;
12    private String issueType;
13    private String damagedComponent;
14    private String repairAction;
15    private String replacedComponent;
16    private BigDecimal repairCost;
17    private String repairedBy;
18    private LocalDate repairDate;
19    private String remarks;
20
21    public Long getId() {
22        return id;
23    }
24
25    public void setId(Long id) {
26        this.id = id;
27    }
28
29    public Long getDeviceId() {
30        return deviceId;
31    }
32
33    public void setDeviceId(Long deviceId) {
34        this.deviceId = deviceId;
35    }
36
37    public String getDeviceName() {
38        return deviceName;
39    }
40
41    public void setDeviceName(String deviceName) {
42        this.deviceName = deviceName;
43    }
44
45    public Long getServiceRequestId() {
46        return serviceRequestId;
47    }
48
49    public void setServiceRequestId(Long serviceRequestId) {
50        this.serviceRequestId = serviceRequestId;
51    }
52
53    public String getServiceRequestStatus() {
54        return serviceRequestStatus;
55    }
56
57    public void setServiceRequestStatus(String serviceRequestStatus) {
58        this.serviceRequestStatus = serviceRequestStatus;
59    }
60
61    public String getIssueType() {
62        return issueType;
63    }
64
65    public void setIssueType(String issueType) {
66        this.issueType = issueType;
67    }
68
69    public String getDamagedComponent() {
70        return damagedComponent;
71    }
```

```
72
73     public void setDamagedComponent(String damagedComponent) {
74         this.damagedComponent = damagedComponent;
75     }
76
77     public String getRepairAction() {
78         return repairAction;
79     }
80
81     public void setRepairAction(String repairAction) {
82         this.repairAction = repairAction;
83     }
84
85     public String getReplacedComponent() {
86         return replacedComponent;
87     }
88
89     public void setReplacedComponent(String replacedComponent) {
90         this.replacedComponent = replacedComponent;
91     }
92
93     public BigDecimal getRepairCost() {
94         return repairCost;
95     }
96
97     public void setRepairCost(BigDecimal repairCost) {
98         this.repairCost = repairCost;
99     }
100
101     public String getRepairedBy() {
102         return repairedBy;
103     }
104
105     public void setRepairedBy(String repairedBy) {
106         this.repairedBy = repairedBy;
107     }
108
109     public LocalDate getRepairDate() {
110         return repairDate;
111     }
112
113     public void setRepairDate(LocalDate repairDate) {
114         this.repairDate = repairDate;
115     }
116
117     public String getRemarks() {
118         return remarks;
119     }
120
121     public void setRemarks(String remarks) {
122         this.remarks = remarks;
123     }
124 }
125
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4
5 import java.util.Set;
6
7 public class RestaurantDTO {
8     private Long id;
9     private String name;
10    private String address;
11    private String description;
12    private Set<MealSlot> supportedMealSlots;
13    private boolean active;
14    private Long vendorId;
15    private String vendorName;
16
17    public Long getId() { return id; }
18    public void setId(Long id) { this.id = id; }
19
20    public String getName() { return name; }
21    public void setName(String name) { this.name = name; }
22
23    public String getAddress() { return address; }
24    public void setAddress(String address) { this.address = address; }
25
26    public String getDescription() { return description; }
27    public void setDescription(String description) { this.description = description; }
28
29    public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
30    public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
31        this.supportedMealSlots = supportedMealSlots;
32    }
33
34    public boolean isActive() { return active; }
35    public void setActive(boolean active) { this.active = active; }
36
37    public Long getVendorId() { return vendorId; }
38    public void setVendorId(Long vendorId) { this.vendorId = vendorId; }
39
40    public String getVendorName() { return vendorName; }
41    public void setVendorName(String vendorName) { this.vendorName = vendorName; }
42 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.NotEmpty;
6 import jakarta.validation.constraints.NotNull;
7
8 import java.util.Set;
9
10 public class RestaurantRequest {
11
12     @NotBlank
13     private String name;
14
15     @NotBlank
16     private String address;
17
18     private String description;
19
20     /**
21      * Which meal slots this restaurant serves.
22      * Must be at least one of: BREAKFAST, LUNCH, DINNER.
23      */
24     @NotNull
25     @NotEmpty
26     private Set<MealSlot> supportedMealSlots;
27
28     /**
29      * The vendor ID of the owner making this request.
30      * Used to verify ownership on updates/deletes.
31      */
32
33     public String getName() { return name; }
34     public void setName(String name) { this.name = name; }
35
36     public String getAddress() { return address; }
37     public void setAddress(String address) { this.address = address; }
38
39     public String getDescription() { return description; }
40     public void setDescription(String description) { this.description = description; }
41
42     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
43     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
44         this.supportedMealSlots = supportedMealSlots;
45     }
46 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;
4
5 public class ServiceRequestDTO {
6
7     private Long deviceId;
8     private ServiceRequestType requestType;
9     private String issueDescription;
10    private boolean urgent;
11
12    public Long getDeviceId() {
13        return deviceId;
14    }
15
16    public void setDeviceId(Long deviceId) {
17        this.deviceId = deviceId;
18    }
19
20    public ServiceRequestType getRequestType() {
21        return requestType;
22    }
23
24    public void setRequestType(ServiceRequestType requestType) {
25        this.requestType = requestType;
26    }
27
28    public String getIssueDescription() {
29        return issueDescription;
30    }
31
32    public void setIssueDescription(String issueDescription) {
33        this.issueDescription = issueDescription;
34    }
35
36    public boolean isUrgent() {
37        return urgent;
38    }
39
40    public void setUrgent(boolean urgent) {
41        this.urgent = urgent;
42    }
43 }
44
```

77 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDateTime;
4
5 public class ServiceRequestResponseDTO
6 {
7     private Long id;
8     private String requestType;
9     private String status;
10    private String issueDescription;
11    private String adminRemarks;
12    private String resolution;
13    private boolean urgent;
14    private LocalDateTime raisedAt;
15    private LocalDateTime resolvedAt;
16    private String deviceName;
17    private String raisedByName;
18    private String reviewedByName;
19
20    public String getDeviceName() {
21        return deviceName;
22    }
23
24    public void setDeviceName(String deviceName) {
25        this.deviceName = deviceName;
26    }
27
28    public String getReviewedByName() {
29        return reviewedByName;
30    }
31
32    public void setReviewedByName(String reviewedByName) {
33        this.reviewedByName = reviewedByName;
34    }
35
36    public LocalDateTime getResolvedAt() {
37        return resolvedAt;
38    }
39
40    public void setResolvedAt(LocalDateTime resolvedAt) {
41        this.resolvedAt = resolvedAt;
42    }
43
44    public LocalDateTime getRaisedAt() {
45        return raisedAt;
46    }
47
48    public void setRaisedAt(LocalDateTime raisedAt) {
49        this.raisedAt = raisedAt;
50    }
51
52    public boolean isUrgent() {
53        return urgent;
54    }
55
56    public void setUrgent(boolean urgent) {
57        this.urgent = urgent;
58    }
59
60    public String getResolution() {
61        return resolution;
62    }
63
64    public void setResolution(String resolution) {
65        this.resolution = resolution;
66    }
67
68    public Long getId() {
69        return id;
70    }
71}
```

```
72     public void setId(Long id) {
73         this.id = id;
74     }
75
76     public String getRequestType() {
77         return requestType;
78     }
79
80     public void setRequestType(String requestType) {
81         this.requestType = requestType;
82     }
83
84     public String getStatus() {
85         return status;
86     }
87
88     public void setStatus(String status) {
89         this.status = status;
90     }
91
92     public String getAdminRemarks() {
93         return adminRemarks;
94     }
95
96     public void setAdminRemarks(String adminRemarks) {
97         this.adminRemarks = adminRemarks;
98     }
99
100    public String getIssueDescription() {
101        return issueDescription;
102    }
103
104    public void setIssueDescription(String issueDescription) {
105        this.issueDescription = issueDescription;
106    }
107
108    public String getRaisedByName() {
109        return raisedByName;
110    }
111
112    public void setRaisedByName(String raisedByName) {
113        this.raisedByName = raisedByName;
114    }
115 }
116
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
6
7 import java.time.DayOfWeek;
8 import java.time.ZonedDateTime;
9
10 public class SubscriptionDTO {
11     private long subscriptionId;
12     private long userId;
13     private MealSlot mealSlot;
14     private ScheduleType scheduleType;
15     private DayOfWeek dayOfWeek;
16     private SubscriptionStatus status;
17     private ZonedDateTime nextDeliveryTime;
18
19     private Long restaurantId;
20     private String restaurantName;
21     private String restaurantAddress;
22
23     public void setRestaurantId(Long restaurantId) {
24         this.restaurantId = restaurantId;
25     }
26
27     public void setRestaurantName(String restaurantName) {
28         this.restaurantName = restaurantName;
29     }
30
31     public void setRestaurantAddress(String restaurantAddress) {
32         this.restaurantAddress = restaurantAddress;
33     }
34
35     public Long getRestaurantId() {
36         return restaurantId;
37     }
38
39     public String getRestaurantName() {
40         return restaurantName;
41     }
42
43     public String getRestaurantAddress() {
44         return restaurantAddress;
45     }
46
47     public long getSubscriptionId() {
48         return subscriptionId;
49     }
50
51     public void setSubscriptionId(long subscriptionId) {
52         this.subscriptionId = subscriptionId;
53     }
54
55     public long getUserId() {
56         return userId;
57     }
58
59     public void setUserId(long userId) {
60         this.userId = userId;
61     }
62
63     public MealSlot getMealSlot() {
64         return mealSlot;
65     }
66
67     public void setMealSlot(MealSlot mealSlot) {
68         this.mealSlot = mealSlot;
69     }
70
71     public ScheduleType getScheduleType() {
```

```
72         return scheduleType;
73     }
74
75     public void setScheduleType(ScheduleType scheduleType) {
76         this.scheduleType = scheduleType;
77     }
78
79     public DayOfWeek getDayOfWeek() {
80         return dayOfWeek;
81     }
82
83     public void setDayOfWeek(DayOfWeek dayOfWeek) {
84         this.dayOfWeek = dayOfWeek;
85     }
86
87     public SubscriptionStatus getStatus() {
88         return status;
89     }
90
91     public void setStatus(SubscriptionStatus status) {
92         this.status = status;
93     }
94
95     public ZonedDateTime getNextDeliveryTime() {
96         return nextDeliveryTime;
97     }
98
99     public void setNextDeliveryTime(ZonedDateTime nextDeliveryTime) {
100         this.nextDeliveryTime = nextDeliveryTime;
101     }
102 }
103
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import jakarta.validation.constraints.NotEmpty;
6 import jakarta.validation.constraints.NotNull;
7
8 import java.time.DayOfWeek;
9 import java.util.List;
10
11 public class SubscriptionRequest {
12     private ScheduleType scheduleType;
13     private DayOfWeek dayOfWeek;
14     @NotNull
15     @NotEmpty
16     private List<MealSlot> mealSlots;
17     @NotNull
18     private List<SlotOrder> slotOrders;
19     public static class SlotOrder {
20         @NotNull
21         private MealSlot mealSlot;
22
23         @NotNull
24         private Long restaurantId;
25
26         public MealSlot getMealSlot() {
27             return mealSlot;
28         }
29
30         public void setMealSlot(MealSlot mealSlot) {
31             this.mealSlot = mealSlot;
32         }
33
34         public Long getRestaurantId() {
35             return restaurantId;
36         }
37
38         public void setRestaurantId(Long restaurantId) {
39             this.restaurantId = restaurantId;
40         }
41     }
42
43     public ScheduleType getScheduleType() {
44         return scheduleType;
45     }
46
47     public void setScheduleType(ScheduleType scheduleType) {
48         this.scheduleType = scheduleType;
49     }
50
51     public List<MealSlot> getMealSlots() {
52         return mealSlots;
53     }
54
55     public void setMealSlots(List<MealSlot> mealSlots) {
56         this.mealSlots = mealSlots;
57     }
58
59     public DayOfWeek getDayOfWeek() {
60         return dayOfWeek;
61     }
62
63     public void setDayOfWeek(DayOfWeek dayOfWeek) {
64         this.dayOfWeek = dayOfWeek;
65     }
66
67
68
69     public List<SlotOrder> getSlotOrders() { return slotOrders; }
70     public void setSlotOrders(List<SlotOrder> slotOrders) { this.slotOrders = slotOrders; }
71 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4
5 import java.time.LocalDate;
6
7 public class VendorDTO {
8     private Long id;
9     private String name;
10    private String email;
11    private String phone;
12    private Role role;
13    private LocalDate registeredAt;
14
15    public Long getId() { return id; }
16    public void setId(Long id) { this.id = id; }
17
18    public String getName() { return name; }
19    public void setName(String name) { this.name = name; }
20
21    public String getEmail() { return email; }
22    public void setEmail(String email) { this.email = email; }
23
24    public String getPhone() { return phone; }
25    public void setPhone(String phone) { this.phone = phone; }
26
27    public Role getRole() { return role; }
28    public void setRole(Role role) { this.role = role; }
29
30    public LocalDate getRegisteredAt() { return registeredAt; }
31    public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
32 }
33
```

81 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationMessageRequest.

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class VendorNegotiationMessageRequest {
4     private String message;
5
6     public String getMessage() { return message; }
7     public void setMessage(String message) { this.message = message; }
8 }
9
```

82 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationMessageResponse

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MessageSenderType;
4
5 import java.time.LocalDateTime;
6
7 public class VendorNegotiationMessageResponse {
8     private Long id;
9     private MessageSenderType senderType;
10    private String senderEmail;
11    private String message;
12    private LocalDateTime createdAt;
13
14    public Long getId() { return id; }
15    public void setId(Long id) { this.id = id; }
16
17    public MessageSenderType getSenderType() { return senderType; }
18    public void setSenderType(MessageSenderType senderType) { this.senderType = senderType; }
19
20    public String getSenderEmail() { return senderEmail; }
21    public void setSenderEmail(String senderEmail) { this.senderEmail = senderEmail; }
22
23    public String getMessage() { return message; }
24    public void setMessage(String message) { this.message = message; }
25
26    public LocalDateTime getCreatedAt() { return createdAt; }
27    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
28 }
29
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.math.BigDecimal;
4
5 public class VendorNegotiationRequest {
6     private Long vendorId;
7     private Long deviceId;
8     private String subject;
9     private String description;
10    private BigDecimal proposedAmount;
11    private String initialMessage;
12
13    public Long getVendorId() { return vendorId; }
14    public void setVendorId(Long vendorId) { this.vendorId = vendorId; }
15
16    public Long getDeviceId() { return deviceId; }
17    public void setDeviceId(Long deviceId) { this.deviceId = deviceId; }
18
19    public String getSubject() { return subject; }
20    public void setSubject(String subject) { this.subject = subject; }
21
22    public String getDescription() { return description; }
23    public void setDescription(String description) { this.description = description; }
24
25    public BigDecimal getProposedAmount() { return proposedAmount; }
26    public void setProposedAmount(BigDecimal proposedAmount) { this.proposedAmount = proposedAmount; }
27
28    public String getInitialMessage() { return initialMessage; }
29    public void setInitialMessage(String initialMessage) { this.initialMessage = initialMessage; }
30 }
31
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.NegotiationStatus;
4
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7 import java.util.List;
8
9 public class VendorNegotiationResponse {
10     private Long id;
11     private Long vendorId;
12     private String vendorName;
13     private Long deviceId;
14     private String deviceName;
15     private String subject;
16     private String description;
17     private BigDecimal proposedAmount;
18     private NegotiationStatus status;
19     private LocalDateTime createdAt;
20     private LocalDateTime updatedAt;
21     private List<VendorNegotiationMessageResponse> messages;
22
23     public Long getId() { return id; }
24     public void setId(Long id) { this.id = id; }
25
26     public Long getVendorId() { return vendorId; }
27     public void setVendorId(Long vendorId) { this.vendorId = vendorId; }
28
29     public String getVendorName() { return vendorName; }
30     public void setVendorName(String vendorName) { this.vendorName = vendorName; }
31
32     public Long getDeviceId() { return deviceId; }
33     public void setDeviceId(Long deviceId) { this.deviceId = deviceId; }
34
35     public String getDeviceName() { return deviceName; }
36     public void setDeviceName(String deviceName) { this.deviceName = deviceName; }
37
38     public String getSubject() { return subject; }
39     public void setSubject(String subject) { this.subject = subject; }
40
41     public String getDescription() { return description; }
42     public void setDescription(String description) { this.description = description; }
43
44     public BigDecimal getProposedAmount() { return proposedAmount; }
45     public void setProposedAmount(BigDecimal proposedAmount) { this.proposedAmount = proposedAmount; }
46
47     public NegotiationStatus getStatus() { return status; }
48     public void setStatus(NegotiationStatus status) { this.status = status; }
49
50     public LocalDateTime getCreatedAt() { return createdAt; }
51     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
52
53     public LocalDateTime getUpdatedAt() { return updatedAt; }
54     public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }
55
56     public List<VendorNegotiationMessageResponse> getMessages() { return messages; }
57     public void setMessages(List<VendorNegotiationMessageResponse> messages) { this.messages = messages; }
58 }
59
```

85 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\VendorNegotiationStatusRequest.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.NegotiationStatus;
4
5 public class VendorNegotiationStatusRequest {
6     private NegotiationStatus status;
7
8     public NegotiationStatus getStatus() { return status; }
9     public void setStatus(NegotiationStatus status) { this.status = status; }
10 }
11
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6
7 public class VendorRequest {
8
9     @NotBlank
10     private String name;
11
12     @Email
13     @NotBlank
14     private String email;
15
16     @NotBlank
17     private String phone;
18
19     private String password;
20
21     private Role role;
22
23     public String getName() { return name; }
24     public void setName(String name) { this.name = name; }
25
26     public String getEmail() { return email; }
27     public void setEmail(String email) { this.email = email; }
28
29     public String getPhone() { return phone; }
30     public void setPhone(String phone) { this.phone = phone; }
31
32     public String getPassword() {
33         return password;
34     }
35
36     public void setPassword(String password) {
37         this.password = password;
38     }
39
40     public Role getRole() {
41         return role;
42     }
43
44     public void setRole(Role role) {
45         this.role = role;
46     }
47 }
```


87 Current:

src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
1 package com.example.EmployeeManagementSystem;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.scheduling.annotation.EnableScheduling;
6
7 @SpringBootApplication
8 @EnableScheduling
9 public class EmployeeManagementSystemApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(EmployeeManagementSystemApplication.class, args);
13     }
14
15 }
16
```

```

1 // src/main/java/com/example/EmployeeManagementSystem/Entity/ApiKey.java
2 package com.example.EmployeeManagementSystem.Entity;
3
4 import jakarta.persistence.*;
5 import java.time.Instant;
6 import java.util.UUID;
7
8 @Entity
9 @Table(name = "api_keys")
10 public class ApiKey {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @Column(nullable = false, unique = true)
17     private String apiKey;
18
19     @Column(nullable = false)
20     private String keyName;
21
22     @ManyToOne
23     @JoinColumn(name = "employee_id")
24     private Employee employee;
25
26     @ManyToOne
27     @JoinColumn(name = "vendor_id")
28     private Vendor vendor;
29
30     @Column(nullable = false)
31     private String permissions; // Comma-separated permissions
32
33     @Column(nullable = false)
34     private Instant createdAt;
35
36     private Instant expiresAt;
37
38     private Instant lastUsedAt;
39
40     @Column(nullable = false)
41     private boolean active = true;
42
43     @PrePersist
44     public void prePersist() {
45         this.apiKey = "ems_" + UUID.randomUUID().toString().replace("-", "");
46         this.createdAt = Instant.now();
47     }
48
49     // Getters and setters
50     public Long getId() { return id; }
51     public void setId(Long id) { this.id = id; }
52
53     public String getApiKey() { return apiKey; }
54     public void setApiKey(String apiKey) { this.apiKey = apiKey; }
55
56     public String getKeyName() { return keyName; }
57     public void setKeyName(String keyName) { this.keyName = keyName; }
58
59     public Employee getEmployee() { return employee; }
60     public void setEmployee(Employee employee) { this.employee = employee; }
61
62     public Vendor getVendor() { return vendor; }
63     public void setVendor(Vendor vendor) { this.vendor = vendor; }
64
65     public String getPermissions() { return permissions; }
66     public void setPermissions(String permissions) { this.permissions = permissions; }
67
68     public Instant getCreatedAt() { return createdAt; }
69     public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
70
71     public Instant getExpiresAt() { return expiresAt; }

```

```
72     public void setExpiresAt(Instant expiresAt) { this.expiresAt = expiresAt; }
73
74     public Instant getLastUsedAt() { return lastUsedAt; }
75     public void setLastUsedAt(Instant lastUsedAt) { this.lastUsedAt = lastUsedAt; }
76
77     public boolean isActive() { return active; }
78     public void setActive(boolean active) { this.active = active; }
79 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
4 import com.example.EmployeeManagementSystem.Enum.MealSlot;
5 import jakarta.persistence.*;
6
7 import java.time.Instant;
8
9 @Entity
10 @Table(name = "delivery", uniqueConstraints = { @UniqueConstraint(name = "uq_subscription_scheduled_time",
11     columnNames = {"subscription_id", "scheduledDeliveryTime"})})
12 public class Delivery {
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @ManyToOne
18     @JoinColumn(name = "subscription_id")
19     private Subscription subscription;
20     private MealSlot mealSlot;
21     private Instant scheduledDeliveryTime;
22     private Instant actualDeliveryTime;
23
24     @Enumerated(EnumType.STRING)
25     private DeliveryStatus status;
26
27     private Instant createdAt;
28
29     @PrePersist
30     public void prePersist() {
31         createdAt = Instant.now();
32         if (status == null) {
33             status = DeliveryStatus.SCHEDULED;
34         }
35     }
36
37     public MealSlot getMealSlot() {
38         return mealSlot;
39     }
40
41     public void setMealSlot(MealSlot mealSlot) {
42         this.mealSlot = mealSlot;
43     }
44
45     public Subscription getSubscription() {
46         return subscription;
47     }
48
49     public void setSubscription(Subscription subscription) {
50         this.subscription = subscription;
51     }
52
53     public Long getId() {
54         return id;
55     }
56
57     public void setId(Long id) {
58         this.id = id;
59     }
60
61     public Instant getScheduledDeliveryTime() {
62         return scheduledDeliveryTime;
63     }
64
65     public void setScheduledDeliveryTime(Instant scheduledDeliveryTime) {
66         this.scheduledDeliveryTime = scheduledDeliveryTime;
67     }
68
69     public Instant getActualDeliveryTime() {
70         return actualDeliveryTime;
71     }
```

```
72
73     public void setActualDeliveryTime(Instant actualDeliveryTime) {
74         this.actualDeliveryTime = actualDeliveryTime;
75     }
76
77     public DeliveryStatus getStatus() {
78         return status;
79     }
80
81     public void setStatus(DeliveryStatus status) {
82         this.status = status;
83     }
84
85     public Instant getCreatedAt() {
86         return createdAt;
87     }
88
89     public void setCreatedAt(Instant createdAt) {
90         this.createdAt = createdAt;
91     }
92 }
93
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
5 import com.example.EmployeeManagementSystem.Enum.DeviceType;
6 import jakarta.persistence.*;
7
8 import java.math.BigDecimal;
9 import java.time.LocalDate;
10 import java.time.LocalDateTime;
11
12 @Entity
13 public class Device {
14     @Id
15     @GeneratedValue(strategy = GenerationType.AUTO)
16     private Long id;
17
18     private String deviceName;        // e.g. "Dell XPS 15"
19     private String brand;
20
21     @ManyToOne
22     @JoinColumn(name = "vendor_id")
23     private Vendor techVendor;
24
25     @OneToOne(mappedBy = "device")
26     private DeviceAssignment currentAssignment;
27
28     @Enumerated(EnumType.STRING)
29     private DeviceStatus deviceStatus;
30
31     private DeviceType deviceType;
32
33     private LocalDateTime createdAt;
34
35     private LocalDate warrantyExpiryDate;
36
37     private BigDecimal rentPerMonth;
38
39     public Long getId() {
40         return id;
41     }
42
43     public void setId(Long id) {
44         this.id = id;
45     }
46
47     public String getDeviceName() {
48         return deviceName;
49     }
50
51     public void setDeviceName(String deviceName) {
52         this.deviceName = deviceName;
53     }
54
55     public String getBrand() {
56         return brand;
57     }
58
59     public void setBrand(String brand) {
60         this.brand = brand;
61     }
62
63     public Vendor getTechVendor() {
64         return techVendor;
65     }
66
67     public void setTechVendor(Vendor techVendor) {
68         this.techVendor = techVendor;
69     }
70
71     public DeviceType getDeviceType() {
```

```
72         return deviceType;
73     }
74
75     public void setDeviceType(DeviceType deviceType) {
76         this.deviceType = deviceType;
77     }
78
79     public LocalDateTime getCreatedAt() {
80         return createdAt;
81     }
82
83     public void setCreatedAt(LocalDateTime createdAt) {
84         this.createdAt = createdAt;
85     }
86
87     public DeviceStatus getDeviceStatus() {
88         return deviceStatus;
89     }
90
91     public void setDeviceStatus(DeviceStatus deviceStatus) {
92         this.deviceStatus = deviceStatus;
93     }
94
95     public DeviceAssignment getCurrentAssignment() {
96         return currentAssignment;
97     }
98
99     public void setCurrentAssignment(DeviceAssignment currentAssignment) {
100         this.currentAssignment = currentAssignment;
101     }
102
103     public LocalDate getWarrantyExpiryDate() {
104         return warrantyExpiryDate;
105     }
106
107     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
108         this.warrantyExpiryDate = warrantyExpiryDate;
109     }
110
111     public BigDecimal getRentPerMonth() {
112         return rentPerMonth;
113     }
114
115     public void setRentPerMonth(BigDecimal rentPerMonth) {
116         this.rentPerMonth = rentPerMonth;
117     }
118 }
119
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
4 import jakarta.persistence.*;
5
6 import java.time.LocalDate;
7
8 @Entity
9 public class DeviceAssignment {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @OneToOne
16     @JoinColumn(name = "device_id")
17     private Device device;
18
19     @ManyToOne
20     @JoinColumn(name = "employee_id")
21     private Employee assignedTo;
22
23     @Enumerated(EnumType.STRING)
24     private AssignmentStatus status;
25
26     private LocalDate assignedDate;
27
28     public Long getId() {
29         return id;
30     }
31
32     public void setId(Long id) {
33         this.id = id;
34     }
35
36     public Device getDevice() {
37         return device;
38     }
39
40     public void setDevice(Device device) {
41         this.device = device;
42     }
43
44     public Employee getAssignedTo() {
45         return assignedTo;
46     }
47
48     public void setAssignedTo(Employee assignedTo) {
49         this.assignedTo = assignedTo;
50     }
51
52     public LocalDate getAssignedDate() {
53         return assignedDate;
54     }
55
56     public void setAssignedDate(LocalDate assignedDate) {
57         this.assignedDate = assignedDate;
58     }
59
60     public AssignmentStatus getStatus() {
61         return status;
62     }
63
64     public void setStatus(AssignmentStatus status) {
65         this.status = status;
66     }
67 }
68
```



```

1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Enum.Status;
6 import com.fasterxml.jackson.annotation.JsonIgnore;
7 import jakarta.persistence.*;
8 import org.springframework.security.core.GrantedAuthority;
9 import org.springframework.security.core.authority.SimpleGrantedAuthority;
10 import org.springframework.security.core.userdetails.UserDetails;
11
12 import java.time.LocalDate;
13 import java.util.Collection;
14 import java.util.HashSet;
15 import java.util.List;
16 import java.util.Set;
17 import java.util.stream.Collectors;
18
19 @Entity
20 public class Employee implements UserDetails, TotpUser {
21     @Id
22     @GeneratedValue(strategy = GenerationType.IDENTITY)
23     private long employeeId;
24     @Column(nullable = true, unique = false)
25     private String name;
26     @Column(unique = true, nullable = false)
27     private String email;
28     @Column(nullable = true)
29     private String dept;
30     @Enumerated(EnumType.STRING)
31     private Status status;
32     private String password;
33
34     // Self-referential – replaces `String manager`
35     @ManyToOne(fetch = FetchType.LAZY)
36     @JoinColumn(name = "manager_id", nullable = true)
37     @JsonIgnore // prevent infinite recursion in serialization
38     private Employee manager;
39
40
41     private LocalDate joined_at;
42     @Enumerated(EnumType.STRING)
43     private Role role;
44     @Column(nullable = true)
45     private String totpSecret;
46     private boolean totpEnabled = false;
47     /** IANA timezone id, e.g. "Asia/Kolkata", "America/New_York". Defaults to UTC. */
48     @Column(nullable = false)
49     private String timezone = "UTC";
50     @Enumerated(EnumType.STRING)
51     @Column(nullable = true)
52     private Gender gender;
53
54     public String getTimezone() {
55         return timezone;
56     }
57
58     public void setTimezone(String timezone) {
59         this.timezone = timezone;
60     }
61
62     public Role getRole() {
63         return role;
64     }
65
66     public void setRole(Role role) {
67         this.role = role;
68     }
69
70     public long getEmployeeId() {
71         return employeeId;

```

```

72     }
73
74     public void setEmployeeId(long employeeId) {
75         this.employeeId = employeeId;
76     }
77
78     public String getName() {
79         return name;
80     }
81
82     public void setName(String name) {
83         this.name = name;
84     }
85
86     public String getEmail() {
87         return email;
88     }
89
90     public void setEmail(String email) {
91         this.email = email;
92     }
93
94     public String getDept() {
95         return dept;
96     }
97
98     public void setDept(String dept) {
99         this.dept = dept;
100    }
101
102    public Status getStatus() {
103        return status;
104    }
105
106    public void setStatus(Status status) {
107        this.status = status;
108    }
109
110    public LocalDate getJoined_at() {
111        return joined_at;
112    }
113
114    public void setJoined_at(LocalDate joined_at) {
115        this.joined_at = joined_at;
116    }
117
118    public Gender getGender() { return gender; }
119    public void setGender(Gender gender) { this.gender = gender; }
120
121    @PrePersist
122    public void initialSetup() {
123        this.status = Status.ACTIVE;
124        this.joined_at = LocalDate.now();
125        if (this.timezone == null || this.timezone.isBlank()) {
126            this.timezone = "UTC";
127        }
128    }
129
130    @Override
131    public Collection<? extends GrantedAuthority> getAuthorities() {
132        Set<SimpleGrantedAuthority> authorities=new HashSet<>();
133        authorities.add(new SimpleGrantedAuthority("ROLE_"+role.name()));
134        authorities.addAll(role.getPermissions().stream()
135            .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
136            .collect(Collectors.toSet()));
137        return authorities;
138    }
139
140    public String getPassword() {
141        return password;
142    }
143
144    @Override
145    public String getUsername() {
146        return email;
147    }

```

```
148
149     @Override
150     public boolean isAccountNonExpired() {
151         return UserDetails.super.isAccountNonExpired();
152     }
153
154     @Override
155     public boolean isAccountNonLocked() {
156         return UserDetails.super.isAccountNonLocked();
157     }
158
159     @Override
160     public boolean isCredentialsNonExpired() {
161         return UserDetails.super.isCredentialsNonExpired();
162     }
163
164     @Override
165     public boolean isEnabled() {
166         return UserDetails.super.isEnabled();
167     }
168
169     public void setPassword(String password) {
170         this.password = password;
171     }
172
173     public String getTotpSecret() {
174         return totpSecret;
175     }
176
177     public boolean isTotpEnabled() {
178         return totpEnabled;
179     }
180
181     public void setTotpSecret(String totpSecret) {
182         this.totpSecret = totpSecret;
183     }
184
185     public void setTotpEnabled(boolean totpEnabled) {
186         this.totpEnabled = totpEnabled;
187     }
188
189     public Employee getManager() {
190         return manager;
191     }
192
193     public void setManager(Employee manager) {
194         this.manager = manager;
195     }
196
197 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.LocalDate;
6
7 @Entity
8 public class FixedHoliday {
9
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     private String name;          // "Diwali", "Christmas"
16     private LocalDate date;       // 2026-10-20
17     private String description;   // optional note
18     private int year;
19
20
21     public Long getId() {
22         return id;
23     }
24
25     public void setId(Long id) {
26         this.id = id;
27     }
28
29     public String getName() {
30         return name;
31     }
32
33     public void setName(String name) {
34         this.name = name;
35     }
36
37     public LocalDate getDate() {
38         return date;
39     }
40
41     public void setDate(LocalDate date) {
42         this.date = date;
43     }
44
45     public int getYear() {
46         return year;
47     }
48
49     public void setYear(int year) {
50         this.year = year;
51     }
52
53     public String getDescription() {
54         return description;
55     }
56
57     public void setDescription(String description) {
58         this.description = description;
59     }
60 }
61
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.InvoiceStatus;
4 import com.example.EmployeeManagementSystem.Enum.InvoiceType;
5 import jakarta.persistence.Entity;
6 import jakarta.persistence.*;
7
8 import java.math.BigDecimal;
9 import java.time.LocalDate;
10
11 @Entity
12 public class Invoice {
13
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17
18     @Enumerated(EnumType.STRING)
19     private InvoiceType invoiceType; // REPAIR, RENTAL
20
21     @OneToOne
22     @JoinColumn(name = "repair_bill_id", unique = true)
23     private RepairBill repairBill; // null if rental
24
25     @OneToOne
26     @JoinColumn(name = "rental_bill_id", unique = true)
27     private RentalBill rentalBill;
28
29     @ManyToOne
30     @JoinColumn(name = "vendor_id")
31     private Vendor vendor;
32
33     private BigDecimal amount;
34
35     private LocalDate issuedDate;
36
37     @Enumerated(EnumType.STRING)
38     private InvoiceStatus status; // SENT, SEEN, ACKNOWLEDGED
39
40     private LocalDate seenAt; // when vendor marked it received
41
42     public Long getId() {
43         return id;
44     }
45
46     public void setId(Long id) {
47         this.id = id;
48     }
49
50     public RepairBill getRepairBill() {
51         return repairBill;
52     }
53
54     public void setRepairBill(RepairBill repairBill) {
55         this.repairBill = repairBill;
56     }
57
58     public InvoiceType getInvoiceType() {
59         return invoiceType;
60     }
61
62     public void setInvoiceType(InvoiceType invoiceType) {
63         this.invoiceType = invoiceType;
64     }
65
66     public RentalBill getRentalBill() {
67         return rentalBill;
68     }
69
70     public void setRentalBill(RentalBill rentalBill) {
71         this.rentalBill = rentalBill;
```

```
72     }
73
74     public Vendor getVendor() {
75         return vendor;
76     }
77
78     public void setVendor(Vendor vendor) {
79         this.vendor = vendor;
80     }
81
82     public BigDecimal getAmount() {
83         return amount;
84     }
85
86     public void setAmount(BigDecimal amount) {
87         this.amount = amount;
88     }
89
90     public LocalDate getIssuedDate() {
91         return issuedDate;
92     }
93
94     public void setIssuedDate(LocalDate issuedDate) {
95         this.issuedDate = issuedDate;
96     }
97
98     public InvoiceStatus getStatus() {
99         return status;
100     }
101
102     public void setStatus(InvoiceStatus status) {
103         this.status = status;
104     }
105
106     public LocalDate getSeenAt() {
107         return seenAt;
108     }
109
110     public void setSeenAt(LocalDate seenAt) {
111         this.seenAt = seenAt;
112     }
113 }
114
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.math.BigDecimal;
6
7 @Entity
8 @Table(name = "leave_entitlement", uniqueConstraints = @UniqueConstraint(columnNames = {"employee_id",
9 "leave_type_id", "year"}))
10 public class LeaveEntitlement {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @ManyToOne(fetch = FetchType.LAZY, optional = false)
16     @JoinColumn(name = "employee_id", nullable = false)
17     private Employee employee;
18
19     @ManyToOne(fetch = FetchType.LAZY, optional = false)
20     @JoinColumn(name = "leave_type_id", nullable = false)
21     private LeaveType leaveType;
22
23     @Column(nullable = false)
24     private Integer year;
25
26     @Column(nullable = false, precision = 5, scale = 2)
27     private BigDecimal openingBalance = BigDecimal.ZERO;
28
29     @Column(nullable = false, precision = 5, scale = 2)
30     private BigDecimal accruedThisYear = BigDecimal.ZERO;
31
32     @Column(nullable = false, precision = 5, scale = 2)
33     private BigDecimal usedThisYear = BigDecimal.ZERO;
34
35     @Column(nullable = false, precision = 5, scale = 2)
36     private BigDecimal closingBalance = BigDecimal.ZERO;
37
38     // Convenience: current available = opening + accrued - used
39     @Transient
40     public BigDecimal getAvailableBalance() {
41         return openingBalance.add(accruedThisYear).subtract(usedThisYear);
42     }
43
44     // --- getters & setters ---
45     public Long getId() {
46         return id;
47     }
48
49     public Employee getEmployee() {
50         return employee;
51     }
52
53     public LeaveType getLeaveType() {
54         return leaveType;
55     }
56
57     public Integer getYear() {
58         return year;
59     }
60
61     public BigDecimal getOpeningBalance() {
62         return openingBalance;
63     }
64
65     public BigDecimal getAccruedThisYear() {
66         return accruedThisYear;
67     }
68
69     public BigDecimal getUsedThisYear() {
70         return usedThisYear;
```

```
71     }
72
73     public BigDecimal getClosingBalance() {
74         return closingBalance;
75     }
76
77     public void setEmployee(Employee e) {
78         this.employee = e;
79     }
80
81     public void setLeaveType(LeaveType lt) {
82         this.leaveType = lt;
83     }
84
85     public void setYear(Integer y) {
86         this.year = y;
87     }
88
89     public void setOpeningBalance(BigDecimal b) {
90         this.openingBalance = b;
91     }
92
93     public void setAccruedThisYear(BigDecimal a) {
94         this.accruedThisYear = a;
95     }
96
97     public void setUsedThisYear(BigDecimal u) {
98         this.usedThisYear = u;
99     }
100
101     public void setClosingBalance(BigDecimal c) {
102         this.closingBalance = c;
103     }
104 }
```



```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5 import jakarta.persistence.*;
6
7 import java.time.LocalDate;
8
9 @Entity
10 public class LeaveRequest {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private long id;
14     @Enumerated(EnumType.STRING)
15     private LeaveType leaveType;
16     private LocalDate startDate;
17     private LocalDate endDate;
18     private String reason;
19     @Enumerated(EnumType.STRING)
20     private LeaveStatus status;
21     private String Manager;
22     private String remarks;
23     @ManyToOne
24     @JoinColumn(name = "employee_id")
25     private Employee employee;
26
27
28     public long getId() {
29         return id;
30     }
31
32     public void setId(long id) {
33         this.id = id;
34     }
35
36     public LeaveType getLeaveType() {
37         return leaveType;
38     }
39
40     public void setLeaveType(LeaveType leaveType) {
41         this.leaveType = leaveType;
42     }
43
44     public LocalDate getStartDate() {
45         return startDate;
46     }
47
48     public void setStartDate(LocalDate startDate) {
49         this.startDate = startDate;
50     }
51
52     public LocalDate getEndDate() {
53         return endDate;
54     }
55
56     public void setEndDate(LocalDate endDate) {
57         this.endDate = endDate;
58     }
59
60     public LeaveStatus getStatus() {
61         return status;
62     }
63
64     public void setStatus(LeaveStatus status) {
65         this.status = status;
66     }
67
68     public String getManager() {
69         return Manager;
70     }
71 }
```

```
72     public void setManager(String manager) {
73         Manager = manager;
74     }
75
76     public String getRemarks() {
77         return remarks;
78     }
79
80     public void setRemarks(String remarks) {
81         this.remarks = remarks;
82     }
83
84     public Employee getEmployee() {
85         return employee;
86     }
87
88     public void setEmployee(Employee employee) {
89         this.employee = employee;
90     }
91
92     public String getReason() {
93         return reason;
94     }
95
96     public void setReason(String reason) {
97         this.reason = reason;
98     }
99
100     @PrePersist
101     public void init(){
102         this.status=LeaveStatus.PENDING;
103     }
104 }
105
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4 import jakarta.persistence.*;
5
6 @Entity
7 @Table(name = "leave_type")
8 public class LeaveType {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Integer id;
13
14     @Column(nullable = false, unique = true, length = 50)
15     private String name;           // SICK, CASUAL, MATERNITY
16
17     @Column(nullable = false)
18     private boolean carriesForward;
19
20     @Column(nullable = false)
21     private boolean monthlyReset;   // true = sick leave wipes at month end
22
23     @Column(nullable = false)
24     private int maxCarryForward;    // 30 for carry-forwardable types
25
26     @Column(nullable = false)
27     private boolean genderRestricted;
28
29     @Enumerated(EnumType.STRING)
30     @Column(nullable = true)
31     private Gender genderRestriction; // null = open to all
32
33     // --- getters & setters ---
34     public Integer getId() { return id; }
35     public String getName() { return name; }
36     public boolean isCarriesForward() { return carriesForward; }
37     public boolean isMonthlyReset() { return monthlyReset; }
38     public int getMaxCarryForward() { return maxCarryForward; }
39     public boolean isGenderRestricted() { return genderRestricted; }
40     public Gender getGenderRestriction() { return genderRestriction; }
41
42     public void setId(Integer id) { this.id = id; }
43     public void setName(String name) { this.name = name; }
44     public void setCarriesForward(boolean cf) { this.carriesForward = cf; }
45     public void setMonthlyReset(boolean mr) { this.monthlyReset = mr; }
46     public void setMaxCarryForward(int max) { this.maxCarryForward = max; }
47     public void setGenderRestricted(boolean gr) { this.genderRestricted = gr; }
48     public void setGenderRestriction(Gender g) { this.genderRestriction = g; }
49 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDate;
7
8 @Entity
9 @Table(name = "leave_warning")
10 public class LeaveWarning {
11
12     public enum WarningType {
13         APPROACHING_28, // 2 months before cap
14         APPROACHING_29, // 1 month before cap
15         CAP_REACHED,    // exactly at 30
16         DAY_WASTED      // already at 30, day lost
17     }
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private Long id;
22
23     @ManyToOne(fetch = FetchType.LAZY, optional = false)
24     @JoinColumn(name = "employee_id", nullable = false)
25     private Employee employee;
26
27     @Enumerated(EnumType.STRING)
28     @Column(nullable = false, length = 30)
29     private WarningType warningType;
30
31     @Column(nullable = false, columnDefinition = "TEXT")
32     private String message;
33
34     @Column(nullable = false, precision = 5, scale = 2)
35     private BigDecimal carryForwardBalance;
36
37     @Column(nullable = false)
38     private LocalDate warningDate;
39
40     @Column(nullable = false)
41     private boolean isRead = false;
42
43     // --- getters & setters ---
44     public Long getId() { return id; }
45     public Employee getEmployee() { return employee; }
46     public WarningType getWarningType() { return warningType; }
47     public String getMessage() { return message; }
48     public BigDecimal getCarryForwardBalance() { return carryForwardBalance; }
49     public LocalDate getWarningDate() { return warningDate; }
50     public boolean isRead() { return isRead; }
51
52     public void setEmployee(Employee e) { this.employee = e; }
53     public void setWarningType(WarningType wt) { this.warningType = wt; }
54     public void setMessage(String m) { this.message = m; }
55     public void setCarryForwardBalance(BigDecimal b) { this.carryForwardBalance = b; }
56     public void setWarningDate(LocalDate d) { this.warningDate = d; }
57     public void setRead(boolean r) { this.isRead = r; }
58 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "notification")
8 public class Notification {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     @ManyToOne(fetch = FetchType.LAZY)
15     @JoinColumn(name = "employee_id", nullable = false)
16     private Employee recipient;
17
18     @Column(nullable = false)
19     private String message;
20
21     @Column(nullable = false)
22     private String type; // LEAVE_APPROVED, LEAVE_REJECTED, LEAVE_REQUEST, HOLIDAY_ADDED
23
24     @Column(nullable = false)
25     private boolean isRead = false;
26
27     @Column(nullable = false)
28     private LocalDateTime createdAt = LocalDateTime.now();
29
30     // getters and setters
31     public Long getId() { return id; }
32     public Employee getRecipient() { return recipient; }
33     public void setRecipient(Employee recipient) { this.recipient = recipient; }
34     public String getMessage() { return message; }
35     public void setMessage(String message) { this.message = message; }
36     public String getType() { return type; }
37     public void setType(String type) { this.type = type; }
38     public boolean isRead() { return isRead; }
39     public void setRead(boolean read) { isRead = read; }
40     public LocalDateTime getCreatedAt() { return createdAt; }
41 }
42
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.Instant;
6 @Entity
7 public class RefreshToken {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    @Column(nullable = false, unique = true)
13    private String token;
14
15    @Column(nullable = false)
16    private Instant expiryTime;
17
18    @ManyToOne
19    @JoinColumn(name = "employee_id")
20    private Employee employee;
21
22    @ManyToOne
23    @JoinColumn(name = "vendor_id")
24    private Vendor vendor;
25
26    public Long getId() {
27        return id;
28    }
29
30    public String getToken() {
31        return token;
32    }
33
34    public void setToken(String token) {
35        this.token = token;
36    }
37
38    public Instant getExpiryTime() {
39        return expiryTime;
40    }
41
42    public void setExpiryTime(Instant expiryTime) {
43        this.expiryTime = expiryTime;
44    }
45
46    public Employee getEmployee() {
47        return employee;
48    }
49
50    public void setEmployee(Employee employee) {
51        this.employee = employee;
52    }
53
54    public Vendor getVendor() {
55        return vendor;
56    }
57
58    public void setVendor(Vendor vendor) {
59        this.vendor = vendor;
60    }
61 }
62
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.PaymentStatus;
4 import jakarta.persistence.*;
5
6 import java.math.BigDecimal;
7 import java.time.LocalDate;
8
9 @Entity
10 public class RentalBill {
11     @Id
12     @GeneratedValue(strategy = GenerationType.AUTO)
13     private Long id;
14
15     private BigDecimal amount;
16
17     private LocalDate billDate;
18
19     private LocalDate dueDate;
20
21     @ManyToOne
22     @JoinColumn(name = "vendor_id")
23     private Vendor vendor;
24
25     @Enumerated(EnumType.STRING)
26     private PaymentStatus paymentStatus;
27
28     public Long getId() {
29         return id;
30     }
31
32     public void setId(Long id) {
33         this.id = id;
34     }
35
36     public LocalDate getBillDate() {
37         return billDate;
38     }
39
40     public void setBillDate(LocalDate billDate) {
41         this.billDate = billDate;
42     }
43
44     public BigDecimal getAmount() {
45         return amount;
46     }
47
48     public void setAmount(BigDecimal amount) {
49         this.amount = amount;
50     }
51
52     public LocalDate getDueDate() {
53         return dueDate;
54     }
55
56     public void setDueDate(LocalDate dueDate) {
57         this.dueDate = dueDate;
58     }
59
60     public Vendor getVendor() {
61         return vendor;
62     }
63
64     public void setVendor(Vendor vendor) {
65         this.vendor = vendor;
66     }
67
68     public PaymentStatus getPaymentStatus() {
69         return paymentStatus;
70     }
71 }
```

```
72     public void setPaymentStatus(PaymentStatus paymentStatus) {  
73         this.paymentStatus = paymentStatus;  
74     }  
75 }  
76
```



```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.PaymentStatus;
4 import jakarta.persistence.*;
5
6 import java.math.BigDecimal;
7 import java.time.LocalDate;
8
9 @Entity
10 public class RepairBill {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @OneToOne
17     private RepairLog repairLog;
18
19     @ManyToOne
20     private Device device;
21
22     private BigDecimal repairCost;
23
24     private LocalDate generatedDate;
25
26     @Enumerated(EnumType.STRING)
27     private PaymentStatus paymentStatus;
28
29     public RepairLog getRepairLog() {
30         return repairLog;
31     }
32
33     public void setRepairLog(RepairLog repairLog) {
34         this.repairLog = repairLog;
35     }
36
37     public Device getDevice() {
38         return device;
39     }
40
41     public void setDevice(Device device) {
42         this.device = device;
43     }
44
45     public Long getId() {
46         return id;
47     }
48
49     public void setId(Long id) {
50         this.id = id;
51     }
52
53     public BigDecimal getRepairCost() {
54         return repairCost;
55     }
56
57     public void setRepairCost(BigDecimal repairCost) {
58         this.repairCost = repairCost;
59     }
60
61     public LocalDate getGeneratedDate() {
62         return generatedDate;
63     }
64
65     public void setGeneratedDate(LocalDate generatedDate) {
66         this.generatedDate = generatedDate;
67     }
68
69     public PaymentStatus getPaymentStatus() {
70         return paymentStatus;
71     }
72 }
```

```
72
73     public void setPaymentStatus(PaymentStatus paymentStatus) {
74         this.paymentStatus = paymentStatus;
75     }
76 }
77
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.math.BigDecimal;
6 import java.time.LocalDate;
7
8 @Entity
9 @Table(name = "repair_log")
10 public class RepairLog {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14     @ManyToOne
15     @JoinColumn(name = "device_id", nullable = false)
16     private Device device;
17
18     @OneToOne
19     @JoinColumn(name = "service_request_id", nullable = false, unique = true)
20     private ServiceRequest serviceRequest;
21
22     private String issueType;
23
24     private String damagedComponent;
25
26     private String repairAction;
27
28     private String replacedComponent;
29
30     private String repairedBy;
31
32     private LocalDate repairDate;
33
34     private String remarks;
35
36     public Long getId() {
37         return id;
38     }
39
40     public void setId(Long id) {
41         this.id = id;
42     }
43
44     public Device getDevice() {
45         return device;
46     }
47
48     public void setDevice(Device device) {
49         this.device = device;
50     }
51
52     public ServiceRequest getServiceRequest() {
53         return serviceRequest;
54     }
55
56     public void setServiceRequest(ServiceRequest serviceRequest) {
57         this.serviceRequest = serviceRequest;
58     }
59
60     public String getIssueType() {
61         return issueType;
62     }
63
64     public void setIssueType(String issueType) {
65         this.issueType = issueType;
66     }
67
68     public String getDamagedComponent() {
69         return damagedComponent;
70     }
71 }
```

```
72     public void setDamagedComponent(String damagedComponent) {
73         this.damagedComponent = damagedComponent;
74     }
75
76     public String getRepairAction() {
77         return repairAction;
78     }
79
80     public void setRepairAction(String repairAction) {
81         this.repairAction = repairAction;
82     }
83
84     public String getReplacedComponent() {
85         return replacedComponent;
86     }
87
88     public void setReplacedComponent(String replacedComponent) {
89         this.replacedComponent = replacedComponent;
90     }
91
92     public String getRepairedBy() {
93         return repairedBy;
94     }
95
96     public void setRepairedBy(String repairedBy) {
97         this.repairedBy = repairedBy;
98     }
99
100    public LocalDate getRepairDate() {
101        return repairDate;
102    }
103
104    public void setRepairDate(LocalDate repairDate) {
105        this.repairDate = repairDate;
106    }
107
108    public String getRemarks() {
109        return remarks;
110    }
111
112    public void setRemarks(String remarks) {
113        this.remarks = remarks;
114    }
115 }
116
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import jakarta.persistence.*;
5 import java.util.Set;
6
7
8 @Entity
9 public class Restaurant {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(nullable = false)
16     private String name;
17
18     @Column(nullable = false)
19     private String address;
20
21     private String description;
22
23     @ElementCollection(targetClass = MealSlot.class, fetch = FetchType.EAGER)
24     @CollectionTable(name = "restaurant_meal_slots",
25         joinColumns = @JoinColumn(name = "restaurant_id"))
26     @Enumerated(EnumType.STRING)
27     @Column(name = "meal_slot")
28     private Set<MealSlot> supportedMealSlots;
29
30     private boolean active = true;
31
32     @ManyToOne(fetch = FetchType.LAZY)
33     @JoinColumn(name = "vendor_id", nullable = false)
34     private Vendor vendor;
35
36     public Long getId() { return id; }
37     public void setId(Long id) { this.id = id; }
38
39     public String getName() { return name; }
40     public void setName(String name) { this.name = name; }
41
42     public String getAddress() { return address; }
43     public void setAddress(String address) { this.address = address; }
44
45     public String getDescription() { return description; }
46     public void setDescription(String description) { this.description = description; }
47
48     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
49     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
50         this.supportedMealSlots = supportedMealSlots;
51     }
52
53     public boolean isActive() { return active; }
54     public void setActive(boolean active) { this.active = active; }
55
56     public Vendor getVendor() { return vendor; }
57     public void setVendor(Vendor vendor) { this.vendor = vendor; }
58 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4 import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;
5 import jakarta.persistence.*;
6
7 import java.time.LocalDateTime;
8
9 @Entity
10 public class ServiceRequest {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @ManyToOne
16     @JoinColumn(name = "device_id")
17     private Device device;
18
19     @ManyToOne
20     @JoinColumn(name = "raised_by")
21     private Employee raisedBy; // employee reporting issue
22
23     @ManyToOne
24     @JoinColumn(name = "reviewed_by")
25     private Employee reviewedBy; // admin who reviewed
26
27     @Enumerated(EnumType.STRING)
28     private ServiceRequestType requestType;
29
30     @Enumerated(EnumType.STRING)
31     private ServiceRequestStatus status;
32
33     private String issueDescription; // "screen cracked", "won't boot"
34     private String adminRemarks; // admin notes on decision
35     private String resolution; // what was done finally
36
37     private boolean urgent; // employee can flag as urgent
38
39     private LocalDateTime raisedAt;
40     private LocalDateTime resolvedAt;
41
42     @PrePersist
43     void onCreate() { this.raisedAt = LocalDateTime.now(); }
44
45     public Employee getRaisedBy() {
46         return raisedBy;
47     }
48
49     public void setRaisedBy(Employee raisedBy) {
50         this.raisedBy = raisedBy;
51     }
52
53     public Device getDevice() {
54         return device;
55     }
56
57     public void setDevice(Device device) {
58         this.device = device;
59     }
60
61     public Long getId() {
62         return id;
63     }
64
65     public void setId(Long id) {
66         this.id = id;
67     }
68
69     public Employee getReviewedBy() {
70         return reviewedBy;
71     }
```

```
72
73     public void setReviewedBy(Employee reviewedBy) {
74         this.reviewedBy = reviewedBy;
75     }
76
77     public ServiceRequestStatus getStatus() {
78         return status;
79     }
80
81     public void setStatus(ServiceRequestStatus status) {
82         this.status = status;
83     }
84
85     public ServiceRequestType getRequestType() {
86         return requestType;
87     }
88
89     public void setRequestType(ServiceRequestType requestType) {
90         this.requestType = requestType;
91     }
92
93     public String getIssueDescription() {
94         return issueDescription;
95     }
96
97     public void setIssueDescription(String issueDescription) {
98         this.issueDescription = issueDescription;
99     }
100
101     public String getAdminRemarks() {
102         return adminRemarks;
103     }
104
105     public void setAdminRemarks(String adminRemarks) {
106         this.adminRemarks = adminRemarks;
107     }
108
109     public String getResolution() {
110         return resolution;
111     }
112
113     public void setResolution(String resolution) {
114         this.resolution = resolution;
115     }
116
117     public boolean isUrgent() {
118         return urgent;
119     }
120
121     public void setUrgent(boolean urgent) {
122         this.urgent = urgent;
123     }
124
125     public LocalDateTime getRaisedAt() {
126         return raisedAt;
127     }
128
129     public void setRaisedAt(LocalDateTime raisedAt) {
130         this.raisedAt = raisedAt;
131     }
132
133     public LocalDateTime getResolvedAt() {
134         return resolvedAt;
135     }
136
137     public void setResolvedAt(LocalDateTime resolvedAt) {
138         this.resolvedAt = resolvedAt;
139     }
140 }
141
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
6 import jakarta.persistence.*;
7
8 import java.time.DayOfWeek;
9 import java.time.Instant;
10 import java.time.LocalDateTime;
11 import java.time.LocalTime;
12 @Entity
13 public class Subscription {
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private long id;
17     @Enumerated(EnumType.STRING)
18     private MealSlot slot;
19     private ScheduleType scheduleType;
20     private DayOfWeek dayOfWeek;
21     @Enumerated(EnumType.STRING)
22     private SubscriptionStatus status;
23     private Instant nextDeliveryTime;
24     private LocalTime created_at;
25
26     @ManyToOne
27     @JoinColumn(name="employee_id")
28     private Employee employee;
29
30     @ManyToOne(fetch = FetchType.EAGER)
31     @JoinColumn(name = "restaurant_id", nullable = false)
32     private Restaurant restaurant;
33
34     @PrePersist
35     public void init(){
36         this.created_at=LocalTime.now();
37     }
38
39     public long getId() {
40         return id;
41     }
42
43     public void setId(long id) {
44         this.id = id;
45     }
46
47     public MealSlot getSlot() {
48         return slot;
49     }
50
51     public void setSlot(MealSlot slot) {
52         this.slot = slot;
53     }
54
55     public ScheduleType getScheduleType() {
56         return scheduleType;
57     }
58
59     public void setScheduleType(ScheduleType scheduleType) {
60         this.scheduleType = scheduleType;
61     }
62
63     public DayOfWeek getDayOfWeek() {
64         return dayOfWeek;
65     }
66
67     public void setDayOfWeek(DayOfWeek dayOfWeek) {
68         this.dayOfWeek = dayOfWeek;
69     }
70
71     public SubscriptionStatus getStatus() {
```



```
72         return status;
73     }
74
75     public void setStatus(SubscriptionStatus status) {
76         this.status = status;
77     }
78
79     public LocalTime getCreated_at() {
80         return created_at;
81     }
82
83     public void setCreated_at(LocalTime created_at) {
84         this.created_at = created_at;
85     }
86
87     public Instant getNextDeliveryTime() {
88         return nextDeliveryTime;
89     }
90
91     public void setNextDeliveryTime(Instant nextDeliveryTime) {
92         this.nextDeliveryTime = nextDeliveryTime;
93     }
94
95     public Employee getEmployee() {
96         return employee;
97     }
98
99     public void setEmployee(Employee employee) {
100         this.employee = employee;
101     }
102     public Restaurant getRestaurant() { return restaurant; }
103     public void setRestaurant(Restaurant restaurant) { this.restaurant = restaurant; }
104 }
105
```

107 Current:

src\main\java\com\example\EmployeeManagementSystem\Entity\TotpUser.java

10
lines

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 public interface TotpUser {
4     String getEmail();
5     String getName();
6     String getTotpSecret();
7     void setTotpSecret(String secret);
8     boolean isTotpEnabled();
9     void setTotpEnabled(boolean enabled);
10 }
```

```

1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import jakarta.persistence.*;
5 import jakarta.xml.bind.annotation.XmlAttribute;
6 import org.jspecify.annotations.Nullable;
7 import org.springframework.security.core.GrantedAuthority;
8 import org.springframework.security.core.authority.SimpleGrantedAuthority;
9 import org.springframework.security.core.userdetails.UserDetails;
10
11 import java.time.LocalDate;
12 import java.util.Collection;
13 import java.util.HashSet;
14 import java.util.List;
15 import java.util.Set;
16 import java.util.stream.Collectors;
17
18 /**
19  * Represents a restaurant vendor/owner.
20  * Vendors can register and manage their restaurants.
21  * They cannot access employee, leave, or subscription APIs.
22  */
23 @Entity
24 @Table(name = "vendor")
25 public class Vendor implements UserDetails, TotpUser {
26
27     @Id
28     @GeneratedValue(strategy = GenerationType.IDENTITY)
29     private Long id;
30
31     @Column(nullable = false)
32     private String name;
33
34     @Column(unique = true, nullable = false)
35     private String email;
36     @Column(name = "totp_secret")
37     private String totpSecret;
38
39     @Column(name = "totp_enabled", nullable = false)
40     private boolean totpEnabled = false;
41     @Column(nullable = false)
42     private String phone;
43
44     @Column(nullable = false)
45     private String password;
46
47     @Enumerated(EnumType.STRING)
48     private Role role;
49
50     private LocalDate registeredAt;
51
52     @PrePersist
53     public void init() {
54         if (this.registeredAt == null) {
55             this.registeredAt = LocalDate.now();
56         }
57     }
58
59     public String getTotpSecret() { return totpSecret; }
60     public void setTotpSecret(String totpSecret) { this.totpSecret = totpSecret; }
61
62     public boolean isTotpEnabled() { return totpEnabled; }
63     public void setTotpEnabled(boolean totpEnabled) { this.totpEnabled = totpEnabled; }
64     public Long getId() { return id; }
65     public void setId(Long id) { this.id = id; }
66
67     public String getName() { return name; }
68     public void setName(String name) { this.name = name; }
69
70     public String getEmail() { return email; }
71     public void setEmail(String email) { this.email = email; }

```

```
72
73     public String getPhone() { return phone; }
74     public void setPhone(String phone) { this.phone = phone; }
75
76     public LocalDate getRegisteredAt() { return registeredAt; }
77     public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
78
79
80     @Override
81     public Collection<? extends GrantedAuthority> getAuthorities() {
82         Role effectiveRole = role;
83         Set<SimpleGrantedAuthority> authorities=new HashSet<>();
84         authorities.add(new SimpleGrantedAuthority("ROLE_"+effectiveRole.name()));
85         authorities.addAll(effectiveRole.getPermissions().stream()
86             .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
87             .collect(Collectors.toSet()));
88         return authorities;
89     }
90
91     @Override
92     public @Nullable String getPassword() {
93         return password;
94     }
95
96     @Override
97     public String getUsername() {
98         return email;
99     }
100
101     @Override
102     public boolean isAccountNonExpired() {
103         return UserDetails.super.isAccountNonExpired();
104     }
105
106     @Override
107     public boolean isAccountNonLocked() {
108         return UserDetails.super.isAccountNonLocked();
109     }
110
111     @Override
112     public boolean isCredentialsNonExpired() {
113         return UserDetails.super.isCredentialsNonExpired();
114     }
115
116     @Override
117     public boolean isEnabled() {
118         return UserDetails.super.isEnabled();
119     }
120
121     public void setPassword(String password) {
122         this.password = password;
123     }
124
125     public Role getRole() {
126         return role;
127     }
128
129     public void setRole(Role role) {
130         this.role = role;
131     }
132 }
133
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.NegotiationStatus;
4 import jakarta.persistence.*;
5
6 import java.math.BigDecimal;
7 import java.time.LocalDateTime;
8
9 @Entity
10 @Table(name = "vendor_negotiation")
11 public class VendorNegotiation {
12
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @ManyToOne(fetch = FetchType.LAZY, optional = false)
18     @JoinColumn(name = "vendor_id", nullable = false)
19     private Vendor vendor;
20
21     @ManyToOne(fetch = FetchType.LAZY)
22     @JoinColumn(name = "device_id")
23     private Device device;
24
25     @Column(nullable = false)
26     private String subject;
27
28     @Column(nullable = false, length = 1000)
29     private String description;
30
31     @Column(precision = 12, scale = 2)
32     private BigDecimal proposedAmount;
33
34     @Enumerated(EnumType.STRING)
35     @Column(nullable = false)
36     private NegotiationStatus status = NegotiationStatus.PENDING;
37
38     @Column(nullable = false)
39     private LocalDateTime createdAt;
40
41     @Column(nullable = false)
42     private LocalDateTime updatedAt;
43
44     @PrePersist
45     public void onCreate() {
46         LocalDateTime now = LocalDateTime.now();
47         this.createdAt = now;
48         this.updatedAt = now;
49         if (this.status == null) {
50             this.status = NegotiationStatus.PENDING;
51         }
52     }
53
54     @PreUpdate
55     public void onUpdate() {
56         this.updatedAt = LocalDateTime.now();
57     }
58
59     public Long getId() { return id; }
60     public void setId(Long id) { this.id = id; }
61
62     public Vendor getVendor() { return vendor; }
63     public void setVendor(Vendor vendor) { this.vendor = vendor; }
64
65     public Device getDevice() { return device; }
66     public void setDevice(Device device) { this.device = device; }
67
68     public String getSubject() { return subject; }
69     public void setSubject(String subject) { this.subject = subject; }
70
71     public String getDescription() { return description; }
```

```
72     public void setDescription(String description) { this.description = description; }
73
74     public BigDecimal getProposedAmount() { return proposedAmount; }
75     public void setProposedAmount(BigDecimal proposedAmount) { this.proposedAmount = proposedAmount; }
76
77     public NegotiationStatus getStatus() { return status; }
78     public void setStatus(NegotiationStatus status) { this.status = status; }
79
80     public LocalDateTime getCreatedAt() { return createdAt; }
81     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
82
83     public LocalDateTime getUpdatedAt() { return updatedAt; }
84     public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }
85 }
86
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.MessageSenderType;
4 import jakarta.persistence.*;
5
6 import java.time.LocalDateTime;
7
8 @Entity
9 @Table(name = "vendor_negotiation_message")
10 public class VendorNegotiationMessage {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne(fetch = FetchType.LAZY, optional = false)
17     @JoinColumn(name = "negotiation_id", nullable = false)
18     private VendorNegotiation negotiation;
19
20     @Enumerated(EnumType.STRING)
21     @Column(nullable = false)
22     private MessageSenderType senderType;
23
24     @Column(nullable = false)
25     private String senderEmail;
26
27     @Column(nullable = false, length = 2000)
28     private String message;
29
30     @Column(nullable = false)
31     private LocalDateTime createdAt;
32
33     @PrePersist
34     public void onCreate() {
35         this.createdAt = LocalDateTime.now();
36     }
37
38     public Long getId() { return id; }
39     public void setId(Long id) { this.id = id; }
40
41     public VendorNegotiation getNegotiation() { return negotiation; }
42     public void setNegotiation(VendorNegotiation negotiation) { this.negotiation = negotiation; }
43
44     public MessageSenderType getSenderType() { return senderType; }
45     public void setSenderType(MessageSenderType senderType) { this.senderType = senderType; }
46
47     public String getSenderEmail() { return senderEmail; }
48     public void setSenderEmail(String senderEmail) { this.senderEmail = senderEmail; }
49
50     public String getMessage() { return message; }
51     public void setMessage(String message) { this.message = message; }
52
53     public LocalDateTime getCreatedAt() { return createdAt; }
54     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
55 }
56
```

```

1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7
8 @Entity
9 @Table(name = "year_end_carry_forward_log")
10 public class YearEndCarryForwardLog {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne(fetch = FetchType.LAZY, optional = false)
17     @JoinColumn(name = "employee_id", nullable = false)
18     private Employee employee;
19
20     @Column(nullable = false)
21     private Integer processedYear;
22
23     @ManyToOne(fetch = FetchType.LAZY, optional = false)
24     @JoinColumn(name = "leave_type_id", nullable = false)
25     private LeaveType leaveType;
26
27     @Column(nullable = false, precision = 5, scale = 2)
28     private BigDecimal closingBalance; // raw closing before cap
29
30     @Column(nullable = false, precision = 5, scale = 2)
31     private BigDecimal cappedBalance; // after global 30-day cap applied
32
33     @Column(nullable = false, precision = 5, scale = 2)
34     private BigDecimal daysWasted = BigDecimal.ZERO;
35
36     @Column(nullable = false, precision = 5, scale = 2)
37     private BigDecimal openingNextYear; // written into next year's entitlement
38
39     @Column(nullable = false)
40     private LocalDateTime processedAt = LocalDateTime.now();
41
42     // --- getters & setters ---
43     public Long getId() { return id; }
44     public Employee getEmployee() { return employee; }
45     public Integer getProcessedYear() { return processedYear; }
46     public LeaveType getLeaveType() { return leaveType; }
47     public BigDecimal getClosingBalance() { return closingBalance; }
48     public BigDecimal getCappedBalance() { return cappedBalance; }
49     public BigDecimal getDaysWasted() { return daysWasted; }
50     public BigDecimal getOpeningNextYear() { return openingNextYear; }
51     public LocalDateTime getProcessedAt() { return processedAt; }
52
53     public void setEmployee(Employee e) { this.employee = e; }
54     public void setProcessedYear(Integer y) { this.processedYear = y; }
55     public void setLeaveType(LeaveType lt) { this.leaveType = lt; }
56     public void setClosingBalance(BigDecimal b) { this.closingBalance = b; }
57     public void setCappedBalance(BigDecimal b) { this.cappedBalance = b; }
58     public void setDaysWasted(BigDecimal d) { this.daysWasted = d; }
59     public void setOpeningNextYear(BigDecimal b) { this.openingNextYear = b; }
60     public void setProcessedAt(LocalDateTime dt) { this.processedAt = dt; }
61 }
62

```



```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum AssignmentStatus {
```

```
4     ACTIVE,
```

```
5     RETURNED,
```

```
6     REVOKED
```

```
7 }
```

```
8
```

113 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java

9
lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeliveryStatus {  
4     SCHEDULED,  
5     IN_PROGRESS,  
6     DELIVERED,  
7     MISSED  
8 }  
9
```

114 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java

12
lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeviceStatus {  
4     AVAILABLE,  
5     ASSIGNED,  
6     SENT_TO_VENDOR,  
7     UNDER_REPAIR,  
8     REPAIR_DONE,  
9     CONDEMNED,  
10    RETURNED_TO_VENDOR  
11 }  
12
```

115 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceType.java

15
lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeviceType {  
4     LAPTOP,  
5     DESKTOP,  
6     PHONE,  
7     TABLET,  
8     MONITOR,  
9     KEYBOARD,  
10    MOUSE,  
11    HEADSET,  
12    OTHER  
13  
14 }  
15
```

116 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\Gender.java

5

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum Gender {  
4     M, F, O  
5 }
```

117 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\InvoiceStatus.java

8
lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum InvoiceStatus {
```

```
4     SENT,
```

```
5     SEEN,
```

```
6     ACKNOWLEDGED
```

```
7 }
```

```
8
```

118 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\InvoiceType.java

7

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum InvoiceType {  
4     REPAIR,  
5     RENTAL  
6 }  
7
```

119 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java

9
lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum LeaveStatus {  
4     PENDING,  
5     APPROVED,  
6     REJECTED,  
7     CANCELLED  
8 }  
9
```


120 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java

9
lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum LeaveType {  
4     SICK,  
5     CASUAL,  
6     MATERNITY,  
7     UNPAID  
8 }  
9
```

121 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java

8

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum MealSlot {  
4     BREAKFAST,  
5     LUNCH,  
6     DINNER  
7 }  
8
```

122 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\MessageSenderType.java

7

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum MessageSenderType {  
4     ADMIN,  
5     VENDOR  
6 }  
7
```

123 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\NegotiationStatus.java

11
lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum NegotiationStatus {  
4     PENDING,  
5     NEGOTIATING,  
6     AGREED,  
7     PAID,  
8     REJECTED,  
9     CLOSED  
10 }  
11
```

124 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\PaymentStatus.java

10
lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum PaymentStatus {  
4     PENDING,  
5     PAID,  
6     ACKNOWLEDGED,  
7     OVERDUE,  
8     CANCELLED  
9 }  
10
```

```
1 package com.example.EmployeeManagementSystem.Enum;
2
3 import java.util.Set;
4
5 public enum Permission {
6     LEAVE_READ,
7     LEAVE_WRITE,
8     LEAVE_UPDATE,
9     LEAVE_CANCEL,
10    SUBSCRIPTION_WRITE,
11    SUBSCRIPTION_READ,
12    SUBSCRIPTION_UPDATE,
13    VENDOR_UPDATE,
14    VENDOR_DELETE,
15    RESTAURANTS_ADD,
16    RESTAURANTS_READ,
17    RESTAURANTS_DELETE,
18    PROFILE_READ,
19    VENDOR_WRITE;
20 }
21
```

```
1 package com.example.EmployeeManagementSystem.Enum;
2
3 import java.util.Collection;
4 import java.util.Set;
5
6 import static com.example.EmployeeManagementSystem.Enum.Permission.*;
7
8
9 public enum Role {
10     EMPLOYEE(Set.of(LEAVE_WRITE,
11         LEAVE_CANCEL,
12         SUBSCRIPTION_WRITE,
13         SUBSCRIPTION_UPDATE,
14         RESTAURANTS_READ,
15         SUBSCRIPTION_READ,
16         PROFILE_READ)),
17     MANAGER(Set.of(LEAVE_UPDATE,
18         LEAVE_WRITE,
19         LEAVE_CANCEL,
20         SUBSCRIPTION_WRITE,
21         RESTAURANTS_READ,
22         SUBSCRIPTION_UPDATE,
23         LEAVE_READ,
24         SUBSCRIPTION_READ,
25         PROFILE_READ)),
26     FOOD_VENDOR(Set.of(
27         VENDOR_WRITE,
28         VENDOR_DELETE,
29         RESTAURANTS_ADD,
30         RESTAURANTS_DELETE,
31         RESTAURANTS_READ,
32         PROFILE_READ
33     )),
34     ADMIN(Set.of(
35         LEAVE_READ,
36         LEAVE_WRITE,
37         LEAVE_UPDATE,
38         LEAVE_CANCEL,
39         RESTAURANTS_READ,
40         SUBSCRIPTION_READ,
41         SUBSCRIPTION_WRITE,
42         SUBSCRIPTION_UPDATE,
43         VENDOR_WRITE,
44         VENDOR_UPDATE,
45         VENDOR_DELETE,
46         PROFILE_READ
47     )),
48     TECH_VENDOR(Set.of(VENDOR_WRITE)),
49     USER(Set.of(PROFILE_READ));
50     private final Set<Permission> permissions;
51
52     Role(Set<Permission> permissions) {
53         this.permissions = permissions;
54     }
55
56     public Set<Permission> getPermissions() {
57         return permissions;
58     }
59 }
60 }
61
```

127 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java

7

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ScheduleType {  
4     DAILY,  
5     WEEKLY  
6 }  
7
```



```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ServiceRequestStatus {  
4     OPEN,  
5     SENT_FOR_REPAIR,  
6     APPROVED, //this is for replacement, return, maintainance  
7     RECEIVED_BY_VENDOR ,  
8     REPAIR_DONE,  
9     REJECTED,  
10    IRREPARABLE,  
11    CLOSED  
12 }  
13
```

129 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestType.java

8

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum ServiceRequestType {
```

```
4     REPAIR,
```

```
5     REPLACEMENT,
```

```
6     RETURN
```

```
7 }
```

```
8
```

130 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\Status.java

8

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum Status {
```

```
4     ACTIVE,
```

```
5     INACTIVE,
```

```
6     ON_LEAVE
```

```
7 }
```

```
8
```

131 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java

8

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum SubscriptionStatus {
```

```
4     ACTIVE,
```

```
5     PAUSED,
```

```
6     EXPIRED
```

```
7 }
```

```
8
```

132 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class DuplicateRequestException extends RuntimeException {
4     public DuplicateRequestException(String message) {
5         super(message);
6     }
7 }
8
```

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 import org.springframework.security.core.userdetails.UsernameNotFoundException;
4
5 public class EmployeeNotFound extends UsernameNotFoundException {
6     public EmployeeNotFound(String message) {
7         super(message);
8     }
9 }
10
```

134 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidEndDateException extends RuntimeException{
4     public InvalidEndDateException(String message) {
5         super(message);
6     }
7 }
8
```

135 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidManagerException extends RuntimeException{
4     public InvalidManagerException(String message) {
5         super(message);
6     }
7 }
8
```


136 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidStartDateException extends RuntimeException{
4     public InvalidStartDateException(String message) {
5         super(message);
6     }
7 }
8
```

137 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundExcep

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class LeaveRequestNotFoundException extends RuntimeException{
4     public LeaveRequestNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

138 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.j

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class OverlappingLeaveException extends RuntimeException{
4     public OverlappingLeaveException(String message) {
5         super(message);
6     }
7 }
8
```

139 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\RentalBillNotFoundException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class RentalBillNotFoundException extends RuntimeException{
4     public RentalBillNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

140 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\RepairBillNotFoundException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class RepairBillNotFoundException extends RuntimeException{
4     public RepairBillNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

141 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class RestaurantNotFoundException extends RuntimeException {
4     public RestaurantNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

142 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class SubscriptionAlreadyExists extends RuntimeException{
4     public SubscriptionAlreadyExists(String message) {
5         super(message);
6     }
7 }
8
```

143 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class UnauthorizedAccessException extends RuntimeException {
4     public UnauthorizedAccessException(String message) {
5         super(message);
6     }
7 }
```


144 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class VendorNotFoundException extends RuntimeException {
4     public VendorNotFoundException(String message) {
5         super(message);
6     }
7 }
```

```
1 // src/main/java/com/example/EmployeeManagementSystem/Filter/ApiKeyFilter.java
2 package com.example.EmployeeManagementSystem.Filter;
3
4 import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationToken;
5 import jakarta.servlet.FilterChain;
6 import jakarta.servlet.ServletException;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.beans.factory.annotation.Qualifier;
11 import org.springframework.security.authentication.AuthenticationManager;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.security.core.context.SecurityContextHolder;
14 import org.springframework.stereotype.Component;
15 import org.springframework.web.filter.OncePerRequestFilter;
16
17 import java.io.IOException;
18 public class ApiKeyFilter extends OncePerRequestFilter {
19
20     private static final String API_KEY_HEADER = "X-API-Key";
21     private final AuthenticationManager authenticationManager;
22
23     public ApiKeyFilter(AuthenticationManager authenticationManager) {
24         this.authenticationManager = authenticationManager;
25     }
26
27     @Override
28     protected void doFilterInternal(HttpServletRequest request,
29                                     HttpServletResponse response,
30                                     FilterChain filterChain) throws ServletException, IOException {
31
32         String apiKey = request.getHeader(API_KEY_HEADER);
33
34         if (apiKey != null && !apiKey.isEmpty()) {
35             try {
36                 // Create the unauthenticated token
37                 ApiKeyAuthenticationToken authRequest = new ApiKeyAuthenticationToken(apiKey);
38                 // Run it through the provider – this hits the DB, checks expiry, loads roles
39                 Authentication authenticated = authenticationManager.authenticate(authRequest);
40                 // Only set it if validation passed
41                 SecurityContextHolder.getContext().setAuthentication(authenticated);
42             } catch (RuntimeException e) {
43                 // Invalid/expired key – clear context and return 401
44                 SecurityContextHolder.clearContext();
45                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
46                 response.setContentType("application/json");
47                 response.getWriter().write("{\"error\": \"" + e.getMessage() + "\"}");
48                 return;
49             }
50         }
51
52         filterChain.doFilter(request, response);
53     }
54
55     @Override
56     protected boolean shouldNotFilter(HttpServletRequest request) {
57         String path = request.getRequestURI();
58         return path.startsWith("/Authenticate") ||
59             path.startsWith("/session/login") ||
60             path.startsWith("/session/logout") ||
61             path.startsWith("/oauth2") ||
62             path.startsWith("/login/oauth2") ||
63             path.startsWith("/swagger-ui") ||
64             path.startsWith("/v3/api-docs") ||
65             path.startsWith("/api-keys/generate"); // Allow API key generation without API key
66     }
67 }
```

```

1  package com.example.EmployeeManagementSystem.Filter;
2
3  import com.example.EmployeeManagementSystem.Util.JWTUtil;
4  import io.jsonwebtoken.ExpiredJwtException;
5  import io.jsonwebtoken.MalformedJwtException;
6  import io.jsonwebtoken.UnsupportedJwtException;
7  import jakarta.servlet.FilterChain;
8  import jakarta.servlet.ServletException;
9  import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.beans.factory.annotation.Qualifier;
13 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
14 import org.springframework.security.core.context.SecurityContextHolder;
15 import org.springframework.security.core.userdetails.UserDetails;
16 import org.springframework.security.core.userdetails.UserDetailsService;
17 import org.springframework.security.core.userdetails.UsernameNotFoundException;
18 import org.springframework.stereotype.Component;
19 import org.springframework.web.filter.OncePerRequestFilter;
20
21 import java.io.IOException;
22
23 @Component
24 public class JwtAuthFilter extends OncePerRequestFilter {
25
26     @Autowired
27     @Qualifier("combinedUserDetailService")
28     private UserDetailsService customUserDetailsService;
29
30     @Autowired
31     private JWTUtil jwtUtil;
32
33     @Override
34     protected void doFilterInternal(HttpServletRequest request,
35                                     HttpServletResponse response,
36                                     FilterChain filterChain) throws ServletException, IOException {
37         String token = null;
38         String username = null;
39
40         String authHeader = request.getHeader("Authorization");
41
42         if (authHeader != null && authHeader.startsWith("Bearer ")) {
43             token = authHeader.substring(7).trim();
44
45             // Reject obviously invalid values before hitting JWT
46             if (token.isEmpty() || token.equals("null") || token.equals("undefined")) {
47                 filterChain.doFilter(request, response);
48                 return;
49             }
50
51             try {
52                 username = jwtUtil.extractUsernameFromToken(token);
53             } catch (ExpiredJwtException e) {
54                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
55                 response.setContentType("application/json");
56                 response.getWriter().write("{\"error\": \"TOKEN_EXPIRED\", \"message\": \"JWT expired. Please login again to get a new token.\"}");
57                 return;
58             } catch (MalformedJwtException | UnsupportedJwtException | IllegalArgumentException e) {
59                 // Token is garbage (e.g. "null" string, garbled value) –
60                 // don't crash. OAuth2 session auth will still work fine.
61                 logger.debug("JwtAuthFilter: skipping malformed token – " + e.getMessage());
62                 filterChain.doFilter(request, response);
63                 return;
64             }
65         }
66     }
67 }

```

```
72         if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
73             UserDetails userDetails;
74             try {
75                 userDetails = customUserDetailsService.loadUserByUsername(username);
76             } catch (UsernameNotFoundException e) {
77                 SecurityContextHolder.clearContext();
78                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
79                 response.setContentType("application/json");
80                 response.getWriter().write("""
81                     {
82                         "error": "USER_NOT_FOUND",
83                         "message": "Token subject no longer matches an active user. Please login again."
84                     }
85                 """);
86                 return;
87             }
88             if (userDetails != null && jwtUtil.validateToken(username, userDetails, token)) {
89                 UsernamePasswordAuthenticationToken authenticationToken =
90                     new UsernamePasswordAuthenticationToken(
91                         userDetails, null, userDetails.getAuthorities());
92                 SecurityContextHolder.getContext().setAuthentication(authenticationToken);
93             }
94         }
95
96         filterChain.doFilter(request, response);
97     }
98 }
99
```

```
1 // src/main/java/com/example/EmployeeManagementSystem/Repository/ApiKeyRepository.java
2 package com.example.EmployeeManagementSystem.Repository;
3
4 import com.example.EmployeeManagementSystem.Entity.ApiKey;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Modifying;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9 import org.springframework.transaction.annotation.Transactional;
10
11 import java.time.Instant;
12 import java.util.List;
13 import java.util.Optional;
14
15 public interface ApiKeyRepository extends JpaRepository<ApiKey, Long> {
16
17     Optional<ApiKey> findByApiKeyAndActiveTrue(String apiKey);
18
19
20     List<ApiKey> findByEmployee_EmployeeIdAndActiveTrue(Long employeeId);
21
22
23     List<ApiKey> findByVendor_IdAndActiveTrue(Long vendorId);
24
25     @Modifying
26     @Transactional
27     @Query("UPDATE ApiKey a SET a.lastUsedAt = :now WHERE a.apiKey = :apiKey")
28     void updateLastUsed(@Param("apiKey") String apiKey, @Param("now") Instant now);
29
30     @Modifying
31     @Transactional
32     @Query("UPDATE ApiKey a SET a.active = false WHERE a.expiresAt < :now AND a.active = true")
33     int expireOldKeys(@Param("now") Instant now);
34 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Delivery;
4 import com.example.EmployeeManagementSystem.Entity.Subscription;
5 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9
10 import java.time.Instant;
11 import java.util.List;
12
13 public interface DeliveryRepository extends JpaRepository<Delivery, Long> {
14     List<Delivery> findByStatusAndScheduledDeliveryTimeLessThanEqual(
15         DeliveryStatus status, Instant time);
16
17     @Query("SELECT d FROM Delivery d WHERE d.subscription = :subscription " +
18         "AND d.status != :status ORDER BY d.scheduledDeliveryTime DESC")
19     List<Delivery> findRecentDeliveriesBySubscription(
20         @Param("subscription") Subscription subscription,
21         @Param("status") DeliveryStatus status);
22 }
23
```

149 Current:**src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java**

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
4 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8
9 import java.util.List;
10 import java.util.Optional;
11
12 public interface DeviceAssignmentRepo extends JpaRepository<DeviceAssignment, Long> {
13     Optional<DeviceAssignment> findByDeviceIdAndStatus(Long deviceId, AssignmentStatus status);
14     List<DeviceAssignment> findByAssignedToEmployeeIdAndStatus(Long employeeId, AssignmentStatus status);
15     List<DeviceAssignment> findByDeviceIdOrderByAssignedDateDescIdDesc(Long deviceId);
16
17     @Query(value = "SELECT da.* FROM device_assignment da JOIN device d ON da.device_id = d.id WHERE d.vendor_id = :vendorId", nativeQuery = true)
18     List<DeviceAssignment> getAllAssignmentsByVendorId(@Param("vendorId") long vendorId);
19
20
21 }
22
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Device;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9
10 import java.util.List;
11
12 public interface DeviceRepository extends JpaRepository<Device,Long> {
13     List<Device> findByTechVendorId(Long vendorId);
14     List<Device> findByDeviceStatus(DeviceStatus deviceStatus);
15     @Query(value = "select * from device d where d.device_status != :deviceStatus",nativeQuery = true)
16     List<Device> findByStatusNot(@Param("deviceStatus") String deviceStatus);
17
18     @Query(value = "select * from device d where d.vendor_id=:vendorId AND d.device_status IN
19 (:status)",nativeQuery = true)
20     List<Device> findByStatus(@Param("vendorId") long vendorId,@Param("status") List<String> status);
21
22     List<Device> findByTechVendorAndDeviceStatus(Vendor vendor,DeviceStatus deviceStatus);
23 }
24
```



```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Enum.Status;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9 import org.springframework.stereotype.Repository;
10
11 import java.util.List;
12 import java.util.Optional;
13
14 @Repository
15 public interface EmployeeRepo extends JpaRepository<Employee, Long> {
16     Optional<Employee> findByEmail(String email);
17     Optional<Employee> findByName(String name);
18     long countByRole(Role role);
19
20     List<Employee> findByManager(Employee manager);
21     List<Employee> findByStatus(Status status);
22     List<Employee> findByRole(Role role);
23     @Query("SELECT e FROM Employee e WHERE e.manager.employeeId = :managerId")
24     List<Employee> findByManagerEmployeeId(@Param("managerId") Long managerId);
25     List<Employee> findByStatusAndRole(Status status, Role role);
26 }
27
```

152 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\FixedHolidayRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.FixedHoliday;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface FixedHolidayRepository extends JpaRepository<FixedHoliday, Long> {
10     List<FixedHoliday> findByYear(int year);
11     List<FixedHoliday> findByDateBetween(LocalDate start, LocalDate end);
12 }
13
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Invoice;
4 import com.example.EmployeeManagementSystem.Entity.RentalBill;
5 import com.example.EmployeeManagementSystem.Entity.RepairBill;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.InvoiceStatus;
8 import org.springframework.data.jpa.repository.JpaRepository;
9
10 import java.util.Collection;
11 import java.util.List;
12
13 public interface InvoiceRepository extends JpaRepository<Invoice, Long> {
14     List<Invoice> findByVendorAndStatus(Vendor current, InvoiceStatus invoiceStatus);
15
16     boolean existsByRepairBill(RepairBill repairBill);
17
18     boolean existsByRentalBill(RentalBill rentalBill);
19 }
20
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7
8 import java.math.BigDecimal;
9 import java.util.List;
10 import java.util.Optional;
11
12 public interface LeaveEntitlementRepository extends JpaRepository<LeaveEntitlement, Long> {
13
14     Optional<LeaveEntitlement> findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
15         Long employeeId, Integer leaveTypeId, Integer year);
16
17     List<LeaveEntitlement> findByEmployeeEmployeeIdAndYear(Long employeeId, Integer year);
18
19     @Query("""
20         SELECT COALESCE(SUM(e.closingBalance), 0)
21         FROM LeaveEntitlement e
22         WHERE e.employee.employeeId = :empId
23             AND e.leaveType.carriesForward = true
24             AND e.year IN :years
25         """)
26     BigDecimal sumCarryForwardBalances(
27         @Param("empId") Long employeeId,
28         @Param("years") List<Integer> years);
29
30     List<LeaveEntitlement> findByLeaveTypeIdAndYear(Integer leaveTypeId, Integer year);
31 }
```

```

1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
5 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
6 import com.example.EmployeeManagementSystem.Enum.LeaveType;
7 import org.springframework.data.jpa.repository.JpaRepository;
8 import org.springframework.data.jpa.repository.Query;
9 import org.springframework.data.repository.query.Param;
10
11 import java.time.LocalDate;
12 import java.util.List;
13
14 public interface LeaveRequestRepo extends JpaRepository<LeaveRequest, Long> {
15
16     List<LeaveRequest> findByStatus(LeaveStatus status);
17
18     List<LeaveRequest> findByStatusAndEmployeeNot(LeaveStatus status, Employee employee);
19
20     /**
21      * Find leave requests by start date and status
22      * Used by scheduler to find leaves starting today
23      */
24     List<LeaveRequest> findByStartDateAndStatus(LocalDate startDate, LeaveStatus status);
25
26     // Fix the checkDuplicate query (fix parameter names)
27     @Query(value = "SELECT COUNT(*) " +
28         "FROM leave_request l " +
29         "WHERE l.employee_id = :employeeId " +
30         "AND l.start_date = :startDate " +
31         "AND l.end_date = :endDate " +
32         "AND l.status=:status",nativeQuery = true)
33     long checkDuplicate(@Param("employeeId") long employeeId,
34         @Param("startDate") LocalDate startDate,
35         @Param("endDate") LocalDate endDate,
36         @Param("status") String status);
37
38     // Fix overlapping leave query (was commented in your code)
39     @Query(value = "select COUNT(*) " +
40         "from leave_request l " +
41         "where l.employee_id=:employeeId " +
42         "AND l.status IN ('APPROVED','PENDING') " +
43         "AND l.start_date <= :endDate " +
44         "AND l.end_date >= :startDate",nativeQuery = true)
45     long countOverlappingLeave(
46         @Param("employeeId") long employeeId,
47         @Param("startDate") LocalDate startDate,
48         @Param("endDate") LocalDate endDate
49     );
50
51     // Add missing methods
52     List<LeaveRequest> findByEmployeeOrderByStartDateDesc(Employee employee);
53
54
55     List<LeaveRequest> findByEndDateAndStatus(LocalDate endDate, LeaveStatus status);
56
57     // Add for leave duration validation
58     @Query("SELECT COALESCE(SUM(DATEDIFF(l.endDate, l.startDate) + 1), 0) " +
59         "FROM LeaveRequest l WHERE l.employee = :employee " +
60         "AND l.leaveType = :leaveType " +
61         "AND YEAR(l.startDate) = :year " +
62         "AND l.status IN ('APPROVED', 'PENDING')")
63     long countDaysByEmployeeAndLeaveTypeAndYear(@Param("employee") Employee employee,
64         @Param("leaveType") LeaveType leaveType,
65         @Param("year") int year);
66
67     @Query("SELECT lr FROM LeaveRequest lr JOIN FETCH lr.employee WHERE lr.status = :status")
68     List<LeaveRequest> findApprovedLeavesWithEmployee(LeaveStatus status);
69
70
71     List<LeaveRequest> findByEmployee_Manager(Employee manager);

```

```

72
73 List<LeaveRequest> findByStatusAndEmployee_Manager(LeaveStatus status, Employee manager);
74
75 @Query("""
76 SELECT COALESCE(SUM(DATEDIFF(lr.endDate, lr.startDate) + 1), 0)
77 FROM LeaveRequest lr
78 WHERE lr.employee = :employee
79 AND lr.leaveType = com.example.EmployeeManagementSystem.Enum.LeaveType.SICK
80 AND lr.status = com.example.EmployeeManagementSystem.Enum.LeaveStatus.APPROVED
81 AND YEAR(lr.startDate) = :year
82 AND MONTH(lr.startDate) = :month
83 """)
84 long countApprovedSickDaysInMonth(
85     @Param("employee") Employee employee,
86     @Param("year") int year,
87     @Param("month") int month);
88
89 @Query("""
90 SELECT COALESCE(SUM(DATEDIFF(lr.endDate, lr.startDate) + 1), 0)
91 FROM LeaveRequest lr
92 WHERE lr.employee = :employee
93 AND lr.leaveType = com.example.EmployeeManagementSystem.Enum.LeaveType.CASUAL
94 AND lr.status IN (
95     com.example.EmployeeManagementSystem.Enum.LeaveStatus.APPROVED,
96     com.example.EmployeeManagementSystem.Enum.LeaveStatus.PENDING
97 )
98 AND YEAR(lr.startDate) = :year
99 AND MONTH(lr.startDate) = :month
100 """)
101 long countCasualDaysInMonth(
102     @Param("employee") Employee employee,
103     @Param("year") int year,
104     @Param("month") int month
105 );
106
107 @Query("SELECT COUNT(l) > 0 FROM LeaveRequest l " +
108     "WHERE l.employee = :employee " +
109     "AND l.leaveType = :leaveType " +
110     "AND YEAR(l.startDate) = :year " +
111     "AND l.status NOT IN (:statuses)")
112 boolean existsMaternityLeaveForYear(
113     @Param("employee") Employee employee,
114     @Param("leaveType") LeaveType leaveType,
115     @Param("year") int year,
116     @Param("statuses") List<LeaveStatus> statuses
117 );
118
119 @Query("""
120 SELECT COALESCE(SUM(DATEDIFF(lr.endDate, lr.startDate) + 1), 0)
121 FROM LeaveRequest lr
122 WHERE lr.employee = :employee
123 AND lr.leaveType = com.example.EmployeeManagementSystem.Enum.LeaveType.UNPAID
124 AND lr.status IN (
125     com.example.EmployeeManagementSystem.Enum.LeaveStatus.APPROVED,
126     com.example.EmployeeManagementSystem.Enum.LeaveStatus.PENDING
127 )
128 AND YEAR(lr.startDate) = :year
129 AND MONTH(lr.startDate) = :month
130 """)
131 long countUnpaidDaysInMonth(
132     @Param("employee") Employee employee,
133     @Param("year") int year,
134     @Param("month") int month
135 );
136
137 // In LeaveRequestRepository
138 @Query("""
139 SELECT COUNT(lr) > 0 FROM LeaveRequest lr
140 WHERE lr.employee.employeeId = :employeeId
141 AND lr.leaveType = 'SICK'
142 AND lr.status = 'APPROVED'
143 AND YEAR(lr.startDate) = :year
144 AND MONTH(lr.startDate) = :month
145 """)
146 boolean existsApprovedSickLeaveForMonth(
147     @Param("employeeId") Long employeeId,
148     @Param("year") int year,

```

```
148         @Param("month") int month
149     );
150 }
151
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import com.example.EmployeeManagementSystem.Entity.LeaveType;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 public interface LeaveTypeRepository extends JpaRepository<LeaveType, Integer> {
11     Optional<LeaveType> findByName(String name);
12     List<LeaveType> findByMonthlyResetTrue();
13 }
```


157 Current:**src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.java**

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface LeaveWarningRepository extends JpaRepository<LeaveWarning, Long> {
10
11     // All unread warnings for an employee – shown on dashboard
12     List<LeaveWarning> findByEmployeeEmployeeIdAndIsReadFalseOrderByWarningDateDesc(Long employeeId);
13
14     // All warnings for an employee (read + unread) – for history view
15     List<LeaveWarning> findByEmployeeEmployeeIdOrderByWarningDateDesc(Long employeeId);
16
17     // Prevent duplicate warnings for same type on same date
18     boolean existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(
19         Long employeeId, LeaveWarning.WarningType warningType, LocalDate date
20     );
21 }
```

158 Current:**src\main\java\com\example\EmployeeManagementSystem\Repository\NotificationRepository.java**

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Notification;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import java.util.List;
7
8 public interface NotificationRepository extends JpaRepository<Notification, Long> {
9     List<Notification> findByRecipientOrderByCreatedAtDesc(Employee recipient);
10    long countByRecipientAndIsReadFalse(Employee recipient);
11 }
12
```

159 Current:**src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java**

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import org.springframework.data.jpa.repository.JpaRepository;
7
8 import java.util.Optional;
9
10 public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {
11
12     Optional<RefreshToken> findByToken(String token);
13
14     // Needed so EmployeeService can clean up tokens before deleting an employee
15     void deleteByEmployee(Employee employee);
16
17     // Needed so VendorService can clean up tokens before deleting a vendor
18     void deleteByVendor(Vendor vendor);
19 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.RentalBill;
4 import com.example.EmployeeManagementSystem.Enum.PaymentStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.math.BigDecimal;
8 import java.util.List;
9
10 public interface RentalBillRepository extends JpaRepository<RentalBill, Long> {
11     List<RentalBill> findByPaymentStatusNotInAndAmountGreaterThan(List<PaymentStatus> paymentStatus,
12     BigDecimal zero);
13 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.RepairBill;
4 import com.example.EmployeeManagementSystem.Enum.PaymentStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.math.BigDecimal;
8 import java.util.List;
9
10 public interface RepairBillRepository extends JpaRepository<RepairBill, Long> {
11     List<RepairBill> findByPaymentStatusNotInAndRepairCostGreaterThan(
12         List<PaymentStatus> statusList,
13         BigDecimal repairCost
14     );
15
16     List<RepairBill> findByDeviceId(Long deviceId);
17 }
18
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.RepairLog;
4 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 public interface RepairLogRepository extends JpaRepository<RepairLog, Long> {
11     Optional<RepairLog> findByServiceRequest(ServiceRequest serviceRequest);
12     List<RepairLog> findByDeviceIdIn(List<Long> deviceIds);
13     List<RepairLog> findByDeviceIdOrderByRepairDateDescIdDesc(Long deviceId);
14     List<RepairLog> findByDeviceIdAndDeviceTechVendorIdOrderByRepairDateDescIdDesc(Long deviceId, Long
15     vendorId);
16 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Restaurant;
4 import com.example.EmployeeManagementSystem.Enum.MealSlot;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8 import org.springframework.stereotype.Repository;
9
10 import java.util.List;
11
12 @Repository
13 public interface RestaurantRepository extends JpaRepository<Restaurant, Long> {
14
15     /** All active restaurants owned by a vendor. */
16     List<Restaurant> findByVendor_EmailAndActiveTrue(String email);
17
18     /** All active restaurants (for employees browsing available options). */
19     List<Restaurant> findByActiveTrue();
20
21     /**
22      * Active restaurants that support a given meal slot.
23      * Employees use this to discover which restaurants they can subscribe to for a given slot.
24      */
25     @Query("SELECT r FROM Restaurant r JOIN r.supportedMealSlots s " +
26            "WHERE r.active = true AND s = :mealSlot")
27     List<Restaurant> findActiveByMealSlot(@Param("mealSlot") MealSlot mealSlot);
28
29     /** Check if a restaurant belongs to a vendor (ownership guard). */
30     boolean existsByIdAndVendor_Email(Long restaurantId, String email);
31 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3
4 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Device;
7 import com.example.EmployeeManagementSystem.Entity.Vendor;
8 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
9 import jdk.dynalink.linker.LinkerServices;
10 import org.springframework.data.jpa.repository.JpaRepository;
11
12 import java.util.List;
13
14 public interface ServiceRequestRepository extends JpaRepository<ServiceRequest, Long> {
15
16     List<ServiceRequest> findByStatus(ServiceRequestStatus status);
17     List<ServiceRequest> findByRaisedByOrderByRaisedAtDesc(Employee employee);
18     List<ServiceRequest> findByDeviceOrderByRaisedAtDesc(Device device);
19     List<ServiceRequest> findByDeviceTechVendorAndStatus(
20         Vendor vendor,
21         ServiceRequestStatus status
22     );
23
24     List<ServiceRequest> findByRaisedByAndStatusNotOrderByRaisedAtDesc(
25         Employee raisedBy,
26         ServiceRequestStatus status
27     );
28
29     List<ServiceRequest> findByDeviceId(Long deviceId);
30
31 }
32
```


165 Current:**src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java**

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Subscription;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7 import org.springframework.stereotype.Repository;
8
9 import java.time.Instant;
10 import java.util.List;
11
12 @Repository
13 public interface SubscriptionRepository extends JpaRepository<Subscription, Long> {
14     @Query(value = "select count(*) from subscription s where s.employee_id=:employeeId and s.slot=:slot and s.status=:status", nativeQuery = true)
15     int checkSubscriptionExists(
16         @Param("employeeId") long employeeId,
17         @Param("slot") String slot,
18         @Param("status") String status
19     );
20
21     List<Subscription> findByEmployee_employeeId(long id);
22
23     @Query(value = "SELECT * FROM subscription s WHERE s.status = :status AND s.next_delivery_time IS NOT NULL AND s.next_delivery_time <= :currentTime", nativeQuery = true)
24     List<Subscription> findDueSubscriptions(
25         @Param("status") String status,
26         @Param("currentTime") Instant currentTime // was LocalDateTime
27     );
28
29     List<Subscription> findByEmployee_name(String name);
30     List<Subscription> findByEmployee_Email(String email);
31 }
32
```

166 Current:**src\main\java\com\example\EmployeeManagementSystem\Repository\VendorNegotiationMessageR**

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.VendorNegotiationMessage;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.util.List;
7
8 public interface VendorNegotiationMessageRepository extends JpaRepository<VendorNegotiationMessage, Long> {
9     List<VendorNegotiationMessage> findByNegotiationIdOrderByCreatedAtAsc(Long negotiationId);
10 }
11
```

167 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\VendorNegotiationRepository

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.VendorNegotiation;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.util.List;
7
8 public interface VendorNegotiationRepository extends JpaRepository<VendorNegotiation, Long> {
9     List<VendorNegotiation> findByVendorIdOrderByUpdatedAtDesc(Long vendorId);
10    List<VendorNegotiation> findAllByOrderByUpdatedAtDesc();
11    List<VendorNegotiation> findByDeviceId(Long deviceId);
12 }
13
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Vendor;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7
8 import java.util.List;
9 import java.util.Optional;
10
11 @Repository
12 public interface VendorRepo extends JpaRepository<Vendor, Long> {
13     Optional<Vendor> findByEmail(String email);
14     boolean existsByEmail(String email);
15     List<Vendor> findByRole(Role role);
16 }
```

169 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\YearEndCarryForwardLogRe

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3
4 import com.example.EmployeeManagementSystem.Entity.YearEndCarryForwardLog;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8
9 public interface YearEndCarryForwardLogRepository
10     extends JpaRepository<YearEndCarryForwardLog, Long> {
11
12     List<YearEndCarryForwardLog> findByEmployeeEmployeeIdAndProcessedYear(
13         Long employeeId, Integer year
14     );
15
16     // Check if year-end has already been run for this year (idempotency guard)
17     boolean existsByProcessedYear(Integer year);
18 }
```

```
1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Service.DeliveryService;
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.scheduling.annotation.Scheduled;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 public class DeliveryScheduler {
11     private static final Logger logger = LoggerFactory.getLogger(DeliveryScheduler.class);
12     private final DeliveryService deliveryService;
13
14     public DeliveryScheduler(DeliveryService deliveryService) {
15         this.deliveryService = deliveryService;
16     }
17
18     // Runs every hour
19     @Scheduled(cron = "0 0 * * * *")
20     public void processDeliveries() {
21         logger.info("Starting delivery processing scheduler...");
22         try {
23             deliveryService.processDueDeliveries();
24             logger.info("Delivery processing completed successfully");
25         } catch (Exception e) {
26             logger.error("Error processing deliveries: ", e);
27         }
28     }
29 }
30
```

```

1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Service.CarryForwardWarningService;
4 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
5 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
6 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
7 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
8 import org.slf4j.Logger;
9 import org.slf4j.LoggerFactory;
10 import org.springframework.scheduling.annotation.Scheduled;
11 import org.springframework.stereotype.Component;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.time.Instant;
15 import java.time.LocalDate;
16 import java.time.YearMonth;
17
18 @Component
19 public class LeaveAccrualScheduler {
20
21     private static final Logger log = LoggerFactory.getLogger(LeaveAccrualScheduler.class);
22
23     private final LeaveAccrualService accrualService;
24     private final SickLeaveResetService resetService;
25     private final CarryForwardWarningService warningService;
26     private final YearEndCarryForwardService yearEndService;
27     private final ApiKeyRepository apiKeyRepository;
28
29     public LeaveAccrualScheduler(LeaveAccrualService accrualService,
30                                 SickLeaveResetService resetService,
31                                 CarryForwardWarningService warningService,
32                                 YearEndCarryForwardService yearEndService,
33                                 ApiKeyRepository apiKeyRepository) {
34         this.accrualService = accrualService;
35         this.resetService = resetService;
36         this.warningService = warningService;
37         this.yearEndService = yearEndService;
38         this.apiKeyRepository = apiKeyRepository;
39     }
40
41     // 1st of month at 00:05 – accrue 1 day for all eligible employees, then warn
42     @Scheduled(cron = "0 5 0 1 * ?")
43     public void runMonthlyAccrual() {
44         LocalDate today = LocalDate.now();
45         accrualService.runMonthlyAccrual(today);
46         warningService.runWarningCheck(today);
47     }
48
49     // Last day of month at 23:55 – reset unused sick leave
50     // FIX: use YearMonth to compute the actual last day of the current month,
51     //      so if the scheduler fires even a minute late (past midnight) on the
52     //      1st, we still reset the correct month and log the correct date.
53     @Scheduled(cron = "0 55 23 L * ?")
54     public void runSickLeaveReset() {
55         LocalDate lastDayOfMonth = YearMonth.now().atEndOfMonth();
56         log.info("Running sick leave reset for end-of-month: {}", lastDayOfMonth);
57         resetService.runMonthlyReset(lastDayOfMonth);
58     }
59
60     // Dec 31 at 22:00 – year-end carry-forward cap
61     @Scheduled(cron = "0 0 22 31 12 ?")
62     public void runYearEnd() {
63         int closingYear = LocalDate.now().getYear();
64         yearEndService.runYearEnd(closingYear);
65     }
66     @Scheduled(cron = "0 0 0 * * *") // runs every midnight
67     @Transactional
68     public void expireKeys() {
69         apiKeyRepository.expireOldKeys(Instant.now());
70     }
71 }

```

```

1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
5 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
9 import org.slf4j.Logger;
10 import org.slf4j.LoggerFactory;
11 import org.springframework.scheduling.annotation.Scheduled;
12 import org.springframework.stereotype.Component;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.time.LocalDate;
16 import java.time.ZoneId;
17 import java.util.List;
18 import java.util.Map;
19 import java.util.stream.Collectors;
20
21 @Component
22 public class LeaveStatusScheduler {
23
24     private static final Logger log = LoggerFactory.getLogger(LeaveStatusScheduler.class);
25
26     private final LeaveRequestRepo leaveRequestRepo;
27     private final EmployeeRepo employeeRepo;
28
29     public LeaveStatusScheduler(LeaveRequestRepo leaveRequestRepo,
30                               EmployeeRepo employeeRepo) {
31         this.leaveRequestRepo = leaveRequestRepo;
32         this.employeeRepo = employeeRepo;
33     }
34
35     /**
36      * Runs every day at midnight UTC
37      */
38     @Scheduled(cron = "0 0 0 * * *", zone = "UTC")
39     @Transactional
40     public void updateEmployeeLeaveStatus() {
41
42         log.info("=== Starting Timezone-Aware Leave Scheduler ===");
43
44         List<LeaveRequest> leaves =
45             leaveRequestRepo.findApprovedLeavesWithEmployee(LeaveStatus.APPROVED);
46
47         Map<Long, List<LeaveRequest>> employeeLeavesMap = leaves.stream()
48             .collect(Collectors.groupingBy(lr -> lr.getEmployee().getEmployeeId()));
49
50         int onLeaveCount = 0;
51         int activeCount = 0;
52
53         for (Map.Entry<Long, List<LeaveRequest>> entry : employeeLeavesMap.entrySet()) {
54
55             Employee employee = entry.getValue().get(0).getEmployee();
56
57             ZoneId zone = ZoneId.of(
58                 employee.getTimezone() != null ? employee.getTimezone() : "UTC"
59             );
60
61             LocalDate today = LocalDate.now(zone);
62
63             boolean isOnLeaveToday = entry.getValue().stream().anyMatch(leave ->
64                 !today.isBefore(leave.getStartDate()) &&
65                 !today.isAfter(leave.getEndDate())
66             );
67
68             // CASE 1: Should be ON_LEAVE
69             if (isOnLeaveToday && employee.getStatus() != Status.ON_LEAVE) {
70
71                 employee.setStatus(Status.ON_LEAVE);

```



```

72         employeeRepo.save(employee);
73         onLeaveCount++;
74
75         log.info("Employee {} ({} marked ON_LEAVE | TZ: {}",
76                 employee.getEmployeeId(),
77                 employee.getName(),
78                 zone);
79     }
80
81     // CASE 2: Should be ACTIVE
82     if (!isOnLeaveToday && employee.getStatus() == Status.ON_LEAVE) {
83
84         employee.setStatus(Status.ACTIVE);
85         employeeRepo.save(employee);
86         activeCount++;
87
88         log.info("Employee {} ({} marked ACTIVE | TZ: {}",
89                 employee.getEmployeeId(),
90                 employee.getName(),
91                 zone);
92     }
93 }
94
95     log.info("=== Scheduler Completed ===");
96     log.info("ON_LEAVE updated: {}", onLeaveCount);
97     log.info("ACTIVE updated: {}", activeCount);
98 }
99
100 /**
101  * Manual trigger (for testing)
102  */
103 public void runManually() {
104     log.info("Manual scheduler trigger started...");
105     updateEmployeeLeaveStatus();
106 }
107 }

```

```

1  package com.example.EmployeeManagementSystem.Scheduler;
2
3  import com.example.EmployeeManagementSystem.Entity.Device;
4  import com.example.EmployeeManagementSystem.Entity.RentalBill;
5  import com.example.EmployeeManagementSystem.Entity.Vendor;
6  import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
7  import com.example.EmployeeManagementSystem.Enum.PaymentStatus;
8  import com.example.EmployeeManagementSystem.Enum.Role;
9  import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
10 import com.example.EmployeeManagementSystem.Repository.RentalBillRepository;
11 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
12 import org.springframework.scheduling.annotation.Scheduled;
13 import org.springframework.stereotype.Component;
14
15 import java.math.BigDecimal;
16 import java.time.LocalDate;
17 import java.util.List;
18
19 @Component
20 public class MonthlyBillScheduler {
21     private final VendorRepo vendorRepo;
22     private final DeviceRepository deviceRepository;
23     private final RentalBillRepository rentalBillRepository;
24
25
26     public MonthlyBillScheduler(VendorRepo vendorRepo, DeviceRepository deviceRepository,
27 RentalBillRepository rentalBillRepository) {
28         this.vendorRepo = vendorRepo;
29         this.deviceRepository = deviceRepository;
30         this.rentalBillRepository = rentalBillRepository;
31     }
32
33     @Scheduled(cron = "0 0 1 1 * *")
34     public void monthlyBillCalculator(){
35         List<Vendor> vendors=vendorRepo.findByRole(Role.TECH_VENDOR);
36
37         for(Vendor vendor:vendors){
38             List<Device> devices=deviceRepository.findByStatus(vendor.getId(),
39                 List.of(DeviceStatus.ASSIGNED.name(),DeviceStatus.UNDER_REPAIR.name()));
40             BigDecimal amount=BigDecimal.ZERO;
41             RentalBill rentalBill=new RentalBill();
42             rentalBill.setBillDate(LocalDate.now());
43             rentalBill.setDueDate(LocalDate.now().plusDays(7));
44             rentalBill.setVendor(vendor);
45             for(Device device:devices){
46                 amount=amount.add(device.getRentPerMonth());
47             }
48             rentalBill.setAmount(amount);
49             rentalBill.setPaymentStatus(PaymentStatus.PENDING);
50             rentalBillRepository.save(rentalBill);
51         }
52     }
53 }
54

```

```

1 // src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationProvider.java
2 package com.example.EmployeeManagementSystem.security;
3
4 import com.example.EmployeeManagementSystem.Entity.ApiKey;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.security.authentication.AuthenticationProvider;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.core.AuthenticationException;
12 import org.springframework.security.core.GrantedAuthority;
13 import org.springframework.security.core.authority.SimpleGrantedAuthority;
14 import org.springframework.stereotype.Component;
15 import org.springframework.transaction.annotation.Transactional;
16
17 import java.time.Instant;
18 import java.util.ArrayList;
19 import java.util.Arrays;
20 import java.util.Collection;
21 import java.util.List;
22 import java.util.stream.Collectors;
23
24 @Component
25 public class ApiKeyAuthenticationProvider implements AuthenticationProvider {
26
27     private final ApiKeyRepository apiKeyRepository;
28
29     public ApiKeyAuthenticationProvider(ApiKeyRepository apiKeyRepository) {
30         this.apiKeyRepository = apiKeyRepository;
31     }
32
33     @Override
34     @Transactional
35     public Authentication authenticate(Authentication authentication) throws AuthenticationException {
36         String apiKey = null;
37
38         // Extract API key from the authentication object
39         if (authentication instanceof ApiKeyAuthenticationToken) {
40             apiKey = ((ApiKeyAuthenticationToken) authentication).getApiKey();
41         }
42
43         if (apiKey == null || apiKey.isEmpty()) {
44             throw new RuntimeException("Invalid API key");
45         }
46
47         ApiKey key = apiKeyRepository.findByApiKeyAndActiveTrue(apiKey)
48             .orElseThrow(() -> new RuntimeException("Invalid or inactive API key"));
49
50         // Check expiration
51         if (key.getExpiresAt() != null && key.getExpiresAt().isBefore(Instant.now())) {
52             throw new RuntimeException("API key has expired");
53         }
54
55         // Update last used timestamp
56         apiKeyRepository.updateLastUsed(apiKey, Instant.now());
57
58         // Build authorities from permissions
59         List<GrantedAuthority> authorities = new ArrayList<>();
60
61         if (key.getPermissions() != null && !key.getPermissions().isEmpty()) {
62             authorities.addAll(Arrays.stream(key.getPermissions().split(","))
63                 .map(String::trim)
64                 .map(SimpleGrantedAuthority::new)
65                 .collect(Collectors.toList()));
66         }
67
68         Object principal;
69         if (key.getEmployee() != null) {
70             principal = key.getEmployee();
71             authorities.add(new SimpleGrantedAuthority("ROLE_" + key.getEmployee().getRole().name()));

```

```
72         } else if (key.getVendor() != null) {
73             principal = key.getVendor();
74             authorities.add(new SimpleGrantedAuthority("ROLE_" + key.getVendor().getRole().name()));
75         } else {
76             throw new RuntimeException("No user associated with API key");
77         }
78
79         return new ApiKeyAuthenticationToken(principal, apiKey, authorities);
80     }
81
82     @Override
83     public boolean supports(Class<?> authentication) {
84         return ApiKeyAuthenticationToken.class.isAssignableFrom(authentication);
85     }
86 }
```

175 Current:

src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java

```
1 // src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationToken.java
2 package com.example.EmployeeManagementSystem.security;
3
4 import org.springframework.security.core.Authentication;
5 import org.springframework.security.core.GrantedAuthority;
6 import java.util.Collection;
7
8 public class ApiKeyAuthenticationToken implements Authentication {
9
10     private static final long serialVersionUID = 1L;
11
12     private final Object principal;
13     private final Object credentials;
14     private final String apiKey;
15     private final Collection<? extends GrantedAuthority> authorities;
16     private boolean authenticated;
17
18     // Constructor for unauthenticated API key (before validation)
19     public ApiKeyAuthenticationToken(String apiKey) {
20         this.apiKey = apiKey;
21         this.principal = null;
22         this.credentials = null;
23         this.authorities = null;
24         this.authenticated = false;
25     }
26
27     // Constructor for authenticated API key (after validation)
28     public ApiKeyAuthenticationToken(Object principal, Object credentials,
29                                     Collection<? extends GrantedAuthority> authorities) {
30         this.apiKey = null;
31         this.principal = principal;
32         this.credentials = credentials;
33         this.authorities = authorities;
34         this.authenticated = true;
35     }
36
37     @Override
38     public Collection<? extends GrantedAuthority> getAuthorities() {
39         return authorities;
40     }
41
42     @Override
43     public Object getCredentials() {
44         return credentials;
45     }
46
47     @Override
48     public Object getDetails() {
49         return null;
50     }
51
52     @Override
53     public Object getPrincipal() {
54         return principal;
55     }
56
57     @Override
58     public boolean isAuthenticated() {
59         return authenticated;
60     }
61
62     @Override
63     public void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException {
64         this.authenticated = isAuthenticated;
65     }
66
67     @Override
68     public String getName() {
69         if (principal instanceof org.springframework.security.core.userdetails.UserDetails) {
70             return ((org.springframework.security.core.userdetails.UserDetails) principal).getUsername();
71         }
```

```
72         if (principal instanceof com.example.EmployeeManagementSystem.Entity.Employee) {
73             return ((com.example.EmployeeManagementSystem.Entity.Employee) principal).getUsername();
74         }
75         if (principal instanceof com.example.EmployeeManagementSystem.Entity.Vendor) {
76             return ((com.example.EmployeeManagementSystem.Entity.Vendor) principal).getUsername();
77         }
78         return principal != null ? principal.toString() : apiKey;
79     }
80
81     public String getApiKey() {
82         return apiKey;
83     }
84 }
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.ApiKey;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.time.Instant;
13 import java.time.temporal.ChronoUnit;
14 import java.util.List;
15
16 @Service
17 public class ApiKeyService {
18
19     @Autowired
20     private ApiKeyRepository apiKeyRepository;
21     @Autowired private EmployeeRepo employeeRepo;
22     @Autowired private VendorRepo vendorRepo;
23
24     @Transactional
25     public ApiKey generateForVendor(Long vendorId, String name,
26                                     String permissions, int validDays) {
27         Vendor vendor = vendorRepo.findById(vendorId)
28             .orElseThrow(() -> new RuntimeException("Vendor not found"));
29
30         ApiKey key = new ApiKey(); // constructor auto-generates "ems_<uuid>"
31         key.setVendor(vendor);
32         key.setKeyName(name);
33         key.setPermissions(permissions); // e.g. "VENDOR_WRITE" or "TECH_VENDOR"
34         key.setExpiresAt(Instant.now().plus(validDays, ChronoUnit.DAYS));
35         key.setActive(true);
36         return apiKeyRepository.save(key);
37     }
38
39     // Same for employee-issued keys if needed
40     public List<ApiKey> getActiveKeysForVendor(Long vendorId) {
41         return apiKeyRepository.findByVendor_IdAndActiveTrue(vendorId);
42     }
43
44     @Transactional
45     public void revokeKey(Long keyId) {
46         ApiKey key = apiKeyRepository.findById(keyId)
47             .orElseThrow(() -> new RuntimeException("Key not found"));
48         key.setActive(false);
49         apiKeyRepository.save(key);
50     }
51 }
52
```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.InvoiceDTO;
4  import com.example.EmployeeManagementSystem.DTO.RentalBillDTO;
5  import com.example.EmployeeManagementSystem.DTO.RepairBillDTO;
6  import com.example.EmployeeManagementSystem.Entity.Invoice;
7  import com.example.EmployeeManagementSystem.Entity.RentalBill;
8  import com.example.EmployeeManagementSystem.Entity.RepairBill;
9  import com.example.EmployeeManagementSystem.Enum.InvoiceStatus;
10 import com.example.EmployeeManagementSystem.Enum.InvoiceType;
11 import com.example.EmployeeManagementSystem.Enum.PaymentStatus;
12 import com.example.EmployeeManagementSystem.Exception.RentalBillNotFoundException;
13 import com.example.EmployeeManagementSystem.Exception.RepairBillNotFoundException;
14 import com.example.EmployeeManagementSystem.Repository.InvoiceRepository;
15 import com.example.EmployeeManagementSystem.Repository.RentalBillRepository;
16 import com.example.EmployeeManagementSystem.Repository.RepairBillRepository;
17 import org.springframework.http.HttpStatus;
18 import org.springframework.stereotype.Service;
19 import org.springframework.web.server.ResponseStatusException;
20
21 import java.math.BigDecimal;
22 import java.time.LocalDate;
23 import java.util.List;
24
25 @Service
26 public class BillService {
27     private final RepairBillRepository repairBillRepository;
28     private final RentalBillRepository rentalBillRepository;
29     private final InvoiceRepository invoiceRepository;
30
31     public BillService(RepairBillRepository repairBillRepository, RentalBillRepository rentalBillRepository,
InvoiceRepository invoiceRepository) {
32         this.repairBillRepository = repairBillRepository;
33         this.rentalBillRepository = rentalBillRepository;
34         this.invoiceRepository = invoiceRepository;
35     }
36
37     public List<RepairBillDTO> getRepairBills(){
38         List<RepairBill> repairBills=repairBillRepository.findByPaymentStatusNotInAndRepairCostGreaterThan(
39             List.of(PaymentStatus.PAID,PaymentStatus.ACKNOWLEDGED),
40             BigDecimal.ZERO
41         );
42         return repairBills.stream().map(this::toRepairBillDTO).toList();
43     }
44
45     public RepairBillDTO toRepairBillDTO(RepairBill repairBill){
46         RepairBillDTO repairBillDTO=new RepairBillDTO();
47         repairBillDTO.setBillId(repairBill.getId());
48         repairBillDTO.setDeviceName(repairBill.getDevice().getDeviceName());
49         repairBillDTO.setRepairedBy(repairBill.getRepairLog().getRepairedBy());
50         repairBillDTO.setRepairedDate(repairBill.getRepairLog().getRepairDate());
51         repairBillDTO.setRepairCost(repairBill.getRepairCost());
52         repairBillDTO.setGeneratedOn(repairBill.getGeneratedDate());
53         repairBillDTO.setStatus(repairBill.getPaymentStatus().name());
54         return repairBillDTO;
55     }
56
57     public List<RentalBillDTO> getRentalBill(){
58         List<RentalBill> rentalBills=rentalBillRepository.findByPaymentStatusNotInAndAmountGreaterThan(
59             List.of(PaymentStatus.PAID,PaymentStatus.ACKNOWLEDGED),
60             BigDecimal.ZERO
61         );
62         return rentalBills.stream().map(this::toRentalBillDTO).toList();
63     }
64
65     public RentalBillDTO toRentalBillDTO(RentalBill rentalBill){
66         RentalBillDTO rentalBillDTO = new RentalBillDTO();
67         rentalBillDTO.setId(rentalBill.getId());
68         rentalBillDTO.setAmount(rentalBill.getAmount());
69         rentalBillDTO.setBillDate(rentalBill.getBillDate());
70         rentalBillDTO.setDueDate(rentalBill.getDueDate());

```



```

71         rentalBillDTO.setVendorName(rentalBill.getVendor().getName());
72         rentalBillDTO.setPaymentStatus(rentalBill.getPaymentStatus().name());
73         return rentalBillDTO;
74     }
75
76     public InvoiceDTO generateRepairInvoice(Long id) {
77         RepairBill repairBill=repairBillRepository.findById(id).orElseThrow(
78             ()->new RepairBillNotFoundException("Bill with Id: "+id+" Not Found")
79         );
80
81         if (invoiceRepository.existsByRepairBill(repairBill)) {
82             throw new ResponseStatusException(
83                 HttpStatus.CONFLICT,
84                 "Invoice already generated for this rental bill"
85             );
86         }
87         repairBill.setPaymentStatus(PaymentStatus.PAID);
88         Invoice invoice=new Invoice();
89         invoice.setInvoiceType(InvoiceType.REPAIR);
90         invoice.setRepairBill(repairBill);
91         invoice.setAmount(repairBill.getRepairCost());
92         invoice.setAmount(repairBill.getRepairCost());
93         invoice.setStatus(InvoiceStatus.SENT);
94         invoice.setIssuedDate(LocalDate.now());
95         invoice.setVendor(repairBill.getDevice().getTechVendor());
96         Invoice saved=invoiceRepository.save(invoice);
97         return toInvoiceDTO(saved);
98     }
99
100    public InvoiceDTO generateRentalInvoice(Long id) {
101        RentalBill rentalBill=rentalBillRepository.findById(id).orElseThrow(
102            ()->new RentalBillNotFoundException("Bill with Id: "+id+" Not Found")
103        );
104        if (invoiceRepository.existsByRentalBill(rentalBill)) {
105            throw new ResponseStatusException(
106                HttpStatus.CONFLICT,
107                "Invoice already generated for this rental bill"
108            );
109        }
110        rentalBill.setPaymentStatus(PaymentStatus.PAID);
111        Invoice invoice=new Invoice();
112        invoice.setInvoiceType(InvoiceType.RENTAL);
113        invoice.setRentalBill(rentalBill);
114        invoice.setAmount(rentalBill.getAmount());
115        invoice.setStatus(InvoiceStatus.SENT);
116        invoice.setIssuedDate(LocalDate.now());
117        invoice.setVendor(rentalBill.getVendor());
118        Invoice saved=invoiceRepository.save(invoice);
119        return toInvoiceDTO(saved);
120    }
121
122    public InvoiceDTO toInvoiceDTO(Invoice invoice) {
123        InvoiceDTO invoiceDTO = new InvoiceDTO();
124        invoiceDTO.setId(invoice.getId());
125        invoiceDTO.setInvoiceType(invoice.getInvoiceType().name());
126        invoiceDTO.setVendorName(invoice.getVendor().getName());
127        invoiceDTO.setAmount(invoice.getAmount());
128        invoiceDTO.setIssuedDate(invoice.getIssuedDate());
129        invoiceDTO.setStatus(invoice.getStatus().name());
130        invoiceDTO.setSeenAt(invoice.getSeenAt());
131        return invoiceDTO;
132    }
133 }
134

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
9 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
10 import org.slf4j.Logger;
11 import org.slf4j.LoggerFactory;
12 import org.springframework.stereotype.Service;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.math.BigDecimal;
16 import java.time.LocalDate;
17 import java.util.List;
18
19 @Service
20 public class CarryForwardWarningService {
21
22     private static final Logger log = LoggerFactory.getLogger(CarryForwardWarningService.class);
23
24     private static final BigDecimal CAP = BigDecimal.valueOf(30);
25     private static final BigDecimal WARN_AT_29 = BigDecimal.valueOf(29);
26     private static final BigDecimal WARN_AT_28 = BigDecimal.valueOf(28);
27
28     private final EmployeeRepo employeeRepository;
29     private final LeaveEntitlementRepository entitlementRepository;
30     private final LeaveWarningRepository warningRepository;
31
32     public CarryForwardWarningService(EmployeeRepo employeeRepository,
33                                     LeaveEntitlementRepository entitlementRepository,
34                                     LeaveWarningRepository warningRepository) {
35         this.employeeRepository = employeeRepository;
36         this.entitlementRepository = entitlementRepository;
37         this.warningRepository = warningRepository;
38     }
39
40     @Transactional
41     public void runWarningCheck(LocalDate checkDate) {
42         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
43
44         for (Employee employee : activeEmployees) {
45             BigDecimal carryForwardTotal = calculateCarryForwardBalance(employee, checkDate);
46             evaluateAndWarn(employee, carryForwardTotal, checkDate);
47         }
48
49         log.info("Warning check complete for {} – {} employees evaluated",
50                checkDate, activeEmployees.size());
51     }
52
53     // -----
54     // Sum available balance across all carry-forwardable types
55     // for the current year (this is what will carry forward if unused)
56     // -----
57     private BigDecimal calculateCarryForwardBalance(Employee employee, LocalDate date) {
58         int year = date.getYear();
59
60         List<LeaveEntitlement> entitlements =
61             entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(), year);
62
63         return entitlements.stream()
64             .filter(e -> e.getLeaveType().isCarriesForward())
65             .map(LeaveEntitlement::getAvailableBalance)
66             .reduce(BigDecimal.ZERO, BigDecimal::add);
67     }
68
69     // -----
70     // Decide which warning to issue based on carry-forward total
71     // -----

```

```

72     private void evaluateAndWarn(Employee employee, BigDecimal balance, LocalDate date) {
73         int cmp30 = balance.compareTo(CAP);
74         int cmp29 = balance.compareTo(WARN_AT_29);
75         int cmp28 = balance.compareTo(WARN_AT_28);
76
77         if (cmp30 > 0) {
78             // Balance EXCEEDS cap – a day was wasted (cap enforcement happens in Phase 5,
79             // but we flag it here so the warning appears immediately after accrual)
80             issueWarning(employee, LeaveWarning.WarningType.DAY_WASTED, balance, date,
81                 "A carry-forward day was wasted this month because your balance exceeded the 30-day
82 cap.");
83         } else if (cmp30 == 0) {
84             issueWarning(employee, LeaveWarning.WarningType.CAP_REACHED, balance, date,
85                 "Carry-forward cap reached (30 days). Next month's unused leave will be wasted.");
86         } else if (cmp29 >= 0) {
87             issueWarning(employee, LeaveWarning.WarningType.APPROACHING_29, balance, date,
88                 "You have " + balance.toPlainString() + " carry-forward days. "
89                 + "1 month until cap. Use leave before it stops accumulating.");
90         } else if (cmp28 >= 0) {
91             issueWarning(employee, LeaveWarning.WarningType.APPROACHING_28, balance, date,
92                 "You have " + balance.toPlainString() + " carry-forward days. "
93                 + "2 months until cap (30). Plan to use leave soon.");
94         }
95         // Below 28 – no warning needed
96     }
97 }
98
99 // -----
100 // Persist warning – skip if same type already issued today
101 // -----
102 private void issueWarning(Employee employee, LeaveWarning.WarningType type,
103     BigDecimal balance, LocalDate date, String message) {
104     boolean alreadyIssued = warningRepository
105         .existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(employee.getEmployeeId(), type,
106 date);
107     if (alreadyIssued) {
108         log.debug("Warning {} already issued for employee {} on {}", type, employee.getEmployeeId(),
109 date);
110         return;
111     }
112
113     LeaveWarning warning = new LeaveWarning();
114     warning.setEmployee(employee);
115     warning.setWarningType(type);
116     warning.setMessage(message);
117     warning.setCarryForwardBalance(balance);
118     warning.setWarningDate(date);
119
120     warningRepository.save(warning);
121
122     log.info("Warning issued – employee={} type={} balance={} date={}",
123         employee.getEmployeeId(), type, balance, date);
124 }
125 }
126 }

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import org.springframework.security.core.userdetails.UserDetails;
8 import org.springframework.security.core.userdetails.UserDetailsService;
9 import org.springframework.security.core.userdetails.UsernameNotFoundException;
10 import org.springframework.stereotype.Service;
11
12 import java.util.Optional;
13
14 @Service("combinedUserDetailsService")
15 public class CombinedUserDetailsService implements UserDetailsService {
16
17     private final VendorRepo vendorRepository;
18     private final EmployeeRepo employeeRepository;
19
20     public CombinedUserDetailsService(VendorRepo vendorRepository, EmployeeRepo employeeRepository) {
21         this.vendorRepository = vendorRepository;
22         this.employeeRepository = employeeRepository;
23     }
24
25     @Override
26     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
27         Optional<Employee> employee = employeeRepository.findByEmail(username);
28         if (employee.isPresent()) {
29             return employee.get(); // must implement UserDetails
30         }
31
32         // Then try Vendor
33         Optional<Vendor> vendor = vendorRepository.findByEmail(username);
34         if (vendor.isPresent()) {
35             return vendor.get();
36         }
37
38         throw new UsernameNotFoundException("User not found: " + username);
39     }
40 }
41
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository VendorRepo;
7 import org.springframework.security.core.GrantedAuthority;
8 import org.springframework.security.core.authority.SimpleGrantedAuthority;
9 import org.springframework.security.crypto.password.PasswordEncoder;
10 import org.springframework.security.oauth2.client.userinfo.DefaultOAuth2UserService;
11 import org.springframework.security.oauth2.client.userinfo.OAuth2UserRequest;
12 import org.springframework.security.oauth2.core.user.DefaultOAuth2User;
13 import org.springframework.security.oauth2.core.user.OAuth2User;
14 import org.springframework.stereotype.Service;
15
16 import java.util.ArrayList;
17 import java.util.List;
18 import java.util.UUID;
19
20 @Service
21 public class CustomOAuth2UserService extends DefaultOAuth2UserService {
22
23     private final EmployeeRepo employeeRepo;
24     private final VendorRepo vendorRepo;
25     private final PasswordEncoder passwordEncoder;
26
27     public CustomOAuth2UserService(EmployeeRepo employeeRepo,
28                                   VendorRepo vendorRepo,
29                                   PasswordEncoder passwordEncoder) {
30         this.employeeRepo = employeeRepo;
31         this.vendorRepo = vendorRepo;
32         this.passwordEncoder = passwordEncoder;
33     }
34
35     @Override
36     public OAuth2User loadUser(OAuth2UserRequest userRequest) {
37         System.out.println("=== CustomOAuth2UserService.loadUser() called ===");
38         OAuth2User oAuth2User = super.loadUser(userRequest);
39
40         String email = oAuth2User.getAttribute("email");
41         String name = oAuth2User.getAttribute("name");
42         System.out.println("OAuth email: " + email);
43
44         boolean employeeExists = employeeRepo.findByEmail(email).isPresent();
45         boolean vendorExists = vendorRepo.findByEmail(email).isPresent();
46         System.out.println("employeeExists=" + employeeExists + ", vendorExists=" + vendorExists);
47
48         List<GrantedAuthority> authorities = new ArrayList<>(oAuth2User.getAuthorities());
49
50         if (employeeExists) {
51             Employee employee = employeeRepo.findByEmail(email).get();
52             authorities.add(new SimpleGrantedAuthority("ROLE_" + employee.getRole().name()));
53
54             // ADD THIS – loads LEAVE_CANCEL, SUBSCRIPTION_READ, etc. into the session
55             employee.getRole().getPermissions().forEach(permission ->
56                 authorities.add(new SimpleGrantedAuthority(permission.name()))
57             );
58
59             System.out.println("Returning existing employee with role: " + employee.getRole());
60             return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
61         }
62
63         if (vendorExists) {
64             // add vendor role if needed
65             authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
66             System.out.println("Returning existing vendor");
67             return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
68         }
69
70         // New user
71         registerAsUser(email, name);

```

```
72     authorities.add(new SimpleGrantedAuthority("ROLE_USER"));
73     return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
74 }
75
76 private void registerAsUser(String email, String name) {
77     Employee e = new Employee();
78     e.setEmail(email);
79     e.setName(name);
80     e.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));
81     e.setRole(Role.USER);
82     e.setTimezone("Asia/Kolkata");
83     e.setDept("UNASSIGNED");
84     employeeRepo.saveAndFlush(e); // flush immediately so OAuth2SuccessHandler sees the record
85     System.out.println("New USER registered and flushed for: " + email);
86 }
87 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.VendorDTO;
4 import com.example.EmployeeManagementSystem.DTO.VendorRequest;
5 import com.example.EmployeeManagementSystem.Entity.Delivery;
6 import com.example.EmployeeManagementSystem.Entity.Subscription;
7 import com.example.EmployeeManagementSystem.Entity.Vendor;
8 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
9 import com.example.EmployeeManagementSystem.Enum.Role;
10 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
11 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
12 import com.example.EmployeeManagementSystem.Repository.DeliveryRepository;
13 import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
14 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
15 import com.example.EmployeeManagementSystem.Util.AuthUtil;
16 import org.slf4j.Logger;
17 import org.slf4j.LoggerFactory;
18 import org.springframework.dao.DataIntegrityViolationException;
19 import org.springframework.security.core.Authentication;
20 import org.springframework.security.crypto.password.PasswordEncoder;
21 import org.springframework.stereotype.Service;
22 import org.springframework.transaction.annotation.Transactional;
23
24 import java.time.Instant;
25 import java.util.List;
26 import java.util.stream.Collectors;
27
28 @Service
29 public class DeliveryService {
30     private final SubscriptionRepository subscriptionRepository;
31     private final DeliveryRepository deliveryRepository;
32     private final DeliveryTimeService deliveryTimeService;
33     private static final Logger logger = LoggerFactory.getLogger(DeliveryService.class);
34
35     public DeliveryService(SubscriptionRepository subscriptionRepository,
36                           DeliveryRepository deliveryRepository,
37                           DeliveryTimeService deliveryTimeService) {
38         this.subscriptionRepository = subscriptionRepository;
39         this.deliveryRepository = deliveryRepository;
40         this.deliveryTimeService = deliveryTimeService;
41     }
42
43     // AFTER
44     @Transactional
45     public void processDueDeliveries() {
46         Instant now = Instant.now();
47         List<Subscription> dueSubscriptions = subscriptionRepository
48             .findDueSubscriptions(SubscriptionStatus.ACTIVE.name(), now);
49
50         for (Subscription subscription : dueSubscriptions) {
51             try {
52                 Delivery delivery = new Delivery();
53                 delivery.setSubscription(subscription);
54                 delivery.setMealSlot(subscription.getSlot());
55                 delivery.setScheduledDeliveryTime(subscription.getNextDeliveryTime());
56                 delivery.setStatus(DeliveryStatus.IN_PROGRESS);
57                 deliveryRepository.save(delivery);
58
59                 Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
60                 subscription.setNextDeliveryTime(nextDelivery);
61                 subscriptionRepository.save(subscription);
62
63             } catch (DataIntegrityViolationException e) {
64                 // Duplicate delivery already exists – just advance the next delivery time
65                 logger.warn("Duplicate delivery skipped for subscription ID: {}", subscription.getId());
66                 Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
67                 subscription.setNextDeliveryTime(nextDelivery);
68                 subscriptionRepository.save(subscription);
69             }
70         }
71     }

```

```

72
73     @Transactional
74     public Delivery updateDeliveryStatus(Long deliveryId, DeliveryStatus status) {
75         Delivery delivery = deliveryRepository.findById(deliveryId)
76             .orElseThrow(() -> new RuntimeException("Delivery not found"));
77
78         delivery.setStatus(status);
79         if (status == DeliveryStatus.DELIVERED) {
80             delivery.setActualDeliveryTime(Instant.now());
81         }
82
83         return deliveryRepository.save(delivery);
84     }
85
86     public List<Delivery> getDeliveriesBySubscription(Long subscriptionId) {
87         Subscription subscription = subscriptionRepository.findById(subscriptionId)
88             .orElseThrow(() -> new RuntimeException("Subscription not found"));
89         return deliveryRepository.findRecentDeliveriesBySubscription(subscription,
DeliveryStatus.SCHEDULED);
90     }
91
92     @Service
93     public static class VendorService {
94
95         private static final Logger log = LoggerFactory.getLogger(VendorService.class);
96         private final VendorRepo vendorRepo;
97         private final PasswordEncoder passwordEncoder;
98
99         public VendorService(VendorRepo vendorRepo, PasswordEncoder passwordEncoder) {
100             this.vendorRepo = vendorRepo;
101             this.passwordEncoder = passwordEncoder;
102         }
103
104         public VendorDTO registerVendor(VendorRequest request) {
105             if (vendorRepo.existsByEmail(request.getEmail())) {
106                 throw new IllegalArgumentException(
107                     "A vendor with email " + request.getEmail() + " already exists");
108             }
109             Vendor vendor = new Vendor();
110             vendor.setName(request.getName());
111             vendor.setEmail(request.getEmail());
112             vendor.setPhone(request.getPhone());
113             vendor.setRole(request.getRole());
114             vendor.setPassword(passwordEncoder.encode(request.getPassword()));
115             Vendor saved = vendorRepo.save(vendor);
116             log.info("Vendor registered: {} (id={})", saved.getName(), saved.getId());
117             return toDTO(saved);
118         }
119
120         public VendorDTO getVendor(Authentication authentication) {
121             String email= AuthUtil.extractEmail(authentication);
122             return toDTO(vendorRepo.findById(email).orElseThrow(
123                 ()->new VendorNotFoundException("Cannot get others account")
124             ));
125         }
126
127         public List<VendorDTO> getAllVendors() {
128             return vendorRepo.findAll().stream()
129                 .map(this::toDTO)
130                 .collect(Collectors.toList());
131         }
132
133         public VendorDTO updateVendor(VendorRequest request, Authentication authentication) {
134             String email= AuthUtil.extractEmail(authentication);
135             Vendor vendor = vendorRepo.findById(email).orElseThrow(
136                 ()->new VendorNotFoundException("Cannot update others account")
137             );
138             if (request.getName() != null) vendor.setName(request.getName());
139             if (request.getPhone() != null) vendor.setPhone(request.getPhone());
140             if (request.getPassword() != null)
vendor.setPassword(passwordEncoder.encode(request.getPassword()));
141             // Email update not allowed to avoid breaking restaurant ownership references
142             return toDTO(vendorRepo.save(vendor));
143         }
144
145         public void deleteVendor(Authentication authentication) {

```



```
146         String email= AuthUtil.extractEmail(authentication);
147         Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
148             ()->new VendorNotFoundException("Cannot delete others account")
149         );
150         vendorRepo.delete(vendor);
151         log.info("Vendor deleted: email={}", email);
152     }
153
154     public void deleteVendorById(Long vendorId) {
155         Vendor vendor = findOrThrow(vendorId);
156         vendorRepo.delete(vendor);
157         log.info("Vendor deleted by admin: id={}, email={}", vendor.getId(), vendor.getEmail());
158     }
159
160     // helpers
161
162     private Vendor findOrThrow(Long id) {
163         return vendorRepo.findById(id)
164             .orElseThrow(() -> new VendorNotFoundException("Vendor with id " + id + " not found"));
165     }
166
167     private VendorDTO toDTO(Vendor vendor) {
168         VendorDTO dto = new VendorDTO();
169         dto.setId(vendor.getId());
170         dto.setName(vendor.getName());
171         dto.setEmail(vendor.getEmail());
172         dto.setPhone(vendor.getPhone());
173         dto.setRole(vendor.getRole());
174         dto.setRegisteredAt(vendor.getRegisteredAt());
175         return dto;
176     }
177 }
178 }
179
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.Subscription;
6 import com.example.EmployeeManagementSystem.Enum.MealSlot;
7 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
8 import org.springframework.stereotype.Service;
9
10 import java.time.*;
11
12 @Service
13 public class DeliveryTimeService {
14
15     private final MealSlotService mealSlotService;
16
17     public DeliveryTimeService(MealSlotService mealSlotService) {
18         this.mealSlotService = mealSlotService;
19     }
20     public Instant calculateNextDelivery(SubscriptionsRequest request, MealSlot slot, Employee employee) {
21         ZoneId userZone=ZoneId.of(employee.getTimezone());
22         if (request.getScheduleType() == ScheduleType.DAILY) {
23             return calculateDailyNextDelivery(slot,userZone);
24         } else {
25             return calculateWeeklyNextDelivery(slot, request.getDayOfWeek(),userZone);
26         }
27     }
28
29     private Instant calculateDailyNextDelivery(MealSlot slot, ZoneId userZone) {
30         LocalDateTime slotTime = mealSlotService.getTime(slot);
31         ZonedDateTime now = ZonedDateTime.now(userZone);
32         ZonedDateTime deliveryTime;
33
34         if (now.toLocalTime().isBefore(slotTime)) {
35             deliveryTime=ZonedDateTime.of(now.toLocalDate(),slotTime,userZone);
36         } else {
37             deliveryTime=ZonedDateTime.of(now.toLocalDate().plusDays(1), slotTime,userZone);
38         }
39         return deliveryTime.toInstant();
40     }
41
42     private Instant calculateWeeklyNextDelivery(MealSlot slot, DayOfWeek dayOfWeek,ZoneId userZone) {
43         LocalDateTime slotTime = mealSlotService.getTime(slot);
44         ZonedDateTime now=ZonedDateTime.now(userZone);
45         DayOfWeek today = now.getDayOfWeek();
46
47         int daysToAdd = dayOfWeek.getValue() - today.getValue();
48
49         if (daysToAdd == 0) {
50             if (now.toLocalTime().isBefore(slotTime)) {
51                 return ZonedDateTime.of(now.toLocalDate(), slotTime,userZone).toInstant();
52             } else {
53                 return ZonedDateTime.of(now.toLocalDate().plusDays(7), slotTime,userZone).toInstant();
54             }
55         }
56         if (daysToAdd < 0) {
57             daysToAdd += 7;
58         }
59
60         ZonedDateTime deliveryTime = ZonedDateTime.of(
61             now.toLocalDate().plusDays(daysToAdd),slotTime,userZone);
62         return deliveryTime.toInstant();
63     }
64
65     public Instant getNextDeliveryTime(Subscriptions subscription) {
66         ZoneId userZone=ZoneId.of(subscription.getEmployee().getTimezone());
67         if (subscription.getScheduleType() == ScheduleType.DAILY) {
68             return calculateDailyNextDelivery(subscription.getSlot(),userZone);
69         } else {
70             return calculateWeeklyNextDelivery(subscription.getSlot(),
71 subscription.getDayOfWeek(),userZone);

```

```
71     }
```

```
72 }
```

```
73 }
```

```
74
```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
4  import com.example.EmployeeManagementSystem.DTO.DeviceActivityDTO;
5  import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
6  import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
7  import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
8  import com.example.EmployeeManagementSystem.DTO.RepairLogResponseDTO;
9  import com.example.EmployeeManagementSystem.Entity.Device;
10 import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
11 import com.example.EmployeeManagementSystem.Entity.Employee;
12 import com.example.EmployeeManagementSystem.Entity.RepairLog;
13 import com.example.EmployeeManagementSystem.Entity.Vendor;
14 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
15 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
16 import com.example.EmployeeManagementSystem.Enum.Role;
17 import com.example.EmployeeManagementSystem.Enum.Status;
18 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
19 import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
20 import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
21 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
22 import com.example.EmployeeManagementSystem.Repository.RepairBillRepository;
23 import com.example.EmployeeManagementSystem.Repository.RepairLogRepository;
24 import com.example.EmployeeManagementSystem.Repository.ServiceRequestRepository;
25 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
26 import com.example.EmployeeManagementSystem.Repository.VendorNegotiationRepository;
27 import com.example.EmployeeManagementSystem.Util.AuthUtil;
28 import org.springframework.http.ResponseEntity;
29 import org.springframework.security.access.AccessDeniedException;
30 import org.springframework.security.core.Authentication;
31 import org.springframework.stereotype.Service;
32 import org.springframework.transaction.annotation.Transactional;
33
34 import java.time.LocalDate;
35 import java.time.LocalDateTime;
36 import java.time.LocalTime;
37 import java.util.ArrayList;
38 import java.util.Comparator;
39 import java.util.List;
40 import java.util.Set;
41
42 @Service
43 public class DeviceService {
44     private final DeviceRepository deviceRepository;
45     private final VendorRepo vendorRepo;
46     private final DeviceAssignmentRepo deviceAssignmentRepo;
47     private final EmployeeRepo employeeRepo;
48     private final RepairLogRepository repairLogRepository;
49     private final ServiceRequestRepository serviceRequestRepository;
50     private final RepairBillRepository repairBillRepository;
51     private final VendorNegotiationRepository vendorNegotiationRepository;
52
53     public DeviceService(DeviceRepository deviceRepository,
54                          VendorRepo vendorRepo,
55                          DeviceAssignmentRepo deviceAssignmentRepo,
56                          EmployeeRepo employeeRepo,
57                          RepairLogRepository repairLogRepository,
58                          ServiceRequestRepository serviceRequestRepository,
59                          RepairBillRepository repairBillRepository,
60                          VendorNegotiationRepository vendorNegotiationRepository) {
61         this.deviceRepository = deviceRepository;
62         this.vendorRepo = vendorRepo;
63         this.deviceAssignmentRepo = deviceAssignmentRepo;
64         this.employeeRepo = employeeRepo;
65         this.repairLogRepository = repairLogRepository;
66         this.serviceRequestRepository = serviceRequestRepository;
67         this.repairBillRepository = repairBillRepository;
68         this.vendorNegotiationRepository = vendorNegotiationRepository;
69     }
70
71     public Device addDevice(DeviceDTO deviceDTO, Authentication authentication) {

```

```

72     String email = AuthUtil.extractEmail(authentication);
73     Vendor vendor = vendorRepo.findByEmail(email)
74         .orElseThrow(() -> new RuntimeException("Vendor not found"));
75
76     Device device = new Device();
77     device.setDeviceName(deviceDTO.getDeviceName());
78     device.setWarrantyExpiryDate(deviceDTO.getWarrantyExpiryDate());
79     device.setRentPerMonth(deviceDTO.getRentPerMonth());
80     device.setBrand(deviceDTO.getBrand());
81     device.setDeviceStatus(DeviceStatus.AVAILABLE);
82     device.setDeviceType(deviceDTO.getDeviceType());
83     device.setTechVendor(vendor);
84     device.setCreatedAt(LocalDateTime.now());
85     return deviceRepository.save(device);
86 }
87
88 public List<DeviceResponseDTO> getDevicesForLoggedInVendor(Authentication authentication) {
89     String email = AuthUtil.extractEmail(authentication);
90     Vendor vendor = vendorRepo.findByEmail(email)
91         .orElseThrow(() -> new RuntimeException("Vendor not found"));
92
93     return deviceRepository.findByTechVendorId(vendor.getId()).stream()
94         .map(this::toDeviceResponse)
95         .toList();
96 }
97
98 public List<DeviceResponseDTO> getDevicesForLoggedInEmployee(Authentication authentication) {
99     String email = AuthUtil.extractEmail(authentication);
100     Employee employee = employeeRepo.findByEmail(email)
101         .orElseThrow(() -> new RuntimeException("Employee not found"));
102
103     return deviceAssignmentRepo.findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(),
AssignmentStatus.ACTIVE)
104         .stream()
105         .map(DeviceAssignment::getDevice)
106         .filter(device -> device.getDeviceStatus() != DeviceStatus.CONDEMNED)
107         .map(this::toDeviceResponse)
108         .toList();
109 }
110 public List<DeviceResponseDTO> getDevicesForEmployee(Authentication authentication) {
111     String email = AuthUtil.extractEmail(authentication);
112     Employee employee = employeeRepo.findByEmail(email)
113         .orElseThrow(() -> new RuntimeException("Employee not found"));
114
115     return deviceAssignmentRepo.findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(),
AssignmentStatus.ACTIVE)
116         .stream()
117         .map(DeviceAssignment::getDevice)
118         .map(this::toDeviceResponse)
119         .toList();
120 }
121
122 public List<DeviceResponseDTO> getUnassignedDevices() {
123     return deviceRepository.findByDeviceStatus(DeviceStatus.AVAILABLE).stream()
124         .map(this::toDeviceResponse)
125         .toList();
126 }
127
128 public List<DeviceResponseDTO> getAllDevices() {
129     return deviceRepository.findByStatusNot(DeviceStatus.RETURNED_TO_VENDOR.name()).stream()
130         .map(this::toDeviceResponse)
131         .toList();
132 }
133
134 public DeviceResponseDTO getDeviceDetails(Long deviceId, Authentication authentication) {
135     Device device = getAuthorizedDevice(deviceId, authentication);
136     return toDeviceResponse(device);
137 }
138
139 public List<DeviceActivityDTO> getDeviceActivity(Long deviceId, Authentication authentication) {
140     Device device = getAuthorizedDevice(deviceId, authentication);
141     List<DeviceActivityDTO> activities = new ArrayList<>();
142
143     if (device.getCreatedAt() != null) {
144         activities.add(activity(device.getCreatedAt(), "DEVICE_CREATED", "Device registered",
145             device.getBrand() + " " + device.getDeviceName() + " was added to inventory",

```

```

146         device.getTechVendor() != null ? device.getTechVendor().getName() : null,
147         device.getDeviceStatus() != null ? device.getDeviceStatus().name() : null));
148     }
149
150     deviceAssignmentRepo.findByDeviceIdOrderByAssignedDateDescIdDesc(deviceId).forEach(assignment -> {
151         LocalDateTime occurredAt = assignment.getAssignedDate() != null
152             ? assignment.getAssignedDate().atStartOfDay()
153             : null;
154         Employee assignedTo = assignment.getAssignedTo();
155         activities.add(activity(occurredAt, "ASSIGNMENT", "Assigned to employee",
156             assignedTo != null ? "Assigned to " + assignedTo.getName() : "Device assignment
updated",
157             assignedTo != null ? assignedTo.getName() : null,
158             assignment.getStatus() != null ? assignment.getStatus().name() : null));
159     });
160
161     serviceRequestRepository.findByDeviceOrderByRaisedAtDesc(device).forEach(request -> {
162         activities.add(activity(request.getRaisedAt(), "SERVICE_REQUEST", "Service request raised",
163             request.getIssueDescription(),
164             request.getRaisedBy() != null ? request.getRaisedBy().getName() : null,
165             request.getStatus() != null ? request.getStatus().name() : null));
166
167         if (request.getAdminRemarks() != null && !request.getAdminRemarks().isBlank()) {
168             activities.add(activity(request.getRaisedAt(), "ADMIN_REVIEW", "Admin reviewed request",
169                 request.getAdminRemarks(),
170                 request.getReviewedBy() != null ? request.getReviewedBy().getName() : null,
171                 request.getStatus() != null ? request.getStatus().name() : null));
172         }
173
174         if (request.getResolvedAt() != null) {
175             activities.add(activity(request.getResolvedAt(), "RESOLUTION", "Request resolved",
176                 request.getResolution(),
177                 request.getDevice() != null && request.getDevice().getTechVendor() != null
178                     ? request.getDevice().getTechVendor().getName()
179                     : null,
180                 request.getStatus() != null ? request.getStatus().name() : null));
181         }
182     });
183
184     repairLogRepository.findByDeviceIdOrderByRepairDateDescIdDesc(deviceId).forEach(repairLog -> {
185         LocalDateTime occurredAt = repairLog.getRepairDate() != null
186             ? repairLog.getRepairDate().atTime(LocalTime.NOON)
187             : null;
188         activities.add(activity(occurredAt, "REPAIR_LOG", "Repair log updated",
189             repairLog.getRepairAction() != null ? repairLog.getRepairAction() :
repairLog.getRemarks(),
190             repairLog.getRepairedBy(),
191             repairLog.getServiceRequest() != null && repairLog.getServiceRequest().getStatus() !=
null
192                 ? repairLog.getServiceRequest().getStatus().name()
193                 : null));
194     });
195
196     return activities.stream()
197         .sorted(Comparator.comparing(DeviceActivityDTO::getOccurredAt,
198             Comparator.nullsLast(Comparator.reverseOrder())))
199         .toList();
200 }
201
202 @Transactional
203 public DeviceAssignment assignDevice(Long deviceId, AssignDeviceRequest request) {
204     if (request.getEmployeeId() == null) {
205         throw new IllegalArgumentException("employeeId is required");
206     }
207
208     Device device = deviceRepository.findById(deviceId)
209         .orElseThrow(() -> new RuntimeException("Device not found"));
210
211     Employee employee = employeeRepo.findById(request.getEmployeeId())
212         .orElseThrow(() -> new RuntimeException("Employee not found"));
213
214     if (employee.getStatus() != Status.ACTIVE) {
215         throw new IllegalArgumentException("Device can only be assigned to an active user");
216     }
217
218     if (!Set.of(Role.EMPLOYEE, Role.MANAGER, Role.ADMIN).contains(employee.getRole())) {

```

```

219         throw new IllegalArgumentException("Devices can only be assigned to employees, managers, or
admins");
220     }
221
222     if (device.getDeviceStatus() != DeviceStatus.AVAILABLE) {
223         throw new IllegalArgumentException("Only available devices can be assigned");
224     }
225
226     deviceAssignmentRepo.findByDeviceIdAndStatus(deviceId, AssignmentStatus.ACTIVE)
227         .ifPresent(existingAssignment -> {
228             throw new IllegalArgumentException("Device is already assigned");
229         });
230
231     DeviceAssignment assignment = new DeviceAssignment();
232     assignment.setDevice(device);
233     assignment.setAssignedTo(employee);
234     assignment.setStatus(AssignmentStatus.ACTIVE);
235     assignment.setAssignedDate(LocalDate.now());
236
237     DeviceAssignment savedAssignment = deviceAssignmentRepo.save(assignment);
238     device.setCurrentAssignment(savedAssignment);
239     device.setDeviceStatus(DeviceStatus.ASSIGNED);
240     deviceRepository.save(device);
241
242     return savedAssignment;
243 }
244
245
246
247 private DeviceResponseDTO toDeviceResponse(Device device) {
248     DeviceResponseDTO response = new DeviceResponseDTO();
249     response.setId(device.getId());
250     response.setDeviceName(device.getDeviceName());
251     response.setBrand(device.getBrand());
252     response.setWarrantyExpiryDate(device.getWarrantyExpiryDate());
253     response.setRentPerMonth(device.getRentPerMonth());
254     response.setDeviceType(device.getDeviceType());
255     response.setDeviceStatus(device.getDeviceStatus());
256     response.setCreatedAt(device.getCreatedAt());
257
258     if (device.getTechVendor() != null) {
259         response.setVendorName(device.getTechVendor().getName());
260     }
261
262     DeviceAssignment currentAssignment = device.getCurrentAssignment();
263     if (currentAssignment != null) {
264         response.setAssignedDate(currentAssignment.getAssignedDate());
265
266         Employee assignedEmployee = currentAssignment.getAssignedTo();
267         if (assignedEmployee != null) {
268             response.setAssignedEmployeeId(assignedEmployee.getEmployeeId());
269             response.setAssignedEmployeeName(assignedEmployee.getName());
270         }
271     }
272
273     return response;
274 }
275
276 private Device getAuthorizedDevice(Long deviceId, Authentication authentication) {
277     Device device = deviceRepository.findById(deviceId)
278         .orElseThrow(() -> new RuntimeException("Device not found"));
279
280     if (hasAuthority(authentication, "ROLE_ADMIN")) {
281         return device;
282     }
283
284     String email = AuthUtil.extractEmail(authentication);
285     if (hasAuthority(authentication, "ROLE_TECH_VENDOR")) {
286         if (device.getTechVendor() != null && device.getTechVendor().getEmail().equals(email)) {
287             return device;
288         }
289         throw new AccessDeniedException("Unauthorized access to this device");
290     }
291
292     DeviceAssignment currentAssignment = device.getCurrentAssignment();
293     if (currentAssignment != null

```

```

294         && currentAssignment.getAssignedTo() != null
295         && currentAssignment.getAssignedTo().getEmail().equals(email)) {
296             return device;
297     }
298
299     throw new AccessDeniedException("Unauthorized access to this device");
300 }
301
302 private boolean hasAuthority(Authentication authentication, String authority) {
303     return authentication != null && authentication.getAuthorities().stream()
304         .anyMatch(grantedAuthority -> authority.equals(grantedAuthority.getAuthority()));
305 }
306
307 private DeviceActivityDTO activity(LocalDateTime occurredAt,
308                                     String type,
309                                     String title,
310                                     String description,
311                                     String actorName,
312                                     String status) {
313     DeviceActivityDTO dto = new DeviceActivityDTO();
314     dto.setOccurredAt(occurredAt);
315     dto.setType(type);
316     dto.setTitle(title);
317     dto.setDescription(description);
318     dto.setActorName(actorName);
319     dto.setStatus(status);
320     return dto;
321 }
322
323 public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfVendor(Authentication authentication) {
324     String email=AuthUtil.extractEmail(authentication);
325     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
326         ()->new VendorNotFoundException("Vendor:"+email+" not found")
327     );
328     List<DeviceAssignment>
deviceAssignments=deviceAssignmentRepo.getAllAssignmentsByVendorId(vendor.getId());
329     return deviceAssignments.stream().map(deviceAssignment -> toAssignmentDTO(deviceAssignment)).toList();
330 }
331
332 private DeviceAssignmentResponseDTO toAssignmentDTO(DeviceAssignment a) {
333     DeviceAssignmentResponseDTO dto = new DeviceAssignmentResponseDTO();
334     dto.setId(a.getId());
335     dto.setStatus(a.getStatus());
336     dto.setAssignedDate(a.getAssignedDate());
337
338     if (a.getDevice() != null) {
339         dto.setDeviceId(a.getDevice().getId());
340         dto.setDeviceName(a.getDevice().getDeviceName());
341         dto.setDeviceBrand(a.getDevice().getBrand());
342         dto.setDeviceType(a.getDevice().getDeviceType());
343     }
344
345     if (a.getAssignedTo() != null) {
346         dto.setEmployeeId(a.getAssignedTo().getEmployeeId());
347         dto.setEmployeeName(a.getAssignedTo().getName()); // adjust to your Employee field
348     }
349
350     return dto;
351 }
352
353 public List<DeviceResponseDTO> getRepairedDevices() {
354     return deviceRepository.findByDeviceStatus(DeviceStatus.REPAIR_DONE).stream()
355         .map(this::toDeviceResponse)
356         .toList();
357 }
358
359 @Transactional
360 public DeviceResponseDTO markDeviceAssignedAfterRepair(Long deviceId) {
361     Device device = deviceRepository.findById(deviceId)
362         .orElseThrow(() -> new RuntimeException("Device not found"));
363
364     if (device.getDeviceStatus() != DeviceStatus.REPAIR_DONE) {
365         throw new IllegalArgumentException("Only REPAIR_DONE devices can be moved to ASSIGNED");
366     }
367
368     device.setDeviceStatus(DeviceStatus.ASSIGNED);

```



```

369         Device savedDevice = deviceRepository.save(device);
370         return toDeviceResponse(savedDevice);
371     }
372
373     public List<RepairLogResponseDTO> getRepairLogsForLoggedInVendorDevice(Long deviceId, Authentication
authentication) {
374         String email = AuthUtil.extractEmail(authentication);
375         Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
376             () -> new VendorNotFoundException("Vendor:" + email + " not found")
377         );
378
379         Device device = deviceRepository.findById(deviceId)
380             .orElseThrow(() -> new RuntimeException("Device not found"));
381
382         if (device.getTechVendor() == null || !device.getTechVendor().getId().equals(vendor.getId())) {
383             throw new RuntimeException("Unauthorized access to device repair logs");
384         }
385
386         return repairLogRepository.findByDeviceIdAndDeviceTechVendorIdOrderByRepairDateDescIdDesc(deviceId,
vendor.getId()).stream()
387             .map(this::toRepairLogResponseDTO)
388             .toList();
389     }
390
391     private RepairLogResponseDTO toRepairLogResponseDTO(RepairLog repairLog) {
392         RepairLogResponseDTO dto = new RepairLogResponseDTO();
393         dto.setId(repairLog.getId());
394         dto.setIssueType(repairLog.getIssueType());
395         dto.setDamagedComponent(repairLog.getDamagedComponent());
396         dto.setRepairAction(repairLog.getRepairAction());
397         dto.setReplacedComponent(repairLog.getReplacedComponent());
398         dto.setRepairedBy(repairLog.getRepairedBy());
399         dto.setRepairDate(repairLog.getRepairDate());
400         dto.setRemarks(repairLog.getRemarks());
401
402         if (repairLog.getDevice() != null) {
403             dto.setDeviceId(repairLog.getDevice().getId());
404             dto.setDeviceName(repairLog.getDevice().getDeviceName());
405         }
406
407         if (repairLog.getServiceRequest() != null) {
408             dto.setServiceRequestId(repairLog.getServiceRequest().getId());
409             dto.setServiceRequestStatus(repairLog.getServiceRequest().getStatus() != null
? repairLog.getServiceRequest().getStatus().name()
410                 : null);
411         }
412     }
413
414     return dto;
415 }
416
417 @Transactional
418 public ResponseEntity<String> returnCondemned(Long deviceId, Authentication authentication) {
419     Device device = getAuthorizedDevice(deviceId, authentication);
420
421     if (device.getDeviceStatus() != DeviceStatus.CONDEMNED) {
422         throw new IllegalArgumentException("Only condemned devices can be returned");
423     }
424
425     DeviceAssignment currentAssignment = device.getCurrentAssignment();
426     device.setCurrentAssignment(null);
427     deviceAssignmentRepo.delete(currentAssignment);
428     device.setDeviceStatus(DeviceStatus.RETURNED_TO_VENDOR);
429     if (currentAssignment != null) {
430         deviceAssignmentRepo.delete(currentAssignment);
431     }
432     deviceRepository.save(device);
433
434     return ResponseEntity.ok("Condemned device returned and assignment removed");
435 }
436
437
438 @Transactional
439 public ResponseEntity<String> removeDevice(Long deviceId, Authentication authentication) {
440
441     Device device=deviceRepository.findById(deviceId).orElseThrow(
442         ()-> new RuntimeException("Device Not found")

```

```

443     );
444     String email=AuthUtil.extractEmail(authentication);
445     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
446         ()->new VendorNotFoundException("vendor not found")
447     );
448
449     if (device.getTechVendor() == null || !vendor.getId().equals(device.getTechVendor().getId())) {
450         throw new AccessDeniedException("Access Denied");
451     }
452
453     vendorNegotiationRepository.findByDeviceId(deviceId).forEach(negotiation ->
negotiation.setDevice(null));
454     repairBillRepository.deleteAll(repairBillRepository.findByDeviceId(deviceId));
455     repairLogRepository.deleteAll(repairLogRepository.findByDeviceIdIn(List.of(deviceId)));
456     serviceRequestRepository.deleteAll(serviceRequestRepository.findByDeviceId(deviceId));
457
458     List<DeviceAssignment> deviceAssignments =
deviceAssignmentRepo.findByDeviceIdOrderByAssignedDateDescIdDesc(deviceId);
459     device.setCurrentAssignment(null);
460     if (!deviceAssignments.isEmpty()) {
461         deviceAssignmentRepo.deleteAll(deviceAssignments);
462     }
463
464     device.setTechVendor(null);
465     deviceRepository.delete(device);
466     return ResponseEntity.ok("Device removed from stock");
467
468 }
469
470 public List<DeviceResponseDTO> getDevicesUnderRepair(Authentication authentication) {
471     String email=AuthUtil.extractEmail(authentication);
472     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
473         ()->new VendorNotFoundException("Vendor Not Found:"+email)
474     );
475     List<Device>
devices=deviceRepository.findByTechVendorAndDeviceStatus(vendor, DeviceStatus.UNDER_REPAIR);
476     return devices.stream().map(device -> toDeviceResponse(device)).toList();
477 }
478 }
479

```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
6  import com.example.EmployeeManagementSystem.Enum.Role;
7  import com.example.EmployeeManagementSystem.Enum.Status;
8  import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
9  import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
11 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
12 import com.example.EmployeeManagementSystem.Util.AuthUtil;
13 import org.slf4j.Logger;
14 import org.slf4j.LoggerFactory;
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.security.crypto.password.PasswordEncoder;
18 import org.springframework.stereotype.Service;
19 import org.springframework.transaction.annotation.Transactional;
20
21 import java.time.ZoneId;
22 import java.util.List;
23 import java.util.stream.Collectors;
24
25 @Service
26 public class EmployeeService {
27
28     private static final Logger log = LoggerFactory.getLogger(EmployeeService.class);
29
30     private final EmployeeRepo employeeRepo;
31     private final LeaveRequestRepo leaveRequestRepo;
32     private final PasswordEncoder passwordEncoder;
33     private final RefreshTokenRepository refreshTokenRepository; // FIX: added
34     private final LeaveAccrualService leaveAccrualService;
35
36     public EmployeeService(EmployeeRepo employeeRepo,
37                             LeaveRequestRepo leaveRequestRepo,
38                             PasswordEncoder passwordEncoder,
39                             RefreshTokenRepository refreshTokenRepository,
40                             LeaveAccrualService leaveAccrualService) {
41         this.employeeRepo = employeeRepo;
42         this.leaveRequestRepo = leaveRequestRepo;
43         this.passwordEncoder = passwordEncoder;
44         this.refreshTokenRepository = refreshTokenRepository; // FIX: injected
45         this.leaveAccrualService = leaveAccrualService;
46     }
47
48     public List<Employee> getAllEmployees() {
49         return employeeRepo.findAll();
50     }
51
52     @Transactional
53     public Employee createEmployee(AdminEmployeeDTO employeeDTO) {
54         log.info("Creating new employee with email: {}", employeeDTO.getEmail());
55         var employee = new Employee();
56         if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
57         if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
58         if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
59             employee.setTimezone(employeeDTO.getTimezone());
60         } else {
61             employee.setTimezone("UTC");
62         }
63         employee.setRole(Role.EMPLOYEE);
64         if (employeeDTO.getPassword() == null) {
65             throw new RuntimeException("Password is required");
66         }
67         employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
68         employee.setGender(employeeDTO.getGender());
69         Employee manager=employeeRepo.findById(employeeDTO.getManagerId()).orElseThrow(
70             ()->new EmployeeNotFound("Employee with not found"+ employeeDTO.getManagerId())
71     );

```

```

72     employee.setDept(manager.getDept());
73     employee.setManager(manager);
74     Employee savedEmployee = employeeRepo.save(employee);
75     leaveAccrualService.grantInitialMonthlyAccrual(savedEmployee, savedEmployee.getJoined_at());
76     return savedEmployee;
77 }
78
79 @Transactional
80 public Employee createManager(EmployeeDTO employeeDTO) {
81     log.info("Creating new manager with email: {}", employeeDTO.getEmail());
82     var employee = new Employee();
83     if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
84     if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
85     if (employeeDTO.getDept() != null) employee.setDept(employeeDTO.getDept());
86     if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
87         employee.setTimezone(employeeDTO.getTimezone());
88     } else {
89         employee.setTimezone("UTC");
90     }
91     employee.setRole(Role.MANAGER);
92     if (employeeDTO.getPassword() == null) {
93         throw new RuntimeException("Password is required");
94     }
95     employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
96     employee.setGender(employeeDTO.getGender());
97     Employee savedEmployee = employeeRepo.save(employee);
98     leaveAccrualService.grantInitialMonthlyAccrual(savedEmployee, savedEmployee.getJoined_at());
99     return savedEmployee;
100 }
101
102 public Employee updateEmployee(long id, EmployeeDTO employee) {
103     var updateEmployee = employeeRepo.findById(id).orElseThrow(
104         () -> new EmployeeNotFound("Employee with id:" + id + " not found")
105     );
106     if (employee.getName() != null)
107         updateEmployee.setName(employee.getName());
108     if (employee.getEmail() != null)
109         updateEmployee.setEmail(employee.getEmail());
110     if (employee.getDept() != null)
111         updateEmployee.setDept(employee.getDept());
112     if (employee.getTimezone() != null && isValidTimezone(employee.getTimezone())) {
113         updateEmployee.setTimezone(employee.getTimezone());
114     }
115     return employeeRepo.save(updateEmployee);
116 }
117
118 /**
119  * FIX: Delete order must be:
120  * 1. refresh_token (FK employee)
121  * 2. leave_request (FK employee)
122  * 3. employee
123  *
124  * Previously only leave_request was deleted first, causing a
125  * FK constraint failure on refresh_token when deleting the employee.
126  */
127 @Transactional
128 public void deleteEmployee(long id) {
129     var employee = employeeRepo.findById(id).orElseThrow(
130         () -> new EmployeeNotFound("Employee with id:" + id + " not found")
131     );
132
133     // Step 1: delete refresh tokens for this employee
134     refreshTokenRepository.deleteByEmployee(employee);
135
136     // Step 2: delete leave requests for this employee
137     List<LeaveRequest> leaveRequests =
138         leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
139     leaveRequestRepo.deleteAll(leaveRequests);
140
141     // Step 3: delete the employee
142     employeeRepo.delete(employee);
143 }
144
145 public ResponseEntity<String> inactivateUser(long employeeId) {
146     var employee = employeeRepo.findById(employeeId).orElseThrow(
147         () -> new EmployeeNotFound("Employee with id:" + employeeId + " not found")
148     );

```

```

148     });
149     if (employee.getStatus() == Status.INACTIVE) {
150         return ResponseEntity.badRequest()
151             .body("Status of the employee with id: " + employeeId + " is already set to inactive");
152     }
153     employee.setStatus(Status.INACTIVE);
154     employeeRepo.save(employee);
155     return ResponseEntity.ok("Status of the employee with id: " + employeeId + " is set to inactive");
156 }
157
158 /** Validate that the given string is a recognized IANA timezone id. */
159 private boolean isValidTimezone(String tz) {
160     try {
161         ZoneId.of(tz);
162         return true;
163     } catch (Exception e) {
164         return false;
165     }
166 }
167
168 public Employee getAccount(Authentication authentication) {
169     String email = AuthUtil.extractEmail(authentication);
170     Employee employee = employeeRepo.findByEmail(email).orElseThrow(
171         () -> new EmployeeNotFound("Employee :" + email + " NOT FOUND")
172     );
173     return employee;
174 }
175
176 public EmployeeAttendanceResponseDTO getEmployeeAttendanceOverview() {
177     List<Employee> employees = employeeRepo.findAll();
178
179     List<EmployeeStatusDTO> workingEmployees = employees.stream()
180         .filter(e -> e.getStatus() == Status.ACTIVE)
181         .map(this::toStatusDTO)
182         .collect(Collectors.toList());
183
184     List<EmployeeStatusDTO> onLeaveEmployees = employees.stream()
185         .filter(e -> e.getStatus() == Status.ON_LEAVE)
186         .map(this::toStatusDTO)
187         .collect(Collectors.toList());
188
189     EmployeeAttendanceResponseDTO response = new EmployeeAttendanceResponseDTO();
190     response.setWorkingCount(workingEmployees.size());
191     response.setOnLeaveCount(onLeaveEmployees.size());
192     response.setWorkingEmployees(workingEmployees);
193     response.setOnLeaveEmployees(onLeaveEmployees);
194     return response;
195 }
196
197 private EmployeeStatusDTO toStatusDTO(Employee employee) {
198     EmployeeStatusDTO dto = new EmployeeStatusDTO();
199     dto.setEmployeeId(employee.getEmployeeId());
200     dto.setName(employee.getName());
201     dto.setEmail(employee.getEmail());
202     dto.setDept(employee.getDept());
203     dto.setRole(employee.getRole());
204     return dto;
205 }
206
207 public ResponseEntity<List<ManagerDTO>> getAllManagers() {
208     List<ManagerDTO> managers = employeeRepo.findByRole(Role.MANAGER).stream()
209         .map(this::toManagerDTO)
210         .collect(Collectors.toList());
211     return ResponseEntity.ok(managers);
212 }
213
214 private ManagerDTO toManagerDTO(Employee employee) {
215     ManagerDTO dto = new ManagerDTO();
216     dto.setManagerId(employee.getEmployeeId());
217     dto.setName(employee.getName());
218     dto.setEmail(employee.getEmail());
219     dto.setDept(employee.getDept());
220     return dto;
221 }
222 }
223

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.HolidayDTO;
4 import com.example.EmployeeManagementSystem.Entity.FixedHoliday;
5 import com.example.EmployeeManagementSystem.Repository.FixedHolidayRepository;
6 import org.springframework.stereotype.Service;
7
8 import java.time.LocalDate;
9 import java.util.List;
10 @Service
11 public class HolidayService {
12     private final FixedHolidayRepository holidayRepository;
13
14     public HolidayService(FixedHolidayRepository holidayRepository) {
15         this.holidayRepository = holidayRepository;
16     }
17
18     // Admin adds a holiday
19     public FixedHoliday addHoliday(HolidayDTO dto) {
20         FixedHoliday h = new FixedHoliday();
21         h.setName(dto.getName());
22         h.setDate(LocalDate.parse(dto.getDate()));
23         h.setDescription(dto.getDescription());
24         h.setYear(LocalDate.parse(dto.getDate()).getYear());
25         return holidayRepository.save(h);
26     }
27
28     // Everyone can view holidays
29     public List<FixedHoliday> getHolidaysByYear(int year) {
30         return holidayRepository.findByYear(year);
31     }
32
33     // Admin deletes a holiday
34     public void deleteHoliday(Long id) {
35         holidayRepository.deleteById(id);
36     }
37 }
38
39
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.InvoiceDTO;
4 import com.example.EmployeeManagementSystem.Entity.Invoice;
5 import com.example.EmployeeManagementSystem.Entity.RentalBill;
6 import com.example.EmployeeManagementSystem.Entity.RepairBill;
7 import com.example.EmployeeManagementSystem.Entity.Vendor;
8 import com.example.EmployeeManagementSystem.Enum.InvoiceStatus;
9 import com.example.EmployeeManagementSystem.Enum.InvoiceType;
10 import com.example.EmployeeManagementSystem.Enum.PaymentStatus;
11 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
12 import com.example.EmployeeManagementSystem.Repository.InvoiceRepository;
13 import com.example.EmployeeManagementSystem.Repository.RentalBillRepository;
14 import com.example.EmployeeManagementSystem.Repository.RepairBillRepository;
15 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
16 import com.example.EmployeeManagementSystem.Util.AuthUtil;
17 import org.springframework.http.ResponseEntity;
18 import org.springframework.security.access.AccessDeniedException;
19 import org.springframework.security.core.Authentication;
20 import org.springframework.stereotype.Service;
21
22 import java.time.LocalDate;
23 import java.util.List;
24 @Service
25 public class InvoiceService {
26     private final VendorRepo vendorRepo;
27     private final InvoiceRepository invoiceRepository;
28     private final RentalBillRepository rentalBillRepository;
29     private final RepairBillRepository repairBillRepository;
30
31
32     public InvoiceService(VendorRepo vendorRepo, InvoiceRepository invoiceRepository, RentalBillRepository
rentalBillRepository, RepairBillRepository repairBillRepository) {
33         this.vendorRepo = vendorRepo;
34         this.invoiceRepository = invoiceRepository;
35         this.rentalBillRepository = rentalBillRepository;
36         this.repairBillRepository = repairBillRepository;
37     }
38
39
40     public List<InvoiceDTO> getPendingInvoicesForCurrentVendor(Authentication authentication) {
41         String email= AuthUtil.extractEmail(authentication);
42         Vendor current = vendorRepo.findByEmail(email).orElseThrow(
43             ()->new VendorNotFoundException("vendor :"+email+" Not Found")
44         );
45         return invoiceRepository
46             .findByVendorAndStatus(current, InvoiceStatus.SENT)
47             .stream().map(this::toInvoiceDTO).toList();
48     }
49
50     private InvoiceDTO toInvoiceDTO(Invoice invoice) {
51         InvoiceDTO invoiceDTO = new InvoiceDTO();
52         invoiceDTO.setId(invoice.getId());
53         invoiceDTO.setInvoiceType(invoice.getInvoiceType().name());
54         invoiceDTO.setVendorName(invoice.getVendor().getName());
55         invoiceDTO.setAmount(invoice.getAmount());
56         invoiceDTO.setIssuedDate(invoice.getIssuedDate());
57         invoiceDTO.setStatus(invoice.getStatus().name());
58         invoiceDTO.setSeenAt(invoice.getSeenAt());
59         return invoiceDTO;
60     }
61
62     public List<InvoiceDTO> getAllInvoices() {
63         List<Invoice> invoices=invoiceRepository.findAll();
64         return invoices.stream().map(invoice -> toInvoiceDTO(invoice)).toList();
65     }
66
67     public ResponseEntity<String> acknowledgeInvoice(Long id,Authentication authentication) {
68         Invoice invoice = invoiceRepository.findById(id)
69             .orElseThrow(() -> new RuntimeException("Invoice not found"));
70

```

```
71 // Security: ensure it belongs to current vendor
72 String email=AuthUtil.extractEmail(authentication);
73 Vendor current = vendorRepo.findByEmail(email).orElseThrow(
74     ()->new VendorNotFoundException("vendor: "+email+" Not Found")
75 );
76 if (!invoice.getVendor().getId().equals(current.getId()))
77     throw new AccessDeniedException("Not your invoice");
78
79 invoice.setStatus(InvoiceStatus.ACKNOWLEDGED);
80 invoice.setSeenAt(LocalDate.now());
81
82 // Mark the underlying bill as PAID
83 if (invoice.getInvoiceType() == InvoiceType.REPAIR && invoice.getRepairBill() != null) {
84     RepairBill bill = invoice.getRepairBill();
85     bill.setPaymentStatus(PaymentStatus.ACKNOWLEDGED);
86     repairBillRepository.save(bill);
87 } else if (invoice.getInvoiceType() == InvoiceType.RENTAL && invoice.getRentalBill() != null) {
88     RentalBill bill = invoice.getRentalBill();
89     bill.setPaymentStatus(PaymentStatus.ACKNOWLEDGED);
90     rentalBillRepository.save(bill);
91 }
92
93 invoiceRepository.save(invoice);
94 return ResponseEntity.ok("Invoice acknowledged");
95 }
96 }
97
```



```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveType;
6 import com.example.EmployeeManagementSystem.Enum.Gender;
7 import com.example.EmployeeManagementSystem.Enum.Role;
8 import com.example.EmployeeManagementSystem.Enum.Status;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
12 import org.slf4j.Logger;
13 import org.slf4j.LoggerFactory;
14 import org.springframework.stereotype.Service;
15 import org.springframework.transaction.annotation.Transactional;
16
17 import java.math.BigDecimal;
18 import java.time.LocalDate;
19 import java.time.YearMonth;
20 import java.util.List;
21 import java.util.Set;
22
23 @Service
24 public class LeaveAccrualService {
25
26     private static final Logger log = LoggerFactory.getLogger(LeaveAccrualService.class);
27     private static final BigDecimal ONE_DAY = BigDecimal.ONE;
28     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL");
29     private static final Set<Role> LEAVE_ACCRUAL_ROLES = Set.of(Role.EMPLOYEE, Role.MANAGER, Role.ADMIN);
30
31     private final EmployeeRepo employeeRepository;
32     private final LeaveTypeRepository leaveTypeRepository;
33     private final LeaveEntitlementRepository entitlementRepository;
34
35     public LeaveAccrualService(EmployeeRepo employeeRepository,
36                               LeaveTypeRepository leaveTypeRepository,
37                               LeaveEntitlementRepository entitlementRepository) {
38         this.employeeRepository = employeeRepository;
39         this.leaveTypeRepository = leaveTypeRepository;
40         this.entitlementRepository = entitlementRepository;
41     }
42
43     @Transactional
44     public void runMonthlyAccrual(LocalDate accrualDate) {
45         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE).stream()
46             .filter(employee -> employee.getRole() != null)
47             .filter(employee -> LEAVE_ACCRUAL_ROLES.contains(employee.getRole()))
48             .toList();
49         List<LeaveType> allLeaveTypes = leaveTypeRepository.findAll();
50
51         log.info("Starting monthly accrual for {} – {} active employees",
52                 accrualDate, activeEmployees.size());
53
54         for (Employee employee : activeEmployees) {
55             accrueForEmployee(employee, allLeaveTypes, accrualDate);
56         }
57
58         log.info("Monthly accrual complete for {}", accrualDate);
59     }
60
61     @Transactional
62     public void grantInitialMonthlyAccrual(Employee employee, LocalDate joinDate) {
63         if (joinDate == null) {
64             joinDate = LocalDate.now();
65         }
66
67         LocalDate firstDayOfLastWeek = YearMonth.from(joinDate).atEndOfMonth().minusDays(6);
68         if (!joinDate.isBefore(firstDayOfLastWeek)) {
69             log.info("Initial leave accrual skipped for employee {} joining in the last week of month {}",
70                     employee.getEmployeeId(), joinDate);
71             return;

```

```

72     }
73
74     accrueForEmployee(employee, leaveTypeRepository.findAll(), joinDate);
75     log.info("Initial monthly leave accrual complete for employee {} on {}",
76             employee.getEmployeeId(), joinDate);
77 }
78
79 private void accrueForEmployee(Employee employee, List<LeaveType> leaveTypes, LocalDate accrualDate) {
80     int year = accrualDate.getYear();
81
82     for (LeaveType leaveType : leaveTypes) {
83         if (!SUPPORTED_LEAVE_TYPES.contains(leaveType.getName())) {
84             continue;
85         }
86         if (!isEligible(employee, leaveType, accrualDate)) {
87             continue;
88         }
89
90         LeaveEntitlement entitlement = entitlementRepository
91             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
92                 employee.getEmployeeId(), leaveType.getId(), year)
93             .orElseGet(() -> createNewEntitlement(employee, leaveType, year));
94
95         entitlement.setAccruedThisYear(
96             entitlement.getAccruedThisYear().add(ONE_DAY)
97         );
98
99         entitlement.setClosingBalance(
100             entitlement.getOpeningBalance()
101                 .add(entitlement.getAccruedThisYear())
102                 .subtract(entitlement.getUsedThisYear())
103         );
104
105         entitlementRepository.save(entitlement);
106         log.debug("Accrued 1 {} day for employee {} (year {})",
107                 leaveType.getName(), employee.getEmployeeId(), year);
108     }
109 }
110
111 // -----
112 // Eligibility check – single place for all rules
113 // -----
114 private boolean isEligible(Employee employee, LeaveType leaveType, LocalDate accrualDate) {
115     // 1. Gender restriction (Maternity = female only)
116     if (leaveType.isGenderRestricted()) {
117         Gender restriction = leaveType.getGenderRestriction();
118         if (restriction != null && restriction != employee.getGender()) {
119             return false;
120         }
121     }
122 }
123
124 // 2. Employee must have joined ON OR BEFORE the accrual date
125 //     (no accrual for employees joining after the 1st of this month)
126 if (employee.getJoined_at() != null && employee.getJoined_at().isAfter(accrualDate)) {
127     return false;
128 }
129
130 return true;
131 }
132
133 // -----
134 // Creates a fresh entitlement row, carrying forward previous year's
135 // closing balance if this is the first row of the year
136 // -----
137 private LeaveEntitlement createNewEntitlement(Employee employee,
138                                             LeaveType leaveType,
139                                             int year) {
140     LeaveEntitlement entitlement = new LeaveEntitlement();
141     entitlement.setEmployee(employee);
142     entitlement.setLeaveType(leaveType);
143     entitlement.setYear(year);
144
145     // Carry forward last year's closing balance (if the type supports it)
146     if (leaveType.isCarriesForward()) {
147         BigDecimal lastYearClosing = entitlementRepository

```

```
148         .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(  
149             employee.getEmployeeId(), leaveType.getId(), year - 1)  
150         .map(LeaveEntitlement::getClosingBalance)  
151         .orElse(BigDecimal.ZERO);  
152  
153         entitlement.setOpeningBalance(lastYearClosing);  
154     }  
155  
156     return entitlementRepository.save(entitlement);  
157 }  
158 }  
159
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6 import com.example.EmployeeManagementSystem.Entity.LeaveType;
7 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
8 import com.example.EmployeeManagementSystem.Enum.Gender;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
12 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
13 import org.springframework.security.access.AccessDeniedException;
14 import org.springframework.stereotype.Service;
15 import org.springframework.transaction.annotation.Transactional;
16
17 import java.math.BigDecimal;
18 import java.time.LocalDate;
19 import java.util.Set;
20 import java.util.LinkedHashMap;
21 import java.util.List;
22 import java.util.Map;
23
24 @Service
25 public class LeaveDashboardService {
26
27     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL", "MATERNITY");
28
29     private final EmployeeRepo employeeRepository;
30     private final LeaveEntitlementRepository leaveEntitlementRepository;
31     private final LeaveTypeRepository leaveTypeRepository;
32     private final LeaveWarningRepository warningRepository;
33
34     public LeaveDashboardService(EmployeeRepo employeeRepository,
35                                 LeaveEntitlementRepository leaveEntitlementRepository,
36                                 LeaveTypeRepository leaveTypeRepository,
37                                 LeaveWarningRepository warningRepository) {
38         this.employeeRepository = employeeRepository;
39         this.leaveEntitlementRepository = leaveEntitlementRepository;
40         this.leaveTypeRepository = leaveTypeRepository;
41         this.warningRepository = warningRepository;
42     }
43
44     @Transactional
45     public LeaveDashboardResponse getDashboard(Long requestedEmployeeId,
46                                              Long callerEmployeeId,
47                                              String callerRole) {
48         assertAccess(requestedEmployeeId, callerEmployeeId, callerRole);
49
50         Employee employee = employeeRepository.findById(requestedEmployeeId)
51             .orElseThrow(() -> new IllegalArgumentException(
52                 "Employee not found: " + requestedEmployeeId));
53
54         int year = LocalDate.now().getYear();
55
56         ensureCurrentYearEntitlements(employee, year);
57         List<LeaveEntitlement> entitlements =
58             leaveEntitlementRepository.findByEmployeeIdAndYear(requestedEmployeeId, year);
59
60         List<LeaveWarning> warnings =
61             warningRepository.findByEmployeeIdAndIsReadFalseOrderByWarningDateDesc(
62                 requestedEmployeeId);
63
64         LeaveDashboardResponse.LeaveBalanceDto sick = buildBalanceDto(entitlements, "SICK");
65         LeaveDashboardResponse.LeaveBalanceDto casual = buildBalanceDto(entitlements, "CASUAL");
66         LeaveDashboardResponse.LeaveBalanceDto maternity = buildBalanceDto(entitlements, "MATERNITY");
67
68         Map<String, LeaveDashboardResponse.LeaveBalanceDto> leaveBalances = buildBalanceMap(entitlements);
69         BigDecimal carryTotal = entitlements.stream()
70             .filter(e -> isSupportedLeaveType(e.getLeaveType()))
71             .filter(e -> e.getLeaveType().isCarriesForward())

```

```

72         .map(LeaveEntitlement::getAvailableBalance)
73         .reduce(BigDecimal.ZERO, BigDecimal::add);
74
75     LeaveDashboardResponse response = new LeaveDashboardResponse();
76     response.setEmployeeId(employee.getEmployeeId());
77     response.setEmployeeName(employee.getName());
78     response.setYear(year);
79     response.setSick(sick);
80     response.setCasual(casual);
81     response.setMaternity(maternity);
82     response.setLeaveBalances(leaveBalances);
83     response.setCarryForwardMeter(new LeaveDashboardResponse.CarryForwardMeterDto(carryTotal, 30));
84     response.setWarnings(warnings.stream()
85         .map(w -> new LeaveDashboardResponse.WarningDto(
86             w.getId(),
87             w.getWarningType().name(),
88             w.getMessage(),
89             w.getWarningDate().toString()))
90         .toList());
91
92     return response;
93 }
94
95 // Mark warning read
96
97 @Transactional
98 public void markWarningRead(Long warningId, Long callerEmployeeId, String callerRole) {
99     LeaveWarning warning = warningRepository.findById(warningId)
100         .orElseThrow(() -> new IllegalArgumentException("Warning not found: " + warningId));
101
102     assertAccess(warning.getEmployee().getEmployeeId(), callerEmployeeId, callerRole);
103
104     warning.setRead(true);
105     warningRepository.save(warning);
106 }
107
108 // Team dashboard (manager/admin)
109
110 @Transactional
111 public List<LeaveDashboardResponse> getTeamDashboard(Long managerId, String callerRole) {
112     if (!"MANAGER".equals(callerRole) && !"ADMIN".equals(callerRole)) {
113         throw new AccessDeniedException("Only managers and admins can view team dashboards");
114     }
115
116     List<Employee> team = employeeRepository.findByManagerEmployeeId(managerId);
117     return team.stream()
118         .map(emp -> getDashboard(emp.getEmployeeId(), managerId, callerRole))
119         .toList();
120 }
121
122 // Helpers
123
124 private void ensureCurrentYearEntitlements(Employee employee, int year) {
125     List<LeaveType> leaveTypes = leaveTypeRepository.findAll();
126
127     for (LeaveType leaveType : leaveTypes) {
128         if (!isSupportedLeaveType(leaveType)) {
129             continue;
130         }
131         if (!isEligible(employee, leaveType)) {
132             continue;
133         }
134
135         leaveEntitlementRepository
136             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
137                 employee.getEmployeeId(), leaveType.getId(), year)
138             .orElseGet(() -> createEntitlement(employee, leaveType, year));
139     }
140 }
141
142 private boolean isEligible(Employee employee, LeaveType leaveType) {
143     if (!leaveType.isGenderRestricted()) {
144         return true;
145     }
146
147     Gender restriction = leaveType.getGenderRestriction();

```

```

148         return restriction == null || restriction == employee.getGender();
149     }
150
151     private LeaveEntitlement createEntitlement(Employee employee, LeaveType leaveType, int year) {
152         LeaveEntitlement entitlement = new LeaveEntitlement();
153         entitlement.setEmployee(employee);
154         entitlement.setLeaveType(leaveType);
155         entitlement.setYear(year);
156
157         if (leaveType.isCarriesForward()) {
158             BigDecimal lastYearClosing = leaveEntitlementRepository
159                 .findByEmployeeIdAndLeaveTypeIdAndYear(
160                     employee.getId(), leaveType.getId(), year - 1)
161                 .map(LeaveEntitlement::getClosingBalance)
162                 .orElse(BigDecimal.ZERO);
163             entitlement.setOpeningBalance(lastYearClosing);
164         }
165
166         return leaveEntitlementRepository.save(entitlement);
167     }
168
169     private Map<String, LeaveDashboardResponse.LeaveBalanceDto> buildBalanceMap(
170         List<LeaveEntitlement> entitlements) {
171         Map<String, LeaveDashboardResponse.LeaveBalanceDto> balances = new LinkedHashMap<>();
172         for (LeaveEntitlement entitlement : entitlements) {
173             String typeName = entitlement.getLeaveType().getName();
174             if (!SUPPORTED_LEAVE_TYPES.contains(typeName)) {
175                 continue;
176             }
177             balances.put(typeName, toBalanceDto(entitlement, typeName));
178         }
179         return balances;
180     }
181
182     private LeaveDashboardResponse.LeaveBalanceDto buildBalanceDto(List<LeaveEntitlement> entitlements,
183                                                                     String typeName) {
184         return entitlements.stream()
185             .filter(e -> typeName.equalsIgnoreCase(e.getLeaveType().getName()))
186             .findFirst()
187             .map(e -> toBalanceDto(e, typeName))
188             .orElse(null); // null = leave type not applicable for this employee
189     }
190
191     private LeaveDashboardResponse.LeaveBalanceDto toBalanceDto(LeaveEntitlement entitlement,
192                                                                 String typeName) {
193         return new LeaveDashboardResponse.LeaveBalanceDto(
194             typeName,
195             entitlement.getOpeningBalance(),
196             entitlement.getAccruedThisYear(),
197             entitlement.getUsedThisYear(),
198             entitlement.getAvailableBalance(),
199             entitlement.getLeaveType().isCarriesForward());
200     }
201
202     private boolean isSupportedLeaveType(LeaveType leaveType) {
203         return leaveType != null && SUPPORTED_LEAVE_TYPES.contains(leaveType.getName());
204     }
205
206     private void assertAccess(Long targetId, Long callerId, String role) {
207         boolean isSelf = targetId.equals(callerId);
208         boolean isManager = "MANAGER".equals(role) || "ADMIN".equals(role);
209         if (!isSelf && !isManager) {
210             throw new AccessDeniedException(
211                 "Employee " + callerId + " cannot view dashboard for " + targetId);
212         }
213     }
214 }
215

```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
7  import com.example.EmployeeManagementSystem.Enum.*;
8  import com.example.EmployeeManagementSystem.Exception.*;
9  import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
12 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
13 import com.example.EmployeeManagementSystem.Util.AuthUtil;
14 import org.slf4j.Logger;
15 import com.example.EmployeeManagementSystem.Enum.LeaveType;
16 import org.slf4j.LoggerFactory;
17 import org.springframework.http.HttpStatus;
18 import org.springframework.http.ResponseEntity;
19 import org.springframework.security.core.Authentication;
20 import org.springframework.stereotype.Service;
21 import org.springframework.transaction.annotation.Transactional;
22
23 import java.math.BigDecimal;
24 import java.time.LocalDate;
25 import java.time.ZoneId;
26 import java.time.temporal.ChronoUnit;
27 import java.util.ArrayList;
28 import java.util.List;
29 import java.util.Map;
30 import java.util.stream.Collectors;
31
32 @Service
33 public class LeaveRequestService {
34     private final LeaveRequestRepo leaveRequestRepo;
35     private final EmployeeRepo employeeRepo;
36     private final LeaveTypeRepository leaveTypeRepository;
37     private final LeaveEntitlementRepository leaveEntitlementRepository;
38     private final NotificationService notificationService;
39     private static final Logger log = LoggerFactory.getLogger(LeaveRequestService.class);
40
41     public LeaveRequestService(LeaveRequestRepo leaveRequestRepo,
42                               EmployeeRepo employeeRepo,
43                               LeaveTypeRepository leaveTypeRepository,
44                               LeaveEntitlementRepository leaveEntitlementRepository,
45                               NotificationService notificationService) {
46         this.leaveRequestRepo = leaveRequestRepo;
47         this.employeeRepo = employeeRepo;
48         this.leaveTypeRepository = leaveTypeRepository;
49         this.leaveEntitlementRepository = leaveEntitlementRepository;
50         this.notificationService=notificationService;
51     }
52
53     public List<LeaveResponseDTO> getAllTheLeaveRequest(){
54         List<LeaveRequest> requestList=leaveRequestRepo.findAll();
55         List<LeaveResponseDTO> dtos=new ArrayList<>();
56         for(LeaveRequest request:requestList){
57             dtos.add(convertToDTO(request));
58         }
59         return dtos;
60     }
61
62
63     public ResponseEntity<?> createRequest(Authentication authentication, LeaveRequestDTO requestDTO){
64         var leaveRequest=new LeaveRequest();
65         String email= AuthUtil.extractEmail(authentication);
66         //Validate employee exists and is active
67         var employee=employeeRepo.findByEmail(email).orElseThrow(()->
68             new EmployeeNotFound("Employee with name: "+email+" is not found")
69         );
70         if (employee.getStatus() != Status.ACTIVE) {
71             return ResponseEntity.badRequest()

```

```

72         .body(Map.of("error", "Only active employees can apply for leave"));
73     }
74
75     if (requestDTO.getLeaveType() == LeaveType.MATERNITY) {
76         if(employee.getGender()==Gender.M){
77             return ResponseEntity.status(HttpStatus.BAD_REQUEST)
78                 .body("Maternity leave is not applicable");
79         }
80         return ResponseEntity.status(HttpStatus.BAD_REQUEST)
81             .body("Please use the maternity leave application process.");
82     }
83
84     leaveRequest.setEmployee(employee);
85     //Checking if the start date is before today
86     String timezone= employee.getTimezone();
87     ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
88         ? timezone
89         : "UTC");
90     LocalDate today = LocalDate.now(zoneId);
91     if (requestDTO.getStartDate().isBefore(today)) {
92         throw new InvalidStartDateException("Start date cannot be before current date");
93     }
94     //Checking if the end date is before start date
95     if (requestDTO.getEndDate().isBefore(requestDTO.getStartDate())) {
96         throw new InvalidEndDateException("End date must be equal to or greater than start date");
97     }
98     //optional
99     long daysRequested = ChronoUnit.DAYS.between(requestDTO.getStartDate(), requestDTO.getEndDate()) +
100     1;
101     if (daysRequested > 30) {
102         return ResponseEntity.badRequest()
103             .body(Map.of("error", "Leave cannot exceed 30 consecutive days"));
104     }
105     // ----- SICK -----
106     if (requestDTO.getLeaveType() == LeaveType.SICK) {
107         if (daysRequested > 1) {
108             return ResponseEntity.badRequest()
109                 .body(Map.of("error", "Sick leave is limited to 1 day per month."));
110         }
111         long sickDaysUsedThisMonth = leaveRequestRepo.countApprovedSickDaysInMonth(
112             employee, today.getYear(), today.getMonthValue());
113         var sickLeaveType = leaveTypeRepository.findByName("SICK")
114             .orElseThrow(() -> new RuntimeException("SICK leave type not configured"));
115         var entitlement = leaveEntitlementRepository
116             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
117                 employee.getEmployeeId(),
118                 sickLeaveType.getId(),
119                 today.getYear())
120             .orElseGet(() -> createEntitlement(employee, sickLeaveType, today.getYear()));
121         BigDecimal availableSick = entitlement.getAvailableBalance();
122         if (sickDaysUsedThisMonth >= 1 || availableSick.compareTo(BigDecimal.ONE) < 0) {
123             requestDTO.setLeaveType(LeaveType.UNPAID);
124         }
125     }
126
127     // ----- CASUAL -----
128     if (requestDTO.getLeaveType() == LeaveType.CASUAL) {
129         com.example.EmployeeManagementSystem.Entity.LeaveType
130         leaveType=leaveTypeRepository.findByName("CASUAL").orElseThrow(
131             ()->new RuntimeException("Leave type Not found")
132         );
133         int leaveYear=requestDTO.getStartDate().getYear();
134         LeaveEntitlement
135         leaveEntitlement=leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
136             employee.getEmployeeId(),
137             leaveType.getId(),
138             leaveYear
139         ).orElseGet(()->createEntitlement(employee, leaveType, leaveYear));
140
141
142
143
144

```



```

145         BigDecimal availableCasual = leaveEntitlement.getAvailableBalance(); // opening + accrued -
used
146         BigDecimal requestedCasual = BigDecimal.valueOf(daysRequested);
147         if (availableCasual.compareTo(BigDecimal.ZERO) <= 0) {
148             return ResponseEntity.badRequest()
149                 .body(Map.of("error", "No casual leave balance available."));
150         }
151
152         if (requestedCasual.compareTo(availableCasual) > 0) {
153             return ResponseEntity.badRequest()
154                 .body(Map.of(
155                     "error",
156                     "Requested casual leave exceeds available balance. Available: " +
availableCasual
157                 ));
158         }
159     }
160 }
161
162 // ----- UNPAID cap -----
163 if (requestDTO.getLeaveType() == LeaveType.UNPAID) {
164     // Max unpaid days = number of leave types exhausted this month
165     // (1 for exhausted sick + 1 for exhausted casual = max 2)
166     long sickUsedThisMonth = leaveRequestRepo.countApprovedSickDaysInMonth(
167         employee, today.getYear(), today.getMonthValue());
168     long casualBalance = getCasualBalance(employee, today.getYear());
169
170     int maxUnpaid = 0;
171     if (sickUsedThisMonth >= 1) maxUnpaid++; // sick exhausted
172     if (casualBalance <= 0) maxUnpaid++; // casual exhausted
173
174     long unpaidDaysThisMonth = leaveRequestRepo.countUnpaidDaysInMonth(
175         employee, requestDTO.getStartDate().getYear(),
176         requestDTO.getStartDate().getMonthValue());
177
178     if (maxUnpaid == 0) {
179         return ResponseEntity.badRequest()
180             .body(Map.of("error",
181                 "You still have paid leave available. Please use sick or casual leave
first."));
182     }
183     if (unpaidDaysThisMonth >= maxUnpaid) {
184         return ResponseEntity.badRequest()
185             .body(Map.of("error",
186                 "Unpaid leave limit (" + maxUnpaid + " day(s) this month) already
reached."));
187     }
188 }
189
190 long duplicateCount= leaveRequestRepo.checkDuplicate(
191     employee.getEmployeeId(),
192     requestDTO.getStartDate(),
193     requestDTO.getEndDate(),
194     LeaveStatus.PENDING.name()
195 );
196 if(duplicateCount>0){
197     throw new DuplicateRequestException("Duplicate leave request");
198 }
199
200 //checking if there is any overlapping leave request of that employee
201 long count= leaveRequestRepo.countOverlappingLeave(
202     employee.getEmployeeId(),
203     requestDTO.getStartDate(),
204     requestDTO.getEndDate()
205 );
206 if(count>0){
207     throw new OverlappingLeaveException("Overlapping leave exists");
208 }
209 leaveRequest.setStartDate(requestDTO.getStartDate());
210 leaveRequest.setEndDate(requestDTO.getEndDate());
211
212 if(requestDTO.getLeaveType()==null){
213     return ResponseEntity.status(HttpStatus.BAD_REQUEST)
214         .body("Leave type is required..");
215 }
216 LeaveType originalLeaveType = requestDTO.getLeaveType();

```

```

217         leaveRequest.setLeaveType(requestDTO.getLeaveType());
218         if(requestDTO.getReason()!=null){
219             leaveRequest.setReason(requestDTO.getReason());
220         }
221         LeaveRequest savedRequest=leaveRequestRepo.save(leaveRequest);
222
223         Employee manager = employee.getManager(); // assuming getManager() exists
224         if (manager != null) {
225             notificationService.notify(
226                 manager,
227                 employee.getName() + " submitted a " + savedRequest.getLeaveType() ,
228                 "LEAVE_REQUEST"
229             );
230         }
231         // After the existing manager notification block
232         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
233             notificationService.notify(
234                 admin,
235                 employee.getName() + " submitted a " + requestDTO.getLeaveType() + " leave request
236                 (" + requestDTO.getStartDate() + " " + requestDTO.getEndDate() + ")",
237                 "LEAVE_REQUEST"
238             )
239         );
240         LeaveResponseDTO responseDTO = convertToDTO(savedRequest);
241         if (savedRequest.getLeaveType() == LeaveType.UNPAID && savedRequest.getLeaveType() !=
242         originalLeaveType) {
243             Map<String, Object> body = new java.util.LinkedHashMap<>();
244             body.put("leave", responseDTO);
245             body.put("warning", "Your " + originalLeaveType.name().toLowerCase() +
246                 " leave quota for this month is exhausted. Request recorded as Unpaid leave.");
247             return ResponseEntity.status(HttpStatus.CREATED).body(body);
248         }
249         return ResponseEntity.status(HttpStatus.CREATED).body(responseDTO);
250     }
251     //Only for manager
252     public List<LeaveResponseDTO> getAllThePendingLeaveRequests(){
253         List<LeaveRequest> requestList=leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
254         List<LeaveResponseDTO> responseDTOS=new ArrayList<>();
255         for(LeaveRequest request:requestList){
256             responseDTOS.add(convertToDTO(request));
257         }
258         return responseDTOS;
259     }
260
261     @Transactional
262     public ResponseEntity<?> updateLeaveRequestStatus(ActionDTO actionDTO,Authentication authentication){
263         String email= AuthUtil.extractEmail(authentication);
264         Employee manager=employeeRepo.findByEmail(email).orElseThrow(
265             ()->new EmployeeNotFound("Manager with name: "+email+" Not Found")
266         );
267         if (actionDTO.getAction() == null) {
268             return ResponseEntity.badRequest().body("Action is required");
269         }
270         log.info("Updating leave request status for id: {}", actionDTO.getLeaveRequestId());
271
272         var leaveRequest=leaveRequestRepo.findById(actionDTO.getLeaveRequestId()).orElseThrow(
273             ()-> new LeaveRequestNotFoundException("LeaveRequest with
274             id:"+actionDTO.getLeaveRequestId()+" not found")
275         );
276         if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
277             return ResponseEntity.badRequest()
278                 .body("Only PENDING requests can be approved or rejected. Current status: "
279                     + leaveRequest.getStatus());
280         }
281
282         if(actionDTO.getAction().equalsIgnoreCase("approved")){
283             leaveRequest.setStatus(LeaveStatus.APPROVED);
284             applyApprovedLeaveToEntitlement(leaveRequest);
285             notificationService.notify(
286                 leaveRequest.getEmployee(),
287                 "Your leave request from " + leaveRequest.getStartDate() + " to " +
288                 leaveRequest.getEndDate() + " has been approved.",
289                 "LEAVE_APPROVED"

```

```

289         );
290         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
291             notificationService.notify(
292                 admin,
293                 leaveRequest.getEmployee().getName() + "'s leave request has been approved by
" + manager.getName(),
294                 "LEAVE_APPROVED"
295             )
296         );
297     }
298     else if(actionDTO.getAction().equalsIgnoreCase("rejected")){
299         leaveRequest.setStatus(LeaveStatus.REJECTED);
300         notificationService.notify(
301             leaveRequest.getEmployee(),
302             "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " was rejected." +
303             (actionDTO.getRemarks() != null ? " Reason: " + actionDTO.getRemarks() : ""),
304             "LEAVE_REJECTED"
305         );
306         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
307             notificationService.notify(
308                 admin,
309                 leaveRequest.getEmployee().getName() + "'s leave request has been rejected by
" + manager.getName(),
310                 "LEAVE_REJECTED"
311             )
312         );
313     }
314     else{
315         return ResponseEntity.badRequest().body("Invalid action,Use APPROVED or REJECTED");
316     }
317     leaveRequest.setManager(manager.getEmail());
318     if(actionDTO.getRemarks()!=null){
319         leaveRequest.setRemarks(actionDTO.getRemarks());
320     }
321     return ResponseEntity.ok(leaveRequestRepo.save(leaveRequest));
322 }
323
324 @Transactional
325 public ResponseEntity<?> cancelLeaveRequest(Authentication authentication, long leaveId){
326     String email= AuthUtil.extractEmail(authentication);
327     var employee=employeeRepo.findByEmail(email).orElseThrow(
328         ()->new EmployeeNotFound("Employee with email:"+email+" not found")
329     );
330     var leaveRequest=leaveRequestRepo.findById(leaveId).orElseThrow(
331         ()->new LeaveRequestNotFoundException("Leave request with id:"+leaveId+" is not found")
332     );
333     if(employee.getEmployeeId()!=leaveRequest.getEmployee().getEmployeeId()){
334         return ResponseEntity.badRequest().body("You cannot cancel others Leave request..");
335     }
336     String timezone= employee.getTimezone();
337     ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
338         ? timezone
339         : "UTC");
340     LocalDate today = LocalDate.now(zoneId);
341     if(leaveRequest.getStartDate().isEqual(today)||leaveRequest.getStartDate().isBefore(today)){
342         return ResponseEntity.badRequest().body("You cannot cancel leave request after the leave has
started");
343     }
344     LeaveStatus previousStatus = leaveRequest.getStatus();
345     leaveRequest.setStatus(LeaveStatus.CANCELLED);
346     if (previousStatus == LeaveStatus.APPROVED) {
347         reverseApprovedLeaveFromEntitlement(leaveRequest);
348     }
349     leaveRequestRepo.save(leaveRequest);
350     return ResponseEntity.ok("Leave Request with id:"+ leaveId+" Successfully cancelled");
351 }
352
353 private LeaveResponseDTO convertToDTO(LeaveRequest request){
354     var responseDTO=new LeaveResponseDTO();
355     responseDTO.setLeaveRequestId(request.getId());
356     responseDTO.setEmployeeName(request.getEmployee().getName());
357     responseDTO.setLeaveType(request.getLeaveType());
358     responseDTO.setStartDate(request.getStartDate());
359     responseDTO.setEndDate(request.getEndDate());
360     responseDTO.setNumberOfDays(requestedDays(request).longValue());

```

```

361         responseDTO.setReason(request.getReason());
362         responseDTO.setStatus(request.getStatus());
363         return responseDTO;
364     }
365
366     public ResponseEntity<?> getLeaveRequestsByEmployee(Authentication authentication) {
367         String email= AuthUtil.extractEmail(authentication);
368         log.info("Fetching leave requests for employee: {}", email);
369
370         Employee employee = employeeRepo.findByEmail(email)
371             .orElseThrow(() -> new EmployeeNotFound("Employee not found: " +email ));
372
373         List<LeaveRequest> requests = leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
374         List<LeaveResponseDTO> response = requests.stream()
375             .map(this::convertToDTO)
376             .collect(Collectors.toList());
377
378         return ResponseEntity.ok(Map.of(
379             "employeeName", email,
380             "totalRequests", response.size(),
381             "requests", response
382         ));
383     }
384     public List<LeaveResponseDTO> getPendingLeaves(Authentication authentication) {
385         String email=AuthUtil.extractEmail(authentication);
386         Employee employee=employeeRepo.findByEmail(email).orElseThrow(
387             ()->new EmployeeNotFound("Employee not found")
388         );
389         List<LeaveRequest> pendingRequests =
390 leaveRequestRepo.findByStatusAndEmployeeNot(LeaveStatus.PENDING,employee);
391         return pendingRequests.stream()
392             .map(this::convertToDTO)
393             .collect(Collectors.toList());
394     }
395     /**
396      * Approve a leave request by ID (simplified version for AdminController).
397      * Returns LeaveResponseDTO directly.
398      */
399     @Transactional
400     public LeaveResponseDTO approveLeave(Long id) {
401         LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
402             .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not
403 found"));
404         if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
405             throw new IllegalStateException("Only PENDING requests can be approved. Current status: " +
406 leaveRequest.getStatus());
407         }
408         leaveRequest.setStatus(LeaveStatus.APPROVED);
409         leaveRequest.setManager("ADMIN"); // Or get from security context if needed
410         applyApprovedLeaveToEntitlement(leaveRequest);
411         notificationService.notify(
412             leaveRequest.getEmployee(),
413             "Your leave request from " + leaveRequest.getStartDate() + " to " +
414 leaveRequest.getEndDate() + " has been approved by admin.",
415             "LEAVE_APPROVED"
416         );
417         LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
418         return convertToDTO(saved);
419     }
420
421     /**
422      * Reject a leave request by ID (simplified version for AdminController).
423      * Returns LeaveResponseDTO directly.
424      */
425     @Transactional
426     public LeaveResponseDTO rejectLeave(Long id) {
427         LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
428             .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not
429 found"));
430         if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
431             throw new IllegalStateException("Only PENDING requests can be rejected. Current status: " +

```

```

leaveRequest.getStatus());
432     }
433
434     leaveRequest.setStatus(LeaveStatus.REJECTED);
435     leaveRequest.setManager("ADMIN");
436     notificationService.notify(
437         leaveRequest.getEmployee(),
438         "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " was rejected by admin.",
439         "LEAVE_REJECTED"
440     );
441
442     LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
443     return convertToDTO(saved);
444 }
445
446 private void applyApprovedLeaveToEntitlement(LeaveRequest leaveRequest) {
447     if (leaveRequest.getLeaveType() == LeaveType.MATERNITY) {
448         grantAndDeductMaternity(leaveRequest); // special maternity flow
449     } else {
450         ensureSufficientBalanceForApproval(leaveRequest);
451         updateUsedLeave(leaveRequest, requestedDays(leaveRequest)); // existing flow
452     }
453 }
454
455 private void grantAndDeductMaternity(LeaveRequest leaveRequest) {
456     int year = leaveRequest.getStartDate().getYear();
457     Employee employee = leaveRequest.getEmployee();
458
459     com.example.EmployeeManagementSystem.Entity.LeaveType maternityType =
460         leaveTypeRepository.findByName("MATERNITY")
461             .orElseThrow(() -> new IllegalStateException(
462                 "Maternity leave type not configured"));
463
464     var entitlement = leaveEntitlementRepository
465         .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
466             employee.getEmployeeId(), maternityType.getId(), year)
467         .orElseGet(() -> createEntitlement(employee, maternityType, year));
468
469     // Grant 180 days one time on approval
470     entitlement.setAccruedThisYear(BigDecimal.valueOf(180));
471
472     // Deduct the days she is taking (always 180)
473     entitlement.setUsedThisYear(requestedDays(leaveRequest));
474
475     leaveEntitlementRepository.save(entitlement);
476 }
477
478 private void reverseApprovedLeaveFromEntitlement(LeaveRequest leaveRequest) {
479     if (leaveRequest.getLeaveType() == LeaveType.MATERNITY) {
480         reverseMaternityEntitlement(leaveRequest); // wipe the grant entirely
481     } else {
482         updateUsedLeave(leaveRequest, requestedDays(leaveRequest).negate());
483     }
484 }
485
486 private void reverseMaternityEntitlement(LeaveRequest leaveRequest) {
487     int year = leaveRequest.getStartDate().getYear();
488     Employee employee = leaveRequest.getEmployee();
489
490     com.example.EmployeeManagementSystem.Entity.LeaveType maternityType =
491         leaveTypeRepository.findByName("MATERNITY")
492             .orElseThrow(() -> new IllegalStateException(
493                 "Maternity leave type not configured"));
494
495     leaveEntitlementRepository
496         .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
497             employee.getEmployeeId(), maternityType.getId(), year)
498         .ifPresent(entitlement -> {
499             // Wipe both the grant and the usage
500             entitlement.setAccruedThisYear(BigDecimal.ZERO);
501             entitlement.setUsedThisYear(BigDecimal.ZERO);
502             leaveEntitlementRepository.save(entitlement);
503         });
504 }
505

```

```

506     private void updateUsedLeave(LeaveRequest leaveRequest, BigDecimal delta) {
507         int year = leaveRequest.getStartDate().getYear();
508         String leaveTypeName = leaveRequest.getLeaveType().name();
509         var leaveType = leaveTypeRepository
510             .findByName(leaveTypeName)
511             .orElseThrow(() -> new IllegalStateException(
512                 "Leave type not configured: " + leaveTypeName));
513
514         var entitlement = leaveEntitlementRepository
515             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
516                 leaveRequest.getEmployee().getEmployeeId(), leaveType.getId(), year)
517             .orElseGet(() -> createEntitlement(leaveRequest.getEmployee(), leaveType, year));
518
519         BigDecimal newUsed = entitlement.getUsedThisYear().add(delta);
520         // Prevent negative
521         newUsed = newUsed.max(BigDecimal.ZERO);
522
523         // Cap used days to the actual entitled total for the year
524         BigDecimal totalEntitled = entitlement.getOpeningBalance().add(entitlement.getAccruedThisYear());
525         if (newUsed.compareTo(totalEntitled) > 0) {
526             newUsed = totalEntitled;
527         }
528         entitlement.setUsedThisYear(newUsed);
529         // Keep closingBalance in sync so it never shows stale/negative values
530         entitlement.setClosingBalance(
531             entitlement.getOpeningBalance()
532                 .add(entitlement.getAccruedThisYear())
533                 .subtract(newUsed)
534         );
535     }
536
537     private void ensureSufficientBalanceForApproval(LeaveRequest leaveRequest) {
538         int year = leaveRequest.getStartDate().getYear();
539         String leaveTypeName = leaveRequest.getLeaveType().name();
540         var leaveType = leaveTypeRepository
541             .findByName(leaveTypeName)
542             .orElseThrow(() -> new IllegalStateException(
543                 "Leave type not configured: " + leaveTypeName));
544
545         var entitlement = leaveEntitlementRepository
546             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
547                 leaveRequest.getEmployee().getEmployeeId(), leaveType.getId(), year)
548             .orElseGet(() -> createEntitlement(leaveRequest.getEmployee(), leaveType, year));
549
550         BigDecimal availableBalance = entitlement.getAvailableBalance();
551         BigDecimal requestedDays = requestedDays(leaveRequest);
552
553         if (requestedDays.compareTo(availableBalance) > 0) {
554             throw new IllegalStateException(
555                 "Insufficient " + leaveTypeName.toLowerCase() + " leave balance for approval. " +
556                 "Available: " + availableBalance + ", requested: " + requestedDays);
557         }
558     }
559
560     private com.example.EmployeeManagementSystem.Entity.LeaveEntitlement createEntitlement(
561         Employee employee,
562         com.example.EmployeeManagementSystem.Entity.LeaveType leaveType,
563         int year) {
564         var entitlement = new com.example.EmployeeManagementSystem.Entity.LeaveEntitlement();
565         entitlement.setEmployee(employee);
566         entitlement.setLeaveType(leaveType);
567         entitlement.setYear(year);
568
569         if (leaveType.isCarriesForward()) {
570             BigDecimal lastYearClosing = leaveEntitlementRepository
571                 .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
572                     employee.getEmployeeId(), leaveType.getId(), year - 1)
573                 .map(com.example.EmployeeManagementSystem.Entity.LeaveEntitlement::getClosingBalance)
574                 .orElse(BigDecimal.ZERO);
575             entitlement.setOpeningBalance(lastYearClosing);
576         }
577
578         return leaveEntitlementRepository.save(entitlement);
579     }
580
581     private BigDecimal requestedDays(LeaveRequest leaveRequest) {

```

```
582         long days = ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1;
583         return BigDecimal.valueOf(days);
584     }
585     private long getCasualBalance(Employee employee, int year) {
586         return leaveTypeRepository.findByName("CASUAL")
587             .flatMap(lt -> leaveEntitlementRepository
588                 .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
589                     employee.getEmployeeId(), lt.getId(), year))
590             .map(e -> e.getAvailableBalance().longValue())
591             .orElse(0L);
592     }
593 }
594
```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Device;
5  import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
6  import com.example.EmployeeManagementSystem.Entity.Employee;
7  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
8  import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
9  import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
10 import com.example.EmployeeManagementSystem.Enum.Role;
11 import com.example.EmployeeManagementSystem.Enum.Status;
12 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
13 import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
14 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
15 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
16 import com.example.EmployeeManagementSystem.Util.AuthUtil;
17 import org.springframework.http.ResponseEntity;
18 import org.springframework.security.core.Authentication;
19 import org.springframework.security.crypto.password.PasswordEncoder;
20 import org.springframework.stereotype.Service;
21 import org.springframework.transaction.annotation.Transactional;
22
23 import java.time.temporal.ChronoUnit;
24 import java.util.List;
25 import java.util.stream.Collectors;
26
27 @Service
28 public class ManagerService {
29     private final EmployeeRepo employeeRepo;
30     private final PasswordEncoder passwordEncoder;
31     private final LeaveRequestRepo leaveRequestRepo;
32     private final DeviceAssignmentRepo deviceAssignmentRepo;
33     private final LeaveAccrualService leaveAccrualService;
34
35     public ManagerService(EmployeeRepo employeeRepo,
36                           PasswordEncoder passwordEncoder,
37                           LeaveRequestRepo leaveRequestRepo,
38                           DeviceAssignmentRepo deviceAssignmentRepo,
39                           LeaveAccrualService leaveAccrualService) {
40         this.employeeRepo = employeeRepo;
41         this.passwordEncoder = passwordEncoder;
42         this.leaveRequestRepo = leaveRequestRepo;
43         this.deviceAssignmentRepo = deviceAssignmentRepo;
44         this.leaveAccrualService = leaveAccrualService;
45     }
46
47     @Transactional
48     public Employee createEmp(EmployeeDetails employeeDetails, Authentication authentication) {
49         String email= AuthUtil.extractEmail(authentication);
50         Employee manager= employeeRepo.findByEmail(email).orElseThrow(
51             ()->new EmployeeNotFound("Manager :"+email+"Not found")
52         );
53         Employee employee=new Employee();
54         employee.setName(employeeDetails.getName());
55         employee.setEmail(employeeDetails.getEmail());
56         employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
57         employee.setGender(employeeDetails.getGender());
58         employee.setDept(manager.getDept());
59         employee.setRole(Role.EMPLOYEE);
60         employee.setManager(manager);
61         employee.setTimezone(employeeDetails.getTimezone());
62         Employee savedEmployee = employeeRepo.save(employee);
63         leaveAccrualService.grantInitialMonthlyAccrual(savedEmployee, savedEmployee.getJoined_at());
64         return savedEmployee;
65     }
66
67
68     public List<Employee> getAllEmployeeByManager(Authentication authentication) {
69         String email=AuthUtil.extractEmail(authentication);
70         Employee manager=employeeRepo.findByEmail(email).orElseThrow(
71             ()->new EmployeeNotFound("Manger:"+email+" not found")

```



```

72     );
73     return employeeRepo.findByManager(manager);
74 }
75
76 public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication) {
77     String email=AuthUtil.extractEmail(authentication);
78     Employee manager=employeeRepo.findByEmail(email).orElseThrow(
79         ()->new EmployeeNotFound("Manager with "+email+" not found")
80     );
81     List<LeaveRequest> requests=leaveRequestRepo.findByEmployee_Manager(manager);
82     return requests.stream().map(request->toManagerLeaveResponseDTO(request)).toList();
83 }
84
85 public List<EmployeeDTO> getUsers(){
86     List<Employee> employeeList=employeeRepo.findByRole(Role.USER);
87     return employeeList.stream().map(this::toDTO).toList();
88 }
89
90 public ResponseEntity<?> promoteUser(Authentication authentication, PromoteRequest promoteRequest){
91     Employee user=employeeRepo.findByEmail(promoteRequest.getEmail()).orElseThrow(
92         ()->new EmployeeNotFound("Employee:"+promoteRequest.getEmail()+" not found")
93     );
94     if(user.getRole()!=Role.USER){
95         return ResponseEntity.ok("Employee already promoted");
96     }
97     String managerEmail=AuthUtil.extractEmail(authentication);
98     Employee manager=employeeRepo.findByEmail(managerEmail).orElseThrow(
99         ()->new EmployeeNotFound("Manager:"+managerEmail+" not found")
100    );
101
102    user.setRole(Role.EMPLOYEE);
103    user.setManager(manager);
104    user.setDept(manager.getDept());
105    user.setPassword(passwordEncoder.encode(promoteRequest.getPassword()));
106    Employee promotedEmployee=employeeRepo.save(user);
107    return ResponseEntity.ok(toDTO(promotedEmployee));
108 }
109
110 public EmployeeDTO toDTO(Employee employee){
111     EmployeeDTO employeeDTO=new EmployeeDTO();
112     employeeDTO.setName(employee.getName());
113     employeeDTO.setEmail(employee.getEmail());
114     employeeDTO.setDept(employee.getDept());
115     employeeDTO.setPassword(employee.getPassword());
116     employeeDTO.setTimezone(employee.getTimezone());
117     return employeeDTO;
118 }
119
120 @Transactional
121 public Employee createAdmin(EmployeeDetails employeeDetails) {
122     Employee employee=new Employee();
123     employee.setName(employeeDetails.getName());
124     employee.setEmail(employeeDetails.getEmail());
125     employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
126     employee.setDept("Main");
127     employee.setRole(Role.ADMIN);
128     employee.setGender(employeeDetails.getGender());
129     employee.setTimezone(employeeDetails.getTimezone());
130     Employee savedEmployee = employeeRepo.save(employee);
131     leaveAccrualService.grantInitialMonthlyAccrual(savedEmployee, savedEmployee.getJoined_at());
132     return savedEmployee;
133 }
134
135 public List<LeaveResponseDTO> getAllThePendingLeaveRequests(Authentication authentication) {
136     String email=AuthUtil.extractEmail(authentication);
137     Employee manager=employeeRepo.findByEmail(email).orElseThrow(
138         ()->new EmployeeNotFound("Manager with "+email+" not found")
139     );
140     List<LeaveRequest>
leaveRequests=leaveRequestRepo.findByStatusAndEmployee_Manager(LeaveStatus.PENDING,manager);
141     return leaveRequests.stream()
142         .map(this::convertToLeaveResponseDTO)
143         .collect(Collectors.toList());
144 }
145
146 public ManagerEmployeeDetailsDTO getEmployeeDetailsByName( String name) {

```

```

147
148     Employee employee = employeeRepo.findByName(name).orElseThrow(
149         () -> new EmployeeNotFound("Employee:" + name + " not found")
150     );
151
152
153
154     ManagerEmployeeDetailsDTO dto = new ManagerEmployeeDetailsDTO();
155     dto.setEmployeeId(employee.getEmployeeId());
156     dto.setName(employee.getName());
157     dto.setEmail(employee.getEmail());
158     dto.setDept(employee.getDept());
159     dto.setStatus(employee.getStatus());
160     dto.setAttendance(toAttendance(employee.getStatus()));
161     dto.setDevices(deviceAssignmentRepo
162         .findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(), AssignmentStatus.ACTIVE)
163         .stream()
164         .map(DeviceAssignment::getDevice)
165         .map(this::toDeviceResponseDTO)
166         .toList());
167     return dto;
168 }
169
170 private LeaveResponseDTO convertToLeaveResponseDTO(LeaveRequest leaveRequest) {
171     LeaveResponseDTO dto = new LeaveResponseDTO();
172     dto.setLeaveRequestId(leaveRequest.getId());
173     dto.setEmployeeName(leaveRequest.getEmployee().getName());
174     dto.setLeaveType(leaveRequest.getLeaveType());
175     dto.setStartDate(leaveRequest.getStartDate());
176     dto.setEndDate(leaveRequest.getEndDate());
177     dto.setReason(leaveRequest.getReason());
178     dto.setStatus(leaveRequest.getStatus());
179     return dto;
180 }
181
182 private ManagerLeaveResponseDTO toManagerLeaveResponseDTO(LeaveRequest leaveRequest){
183     ManagerLeaveResponseDTO managerLeaveResponseDTO=new ManagerLeaveResponseDTO();
184     managerLeaveResponseDTO.setEmployeeName(leaveRequest.getEmployee().getName());
185     managerLeaveResponseDTO.setLeaveType(leaveRequest.getLeaveType());
186     managerLeaveResponseDTO.setStartDate(leaveRequest.getStartDate());
187     managerLeaveResponseDTO.setEndDate(leaveRequest.getEndDate());
188     managerLeaveResponseDTO.setNumberOfDays(
189         ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1);
190     managerLeaveResponseDTO.setStatus(leaveRequest.getStatus());
191     managerLeaveResponseDTO.setReason(leaveRequest.getReason());
192     return managerLeaveResponseDTO;
193 }
194
195 private String toAttendance(Status status) {
196     if (status == Status.ON_LEAVE) {
197         return "ON_LEAVE";
198     }
199     if (status == Status.ACTIVE) {
200         return "PRESENT";
201     }
202     return status.name();
203 }
204
205 private DeviceResponseDTO toDeviceResponseDTO(Device device) {
206     DeviceResponseDTO dto = new DeviceResponseDTO();
207     dto.setId(device.getId());
208     dto.setDeviceName(device.getDeviceName());
209     dto.setBrand(device.getBrand());
210     dto.setDeviceType(device.getDeviceType());
211     dto.setDeviceStatus(device.getDeviceStatus());
212     dto.setWarrantyExpiryDate(device.getWarrantyExpiryDate());
213     dto.setRentPerMonth(device.getRentPerMonth());
214
215     if (device.getTechVendor() != null) {
216         dto.setVendorName(device.getTechVendor().getName());
217     }
218
219     DeviceAssignment assignment = device.getCurrentAssignment();
220     if (assignment != null && assignment.getStatus() == AssignmentStatus.ACTIVE) {
221         dto.setAssignedDate(assignment.getAssignedDate());
222         if (assignment.getAssignedTo() != null) {

```

```
223         dto.setAssignedEmployeeId(assignment.getAssignedTo().getEmployeeId());
224         dto.setAssignedEmployeeName(assignment.getAssignedTo().getName());
225     }
226 }
227
228     return dto;
229 }
230 }
231
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
4 import com.example.EmployeeManagementSystem.DTO.MaternityLeaveRequestDTO;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
7 import com.example.EmployeeManagementSystem.Enum.*;
8 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
9 import com.example.EmployeeManagementSystem.Exception.InvalidStartDateException;
10 import com.example.EmployeeManagementSystem.Exception.OverlappingLeaveException;
11 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
12 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
13 import com.example.EmployeeManagementSystem.Util.AuthUtil;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.stereotype.Service;
18
19 import java.math.BigDecimal;
20 import java.time.LocalDate;
21 import java.time.temporal.ChronoUnit;
22 import java.util.List;
23 import java.util.Map;
24
25 @Service
26 public class MaternityLeaveService {
27
28     private final EmployeeRepo employeeRepo;
29     private final LeaveRequestRepo leaveRequestRepo;
30     private final NotificationService notificationService;
31
32     public MaternityLeaveService(EmployeeRepo employeeRepo, LeaveRequestRepo leaveRequestRepo,
33 NotificationService notificationService) {
34         this.employeeRepo = employeeRepo;
35         this.leaveRequestRepo = leaveRequestRepo;
36         this.notificationService = notificationService;
37     }
38
39     public ResponseEntity<?> applyMaternityLeave(Authentication authentication,
40 MaternityLeaveRequestDTO maternityLeaveRequestDTO){
41         String email = AuthUtil.extractEmail(authentication);
42         Employee employee = employeeRepo.findByEmail(email)
43             .orElseThrow(() -> new EmployeeNotFound("Employee not found: " + email));
44
45         if (employee.getStatus() != Status.ACTIVE) {
46             return ResponseEntity.badRequest()
47                 .body(Map.of("error", "Only active employees can apply"));
48         }
49
50         if (employee.getGender() != Gender.F) {
51             return ResponseEntity.badRequest()
52                 .body(Map.of("error", "Only female employees can apply for maternity leave"));
53         }
54
55         boolean alreadyApplied = leaveRequestRepo
56             .existsMaternityLeaveForYear(
57                 employee,
58                 LeaveType.MATERNITY,
59                 maternityLeaveRequestDTO.getStartDate().getYear(),
60                 List.of(LeaveStatus.CANCELLED, LeaveStatus.REJECTED));
61         if (alreadyApplied) {
62             return ResponseEntity.badRequest()
63                 .body(Map.of("error", "Maternity leave already applied for this year"));
64         }
65
66         LocalDate today = LocalDate.now();
67         if (maternityLeaveRequestDTO.getStartDate().isBefore(today)) {
68             throw new InvalidStartDateException("Start date cannot be before today");
69         }
70     }

```

```

71
72     // 5. Overlapping check
73     LocalDate endDate = maternityLeaveRequestDTO.getStartDate().plusDays(179); // always 180 days
74     long overlapping = leaveRequestRepo.countOverlappingLeave(
75         employee.getId(),
76         maternityLeaveRequestDTO.getStartDate(),
77         endDate);
78     if (overlapping > 0) {
79         throw new OverlappingLeaveException("Overlapping leave exists");
80     }
81
82     LeaveRequest request = new LeaveRequest();
83     request.setEmployee(employee);
84     request.setLeaveType(LeaveType.MATERNITY);
85     request.setStartDate(maternityLeaveRequestDTO.getStartDate());
86     request.setEndDate(endDate); // forced 180 days
87     request.setStatus(LeaveStatus.PENDING);
88     request.setReason(maternityLeaveRequestDTO.getReason());
89
90     LeaveRequest saved = leaveRequestRepo.save(request);
91     Employee manager = employee.getManager();
92     if (manager != null) {
93         notificationService.notify(
94             manager,
95             employee.getName() + " has applied for maternity leave.",
96             "MATERNITY_LEAVE_REQUEST"
97         );
98     }
99     employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
100         notificationService.notify(
101             admin,
102             employee.getName() + " applied for maternity leave (" +
103                 maternityLeaveRequestDTO.getStartDate() + " " + endDate + ")",
104             "MATERNITY_LEAVE_REQUEST"
105         )
106     );
107
108     return ResponseEntity.status(HttpStatus.CREATED).body(convertToDTO(saved));
109 }
110
111 private LeaveResponseDTO convertToDTO(LeaveRequest request){
112     var responseDTO=new LeaveResponseDTO();
113     responseDTO.setLeaveRequestId(request.getId());
114     responseDTO.setEmployeeName(request.getEmployee().getName());
115     responseDTO.setLeaveType(request.getLeaveType());
116     responseDTO.setStartDate(request.getStartDate());
117     responseDTO.setEndDate(request.getEndDate());
118     responseDTO.setNumberOfDays(requestedDays(request).longValue());
119     responseDTO.setReason(request.getReason());
120     responseDTO.setStatus(request.getStatus());
121     return responseDTO;
122 }
123
124 private BigDecimal requestedDays(LeaveRequest leaveRequest) {
125     long days = ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1;
126     return BigDecimal.valueOf(days);
127 }
128 }
129

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import org.springframework.stereotype.Service;
5
6 import java.time.LocalDateTime;
7 import java.util.Map;
8
9 @Service
10 public class MealSlotService {
11
12     private final Map<MealSlot, LocalDateTime> slotLocalTimeMap = Map.of(
13         MealSlot.BREAKFAST, LocalDateTime.of(8, 0),
14         MealSlot.LUNCH, LocalDateTime.of(13, 0),
15         MealSlot.DINNER, LocalDateTime.of(21, 0)
16     );
17
18     public LocalDateTime getTime(MealSlot slot) {
19         return slotLocalTimeMap.get(slot);
20     }
21 }
22
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Notification;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.NotificationRepository;
7 import org.springframework.stereotype.Service;
8 import java.util.List;
9
10 @Service
11 public class NotificationService {
12
13     private final NotificationRepository notificationRepo;
14     private final EmployeeRepo employeeRepo;
15
16     public NotificationService(NotificationRepository notificationRepo,
17                               EmployeeRepo employeeRepo) {
18         this.notificationRepo = notificationRepo;
19         this.employeeRepo = employeeRepo;
20     }
21
22     /** Create a notification for a specific employee */
23     public void notify(Employee recipient, String message, String type) {
24         Notification n = new Notification();
25         n.setRecipient(recipient);
26         n.setMessage(message);
27         n.setType(type);
28         notificationRepo.save(n);
29     }
30
31     /** Notify every employee in the system (e.g. new public holiday) */
32     public void notifyAll(String message, String type) {
33         List<Employee> all = employeeRepo.findAll();
34         for (Employee e : all) {
35             notify(e, message, type);
36         }
37     }
38
39     public List<Notification> getForEmployee(Employee employee) {
40         return notificationRepo.findByRecipientOrderByCreatedAtDesc(employee);
41     }
42
43     public long getUnreadCount(Employee employee) {
44         return notificationRepo.countByRecipientAndIsReadFalse(employee);
45     }
46
47     public void markAllRead(Employee employee) {
48         List<Notification> unread = notificationRepo
49             .findByRecipientOrderByCreatedAtDesc(employee)
50             .stream()
51             .filter(n -> !n.isRead())
52             .toList();
53         unread.forEach(n -> n.setRead(true));
54         notificationRepo.saveAll(unread);
55     }
56 }
57
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
8 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
9 import org.springframework.stereotype.Service;
10
11 import java.time.Instant;
12 import java.time.temporal.ChronoUnit;
13 import java.util.UUID;
14
15 @Service
16 public class RefreshTokenService {
17     private final EmployeeRepo employeeRepo;
18     private final VendorRepo vendorRepo;
19
20     public RefreshTokenService(RefreshTokenRepository refreshTokenRepository, EmployeeRepo employeeRepo,
21     VendorRepo vendorRepo) {
22         this.employeeRepo = employeeRepo;
23         this.vendorRepo = vendorRepo;
24     }
25
26     public RefreshToken createForEmployee(Employee employee) {
27         RefreshToken refreshToken = new RefreshToken();
28         refreshToken.setToken(UUID.randomUUID().toString());
29         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
30         refreshToken.setEmployee(employee);
31         return refreshToken;
32     }
33
34     public RefreshToken createForVendor(Vendor vendor) {
35         RefreshToken refreshToken = new RefreshToken();
36         refreshToken.setToken(UUID.randomUUID().toString());
37         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
38         refreshToken.setVendor(vendor);
39         return refreshToken;
40     }
41 }
```



```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
4 import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
5 import com.example.EmployeeManagementSystem.Entity.Restaurant;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.MealsSlot;
8 import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
9 import com.example.EmployeeManagementSystem.Exception.UnauthorizedAccessException;
10 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
11 import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
12 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
13 import com.example.EmployeeManagementSystem.Util.AuthUtil;
14 import org.slf4j.Logger;
15 import org.slf4j.LoggerFactory;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.stereotype.Service;
18
19 import java.util.List;
20 import java.util.stream.Collectors;
21
22 @Service
23 public class RestaurantService {
24
25     private static final Logger log = LoggerFactory.getLogger(RestaurantService.class);
26
27     private final RestaurantRepository restaurantRepository;
28     private final VendorRepo vendorRepo;
29
30     public RestaurantService(RestaurantRepository restaurantRepository, VendorRepo vendorRepo) {
31         this.restaurantRepository = restaurantRepository;
32         this.vendorRepo = vendorRepo;
33     }
34
35     /**
36      * Create a new restaurant.
37      * Only the owning vendor (matched by vendorId in the request) may do this.
38      */
39     public RestaurantDTO createRestaurant(RestaurantRequest request, Authentication authentication) {
40         String email = AuthUtil.extractEmail(authentication);
41         Vendor vendor = vendorRepo.findByEmail(email)
42             .orElseThrow(() -> new VendorNotFoundException(
43                 "Vendor with email " + email + " not found"));
44
45         Restaurant restaurant = new Restaurant();
46         restaurant.setName(request.getName());
47         restaurant.setAddress(request.getAddress());
48         restaurant.setDescription(request.getDescription());
49         restaurant.setSupportedMealsSlots(request.getSupportedMealsSlots());
50         restaurant.setVendor(vendor);
51         restaurant.setActive(true);
52
53         Restaurant saved = restaurantRepository.save(restaurant);
54         log.info("Restaurant created: {} by vendor {}", saved.getName(), vendor.getId());
55         return toDTO(saved);
56     }
57
58     /**
59      * Update restaurant details.
60      * Vendor must own this restaurant – otherwise 403.
61      */
62     public RestaurantDTO updateRestaurant(Long restaurantId, RestaurantRequest request, Authentication authentication) {
63         Restaurant restaurant = findOrThrow(restaurantId);
64         String email = AuthUtil.extractEmail(authentication);
65         Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
66             () -> new VendorNotFoundException("Vendor Not found")
67         );
68         assertOwnership(restaurantId, vendor.getEmail());
69
70         if (request.getName() != null) restaurant.setName(request.getName());

```

```

71         if (request.getAddress() != null) restaurant.setAddress(request.getAddress());
72         if (request.getDescription() != null) restaurant.setDescription(request.getDescription());
73         if (request.getSupportedMealSlots() != null && !request.getSupportedMealSlots().isEmpty()) {
74             restaurant.setSupportedMealSlots(request.getSupportedMealSlots());
75         }
76         return toDTO(restaurantRepository.save(restaurant));
77     }
78
79     /**
80      * Deactivate a restaurant (soft delete).
81      * Only the owning vendor may do this.
82      * Active subscriptions to this restaurant will no longer receive new deliveries
83      * – the Subscription status should be handled separately.
84      */
85     public void deactivateRestaurant(Long restaurantId, Authentication authentication) {
86         Restaurant restaurant = findOrThrow(restaurantId);
87         String email= AuthUtil.extractEmail(authentication);
88         Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
89             ()->new VendorNotFoundException("Vendor Not found")
90         );
91         assertOwnership(restaurantId, vendor.getEmail());
92         restaurant.setActive(false);
93         restaurantRepository.save(restaurant);
94         log.info("Restaurant deactivated: id={}", restaurantId);
95     }
96
97     /**
98      * Get all restaurants owned by a vendor.
99      * VENDOR-only endpoint.
100     */
101     public List<RestaurantDTO> getRestaurantsByVendor(Authentication authentication) {
102         String email= AuthUtil.extractEmail(authentication);
103         vendorRepo.findByEmail(email)
104             .orElseThrow(() -> new VendorNotFoundException("Vendor " + email + " not found"));
105         return restaurantRepository.findByVendor_EmailAndActiveTrue(email)
106             .stream().map(this::toDTO).collect(Collectors.toList());
107     }
108
109     /**
110      * Get all active restaurants.
111      * Available to EMPLOYEE so they can browse and choose.
112      */
113     public List<RestaurantDTO> getAllActiveRestaurants() {
114         return restaurantRepository.findByActiveTrue()
115             .stream().map(this::toDTO).collect(Collectors.toList());
116     }
117
118     /**
119      * Get active restaurants that serve a specific meal slot.
120      * Employees call this when subscribing, to see valid restaurant options for a slot.
121      */
122     public List<RestaurantDTO> getRestaurantsByMealSlot(MealSlot mealSlot) {
123         return restaurantRepository.findActiveByMealSlot(mealSlot)
124             .stream().map(this::toDTO).collect(Collectors.toList());
125     }
126
127     public RestaurantDTO getRestaurant(Long id) {
128         return toDTO(findOrThrow(id));
129     }
130
131     // helpers
132
133     private Restaurant findOrThrow(Long id) {
134         return restaurantRepository.findById(id)
135             .orElseThrow(() -> new RestaurantNotFoundException(
136                 "Restaurant with id " + id + " not found"));
137     }
138
139     /**
140      * Throws UnauthorizedAccessException if the restaurant does not belong to the vendor.
141      * This is the core access-control guard for all mutating vendor operations.
142      */
143     private void assertOwnership(Long restaurantId, String email) {
144         if (!restaurantRepository.existsByIdAndVendor_Email(restaurantId, email)) {
145             throw new UnauthorizedAccessException(
146                 "Vendor " + email + " does not own restaurant " + restaurantId);

```

```
147     }
148 }
149
150 public RestaurantDTO toDTO(Restaurant r) {
151     RestaurantDTO dto = new RestaurantDTO();
152     dto.setId(r.getId());
153     dto.setName(r.getName());
154     dto.setAddress(r.getAddress());
155     dto.setDescription(r.getDescription());
156     dto.setSupportedMealSlots(r.getSupportedMealSlots());
157     dto.setActive(r.isActive());
158     dto.setVendorId(r.getVendor().getId());
159     dto.setVendorName(r.getVendor().getName());
160     return dto;
161 }
162 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.*;
4 import com.example.EmployeeManagementSystem.Entity.*;
5 import com.example.EmployeeManagementSystem.Enum.*;
6 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
7 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
8 import com.example.EmployeeManagementSystem.Repository.*;
9 import com.example.EmployeeManagementSystem.Util.AuthUtil;
10 import org.jspecify.annotations.Nullable;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.stereotype.Service;
14 import org.springframework.transaction.annotation.Transactional;
15
16 import java.math.BigDecimal;
17 import java.time.LocalDate;
18 import java.time.LocalDateTime;
19 import java.util.List;
20
21 @Service
22 public class ServiceRequestService {
23     private final ServiceRequestRepository serviceRequestRepository;
24     private final EmployeeRepo employeeRepo;
25     private final DeviceRepository deviceRepository;
26     private final RepairLogRepository repairLogRepository;
27     private final VendorRepo vendorRepo;
28     private final RepairBillRepository repairBillRepository;
29     private final DeviceAssignmentRepo deviceAssignmentRepo;
30     private final NotificationService notificationService;
31
32     public ServiceRequestService(ServiceRequestRepository serviceRequestRepository, EmployeeRepo
employeeRepo, DeviceRepository deviceRepository, RepairLogRepository repairLogRepository, VendorRepo...
33         this.serviceRequestRepository = serviceRequestRepository;
34         this.employeeRepo = employeeRepo;
35         this.deviceRepository = deviceRepository;
36         this.repairLogRepository = repairLogRepository;
37         this.vendorRepo = vendorRepo;
38         this.repairBillRepository = repairBillRepository;
39         this.deviceAssignmentRepo = deviceAssignmentRepo;
40         this.notificationService = notificationService;
41     }
42
43     public ServiceRequest createServiceRequest(ServiceRequestDTO dto, Authentication authentication) {
44         String email= AuthUtil.extractEmail(authentication);
45         Employee employee = employeeRepo.findByEmail(email)
46             .orElseThrow(() -> new RuntimeException("Employee not found"));
47
48         // 2. Fetch device
49         Device device = deviceRepository.findById(dto.getDeviceId())
50             .orElseThrow(() -> new RuntimeException("Device not found"));
51
52         // 3. Validate device is assigned to employee
53         if (device.getCurrentAssignment() == null ||
54             !device.getCurrentAssignment().getAssignedTo().equals(employee)) {
55             throw new RuntimeException("Device not assigned to this employee");
56         }
57
58         // 4. Validate device status
59         if (device.getDeviceStatus() != DeviceStatus.ASSIGNED) {
60             throw new RuntimeException("Device is not in usable state");
61         }
62
63         // 5. Create Service Request
64         ServiceRequest request = new ServiceRequest();
65         request.setDevice(device);
66         request.setRaisedBy(employee);
67         request.setRequestType(dto.getRequestType());
68         request.setIssueDescription(dto.getIssueDescription());
69         request.setUrgent(dto.isUrgent());
70         request.setStatus(ServiceRequestStatus.OPEN);

```

```

71         return serviceRequestRepository.save(request);
72     }
73
74
75     public List<ServiceRequestResponseDTO> getAllOpenServiceRequests(){
76         List<ServiceRequest>
serviceRequestList=serviceRequestRepository.findByStatus(ServiceRequestStatus.OPEN);
77         return serviceRequestList.stream().map(serviceRequest ->
toServiceRequestResponseDTO(serviceRequest)).toList();
78
79     }
80
81     public List<ServiceRequestResponseDTO> getMyServiceRequests(Authentication authentication) {
82         String email = AuthUtil.extractEmail(authentication);
83         Employee employee = employeeRepo.findByEmail(email)
84             .orElseThrow(() -> new EmployeeNotFound("Employee not found: " + email));
85
86         List<ServiceRequest> serviceRequestList =
serviceRequestRepository.findByRaisedByAndStatusNotOrderByRaisedAtDesc(employee, ServiceRequestStatus.CLOSED);
87         return serviceRequestList.stream()
88             .map(this::toServiceRequestResponseDTO)
89             .toList();
90     }
91
92     private ServiceRequestDTO toServiceRequestDTO(ServiceRequest serviceRequest){
93         ServiceRequestDTO requestDTO=new ServiceRequestDTO();
94         requestDTO.setDeviceId(serviceRequest.getDevice().getId());
95         requestDTO.setRequestType(serviceRequest.getRequestType());
96         requestDTO.setIssueDescription(serviceRequest.getIssueDescription());
97         requestDTO.setUrgent(serviceRequest.isUrgent());
98         return requestDTO;
99     }
100
101     private ServiceRequestResponseDTO toServiceRequestResponseDTO(ServiceRequest serviceRequest) {
102         ServiceRequestResponseDTO responseDTO = new ServiceRequestResponseDTO();
103         responseDTO.setId(serviceRequest.getId());
104         responseDTO.setRequestType(serviceRequest.getRequestType() != null
105             ? serviceRequest.getRequestType().name()
106             : null);
107         responseDTO.setStatus(serviceRequest.getStatus() != null
108             ? serviceRequest.getStatus().name()
109             : null);
110         responseDTO.setIssueDescription(serviceRequest.getIssueDescription());
111         responseDTO.setAdminRemarks(serviceRequest.getAdminRemarks());
112         responseDTO.setResolution(serviceRequest.getResolution());
113         responseDTO.setUrgent(serviceRequest.isUrgent());
114         responseDTO.setRaisedAt(serviceRequest.getRaisedAt());
115         responseDTO.setResolvedAt(serviceRequest.getResolvedAt());
116         responseDTO.setDeviceName(serviceRequest.getDevice() != null
117             ? serviceRequest.getDevice().getDeviceName()
118             : null);
119         responseDTO.setReviewedByName(serviceRequest.getReviewedBy() != null
120             ? serviceRequest.getReviewedBy().getName()
121             : null);
122         responseDTO.setRaisedByName(serviceRequest.getRaisedBy().getName());
123         return responseDTO;
124     }
125
126
127     @Transactional
128     public ServiceRequestResponseDTO updateServiceRequestForRepairByAdmin(Authentication authentication,
AdminActionDTO actionDTO){
129         ServiceRequest
serviceRequest=serviceRequestRepository.findById(actionDTO.getServiceRequestId()).orElseThrow(
130             ()->new RuntimeException("Service Request Not Found")
131         );
132         String email=AuthUtil.extractEmail(authentication);
133         Employee admin=employeeRepo.findByEmail(email).orElseThrow(
134             ()->new EmployeeNotFound("Admin not found:"+email)
135         );
136         Device device=serviceRequest.getDevice();
137         switch (actionDTO.getStatus()){
138             case SENT_FOR_REPAIR :
139                 device.setDeviceStatus(DeviceStatus.SENT_TO_VENDOR);
140                 deviceRepository.save(device);
141                 createRepairLogIfMissing(serviceRequest, actionDTO.getAdminRemarks());

```

```

142         break;
143     case REJECTED :
144         device.setDeviceStatus(DeviceStatus.ASSIGNED);
145         break;
146     }
147     serviceRequest.setReviewedBy(admin);
148     serviceRequest.setAdminRemarks(actionDTO.getAdminRemarks());
149     serviceRequest.setStatus(actionDTO.getStatus());
150     ServiceRequest saved=serviceRequestRepository.save(serviceRequest);
151     return toServiceRequestResponseDTO(saved);
152 }
153
154 public ServiceRequestResponseDTO updateServiceRequestForOtherServicesByAdmin(Authentication
authentication, ApprovalActionDTO actionDTO) {
155     ServiceRequest
serviceRequest=serviceRequestRepository.findById(actionDTO.getServiceRequestId()).orElseThrow(
156         ()->new RuntimeException("Service Request Not Found")
157     );
158     String email=AuthUtil.extractEmail(authentication);
159     Employee admin=employeeRepo.findById(email).orElseThrow(
160         ()->new EmployeeNotFound("Admin not found:"+email)
161     );
162     Device device=serviceRequest.getDevice();
163     switch(actionDTO.getStatus()){
164         case APPROVED :
165             handelApproval(serviceRequest,device,actionDTO.getReplacementDeviceId());
166             break;
167         case REJECTED:
168             device.setDeviceStatus(DeviceStatus.ASSIGNED);
169             break;
170     }
171     deviceRepository.save(device);
172     serviceRequest.setReviewedBy(admin);
173     serviceRequest.setAdminRemarks(actionDTO.getAdminRemarks());
174     serviceRequest.setStatus(ServiceRequestStatus.CLOSED);
175     ServiceRequest saved=serviceRequestRepository.save(serviceRequest);
176     return toServiceRequestResponseDTO(saved);
177 }
178
179 public void handelApproval(ServiceRequest serviceRequest, Device device,Long deviceId) {
180     ServiceRequestType type=serviceRequest.getRequestType();
181     DeviceAssignment current= device.getCurrentAssignment();
182     if(type==ServiceRequestType.RETURN){
183         device.setCurrentAssignment(null);
184         device.setDeviceStatus(DeviceStatus.AVAILABLE);
185         deviceAssignmentRepo.delete(current);
186         System.out.println("deleted related assignment details for device:"+device.getDeviceName());
187     }
188     if(type==ServiceRequestType.REPLACEMENT){
189         device.setCurrentAssignment(null);
190         device.setDeviceStatus(DeviceStatus.AVAILABLE);
191         deviceAssignmentRepo.delete(current);
192         System.out.println("deleted related assignment details for device:"+device.getDeviceName());
193         Device newDevice=deviceRepository.findById(deviceId).orElseThrow(
194             ()->new RuntimeException("Device not found:"+deviceId)
195         );
196         if(newDevice.getDeviceStatus()!=DeviceStatus.AVAILABLE){
197             throw new RuntimeException("Device not available");
198         }
199         if(newDevice.getCurrentAssignment()!=null){
200             throw new RuntimeException("Device already assigned");
201         }
202         Employee employee=employeeRepo.findByName(serviceRequest.getRaisedBy().getName()).orElseThrow(
203             ()->new EmployeeNotFound("Employee Not found:"+serviceRequest.getRaisedBy().getName())
204         );
205         DeviceAssignment deviceAssignment=new DeviceAssignment();
206         deviceAssignment.setAssignedTo(employee);
207         deviceAssignment.setAssignedDate(LocalDate.now());
208         deviceAssignment.setDevice(newDevice);
209         deviceAssignment.setStatus(AssignmentStatus.ACTIVE);
210         DeviceAssignment saved=deviceAssignmentRepo.save(deviceAssignment);
211         newDevice.setCurrentAssignment(saved);
212         newDevice.setDeviceStatus(DeviceStatus.ASSIGNED);
213         deviceRepository.save(newDevice);
214     }
215 }

```

```

216     }
217
218     public List<ServiceRequestResponseDTO> getAllServiceRequestByVendor(Authentication authentication){
219         String email=AuthUtil.extractEmail(authentication);
220         Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
221             ()->new VendorNotFoundException("Vendor not found: "+email)
222         );
223
224         List<ServiceRequest>
serviceRequestList=serviceRequestRepository.findById(deviceTechVendorAndStatus(vendor,
225             ServiceRequestStatus.SENT_FOR_REPAIR);
226         return serviceRequestList.stream()
227             .map(serviceRequest -> toServiceRequestResponseDTO(serviceRequest))
228             .toList();
229     }
230
231     public ResponseEntity<String> updateServiceRequestToUnderRepair(Authentication authentication, long
serviceRequestId) {
232         ServiceRequest serviceRequest=serviceRequestRepository.findById(serviceRequestId).orElseThrow(
233             ()->new RuntimeException("Service Request Not Found")
234         );
235         Device device=serviceRequest.getDevice();
236         serviceRequest.setStatus(ServiceRequestStatus.RECEIVED_BY_VENDOR);
237         device.setDeviceStatus(DeviceStatus.UNDER_REPAIR);
238         serviceRequestRepository.save(serviceRequest);
239         deviceRepository.save(device);
240         notificationService.notify(serviceRequest.getReviewedBy(),"Device:"+device+"is being
repaired","DEVICE UNDER REPAIR");
241         return ResponseEntity.ok(
242             "Recieved the device and started the repair"
243         );
244     }
245
246     @Transactional
247     public ServiceRequestResponseDTO updateServiceRequestByVendor(Authentication authentication, RepairDTO
repairDTO) {
248
249         // 1. Get vendor
250         String email = AuthUtil.extractEmail(authentication);
251         Vendor vendor = vendorRepo.findByEmail(email)
252             .orElseThrow(() -> new RuntimeException("Vendor not found"));
253
254         // 2. Get request
255         ServiceRequest request = serviceRequestRepository.findById(repairDTO.getRequestId())
256             .orElseThrow(() -> new RuntimeException("Request not found"));
257
258         // 3. Validate request belongs to vendor
259         if (!request.getDevice().getTechVendor().getId().equals(vendor.getId())) {
260             throw new RuntimeException("Unauthorized access");
261         }
262
263         // 4. Validate status
264         if (request.getStatus() != ServiceRequestStatus.RECEIVED_BY_VENDOR) {
265             throw new RuntimeException("Request is not in repair stage");
266         }
267
268         Device device = request.getDevice();
269         RepairLog repairLog = repairLogRepository.findByServiceRequest(request)
270             .orElseThrow(() -> new RuntimeException("Repair log not found for request " +
request.getId()));
271
272         if (repairDTO.getStatus() == null) {
273             throw new RuntimeException("Repair status is required");
274         }
275
276         RepairBill repairBill=new RepairBill();
277         repairBill.setDevice(device);
278         repairBill.setGeneratedDate(LocalDate.now());
279         repairBill.setPaymentStatus(PaymentStatus.PENDING);
280
281         // 5. Update based on repair result
282         if (repairDTO.getStatus()==ServiceRequestStatus.REPAIR_DONE) {
283             request.setResolution(repairDTO.getResolution());
284             request.setStatus(repairDTO.getStatus());
285             repairBill.setRepairCost(repairDTO.getRepairCost());
286             device.setDeviceStatus(DeviceStatus.REPAIR_DONE);

```

```

287         repairLog.setRemarks("Repair completed successfully");
288     } else if(repairDTO.getStatus()==ServiceRequestStatus.IRREPARABLE){
289         request.setResolution(repairDTO.getResolution());
290         request.setStatus(repairDTO.getStatus());
291         repairBill.setRepairCost(BigDecimal.ZERO);
292         device.setDeviceStatus(DeviceStatus.CONDEMNED);
293         repairLog.setRemarks("Repair closed without successful fix");
294     }
295
296     request.setResolvedAt(LocalDateTime.now());
297     repairLog.setRepairAction(repairDTO.getResolution());
298     repairLog.setRepairedBy(vendor.getName());
299     repairLog.setRepairDate(LocalDate.now());
300     if(repairDTO.getDamagedComponent()!=null){
301         repairLog.setDamagedComponent(repairDTO.getDamagedComponent());
302     }
303     if(repairDTO.getReplacedComponent()!=null){
304         repairLog.setReplacedComponent(repairDTO.getReplacedComponent());
305     }
306
307     RepairLog savedLog=repairLogRepository.save(repairLog);
308     repairBill.setRepairLog(savedLog);
309     repairBillRepository.save(repairBill);
310     deviceRepository.save(device);
311     request.setStatus(ServiceRequestStatus.CLOSED);
312     ServiceRequest serviceRequest=serviceRequestRepository.save(request);
313     return toServiceRequestResponseDTO(serviceRequest);
314 }
315
316 private void createRepairLogIfMissing(ServiceRequest serviceRequest, String adminRemarks) {
317     if (repairLogRepository.findByServiceRequest(serviceRequest).isPresent()) {
318         return;
319     }
320
321     RepairLog repairLog = new RepairLog();
322     repairLog.setDevice(serviceRequest.getDevice());
323     repairLog.setServiceRequest(serviceRequest);
324     repairLog.setIssueType(serviceRequest.getRequestType() != null
325         ? serviceRequest.getRequestType().name()
326         : null);
327     repairLog.setRemarks(adminRemarks);
328     repairLogRepository.save(repairLog);
329 }
330
331
332 public List<ServiceRequestResponseDTO> getServiceRequestRecievedByVendor(Authentication authentication)
333 {
334     String email=AuthUtil.extractEmail(authentication);
335     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
336         ()->new VendorNotFoundException("Vendor Not found:"+email)
337     );
338     List<ServiceRequest>
339     serviceRequestList=serviceRequestRepository.findByDeviceTechVendorAndStatus(vendor,ServiceRequestStatus.RECEIVED
340     _BY_VENDOR);
341     return serviceRequestList
342         .stream()
343         .map(serviceRequest -> toServiceRequestResponseDTO(serviceRequest))
344         .toList();
345 }
346

```



```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import com.example.EmployeeManagementSystem.Entity.LeaveType;
5 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
6 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
7 import org.slf4j.Logger;
8 import org.slf4j.LoggerFactory;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.math.BigDecimal;
13 import java.time.LocalDate;
14 import java.util.List;
15
16 @Service
17 public class SickLeaveResetService {
18
19     private static final Logger log = LoggerFactory.getLogger(SickLeaveResetService.class);
20
21     private final LeaveTypeRepository leaveTypeRepository;
22     private final LeaveEntitlementRepository entitlementRepository;
23
24     public SickLeaveResetService(LeaveTypeRepository leaveTypeRepository,
25                               LeaveEntitlementRepository entitlementRepository) {
26         this.leaveTypeRepository = leaveTypeRepository;
27         this.entitlementRepository = entitlementRepository;
28     }
29
30     @Transactional
31     public void runMonthlyReset(LocalDate resetDate) {
32         // resetDate = last day of the month being reset (e.g. 2025-06-30)
33         int year = resetDate.getYear();
34         int month = resetDate.getMonthValue(); // FIX: scope reset to this month only
35
36         // Find all leave types that reset monthly (only SICK in your design)
37         List<LeaveType> resetTypes = leaveTypeRepository.findByMonthlyResetTrue();
38
39         for (LeaveType leaveType : resetTypes) {
40             // FIX: was findByLeaveTypeIdAndYear – fetched ALL entitlements for the year
41             // and re-reset them every month. Now scoped to current month's accruals.
42             List<LeaveEntitlement> entitlements =
43                 entitlementRepository.findByLeaveTypeIdAndYear(leaveType.getId(), year);
44
45             for (LeaveEntitlement entitlement : entitlements) {
46
47                 BigDecimal available = entitlement.getAvailableBalance();
48
49                 if (available.compareTo(BigDecimal.ZERO) > 0) {
50                     // Wipe unused days – set accrued = used so available = 0
51                     // Preserves the audit trail of what was used
52                     entitlement.setAccruedThisYear(entitlement.getUsedThisYear());
53
54                     log.info("Sick leave reset for employee {} – {} unused day(s) expired at end of {}-{}",
55                             entitlement.getEmployee().getEmployeeId(), available, year, month);
56                 } else {
57                     log.debug("Sick leave reset for employee {} – nothing to expire (month {}-{})",
58                             entitlement.getEmployee().getEmployeeId(), year, month);
59                 }
60
61                 entitlementRepository.save(entitlement);
62             }
63         }
64
65         log.info("Sick leave monthly reset complete for {}", resetDate);
66     }
67 }

```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
4  import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
5  import com.example.EmployeeManagementSystem.Entity.Employee;
6  import com.example.EmployeeManagementSystem.Entity.Restaurant;
7  import com.example.EmployeeManagementSystem.Entity.Subscription;
8  import com.example.EmployeeManagementSystem.Enum.MealSlot;
9  import com.example.EmployeeManagementSystem.Enum.ScheduleType;
10 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
11 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
12 import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
13 import com.example.EmployeeManagementSystem.Exception.SubscriptionAlreadyExists;
14 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
15 import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
16 import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
17 import com.example.EmployeeManagementSystem.Util.AuthUtil;
18 import org.springframework.security.core.Authentication;
19 import org.springframework.stereotype.Service;
20
21 import java.time.ZoneId;
22 import java.time.ZonedDateTime;
23 import java.util.ArrayList;
24 import java.util.List;
25
26 @Service
27 public class SubscriptionService {
28
29     private final SubscriptionRepository subscriptionRepository;
30     private final EmployeeRepo employeeRepo;
31     private final DeliveryTimeService deliveryTimeService;
32     private final RestaurantRepository restaurantRepository;
33
34     public SubscriptionService(SubscriptionRepository subscriptionRepository,
35                               EmployeeRepo employeeRepo,
36                               DeliveryTimeService deliveryTimeService,
37                               RestaurantRepository restaurantRepository) {
38         this.subscriptionRepository = subscriptionRepository;
39         this.employeeRepo = employeeRepo;
40         this.deliveryTimeService = deliveryTimeService;
41         this.restaurantRepository = restaurantRepository;
42     }
43
44     /**
45      * Add subscriptions for an employee.
46      *
47      * Each SlotOrder specifies a meal slot AND the restaurant for that slot,
48      * so an employee can have breakfast from Restaurant A and lunch from Restaurant B.
49      *
50      * Validations:
51      * - Employee must exist and be ACTIVE
52      * - Restaurant must exist and be active
53      * - Restaurant must support the requested meal slot
54      * - No duplicate active subscription for the same employee + slot combination
55      */
56     public void addSubscription(SubscriptionRequest request, Authentication authentication) {
57         String email = AuthUtil.extractEmail(authentication);
58         Employee employee = employeeRepo.findByEmail(email)
59             .orElseThrow(() -> new EmployeeNotFound(
60                 "Employee with id " + email + " not found"));
61
62         if (request.getSlotOrders() == null || request.getSlotOrders().isEmpty()) {
63             throw new IllegalArgumentException("slotOrders cannot be null or empty");
64         }
65
66         if (request.getScheduleType() == ScheduleType.DAILY) {
67             request.setDayOfWeek(null);
68         } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
69             if (request.getDayOfWeek() == null) {
70                 throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
71             }

```

```

72     }
73
74     for (SubscriptionRequest.SlotOrder slotOrder : request.getSlotOrders()) {
75         MealSlot slot = slotOrder.getMealSlot();
76         Long restaurantId = slotOrder.getRestaurantId();
77
78         // 1. Validate restaurant exists and is active
79         Restaurant restaurant = restaurantRepository.findById(restaurantId)
80             .orElseThrow(() -> new RestaurantNotFoundException(
81                 "Restaurant with id " + restaurantId + " not found"));
82         if (!restaurant.isActive()) {
83             throw new IllegalArgumentException(
84                 "Restaurant '" + restaurant.getName() + "' is not currently active");
85         }
86
87         // 2. Validate the restaurant actually serves this meal slot
88         if (!restaurant.getSupportedMealSlots().contains(slot)) {
89             throw new IllegalArgumentException(
90                 "Restaurant '" + restaurant.getName() + "' does not serve " + slot +
91                 ". Supported slots: " + restaurant.getSupportedMealSlots());
92         }
93
94         // 3. Check for duplicate active subscription (same employee + slot)
95         if (subscriptionRepository.checkSubscriptionExists(
96             employee.getEmployeeId(), slot.name(), SubscriptionStatus.ACTIVE.name()) > 0) {
97             throw new SubscriptionAlreadyExists(
98                 "Employee " + employee.getEmployeeId() +
99                 " already has an active " + slot + " subscription. " +
100                 "Cancel the existing one before subscribing to a new restaurant.");
101         }
102
103         // 4. Create subscription
104         Subscription subscription = new Subscription();
105         subscription.setEmployee(employee);
106         subscription.setSlot(slot);
107         subscription.setRestaurant(restaurant);
108         subscription.setScheduleType(request.getScheduleType());
109         subscription.setDayOfWeek(request.getDayOfWeek());
110         subscription.setStatus(SubscriptionStatus.ACTIVE);
111         subscription.setNextDeliveryTime(
112             deliveryTimeService.calculateNextDelivery(request, slot, employee));
113
114         subscriptionRepository.save(subscription);
115     }
116 }
117
118 public List<SubscriptionDTO> getAllSubscriptions() {
119     List<Subscription> subscriptions = subscriptionRepository.findAll();
120     List<SubscriptionDTO> dtoList = new ArrayList<>();
121     for (Subscription s : subscriptions) {
122         dtoList.add(convertToDTO(s));
123     }
124     return dtoList;
125 }
126
127 public Subscription updateSubscription(Long id, SubscriptionRequest request) {
128     Subscription subscription = subscriptionRepository.findById(id)
129         .orElseThrow(() -> new RuntimeException("Subscription not found"));
130     Employee employee = subscription.getEmployee();
131
132     if (request.getScheduleType() == ScheduleType.DAILY) {
133         subscription.setDayOfWeek(null);
134     } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
135         if (request.getDayOfWeek() == null) {
136             throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
137         }
138         subscription.setDayOfWeek(request.getDayOfWeek());
139     }
140     subscription.setScheduleType(request.getScheduleType());
141
142     // Update slot & restaurant if a single slotOrder is provided
143     if (request.getSlotOrders() != null && !request.getSlotOrders().isEmpty()) {
144         if (request.getSlotOrders().size() > 1) {
145             throw new IllegalArgumentException(
146                 "Update supports one slot at a time. Use separate subscriptions for multiple
slots.");

```

```

147     }
148     SubscriptionRequest.SlotOrder slotOrder = request.getSlotOrders().get(0);
149     MealSlot newSlot = slotOrder.getMealSlot();
150     Restaurant restaurant = restaurantRepository.findById(slotOrder.getRestaurantId())
151         .orElseThrow(() -> new RestaurantNotFoundException(
152             "Restaurant " + slotOrder.getRestaurantId() + " not found"));
153     if (!restaurant.isActive()) {
154         throw new IllegalArgumentException("Restaurant '" + restaurant.getName() + "' is not
active");
155     }
156     if (!restaurant.getSupportedMealSlots().contains(newSlot)) {
157         throw new IllegalArgumentException(
158             "Restaurant '" + restaurant.getName() + "' does not serve " + newSlot);
159     }
160     subscription.setSlot(newSlot);
161     subscription.setRestaurant(restaurant);
162 }
163
164     subscription.setNextDeliveryTime(
165         deliveryTimeService.calculateNextDelivery(request, subscription.getSlot(), employee));
166
167     return subscriptionRepository.save(subscription);
168 }
169
170 public Subscription pauseSubscription(Long id) {
171     Subscription subscription = subscriptionRepository.findById(id)
172         .orElseThrow(() -> new RuntimeException("Subscription not found"));
173     subscription.setStatus(SubscriptionStatus.PAUSED);
174     subscription.setNextDeliveryTime(null);
175     return subscriptionRepository.save(subscription);
176 }
177
178 public Subscription resumeSubscription(Long id) {
179     Subscription subscription = subscriptionRepository.findById(id)
180         .orElseThrow(() -> new RuntimeException("Subscription not found"));
181     subscription.setStatus(SubscriptionStatus.ACTIVE);
182     subscription.setNextDeliveryTime(deliveryTimeService.getNextDeliveryTime(subscription));
183     return subscriptionRepository.save(subscription);
184 }
185
186 public Subscription expireSubscription(Long id) {
187     Subscription subscription = subscriptionRepository.findById(id)
188         .orElseThrow(() -> new RuntimeException("Subscription not found"));
189     subscription.setStatus(SubscriptionStatus.EXPIRED);
190     subscription.setNextDeliveryTime(null);
191     return subscriptionRepository.save(subscription);
192 }
193
194 public List<SubscriptionDTO> getSubscriptionOfUser(Authentication authentication) {
195     String email= AuthUtil.extractEmail(authentication);
196     List<Subscription> subscriptions = subscriptionRepository.findByEmployee_Email(email);
197     List<SubscriptionDTO> dtoList = new ArrayList<>();
198     for (Subscription s : subscriptions) {
199         dtoList.add(convertToDTO(s));
200     }
201     return dtoList;
202 }
203
204 // helpers
205
206 private SubscriptionDTO convertToDTO(Subscription s) {
207     SubscriptionDTO dto = new SubscriptionDTO();
208     dto.setSubscriptionId(s.getId());
209     dto.setUserId(s.getEmployee().getEmployeeId());
210     dto.setMealSlot(s.getSlot());
211     dto.setScheduleType(s.getScheduleType());
212     if (s.getScheduleType() == ScheduleType.WEEKLY) {
213         dto.setDayOfWeek(s.getDayOfWeek());
214     }
215     dto.setStatus(s.getStatus());
216     if (s.getNextDeliveryTime() != null) {
217         ZoneId userZone = ZoneId.of(s.getEmployee().getTimezone());
218         dto.setNextDeliveryTime(s.getNextDeliveryTime().atZone(userZone));
219     }
220     // Restaurant info
221     if (s.getRestaurant() != null) {

```

```
222         dto.setRestaurantId(s.getRestaurant().getId());
223         dto.setRestaurantName(s.getRestaurant().getName());
224         dto.setRestaurantAddress(s.getRestaurant().getAddress());
225     }
226     return dto;
227 }
228 }
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import dev.samstevens.totp.code.*;
4 import dev.samstevens.totp.secret.DefaultSecretGenerator;
5 import dev.samstevens.totp.secret.SecretGenerator;
6 import dev.samstevens.totp.time.SystemTimeProvider;
7 import dev.samstevens.totp.time.TimeProvider;
8 import org.springframework.stereotype.Service;
9
10 @Service
11 public class TotpService {
12
13     private final SecretGenerator secretGenerator = new DefaultSecretGenerator();
14     private final TimeProvider timeProvider = new SystemTimeProvider();
15     private final CodeGenerator codeGenerator = new DefaultCodeGenerator();
16     private final CodeVerifier codeVerifier = new DefaultCodeVerifier(codeGenerator, timeProvider);
17
18     /** Generate a new Base32 secret for a user */
19     public String generateSecret() {
20         return secretGenerator.generate();
21     }
22
23     /**
24      * Generate a QR code URI – user scans this with Google Authenticator / Authy.
25      * Format: otpauth://totp/{issuer}:{username}?secret={secret}&issuer={issuer}
26      */
27     public String getQrCodeUri(String secret, String username) {
28         return String.format(
29             "otpauth://totp/EmployeeMS:%s?secret=%s&issuer=EmployeeManagementSystem",
30             username, secret
31         );
32     }
33
34     /**
35      * Verify the 6-digit code the user typed.
36      * DefaultCodeVerifier already allows ±1 time step (±30 seconds) for clock skew.
37      */
38     public boolean verifyCode(String secret, String code) {
39         try {
40             return codeVerifier.isValidCode(secret, code);
41         } catch (Exception e) {
42             return false;
43         }
44     }
45 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.*;
4 import com.example.EmployeeManagementSystem.Entity.Device;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import com.example.EmployeeManagementSystem.Entity.VendorNegotiation;
7 import com.example.EmployeeManagementSystem.Entity.VendorNegotiationMessage;
8 import com.example.EmployeeManagementSystem.Enum.MessageSenderType;
9 import com.example.EmployeeManagementSystem.Enum.NegotiationStatus;
10 import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
11 import com.example.EmployeeManagementSystem.Repository.VendorNegotiationMessageRepository;
12 import com.example.EmployeeManagementSystem.Repository.VendorNegotiationRepository;
13 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
14 import com.example.EmployeeManagementSystem.Util.AuthUtil;
15 import org.springframework.security.access.AccessDeniedException;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.stereotype.Service;
18 import org.springframework.transaction.annotation.Transactional;
19
20 import java.util.List;
21
22 @Service
23 public class VendorNegotiationService {
24
25     private final VendorNegotiationRepository negotiationRepository;
26     private final VendorNegotiationMessageRepository messageRepository;
27     private final VendorRepo vendorRepository;
28     private final DeviceRepository deviceRepository;
29
30     public VendorNegotiationService(VendorNegotiationRepository negotiationRepository,
31                                     VendorNegotiationMessageRepository messageRepository,
32                                     VendorRepo vendorRepository,
33                                     DeviceRepository deviceRepository) {
34         this.negotiationRepository = negotiationRepository;
35         this.messageRepository = messageRepository;
36         this.vendorRepository = vendorRepository;
37         this.deviceRepository = deviceRepository;
38     }
39
40     @Transactional
41     public VendorNegotiationResponse createNegotiation(VendorNegotiationRequest request,
42                                                         Authentication authentication) {
43         validateCreateRequest(request);
44
45         MessageSenderType senderType = senderType(authentication);
46         Vendor vendor = resolveVendor(request, authentication, senderType);
47         Device device = resolveDevice(request.getDeviceId(), vendor);
48
49         VendorNegotiation negotiation = new VendorNegotiation();
50         negotiation.setVendor(vendor);
51         negotiation.setDevice(device);
52         negotiation.setSubject(request.getSubject().trim());
53         negotiation.setDescription(request.getDescription().trim());
54         negotiation.setProposedAmount(request.getProposedAmount());
55         negotiation.setStatus(NegotiationStatus.PENDING);
56
57         VendorNegotiation saved = negotiationRepository.save(negotiation);
58
59         if (hasText(request.getInitialMessage())) {
60             saveMessage(saved, senderType, AuthUtil.extractEmail(authentication),
61 request.getInitialMessage());
62             saved.setStatus(NegotiationStatus.NEGOTIATING);
63             saved = negotiationRepository.save(saved);
64         }
65
66         return toResponse(saved, true);
67     }
68
69     @Transactional(readOnly = true)
70     public List<VendorNegotiationResponse> listNegotiations(Authentication authentication) {
71         if (isAdmin(authentication)) {

```

```

71         return negotiationRepository.findAllByOrderByUpdatedAtDesc()
72             .stream()
73             .map(n -> toResponse(n, false))
74             .toList();
75     }
76
77     Vendor vendor = vendorForAuthentication(authentication);
78     return negotiationRepository.findByVendorIdOrderByUpdatedAtDesc(vendor.getId())
79         .stream()
80         .map(n -> toResponse(n, false))
81         .toList();
82 }
83
84 @Transactional(readOnly = true)
85 public VendorNegotiationResponse getNegotiation(Long negotiationId, Authentication authentication) {
86     VendorNegotiation negotiation = findNegotiation(negotiationId);
87     assertParticipantAccess(negotiation, authentication);
88     return toResponse(negotiation, true);
89 }
90
91 @Transactional
92 public VendorNegotiationResponse addMessage(Long negotiationId,
93     VendorNegotiationMessageRequest request,
94     Authentication authentication) {
95     if (request == null || !hasText(request.getMessage())) {
96         throw new IllegalArgumentException("Message is required");
97     }
98
99     VendorNegotiation negotiation = findNegotiation(negotiationId);
100     assertParticipantAccess(negotiation, authentication);
101
102     saveMessage(negotiation, senderType(authentication), AuthUtil.extractEmail(authentication),
103 request.getMessage());
104     if (negotiation.getStatus() == NegotiationStatus.PENDING) {
105         negotiation.setStatus(NegotiationStatus.NEGOTIATING);
106     }
107     VendorNegotiation saved = negotiationRepository.save(negotiation);
108     return toResponse(saved, true);
109 }
110
111 @Transactional
112 public VendorNegotiationResponse updateStatus(Long negotiationId,
113     VendorNegotiationStatusRequest request,
114     Authentication authentication) {
115     if (request == null || request.getStatus() == null) {
116         throw new IllegalArgumentException("Status is required");
117     }
118
119     VendorNegotiation negotiation = findNegotiation(negotiationId);
120     assertParticipantAccess(negotiation, authentication);
121
122     if (!isAdmin(authentication) && request.getStatus() == NegotiationStatus.PAID) {
123         throw new AccessDeniedException("Only admin can mark a negotiation as paid");
124     }
125
126     negotiation.setStatus(request.getStatus());
127     return toResponse(negotiationRepository.save(negotiation), true);
128 }
129
130 private void validateCreateRequest(VendorNegotiationRequest request) {
131     if (request == null) {
132         throw new IllegalArgumentException("Negotiation request is required");
133     }
134     if (!hasText(request.getSubject())) {
135         throw new IllegalArgumentException("Subject is required");
136     }
137     if (!hasText(request.getDescription())) {
138         throw new IllegalArgumentException("Description is required");
139     }
140 }
141
142 private Vendor resolveVendor(VendorNegotiationRequest request,
143     Authentication authentication,
144     MessageSenderType senderType) {
145     if (senderType == MessageSenderType.ADMIN) {
146         if (request.getVendorId() == null) {

```



```

146         throw new IllegalArgumentException("vendorId is required when admin creates a
negotiation");
147     }
148     return vendorRepository.findById(request.getVendorId())
149         .orElseThrow(() -> new IllegalArgumentException("Vendor not found: " +
request.getVendorId()));
150 }
151
152     return vendorForAuthentication(authentication);
153 }
154
155     private Device resolveDevice(Long deviceId, Vendor vendor) {
156         if (deviceId == null) {
157             return null;
158         }
159
160         Device device = deviceRepository.findById(deviceId)
161             .orElseThrow(() -> new IllegalArgumentException("Device not found: " + deviceId));
162
163         if (device.getTechVendor() != null && !device.getTechVendor().getId().equals(vendor.getId())) {
164             throw new AccessDeniedException("Device does not belong to this vendor");
165         }
166
167         return device;
168     }
169
170     private Vendor vendorForAuthentication(Authentication authentication) {
171         String email = AuthUtil.extractEmail(authentication);
172         return vendorRepository.findByEmail(email)
173             .orElseThrow(() -> new AccessDeniedException("Vendor account not found for " + email));
174     }
175
176     private VendorNegotiation findNegotiation(Long negotiationId) {
177         return negotiationRepository.findById(negotiationId)
178             .orElseThrow(() -> new IllegalArgumentException("Negotiation not found: " +
negotiationId));
179     }
180
181     private void assertParticipantAccess(VendorNegotiation negotiation, Authentication authentication) {
182         if (isAdmin(authentication)) {
183             return;
184         }
185
186         Vendor vendor = vendorForAuthentication(authentication);
187         if (!negotiation.getVendor().getId().equals(vendor.getId())) {
188             throw new AccessDeniedException("You cannot access another vendor's negotiation");
189         }
190     }
191
192     private void saveMessage(VendorNegotiation negotiation,
193                             MessageSenderType senderType,
194                             String senderEmail,
195                             String text) {
196         VendorNegotiationMessage message = new VendorNegotiationMessage();
197         message.setNegotiation(negotiation);
198         message.setSenderType(senderType);
199         message.setSenderEmail(senderEmail);
200         message.setMessage(text.trim());
201         messageRepository.save(message);
202     }
203
204     private VendorNegotiationResponse toResponse(VendorNegotiation negotiation, boolean includeMessages) {
205         VendorNegotiationResponse response = new VendorNegotiationResponse();
206         response.setId(negotiation.getId());
207         response.setVendorId(negotiation.getVendor().getId());
208         response.setVendorName(negotiation.getVendor().getName());
209         response.setSubject(negotiation.getSubject());
210         response.setDescription(negotiation.getDescription());
211         response.setProposedAmount(negotiation.getProposedAmount());
212         response.setStatus(negotiation.getStatus());
213         response.setCreatedAt(negotiation.getCreatedAt());
214         response.setUpdatedAt(negotiation.getUpdatedAt());
215
216         if (negotiation.getDevice() != null) {
217             response.setDeviceId(negotiation.getDevice().getId());
218             response.setDeviceName(negotiation.getDevice().getDeviceName());

```

```

219     }
220
221     if (includeMessages) {
222
223         response.setMessages(messageRepository.findByNegotiationIdOrderByCreatedAtAsc(negotiation.getId())
224             .stream()
225             .map(this::toMessageResponse)
226             .toList());
227     }
228     return response;
229 }
230
231 private VendorNegotiationMessageResponse toMessageResponse(VendorNegotiationMessage message) {
232     VendorNegotiationMessageResponse response = new VendorNegotiationMessageResponse();
233     response.setId(message.getId());
234     response.setSenderType(message.getSenderType());
235     response.setSenderEmail(message.getSenderEmail());
236     response.setMessage(message.getMessage());
237     response.setCreatedAt(message.getCreatedAt());
238     return response;
239 }
240
241 private MessageSenderType senderType(Authentication authentication) {
242     return isAdmin(authentication) ? MessageSenderType.ADMIN : MessageSenderType.VENDOR;
243 }
244
245 private boolean isAdmin(Authentication authentication) {
246     return authentication != null && authentication.getAuthorities().stream()
247         .anyMatch(authority -> "ROLE_ADMIN".equals(authority.getAuthority()));
248 }
249
250 private boolean hasText(String value) {
251     return value != null && !value.isBlank();
252 }
253 }
254

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.*;
4 import com.example.EmployeeManagementSystem.Enum.Status;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
7 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
8 import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
9 import org.slf4j.Logger;
10 import org.slf4j.LoggerFactory;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.math.BigDecimal;
15 import java.math.RoundingMode;
16 import java.time.LocalDate;
17 import java.util.List;
18 import java.util.Map;
19 import java.util.Set;
20 import java.util.stream.Collectors;
21
22 @Service
23 public class YearEndCarryForwardService {
24
25     private static final Logger log = LoggerFactory.getLogger(YearEndCarryForwardService.class);
26     private static final BigDecimal CAP = BigDecimal.valueOf(30);
27     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL", "MATERNITY");
28
29     private final EmployeeRepo employeeRepository;
30     private final LeaveEntitlementRepository entitlementRepository;
31     private final YearEndCarryForwardLogRepository logRepository;
32     private final LeaveWarningRepository warningRepository;
33
34     public YearEndCarryForwardService(
35         EmployeeRepo employeeRepository,
36         LeaveEntitlementRepository entitlementRepository,
37         YearEndCarryForwardLogRepository logRepository,
38         LeaveWarningRepository warningRepository) {
39         this.employeeRepository = employeeRepository;
40         this.entitlementRepository = entitlementRepository;
41         this.logRepository = logRepository;
42         this.warningRepository = warningRepository;
43     }
44
45     // -----
46     // Main entry point – called by scheduler on Dec 31 at 22:00
47     // -----
48     @Transactional
49     public void runYearEnd(int closingYear) {
50
51         // Idempotency guard – never run twice for same year
52         if (logRepository.existsByProcessedYear(closingYear)) {
53             log.warn("Year-end carry-forward already processed for {}. Skipping.", closingYear);
54             return;
55         }
56
57         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
58         log.info("Starting year-end carry-forward for {} – {} employees", closingYear,
59             activeEmployees.size());
60         for (Employee employee : activeEmployees) {
61             processEmployeeYearEnd(employee, closingYear);
62         }
63
64         log.info("Year-end carry-forward complete for {}", closingYear);
65     }
66
67     // -----
68     // Per-employee processing
69     // -----
70     private void processEmployeeYearEnd(Employee employee, int closingYear) {

```

```

71     int nextYear = closingYear + 1;
72
73     // Fetch all entitlements for the closing year
74     List<LeaveEntitlement> entitlements =
75         entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(),
closingYear);
76
77     // Separate carry-forwardable from non-carry-forwardable
78     List<LeaveEntitlement> carryForwardable = entitlements.stream()
79         .filter(e -> SUPPORTED_LEAVE_TYPES.contains(e.getLeaveType().getName()))
80         .filter(e -> e.getLeaveType().isCarriesForward())
81         .collect(Collectors.toList());
82
83     // Step 1 – compute raw closing balance for each type
84     //     closing = opening + accrued - used
85     Map<LeaveEntitlement, BigDecimal> rawClosing = carryForwardable.stream()
86         .collect(Collectors.toMap(
87             e -> e,
88             e -> e.getOpeningBalance()
89                 .add(e.getAccruedThisYear())
90                 .subtract(e.getUsedThisYear())
91                 .max(BigDecimal.ZERO) // never negative
92         ));
93
94     // Step 2 – sum all carry-forwardable closing balances
95     BigDecimal totalRaw = rawClosing.values().stream()
96         .reduce(BigDecimal.ZERO, BigDecimal::add);
97
98     // Step 3 – apply cap proportionally if total exceeds 30
99     Map<LeaveEntitlement, BigDecimal> cappedClosing = applyCap(rawClosing, totalRaw);
100
101     // Step 4 – persist: update closing balance on current year row,
102     //     create/update opening balance on next year row, write log
103     for (LeaveEntitlement entitlement : carryForwardable) {
104         BigDecimal raw = rawClosing.get(entitlement);
105         BigDecimal capped = cappedClosing.get(entitlement);
106         BigDecimal wasted = raw.subtract(capped);
107
108         // Update closing balance on the current year's row
109         entitlement.setClosingBalance(capped);
110         entitlementRepository.save(entitlement);
111
112         // Write or update next year's opening balance
113         LeaveEntitlement nextYearRow = entitlementRepository
114             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
115                 employee.getEmployeeId(), entitlement.getLeaveType().getId(), nextYear)
116             .orElseGet(() -> buildNextYearRow(employee, entitlement.getLeaveType(), nextYear));
117
118         nextYearRow.setOpeningBalance(capped);
119         entitlementRepository.save(nextYearRow);
120
121         // Write audit log
122         writeLog(employee, closingYear, entitlement.getLeaveType(), raw, capped, wasted);
123
124         // Issue DAY_WASTED warning if any days were lost
125         if (wasted.compareTo(BigDecimal.ZERO) > 0) {
126             issueWastedWarning(employee, wasted, capped);
127             log.info("Employee {} lost {} day(s) of {} to cap at year end",
128                 employee.getEmployeeId(), wasted, entitlement.getLeaveType().getName());
129         }
130     }
131
132     // Step 5 – zero out sick leave closing balance (it never carries)
133     entitlements.stream()
134         .filter(e -> SUPPORTED_LEAVE_TYPES.contains(e.getLeaveType().getName()))
135         .filter(e -> !e.getLeaveType().isCarriesForward())
136         .forEach(e -> {
137             e.setClosingBalance(BigDecimal.ZERO);
138             entitlementRepository.save(e);
139         });
140
141     log.info("Year-end processed for employee {} – total raw={{} capped={}",
142         employee.getEmployeeId(), totalRaw, CAP.min(totalRaw));
143 }
144
145 // -----

```

```

146 // Cap logic – distribute cap proportionally across leave types
147 //
148 // Example: casual=20, maternity=16 total=36, over cap by 6
149 // casual gets: 20/36 * 30 = 16.67
150 // maternity gets: 16/36 * 30 = 13.33
151 // total = 30 exactly
152 // -----
153 private Map<LeaveEntitlement, BigDecimal> applyCap(
154     Map<LeaveEntitlement, BigDecimal> rawClosing,
155     BigDecimal totalRaw) {
156
157     if (totalRaw.compareTo(CAP) <= 0) {
158         // Under cap – no adjustment needed
159         return rawClosing;
160     }
161
162     return rawClosing.entrySet().stream()
163         .collect(Collectors.toMap(
164             Map.Entry::getKey,
165             entry -> {
166                 BigDecimal raw = entry.getValue();
167                 // proportion = raw / totalRaw * CAP
168                 return raw
169                     .divide(totalRaw, 10, RoundingMode.HALF_UP)
170                     .multiply(CAP)
171                     .setScale(2, RoundingMode.HALF_UP);
172             }
173         ));
174 }
175
176 // -----
177 // Build a blank next-year entitlement row
178 // -----
179 private LeaveEntitlement buildNextYearRow(Employee employee, LeaveType leaveType, int year) {
180     LeaveEntitlement row = new LeaveEntitlement();
181     row.setEmployee(employee);
182     row.setLeaveType(leaveType);
183     row.setYear(year);
184     row.setOpeningBalance(BigDecimal.ZERO);
185     row.setAccruedThisYear(BigDecimal.ZERO);
186     row.setUsedThisYear(BigDecimal.ZERO);
187     row.setClosingBalance(BigDecimal.ZERO);
188     return entitlementRepository.save(row);
189 }
190
191 // -----
192 // Write audit log entry
193 // -----
194 private void writeLog(Employee employee, int year, LeaveType leaveType,
195     BigDecimal raw, BigDecimal capped, BigDecimal wasted) {
196     YearEndCarryForwardLog entry = new YearEndCarryForwardLog();
197     entry.setEmployee(employee);
198     entry.setProcessedYear(year);
199     entry.setLeaveType(leaveType);
200     entry.setClosingBalance(raw);
201     entry.setCappedBalance(capped);
202     entry.setDaysWasted(wasted);
203     entry.setOpeningNextYear(capped);
204     logRepository.save(entry);
205 }
206
207 // -----
208 // Issue a DAY_WASTED warning on the dashboard
209 // -----
210 private void issueWastedWarning(Employee employee, BigDecimal wasted, BigDecimal newBalance) {
211     LeaveWarning warning = new LeaveWarning();
212     warning.setEmployee(employee);
213     warning.setWarningType(LeaveWarning.WarningType.DAY_WASTED);
214     warning.setCarryForwardBalance(newBalance);
215     warning.setWarningDate(LocalDate.now());
216     warning.setMessage(
217         wasted.toPlainString() + " carry-forward day(s) were lost at year-end " +
218         "because your balance exceeded the 30-day cap. " +
219         "Your opening balance for next year is " + newBalance.toPlainString() + " days."
220     );
221     warningRepository.save(warning);

```

```
222     }
223 }
224
```

```
1 package com.example.EmployeeManagementSystem.Util;
2
3 import org.springframework.security.core.Authentication;
4 import org.springframework.security.oauth2.core.user.OAuth2User;
5 import org.springframework.stereotype.Component;
6
7 @Component
8 public class AuthUtil {
9     public static String extractEmail(Authentication authentication) {
10         if (authentication == null) return null;
11
12         // OAuth2 login (session-based) – principal is OAuth2User
13         if (authentication.getPrincipal() instanceof OAuth2User oAuth2User) {
14             return oAuth2User.getAttribute("email"); // always the actual email
15         }
16
17         // JWT or session username/password login – getName() is already the email
18         return authentication.getName();
19     }
20 }
21
```

```
1 package com.example.EmployeeManagementSystem.Util;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Enum.Role;
6 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
7 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
10 import io.jsonwebtoken.Claims;
11 import io.jsonwebtoken.Jwts;
12 import io.jsonwebtoken.security.Keys;
13 import jakarta.annotation.PostConstruct;
14 import org.springframework.beans.factory.annotation.Value;
15 import org.springframework.security.core.userdetails.UserDetails;
16 import org.springframework.stereotype.Component;
17
18 import javax.crypto.SecretKey;
19 import java.util.Date;
20 import java.util.Optional;
21
22 @Component
23 public class JWTUtil {
24
25     private final EmployeeRepo employeeRepo;
26     private final VendorRepo vendorRepo;
27
28     // FIX 1: Was 1000*60*60 (1 hour) – caused token expiry mid-session.
29     // Extended to 24 hours to match the session timeout in application.yaml.
30     private final long Expiration_time = 1000L * 60 * 60 * 24;
31
32     @Value("${jwt.secret}")
33     private String secret;
34     private SecretKey key;
35
36     public JWTUtil(EmployeeRepo employeeRepo, VendorRepo vendorRepo) {
37         this.employeeRepo = employeeRepo;
38         this.vendorRepo = vendorRepo;
39     }
40
41     @PostConstruct
42     public void init() {
43         key = Keys.hmacShaKeyFor(secret.getBytes());
44     }
45
46     public String generateToken(String username, String role) {
47         return Jwts.builder()
48             .subject(username)
49             .claim("role", role)
50             .issuedAt(new Date())
51             .expiration(new Date(System.currentTimeMillis() + Expiration_time))
52             .signWith(key)
53             .compact();
54     }
55
56     public String extractUsernameFromToken(String token) {
57         Claims body = extractClaims(token);
58         return body.getSubject();
59     }
60
61     private Claims extractClaims(String token) {
62         return Jwts.parser()
63             .verifyWith(key)
64             .build()
65             .parseSignedClaims(token)
66             .getPayload();
67     }
68
69     public boolean validateToken(String username, UserDetails userDetails, String token) {
70         return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
71     }
```



```
72
73     private boolean isTokenExpired(String token) {
74         return extractClaims(token).getExpiration().before(new Date());
75     }
76 }
```

```

1 package com.example.EmployeeManagementSystem.Util;
2
3 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
8 import jakarta.servlet.http.HttpServletRequest;
9 import jakarta.servlet.http.HttpServletResponse;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.oauth2.core.user.OAuth2User;
12 import org.springframework.security.web.authentication.SimpleUrlAuthenticationSuccessHandler;
13 import org.springframework.stereotype.Service;
14
15 import java.io.IOException;
16 import java.net.URLEncoder;
17 import java.nio.charset.StandardCharsets;
18
19 @Service
20 public class OAuth2SuccessHandler extends SimpleUrlAuthenticationSuccessHandler{
21     private final JWTUtil jwtUtil;
22     private final EmployeeRepo employeeRepo;
23     private final VendorRepo vendorRepo;
24     private final RefreshTokenService refreshTokenService;
25     private final RefreshTokenRepository refreshTokenRepository;
26
27     public OAuth2SuccessHandler(JWTUtil jwtUtil,
28                               EmployeeRepo employeeRepo,
29                               VendorRepo vendorRepo,
30                               RefreshTokenService refreshTokenService,
31                               RefreshTokenRepository refreshTokenRepository) {
32         this.jwtUtil = jwtUtil;
33         this.employeeRepo = employeeRepo;
34         this.vendorRepo = vendorRepo;
35         this.refreshTokenService = refreshTokenService;
36         this.refreshTokenRepository = refreshTokenRepository;
37     }
38
39     @Override
40     public void onAuthenticationSuccess(HttpServletRequest request,
41                                       HttpServletResponse response,
42                                       Authentication authentication) throws IOException {
43
44         try{
45             OAuth2User oAuth2User = (OAuth2User) authentication.getPrincipal();
46             String email = oAuth2User.getAttribute("email");
47
48             // Generate refresh token – check which entity type owns this email
49             RefreshToken refreshToken;
50             String role;
51             var employeeOpt = employeeRepo.findByEmail(email);
52             var vendorOpt = vendorRepo.findByEmail(email);
53
54             if (employeeOpt.isPresent()) {
55                 role = employeeOpt.get().getRole().name();
56                 refreshToken = refreshTokenService.createForEmployee(employeeOpt.get());
57             } else if (vendorOpt.isPresent()) {
58                 role = vendorOpt.get().getRole().name();
59                 refreshToken = refreshTokenService.createForVendor(vendorOpt.get());
60             } else {
61                 response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "User not found post-
62 OAuth");
63                 return;
64             }
65             String accessToken = jwtUtil.generateToken(email, role);
66             refreshTokenRepository.save(refreshToken);
67
68             // Redirect frontend to a callback page with tokens as query params
69             // (or use a fragment # so tokens don't hit server logs)
70             String redirectUrl = "http://localhost:8080/google.html" // your frontend page

```

```
71         + "?accessToken=" + URLEncoder.encode(accessToken, StandardCharsets.UTF_8)
72         + "&refreshToken=" + URLEncoder.encode(refreshToken.getToken(),
StandardCharsets.UTF_8);
73
74         getRedirectStrategy().sendRedirect(request, response, redirectUrl);
75     }
76     catch (Exception e) {
77         // This will now print the REAL error in your console
78         System.err.println("=== OAuth2SuccessHandler FAILED ===");
79         e.printStackTrace();
80         response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
81             "OAuth2 login failed: " + e.getMessage());
82     }
83 }
84 }
85
```

```
1  spring:
2    application:
3      name: EmployeeManagementSystem
4
5    datasource :
6      url : jdbc:mysql://localhost:3306/employee_management_db
7      username : ${USER_NAME}
8      password : ${PASSWORD}
9
10   jpa:
11     hibernate :
12       ddl-auto: update
13     show-sql: true
14
15   springdoc:
16     api-docs:
17       path: /v3/api-docs
18       enabled: true
19     swagger-ui:
20       path: /swagger-ui.html
21       enabled: true
22       operations-sorter: method
23       tags-sorter: alpha
24       try-it-out-enabled: true
25       filter: true
26     show-actuator: true
27   mvc:
28     static-path-pattern: /**
29   session:
30     tracking-modes: COOKIE
31   web:
32     resources:
33       static-locations: classpath:/static/
34       add-mappings: true
35   session:
36     timeout: 86400s # 24 hours in seconds (was 1440m which is incorrect format)
37     store-type: none # Use default cookie-based session
38
39   servlet:
40     session:
41       cookie:
42         http-only: true
43         same-site: strict
44         secure: false # set true in production with HTTPS
45
46   security:
47     debug: true
48     oauth2:
49       client:
50         registration:
51           google:
52             client-id: ${client_id}
53             client-secret: ${client_secret}
54             scope:
55               - email
56               - profile
57
58   logging:
59     level:
60       org.springframework.security: DEBUG
61       com.example.EmployeeManagementSystem: DEBUG
62   jwt:
63     secret: ${JWT_SECRET}
64     # Add this to your application.yaml
65   api:
66     key:
67       enabled: true
68       header-name: X-API-Key
69   app:
70     cors:
71       allowed-origins: http://192.168.1.3:8080,http://localhost:8080,http://smartems.local:8080
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS - Admin Console</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
8      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
9      <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
10     <style>
11         * { margin: 0; padding: 0; box-sizing: border-box; }
12         :root {
13             --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
14             --panel: #1e2436; --panel2: #252c40;
15             --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
16             --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
17             --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
18             --green: #36d399; --green-dim: rgba(54,211,153,0.12);
19             --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
20             --red: #f87171; --red-dim: rgba(248,113,113,0.12);
21             --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
22             --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
23         }
24         body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-
height: 1.5; overflow: hidden; }
25         .app { display: flex; height: 100vh; }
26         .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display:
flex; flex-direction: column; overflow-y: auto; }
27         .logo { padding: 20px; border-bottom: 1px solid var(--border); }
28         .logo h2 { font-size: 20px; color: var(--text); }
29         .logo span { color: var(--accent); }
30         .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
31         .nav-item { padding: 12px 20px; margin: 4px 12px; border-radius: 8px; cursor: pointer; transition:
all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; }
32         .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
33         .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
34         .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border); }
35         .page-title { font-size: 20px; font-weight: 600; }
36         .user-info { display: flex; align-items: center; gap: 16px; }
37         .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid
rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; }
38         .content { flex: 1; overflow-y: auto; padding: 24px; }
39         .view { display: none; }
40         .view.active { display: block; }
41         .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px;
}
42         .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--
border); }
43         .stat-label { font-size: 11px; color: var(--text3); text-transform: uppercase; margin-bottom: 8px; }
44         .stat-value { font-size: 28px; font-weight: 600; }
45         .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 24px; max-width: 600px; }
46         .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
47         .form-group { display: flex; flex-direction: column; gap: 6px; }
48         .form-group.full { grid-column: 1 / -1; }
49         label { font-size: 11px; font-weight: 600; color: var(--text3); text-transform: uppercase; }
50         input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--
border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; }
51         input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
52         .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
overflow-x: auto; }
53         table { width: 100%; border-collapse: collapse; }
54         th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text3); font-size:
11px; font-weight: 600; text-transform: uppercase; }
55         td { padding: 12px 16px; border-top: 1px solid var(--border); }
56         .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-b
lock; }
57         .chip.active, .chip.approved, .chip.delivered, .chip.repair_done { background: var(--green-dim);
color: var(--green); }

```

```

58     .chip.pending, .chip.scheduled, .chip.sent_for_repair { background: var(--amber-dim); color: var(--
amber); }
59     .chip.open { background: var(--accent-dim); color: var(--accent); }
60     .chip.negotiating, .chip.agreed, .chip.paid { background: var(--green-dim); color: var(--green); }
61     .chip.inactive, .chip.rejected, .chip.cancelled, .chip.expired { background: var(--red-dim); color:
var(--red); }
62     .chip.closed { background: var(--bg3); color: var(--text2); }
63     .chip.on_leave { background: var(--violet-dim); color: var(--violet); }
64     .chip.employee { background: var(--accent-dim); color: var(--accent); }
65     .chip.manager { background: var(--violet-dim); color: var(--violet); }
66     .chip.vendor { background: var(--red-dim); color: var(--red); }
67     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
68     .chip.unpaid, .chip.UNPAID { background: var(--violet-dim); color: var(--violet); }
69     .chip.assigned { background: var(--green-dim); color: var(--green); }
70     .chip.condemned { background: var(--red-dim); color: var(--red); }
71     .chip.admin { background: var(--amber-dim); color: var(--amber); }
72     .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size:
13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
73     .btn-primary { background: var(--accent); color: white; }
74     .btn-primary:hover { background: #6ba0ff; }
75     .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
76     .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
77     .btn-sm { padding: 4px 10px; font-size: 11px; margin: 2px; }
78     .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom:
16px; }
79     .section-title { font-size: 16px; font-weight: 600; }
80     .empty-state { text-align: center; padding: 40px; color: var(--text3); }
81     .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
82     .modal-overlay.open { display: flex; }
83     .modal { background: var(--panel); border-radius: 12px; padding: 24px; width: 450px; max-width: 90%;
}
84     .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
85     .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
86     .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index:
1100; animation: slideIn 0.3s ease; }
87     .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid var(--green);
}
88     .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid var(--red); }
89     @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
90     .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
91     .negotiation-layout { display: grid; grid-template-columns: minmax(0, 1fr) 360px; gap: 18px; align-
items: start; }
92     .negotiation-panel { background: var(--panel); border: 1px solid var(--border); border-radius: 12px;
overflow: hidden; }
93     .negotiation-panel-body { padding: 18px; }
94     .message-list { display: flex; flex-direction: column; gap: 10px; max-height: 360px; overflow-y:
auto; margin: 14px 0; }
95     .message-item { background: var(--bg3); border: 1px solid var(--border); border-radius: 8px;
padding: 10px 12px; }
96     .message-meta { color: var(--text3); font-size: 10px; font-family: var(--mono); margin-bottom: 4px;
text-transform: uppercase; }
97     .notif-wrapper { position: relative; cursor: pointer; }
98
99     .notif-badge {
100         position: absolute; top: -5px; right: -6px;
101         background: #e53e3e; color: white;
102         font-size: 10px; font-weight: 600;
103         min-width: 16px; height: 16px;
104         border-radius: 50%; display: flex;
105         align-items: center; justify-content: center;
106         padding: 0 3px;
107     }
108     .notif-panel {
109         position: absolute; top: 48px; right: 0;
110         width: 320px; max-height: 400px; overflow-y: auto;
111         background: var(--panel);
112         border: 1px solid var(--border2);
113         border-radius: 10px;
114         box-shadow: 0 4px 24px rgba(0,0,0,0.4);
115         z-index: 999;
116     }
117     .notif-header {
118         padding: 12px 16px; font-weight: 600;

```

```

119         color: var(--text);
120         border-bottom: 1px solid var(--border);
121     }
122     .notif-item {
123         padding: 12px 16px; font-size: 13px;
124         color: var(--text2);
125         border-bottom: 1px solid var(--border);
126     }
127     .notif-item.unread { background: var(--accent-dim); color: var(--text); }
128     .notif-time { font-size: 11px; color: var(--text3); margin-top: 4px; }
129
130     /* ===== CALENDAR STYLES ===== */
131     .leave-balance-grid {
132         display: grid;
133         grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
134         gap: 14px; padding: 20px;
135         border-bottom: 1px solid var(--border);
136     }
137     .leave-bal-card {
138         background: var(--bg3); border: 1px solid var(--border);
139         border-radius: 10px; padding: 16px 18px;
140         display: flex; flex-direction: column; gap: 6px;
141         position: relative; overflow: hidden; transition: border-color 0.2s;
142     }
143     .leave-bal-card:hover { border-color: var(--border2); }
144     .leave-bal-card::before {
145         content: ''; position: absolute;
146         left: 0; top: 0; bottom: 0; width: 3px;
147         border-radius: 3px 0 0 3px;
148     }
149     .leave-bal-card.sick::before { background: var(--red); }
150     .leave-bal-card.casual::before { background: var(--amber); }
151     .leave-bal-card.mat::before { background: var(--accent); }
152     .leave-bal-card.total::before { background: var(--green); }
153     .lbc-icon { font-size: 18px; }
154     .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px; font-weight: 600; }
155     .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-height: 1; }
156     .lbc-sub { font-size: 11px; color: var(--text3); }
157     .cal-legend {
158         display: flex; align-items: center; gap: 20px;
159         padding: 12px 20px; border-bottom: 1px solid var(--border); flex-wrap: wrap;
160     }
161     .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase; letter-spacing: 0.8px; margin-right: 4px; }
162     .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2); }
163     .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
164     .cal-section { background: var(--panel); border: 1px solid var(--border); border-radius: 12px; margin-bottom: 20px; overflow: hidden; }
165     .cal-section-head { padding: 16px 20px; border-bottom: 1px solid var(--border); display: flex; justify-content: space-between; align-items: center; }
166     .cal-section-title { font-size: 16px; font-weight: 600; color: var(--text); }
167     .cal-section-sub { font-size: 12px; color: var(--text2); margin-top: 2px; }
168     #employee-calendar { padding: 16px 20px 20px; }
169     #employee-calendar .fc .fc-toolbar-title { font-family: var(--font); font-size: 15px; font-weight: 600; color: var(--text); }
170     #employee-calendar .fc .fc-button {
171         background: var(--panel2); border: 1px solid var(--border2);
172         color: var(--text2); font-family: var(--font); font-size: 12px;
173         font-weight: 600; border-radius: 6px; padding: 5px 12px;
174         transition: all 0.2s; box-shadow: none; text-transform: capitalize;
175     }
176     #employee-calendar .fc .fc-button:hover,
177     #employee-calendar .fc .fc-button-active,
178     #employee-calendar .fc .fc-button:not(:disabled):active {
179         background: var(--accent-dim); border-color: var(--accent); color: var(--accent); box-shadow: none;
180     }
181     #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
182     #employee-calendar .fc-theme-standard td,
183     #employee-calendar .fc-theme-standard th,
184     #employee-calendar .fc-theme-standard .fc-scrollgrid { border-color: var(--border); }
185     #employee-calendar .fc .fc-col-header-cell-cushion {
186         font-family: var(--font); font-size: 11px; font-weight: 600;
187         color: var(--text3); text-transform: uppercase; letter-spacing: 0.8px; padding: 8px 4px;
188     }

```

```

189 #employee-calendar .fc .fc-daygrid-day-number { font-family: var(--mono); font-size: 12px; color: var(--
text3); padding: 6px 8px; }
190 #employee-calendar .fc .fc-daygrid-day.fc-day-today { background: var(--accent-dim) !important; }
191 #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number { color: var(--accent); font-
weight: 700; }
192 #employee-calendar .fc-daygrid-day-frame { background: var(--bg3); }
193 #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame { background: var(--bg2); }
194 #employee-calendar .fc-event {
195     border: none !important; border-radius: 4px;
196     font-family: var(--font); font-size: 11px; font-weight: 600; padding: 2px 6px; cursor: pointer;
197 }
198 .cal-tooltip {
199     position: fixed; background: var(--panel2); border: 1px solid var(--border2);
200     border-radius: 8px; padding: 10px 14px; font-size: 12px; color: var(--text);
201     z-index: 9999; pointer-events: none; max-width: 200px;
202     box-shadow: 0 8px 24px rgba(0,0,0,0.4);
203 }
204 .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
205 .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
206     </style>
207 </head>
208 <body>
209 <div class="app">
210     <!-- Sidebar -->
211     <div class="sidebar">
212         <div class="logo">
213             <h2>E<span>M</span>S</h2>
214             <p>Admin Control Center</p>
215         </div>
216         <div style="flex: 1;">
217             <div class="nav-item active" onclick="showView('dashboard', event)">&#9632; Dashboard</div>
218             <div class="nav-item" onclick="showView('employees', event)">&#9632; Employees</div>
219             <div class="nav-item" onclick="showView('add-employee', event)">&#9632; Add Employee</div>
220             <div class="nav-item" onclick="showView('add-manager', event)">&#9632; Add Manager</div>
221             <div class="nav-item" onclick="showView('create-admin', event)">&#9632; Create Admin</div>
222             <div class="nav-item" onclick="showView('add-vendor', event)">&#9632; Add Vendor</div>
223             <div class="nav-item" onclick="window.location.href='/leave_management.html'">&#9632; Leave
Management</div>
224             <div class="nav-item" onclick="showView('holidays', event)">&#9632; Holidays</div>
225             <div class="nav-item" onclick="showView('restaurants', event)">&#9632; Restaurants</div>
226             <div class="nav-item" onclick="showView('subscriptions', event)">&#9632; Subscriptions</div>
227             <div class="nav-item" onclick="showView('add-subscription', event)">&#9632; Add
Subscription</div>
228             <div class="nav-item" onclick="showView('vendors', event)">&#9632; Vendors</div>
229             <div class="nav-item" onclick="showView('vendor-negotiations', event)">&#9632; Vendor
Negotiations</div>
230             <div class="nav-item" onclick="window.location.href='/device_management.html'">&#9632; Device
Management</div>
231             <div class="nav-item" onclick="showView('my-calendar', event)">&#9632; My Calendar</div>
232             <div class="nav-item" onclick="showView('my-profile', event)">&#9632; My Profile</div>
233             <div class="nav-item" onclick="showView('two-factor', event)">&#9632; Two-Factor Auth</div>
234         </div>
235     </div>
236
237     <!-- Main Content -->
238     <div class="main-content">
239         <div class="topbar">
240             <div class="page-title" id="pageTitle">Dashboard</div>
241             <!-- Bell icon with badge -->
242             <div class="notif-wrapper" onclick="toggleNotifPanel()">
243                 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
244                     stroke="currentColor" stroke-width="2" stroke-linecap="round">
245                     <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
246                     <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
247                 </svg>
248                 <span id="notif-badge" class="notif-badge" style="display:none"></span>
249             </div>
250
251             <!-- Dropdown panel -->
252             <div id="notif-panel" class="notif-panel" style="display:none">
253                 <div class="notif-header">Notifications</div>
254                 <div id="notif-list"></div>
255             </div>
256             <div class="user-info">
257                 <span id="username-display"></span>
258                 <button class="logout-btn" onclick="logout()">Logout</button>

```



```

259         </div>
260     </div>
261     <div class="content" id="content">
262
263         <!-- DASHBOARD VIEW -->
264         <div id="dashboard-view" class="view active">
265             <div class="stats-grid">
266                 <div class="stat-card"><div class="stat-label">Total Employees</div><div class="stat-
267 value" id="total-employees">--</div></div>
268                 <div class="stat-card"><div class="stat-label">Active</div><div class="stat-value"
269 id="active-employees">--</div></div>
270                 <div class="stat-card"><div class="stat-label">On Leave</div><div class="stat-value"
271 id="onleave-employees">--</div></div>
272                 <div class="stat-card"><div class="stat-label">Pending Leaves</div><div class="stat-
273 value" id="pending-leaves">--</div></div>
274             </div>
275             <div class="section-head"><div class="section-title">Recent Leave Requests</div><button
276 class="btn btn-secondary btn-sm" onclick="loadDashboard()">Refresh</button></div>
277             <div class="table-
278 wrap"><table><thead><tr><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Status</th></tr></thead><tbo
279 dy id="recent-leaves"></tbody></table></div>
280         </div>
281
282         <!-- EMPLOYEES VIEW -->
283         <div id="employees-view" class="view">
284             <div class="section-head"><div class="section-title">All Employees</div><button class="btn
285 btn-primary btn-sm" onclick="loadEmployees()">Refresh</button></div>
286             <div class="table-
287 wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>
288 >Timezone</th><th>Actions</th></tr></thead><tbody i...
289         </div>
290
291         <div id="add-employee-view" class="view">
292             <div class="section-title" style="margin-bottom: 20px;">Add Employee</div>
293             <div class="form-card">
294                 <div class="form-grid">
295                     <div class="form-group"><label>Name</label><input type="text" id="employee-name"
296 placeholder="Employee name"></div>
297                     <div class="form-group"><label>Email</label><input type="email" id="employee-email"
298 placeholder="employee@company.com"></div>
299                     <div class="form-group"><label>Password</label><input type="password" id="employee-
300 password" placeholder="Temporary password"></div>
301                     <div class="form-group">
302                         <label>Gender</label>
303                         <select id="employee-gender">
304                             <option value="M">Male</option>
305                             <option value="F">Female</option>
306                             <option value="O">Other</option>
307                         </select>
308                     </div>
309                     <div class="form-group full">
310                         <label>Manager</label>
311                         <select id="employee-manager-select">
312                             <option value="">-- Select Manager --</option>
313                         </select>
314                     </div>
315                     <div class="form-group full"><label>Timezone</label>
316                     <select id="employee-timezone">
317                         <option value="Asia/Kolkata">India – IST UTC+5:30</option>
318                         <option value="UTC">UTC</option>
319                         <option value="America/New_York">New York – EST UTC-5</option>
320                         <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
321                         <option value="Europe/London">London – GMT UTC+0</option>
322                         <option value="Europe/Paris">Paris – CET UTC+1</option>
323                         <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
324                         <option value="Asia/Dubai">Dubai – GST UTC+4</option>
325                         <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
326                         <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
327                     </select>
328                     </div>
329                 </div>
330             </div>
331             <div class="modal-actions" style="margin-top: 20px;">
332                 <button class="btn btn-primary" onclick="createEmployee()">Create Employee</button>
333                 <button class="btn btn-secondary"
334 onclick="clearEmployeeForm('employee')">Clear</button>
335             </div>

```

```

321         </div>
322     </div>
323
324     <div id="add-manager-view" class="view">
325         <div class="section-title" style="margin-bottom: 20px;">Add Manager</div>
326         <div class="form-card">
327             <div class="form-grid">
328                 <div class="form-group"><label>Name</label><input type="text" id="manager-name-
create" placeholder="Manager name"></div>
329                 <div class="form-group"><label>Email</label><input type="email" id="manager-email-
create" placeholder="manager@company.com"></div>
330                 <div class="form-group"><label>Password</label><input type="password" id="manager-
password-create" placeholder="Temporary password"></div>
331                 <div class="form-group"><label>Department</label><input type="text" id="manager-
dept-create" placeholder="Department"></div>
332                 <div class="form-group full"><label>Timezone</label>
333                     <select id="manager-timezone-create">
334                         <option value="Asia/Kolkata">India – IST UTC+5:30</option>
335                         <option value="UTC">UTC</option>
336                         <option value="America/New_York">New York – EST UTC-5</option>
337                         <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
338                         <option value="Europe/London">London – GMT UTC+0</option>
339                         <option value="Europe/Paris">Paris – CET UTC+1</option>
340                         <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
341                         <option value="Asia/Dubai">Dubai – GST UTC+4</option>
342                         <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
343                         <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
344                     </select>
345                 </div>
346                 <div class="form-group"><label>Gender</label>
347                     <select id="manager-gender">
348                         <option value="M">Male</option>
349                         <option value="F">Female</option>
350                         <option value="O">Other</option>
351                     </select>
352                 </div>
353             </div>
354             <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="createManager()">Create Manager</button><button class="btn btn-secondary" onclick="...
355             </div>
356         </div>
357
358     <!-- CREATE ADMIN VIEW -->
359     <div id="create-admin-view" class="view">
360         <div class="section-title" style="margin-bottom: 20px;">Create Admin</div>
361         <div class="form-card">
362             <div class="form-grid">
363                 <div class="form-group">
364                     <label>Name</label>
365                     <input type="text" id="admin-name" placeholder="Admin name">
366                 </div>
367                 <div class="form-group">
368                     <label>Email</label>
369                     <input type="email" id="admin-email" placeholder="admin@company.com">
370                 </div>
371                 <div class="form-group">
372                     <label>Password</label>
373                     <input type="password" id="admin-password" placeholder="Temporary password">
374                 </div>
375                 <div class="form-group">
376                     <label>Gender</label>
377                     <select id="admin-gender">
378                         <option value="M">Male</option>
379                         <option value="F">Female</option>
380                         <option value="O">Other</option>
381                     </select>
382                 </div>
383                 <div class="form-group full">
384                     <label>Timezone</label>
385                     <select id="admin-timezone">
386                         <option value="Asia/Kolkata">India – IST UTC+5:30</option>
387                         <option value="UTC">UTC</option>
388                         <option value="America/New_York">New York – EST UTC-5</option>
389                         <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
390                         <option value="Europe/London">London – GMT UTC+0</option>
391                         <option value="Europe/Paris">Paris – CET UTC+1</option>

```

```

392         <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
393         <option value="Asia/Dubai">Dubai – GST UTC+4</option>
394         <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
395         <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
396     </select>
397 </div>
398 </div>
399     <div class="modal-actions" style="margin-top: 20px;">
400         <button class="btn btn-primary" onclick="createAdmin()">Create Admin</button>
401         <button class="btn btn-secondary" onclick="clearAdminForm()">Clear</button>
402     </div>
403 </div>
404 </div>
405
406     <div id="add-vendor-view" class="view">
407         <div class="section-title" style="margin-bottom: 20px;">Add Vendor</div>
408         <div class="form-card">
409             <div class="form-grid">
410                 <div class="form-group"><label>Name</label><input type="text" id="vendor-name-
create" placeholder="Vendor name"></div>
411                 <div class="form-group"><label>Email</label><input type="email" id="vendor-email-
create" placeholder="vendor@company.com"></div>
412                 <div class="form-group"><label>Password</label><input type="password" id="vendor-
password-create" placeholder="Temporary password"></div>
413                 <div class="form-group"><label>Phone</label><input type="text" id="vendor-phone-
create" placeholder="+91 90000 00000"></div>
414                 <div class="form-group">
415                     <label>Role</label>
416                     <select id="vendor-role">
417                         <option value="TECH_VENDOR">tech vendor</option>
418                         <option value="FOOD_VENDOR">Food Vendor</option>
419                     </select>
420                 </div>
421             </div>
422             <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="createVendor()">Create Vendor</button><button class="btn btn-secondary" onclick="cl...
423             </div>
424         </div>
425
426     <!-- APPLY LEAVE VIEW -->
427     <div id="apply-leave-view" class="view">
428         <div class="section-title" style="margin-bottom: 20px;">Apply for Leave</div>
429         <div class="form-card">
430             <div class="form-grid">
431                 <div class="form-group full"><label>Logged In User</label><input type="email"
id="leave-email" placeholder="your.email@company.com" readonly></div>
432                 <div class="form-group"><label>Leave Type</label><select id="leave-type"><option
value="SICK">Sick Leave</option><option value="CASUAL">Casual Leave</option><option value="M...
433                 Sick leave is limited to <strong>1 day per month</strong>. If already used
this month, please select Unpaid Leave.
434             </div>
435             <div id="casual-leave-hint" style="display:none; margin-top:6px; font-
size:11px; color:var(--amber); background:var(--amber-dim); border:1px solid rgba(251,191,36,0.2); ...
436             Casual leave is limited to <strong>1 day per month</strong>. If already
used this month, it will be automatically recorded as <strong>Unpaid Leave</strong>.
437             </div>
438         </div>
439         <div class="form-group"><label>Start Date</label><input type="date" id="leave-
start"></div>
440         <div class="form-group"><label>End Date</label><input type="date" id="leave-
end"></div>
441         <div class="form-group full"><label>Reason</label><textarea id="leave-reason"
rows="3" placeholder="Please provide reason for leave..."></textarea></div>
442     </div>
443     <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="applyLeave()">Submit Leave Request</button><button class="btn btn-secondary" oncl...
444     </div>
445 </div>
446
447     <!-- HOLIDAYS VIEW -->
448     <div id="holidays-view" class="view">
449         <div class="section-head">
450             <div class="section-title">Fixed Holidays</div>
451             <button class="btn btn-primary btn-sm" onclick="loadHolidays()">Refresh</button>
452         </div>
453

```

```
454         <!-- Add Holiday Form -->
455         <div class="form-card" style="margin-bottom: 24px;">
456             <div class="form-grid">
457                 <div class="form-group">
458                     <label>Holiday Name</label>
459                     <input type="text" id="h-name" placeholder="e.g. Diwali, Christmas">
460                 </div>
461                 <div class="form-group">
462                     <label>Date</label>
463                     <input type="date" id="h-date">
464                 </div>
465                 <div class="form-group full">
466                     <label>Description (optional)</label>
467                     <input type="text" id="h-desc" placeholder="Brief description">
468                 </div>
469             </div>
470             <div class="modal-actions" style="margin-top: 20px;">
471                 <button class="btn btn-primary" onclick="addHoliday()">Add Holiday</button>
472                 <button class="btn btn-secondary" onclick="clearHolidayForm()">Clear</button>
473             </div>
474         </div>
475
476         <!-- Holidays Table -->
477         <div class="table-wrap">
478             <table>
479                 <thead>
480                     <tr>
481                         <th>ID</th>
482                         <th>Name</th>
483                         <th>Date</th>
484                         <th>Description</th>
485                         <th>Action</th>
486                     </tr>
487                 </thead>
488                 <tbody id="holidays-list">
489                     <tr><td colspan="5" class="empty-state">Loading...</td></tr>
490                 </tbody>
491             </table>
492         </div>
493     </div>
494
495     <!-- MY LEAVES VIEW -->
496     <div id="my-leaves-view" class="view">
497         <div class="section-head"><div class="section-title">My Leave Requests</div><button
498         class="btn btn-primary btn-sm" onclick="loadMyLeaves()">Refresh</button></div>
499         <div class="table-wrap"><table><thead><tr><th>ID</th><th>Type</th><th>Start
500         Date</th><th>End Date</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead><tbody id="my-leaves-...
501         </div>
502     </div>
503
504     <!-- PENDING LEAVES VIEW -->
505     <div id="pending-leaves-view" class="view">
506         <div class="section-head"><div class="section-title">Pending Leave Approvals</div><button
507         class="btn btn-primary btn-sm" onclick="loadPendingLeaves()">Refresh</button></div>
508         <div class="table-wrap"><table><thead><tr><th>ID</th><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Reason</th><th>Ac
509         tions</th></tr></thead><tbody id="pending-leaves-li...
510         </div>
511     </div>
512
513     <!-- RESTAURANTS VIEW -->
514     <div id="restaurants-view" class="view">
515         <div class="section-head"><div class="section-title">Restaurants</div><div><select
516         id="slot-filter" onchange="loadRestaurants()"><option value="">All Slots</option><option value="BR...
517         <div class="table-wrap"><table><thead><tr><th>Name</th><th>Address</th><th>Meal
518         Slots</th><th>Status</th><th>Vendor</th></tr></thead><tbody id="restaurants-list"></tbody></table></div>
519     </div>
520
521     <!-- MY SUBSCRIPTIONS VIEW -->
522     <div id="subscriptions-view" class="view">
523         <div class="section-head"><div class="section-title">My Meal Subscriptions</div><button
524         class="btn btn-primary btn-sm" onclick="loadSubscriptions()">Refresh</button></div>
525         <div class="table-wrap"><table><thead><tr><th>ID</th><th>Restaurant</th><th>Meal
526         Slot</th><th>Schedule</th><th>Status</th><th>Next Delivery</th><th>Actions</th></tr></thead><tbody i...
527     </div>
528
529     <!-- ADD SUBSCRIPTION VIEW -->
530     <div id="add-subscription-view" class="view">
```

```

521         <div class="section-title" style="margin-bottom: 20px;">Create Meal Subscription</div>
522         <div class="form-card">
523             <div class="form-grid">
524                 <div class="form-group full">
525                     <label>Employee Email</label>
526                     <input type="email" id="sub-email" placeholder="employee@company.com">
527                 </div>
528                 <div class="form-group full">
529                     <label>Restaurant</label>
530                     <select id="sub-restaurant-select">
531                         <option value="">-- Select Restaurant --</option>
532                     </select>
533                 </div>
534                 <div class="form-group full">
535                     <label>Meal Slots</label>
536                     <div id="meal-slot-checkboxes" style="display:flex; gap:16px; flex-wrap:wrap;
margin-top:6px;">
537                         <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
538                             <input type="checkbox" value="BREAKFAST"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Breakfast
539                         </label>
540                         <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
541                             <input type="checkbox" value="LUNCH"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Lunch
542                         </label>
543                         <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
544                             <input type="checkbox" value="DINNER"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Dinner
545                         </label>
546                     </div>
547                 </div>
548                 <div class="form-group">
549                     <label>Schedule Type</label>
550                     <select id="sub-schedule">
551                         <option value="DAILY">Daily</option>
552                         <option value="WEEKLY">Weekly</option>
553                     </select>
554                 </div>
555                 <div class="form-group" id="day-of-week-group" style="display:none;">
556                     <label>Day of Week</label>
557                     <select id="sub-day">
558                         <option value="MONDAY">Monday</option>
559                         <option value="TUESDAY">Tuesday</option>
560                         <option value="WEDNESDAY">Wednesday</option>
561                         <option value="THURSDAY">Thursday</option>
562                         <option value="FRIDAY">Friday</option>
563                         <option value="SATURDAY">Saturday</option>
564                         <option value="SUNDAY">Sunday</option>
565                     </select>
566                 </div>
567             </div>
568             <div class="modal-actions" style="margin-top: 20px;">
569                 <button class="btn btn-primary" onclick="addSubscription()">Create
Subscription</button>
570                 <button class="btn btn-secondary" onclick="clearSubscriptionForm()">Clear</button>
571             </div>
572         </div>
573     </div>
574
575     <!-- DELIVERIES VIEW -->
576     <div id="deliveries-view" class="view">
577         <div class="section-head"><div class="section-title">My Deliveries</div><button class="btn
btn-primary btn-sm" onclick="loadDeliveries()">Refresh</button></div>
578         <div class="table-wrap"><table><thead><tr><th>ID</th><th>Subscription ID</th><th>Meal
Slot</th><th>Scheduled Time</th><th>Status</th></tr></thead><tbody id="deliveries-list"></tbody...
579     </div>
580
581     <!-- VENDORS VIEW -->
582     <div id="vendors-view" class="view">
583         <div class="section-head"><div class="section-title">Vendors</div><button class="btn btn-
primary btn-sm" onclick="loadVendors()">Refresh</button></div>
584         <div class="table-
wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th><th>Registered</th><th>Actions</th><

```

```

/tr></thead><tbody id="vendors-list"></tbody></tbl...
585         </div>
586
587         <!-- VENDOR NEGOTIATIONS VIEW -->
588         <div id="vendor-negotiations-view" class="view">
589             <div class="section-head">
590                 <div class="section-title">Vendor Negotiations</div>
591                 <button class="btn btn-primary btn-sm"
onclick="loadVendorNegotiations()">Refresh</button>
592             </div>
593             <div class="negotiation-layout">
594                 <div>
595                     <div class="table-wrap" style="margin-bottom:18px">
596                         <table>
597
<thead><tr><th>ID</th><th>Vendor</th><th>Device</th><th>Subject</th><th>Amount</th><th>Status</th><th>Updated</t
h><th>Actions</th></tr></thead>
598                             <tbody id="vendor-negotiations-list"><tr><td colspan="8" class="empty-
state">Loading...</td></tr></tbody>
599                         </table>
600                     </div>
601                     <div class="form-card" style="max-width:none">
602                         <div class="section-title" style="margin-bottom:14px">Start Negotiation</div>
603                         <div class="form-grid">
604                             <div class="form-group">
605                                 <label>Vendor</label>
606                                 <select id="neg-vendor-id"><option value="">-- Select Vendor --
</option></select>
607                             </div>
608                             <div class="form-group">
609                                 <label>Device</label>
610                                 <select id="neg-device-id"><option value="">No device</option></select>
611                             </div>
612                             <div class="form-group">
613                                 <label>Proposed Amount</label>
614                                 <input id="neg-amount" type="number" min="0" step="0.01"
placeholder="2500">
615                             </div>
616                             <div class="form-group">
617                                 <label>Subject</label>
618                                 <input id="neg-subject" placeholder="Repair cost discussion">
619                             </div>
620                             <div class="form-group full">
621                                 <label>Description</label>
622                                 <textarea id="neg-description" rows="3" placeholder="Scope, issue, or
billing context"></textarea>
623                             </div>
624                             <div class="form-group full">
625                                 <label>Initial Message</label>
626                                 <textarea id="neg-initial-message" rows="3" placeholder="Message to the
vendor"></textarea>
627                             </div>
628                         </div>
629                         <div style="margin-top:16px;display:flex;gap:10px">
630                             <button class="btn btn-primary" onclick="createVendorNegotiation()">Create
Negotiation</button>
631                             <button class="btn btn-secondary"
onclick="clearVendorNegotiationForm()">Clear</button>
632                         </div>
633                     </div>
634                 </div>
635                 <div class="negotiation-panel">
636                     <div class="section-head"><div class="section-title" id="neg-detail-
title">Negotiation Details</div></div>
637                     <div class="negotiation-panel-body" id="neg-detail-body">
638                         <div class="empty-state">Select a negotiation to view messages.</div>
639                     </div>
640                 </div>
641             </div>
642         </div>
643
644         <!-- DEVICES VIEW -->
645         <div id="devices-view" class="view">
646             <div class="section-head">
647                 <div class="section-title">Unassigned Devices</div>
648                 <button class="btn btn-primary btn-sm"

```

```

onclick="loadUnassignedDevices()">Refresh</button>
649         </div>
650         <div class="table-wrap">
651             <table>
652                 <thead>
653                     <tr>
654                         <th>ID</th>
655                         <th>Device Name</th>
656                         <th>Brand</th>
657                         <th>Type</th>
658                         <th>Warranty</th>
659                         <th>Vendor</th>
660                         <th>Actions</th>
661                     </tr>
662                 </thead>
663                 <tbody id="devices-list"></tbody>
664             </table>
665         </div>
666     </div>
667
668     <!-- ASSIGN DEVICE MODAL -->
669     <div id="assign-modal" class="modal-overlay">
670         <div class="modal">
671             <div class="modal-title">Assign Device to Employee</div>
672             <input type="hidden" id="assign-device-id">
673             <div class="form-group" style="margin-bottom:12px">
674                 <label>Device</label>
675                 <input type="text" id="assign-device-name" readonly style="color:var(--text2)">
676             </div>
677             <div class="form-group" style="margin-bottom:12px">
678                 <label>Employee</label>
679                 <select id="assign-employee-select">
680                     <option value="">-- Select Employee --</option>
681                 </select>
682             </div>
683             <div class="modal-actions">
684                 <button class="btn btn-primary" onclick="confirmAssign()">Assign</button>
685                 <button class="btn btn-secondary" onclick="closeAssignModal()">Cancel</button>
686             </div>
687         </div>
688     </div>
689
690     <!-- MY DEVICES VIEW -->
691     <div id="my-devices-view" class="view">
692         <div class="section-head">
693             <div class="section-title">My Assigned Devices</div>
694             <button class="btn btn-primary btn-sm"
onclick="loadMyAssignedDevices()">Refresh</button>
695         </div>
696         <div class="table-wrap">
697             <table>
698                 <thead>
699                     <tr>
700                         <th>ID</th>
701                         <th>Device Name</th>
702                         <th>Brand</th>
703                         <th>Type</th>
704                         <th>Warranty</th>
705                         <th>Assigned On</th>
706                         <th>Status</th>
707                     </tr>
708                 </thead>
709                 <tbody id="my-devices-list"></tbody>
710             </table>
711         </div>
712     </div>
713
714     <!-- MY SERVICE REQUESTS VIEW -->
715     <div id="service-requests-view" class="view">
716         <div class="section-head">
717             <div class="section-title">My Service Requests</div>
718             <button class="btn btn-primary btn-sm" onclick="loadServiceRequests()">Refresh</button>
719         </div>
720         <div class="table-wrap" style="margin-bottom:24px">
721             <table>
722                 <thead><tr>

```

```

723         <th>ID</th><th>Device</th><th>Type</th>
724         <th>Description</th><th>Urgent</th><th>Status</th><th>Raised At</th>
725     </tr></thead>
726     <tbody id="service-requests-list"></tbody>
727 </table>
728 </div>
729
730 <div class="section-title" style="margin-bottom:16px">Raise a New Request</div>
731 <div class="form-card">
732     <div class="form-grid">
733         <div class="form-group full">
734             <label>Device</label>
735             <select id="sr-device-select">
736                 <option value="">-- Select your device --</option>
737             </select>
738         </div>
739         <div class="form-group">
740             <label>Request Type</label>
741             <select id="sr-type">
742                 <option value="REPAIR">Repair</option>
743                 <option value="REPLACEMENT">Replacement</option>
744                 <option value="UPGRADE">Upgrade</option>
745                 <option value="MAINTENANCE">Maintenance</option>
746                 <option value="RETURN">Return</option>
747             </select>
748         </div>
749         <div class="form-group" style="display:flex;align-items:center;gap:10px;padding-top:20px">
750             <input type="checkbox" id="sr-urgent" style="width:16px;height:16px;padding:0">
751             <label style="text-transform:none;font-size:13px;font-weight:400;color:var(--text);cursor:pointer" for="sr-urgent">Mark as Urgent</label>
752         </div>
753         <div class="form-group full">
754             <label>Issue Description</label>
755             <textarea id="sr-description" rows="3" placeholder="Describe the issue in detail..."></textarea>
756         </div>
757     </div>
758     <div class="modal-actions" style="margin-top:20px">
759         <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
760         Request</button>
761         <button class="btn btn-secondary"
762         onclick="clearServiceRequestForm()">Clear</button>
763     </div>
764
765     <!-- MY CALENDAR VIEW -->
766     <div id="my-calendar-view" class="view">
767         <div class="cal-section">
768             <div class="cal-section-head">
769                 <div>
770                     <div class="cal-section-title">My Leave Calendar</div>
771                     <div class="cal-section-sub">All approved leaves and public holidays</div>
772                 </div>
773                 <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
774                 Refresh</button>
775             </div>
776
777     <!-- Balance Cards -->
778     <div class="leave-balance-grid" style="grid-template-columns: 1fr 1fr;">
779
780         <!-- Sick Leave Card -->
781         <div class="leave-bal-card sick">
782             <span class="lbc-icon"></span>
783             <span class="lbc-label">Sick Leave</span>
784             <span class="lbc-count" id="cal-sick-remaining">--</span>
785             <span class="lbc-sub">day available this month</span>
786             <div id="cal-sick-month-badge" style="
787             display:inline-flex; align-items:center; gap:5px;
788             margin-top:8px; padding:4px 10px;
789             border-radius:20px; font-size:10px; font-weight:700;
790             letter-spacing:0.8px; text-transform:uppercase;
791             ">...</div>
792             <span class="lbc-sub" id="cal-sick-accrued-sub" style="margin-top:4px"></span>
793         </div>

```



```

793
794         <!-- Casual Leave Card -->
795         <div class="leave-bal-card casual">
796             <span class="lbc-icon"></span>
797             <span class="lbc-label">Casual Leave</span>
798             <span class="lbc-count" id="cal-casual-remaining">--</span>
799             <span class="lbc-sub" id="cal-casual-sub">loading...</span>
800             <div style="margin-top:10px;display:flex;flex-direction:column;gap:3px">
801                 <span class="lbc-sub" id="cal-casual-opening"></span>
802                 <span class="lbc-sub" id="cal-casual-accrued"></span>
803                 <span class="lbc-sub" id="cal-casual-used"></span>
804             </div>
805         </div>
806
807     </div>
808
809     <!-- Legend -->
810     <div class="cal-legend">
811         <span class="legend-label">Legend:</span>
812         <div class="legend-item"><div class="legend-dot"
813 style="background:#36d399"></div>Approved</div>
814         <div class="legend-item"><div class="legend-dot"
815 style="background:#f9c74f"></div>Pending</div>
816         <div class="legend-item"><div class="legend-dot"
817 style="background:#f87171"></div>Rejected</div>
818         <div class="legend-item"><div class="legend-dot"
819 style="background:#a78bfa"></div>Holiday</div>
820     </div>
821
822     <!-- Calendar -->
823     <div id="employee-calendar"></div>
824 </div>
825
826 <!-- MY PROFILE VIEW -->
827 <div id="my-profile-view" class="view">
828     <div class="section-title" style="margin-bottom:20px">My Profile</div>
829
830     <!-- Profile Info -->
831     <div class="form-card" style="max-width:100%;margin-bottom:24px">
832         <div class="form-grid">
833             <div class="form-group">
834                 <label>Name</label>
835                 <input type="text" id="p-name" readonly style="color:var(--text2)">
836             </div>
837             <div class="form-group">
838                 <label>Email</label>
839                 <input type="text" id="p-email" readonly style="color:var(--text2)">
840             </div>
841             <div class="form-group">
842                 <label>Department</label>
843                 <input type="text" id="p-dept" readonly style="color:var(--text2)">
844             </div>
845             <div class="form-group">
846                 <label>Role</label>
847                 <input type="text" id="p-role" readonly style="color:var(--text2)">
848             </div>
849             <div class="form-group">
850                 <label>Status</label>
851                 <input type="text" id="p-status" readonly style="color:var(--text2)">
852             </div>
853             <div class="form-group">
854                 <label>Timezone</label>
855                 <input type="text" id="p-tz" readonly style="color:var(--text2)">
856             </div>
857         </div>
858     </div>
859
860     <!-- Maternity Section – only visible for female admins -->
861     <div id="maternity-apply-btn" style="display:none">
862         <div class="form-card" style="max-width:100%">
863             <div class="section-title" style="margin-bottom:20px">Maternity Leave</div>
864             <div class="form-grid">
865                 <div class="form-group">
866                     <label>Start Date</label>
867                     <input type="date" id="mat-start-date">
868                 </div>

```

```

865         </div>
866         <div class="form-group">
867             <label>End Date (Auto)</label>
868             <input type="text" id="mat-end-date"
869                 placeholder="Auto: start + 180 days"
870                 disabled style="color:var(--text2)">
871         </div>
872         <div class="form-group full">
873             <label>Reason / Notes</label>
874             <textarea id="mat-reason" rows="3"
875                 placeholder="Optional notes..."></textarea>
876         </div>
877         <div class="form-group full">
878             <div style="background:var(--accent-dim);
879                 border:1px solid rgba(79,140,255,0.2);
880                 border-radius:8px;padding:12px 16px;
881                 font-size:12px;color:var(--text2)">
882                 Maternity leave is fixed at
883                 <strong style="color:var(--accent)">180 days</strong>.
884                 End date is auto-calculated. You can apply
885                 <strong style="color:var(--accent)">once per year</strong>.
886             </div>
887         </div>
888     </div>
889     <div class="modal-actions" style="margin-top:20px">
890         <button class="btn btn-primary" onclick="applyMaternityLeave()">
891             Submit Maternity Leave
892         </button>
893     </div>
894 </div>
895 </div>
896 </div>
897
898 <!-- 2FA VIEW -->
899 <div id="two-factor-view" class="view">
900     <div class="section-title" style="margin-bottom:20px"> Two-Factor Authentication</div>
901
902     <div class="form-card" style="max-width:500px;margin-bottom:24px">
903         <!-- Status -->
904         <div style="display:flex;align-items:center;justify-content:space-between;margin-
bottom:20px">
905             <div>
906                 <div style="font-size:14px;font-weight:600;color:var(--text)">Authenticator App
(TOTP)</div>
907                 <div style="font-size:12px;color:var(--text2);margin-top:4px">
908                     Google Authenticator, Authy, or any TOTP app
909                 </div>
910             </div>
911             <span id="totp-status-chip" class="chip">Checking...</span>
912         </div>
913
914         <!-- When TOTP is DISABLED – show setup -->
915         <div id="totp-setup-section">
916             <p style="font-size:13px;color:var(--text2);margin-bottom:16px">
917                 Scan the QR code with your authenticator app, then enter the 6-digit code to
activate.
918             </p>
919             <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-btn">
920                 Generate QR Code
921             </button>
922
923             <!-- QR code area – hidden until generated -->
924             <div id="qr-area" style="display:none;margin-top:20px;text-align:center">
925                 <img id="qr-img" src="" alt="QR Code"
926                     style="width:180px;height:180px;border-radius:10px;border:1px solid var(--
border2)">
927                 <div style="margin-top:12px;font-size:11px;color:var(--text2)">
928                     Can't scan? Enter this key manually:
929                 </div>
930                 <div id="totp-secret-display"
931                     style="font-family:var(--mono);font-size:13px;color:var(--accent);
932                     background:var(--bg3);border:1px solid var(--border2);
933                     border-radius:6px;padding:8px 12px;margin-top:6px;
934                     word-break:break-all;text-align:center"></div>
935
936             <!-- Verify step -->

```

```

937         <div style="margin-top:20px">
938             <label style="font-size:11px;color:var(--text3);font-weight:600;
939                 text-transform:uppercase;display:block;margin-bottom:8px">
940                 Enter 6-digit code to activate
941             </label>
942             <input id="totp-verify-input" type="text" inputmode="numeric"
943                 maxlength="6" placeholder="000000"
944                 style="text-align:center;font-size:20px;font-family:var(--mono);
945                 letter-spacing:8px;padding:12px;width:100%"
946                 oninput="this.value=this.value.replace(/\D/g, '')">
947             <button class="btn btn-primary" onclick="verifyAndActivateTotp()"
948                 style="width:100%;margin-top:12px">
949                 Activate 2FA
950             </button>
951             <div id="totp-verify-error"
952                 style="display:none;margin-top:8px;font-size:12px;color:var(--
red)"></div>
953         </div>
954     </div>
955 </div>
956
957 <!-- When TOTP is ENABLED – show disable -->
958 <div id="totp-disable-section" style="display:none">
959     <div style="background:var(--green-dim);border:1px solid rgba(54,211,153,0.2);
960         border-radius:8px;padding:12px 16px;font-size:13px;
961         color:var(--green);margin-bottom:16px">
962         Two-factor authentication is active on your account.
963     </div>
964     <button class="btn btn-danger" onclick="disableTotp()">
965         Disable 2FA
966     </button>
967 </div>
968 </div>
969 </div>
970
971 <!-- ===== ADMIN SERVICE REQUESTS VIEW ===== -->
972 <div id="admin-service-requests-view" class="view">
973     <div class="section-head">
974         <div class="section-title">All Open Service Requests</div>
975         <button class="btn btn-primary btn-sm"
onclick="loadAdminServiceRequests()">Refresh</button>
976     </div>
977     <div class="table-wrap">
978         <table>
979             <thead>
980                 <tr>
981                     <th>ID</th>
982                     <th>Employee</th>
983                     <th>Device</th>
984                     <th>Type</th>
985                     <th>Description</th>
986                     <th>Urgent</th>
987                     <th>Status</th>
988                     <th>Raised At</th>
989                     <th>Actions</th>
990                 </tr>
991             </thead>
992             <tbody id="admin-service-requests-list"></tbody>
993         </table>
994     </div>
995 </div>
996
997 </div>
998 </div>
999 </div>
1000
1001 <!-- Approve/Reject Modal -->
1002 <div id="action-modal" class="modal-overlay">
1003     <div class="modal">
1004         <div class="modal-title" id="modal-title">Approve Leave Request</div>
1005         <input type="hidden" id="action-leave-id">
1006         <input type="hidden" id="action-type">
1007         <div class="form-group"><label>Reviewed By</label><input type="email" id="manager-email"
placeholder="admin@company.com" readonly></div>
1008         <div class="form-group"><label>Remarks</label><textarea id="action-remarks" rows="3"
placeholder="Add remarks..."></textarea></div>

```

```

1009         <div class="modal-actions"><button class="btn btn-primary" id="action-btn"
onclick="submitApproval()">Approve</button><button class="btn btn-secondary"
onclick="closeModal()">Cancel</button>...
1010     </div>
1011 </div>
1012
1013 <!-- Cancel Leave Modal -->
1014 <div id="cancel-modal" class="modal-overlay">
1015     <div class="modal">
1016         <div class="modal-title">Cancel Leave Request</div>
1017         <input type="hidden" id="cancel-leave-id">
1018         <div class="form-group"><label>Cancelled By</label><input type="email" id="cancel-email"
placeholder="admin@company.com" readonly></div>
1019         <div class="modal-actions"><button class="btn btn-danger" onclick="confirmCancelLeave()">Cancel
Leave</button><button class="btn btn-secondary" onclick="closeCancelModal()">Close</button></div>
1020     </div>
1021 </div>
1022
1023 <!-- Admin Service Request Action Modal -->
1024 <div id="admin-sr-modal" class="modal-overlay">
1025     <div class="modal">
1026         <div class="modal-title">Update Service Request <span id="admin-sr-modal-id" style="color:var(--
accent)"></span></div>
1027         <input type="hidden" id="admin-sr-id">
1028
1029         <div class="form-group" style="margin-bottom:14px">
1030             <label>Device</label>
1031             <input type="text" id="admin-sr-device-display" readonly style="color:var(--text2)">
1032         </div>
1033         <div class="form-group" style="margin-bottom:14px">
1034             <label>Employee</label>
1035             <input type="text" id="admin-sr-employee-display" readonly style="color:var(--text2)">
1036         </div>
1037         <div class="form-group" style="margin-bottom:14px">
1038             <label>Update Status</label>
1039             <select id="admin-sr-status">
1040                 <option value="SENT_FOR_REPAIR">Sent for Repair – device will be marked Under
Repair</option>
1041                 <option value="REPAIR_DONE">Repair Done</option>
1042                 <option value="REJECTED">Rejected</option>
1043             </select>
1044         </div>
1045         <div class="form-group" style="margin-bottom:4px">
1046             <label>Admin Remarks</label>
1047             <textarea id="admin-sr-remarks" rows="3" placeholder="Add remarks or instructions for the
vendor..."></textarea>
1048         </div>
1049         <div id="admin-sr-repair-hint" style="font-size:11px;color:var(--amber);margin-
top:6px;display:none;">
1050             Selecting "Sent for Repair" will automatically mark the device as <strong>Under
Repair</strong>.
1051         </div>
1052
1053         <div class="modal-actions">
1054             <button class="btn btn-primary" onclick="submitAdminSRUpdate()">Update Request</button>
1055             <button class="btn btn-secondary" onclick="closeAdminSRModal()">Cancel</button>
1056         </div>
1057     </div>
1058 </div>
1059
1060 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
1061
1062 <script>
1063     const API = 'http://localhost:8080';
1064     let currentUser = null;
1065     const ROLE_DASHBOARDS = {
1066         EMPLOYEE: '/employee-dashboard.html',
1067         MANAGER: '/manager-dashboard.html',
1068         VENDOR: '/vendor-dashboard.html',
1069         ADMIN: '/dashboard.html'
1070     };
1071
1072     (async function initAuth() {
1073         const token = localStorage.getItem('jwt_token');
1074         const authType = localStorage.getItem('auth_type');
1075         if (!token && authType !== 'session') {

```

```

1076         window.location.href = '/google.html';
1077         return;
1078     }
1079
1080     try {
1081         const response = await fetch(`${API}/employee/me`, {
1082             headers: getAuthHeader(),
1083             credentials: 'include'
1084         });
1085
1086         if (response.status === 401) { window.location.href = '/google.html'; return; }
1087         if (!response.ok) { window.location.href = '/google.html'; return; }
1088
1089         const employee = await response.json();
1090         const role = employee.role || 'EMPLOYEE';
1091         if (role !== 'ADMIN') {
1092             window.location.href = ROLE_DASHBOARDS[role] || '/employee-dashboard.html';
1093             return;
1094         }
1095
1096         currentUser = employee;
1097         localStorage.setItem('last_role', role);
1098         document.getElementById('username-display').textContent = employee.email || employee.name ||
1099 'admin';
1100         document.getElementById('leave-email').value = employee.email || '';
1101         document.getElementById('manager-email').value = employee.email || '';
1102         document.getElementById('cancel-email').value = employee.email || '';
1103         loadDashboard();
1104     } catch (e) {
1105         window.location.href = '/google.html';
1106     }
1107 })();
1108
1109 function getAuthHeader() {
1110     const token = localStorage.getItem('jwt_token');
1111     return token ? { 'Authorization': `Bearer ${token}` } : {};
1112 }
1113
1114 async function apiFetch(path, opts = {}) {
1115     try {
1116         const response = await fetch(`${API}${path}`, {
1117             ...opts,
1118             headers: {
1119                 'Content-Type': 'application/json',
1120                 ...getAuthHeader(),
1121                 ...opts.headers
1122             },
1123             credentials: 'include'
1124         });
1125
1126         if (response.status === 401) {
1127             localStorage.removeItem('jwt_token');
1128             localStorage.removeItem('auth_type');
1129             showAlert('Session expired. Please login again.', 'error');
1130             setTimeout(() => window.location.href = '/google.html', 1500);
1131             throw new Error('Unauthorized');
1132         }
1133
1134         if (!response.ok) {
1135             const error = await response.text();
1136             throw new Error(error || 'Request failed');
1137         }
1138
1139         const text = await response.text();
1140         if (!text) return null;
1141         try { return JSON.parse(text); } catch { return text; }
1142     } catch (e) {
1143         if (!e.message.includes('Unauthorized')) showAlert(e.message, 'error');
1144         throw e;
1145     }
1146 }
1147
1148 function showAlert(message, type) {
1149     const alert = document.createElement('div');
1150     alert.className = `alert alert-${type}`;
1151     alert.textContent = message;

```

```

1151     document.body.appendChild(alert);
1152     setTimeout(() => alert.remove(), 3000);
1153 }
1154
1155 function showView(viewName, event) {
1156     document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
1157     document.getElementById(`${viewName}-view`).classList.add('active');
1158     document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
1159     if (event && event.target) event.target.classList.add('active');
1160     document.getElementById('pageTitle').textContent = viewName.split('-').map(w =>
1161         w.charAt(0).toUpperCase() + w.slice(1)).join(' ');
1162     const loaders = {
1163         'dashboard': loadDashboard,
1164         'employees': loadEmployees,
1165         'add-employee': loadManagersDropdown,
1166         'holidays': loadHolidays,
1167         'add-manager': () => {},
1168         'add-vendor': () => {},
1169         'my-leaves': loadMyLeaves,
1170         'pending-leaves': loadPendingLeaves,
1171         'restaurants': loadRestaurants,
1172         'subscriptions': loadSubscriptions,
1173         'add-subscription': loadRestaurantDropdown,
1174         'deliveries': loadDeliveries,
1175         'vendors': loadVendors,
1176         'vendor-negotiations': loadVendorNegotiations,
1177         'devices': loadUnassignedDevices,
1178         'my-devices': loadMyAssignedDevices,
1179         'service-requests': loadServiceRequests,
1180         'admin-service-requests': loadAdminServiceRequests,
1181         'my-profile': loadAdminProfile,
1182         'two-factor': loadTotpStatus,
1183         'my-calendar': loadMyCalendar,
1184         'create-admin': () => {}
1185     };
1186     if (loaders[viewName]) loaders[viewName]();
1187 }
1188
1189 // ===== DASHBOARD =====
1190 async function loadDashboard() {
1191     try {
1192         const [employees, leaves] = await Promise.all([
1193             apiFetch('/employee'),
1194             apiFetch('/leave_requests')
1195         ]);
1196         document.getElementById('total-employees').textContent = employees.length;
1197         document.getElementById('active-employees').textContent = employees.filter(e => e.status ===
1198 'ACTIVE').length;
1199         document.getElementById('onleave-employees').textContent = employees.filter(e => e.status ===
1200 'ON_LEAVE').length;
1201         document.getElementById('pending-leaves').textContent = leaves.filter(l => l.status ===
1202 'PENDING').length;
1203         const recent = leaves.slice(-5).reverse();
1204         const tbody = document.getElementById('recent-leaves');
1205         if (!recent.length) {
1206             tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
1207         } else {
1208             tbody.innerHTML = recent.map(l => `<tr>
1209                 <td>${l.employeeName || 'N/A'}</td>
1210                 <td><span class="chip">${l.leaveType || 'N/A'}</span></td>
1211                 <td>${l.startDate || 'N/A'}</td>
1212                 <td>${l.endDate || 'N/A'}</td>
1213                 <td><span class="chip ${l.status?.toLowerCase()}>${l.status || 'N/A'}</span></td>
1214             </tr>`).join('');
1215         }
1216     } catch (e) { console.error(e); }
1217 }
1218
1219 // ===== EMPLOYEES =====
1220 async function loadEmployees() {
1221     try {
1222         const employees = await apiFetch('/employee');
1223         const tbody = document.getElementById('employees-list');
1224         if (!employees.length) {
1225             tbody.innerHTML = '<tr><td colspan="8" class="empty-state">No employees found</td></tr>';
1226         } else {

```

```

1224         tbody.innerHTML = employees.map(e => `<tr>
1225             <td>${e.employeeId}</td>
1226             <td>${e.name || 'N/A'}</td>
1227             <td>${e.email || 'N/A'}</td>
1228             <td>${e.dept || 'N/A'}</td>
1229             <td><span class="chip ${e.role?.toLowerCase()}">${e.role || 'N/A'}</span></td>
1230             <td><span class="chip ${e.status?.toLowerCase()}">${e.status || 'N/A'}</span></td>
1231             <td>${e.timezone || 'N/A'}</td>
1232             <td><button class="btn btn-danger btn-sm"
onclick="deleteEmployee(${e.employeeId})">Delete</button></td>
1233         </tr>`).join('');
1234     }
1235     } catch (e) { console.error(e); }
1236 }
1237
1238 async function loadManagersDropdown() {
1239     try {
1240         const managers = await apiFetch('/admin/managers');
1241         const select = document.getElementById('employee-manager-select');
1242         select.innerHTML = '<option value="">-- Select Manager --</option>';
1243         (managers || []).forEach(m => {
1244             const opt = document.createElement('option');
1245             opt.value = m.managerId; // matches ManagerDTO field
1246             opt.textContent = `${m.name} - ${m.dept || 'N/A'} (${m.email})`;
1247             select.appendChild(opt);
1248         });
1249     } catch (e) {
1250         showAlert('Failed to load managers', 'error');
1251     }
1252 }
1253
1254
1255
1256 async function createEmployee() {
1257     const managerId = document.getElementById('employee-manager-select').value;
1258     const body = {
1259         name: document.getElementById('employee-name').value.trim(),
1260         email: document.getElementById('employee-email').value.trim(),
1261         password: document.getElementById('employee-password').value,
1262         gender: document.getElementById('employee-gender').value,
1263         timezone: document.getElementById('employee-timezone').value || 'UTC',
1264         managerId: parseInt(managerId)
1265     };
1266     if (!body.name || !body.email || !body.password) {
1267         showAlert('Please fill all employee fields', 'error');
1268         return;
1269     }
1270     if (!managerId) {
1271         showAlert('Please select a manager', 'error');
1272         return;
1273     }
1274     try {
1275         await apiFetch('/admin/employees', { method: 'POST', body: JSON.stringify(body) });
1276         showAlert('Employee created successfully!', 'success');
1277         clearEmployeeForm('employee');
1278         loadEmployees();
1279     } catch (e) {
1280         showAlert('Failed to create employee: ' + e.message, 'error');
1281     }
1282 }
1283
1284 async function createManager() {
1285     const body = {
1286         name: document.getElementById('manager-name-create').value.trim(),
1287         email: document.getElementById('manager-email-create').value.trim(),
1288         password: document.getElementById('manager-password-create').value,
1289         dept: document.getElementById('manager-dept-create').value.trim(),
1290         gender: document.getElementById('manager-gender').value,
1291         timezone: document.getElementById('manager-timezone-create').value || 'UTC'
1292     };
1293     if (!body.name || !body.email || !body.password || !body.dept || !body.gender) {
1294         showAlert('Please fill all manager fields', 'error');
1295         return;
1296     }
1297     try {
1298         await apiFetch('/admin/employees/manager', { method: 'POST', body: JSON.stringify(body) });

```

```

1299         showAlert('Manager created successfully!', 'success');
1300         clearEmployeeForm('manager');
1301         loadEmployees();
1302     } catch (e) { showAlert('Failed to create manager: ' + e.message, 'error'); }
1303 }
1304
1305 function clearEmployeeForm(type) {
1306     const fields = type === 'manager'
1307         ? ['manager-name-create', 'manager-email-create', 'manager-password-create', 'manager-dept-
create', 'manager-timezone-create']
1308         : ['employee-name', 'employee-email', 'employee-password', 'employee-timezone'];
1309     fields.forEach(id => document.getElementById(id).value = '');
1310     if (type === 'employee') {
1311         document.getElementById('employee-manager-select').value = '';
1312     }
1313 }
1314
1315 async function deleteEmployee(employeeId) {
1316     if (!confirm(`Delete employee #${employeeId}?`)) return;
1317     try {
1318         await apiFetch(`/employee/${employeeId}`, { method: 'DELETE' });
1319         showAlert('Employee deleted successfully!', 'success');
1320         loadEmployees();
1321         loadDashboard();
1322     } catch (e) { showAlert('Failed to delete employee: ' + e.message, 'error'); }
1323 }
1324
1325 // ===== LEAVE REQUESTS =====
1326 async function applyLeave() {
1327     const leaveType = document.getElementById('leave-type').value;
1328     const startDate = document.getElementById('leave-start').value;
1329     const endDate = document.getElementById('leave-end').value;
1330     const reason = document.getElementById('leave-reason').value.trim();
1331     if (!startDate || !endDate) { showAlert('Please fill all required fields', 'error'); return; }
1332     try {
1333         const data = await apiFetch('/leave_requests', {
1334             method: 'POST',
1335             body: JSON.stringify({ leaveType, startDate, endDate, reason })
1336         });
1337         if (data && data.warning) {
1338             showAlert(data.warning, 'warning'); // amber toast
1339         } else {
1340             showAlert('Leave request submitted successfully!', 'success');
1341         }
1342         clearLeaveForm();
1343         loadMyLeaves();
1344     } catch (e) { showAlert('Failed to submit: ' + e.message, 'error'); }
1345 }
1346 document.getElementById('leave-type').addEventListener('change', function() {
1347     document.getElementById('sick-leave-hint').style.display =
1348         this.value === 'SICK' ? 'block' : 'none';
1349     document.getElementById('casual-leave-hint').style.display =
1350         this.value === 'CASUAL' ? 'block' : 'none';
1351 });
1352
1353 async function loadMyLeaves() {
1354     try {
1355         const data = await apiFetch('/leave_requests/employee');
1356         const leaves = data.requests || [];
1357         const tbody = document.getElementById('my-leaves-list');
1358         if (!leaves.length) {
1359             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No leave requests
found</td></tr>';
1360         } else {
1361             tbody.innerHTML = leaves.map(l => `<tr>
1362                 <td>${l.leaveRequestId}</td>
1363                 <td><span class="chip">${l.leaveType}</span></td>
1364                 <td>${l.startDate}</td>
1365                 <td>${l.endDate}</td>
1366                 <td><span class="chip ${l.status?.toLowerCase()}>${l.status}</span></td>
1367                 <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-
'}</td>
1368                 <td>${l.status === 'PENDING' ? `<button class="btn btn-danger btn-sm"
onclick="openCancelModal(${l.leaveRequestId})">Cancel</button>` : '-'}</td>
1369             </tr>`).join('');
1370         }
1371     }

```



```

1371     } catch (e) { console.error(e); }
1372   }
1373
1374   async function loadPendingLeaves() {
1375     try {
1376       const leaves = await apiFetch('/leave_requests/pending');
1377       const tbody = document.getElementById('pending-leaves-list');
1378       if (!leaves.length) {
1379         tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No pending leave
requests</td></tr>';
1380       } else {
1381         tbody.innerHTML = leaves.map(l => `<tr>
1382           <td>${l.leaveRequestId}</td>
1383           <td>${l.employeeName}</td>
1384           <td><span class="chip">${l.leaveType}</span></td>
1385           <td>${l.startDate}</td>
1386           <td>${l.endDate}</td>
1387           <td style="max-width:150px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-
'}</td>
1388           <td class="action-row">
1389             <button class="btn btn-primary btn-sm"
onclick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
1390             <button class="btn btn-danger btn-sm"
onclick="openActionModal(${l.leaveRequestId}, 'REJECTED')">Reject</button>
1391           </td>
1392         </tr>`).join('');
1393       }
1394     } catch (e) { console.error(e); }
1395   }
1396
1397   function openActionModal(leaveId, action) {
1398     document.getElementById('action-leave-id').value = leaveId;
1399     document.getElementById('action-type').value = action;
1400     document.getElementById('modal-title').textContent = `${action} Leave Request #${leaveId}`;
1401     document.getElementById('action-btn').textContent = action;
1402     document.getElementById('action-btn').className = action === 'APPROVED' ? 'btn btn-primary' : 'btn
btn-danger';
1403     document.getElementById('action-modal').classList.add('open');
1404   }
1405
1406   async function submitApproval() {
1407     const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
1408     const action = document.getElementById('action-type').value;
1409     const remarks = document.getElementById('action-remarks').value;
1410     try {
1411       await apiFetch('/leave_requests/approval', {
1412         method: 'PUT',
1413         body: JSON.stringify({ leaveRequestId, action, remarks })
1414       });
1415       showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
1416       closeModal();
1417       loadPendingLeaves();
1418     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1419   }
1420
1421   function openCancelModal(leaveId) {
1422     document.getElementById('cancel-leave-id').value = leaveId;
1423     document.getElementById('cancel-modal').classList.add('open');
1424   }
1425
1426   async function confirmCancelLeave() {
1427     const leaveId = document.getElementById('cancel-leave-id').value;
1428     try {
1429       await apiFetch(`/leave_requests/cancel/${leaveId}`, { method: 'PUT' });
1430       showAlert('Leave cancelled successfully!', 'success');
1431       closeModal();
1432       loadMyLeaves();
1433     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1434   }
1435
1436   function clearLeaveForm() {
1437     document.getElementById('leave-start').value = '';
1438     document.getElementById('leave-end').value = '';
1439     document.getElementById('leave-reason').value = '';
1440   }
1441

```

```

1442 // ===== RESTAURANTS =====
1443 async function loadRestaurants() {
1444     try {
1445         const slot = document.getElementById('slot-filter').value;
1446         const path = slot ? `/restaurants/by-slot?slot=${slot}` : '/restaurants';
1447         const restaurants = await apiFetch(path);
1448         const tbody = document.getElementById('restaurants-list');
1449         if (!restaurants.length) {
1450             tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No restaurants
found</td></tr>';
1451         } else {
1452             tbody.innerHTML = restaurants.map(r => `<tr>
1453                 <td><strong>${r.name}</strong></td>
1454                 <td>${r.address || 'N/A'}</td>
1455                 <td>${r.supportedMealSlots ? r.supportedMealSlots.map(s => `<span
class="chip">${s}</span>`).join(' ') : 'N/A'}</td>
1456                 <td><span class="chip ${r.active ? 'active' : 'inactive'}">${r.active ? 'ACTIVE' :
'INACTIVE'}</span></td>
1457                 <td>${r.vendorName || r.vendorId || 'N/A'}</td>
1458             </tr>`).join('');
1459         }
1460     } catch (e) { console.error(e); }
1461 }
1462
1463 // ===== SUBSCRIPTIONS =====
1464 async function loadRestaurantDropdown() {
1465     try {
1466         const restaurants = await apiFetch('/restaurants');
1467         const arr = Array.isArray(restaurants) ? restaurants : [];
1468         const select = document.getElementById('sub-restaurant-select');
1469         select.innerHTML = '<option value="">-- Select Restaurant --</option>';
1470         arr.filter(r => r.active).forEach(r => {
1471             const opt = document.createElement('option');
1472             opt.value = r.id || r.restaurantId;
1473             opt.textContent = r.name;
1474             opt.dataset.slots = JSON.stringify(r.supportedMealSlots || []);
1475             select.appendChild(opt);
1476         });
1477     } catch (e) { console.error('Failed to load restaurants:', e); }
1478 }
1479
1480 document.getElementById('sub-restaurant-select')?.addEventListener('change', function () {
1481     const selected = this.options[this.selectedIndex];
1482     const supported = selected?.dataset?.slots ? JSON.parse(selected.dataset.slots) : [];
1483     const boxes = document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]');
1484     boxes.forEach(cb => {
1485         const wrapper = cb.parentElement;
1486         if (!supported.length || supported.includes(cb.value)) {
1487             wrapper.style.display = 'flex'; wrapper.style.opacity = '1'; cb.disabled = false;
1488         } else {
1489             cb.checked = false; cb.disabled = true; wrapper.style.opacity = '0.3';
1490         }
1491     });
1492 });
1493
1494 async function loadSubscriptions() {
1495     try {
1496         const subscriptions = await apiFetch('/subscription/user');
1497         const arr = Array.isArray(subscriptions) ? subscriptions : [];
1498         const tbody = document.getElementById('subscriptions-list');
1499         tbody.innerHTML = arr.length
1500             ? arr.map(s => `<tr>
1501                 <td>${s.subscriptionId}</td>
1502                 <td>${s.restaurantName || 'N/A'}</td>
1503                 <td><span class="chip">${s.mealSlot}</span></td>
1504                 <td>${s.scheduleType}</td>
1505                 <td><span class="chip ${s.status?.toLowerCase()}>${s.status}</span></td>
1506                 <td>${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleDateString() : 'Not
scheduled'}</td>
1507                 <td class="action-row">
1508                     ${s.status === 'ACTIVE' ? `<button class="btn btn-secondary btn-sm"
onclick="pauseSubscription(${s.subscriptionId})">Pause</button>` : ''}
1509                     ${s.status === 'PAUSED' ? `<button class="btn btn-primary btn-sm"
onclick="resumeSubscription(${s.subscriptionId})">Resume</button>` : ''}
1510                     <button class="btn btn-danger btn-sm"
onclick="expireSubscription(${s.subscriptionId})">Expire</button>

```

```

1511         </td>
1512     </tr>`)).join('')
1513     : '<tr><td colspan="7" class="empty-state">No subscriptions found</td></tr>';
1514 } catch (e) { console.error(e); }
1515 }
1516
1517 async function addSubscription() {
1518     const email = document.getElementById('sub-email').value.trim();
1519     const restaurantId = parseInt(document.getElementById('sub-restaurant-select').value);
1520     const scheduleType = document.getElementById('sub-schedule').value;
1521     const dayOfWeek = document.getElementById('sub-day').value;
1522     const selectedSlots = [...document.querySelectorAll('#meal-slot-checkboxes
input[type=checkbox]:checked')].map(cb => cb.value);
1523     if (!email) { showAlert('Please enter employee email', 'error'); return; }
1524     if (!restaurantId) { showAlert('Please select a restaurant', 'error'); return; }
1525     if (!selectedSlots.length) { showAlert('Please select at least one meal slot', 'error'); return; }
1526     const body = {
1527         scheduleType,
1528         mealsSlots: selectedSlots,
1529         slotOrders: selectedSlots.map(slot => ({ mealSlot: slot, restaurantId })))
1530     };
1531     if (scheduleType === 'WEEKLY') body.dayOfWeek = dayOfWeek;
1532     try {
1533         await apiFetch('/subscription', { method: 'POST', body: JSON.stringify(body) });
1534         showAlert('Subscription created successfully!', 'success');
1535         clearSubscriptionForm();
1536         loadSubscriptions();
1537     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1538 }
1539
1540 async function pauseSubscription(id) {
1541     try { await apiFetch(`/subscription/${id}/pause`, { method: 'PUT' }); showAlert('Subscription
paused', 'success'); loadSubscriptions(); }
1542     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1543 }
1544
1545 async function resumeSubscription(id) {
1546     try { await apiFetch(`/subscription/${id}/resume`, { method: 'PUT' }); showAlert('Subscription
resumed', 'success'); loadSubscriptions(); }
1547     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1548 }
1549
1550 async function expireSubscription(id) {
1551     try { await apiFetch(`/subscription/${id}/expire`, { method: 'PUT' }); showAlert('Subscription
expired', 'success'); loadSubscriptions(); }
1552     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1553 }
1554
1555 function clearSubscriptionForm() {
1556     document.getElementById('sub-email').value = '';
1557     document.getElementById('sub-restaurant-select').value = '';
1558     document.getElementById('sub-schedule').value = 'DAILY';
1559     document.getElementById('day-of-week-group').style.display = 'none';
1560     document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]').forEach(cb => {
1561         cb.checked = false; cb.disabled = false;
1562         cb.parentElement.style.opacity = '1'; cb.parentElement.style.display = 'flex';
1563     });
1564 }
1565
1566 document.getElementById('sub-schedule')?.addEventListener('change', function () {
1567     document.getElementById('day-of-week-group').style.display = this.value === 'WEEKLY' ? 'block' :
'none';
1568 });
1569
1570 // ===== DELIVERIES =====
1571 async function loadDeliveries() {
1572     try {
1573         const subscriptions = await apiFetch('/subscription/user');
1574         let allDeliveries = [];
1575         for (const sub of subscriptions) {
1576             try {
1577                 const deliveries = await apiFetch(`/delivery/subscription/${sub.subscriptionId}`);
1578                 allDeliveries.push(...deliveries);
1579             } catch (e) {}
1580         }
1581         const tbody = document.getElementById('deliveries-list');

```

```

1582         if (!allDeliveries.length) {
1583             tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No deliveries found</td></tr>';
1584         } else {
1585             tbody.innerHTML = allDeliveries.map(d => `<tr>
1586                 <td>${d.id}</td>
1587                 <td>${d.subscription?.id || 'N/A'}</td>
1588                 <td><span class="chip">${d.mealSlot || 'N/A'}</span></td>
1589                 <td>${d.scheduledDeliveryTime ? new Date(d.scheduledDeliveryTime).toLocaleString() :
1590 'N/A'}</td>
1591                 <td><span class="chip ${d.status?.toLowerCase()}>${d.status ||
1592 'SCHEDULED'}</span></td>
1593             </tr>`).join('');
1594         }
1595     } catch (e) { console.error(e); }
1596 }
1597
1598 // ===== VENDORS =====
1599 async function loadVendors() {
1600     try {
1601         const vendors = await apiFetch('/vendors');
1602         const tbody = document.getElementById('vendors-list');
1603         if (!vendors.length) {
1604             tbody.innerHTML = '<tr><td colspan="6" class="empty-state">No vendors found</td></tr>';
1605         } else {
1606             tbody.innerHTML = vendors.map(v => `<tr>
1607                 <td>${v.id}</td>
1608                 <td><strong>${v.name}</strong></td>
1609                 <td>${v.email}</td>
1610                 <td>${v.phone || 'N/A'}</td>
1611                 <td>${v.registeredAt || 'N/A'}</td>
1612                 <td><button class="btn btn-danger btn-sm"
1613 onclick="deleteVendor(${v.id})">Delete</button></td>
1614             </tr>`).join('');
1615         }
1616     } catch (e) { console.error(e); }
1617 }
1618
1619 async function createVendor() {
1620     const body = {
1621         name: document.getElementById('vendor-name-create').value.trim(),
1622         email: document.getElementById('vendor-email-create').value.trim(),
1623         password: document.getElementById('vendor-password-create').value,
1624         phone: document.getElementById('vendor-phone-create').value.trim(),
1625         role: document.getElementById('vendor-role').value
1626     };
1627     if (!body.name || !body.email || !body.password || !body.phone) {
1628         showAlert('Please fill all vendor fields', 'error'); return;
1629     }
1630     try {
1631         await apiFetch('/vendors/register', { method: 'POST', body: JSON.stringify(body) });
1632         showAlert('Vendor created successfully!', 'success');
1633         clearVendorForm();
1634         loadVendors();
1635     } catch (e) { showAlert('Failed to create vendor: ' + e.message, 'error'); }
1636 }
1637
1638 function clearVendorForm() {
1639     ['vendor-name-create', 'vendor-email-create', 'vendor-password-create', 'vendor-phone-create']
1640     .forEach(id => document.getElementById(id).value = '');
1641 }
1642
1643 async function deleteVendor(vendorId) {
1644     if (!confirm('Delete vendor #${vendorId}?')) return;
1645     try {
1646         await apiFetch(`/vendors/${vendorId}`, { method: 'DELETE' });
1647         showAlert('Vendor deleted successfully!', 'success');
1648         loadVendors();
1649     } catch (e) { showAlert('Failed to delete vendor: ' + e.message, 'error'); }
1650 }
1651
1652 // ===== VENDOR NEGOTIATIONS =====
1653 let vendorNegotiationsCache = [];
1654 let selectedNegotiationId = null;
1655
1656 function escHtml(value) {
1657     const div = document.createElement('div');

```

```

1655     div.textContent = value == null ? '' : String(value);
1656     return div.innerHTML;
1657 }
1658
1659 function formatMoney(value) {
1660     return value == null || value === '' ? '-' : `${Number(value).toLocaleString('en-IN')}`;
1661 }
1662
1663 async function loadVendorNegotiationOptions() {
1664     const [vendors, devices] = await Promise.all([
1665         apiFetch('/vendors').catch(() => []),
1666         apiFetch('/devices/all').catch(() => apiFetch('/devices').catch(() => []))
1667     ]);
1668     const vendorSelect = document.getElementById('neg-vendor-id');
1669     const deviceSelect = document.getElementById('neg-device-id');
1670     vendorSelect.innerHTML = '<option value="">-- Select Vendor --</option>' + (vendors || []).map(v =>
1671         `<option value="${v.id}">${escHtml(v.name || v.email || `Vendor
1672 #${v.id}`)}</option>`).join('');
1673     deviceSelect.innerHTML = '<option value="">No device</option>' + (devices || []).map(d =>
1674         `<option value="${d.id}">${escHtml(d.deviceName || `Device #${d.id}`)}${d.techVendorName ? `
1675 ${escHtml(d.techVendorName)}` : ''}</option>`).join('');
1676 }
1677
1678 async function loadVendorNegotiations() {
1679     await loadVendorNegotiationOptions();
1680     try {
1681         vendorNegotiationsCache = await apiFetch('/vendor-negotiations') || [];
1682         const tbody = document.getElementById('vendor-negotiations-list');
1683         tbody.innerHTML = vendorNegotiationsCache.length ? vendorNegotiationsCache.map(n => `
1684             <tr>
1685                 <td>#${n.id}</td>
1686                 <td>${escHtml(n.vendorName || n.vendorId || '-')}</td>
1687                 <td>${escHtml(n.deviceName || n.deviceId || '-')}</td>
1688                 <td>${escHtml(n.subject || '-')}</td>
1689                 <td>${formatMoney(n.proposedAmount)}</td>
1690                 <td><span class="chip ${n.status || 'pending'}.toLowerCase()}>${escHtml(n.status ||
1691 'PENDING')}</span></td>
1692                 <td>${n.updatedAt ? new Date(n.updatedAt).toLocaleString() : '-'}</td>
1693                 <td><button class="btn btn-secondary btn-sm
1694 onclick="openVendorNegotiation(${n.id})">Open</button></td>
1695             </tr>`).join('') : '<tr><td colspan="8" class="empty-state">No negotiations
1696 yet</td></tr>';
1697         if (selectedNegotiationId) {
1698             const exists = vendorNegotiationsCache.some(n => n.id === selectedNegotiationId);
1699             if (exists) await openVendorNegotiation(selectedNegotiationId);
1700         }
1701     } catch (e) {
1702         document.getElementById('vendor-negotiations-list').innerHTML = '<tr><td colspan="8"
1703 class="empty-state">Could not load negotiations</td></tr>';
1704     }
1705 }
1706
1707 async function openVendorNegotiation(id) {
1708     selectedNegotiationId = id;
1709     const n = await apiFetch(`/vendor-negotiations/${id}`);
1710     document.getElementById('neg-detail-title').textContent = `Negotiation #${n.id}`;
1711     document.getElementById('neg-detail-body').innerHTML = `
1712     <div style="display:grid;gap:10px">
1713         <div><label>Subject</label><div>${escHtml(n.subject || '-')}</div></div>
1714         <div><label>Vendor</label><div>${escHtml(n.vendorName || n.vendorId || '-')}</div></div>
1715         <div><label>Device</label><div>${escHtml(n.deviceName || n.deviceId || '-')}</div></div>
1716         <div><label>Amount</label><div>${formatMoney(n.proposedAmount)}</div></div>
1717         <div><label>Status</label><select id="neg-status-
1718 select">${['PENDING', 'NEGOTIATING', 'AGREED', 'REJECTED', 'PAID', 'CLOSED'].map(s => `<option value="${s}" $${s ===
1719 n.status ? 'selected'}...
1720         <button class="btn btn-secondary btn-sm"
1721 onclick="updateVendorNegotiationStatus(${n.id})">Update Status</button>
1722         <div><label>Description</label><div style="color:var(--text2)">${escHtml(n.description ||
1723 '-')}</div></div>
1724     </div>
1725     <div class="message-list">${(n.messages || []).length ? n.messages.map(m => `
1726         <div class="message-item">
1727             <div class="message-meta">${escHtml(m.senderType || '-')} · ${escHtml(m.senderEmail ||
1728 '')} · ${m.createdAt ? new Date(m.createdAt).toLocaleString() : ''}</div>
1729             <div>${escHtml(m.message || '')}</div>

```

```

1719         </div>`).join('') : '<div class="empty-state">No messages yet</div>'}</div>
1720     <div class="form-group">
1721         <label>Reply</label>
1722         <textarea id="neg-reply-message" rows="3" placeholder="Write a reply"></textarea>
1723     </div>
1724     <button class="btn btn-primary" style="margin-top:10px"
onclick="replyVendorNegotiation(${n.id})">Send Reply</button>`;
1725     }
1726
1727     async function createVendorNegotiation() {
1728         const body = {
1729             vendorId: Number(document.getElementById('neg-vendor-id').value) || null,
1730             deviceId: Number(document.getElementById('neg-device-id').value) || null,
1731             proposedAmount: document.getElementById('neg-amount').value || null,
1732             subject: document.getElementById('neg-subject').value.trim(),
1733             description: document.getElementById('neg-description').value.trim(),
1734             initialMessage: document.getElementById('neg-initial-message').value.trim()
1735         };
1736         if (!body.vendorId || !body.subject || !body.description) {
1737             showAlert('Vendor, subject, and description are required', 'error');
1738             return;
1739         }
1740         try {
1741             const created = await apiFetch('/vendor-negotiations', { method: 'POST', body:
JSON.stringify(body) });
1742             showAlert('Negotiation created', 'success');
1743             clearVendorNegotiationForm();
1744             selectedNegotiationId = created.id;
1745             await loadVendorNegotiations();
1746         } catch (e) { showAlert('Failed to create negotiation: ' + e.message, 'error'); }
1747     }
1748
1749     async function replyVendorNegotiation(id) {
1750         const message = document.getElementById('neg-reply-message').value.trim();
1751         if (!message) { showAlert('Reply message is required', 'error'); return; }
1752         try {
1753             await apiFetch(`/vendor-negotiations/${id}/messages`, { method: 'POST', body: JSON.stringify({
message }) });
1754             showAlert('Reply sent', 'success');
1755             await loadVendorNegotiations();
1756         } catch (e) { showAlert('Failed to send reply: ' + e.message, 'error'); }
1757     }
1758
1759     async function updateVendorNegotiationStatus(id) {
1760         const status = document.getElementById('neg-status-select').value;
1761         try {
1762             await apiFetch(`/vendor-negotiations/${id}/status`, { method: 'PUT', body: JSON.stringify({
status }) });
1763             showAlert('Status updated', 'success');
1764             await loadVendorNegotiations();
1765         } catch (e) { showAlert('Failed to update status: ' + e.message, 'error'); }
1766     }
1767
1768     function clearVendorNegotiationForm() {
1769         ['neg-amount', 'neg-subject', 'neg-description', 'neg-initial-message'].forEach(id =>
document.getElementById(id).value = '');
1770         document.getElementById('neg-vendor-id').value = '';
1771         document.getElementById('neg-device-id').value = '';
1772     }
1773
1774     // ===== MODAL HELPERS =====
1775     function closeModal() {
1776         document.getElementById('action-modal').classList.remove('open');
1777         document.getElementById('action-remarks').value = '';
1778         document.getElementById('manager-email').value = currentUser?.email || '';
1779     }
1780
1781     function closeCancelModal() {
1782         document.getElementById('cancel-modal').classList.remove('open');
1783         document.getElementById('cancel-email').value = currentUser?.email || '';
1784     }
1785
1786     // ===== DEVICES =====
1787     async function loadUnassignedDevices() {
1788         try {
1789             const devices = await apiFetch('/devices/unassigned');

```

```

1790         const tbody = document.getElementById('devices-list');
1791         if (!devices || !devices.length) {
1792             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No unassigned
devices</td></tr>';
1793             return;
1794         }
1795         const today = new Date();
1796         tbody.innerHTML = devices.map(d => {
1797             const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1798             const warrantyDisplay = warrantyExpiry
1799                 ? (warrantyExpiry > today
1800                     ? `<span style="color:var(--green);font-size:11px">
${d.warrantyExpiryDate}</span>`
1801                     : `<span style="color:var(--red);font-size:11px"> ${d.warrantyExpiryDate}</span>`)
1802                 : `<span style="color:var(--text3);font-size:11px">--</span>`;
1803             return `<tr>
1804                 <td>#${d.id}</td>
1805                 <td><strong>${d.deviceName} || '-'</strong></td>
1806                 <td>${d.brand} || '-'</td>
1807                 <td><span class="chip">${d.deviceType} || '-'</span></td>
1808                 <td>${warrantyDisplay}</td>
1809                 <td>${d.vendorName} || '-'</td>
1810                 <td><button class="btn btn-primary btn-sm" onclick="openAssignModal(${d.id},
'${d.deviceName}')">Assign</button></td>
1811             </tr>`;
1812         }).join('');
1813     } catch (e) { console.error(e); }
1814 }
1815
1816 async function openAssignModal(deviceId, deviceName) {
1817     document.getElementById('assign-device-id').value = deviceId;
1818     document.getElementById('assign-device-name').value = deviceName;
1819     try {
1820         const employees = await apiFetch('/employee');
1821         const select = document.getElementById('assign-employee-select');
1822         select.innerHTML = '<option value="">-- Select Employee --</option>';
1823         employees.filter(e => e.status === 'ACTIVE').forEach(e => {
1824             const opt = document.createElement('option');
1825             opt.value = e.employeeId;
1826             opt.textContent = `${e.name} (${e.email}) - ${e.dept} || 'N/A'`;
1827             select.appendChild(opt);
1828         });
1829     } catch (e) { showAlert('Failed to load employees', 'error'); return; }
1830     document.getElementById('assign-modal').classList.add('open');
1831 }
1832
1833 async function confirmAssign() {
1834     const deviceId = document.getElementById('assign-device-id').value;
1835     const employeeId = document.getElementById('assign-employee-select').value;
1836     if (!employeeId) { showAlert('Please select an employee', 'error'); return; }
1837     try {
1838         const message = await apiFetch(`/devices/${deviceId}/assign`, {
1839             method: 'POST',
1840             body: JSON.stringify({ employeeId: parseInt(employeeId) })
1841         });
1842         showAlert(message || 'Device assigned successfully!', 'success');
1843         closeAssignModal();
1844         loadUnassignedDevices();
1845     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1846 }
1847
1848 function closeAssignModal() {
1849     document.getElementById('assign-modal').classList.remove('open');
1850     document.getElementById('assign-employee-select').value = '';
1851     document.getElementById('assign-device-id').value = '';
1852     document.getElementById('assign-device-name').value = '';
1853 }
1854
1855 // ===== MY DEVICES =====
1856 async function loadMyAssignedDevices() {
1857     try {
1858         const devices = await apiFetch('/devices/my');
1859         const tbody = document.getElementById('my-devices-list');
1860         if (!devices || !devices.length) {
1861             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';

```

```

1862         return;
1863     }
1864     const today = new Date();
1865     tbody.innerHTML = devices.map(d => {
1866         const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1867         const warrantyDisplay = warrantyExpiry
1868             ? (warrantyExpiry > today
1869                 ? `<span style="color:var(--green);font-size:11px">
1870 ${d.warrantyExpiryDate}</span>`
1871                 : `<span style="color:var(--red);font-size:11px"> Expired
1872 ${d.warrantyExpiryDate}</span>`)
1873             : `<span style="color:var(--text3);font-size:11px">--</span>`;
1874         return `<tr>
1875             <td>#${d.id}</td>
1876             <td><strong>${d.deviceName || '-'}</strong></td>
1877             <td>${d.brand || '-'}</td>
1878             <td><span class="chip">${(d.deviceType || '').toLowerCase()}</span></td>
1879             <td>${warrantyDisplay}</td>
1880             <td style="font-size:12px;color:var(--text2)">${d.assignedDate || '-'}</td>
1881             <td><span class="chip ${d.deviceStatus || ''}.toLowerCase()">${d.deviceStatus || '-'}</span></td>
1882         </tr>`;
1883     }).join('');
1884     } catch (e) { console.error(e); }
1885 }
1886
1887 // ===== MY SERVICE REQUESTS =====
1888 async function loadServiceRequests() {
1889     await Promise.all([loadServiceRequestsList(), loadMyDevicesForSR()]);
1890 }
1891
1892 async function loadServiceRequestsList() {
1893     try {
1894         const requests = await apiFetch('/service-requests/my');
1895         const tbody = document.getElementById('service-requests-list');
1896         if (!requests || !requests.length) {
1897             tbody.innerHTML = `<tr><td colspan="7" class="empty-state">No service requests
1898 yet</td></tr>`;
1899         }
1900         return;
1901     } catch (e) { console.error(e); }
1902 }
1903
1904 tbody.innerHTML = requests.map(r => `<tr>
1905     <td>#${r.id}</td>
1906     <td>${r.deviceName || '-'}</td>
1907     <td><span class="chip">${r.requestType}</span></td>
1908     <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
1909 space:nowrap">${r.issueDescription || '-'}</td>
1910     <td>${r.urgent ? `<span class="chip pending">YES</span>` : `<span style="color:var(--
1911 text2)">No</span>`}</td>
1912     <td><span class="chip ${r.status?.toLowerCase()}>${r.status}</span></td>
1913     <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new
1914 Date(r.raisedAt).toLocaleDateString() : '-'}</td>
1915     </tr>`).join('');
1916     } catch (e) { console.error(e); }
1917 }
1918
1919 async function loadMyDevicesForSR() {
1920     try {
1921         const devices = await apiFetch('/devices/my');
1922         const select = document.getElementById('sr-device-select');
1923         select.innerHTML = `<option value="">-- Select your device --</option>`;
1924         (devices || []).forEach(d => {
1925             const opt = document.createElement('option');
1926             opt.value = d.id;
1927             opt.textContent = `${d.deviceName} (${d.brand || d.deviceType || 'Device'})`;
1928             select.appendChild(opt);
1929         });
1930     } catch (e) { console.error('Failed to load devices for SR', e); }
1931 }
1932
1933 async function submitServiceRequest() {
1934     const deviceId = parseInt(document.getElementById('sr-device-select').value);
1935     const requestType = document.getElementById('sr-type').value;
1936     const issueDescription = document.getElementById('sr-description').value.trim();
1937     const urgent = document.getElementById('sr-urgent').checked;
1938     if (!deviceId) { showAlert('Please select a device', 'error'); return; }
1939     if (!issueDescription) { showAlert('Please describe the issue', 'error'); return; }
1940     try {

```



```

1932         await apiFetch('/service-requests', {
1933             method: 'POST',
1934             body: JSON.stringify({ deviceId, requestType, issueDescription, urgent })
1935         });
1936         showAlert('Service request submitted!', 'success');
1937         clearServiceRequestForm();
1938         loadServiceRequestsList();
1939     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1940 }
1941
1942 function clearServiceRequestForm() {
1943     document.getElementById('sr-device-select').value = '';
1944     document.getElementById('sr-type').value = 'REPAIR';
1945     document.getElementById('sr-description').value = '';
1946     document.getElementById('sr-urgent').checked = false;
1947 }
1948
1949 // ===== ADMIN SERVICE REQUESTS =====
1950 async function loadAdminServiceRequests() {
1951     try {
1952         const requests = await apiFetch('/service-requests/open');
1953         const tbody = document.getElementById('admin-service-requests-list');
1954         if (!requests || !requests.length) {
1955             tbody.innerHTML = '<tr><td colspan="9" class="empty-state">No open service
requests</td></tr>';
1956             return;
1957         }
1958         tbody.innerHTML = requests.map(r => `<tr>
1959             <td>#${r.id}</td>
1960             <td style="font-size:12px;color:var(--text2)">${r.raisedByName || r.employeeId ||
'-'</td>
1961             <td><strong>${r.deviceName || '-'</strong></td>
1962             <td><span class="chip">${r.requestType}</span></td>
1963             <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;font-
size:12px;color:var(--text2)" title="${r.issueDescription || ''}">${r.issueDescription || '...
1964             <td>${r.urgent ? '<span class="chip pending">YES</span>' : '<span style="color:var(--
text2);font-size:12px">No</span>'}</td>
1965             <td><span class="chip ${r.status?.toLowerCase()}>${r.status}</span></td>
1966             <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new
Date(r.raisedAt).toLocaleDateString() : '-'</td>
1967             <td>
1968                 <button class="btn btn-primary btn-sm" onclick="openAdminSRModal(${r.id},
'${(r.deviceName || '').replace(/'/g, '\\')}','${(r.raisedByName || r.employeeId || '').replace(/'/g,...
1969                 Update
1970                 </button>
1971             </td>
1972             </tr>`).join('');
1973     } catch (e) { console.error(e); }
1974 }
1975
1976 function openAdminSRModal(requestId, deviceName, employeeName) {
1977     document.getElementById('admin-sr-id').value = requestId;
1978     document.getElementById('admin-sr-modal-id').textContent = `#${requestId}`;
1979     document.getElementById('admin-sr-device-display').value = deviceName;
1980     document.getElementById('admin-sr-employee-display').value = employeeName;
1981     document.getElementById('admin-sr-status').value = 'SENT_FOR_REPAIR';
1982     document.getElementById('admin-sr-remarks').value = '';
1983     // Show hint for default selection
1984     document.getElementById('admin-sr-repair-hint').style.display = 'block';
1985     document.getElementById('admin-sr-modal').classList.add('open');
1986 }
1987
1988 // Show/hide the repair hint based on status selection
1989 document.getElementById('admin-sr-status')?.addEventListener('change', function () {
1990     document.getElementById('admin-sr-repair-hint').style.display =
1991     this.value === 'SENT_FOR_REPAIR' ? 'block' : 'none';
1992 });
1993
1994 async function submitAdminSRUpdate() {
1995     const serviceRequestId = parseInt(document.getElementById('admin-sr-id').value);
1996     const status = document.getElementById('admin-sr-status').value;
1997     const adminRemarks = document.getElementById('admin-sr-remarks').value.trim();
1998
1999     if (!adminRemarks) {
2000         showAlert('Please add remarks before updating', 'error');
2001         return;

```

```

2002     }
2003
2004     try {
2005         await apiFetch('/service-requests/admin', {
2006             method: 'PUT',
2007             body: JSON.stringify({ serviceRequestId, status, adminRemarks })
2008         });
2009         const statusLabel = status.replace(/_/g, ' ').toLowerCase();
2010         showAlert(`Service request marked as "${statusLabel}" successfully!`, 'success');
2011         closeAdminSRModal();
2012         loadAdminServiceRequests();
2013     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
2014 }
2015
2016 function closeAdminSRModal() {
2017     document.getElementById('admin-sr-modal').classList.remove('open');
2018     document.getElementById('admin-sr-id').value = '';
2019     document.getElementById('admin-sr-remarks').value = '';
2020     document.getElementById('admin-sr-repair-hint').style.display = 'none';
2021 }
2022
2023 // ===== LOGOUT =====
2024 function logout() {
2025     localStorage.removeItem('jwt_token');
2026     localStorage.removeItem('refresh_token');
2027     localStorage.removeItem('auth_type');
2028     localStorage.removeItem('last_role');
2029     fetch(`${API}/session/logout`, { method: 'POST', credentials: 'include' })
2030         .finally(() => {
2031             document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
2032             window.location.href = '/google.html';
2033         });
2034 }
2035 async function loadNotifCount() {
2036     try {
2037         const data = await apiFetch('/notifications/unread-count');
2038         const badge = document.getElementById('notif-badge');
2039
2040         // Handles: plain number, { count: N }, or { unreadCount: N }
2041         const count = typeof data === 'number'
2042             ? data
2043             : (data?.count ?? data?.unreadCount ?? 0);
2044
2045         if (count > 0) {
2046             badge.textContent = count;
2047             badge.style.display = 'flex';
2048         } else {
2049             badge.style.display = 'none';
2050         }
2051     } catch(e) { console.warn('Notif count error', e); }
2052 }
2053
2054 async function toggleNotifPanel() {
2055     const panel = document.getElementById('notif-panel');
2056     if (panel.style.display === 'none') {
2057         panel.style.display = 'block';
2058         await loadNotifications();
2059         try { await apiFetch('/notifications/mark-read', { method: 'POST' }); } catch(e) {}
2060         document.getElementById('notif-badge').style.display = 'none';
2061     } else {
2062         panel.style.display = 'none';
2063     }
2064 }
2065
2066 async function loadNotifications() {
2067     try {
2068         const data = await apiFetch('/notifications');
2069         const list = document.getElementById('notif-list');
2070         list.innerHTML = (data && data.length)
2071             ? data.map(n => `
2072                 <div class="notif-item ${n.read ? '' : 'unread'}">
2073                     <div>${n.message}</div>
2074                     <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
2075                 </div>`).join('')
2076             : '<div class="notif-item">No notifications yet.</div>';
2077     } catch(e) { console.warn('Notif load error', e); }

```

```

2078     }
2079
2080     loadNotifCount();
2081     setInterval(loadNotifCount, 30000);
2082
2083     document.addEventListener('click', e => {
2084         const panel = document.getElementById('notif-panel');
2085         const wrapper = document.querySelector('.notif-wrapper');
2086         if (panel && wrapper && !panel.contains(e.target) && !wrapper.contains(e.target)) {
2087             panel.style.display = 'none';
2088         }
2089     });
2090
2091     // Set today's date as default for leave form
2092     document.getElementById('leave-start').valueAsDate = new Date();
2093     document.getElementById('leave-end').valueAsDate = new Date();
2094
2095     // ===== HOLIDAYS =====
2096     async function loadHolidays() {
2097         try {
2098             const year = new Date().getFullYear();
2099             const holidays = await apiFetch(`/holidays?year=${year}`);
2100             const tbody = document.getElementById('holidays-list');
2101             if (!holidays || !holidays.length) {
2102                 tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No holidays added yet</td></tr>';
2103                 return;
2104             }
2105             tbody.innerHTML = holidays.map(h => `<tr>
2106                 <td>#${h.id}</td>
2107                 <td><strong>${h.name}</strong></td>
2108                 <td>${h.date}</td>
2109                 <td>${h.description || '-'}</td>
2110                 <td>
2111                     <button class="btn btn-danger btn-sm" onclick="deleteHoliday('${h.id}')">Delete</button>
2112                 </td>
2113             </tr>`).join('');
2114         } catch(e) { console.error(e); }
2115     }
2116
2117     async function addHoliday() {
2118         const name = document.getElementById('h-name').value.trim();
2119         const date = document.getElementById('h-date').value;
2120         const description = document.getElementById('h-desc').value.trim();
2121         if (!name || !date) { showAlert('Please enter name and date', 'error'); return; }
2122         try {
2123             await apiFetch('/holidays', {
2124                 method: 'POST',
2125                 body: JSON.stringify({ name, date, description })
2126             });
2127             showAlert('Holiday added!', 'success');
2128             clearHolidayForm();
2129             loadHolidays();
2130         } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
2131     }
2132
2133     async function deleteHoliday(id) {
2134         if (!confirm('Delete this holiday?')) return;
2135         try {
2136             await apiFetch(`/holidays/${id}`, { method: 'DELETE' });
2137             showAlert('Holiday deleted', 'success');
2138             loadHolidays();
2139         } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
2140     }
2141
2142     function clearHolidayForm() {
2143         document.getElementById('h-name').value = '';
2144         document.getElementById('h-date').value = '';
2145         document.getElementById('h-desc').value = '';
2146     }
2147
2148     // ===== MY PROFILE =====
2149     async function loadAdminProfile() {
2150         try {
2151             const me = await apiFetch('/employee/me');
2152             if(me) {
2153                 document.getElementById('p-name').value = me.name || '-';

```

```

2154         document.getElementById('p-email').value = me.email    || '-';
2155         document.getElementById('p-dept').value = me.dept      || '-';
2156         document.getElementById('p-role').value = me.role      || 'ADMIN';
2157         document.getElementById('p-status').value = me.status   || '-';
2158         document.getElementById('p-tz').value = me.timezone    || '-';
2159
2160         // Show maternity section only for female admins
2161         document.getElementById('maternity-apply-btn').style.display =
2162             me.gender === 'F' ? 'block' : 'none';
2163     }
2164 } catch(e) {
2165     showAlert('Could not load profile', 'error');
2166 }
2167 }
2168
2169 // Auto calculate maternity end date
2170 document.getElementById('mat-start-date').addEventListener('change', function(){
2171     if(this.value) {
2172         const end = new Date(this.value);
2173         end.setDate(end.getDate() + 179);
2174         document.getElementById('mat-end-date').value =
2175             end.toISOString().split('T')[0];
2176     } else {
2177         document.getElementById('mat-end-date').value = '';
2178     }
2179 });
2180
2181 // Submit maternity leave
2182 async function applyMaternityLeave() {
2183     const startDate = document.getElementById('mat-start-date').value;
2184     const reason = document.getElementById('mat-reason').value.trim();
2185
2186     if(!startDate) {
2187         showAlert('Please select a start date', 'error');
2188         return;
2189     }
2190
2191     try {
2192         await apiFetch('/leave_requests/maternity/apply', {
2193             method: 'POST',
2194             body: JSON.stringify({ startDate, reason })
2195         });
2196         showAlert('Maternity leave submitted successfully!', 'success');
2197         document.getElementById('mat-start-date').value = '';
2198         document.getElementById('mat-end-date').value = '';
2199         document.getElementById('mat-reason').value = '';
2200         loadMyLeaves();
2201     } catch(e) {
2202         showAlert('Failed: ' + e.message, 'error');
2203     }
2204 }
2205
2206 // ===== CALENDAR =====
2207 let employeeCalendar = null;
2208 let calendarInitialized = false;
2209
2210 const STATUS_COLORS = {
2211     APPROVED: { bg:'rgba(54,211,153,0.25)', border:'#36d399', text:'#36d399' },
2212     PENDING: { bg:'rgba(249,199,79,0.25)', border:'#f9c74f', text:'#f9c74f' },
2213     REJECTED: { bg:'rgba(248,113,113,0.25)', border:'#f87171', text:'#f87171' },
2214     HOLIDAY: { bg:'rgba(167,139,250,0.25)', border:'#a78bfa', text:'#a78bfa' },
2215 };
2216 const TYPE_LABEL = {
2217     SICK:' Sick', CASUAL:' Casual',
2218     MATERNITY:' Maternity', UNPAID:' Unpaid'
2219 };
2220
2221 async function loadMyCalendar() {
2222     await loadCalendarBalanceCards();
2223     if(!calendarInitialized) {
2224         initFullCalendar();
2225     } else {
2226         await refreshCalendarEvents();
2227     }
2228 }
2229

```

```

2230 async function loadCalendarBalanceCards() {
2231     try {
2232         const [dashboard, sickMonthData] = await Promise.all([
2233             apiFetch('/api/employees/me/leave-dashboard'),
2234             apiFetch('/leave_requests/sick/used-this-month')
2235         ]);
2236
2237         /* SICK */
2238         const sickBadge = document.getElementById('cal-sick-month-badge');
2239         const sick = dashboard?.sick;
2240         const sickAvail = Number(sick?.available ?? 0);
2241         const usedThisMonth = sickMonthData?.usedThisMonth === true;
2242
2243         if (usedThisMonth) {
2244             // Explicitly used this month
2245             document.getElementById('cal-sick-remaining').textContent = '0';
2246             sickBadge.textContent = 'Used this month';
2247             sickBadge.style.background = 'var(--red-dim)';
2248             sickBadge.style.color = 'var(--red)';
2249             sickBadge.style.border = '1px solid rgba(248,113,113,0.25)';
2250         } else if (sickAvail === 0) {
2251             // New employee – accrual hasn't run yet
2252             document.getElementById('cal-sick-remaining').textContent = '0';
2253             sickBadge.textContent = 'Accrual pending';
2254             sickBadge.style.background = 'var(--amber-dim)';
2255             sickBadge.style.color = 'var(--amber)';
2256             sickBadge.style.border = '1px solid rgba(251,191,36,0.25)';
2257         } else {
2258             // Has balance and not used
2259             document.getElementById('cal-sick-remaining').textContent = sickAvail;
2260             sickBadge.textContent = 'Available this month';
2261             sickBadge.style.background = 'var(--green-dim)';
2262             sickBadge.style.color = 'var(--green)';
2263             sickBadge.style.border = '1px solid rgba(52,211,153,0.25)';
2264         }
2265
2266         /* CASUAL */
2267         const cas = dashboard?.casual;
2268         const casAvail = Number(cas?.available ?? 0);
2269         const casTotal = Number(cas?.opening ?? 0) + Number(cas?.accrued ?? 0);
2270         const casUsed = Number(cas?.used ?? 0);
2271
2272         document.getElementById('cal-casual-remaining').textContent = casAvail;
2273         document.getElementById('cal-casual-sub').textContent =
2274             `${casUsed} used · ${casTotal} total this year`;
2275
2276     } catch(e) {
2277         console.error('Could not load leave balances:', e);
2278         document.getElementById('cal-sick-remaining').textContent = '--';
2279         document.getElementById('cal-casual-remaining').textContent = '--';
2280     }
2281 }
2282
2283 async function buildCalendarEvents() {
2284     const events = [];
2285     try {
2286         const data = await apiFetch('/leave_requests/employee');
2287         const leaves = data?.requests || data || [];
2288         leaves.forEach(l => {
2289             const s = (l.status || 'PENDING').toUpperCase();
2290             const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
2291             let endDate = l.endDate;
2292             if(endDate) {
2293                 const d = new Date(endDate);
2294                 d.setDate(d.getDate() + 1);
2295                 endDate = d.toISOString().split('T')[0];
2296             }
2297             events.push({
2298                 id: `leave-${l.leaveRequestId}`,
2299                 title: `${TYPE_LABEL[l.leaveType] || l.leaveType}
2300 ($${s.charAt(0)+s.slice(1).toLowerCase()})`,
2301                 start: l.startDate, end: endDate, allDay: true,
2302                 backgroundColor: c.bg, borderColor: c.border, textColor: c.text,
2303                 extendedProps: { type:'leave', status:s, reason:l.reason||'-', leaveType:l.leaveType }
2304             });
2305         });
2306     }

```

```

2305     } catch(e) { console.warn('Could not load leaves for calendar'); }
2306
2307     try {
2308         const holidays = await apiFetch('/holidays');
2309         (holidays || []).forEach(h => {
2310             const c = STATUS_COLORS.HOLIDAY;
2311             events.push({
2312                 id: `holiday-${h.id||Math.random()}`,
2313                 title: ` ${h.name || 'Holiday'}`,
2314                 start: h.date || h.holidayDate, allDay: true,
2315                 backgroundColor: c.bg, borderColor: c.border, textColor: c.text,
2316                 extendedProps: { type:'holiday', description:h.description||'' }
2317             });
2318         });
2319     } catch(e) { console.warn('Could not load holidays for calendar'); }
2320
2321     return events;
2322 }
2323
2324 function initFullCalendar() {
2325     employeeCalendar = new FullCalendar.Calendar(
2326         document.getElementById('employee-calendar'), {
2327             initialView: 'dayGridMonth',
2328             height: 'auto',
2329             headerToolbar: {
2330                 left:'prev,next today', center:'title', right:'dayGridMonth,dayGridWeek,listMonth'
2331             },
2332             buttonText: { today:'Today', month:'Month', week:'Week', listMonth:'List' },
2333             dayMaxEvents: 3,
2334             navLinks: true,
2335             events: [],
2336             eventMouseEnter: function(info) {
2337                 const ep = info.event.extendedProps;
2338                 const tt = document.getElementById('cal-tooltip');
2339                 let html = `<div class="tt-title">${info.event.title}</div>`;
2340                 if(ep.type === 'leave') {
2341                     html += `<div class="tt-row"> ${info.event.startStr}</div>`;
2342                     if(ep.reason && ep.reason !== '-')
2343                         html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
2344                     html += `<div class="tt-row">Status: <strong
style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
2345                 } else if(ep.description) {
2346                     html += `<div class="tt-row">${ep.description}</div>`;
2347                 }
2348                 tt.innerHTML = html;
2349                 tt.style.display = 'block';
2350             },
2351             eventMouseLeave: function() {
2352                 document.getElementById('cal-tooltip').style.display = 'none';
2353             },
2354             eventClick: function(info) {
2355                 if(info.event.extendedProps.type === 'leave') {
2356                     showView('my-leaves', null);
2357                 }
2358             }
2359         });
2360     employeeCalendar.render();
2361     calendarInitialized = true;
2362     refreshCalendarEvents();
2363 }
2364
2365 async function refreshCalendarEvents() {
2366     if(!employeeCalendar) return;
2367     const events = await buildCalendarEvents();
2368     employeeCalendar.removeAllEvents();
2369     events.forEach(e => employeeCalendar.addEvent(e));
2370 }
2371
2372 async function refreshCalendar() {
2373     await loadCalendarBalanceCards();
2374     await refreshCalendarEvents();
2375     showAlert('Calendar refreshed!', 'success');
2376 }
2377
2378 document.addEventListener('mousemove', function(e) {
2379     const tt = document.getElementById('cal-tooltip');

```

```

2380     if(tt && tt.style.display === 'block') {
2381         tt.style.left = (e.clientX + 14) + 'px';
2382         tt.style.top = (e.clientY + 14) + 'px';
2383         const rect = tt.getBoundingClientRect();
2384         if(rect.right > window.innerWidth)
2385             tt.style.left = (e.clientX - rect.width - 10) + 'px';
2386         if(rect.bottom > window.innerHeight)
2387             tt.style.top = (e.clientY - rect.height - 10) + 'px';
2388     }
2389 });
2390 // ===== TWO-FACTOR AUTH =====
2391 async function loadTotpStatus() {
2392     const chip = document.getElementById('totp-status-chip');
2393     chip.textContent = 'Checking...';
2394     chip.className = 'chip';
2395     // Reset UI
2396     document.getElementById('qr-area').style.display = 'none';
2397     document.getElementById('totp-verify-input').value = '';
2398     document.getElementById('totp-verify-error').style.display = 'none';
2399
2400     try {
2401         const data = await apiFetch('/totp/status');
2402         if (data.totpEnabled) {
2403             chip.textContent = 'ENABLED';
2404             chip.className = 'chip active';
2405             document.getElementById('totp-setup-section').style.display = 'none';
2406             document.getElementById('totp-disable-section').style.display = 'block';
2407         } else {
2408             chip.textContent = 'DISABLED';
2409             chip.className = 'chip inactive';
2410             document.getElementById('totp-setup-section').style.display = 'block';
2411             document.getElementById('totp-disable-section').style.display = 'none';
2412         }
2413     } catch(e) {
2414         chip.textContent = 'Error';
2415         chip.className = 'chip pending';
2416     }
2417 }
2418
2419 async function generateTotpQr() {
2420     const btn = document.getElementById('generate-qr-btn');
2421     btn.textContent = 'Generating...';
2422     btn.disabled = true;
2423     try {
2424         const data = await apiFetch('/totp/setup', { method: 'POST' });
2425         // Show secret as manual entry fallback
2426         document.getElementById('totp-secret-display').textContent = data.secret || '';
2427         // Build QR image from the otpauth URI using a free QR API
2428         const qrUri = encodeURIComponent(data.qrCodeUri || '');
2429         document.getElementById('qr-img').src =
2430             `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
2431         document.getElementById('qr-area').style.display = 'block';
2432     } catch(e) {
2433         showAlert('Failed to generate QR: ' + e.message, 'error');
2434     } finally {
2435         btn.textContent = 'Regenerate QR Code';
2436         btn.disabled = false;
2437     }
2438 }
2439
2440 async function verifyAndActivateTotp() {
2441     const code = document.getElementById('totp-verify-input').value.trim();
2442     const errEl = document.getElementById('totp-verify-error');
2443     errEl.style.display = 'none';
2444     if (code.length !== 6) {
2445         errEl.textContent = 'Please enter a 6-digit code.';
2446         errEl.style.display = 'block';
2447         return;
2448     }
2449     try {
2450         await apiFetch('/totp/verify-setup', {
2451             method: 'POST',
2452             body: JSON.stringify({ code })
2453         });
2454         showAlert(' 2FA activated! You will need your authenticator on next login.', 'success');
2455         loadTotpStatus(); // refresh UI

```

```

2456     } catch(e) {
2457         errEl.textContent = 'Invalid code. Try again.';
2458         errEl.style.display = 'block';
2459     }
2460 }
2461
2462 async function disableTotp() {
2463     if (!confirm('Are you sure you want to disable two-factor authentication?')) return;
2464     try {
2465         await apiFetch('/totp/disable', { method: 'DELETE' });
2466         showAlert('2FA has been disabled.', 'success');
2467         loadTotpStatus();
2468     } catch(e) {
2469         showAlert('Failed to disable 2FA: ' + e.message, 'error');
2470     }
2471 }
2472
2473 async function createAdmin() {
2474     const body = {
2475         name: document.getElementById('admin-name').value.trim(),
2476         email: document.getElementById('admin-email').value.trim(),
2477         password: document.getElementById('admin-password').value,
2478         gender: document.getElementById('admin-gender').value,
2479         timezone: document.getElementById('admin-timezone').value || 'UTC'
2480     };
2481     if (!body.name || !body.email || !body.password) {
2482         showAlert('Please fill all required fields', 'error');
2483         return;
2484     }
2485     try {
2486         await apiFetch('/manager/createAdmin', { method: 'POST', body: JSON.stringify(body) });
2487         showAlert('Admin created successfully!', 'success');
2488         clearAdminForm();
2489     } catch(e) {
2490         showAlert('Failed to create admin: ' + e.message, 'error');
2491     }
2492 }
2493
2494 function clearAdminForm() {
2495     ['admin-name', 'admin-email', 'admin-password'].forEach(id =>
2496         document.getElementById(id).value = '');
2497     document.getElementById('admin-gender').selectedIndex = 0;
2498     document.getElementById('admin-timezone').value = 'Asia/Kolkata';
2499 }
2500 </script>
2501 </body>
2502 </html>
2503

```



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS - Device Management</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
8      <style>
9          * { margin: 0; padding: 0; box-sizing: border-box; }
10         :root {
11             --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
12             --panel: #1e2436; --panel2: #252c40;
13             --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
14             --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
15             --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
16             --green: #36d399; --green-dim: rgba(54,211,153,0.12);
17             --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
18             --red: #f87171; --red-dim: rgba(248,113,113,0.12);
19             --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
20             --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
21         }
22         body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-
height: 1.5; overflow: hidden; }
23         .app { display: flex; height: 100vh; }
24         .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display:
flex; flex-direction: column; overflow-y: auto; }
25         .logo { padding: 20px; border-bottom: 1px solid var(--border); }
26         .logo h2 { font-size: 20px; color: var(--text); }
27         .logo span { color: var(--accent); }
28         .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
29         .nav-item { padding: 12px 20px; margin: 4px 12px; border-radius: 8px; cursor: pointer; transition:
all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; }
30         .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
31         .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
32         .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border); }
33         .page-title { font-size: 20px; font-weight: 600; }
34         .user-info { display: flex; align-items: center; gap: 16px; }
35         .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid
rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; }
36         .content { flex: 1; overflow-y: auto; padding: 24px; }
37         .view { display: none; }
38         .view.active { display: block; }
39         .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px;
}
40         .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--
border); }
41         .stat-label { font-size: 11px; color: var(--text3); text-transform: uppercase; margin-bottom: 8px; }
42         .stat-value { font-size: 28px; font-weight: 600; }
43         .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 24px; max-width: 600px; }
44         .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
45         .form-group { display: flex; flex-direction: column; gap: 6px; }
46         .form-group.full { grid-column: 1 / -1; }
47         label { font-size: 11px; font-weight: 600; color: var(--text3); text-transform: uppercase; }
48         input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--
border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; }
49         input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
50         .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
overflow-x: auto; }
51         table { width: 100%; border-collapse: collapse; }
52         th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text3); font-size:
11px; font-weight: 600; text-transform: uppercase; }
53         td { padding: 12px 16px; border-top: 1px solid var(--border); }
54         .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-b
lock; }
55         .chip.active, .chip.approved, .chip.delivered, .chip.repair_done { background: var(--green-dim);
color: var(--green); }
56         .chip.pending, .chip.scheduled, .chip.sent_for_repair { background: var(--amber-dim); color: var(--
amber); }

```

```

57     .chip.open { background: var(--accent-dim); color: var(--accent); }
58     .chip.inactive, .chip.rejected, .chip.cancelled, .chip.expired { background: var(--red-dim); color:
var(--red); }
59     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
60     .chip.assigned { background: var(--green-dim); color: var(--green); }
61     .chip.condemned { background: var(--red-dim); color: var(--red); }
62     .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size:
13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
63     .btn-primary { background: var(--accent); color: white; }
64     .btn-primary:hover { background: #6ba0ff; }
65     .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
66     .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
67     .btn-sm { padding: 4px 10px; font-size: 11px; margin: 2px; }
68     .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom:
16px; }
69     .section-title { font-size: 16px; font-weight: 600; }
70     .empty-state { text-align: center; padding: 40px; color: var(--text3); }
71     .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
72     .modal-overlay.open { display: flex; }
73     .modal { background: var(--panel); border-radius: 12px; padding: 24px; width: 450px; max-width: 90%;
}
74     .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
75     .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
76     .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index:
1100; animation: slideIn 0.3s ease; }
77     .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid var(--green);
}
78     .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid var(--red); }
79     @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
80     .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
81     .back-btn { padding: 8px 16px; background: var(--bg3); border: 1px solid var(--border2); border-
radius: 6px; color: var(--text2); cursor: pointer; text-decoration: none; display: inline-fle...
82     .back-btn:hover { background: var(--panel2); color: var(--text); }
83     .device-card-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
gap: 16px; }
84     .device-card { background: var(--panel); border: 1px solid var(--border); border-radius: 8px;
padding: 16px; display: flex; flex-direction: column; gap: 14px; min-height: 210px; }
85     .device-card-head { display: flex; justify-content: space-between; gap: 12px; align-items: flex-
start; }
86     .device-icon { width: 42px; height: 42px; border-radius: 8px; display: grid; place-items: center;
background: var(--accent-dim); color: var(--accent); font-size: 22px; flex: 0 0 auto; }
87     .device-name { font-size: 16px; font-weight: 700; line-height: 1.25; }
88     .device-subtitle { color: var(--text2); font-size: 12px; margin-top: 4px; }
89     .device-meta { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-top: auto; }
90     .meta-box { background: var(--bg3); border: 1px solid var(--border); border-radius: 8px; padding:
10px; min-width: 0; }
91     .meta-label { color: var(--text3); text-transform: uppercase; font-size: 10px; font-weight: 700;
margin-bottom: 4px; }
92     .meta-value { color: var(--text); font-size: 12px; overflow-wrap: anywhere; }
93     .detail-layout { display: grid; grid-template-columns: minmax(260px, 360px) 1fr; gap: 18px; align-
items: start; }
94     .detail-panel { background: var(--panel); border: 1px solid var(--border); border-radius: 8px;
padding: 18px; }
95     .detail-title-row { display: flex; justify-content: space-between; gap: 12px; align-items: flex-
start; margin-bottom: 18px; }
96     .timeline { position: relative; display: flex; flex-direction: column; gap: 12px; }
97     .timeline-item { position: relative; display: grid; grid-template-columns: 34px 1fr; gap: 12px; }
98     .timeline-dot { width: 14px; height: 14px; border-radius: 50%; background: var(--accent); margin:
5px auto 0; box-shadow: 0 0 0 5px var(--accent-dim); }
99     .timeline-body { background: var(--bg3); border: 1px solid var(--border); border-radius: 8px;
padding: 12px; }
100     .timeline-title { font-weight: 700; margin-bottom: 4px; }
101     .timeline-description { color: var(--text2); font-size: 13px; }
102     .timeline-meta { display: flex; flex-wrap: wrap; gap: 8px; margin-top: 10px; color: var(--text3);
font-size: 11px; }
103     @media (max-width: 900px) {
104         body { overflow: auto; }
105         .app { min-height: 100vh; height: auto; }
106         .sidebar { width: 220px; }
107         .stats-grid { grid-template-columns: repeat(2, 1fr); }
108         .detail-layout { grid-template-columns: 1fr; }
109     }
110     @media (max-width: 700px) {

```

```

111     .app { display: block; }
112     .sidebar { width: 100%; height: auto; }
113     .topbar { align-items: flex-start; gap: 12px; flex-direction: column; }
114     .form-grid, .stats-grid { grid-template-columns: 1fr; }
115 }
116
117     .bill-card-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap:
16px; }
118 .bill-card { background: var(--panel); border: 1px solid var(--border); border-radius: 12px; padding: 18px;
display: flex; flex-direction: column; gap: 12px; }
119 .bill-card-head { display: flex; justify-content: space-between; align-items: flex-start; }
120 .bill-card-id { font-size: 11px; color: var(--text3); font-family: var(--mono); }
121 .bill-card-device { font-size: 15px; font-weight: 700; }
122 .bill-card-brand { font-size: 11px; color: var(--text2); margin-top: 2px; }
123 .bill-card-cost { font-size: 26px; font-weight: 700; color: var(--amber); font-family: var(--mono); }
124 .bill-card-divider { height: 1px; background: var(--border); }
125 .bill-card-meta { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
126 .bill-meta-item { display: flex; flex-direction: column; gap: 3px; }
127 .bill-meta-label { font-size: 10px; font-weight: 600; color: var(--text3); text-transform: uppercase;
letter-spacing: .5px; }
128 .bill-meta-value { font-size: 12px; color: var(--text2); font-family: var(--mono); }
129
130     .chip.overdue { background: var(--red-dim); color: var(--red); }
131 .chip.unpaid { background: var(--amber-dim); color: var(--amber); }
132 
133 </head>
134 <body>
135 <div class="app">
136     <!-- Sidebar -->
137     <div class="sidebar">
138         <div class="logo">
139             <h2>E<span>M</span>S</h2>
140             <p>Device Management</p>
141         </div>
142         <div style="flex: 1;">
143             <div class="nav-item" onclick="window.location.href='/dashboard.html'">
144                 <i class="ti ti-arrow-left"></i> Back to dashboard
145             </div>
146
147             <!-- OVERVIEW -->
148             <div style="padding: 10px 20px 4px; font-size: 10px; font-weight: 600; color: var(--text3);
text-transform: uppercase; letter-spacing: .8px;">Overview</div>
149             <div class="nav-item active" onclick="showView('overview', event)">&#9632; Overview</div>
150             <div class="nav-item" onclick="showView('all-devices', event)">&#9632; All Devices</div>
151             <div class="nav-item" onclick="showView('admin-service-requests', event)">&#9632; All Service
Requests</div>
152             <div class="nav-item" onclick="showView('bills', event)">&#9632; Repair Bills</div>
153             <div class="nav-item" onclick="showView('bills-rental', event)">&#9632; Rental Bills</div>
154
155             <!-- MY WORKSPACE -->
156             <div style="height: 1px; background: var(--border); margin: 6px 12px;"></div>
157             <div style="padding: 6px 20px 4px; font-size: 10px; font-weight: 600; color: var(--text3);
text-transform: uppercase; letter-spacing: .8px;">My Workspace</div>
158             <div class="nav-item" onclick="showView('my-devices', event)">&#9632; My Devices</div>
159             <div class="nav-item" onclick="showView('service-requests', event)">&#9632; My Service
Requests</div>
160
161         </div>
162     </div>
163
164     <!-- Main Content -->
165     <div class="main-content">
166         <div class="topbar">
167             <div class="page-title" id="pageTitle">Device Overview</div>
168             <div class="user-info">
169                 <span id="username-display"></span>
170                 <button class="logout-btn" onclick="logout()">Logout</button>
171             </div>
172         </div>
173         <div class="content" id="content">
174
175             <!-- OVERVIEW VIEW -->
176             <div id="overview-view" class="view active">
177                 <div class="stats-grid">
178                     <div class="stat-card">
179                         <div class="stat-label">Total Devices</div>

```

```

180         <div class="stat-value" id="total-devices">--</div>
181     </div>
182     <div class="stat-card">
183         <div class="stat-label">Assigned</div>
184         <div class="stat-value" id="assigned-devices">--</div>
185     </div>
186     <div class="stat-card">
187         <div class="stat-label">Under Repair</div>
188         <div class="stat-value" id="repair-devices">--</div>
189     </div>
190     <div class="stat-card">
191         <div class="stat-label">Open Requests</div>
192         <div class="stat-value" id="open-requests">--</div>
193     </div>
194 </div>
195
196 <div class="section-head">
197     <div class="section-title">Recent Service Requests</div>
198     <button class="btn btn-secondary btn-sm" onclick="loadOverview()">Refresh</button>
199 </div>
200 <div class="table-wrap">
201     <table>
202         <thead>
203             <tr>
204                 <th>ID</th>
205                 <th>Employee</th>
206                 <th>Device</th>
207                 <th>Type</th>
208                 <th>Status</th>
209                 <th>Raised At</th>
210             </tr>
211         </thead>
212         <tbody id="recent-requests"></tbody>
213     </table>
214 </div>
215 </div>
216
217 <!-- ALL DEVICES VIEW -->
218 <div id="all-devices-view" class="view">
219     <div class="section-head">
220         <div>
221             <div class="section-title">Device Cards</div>
222             <div style="color:var(--text2);font-size:12px">Scan inventory status, assignment,
warranty, and open a full device record.</div>
223         </div>
224         <button class="btn btn-primary btn-sm" onclick="loadAllDevices()">Refresh</button>
225     </div>
226     <div id="all-devices-list" class="device-card-grid"></div>
227 </div>
228
229 <!-- DEVICE DETAILS VIEW -->
230 <div id="device-detail-view" class="view">
231     <div class="section-head">
232         <button class="back-btn" onclick="showView('all-devices')"> Back to Devices</button>
233         <button class="btn btn-secondary btn-sm"
onclick="reloadCurrentDeviceDetail()">Refresh</button>
234     </div>
235     <div class="detail-layout">
236         <div class="detail-panel">
237             <div class="detail-title-row">
238                 <div>
239                     <div class="device-name" id="detail-device-name">Device</div>
240                     <div class="device-subtitle" id="detail-device-subtitle">Loading
details...</div>
241                 </div>
242                 <span class="chip" id="detail-device-status">--</span>
243             </div>
244             <div class="device-meta" style="grid-template-columns:1fr">
245                 <div class="meta-box"><div class="meta-label">Assigned To</div><div
class="meta-value" id="detail-assigned-to">--</div></div>
246                 <div class="meta-box"><div class="meta-label">Assigned On</div><div
class="meta-value" id="detail-assigned-date">--</div></div>
247                 <div class="meta-box"><div class="meta-label">Vendor</div><div class="meta-
value" id="detail-vendor">--</div></div>
248                 <div class="meta-box"><div class="meta-label">Warranty</div><div class="meta-
value" id="detail-warranty">--</div></div>

```

```

249         <div class="meta-box"><div class="meta-label">Created At</div><div class="meta-
value" id="detail-created-at">--</div></div>
250     </div>
251 </div>
252     <div class="detail-panel">
253         <div class="section-title" style="margin-bottom:16px">Timeline / Activity
Logs</div>
254         <div id="device-activity-list" class="timeline"></div>
255     </div>
256 </div>
257 </div>
258
259 <!-- MY DEVICES VIEW -->
260 <div id="my-devices-view" class="view">
261     <div class="section-head">
262         <div class="section-title">My Assigned Devices</div>
263         <button class="btn btn-primary btn-sm"
onclick="loadMyAssignedDevices()">Refresh</button>
264     </div>
265     <div class="table-wrap">
266         <table>
267             <thead>
268                 <tr>
269                     <th>ID</th>
270                     <th>Device Name</th>
271                     <th>Brand</th>
272                     <th>Type</th>
273                     <th>Warranty</th>
274                     <th>Assigned On</th>
275                     <th>Status</th>
276                 </tr>
277             </thead>
278             <tbody id="my-devices-list"></tbody>
279         </table>
280     </div>
281 </div>
282
283 <!-- MY SERVICE REQUESTS VIEW -->
284 <div id="service-requests-view" class="view">
285     <div class="section-head">
286         <div class="section-title">My Service Requests</div>
287         <button class="btn btn-primary btn-sm" onclick="loadServiceRequests()">Refresh</button>
288     </div>
289     <div class="table-wrap" style="margin-bottom:24px">
290         <table>
291             <thead>
292                 <tr>
293                     <th>ID</th>
294                     <th>Device</th>
295                     <th>Type</th>
296                     <th>Description</th>
297                     <th>Urgent</th>
298                     <th>Status</th>
299                     <th>Raised At</th>
300                 </tr>
301             </thead>
302             <tbody id="service-requests-list"></tbody>
303         </table>
304     </div>
305
306     <div class="section-title" style="margin-bottom:16px">Raise a New Request</div>
307     <div class="form-card">
308         <div class="form-grid">
309             <div class="form-group full">
310                 <label>Device</label>
311                 <select id="sr-device-select">
312                     <option value="">-- Select your device --</option>
313                 </select>
314             </div>
315             <div class="form-group">
316                 <label>Request Type</label>
317                 <select id="sr-type">
318                     <option value="REPAIR">Repair</option>
319                     <option value="REPLACEMENT">Replacement</option>
320                     <option value="RETURN">Return</option>
321                 </select>

```

```

322                 </div>
323                 <div class="form-group" style="display:flex;align-items:center;gap:10px;padding-
top:20px">
324                     <input type="checkbox" id="sr-urgent" style="width:16px;height:16px;padding:0">
325                     <label style="text-transform:none;font-size:13px;font-weight:400;color:var(--
text);cursor:pointer" for="sr-urgent">Mark as Urgent</label>
326                 </div>
327                 <div class="form-group full">
328                     <label>Issue Description</label>
329                     <textarea id="sr-description" rows="3" placeholder="Describe the issue in
detail..."></textarea>
330                 </div>
331             </div>
332             <div class="modal-actions" style="margin-top:20px">
333                 <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
Request</button>
334                 <button class="btn btn-secondary"
onclick="clearServiceRequestForm()">Clear</button>
335             </div>
336         </div>
337     </div>
338
339     <!-- ADMIN SERVICE REQUESTS VIEW -->
340     <div id="admin-service-requests-view" class="view">
341         <div class="section-head">
342             <div class="section-title">All Open Service Requests</div>
343             <button class="btn btn-primary btn-sm"
onclick="loadAdminServiceRequests()">Refresh</button>
344         </div>
345         <div class="table-wrap">
346             <table>
347                 <thead>
348                     <tr>
349                         <th>ID</th>
350                         <th>Employee</th>
351                         <th>Device</th>
352                         <th>Type</th>
353                         <th>Description</th>
354                         <th>Urgent</th>
355                         <th>Status</th>
356                         <th>Raised At</th>
357                         <th>Actions</th>
358                     </tr>
359                 </thead>
360                 <tbody id="admin-service-requests-list"></tbody>
361             </table>
362         </div>
363     </div>
364
365     <!-- REPAIR BILLS VIEW -->
366     <div id="bills-view" class="view">
367         <div class="section-head">
368             <div>
369                 <div class="section-title">Unpaid Repair Bills</div>
370                 <div style="color:var(--text2);font-size:12px">All outstanding bills pending
payment</div>
371             </div>
372             <button class="btn btn-primary btn-sm" onclick="loadRepairBills()">Refresh</button>
373         </div>
374         <div id="bills-card-grid" class="bill-card-grid"></div>
375     </div>
376
377     <!-- RENTAL BILLS VIEW -->
378     <div id="bills-rental-view" class="view">
379         <div class="section-head">
380             <div>
381                 <div class="section-title">Unpaid Rental Bills</div>
382                 <div style="color:var(--text2);font-size:12px">Outstanding vendor rental dues
pending payment</div>
383             </div>
384             <button class="btn btn-primary btn-sm" onclick="loadRentalBills()">Refresh</button>
385         </div>
386         <div id="rental-bills-card-grid" class="bill-card-grid"></div>
387     </div>
388
389 </div>

```

```

390     </div>
391 </div>
392
393 <!-- ASSIGN DEVICE MODAL -->
394 <div id="assign-modal" class="modal-overlay">
395     <div class="modal">
396         <div class="modal-title">Assign Device to Employee</div>
397         <input type="hidden" id="assign-device-id">
398         <div class="form-group" style="margin-bottom:12px">
399             <label>Device</label>
400             <input type="text" id="assign-device-name" readonly style="color:var(--text2)">
401         </div>
402         <div class="form-group" style="margin-bottom:12px">
403             <label>Employee</label>
404             <select id="assign-employee-select">
405                 <option value="">-- Select Employee --</option>
406             </select>
407         </div>
408         <div class="modal-actions">
409             <button class="btn btn-primary" onclick="confirmAssign()">Assign</button>
410             <button class="btn btn-secondary" onclick="closeAssignModal()">Cancel</button>
411         </div>
412     </div>
413 </div>
414
415 <!-- ADMIN SERVICE REQUEST ACTION MODAL -->
416 <div id="admin-sr-modal" class="modal-overlay">
417     <div class="modal">
418         <div class="modal-title">Update Service Request <span id="admin-sr-modal-id" style="color:var(--
accent)"></span></div>
419         <input type="hidden" id="admin-sr-id">
420
421         <div class="form-group" style="margin-bottom:14px">
422             <label>Device</label>
423             <input type="text" id="admin-sr-device-display" readonly style="color:var(--text2)">
424         </div>
425         <div class="form-group" style="margin-bottom:14px">
426             <label>Employee</label>
427             <input type="text" id="admin-sr-employee-display" readonly style="color:var(--text2)">
428         </div>
429         <div class="form-group" style="margin-bottom:14px">
430             <label>Update Status</label>
431             <select id="admin-sr-status">
432                 <option value="SENT_FOR_REPAIR">Sent for Repair – device will be marked Under
Repair</option>
433                 <option value="REPAIR_DONE">Repair Done</option>
434                 <option value="REJECTED">Rejected</option>
435             </select>
436         </div>
437         <div class="form-group" style="margin-bottom:4px">
438             <label>Admin Remarks</label>
439             <textarea id="admin-sr-remarks" rows="3" placeholder="Add remarks or instructions for the
vendor..."></textarea>
440         </div>
441         <div id="admin-sr-repair-hint" style="font-size:11px;color:var(--amber);margin-
top:6px;display:none;">
442             Selecting "Sent for Repair" will automatically mark the device as <strong>Under
Repair</strong>.
443         </div>
444
445         <input type="hidden" id="admin-sr-request-type">
446
447         <!-- Show only for REPLACEMENT -->
448         <div class="form-group" id="replacement-device-group" style="display:none;margin-bottom:14px">
449             <label>Replacement Device</label>
450             <select id="admin-sr-replacement-device">
451                 <option value="">-- Select Available Device --</option>
452             </select>
453         </div>
454
455         <div class="modal-actions">
456             <button class="btn btn-primary" onclick="submitAdminSRUpdate()">Update Request</button>
457             <button class="btn btn-secondary" onclick="closeAdminSRModal()">Cancel</button>
458         </div>
459     </div>
460 </div>

```

```

461
462 <script>
463     const API = 'http://localhost:8080';
464     let currentUser = null;
465     let currentDeviceId = null;
466
467     function escapeHtml(value) {
468         return String(value ?? '').replace(/[\<>"]/g, ch => ({
469             '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;';
470         }[ch]));
471     }
472
473     function escapeJs(value) {
474         return String(value ?? '').replace(/\\/g, '\\\\').replace(/'/g, '\\\'');
475     }
476
477     function formatDate(value, includeTime = false) {
478         if (!value) return '--';
479         const date = new Date(value);
480         if (Number.isNaN(date.getTime())) return value;
481         return includeTime ? date.toLocaleString() : date.toLocaleDateString();
482     }
483
484     function titleCase(value) {
485         return String(value || '--').replace(/_/g, ' ').toLowerCase().replace(/b\w/g, c =>
486             c.toUpperCase());
487     }
488
489     function deviceIcon(type) {
490         const icons = { LAPTOP: '', DESKTOP: '', MOBILE: '', TABLET: '', MONITOR: '', KEYBOARD: '', MOUSE:
491             '', HEADSET: '' };
492         return icons[type] || '';
493     }
494
495     (async function initAuth() {
496         const token = localStorage.getItem('jwt_token');
497         const authType = localStorage.getItem('auth_type');
498         if (!token && authType !== 'session') {
499             window.location.href = '/google.html';
500             return;
501         }
502
503         try {
504             const response = await fetch(`${API}/employee/me`, {
505                 headers: getAuthHeader(),
506                 credentials: 'include'
507             });
508
509             if (response.status === 401) { window.location.href = '/google.html'; return; }
510             if (!response.ok) { window.location.href = '/google.html'; return; }
511
512             const employee = await response.json();
513             currentUser = employee;
514             document.getElementById('username-display').textContent = employee.email || employee.name ||
515                 'User';
516             loadOverview();
517         } catch (e) {
518             window.location.href = '/google.html';
519         }
520     })();
521
522     function getAuthHeader() {
523         const token = localStorage.getItem('jwt_token');
524         return token ? { 'Authorization': `Bearer ${token}` } : {};
525     }
526
527     async function apiFetch(path, opts = {}) {
528         const { silent = false, ...fetchOptions } = opts;
529         try {
530             const response = await fetch(`${API}${path}`, {
531                 ...fetchOptions,
532                 headers: {
533                     'Content-Type': 'application/json',
534                     ...getAuthHeader(),
535                     ...fetchOptions.headers
536                 },

```



```

534         credentials: 'include'
535     });
536
537     if (response.status === 401) {
538         localStorage.removeItem('jwt_token');
539         localStorage.removeItem('auth_type');
540         showAlert('Session expired. Please login again.', 'error');
541         setTimeout(() => window.location.href = '/google.html', 1500);
542         throw new Error('Unauthorized');
543     }
544
545     if (!response.ok) {
546         const error = await response.text();
547         throw new Error(error || 'Request failed');
548     }
549
550     const text = await response.text();
551     if (!text) return null;
552     try { return JSON.parse(text); } catch { return text; }
553 } catch (e) {
554     if (!silent && !e.message.includes('Unauthorized')) showAlert(e.message, 'error');
555     throw e;
556 }
557 }
558
559 function showAlert(message, type) {
560     const alert = document.createElement('div');
561     alert.className = `alert alert-${type}`;
562     alert.textContent = message;
563     document.body.appendChild(alert);
564     setTimeout(() => alert.remove(), 3000);
565 }
566
567 function showView(viewName, event) {
568     document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
569     document.getElementById(`${viewName}-view`).classList.add('active');
570     document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
571     if (event && event.target) event.target.classList.add('active');
572     if (!event) {
573         const nav = Array.from(document.querySelectorAll('.nav-item'))
574             .find(n => n.getAttribute('onclick')?.includes(`${viewName}`));
575         if (nav) nav.classList.add('active');
576     }
577     document.getElementById('pageTitle').textContent = viewName.split('-').map(w =>
578         w.charAt(0).toUpperCase() + w.slice(1)).join(' ');
579
580     const loaders = {
581         'overview': loadOverview,
582         'all-devices': loadAllDevices,
583         'my-devices': loadMyAssignedDevices,
584         'service-requests': loadServiceRequests,
585         'admin-service-requests': loadAdminServiceRequests,
586         'add-device': loadVendorsForDevice,
587         'bills': loadRepairBills,
588         'bills-rental': loadRentalBills
589     };
590     if (loaders[viewName]) loaders[viewName]();
591 }
592
593 // ===== OVERVIEW =====
594 async function loadOverview() {
595     try {
596         const [allDevices, requests] = await Promise.all([
597             apiFetch('/devices/all'),
598             apiFetch('/service-requests/open')
599         ]);
600
601         document.getElementById('total-devices').textContent = allDevices?.length || 0;
602         document.getElementById('assigned-devices').textContent =
603             allDevices?.filter(d => d.deviceStatus === 'ASSIGNED').length || 0;
604         document.getElementById('repair-devices').textContent =
605             allDevices?.filter(d => d.deviceStatus === 'UNDER_REPAIR').length || 0;
606         document.getElementById('open-requests').textContent = requests?.length || 0;
607
608         const recent = (requests || []).slice(0, 5);
609         const tbody = document.getElementById('recent-requests');

```

```

610         if (!recent.length) {
611             tbody.innerHTML = '<tr><td colspan="6" class="empty-state">No recent requests</td></tr>';
612         } else {
613             tbody.innerHTML = recent.map(r => `<tr>
614                 <td>#${r.id}</td>
615                 <td>${r.raisedByName || '-'}</td>
616                 <td>${r.deviceName || '-'}</td>
617                 <td><span class="chip">${r.requestType}</span></td>
618                 <td><span class="chip ${r.status?.toLowerCase()}>${r.status}</span></td>
619                 <td style="font-size:12px">${r.raisedAt ? new Date(r.raisedAt).toLocaleDateString() :
        '-'}</td>
620             </tr>`).join('');
621         }
622     } catch (e) { console.error(e); }
623 }
624
625 // ===== ALL DEVICES =====
626 async function loadAllDevices() {
627     try {
628         const devices = await loadAccessibleDevices();
629         const list = document.getElementById('all-devices-list');
630         if (!devices || !devices.length) {
631             list.innerHTML = '<div class="empty-state">No devices found</div>';
632             return;
633         }
634
635         const today = new Date();
636         list.innerHTML = devices.map(d => {
637             const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
638             const warrantyDisplay = warrantyExpiry
639                 ? (warrantyExpiry > today
640                     ? `<span style="color:var(--green)">Active until
641                       ${escapeHtml(d.warrantyExpiryDate)}</span>`
642                     : `<span style="color:var(--red)">Expired
643                       ${escapeHtml(d.warrantyExpiryDate)}</span>`)
644                 : `<span style="color:var(--text3)">--</span>`;
645
646             let actions = '';
647             if (d.deviceStatus === 'AVAILABLE') {
648                 actions = `<button class="btn btn-primary btn-sm" onclick="event.stopPropagation();
649 openAssignModal(${d.id}, '${escapeJs(d.deviceName)}')">Assign</button>`;
650             } else if (d.deviceStatus === 'REPAIR_DONE') {
651                 actions = `<button class="btn btn-sm" style="background:var(--violet-dim);color:var(--
652 violet);border:1px solid rgba(167,139,250,0.3)" onclick="event.stopPropagation(); reassignRepairedDevice(${...
653 }else if (d.deviceStatus === 'CONDEMNED') {
654                 actions = `<button class="btn btn-danger btn-sm" onclick="event.stopPropagation();
655 returnCondemnedDevice(${d.id}, '${escapeJs(d.deviceName)}')">Return to Vendor</button>`;
656             }
657
658             return `<div class="device-card" onclick="openDeviceDetail(${d.id})">
659                 <div class="device-card-head">
660                     <div style="display:flex;gap:12px;min-width:0">
661                         <div class="device-icon">${deviceIcon(d.deviceType)}</div>
662                         <div style="min-width:0">
663                             <div class="device-name">${escapeHtml(d.deviceName || '-')}</div>
664                             <div class="device-subtitle">#${d.id} · ${escapeHtml(d.brand || '-')} ·
665 ${titleCase(d.deviceType)}</div>
666                         </div>
667                     </div>
668                     <div>
669                         <span class="chip ${((d.deviceStatus ||
670 '').toLowerCase())}>${titleCase(d.deviceStatus)}</span>
671                     </div>
672                     <div class="device-meta">
673                         <div class="meta-box"><div class="meta-label">Assigned To</div><div class="meta-
674 value">${escapeHtml(d.assignedEmployeeName || '-')}</div></div>
675                         <div class="meta-box"><div class="meta-label">Assigned On</div><div class="meta-
676 value">${escapeHtml(d.assignedDate || '-')}</div></div>
677                         <div class="meta-box"><div class="meta-label">Vendor</div><div class="meta-
678 value">${escapeHtml(d.vendorName || '-')}</div></div>
679                         <div class="meta-box"><div class="meta-label">Warranty</div><div class="meta-
680 value">${warrantyDisplay}</div></div>
681                     </div>
682                     <div class="action-row">
683                         <button class="btn btn-secondary btn-sm" onclick="event.stopPropagation();
684 openDeviceDetail(${d.id})">Details</button>
685                         ${actions}

```

```

673         </div>
674     </div>`;
675     }).join('');
676     } catch (e) { console.error(e); }
677 }
678
679 async function loadAccessibleDevices() {
680     const endpoints = ['/devices/all', '/devices/vendor/me', '/devices/my'];
681     for (const endpoint of endpoints) {
682         try {
683             return await apiFetch(endpoint, { silent: endpoint !== endpoints[endpoints.length - 1] });
684         } catch (e) {
685             if (endpoint === endpoints[endpoints.length - 1]) throw e;
686         }
687     }
688     return [];
689 }
690
691 async function openDeviceDetail(deviceId) {
692     currentDeviceId = deviceId;
693     showView('device-detail');
694     await reloadCurrentDeviceDetail();
695 }
696
697 async function reloadCurrentDeviceDetail() {
698     if (!currentDeviceId) return;
699     try {
700         const [device, activity] = await Promise.all([
701             apiFetch(`/devices/${currentDeviceId}`),
702             apiFetch(`/devices/${currentDeviceId}/activity`)
703         ]);
704
705         document.getElementById('pageTitle').textContent = `Device #${device.id}`;
706         document.getElementById('detail-device-name').textContent = device.deviceName || '--';
707         document.getElementById('detail-device-subtitle').textContent = `${device.brand || '--'} .
708         ${titleCase(device.deviceType)}`;
709         const status = document.getElementById('detail-device-status');
710         status.className = `chip ${device.deviceStatus || ''}.toLowerCase()`;
711         status.textContent = titleCase(device.deviceStatus);
712         document.getElementById('detail-assigned-to').textContent = device.assignedEmployeeName || '--';
713         document.getElementById('detail-assigned-date').textContent = device.assignedDate || '--';
714         document.getElementById('detail-vendor').textContent = device.vendorName || '--';
715         document.getElementById('detail-warranty').textContent = device.warrantyExpiryDate || '--';
716         document.getElementById('detail-created-at').textContent = formatDate(device.createdAt, true);
717
718         const timeline = document.getElementById('device-activity-list');
719         if (!activity || !activity.length) {
720             timeline.innerHTML = '<div class="empty-state">No activity logs found for this
721             device</div>';
722             return;
723         }
724
725         timeline.innerHTML = activity.map(item => `<div class="timeline-item">
726             <div class="timeline-dot"></div>
727             <div class="timeline-body">
728                 <div style="display:flex;justify-content:space-between;gap:12px;align-items:flex-
729                 start">
730                     <div class="timeline-title">${escapeHtml(item.title || titleCase(item.type))}</div>
731                     ${item.status ? `<span class="chip ${item.status ||
732                     ''}.toLowerCase()>${titleCase(item.status)}</span>` : ''}
733                 </div>
734                 <div class="timeline-description">${escapeHtml(item.description || '--')}</div>
735                 <div class="timeline-meta">
736                     <span>${formatDate(item.occurredAt, true)}</span>
737                     <span>By ${escapeHtml(item.actorName)}</span>` : ''}
738                     <span>${item.type ? `<span>${titleCase(item.type)}</span>` : ''}
739                 </div>
740             </div>
741             </div>`).join('');
742     } catch (e) { console.error(e); }
743 }
744
745 // ===== MY DEVICES =====
746 async function loadMyAssignedDevices() {
747     try {

```

```

744         const devices = await apiFetch('/devices');
745         const tbody = document.getElementById('my-devices-list');
746         if (!devices || !devices.length) {
747             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
748             return;
749         }
750
751         const today = new Date();
752         tbody.innerHTML = devices.map(d => {
753             const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
754             const warrantyDisplay = warrantyExpiry
755                 ? (warrantyExpiry > today
756                     ? `<span style="color:var(--green);font-size:11px"> ${d.warrantyExpiryDate}</span>`
757                     : `<span style="color:var(--red);font-size:11px"> Expired
758 $${d.warrantyExpiryDate}</span>`)
759                 : `<span style="color:var(--text3);font-size:11px">--</span>`;
760             return `<tr>
761                 <td>#${d.id}</td>
762                 <td><strong>${d.deviceName || '-'}</strong></td>
763                 <td>${d.brand || '-'}</td>
764                 <td><span class="chip">${(d.deviceType || '').toLowerCase()}</span></td>
765                 <td>${warrantyDisplay}</td>
766                 <td style="font-size:12px;color:var(--text2)">${d.assignedDate || '-'}</td>
767                 <td><span class="chip ${d.deviceStatus || ''}.toLowerCase()}>${d.deviceStatus ||
768 '-'}</span></td>
769             </tr>`;
770         }).join('');
771     } catch (e) { console.error(e); }
772 }
773
774 // ===== SERVICE REQUESTS =====
775 async function loadServiceRequests() {
776     await Promise.all([loadServiceRequestsList(), loadMyDevicesForSR()]);
777 }
778
779 async function loadServiceRequestsList() {
780     try {
781         const requests = await apiFetch('/service-requests/my');
782         const tbody = document.getElementById('service-requests-list');
783         if (!requests || !requests.length) {
784             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No service requests
yet</td></tr>';
785             return;
786         }
787         tbody.innerHTML = requests.map(r => `<tr>
788             <td>#${r.id}</td>
789             <td>${r.deviceName || '-'}</td>
790             <td><span class="chip">${r.requestType}</span></td>
791             <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap">${r.issueDescription || '-'}</td>
792             <td>${r.urgent ? `<span class="chip pending">YES</span>` : `<span style="color:var(--
text2)">No</span>`}</td>
793             <td><span class="chip ${r.status?.toLowerCase()}>${r.status}</span></td>
794             <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new
Date(r.raisedAt).toLocaleDateString() : '-'}</td>
795             </tr>`).join('');
796     } catch (e) { console.error(e); }
797 }
798
799 async function loadMyDevicesForSR() {
800     try {
801         const devices = await apiFetch('/devices/my');
802         const select = document.getElementById('sr-device-select');
803         select.innerHTML = '<option value="">-- Select your device --</option>';
804         (devices || []).forEach(d => {
805             const opt = document.createElement('option');
806             opt.value = d.id;
807             opt.textContent = `${d.deviceName} (${d.brand || d.deviceType || 'Device'})`;
808             select.appendChild(opt);
809         });
810     } catch (e) { console.error('Failed to load devices for SR', e); }
811 }
812
813 async function submitServiceRequest() {

```

```

813     const deviceId = parseInt(document.getElementById('sr-device-select').value);
814     const requestType = document.getElementById('sr-type').value;
815     const issueDescription = document.getElementById('sr-description').value.trim();
816     const urgent = document.getElementById('sr-urgent').checked;
817
818     if (!deviceId) { showAlert('Please select a device', 'error'); return; }
819     if (!issueDescription) { showAlert('Please describe the issue', 'error'); return; }
820
821     try {
822         await apiFetch('/service-requests', {
823             method: 'POST',
824             body: JSON.stringify({ deviceId, requestType, issueDescription, urgent })
825         });
826         showAlert('Service request submitted!', 'success');
827         clearServiceRequestForm();
828         loadServiceRequestsList();
829         loadOverview();
830     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
831 }
832
833 function clearServiceRequestForm() {
834     document.getElementById('sr-device-select').value = '';
835     document.getElementById('sr-type').value = 'REPAIR';
836     document.getElementById('sr-description').value = '';
837     document.getElementById('sr-urgent').checked = false;
838 }
839
840 // ===== ADMIN SERVICE REQUESTS =====
841 async function loadAdminServiceRequests() {
842     try {
843         const requests = await apiFetch('/service-requests/open');
844         const tbody = document.getElementById('admin-service-requests-list');
845         if (!requests || !requests.length) {
846             requests</td></tr>';
847             return;
848         }
849         tbody.innerHTML = requests.map(r => `<tr>
850             <td>#${r.id}</td>
851             <td style="font-size:12px;color:var(--text2)">${r.raisedByName || r.employeeId || '-'}</td>
852             <td><strong>${r.deviceName || '-'}</strong></td>
853             <td><span class="chip">${r.requestType}</span></td>
854             <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;font-size:12px;color:var(--text2)" title="${r.issueDescription || ''}">${r.issueDescription || '...'}
855             <td>${r.urgent ? '<span class="chip pending">YES</span>' : '<span style="color:var(--text2);font-size:12px">No</span>'}</td>
856             <td><span class="chip ${r.status?.toLowerCase()}>${r.status}</span></td>
857             <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new
Date(r.raisedAt).toLocaleDateString() : '-'}</td>
858             <td>
859                 <button class="btn btn-primary btn-sm" onclick="openAdminSRModal(${r.id},
'${(r.deviceName||'').replace(/'/g,"\\'")}', '${(r.raisedByName||'').replace(/'/g,"\\'")}', '${r.reques...
860                 Update
861             </button>
862         </td>
863     </tr>`).join('');
864     } catch (e) { console.error(e); }
865 }
866
867 function openAdminSRModal(requestId, deviceName, employeeName, requestType) {
868     document.getElementById('admin-sr-id').value = requestId;
869     document.getElementById('admin-sr-request-type').value = requestType;
870     document.getElementById('admin-sr-modal-id').textContent = `#${requestId}`;
871     document.getElementById('admin-sr-device-display').value = deviceName;
872     document.getElementById('admin-sr-employee-display').value = employeeName;
873     document.getElementById('admin-sr-remarks').value = '';
874     document.getElementById('admin-sr-repair-hint').style.display = 'none';
875     document.getElementById('replacement-device-group').style.display = 'none';
876
877     const statusSelect = document.getElementById('admin-sr-status');
878
879     if (requestType === 'REPAIR') {
880         // REPAIR: SENT_FOR_REPAIR or REJECTED
881         statusSelect.innerHTML = `
882             <option value="SENT_FOR_REPAIR">Sent for Repair – device marked Under Repair</option>
883             <option value="REJECTED">Rejected</option>

```

```

884     `;
885     document.getElementById('admin-sr-repair-hint').style.display = 'block';
886
887     } else if (requestType === 'REPLACEMENT') {
888         statusSelect.innerHTML = `
889             <option value="APPROVED">Approved – assign replacement device</option>
890             <option value="REJECTED">Rejected</option>
891         `;
892         // Show the dropdown immediately since default is APPROVED
893         document.getElementById('replacement-device-group').style.display = 'block';
894         loadAvailableDevicesForReplacement();
895
896     } else if (requestType === 'RETURN') {
897         // RETURN: APPROVED or REJECTED
898         statusSelect.innerHTML = `
899             <option value="APPROVED">Approved – device will be unassigned</option>
900             <option value="REJECTED">Rejected</option>
901         `;
902     }
903
904     document.getElementById('admin-sr-modal').classList.add('open');
905 }
906
907 document.getElementById('admin-sr-status')?.addEventListener('change', function () {
908     const requestType = document.getElementById('admin-sr-request-type').value;
909
910     // REPAIR hint
911     document.getElementById('admin-sr-repair-hint').style.display =
912         (requestType === 'REPAIR' && this.value === 'SENT_FOR_REPAIR') ? 'block' : 'none';
913
914     // Replacement device dropdown
915     document.getElementById('replacement-device-group').style.display =
916         (requestType === 'REPLACEMENT' && this.value === 'APPROVED') ? 'block' : 'none';
917 });
918
919 async function submitAdminSRUpdate() {
920     const serviceRequestId = parseInt(document.getElementById('admin-sr-id').value);
921     const status = document.getElementById('admin-sr-status').value;
922     const adminRemarks = document.getElementById('admin-sr-remarks').value.trim();
923     const requestType = document.getElementById('admin-sr-request-type').value;
924
925     if (!adminRemarks) {
926         showAlert('Please add remarks before updating', 'error');
927         return;
928     }
929
930     try {
931         if (requestType === 'REPAIR') {
932             // POST to /admin/repair with AdminActionDTO
933             await apiFetch('/service-requests/admin/repair', {
934                 method: 'PUT',
935                 body: JSON.stringify({ serviceRequestId, status, adminRemarks })
936             });
937
938         } else {
939             // REPLACEMENT or RETURN /admin/other with ApprovalActionDTO
940             const replacementDeviceId = requestType === 'REPLACEMENT' && status === 'APPROVED'
941                 ? parseInt(document.getElementById('admin-sr-replacement-device').value) || null
942                 : null;
943
944             if (requestType === 'REPLACEMENT' && status === 'APPROVED' && !replacementDeviceId) {
945                 showAlert('Please select a replacement device', 'error');
946                 return;
947             }
948
949             await apiFetch('/service-requests/admin/other', {
950                 method: 'PUT',
951                 body: JSON.stringify({ serviceRequestId, status, adminRemarks, replacementDeviceId })
952             });
953         }
954
955         showAlert('Service request updated successfully!', 'success');
956         closeAdminSRModal();
957         loadAdminServiceRequests();
958         loadOverview();
959     }

```

```

960     } catch (e) {
961         showAlert('Failed: ' + e.message, 'error');
962     }
963 }
964
965 function closeAdminSRModal() {
966     document.getElementById('admin-sr-modal').classList.remove('open');
967     document.getElementById('admin-sr-id').value = '';
968     document.getElementById('admin-sr-request-type').value = '';
969     document.getElementById('admin-sr-remarks').value = '';
970     document.getElementById('admin-sr-repair-hint').style.display = 'none';
971     document.getElementById('replacement-device-group').style.display = 'none';
972 }
973
974 async function loadAvailableDevicesForReplacement() {
975     const select = document.getElementById('admin-sr-replacement-device');
976     select.innerHTML = '<option value="">Loading...</option>';
977     try {
978         const devices = await apiFetch('/devices/all');
979         const available = (devices || []).filter(d => d.deviceStatus === 'AVAILABLE');
980         if (!available.length) {
981             select.innerHTML = '<option value="">No available devices</option>';
982             return;
983         }
984         select.innerHTML = '<option value="">-- Select Available Device --</option>' +
985             available.map(d =>
986                 `<option value="${d.id}">${escapeHtml(d.deviceName)} - ${escapeHtml(d.brand || '')}
987                 (${titleCase(d.deviceType)})</option>`
988             ).join('');
989     } catch (e) {
990         select.innerHTML = '<option value="">Failed to load devices</option>';
991     }
992 }
993 // ===== ASSIGN DEVICE =====
994 async function openAssignModal(deviceId, deviceName) {
995     document.getElementById('assign-device-id').value = deviceId;
996     document.getElementById('assign-device-name').value = deviceName;
997     try {
998         const employees = await apiFetch('/employee');
999         const select = document.getElementById('assign-employee-select');
1000         select.innerHTML = '<option value="">-- Select Employee --</option>';
1001         employees.filter(e => e.status === 'ACTIVE').forEach(e => {
1002             const opt = document.createElement('option');
1003             opt.value = e.employeeId;
1004             opt.textContent = `${e.name} (${e.email}) - ${e.dept || 'N/A'}`;
1005             select.appendChild(opt);
1006         });
1007     } catch (e) { showAlert('Failed to load employees', 'error'); return; }
1008     document.getElementById('assign-modal').classList.add('open');
1009 }
1010
1011 async function confirmAssign() {
1012     const deviceId = document.getElementById('assign-device-id').value;
1013     const employeeId = document.getElementById('assign-employee-select').value;
1014     if (!employeeId) { showAlert('Please select an employee', 'error'); return; }
1015     try {
1016         const message = await apiFetch(`/devices/${deviceId}/assign`, {
1017             method: 'POST',
1018             body: JSON.stringify({ employeeId: parseInt(employeeId) })
1019         });
1020         showAlert(message || 'Device assigned successfully!', 'success');
1021         closeAssignModal();
1022         loadAllDevices();
1023         loadOverview();
1024     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1025 }
1026
1027 function closeAssignModal() {
1028     document.getElementById('assign-modal').classList.remove('open');
1029     document.getElementById('assign-employee-select').value = '';
1030     document.getElementById('assign-device-id').value = '';
1031     document.getElementById('assign-device-name').value = '';
1032 }
1033
1034 async function reassignRepairedDevice(deviceId, deviceName) {
1035     if (!confirm(`Reassign "${deviceName}" back to the previous employee?`)) return;

```

```

1035     try {
1036         const device = await apiFetch(`/devices/${deviceId}/mark-assigned`, { method: 'PUT' });
1037         showAlert(`Device reassigned to ${device.assignedEmployeeName} successfully!`, 'success');
1038         loadAllDevices();
1039         loadOverview();
1040     } catch (e) {
1041         showAlert('Failed: ' + e.message, 'error');
1042     }
1043 }
1044 async function returnCondemnedDevice(deviceId, deviceName) {
1045     if (!confirm(`Return condemned device "${deviceName}" to vendor? This will remove the assignment.`))
1046         return;
1047     try {
1048         const message = await apiFetch(`/devices/condemned/remove/${deviceId}`, { method: 'PUT' });
1049         showAlert(message || 'Device returned to vendor successfully!', 'success');
1050         loadAllDevices();
1051         loadOverview();
1052     } catch (e) {
1053         showAlert('Failed: ' + e.message, 'error');
1054     }
1055 }
1056 // ===== ADD DEVICE =====
1057 async function loadVendorsForDevice() {
1058     try {
1059         const vendors = await apiFetch('/vendors');
1060         const select = document.getElementById('device-vendor');
1061         select.innerHTML = '<option value="">-- Select Vendor --</option>';
1062         (vendors || []).forEach(v => {
1063             const opt = document.createElement('option');
1064             opt.value = v.id;
1065             opt.textContent = `${v.name} (${v.email})`;
1066             select.appendChild(opt);
1067         });
1068     } catch (e) { console.error('Failed to load vendors', e); }
1069 }
1070
1071 async function addDevice() {
1072     const body = {
1073         deviceName: document.getElementById('device-name').value.trim(),
1074         brand: document.getElementById('device-brand').value.trim(),
1075         deviceType: document.getElementById('device-type').value,
1076         warrantyExpiryDate: document.getElementById('device-warranty').value,
1077         vendorId: parseInt(document.getElementById('device-vendor').value) || null,
1078         description: document.getElementById('device-description').value.trim()
1079     };
1080
1081     if (!body.deviceName || !body.brand) {
1082         showAlert('Please enter device name and brand', 'error');
1083         return;
1084     }
1085
1086     try {
1087         await apiFetch('/devices', {
1088             method: 'POST',
1089             body: JSON.stringify(body)
1090         });
1091         showAlert('Device added successfully!', 'success');
1092         clearDeviceForm();
1093         loadAllDevices();
1094         loadOverview();
1095     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1096 }
1097
1098 function clearDeviceForm() {
1099     document.getElementById('device-name').value = '';
1100     document.getElementById('device-brand').value = '';
1101     document.getElementById('device-type').value = 'LAPTOP';
1102     document.getElementById('device-warranty').value = '';
1103     document.getElementById('device-vendor').value = '';
1104     document.getElementById('device-description').value = '';
1105 }
1106
1107 // ===== REPAIR BILLS =====
1108 async function loadRepairBills() {
1109     try {

```



```

1110     const bills = await apiFetch('/bills/repairBill');
1111     const grid = document.getElementById('bills-card-grid');
1112     if (!bills || !bills.length) {
1113         grid.innerHTML = '<div class="empty-state">No unpaid bills found</div>';
1114         return;
1115     }
1116     grid.innerHTML = bills.map(b => `
1117         <div class="bill-card">
1118             <div class="bill-card-head">
1119                 <div>
1120                     <div class="bill-card-id">Bill #${b.billId}</div>
1121                     <div class="bill-card-device">${escapeHtml(b.deviceName || '-')}</div>
1122                     <div class="bill-card-brand">${escapeHtml(b.repairedBy || '-')}</div>
1123                 </div>
1124                 <span class="chip ${b.status || ''}.toLowerCase()">${b.status || '-'}</span>
1125             </div>
1126             <div class="bill-card-cost">${b.repairCost ? '' + b.repairCost : '-'}</div>
1127             <div class="bill-card-divider"></div>
1128             <div class="bill-card-meta">
1129                 <div class="bill-meta-item">
1130                     <div class="bill-meta-label">Repaired By</div>
1131                     <div class="bill-meta-value">${escapeHtml(b.repairedBy || '-')}</div>
1132                 </div>
1133                 <div class="bill-meta-item">
1134                     <div class="bill-meta-label">Repair Date</div>
1135                     <div class="bill-meta-value">${b.repairedDate || '-'}</div>
1136                 </div>
1137                 <div class="bill-meta-item">
1138                     <div class="bill-meta-label">Generated On</div>
1139                     <div class="bill-meta-value">${b.generatedOn || '-'}</div>
1140                 </div>
1141                 <div class="bill-meta-item">
1142                     <div class="bill-meta-label">Vendor</div>
1143                     <div class="bill-meta-value">${escapeHtml(b.repairedBy || '-')}</div>
1144                 </div>
1145             </div>
1146             <div class="bill-card-divider"></div>
1147             <button
1148                 class="btn btn-primary"
1149                 style="width:100%"
1150                 id="pay-repair-btn-${b.billId}"
1151                 onclick="payRepairBill(${b.billId})">
1152                 Pay & Send Invoice
1153             </button>
1154         </div>
1155     `).join('');
1156     } catch (e) { console.error(e); }
1157 }
1158
1159 // ===== RENTAL BILLS =====
1160 async function loadRentalBills() {
1161     try {
1162         const bills = await apiFetch('/bills/rentalBill');
1163         const grid = document.getElementById('rental-bills-card-grid');
1164         if (!bills || !bills.length) {
1165             grid.innerHTML = '<div class="empty-state">No unpaid rental bills found</div>';
1166             return;
1167         }
1168         grid.innerHTML = bills.map(b => `
1169             <div class="bill-card">
1170                 <div class="bill-card-head">
1171                     <div>
1172                         <div class="bill-card-id">Rental Bill #${b.id}</div>
1173                         <div class="bill-card-device">${escapeHtml(b.vendorName || '-')}</div>
1174                         <div class="bill-card-brand">Rental charge</div>
1175                     </div>
1176                     <span class="chip ${b.paymentStatus || ''}.toLowerCase()">${b.paymentStatus ||
1177 '-'}</span>
1178                 </div>
1179                 <div class="bill-card-cost">${b.amount != null ? '' + b.amount : '-'}</div>
1180                 <div class="bill-card-divider"></div>
1181                 <div class="bill-card-meta">
1182                     <div class="bill-meta-item">
1183                         <div class="bill-meta-label">Vendor</div>
1184                         <div class="bill-meta-value">${escapeHtml(b.vendorName || '-')}</div>
1185                     </div>

```

```

1185         <div class="bill-meta-item">
1186             <div class="bill-meta-label">Bill Date</div>
1187             <div class="bill-meta-value">${b.billDate || '-'}</div>
1188         </div>
1189         <div class="bill-meta-item">
1190             <div class="bill-meta-label">Due Date</div>
1191             <div class="bill-meta-value" style="color:${b.paymentStatus === 'OVERDUE' ? 'var(-
-red)' : 'inherit'}">${b.dueDate || '-'}</div>
1192         </div>
1193         <div class="bill-meta-item">
1194             <div class="bill-meta-label">Status</div>
1195             <div class="bill-meta-value">${b.paymentStatus || '-'}</div>
1196         </div>
1197     </div>
1198     <div class="bill-card-divider"></div>
1199     <button
1200         class="btn btn-primary"
1201         style="width:100%"
1202         id="pay-rental-btn-${b.id}"
1203         onclick="payRentalBill(${b.id})">
1204         Pay & Send Invoice
1205     </button>
1206 </div>
1207 `).join('');
1208 } catch (e) { console.error(e); }
1209 }
1210 async function payRepairBill(billId) {
1211     if (!confirm('Generate invoice and send to vendor?')) return;
1212     const btn = document.getElementById(`pay-repair-btn-${billId}`);
1213     btn.disabled = true;
1214     btn.textContent = 'Processing...';
1215     try {
1216         await apiFetch(`/bills/repairBill/${billId}/pay`, { method: 'POST' });
1217         showAlert('Invoice sent to vendor!', 'success');
1218         loadRepairBills(); // refresh – paid bill disappears if your API filters it out
1219     } catch (e) {
1220         showAlert('Failed: ' + e.message, 'error');
1221         btn.disabled = false;
1222         btn.textContent = 'Pay & Send Invoice';
1223     }
1224 }
1225
1226 async function payRentalBill(billId) {
1227     if (!confirm('Generate invoice and send to vendor?')) return;
1228     const btn = document.getElementById(`pay-rental-btn-${billId}`);
1229     btn.disabled = true;
1230     btn.textContent = 'Processing...';
1231     try {
1232         await apiFetch(`/bills/rentalBill/${billId}/pay`, { method: 'POST' });
1233         showAlert('Invoice sent to vendor!', 'success');
1234         loadRentalBills();
1235     } catch (e) {
1236         showAlert('Failed: ' + e.message, 'error');
1237         btn.disabled = false;
1238         btn.textContent = 'Pay & Send Invoice';
1239     }
1240 }
1241
1242 // ===== LOGOUT =====
1243 function logout() {
1244     localStorage.removeItem('jwt_token');
1245     localStorage.removeItem('refresh_token');
1246     localStorage.removeItem('auth_type');
1247     localStorage.removeItem('last_role');
1248     fetch(`${API}/session/logout`, { method: 'POST', credentials: 'include' })
1249         .finally(() => {
1250             document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
1251             window.location.href = '/google.html';
1252         });
1253 }
1254 </script>
1255 </body>
1256 </html>
1257

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Employee Portal</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&
display=swap" rel="stylesheet">
8      <!-- FullCalendar CDN -->
9      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
10     <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
11     <style>
12         *{margin:0;padding:0;box-sizing:border-box}
13         :root{
14             --bg:#080c10;--bg2:#0d1117;--bg3:#111820;
15             --panel:#141c26;--panel2:#1a2332;
16             --border:rgba(99,179,237,0.08);--border2:rgba(99,179,237,0.15);
17             --text:#d4e6f1;--text2:#7a9bb5;--text3:#4a6d87;
18             --accent:#38bdf8;--accent-dim:rgba(56,189,248,0.1);
19             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
20             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
21             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
22             --font:'Sora', sans-serif;--mono:'JetBrains Mono', monospace;
23             --sidebar:240px;
24         }
25         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
26         .app{display:flex;height:100vh}
27
28         /* SIDEBAR */
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;position:relative;flex-shrink:0}
30         .sidebar::before{content:'';position:absolute;top:0;left:0;right:0;height:1px;background:linear-
gradient(90deg,transparent,var(--accent),transparent)}
31         .logo{padding:24px 20px;border-bottom:1px solid var(--border)}
32         .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
33         .logo-icon{width:32px;height:32px;background:linear-gradient(135deg,var(--accent),#0ea5e9);border-
radius:8px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size...
34         .logo h2{font-size:16px;font-weight:600;color:var(--text);letter-spacing:0.5px}
35         .logo p{font-size:11px;color:var(--text3);font-family:var(--mono);margin-top:2px}
36         .role-badge{display:inline-flex;align-items:center;gap:5px;padding:4px 10px;background:var(--accent-dim);border:1px solid rgba(56,189,248,0.25);border-radius:20px;font-size:10px;font-weight...
37         .role-badge::before{content:'';width:6px;height:6px;background:var(--accent);border-
radius:50%;animation:pulse 2s infinite}
38         @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
39         .nav{flex:1;padding:12px 0;overflow-y:auto}
40         .nav-section{padding:4px 20px 8px;font-size:10px;font-weight:600;color:var(--text3);letter-
spacing:1.5px;text-transform:uppercase;margin-top:8px}
41         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
42         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
43         .nav-item.active{background:var(--accent-dim);color:var(--accent);border:1px solid
rgba(56,189,248,0.2)}
44         .nav-item .icon{width:20px;text-align:center;font-size:14px;flex-shrink:0}
45         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
46         .user-card{display:flex;align-items:center;gap:10px}
47         .avatar{width:32px;height:32px;background:linear-gradient(135deg,#0ea5e9,var(--accent));border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:13px;font-weight:6...
48         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
49         .user-info-side .email{font-size:10px;color:var(--text3);font-family:var(--mono);white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;max-width:140px}
50
51         /* MAIN */
52         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
53         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
54         .page-title{font-size:16px;font-weight:600;color:var(--text)}
55         .topbar-right{display:flex;align-items:center;gap:12px}
56         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
57         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}

```

```

58     .notif-wrapper { position: relative; cursor: pointer; }
59
60     .notif-badge {
61         position: absolute; top: -5px; right: -6px;
62         background: #e53e3e; color: white;
63         font-size: 10px; font-weight: 600;
64         min-width: 16px; height: 16px;
65         border-radius: 50%; display: flex;
66         align-items: center; justify-content: center;
67         padding: 0 3px;
68     }
69
70     .notif-panel {
71         position: absolute; top: 48px; right: 16px;
72         width: 320px; max-height: 400px; overflow-y: auto;
73         background: var(--panel); border: 1px solid #e2e8f0;
74         border-radius: 10px; box-shadow: 0 4px 16px rgba(0,0,0,0.1);
75         z-index: 999;
76     }
77
78     .notif-header {
79         padding: 12px 16px; font-weight: 600;
80         border-bottom: 1px solid #e2e8f0;
81     }
82
83     .notif-item {
84         padding: 12px 16px; font-size: 14px;
85         border-bottom: 1px solid #f1f5f9;
86     }
87
88     .notif-item.unread { background: #eff6ff; }
89     .notif-time { font-size: 11px; color: #94a3b8; margin-top: 4px; }
90
91     /* CONTENT */
92     .content{flex:1;overflow-y:auto;padding:24px}
93     .view{display:none}
94     .view.active{display:block;animation:fadeIn 0.3s ease}
95     @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
96
97     /* CARDS */
98     .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
99     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:12px;padding:20px;position:relative;overflow:hidden;transition:border-color 0.2s}
100     .stat-card:hover{border-color:var(--border2)}
101     .stat-card::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
102     .stat-card.blue::after{background:var(--accent)}
103     .stat-card.green::after{background:var(--green)}
104     .stat-card.amber::after{background:var(--amber)}
105     .stat-card.red::after{background:var(--red)}
106     .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;margin-
bottom:10px;font-weight:500}
107     .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
108     .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
109
110     /* SECTION */
111     .section{background:var(--panel);border:1px solid var(--border);border-radius:12px;margin-
bottom:20px;overflow:hidden}
112     .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
113     .section-title{font-size:14px;font-weight:600;color:var(--text)}
114     .section-sub{font-size:12px;color:var(--text3)}
115
116     /* TABLE */
117     .tablewrap{overflow-x:auto}
118     table{width:100%;border-collapse:collapse}
119     th{padding:10px 16px;text-align:left;font-size:11px;color:var(--text3);font-weight:600;letter-
spacing:0.8px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--bord...
120     td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
121     tr:last-child td{border-bottom:none}
122     tr:hover td{background:rgba(56,189,248,0.03);color:var(--text)}
123
124     /* CHIPS */
125     .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600;letter-spacing:0.5px}
126     .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}

```

```

127     .chip.open{background:var(--accent-dim);color:var(--accent)}
128     .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
129     .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
130     .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
131     .chip.sick{background:var(--red-dim);color:var(--red)}
132     .chip.casual{background:var(--amber-dim);color:var(--amber)}
133     .chip.maternity{background:var(--accent-dim);color:var(--accent)}
134     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
135     .chip.unpaid { background: rgba(139,92,246,0.15); color: #a78bfa; border: 1px solid
rgba(139,92,246,0.2); }
136     .chip.assigned{background:var(--green-dim);color:var(--green)}
137     .chip.condemned{background:var(--red-dim);color:var(--red)}
138
139     /* FORM */
140     .form-card{padding:24px}
141     .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
142     .form-group{display:flex;flex-direction:column;gap:6px}
143     .form-group.full{grid-column:1/-1}
144     label{font-size:11px;font-weight:600;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
145     input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
146     input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent);background:var(--
bg2)}
147     input::placeholder,textarea::placeholder{color:var(--text3)}
148     select option{background:var(--bg2)}
149
150     /* BUTTONS */
151     .btn{padding:9px 18px;border-radius:8px;border:none;font-family:var(--font);font-size:13px;font-
weight:600;cursor:pointer;transition:all 0.2s}
152     .btn-primary{background:var(--accent);color:#fff}
153     .btn-primary:hover{background:#0ea5e9;transform:translateY(-1px);box-shadow:0 4px 12px
rgba(56,189,248,0.3)}
154     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
155     .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
156     .btn-sm{padding:6px 12px;font-size:12px}
157     .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
158     .btn-danger:hover{background:var(--red);color:#fff}
159     .form-actions{display:flex;gap:10px;margin-top:20px}
160
161     /* ALERT */
162     .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-
index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
163     .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.3)}
164     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
165     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
166
167     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
168     .profile-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;padding:24px}
169     .profile-field{display:flex;flex-direction:column;gap:4px}
170     .profile-field .field-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px;font-weight:600}
171     .profile-field .field-value{font-size:14px;color:var(--text);padding:10px 14px;background:var(--
bg3);border-radius:8px;border:1px solid var(--border);font-family:var(--mono)}
172
173     .subscription-card{display:grid;grid-template-columns:repeat(auto-
fill,minmax(200px,1fr));gap:12px;padding:20px}
174     .sub-item{background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px;transition:border-color 0.2s}
175     .sub-item:hover{border-color:var(--border2)}
176     .sub-item .meal-name{font-weight:600;color:var(--text);margin-bottom:6px}
177     .sub-item .meal-meta{font-size:11px;color:var(--text3)}
178
179     .chip.paused{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
180     .chip.expired{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
181
182     /* ===== */
183     /* CALENDAR VIEW STYLES */
184     /* ===== */
185
186     /* Leave balance summary cards above the calendar */
187     .leave-balance-grid {
188         display: grid;
189         grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));

```

```

190         gap: 14px;
191         padding: 20px;
192         border-bottom: 1px solid var(--border);
193     }
194     .leave-bal-card {
195         background: var(--bg3);
196         border: 1px solid var(--border);
197         border-radius: 10px;
198         padding: 16px 18px;
199         display: flex;
200         flex-direction: column;
201         gap: 6px;
202         position: relative;
203         overflow: hidden;
204         transition: border-color 0.2s;
205     }
206     .leave-bal-card:hover { border-color: var(--border2); }
207     .leave-bal-card::before {
208         content: '';
209         position: absolute;
210         left: 0; top: 0; bottom: 0;
211         width: 3px;
212         border-radius: 3px 0 0 3px;
213     }
214     .leave-bal-card.sick::before { background: var(--red); }
215     .leave-bal-card.casual::before { background: var(--amber); }
216     .leave-bal-card.mat::before { background: var(--accent); }
217     .leave-bal-card.total::before { background: var(--green); }
218     .lbc-icon { font-size: 18px; }
219     .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }
220     .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-
height: 1; }
221     .lbc-sub { font-size: 11px; color: var(--text3); }
222
223     /* Legend row */
224     .cal-legend {
225         display: flex;
226         align-items: center;
227         gap: 20px;
228         padding: 12px 20px;
229         border-bottom: 1px solid var(--border);
230         flex-wrap: wrap;
231     }
232     .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase;
letter-spacing: 0.8px; margin-right: 4px; }
233     .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2);
}
234     .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
235
236     /* FullCalendar dark-theme overrides */
237     #employee-calendar {
238         padding: 16px 20px 20px;
239     }
240     /* Toolbar */
241     #employee-calendar .fc .fc-toolbar-title {
242         font-family: var(--font);
243         font-size: 15px;
244         font-weight: 600;
245         color: var(--text);
246     }
247     #employee-calendar .fc .fc-button {
248         background: var(--panel2);
249         border: 1px solid var(--border2);
250         color: var(--text2);
251         font-family: var(--font);
252         font-size: 12px;
253         font-weight: 600;
254         border-radius: 6px;
255         padding: 5px 12px;
256         transition: all 0.2s;
257         box-shadow: none;
258         text-transform: capitalize;
259     }
260     #employee-calendar .fc .fc-button:hover,
261     #employee-calendar .fc .fc-button-active,

```

```

262     #employee-calendar .fc .fc-button:not(:disabled):active {
263         background: var(--accent-dim);
264         border-color: var(--accent);
265         color: var(--accent);
266         box-shadow: none;
267     }
268     #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
269
270     /* Grid */
271     #employee-calendar .fc-theme-standard td,
272     #employee-calendar .fc-theme-standard th,
273     #employee-calendar .fc-theme-standard .fc-scrollgrid {
274         border-color: var(--border);
275     }
276     #employee-calendar .fc .fc-col-header-cell-cushion {
277         font-family: var(--font);
278         font-size: 11px;
279         font-weight: 600;
280         color: var(--text3);
281         text-transform: uppercase;
282         letter-spacing: 0.8px;
283         padding: 8px 4px;
284     }
285     #employee-calendar .fc .fc-daygrid-day-number {
286         font-family: var(--mono);
287         font-size: 12px;
288         color: var(--text3);
289         padding: 6px 8px;
290     }
291     #employee-calendar .fc .fc-daygrid-day.fc-day-today {
292         background: var(--accent-dim) !important;
293     }
294     #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number {
295         color: var(--accent);
296         font-weight: 700;
297     }
298     #employee-calendar .fc .fc-daygrid-day:hover {
299         background: rgba(56,189,248,0.03);
300     }
301     #employee-calendar .fc-daygrid-day-frame {
302         background: var(--bg3);
303     }
304     #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame {
305         background: var(--bg2);
306     }
307
308     /* Events */
309     #employee-calendar .fc-event {
310         border: none !important;
311         border-radius: 4px;
312         font-family: var(--font);
313         font-size: 11px;
314         font-weight: 600;
315         padding: 2px 6px;
316         cursor: pointer;
317     }
318     #employee-calendar .fc-event-title { font-weight: 600; }
319
320     /* Event tooltip */
321     .cal-tooltip {
322         position: fixed;
323         background: var(--panel2);
324         border: 1px solid var(--border2);
325         border-radius: 8px;
326         padding: 10px 14px;
327         font-size: 12px;
328         color: var(--text);
329         z-index: 9999;
330         pointer-events: none;
331         max-width: 200px;
332         box-shadow: 0 8px 24px rgba(0,0,0,0.4);
333     }
334     .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
335     .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
336
337     /* balance loading state */

```

```
338         .lbc-count.loading { color: var(--text3); font-size: 16px; }
339     </style>
340 </head>
341 <body>
342     <div class="app">
343         <!-- SIDEBAR -->
344         <div class="sidebar">
345             <div class="logo">
346                 <div class="logo-mark">
347                     <div class="logo-icon">E</div>
348                     <h2>EMS Portal</h2>
349                 </div>
350                 <p>Employee Management</p>
351                 <div class="role-badge">Employee</div>
352             </div>
353             <nav class="nav">
354                 <div class="nav-section">Overview</div>
355                 <div class="nav-item active" onclick="showView('dashboard',this)"><span class="icon"></span>
Dashboard</div>
356                 <div class="nav-section">Leave</div>
357                 <div class="nav-item" onclick="showView('apply-leave',this)"><span class="icon"></span> Apply
Leave</div>
358                 <div class="nav-item" onclick="showView('my-leaves',this)"><span class="icon"></span> My Leave
History</div>
359                 <div class="nav-item" onclick="showView('my-calendar',this)"><span class="icon"></span> My
Calendar</div>
360                 <div class="nav-section">Meals</div>
361                 <div class="nav-item" onclick="showView('restaurants',this)"><span class="icon"></span>
Restaurants</div>
362                 <div class="nav-item" onclick="showView('subscriptions',this)"><span class="icon"></span> My
Subscriptions</div>
363                 <div class="nav-item" onclick="showView('add-subscription',this)"><span class="icon"></span>
Add Subscription</div>
364                 <div class="nav-section">Account</div>
365                 <div class="nav-item" onclick="showView('profile',this)"><span class="icon"></span> My
Profile</div>
366                 <div class="nav-item" onclick="showView('my-devices',this)"><span class="icon"></span> My
Devices</div>
367                 <div class="nav-item" onclick="showView('service-requests',this)"><span class="icon"></span>
Service Requests</div>
368                 <div class="nav-item" onclick="showView('two-factor',this)"><span class="icon"></span> Two-
Factor Auth</div>
369             </nav>
370             <div class="sidebar-footer">
371                 <div class="user-card">
372                     <div class="avatar" id="avatar-initials">?</div>
373                     <div class="user-info-side">
374                         <div class="name" id="sidebar-name">Loading...</div>
375                         <div class="email" id="sidebar-email">--</div>
376                     </div>
377                 </div>
378             </div>
379         </div>
380
381         <!-- MAIN -->
382         <div class="main">
383             <div class="topbar">
384                 <div class="page-title" id="pageTitle">Dashboard</div>
385                 <!-- Bell icon with badge -->
386                 <div class="notif-wrapper" onclick="toggleNotifPanel()">
387                     <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
388                         stroke="currentColor" stroke-width="2" stroke-linecap="round">
389                         <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
390                         <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
391                     </svg>
392                     <span id="notif-badge" class="notif-badge" style="display:none"></span>
393                 </div>
394
395                 <!-- Dropdown panel -->
396                 <div id="notif-panel" class="notif-panel" style="display:none">
397                     <div class="notif-header">Notifications</div>
398                     <div id="notif-list"></div>
399                 </div>
400                 <div class="topbar-right">
401                     <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
```



```

402         <button class="logout-btn" onclick="logout()">Sign Out</button>
403     </div>
404 </div>
405 <div class="content">
406
407     <!-- DASHBOARD -->
408     <div id="dashboard-view" class="view active">
409         <div class="stats-grid">
410             <div class="stat-card blue">
411                 <div class="stat-label">Leave Balance</div>
412                 <div class="stat-value" id="stat-balance">--</div>
413                 <div class="stat-sub">days remaining</div>
414             </div>
415             <div class="stat-card amber">
416                 <div class="stat-label">Pending Leaves</div>
417                 <div class="stat-value" id="stat-pending">--</div>
418                 <div class="stat-sub">awaiting approval</div>
419             </div>
420             <div class="stat-card green">
421                 <div class="stat-label">Approved Leaves</div>
422                 <div class="stat-value" id="stat-approved">--</div>
423                 <div class="stat-sub">this year</div>
424             </div>
425             <div class="stat-card red">
426                 <div class="stat-label">Rejected Leaves</div>
427                 <div class="stat-value" id="stat-rejected">--</div>
428                 <div class="stat-sub">this year</div>
429             </div>
430         </div>
431         <div class="section">
432             <div class="section-head">
433                 <div>
434                     <div class="section-title">Recent Leave Requests</div>
435                     <div class="section-sub">Your last 5 requests</div>
436                 </div>
437                 <button class="btn btn-secondary btn-sm" onclick="loadDashboard()">
Refresh</button>
438             </div>
439             <div class="tablewrap">
440                 <table>
441
442 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
442                 <tbody id="recent-leaves"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
443                 </table>
444             </div>
445         </div>
446         <div class="section">
447             <div class="section-head">
448                 <div class="section-title">Active Subscriptions</div>
449             </div>
450             <div id="dashboard-subs" class="subscription-card"></div>
451         </div>
452     </div>
453
454     <!-- APPLY LEAVE -->
455     <div id="apply-leave-view" class="view">
456         <div class="section">
457             <div class="section-head"><div class="section-title">Apply for Leave</div></div>
458             <div class="form-card">
459                 <div class="form-grid">
460                     <div class="form-group">
461                         <label>Leave Type</label>
462                         <select id="leave-type">
463                             <option value="SICK">Sick Leave</option>
464                             <option value="CASUAL">Casual Leave</option>
465                             <option value="UNPAID">Unpaid Leave</option>
466                         </select>
467                         <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(251,191,36,0.2);bord...
468                             Sick leave is limited to <strong>1 day per month</strong>. If already
used this month, please select Unpaid Leave.
469                         </div>
470                         <div id="casual-leave-hint" style="display:none; margin-top:6px; font-
size:11px; color:var(--accent); background:var(--accent-dim); border:1px solid rgba(56,189,248,...
471                             You have <strong id="casual-balance-text">...</strong> casual leave

```

```

days available.
472                 </div>
473             </div>
474             <div class="form-group">
475                 <label>Start Date</label>
476                 <input type="date" id="leave-start">
477             </div>
478             <div class="form-group">
479                 <label>End Date</label>
480                 <input type="date" id="leave-end">
481             </div>
482             <div class="form-group full">
483                 <label>Reason</label>
484                 <textarea id="leave-reason" rows="3" placeholder="Please describe the
reason for your leave..."></textarea>
485             </div>
486         </div>
487         <div class="form-actions">
488             <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
489             <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
490         </div>
491     </div>
492 </div>
493 </div>
494
495 <!-- MY LEAVES -->
496 <div id="my-leaves-view" class="view">
497     <div class="section">
498         <div class="section-head">
499             <div><div class="section-title">My Leave History</div><div class="section-sub">All
your leave requests</div></div>
500             <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
501         </div>
502         <div class="tablewrap">
503             <table>
504
505 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Status</th><th>Action</th></tr><
/thead>
506             <tbody id="my-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
507         </div>
508     </div>
509 </div>
510
511 <!-- ===== -->
512 <!-- MY CALENDAR VIEW (NEW) -->
513 <!-- ===== -->
514 <div id="my-calendar-view" class="view">
515     <div class="section">
516         <div class="section-head">
517             <div>
518                 <div class="section-title">My Leave Calendar</div>
519                 <div class="section-sub">All approved leaves and public holidays</div>
520             </div>
521             <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
Refresh</button>
522         </div>
523     </div>
524     <div class="leave-balance-grid" style="grid-template-columns: 1fr 1fr;">
525
526         <!-- Sick Leave Card -->
527         <div class="leave-bal-card sick">
528             <span class="lbc-icon"></span>
529             <span class="lbc-label">Sick Leave</span>
530             <span class="lbc-count" id="cal-sick-remaining">--</span>
531             <span class="lbc-sub">day available this month</span>
532             <div id="cal-sick-month-badge" style="
533                 display:inline-flex; align-items:center; gap:5px;
534                 margin-top:8px; padding:4px 10px;
535                 border-radius:20px; font-size:10px; font-weight:700;
536                 letter-spacing:0.8px; text-transform:uppercase;
537             ">...</div>
538             <span class="lbc-sub" id="cal-sick-accrued-sub" style="margin-top:4px"></span>
539         </div>
540

```

```

541         <!-- Casual Leave Card -->
542         <div class="leave-bal-card casual">
543             <span class="lbc-icon"></span>
544             <span class="lbc-label">Casual Leave</span>
545             <span class="lbc-count" id="cal-casual-remaining">--</span>
546             <span class="lbc-sub" id="cal-casual-sub">loading...</span>
547             <div style="margin-top:10px;display:flex;flex-direction:column;gap:3px">
548                 <span class="lbc-sub" id="cal-casual-opening"></span>
549                 <span class="lbc-sub" id="cal-casual-accrued"></span>
550                 <span class="lbc-sub" id="cal-casual-used"></span>
551             </div>
552         </div>
553
554     </div>
555
556     <!-- Legend -->
557     <div class="cal-legend">
558         <span class="legend-label">Legend:</span>
559         <div class="legend-item">
560             <div class="legend-dot" style="background:#34d399"></div>
561             Approved Leave
562         </div>
563         <div class="legend-item">
564             <div class="legend-dot" style="background:#fbbf24"></div>
565             Pending Leave
566         </div>
567         <div class="legend-item">
568             <div class="legend-dot" style="background:#f87171"></div>
569             Rejected Leave
570         </div>
571         <div class="legend-item">
572             <div class="legend-dot" style="background:#38bdf8"></div>
573             Public Holiday
574         </div>
575     </div>
576
577     <!-- FullCalendar Mount Point -->
578     <div id="employee-calendar"></div>
579 </div>
580 </div>
581 <!-- END CALENDAR VIEW -->
582
583 <!-- RESTAURANTS -->
584 <div id="restaurants-view" class="view">
585     <div class="section">
586         <div class="section-head">
587             <div class="section-title">Available Restaurants</div>
588             <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">
Refresh</button>
589         </div>
590         <div class="tablewrap">
591             <table>
592                 <thead><tr><th>Name</th><th>Meal
Slot</th><th>Status</th><th>Action</th></tr></thead>
593                 <tbody id="restaurants-list"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
594             </table>
595         </div>
596     </div>
597 </div>
598
599 <!-- SUBSCRIPTIONS -->
600 <div id="subscriptions-view" class="view">
601     <div class="section">
602         <div class="section-head">
603             <div class="section-title">My Meal Subscriptions</div>
604             <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">
Refresh</button>
605         </div>
606         <div class="tablewrap">
607             <table>
608                 <thead>
609                     <tr>
610                         <th>Restaurant</th><th>Meal Slot</th><th>Schedule</th>
611                         <th>Next Delivery</th><th>Status</th><th>Action</th>
612                     </tr>

```

```
613                                     </thead>
614                                     <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-
state">Loading...</td></tr></tbody>
615                                     </table>
616                                 </div>
617 </div>
618 </div>
619
620 <!-- ADD SUBSCRIPTION -->
621 <div id="add-subscription-view" class="view">
622     <div class="section">
623         <div class="section-head"><div class="section-title">Subscribe to Meal Plan</div></div>
624         <div class="form-card">
625             <div class="form-grid">
626                 <div class="form-group">
627                     <label>Meal Slot</label>
628                     <select id="sub-meal-slot">
629                         <option value="BREAKFAST">Breakfast</option>
630                         <option value="LUNCH">Lunch</option>
631                         <option value="DINNER">Dinner</option>
632                     </select>
633                 </div>
634                 <div class="form-group">
635                     <label>Restaurant</label>
636                     <select id="sub-restaurant-id">
637                         <option value="">Select a restaurant</option>
638                     </select>
639                 </div>
640                 <div class="form-group">
641                     <label>Schedule Type</label>
642                     <select id="sub-schedule-type">
643                         <option value="DAILY">Daily</option>
644                         <option value="WEEKLY">Weekly</option>
645                     </select>
646                 </div>
647                 <div class="form-group" id="day-of-week-group" style="display:none">
648                     <label>Day of Week</label>
649                     <select id="sub-day-of-week">
650                         <option value="MONDAY">Monday</option>
651                         <option value="TUESDAY">Tuesday</option>
652                         <option value="WEDNESDAY">Wednesday</option>
653                         <option value="THURSDAY">Thursday</option>
654                         <option value="FRIDAY">Friday</option>
655                         <option value="SATURDAY">Saturday</option>
656                         <option value="SUNDAY">Sunday</option>
657                     </select>
658                 </div>
659             </div>
660             <div class="form-actions">
661                 <button class="btn btn-primary" onclick="addSubscription()">Subscribe</button>
662             </div>
663         </div>
664     </div>
665 </div>
666
667 <!-- MY DEVICES -->
668 <div id="my-devices-view" class="view">
669     <div class="section">
670         <div class="section-head">
671             <div>
672                 <div class="section-title">My Assigned Devices</div>
673                 <div class="section-sub">Devices currently assigned to you</div>
674             </div>
675             <button class="btn btn-secondary btn-sm" onclick="loadMyAssignedDevices()">
Refresh</button>
676         </div>
677         <div class="tablewrap">
678             <table>
679                 <thead>
680                     <tr>
681                         <th>ID</th><th>Device Name</th><th>Brand</th>
682                         <th>Type</th><th>Warranty</th><th>Assigned On</th><th>Status</th>
683                     </tr>
684                 </thead>
685                 <tbody id="my-devices-list"></tbody>
686             </table>
```

```

687         </div>
688     </div>
689 </div>
690
691 <!-- SERVICE REQUESTS -->
692 <div id="service-requests-view" class="view">
693     <div class="section" style="margin-bottom:20px">
694         <div class="section-head">
695             <div>
696                 <div class="section-title">My Service Requests</div>
697                 <div class="section-sub">All requests you have raised</div>
698             </div>
699             <button class="btn btn-secondary btn-sm" onclick="loadServiceRequestsList()">
Refresh</button>
700         </div>
701         <div class="tablewrap">
702             <table>
703                 <thead>
704                     <tr>
705                         <th>ID</th><th>Device</th><th>Type</th>
706                         <th>Description</th><th>Urgent</th><th>Status</th><th>Raised At</th>
707                     </tr>
708                 </thead>
709                 <tbody id="service-requests-list">
710                     <tr><td colspan="7" class="empty-state">Loading...</td></tr>
711                 </tbody>
712             </table>
713         </div>
714     </div>
715     <div class="section">
716         <div class="section-head"><div class="section-title">Raise a New Request</div></div>
717         <div class="form-card">
718             <div class="form-grid">
719                 <div class="form-group full">
720                     <label>Device</label>
721                     <select id="sr-device-select">
722                         <option value="">-- Select your device --</option>
723                     </select>
724                 </div>
725                 <div class="form-group">
726                     <label>Request Type</label>
727                     <select id="sr-type">
728                         <option value="REPAIR">Repair</option>
729                         <option value="REPLACEMENT">Replacement</option>
730                         <option value="RETURN">Return</option>
731                     </select>
732                 </div>
733                 <div class="form-group" style="display:flex;align-
items:center;gap:10px;padding-top:22px">
734                     <input type="checkbox" id="sr-urgent"
style="width:16px;height:16px;padding:0;flex-shrink:0">
735                     <label for="sr-urgent" style="text-transform:none;font-size:13px;font-
weight:500;color:var(--text);cursor:pointer;letter-spacing:0">Mark as Urgent</label>
736                 </div>
737                 <div class="form-group full">
738                     <label>Issue Description</label>
739                     <textarea id="sr-description" rows="3" placeholder="Describe the issue in
detail..."></textarea>
740                 </div>
741             </div>
742             <div class="form-actions">
743                 <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
Request</button>
744                 <button class="btn btn-secondary"
onclick="clearServiceRequestForm()">Clear</button>
745             </div>
746         </div>
747     </div>
748 </div>
749 <!-- TWO-FACTOR AUTH -->
750 <div id="two-factor-view" class="view">
751     <div class="section">
752         <div class="section-head">
753             <div>
754                 <div class="section-title"> Two-Factor Authentication</div>
755                 <div class="section-sub">Protect your account with an authenticator app</div>

```

```

756         </div>
757         <span id="totp-status-chip" class="chip">Checking...</span>
758     </div>
759
760     <div style="padding:24px;max-width:480px">
761
762         <!-- DISABLED state – setup flow -->
763         <div id="totp-setup-section">
764             <p style="font-size:13px;color:var(--text2);margin-bottom:16px;line-
height:1.6">
765                 Use Google Authenticator, Authy, or any TOTP app.
766                 Scan the QR code below, then enter the 6-digit code to activate.
767             </p>
768             <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-
btn">
769                 Generate QR Code
770             </button>
771
772             <div id="qr-area" style="display:none;margin-top:24px;text-align:center">
773                 <img id="qr-img" src="" alt="QR Code"
774                     style="width:180px;height:180px;border-radius:10px;
775                     border:1px solid var(--border2);display:block;margin:0 auto">
776                 <div style="margin-top:12px;font-size:11px;color:var(--text3)">
777                     Can't scan? Enter this key manually:
778                 </div>
779                 <div id="totp-secret-display"
780                     style="font-family:var(--mono);font-size:13px;color:var(--accent);
781                     background:var(--bg3);border:1px solid var(--border2);
782                     border-radius:6px;padding:8px 12px;margin-top:6px;
783                     word-break:break-all"></div>
784
785                 <div style="margin-top:20px;text-align:left">
786                     <label style="font-size:11px;color:var(--text3);font-weight:600;
787                     text-transform:uppercase;letter-spacing:0.8px;
788                     display:block;margin-bottom:8px">
789                         Enter 6-digit code to activate
790                     </label>
791                     <input id="totp-verify-input" type="text" inputmode="numeric"
792                         maxlength="6" placeholder="000000"
793                         style="text-align:center;font-size:20px;
794                         font-family:var(--mono);letter-spacing:8px;padding:12px"
795                         oninput="this.value=this.value.replace(/\D/g, '')">
796                     <button class="btn btn-primary"
797                         onclick="verifyAndActivateTotp()"
798                         style="width:100%;margin-top:10px">
799                         Activate 2FA
800                     </button>
801                     <div id="totp-verify-error"
802                         style="display:none;margin-top:8px;font-size:12px;
803                         color:var(--red)"></div>
804                 </div>
805             </div>
806         </div>
807
808         <!-- ENABLED state -->
809         <div id="totp-disable-section" style="display:none">
810             <div style="background:var(--green-dim);border:1px solid rgba(52,211,153,0.2);
811                 border-radius:8px;padding:12px 16px;font-size:13px;
812                 color:var(--green);margin-bottom:16px">
813                 Two-factor authentication is active on your account.
814             </div>
815             <button class="btn btn-danger" onclick="disableTotp()">
816                 Disable 2FA
817             </button>
818         </div>
819     </div>
820 </div>
821 </div>
822 <!-- PROFILE -->
823 <div id="profile-view" class="view">
824     <div class="section">
825         <div class="section-head"><div class="section-title">My Profile</div></div>
826         <div class="profile-grid" id="profile-data">
827             <div class="profile-field"><div class="field-label">Name</div><div class="field-
value" id="p-name"></div></div>
828             <div class="profile-field"><div class="field-label">Email</div><div class="field-

```

```

value" id="p-email"></div></div>
829         <div class="profile-field"><div class="field-label">Department</div><div
class="field-value" id="p-dept"></div></div>
830         <div class="profile-field"><div class="field-label">Role</div><div class="field-
value" id="p-role"></div></div>
831         <div class="profile-field"><div class="field-label">Status</div><div class="field-
value" id="p-status"></div></div>
832         <div class="profile-field"><div class="field-label">Timezone</div><div
class="field-value" id="p-tz"></div></div>
833     </div>
834 </div>
835
836     <div class="section" id="maternity-apply-btn" style="display:none">
837         <div class="section-head">
838             <div>
839                 <div class="section-title"> Maternity Leave</div>
840                 <div class="section-sub">Apply for 180 days maternity leave</div>
841             </div>
842         </div>
843         <div class="form-card">
844             <div class="form-grid">
845                 <div class="form-group">
846                     <label>Start Date</label>
847                     <input type="date" id="mat-start-date">
848                 </div>
849                 <div class="form-group">
850                     <label>End Date (Auto)</label>
851                     <input type="text" id="mat-end-date"
852                         placeholder="Auto: start + 180 days"
853                         disabled
854                         style="color:var(--text3);cursor:not-allowed">
855                 </div>
856                 <div class="form-group full">
857                     <label>Reason / Notes</label>
858                     <textarea id="mat-reason" rows="3"
859                         placeholder="Optional notes..."></textarea>
860                 </div>
861                 <!-- Info box -->
862                 <div class="form-group full">
863                     <div style="background:var(--accent-dim);border:1px solid
rgba(56,189,248,0.2);
864                         border-radius:8px;padding:12px 16px;font-size:12px;color:var(--text2)">
865                         Maternity leave is fixed at <strong style="color:var(--accent)">180
days</strong>.
866                         The end date will be automatically calculated.
867                         You can apply <strong style="color:var(--accent)">once per
year</strong>.
868                     </div>
869                 </div>
870             </div>
871             <div class="form-actions">
872                 <button class="btn btn-primary" onclick="applyMaternityLeave()">
873                     Submit Maternity Leave
874                 </button>
875             </div>
876         </div>
877     </div>
878
879 </div>
880
881 </div><!-- /content -->
882 </div><!-- /main -->
883 </div><!-- /app -->
884
885 <!-- Tooltip for calendar events -->
886 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
887
888 <script>
889     const API = 'http://localhost:8080';
890     let availableRestaurants = [];
891
892     /* =====
893     CALENDAR GLOBALS
894     ===== */
895     let employeeCalendar = null; // FullCalendar instance
896     let calendarInitialized = false;

```

```

897     let currentEmployeeGender = null;
898     let currentEmployeeId = null;
899
900     // Event color map
901     const STATUS_COLORS = {
902         APPROVED : { bg:'rgba(52,211,153,0.25)', border:'#34d399', text:'#34d399' },
903         PENDING  : { bg:'rgba(251,191,36,0.25)',   border:'#fbbf24', text:'#fbbf24' },
904         REJECTED : { bg:'rgba(248,113,113,0.25)',   border:'#f87171', text:'#f87171' },
905         HOLIDAY  : { bg:'rgba(56,189,248,0.25)',   border:'#38bdf8', text:'#38bdf8' },
906     };
907
908     const TYPE_LABEL = { SICK:' Sick', CASUAL:' Casual', MATERNITY:' Maternity', UNPAID:' Unpaid'
909     };
910
911     /* =====
912     AUTH / INIT
913     ===== */
914     function extractRoleFromToken(token) {
915         try {
916             const payload = JSON.parse(atob(token.split('.')[1]));
917             return payload.role || payload.roles || payload.authorities || '';
918         } catch(e) { return ''; }
919     }
920
921     (function(){
922         const token = localStorage.getItem('jwt_token');
923         const authType = localStorage.getItem('auth_type');
924         if(!token && authType !== 'session') { window.location.href = '/google.html'; return; }
925         if(token) {
926             try {
927                 const roles = String(extractRoleFromToken(token));
928                 if(roles.includes('ROLE_MANAGER')) { window.location.href='/manager-dashboard.html';
929 return; }
930                 if(roles.includes('ROLE_VENDOR')) { window.location.href='/vendor-dashboard.html';
931 return; }
932                 if(roles.includes('ROLE_ADMIN') || roles.includes('ADMIN')) {
933 window.location.href='/dashboard.html'; return; }
934             } catch(e) { console.warn('Could not decode JWT for role check'); }
935         }
936         initUser();
937     })();
938
939     function getAuthHeader(){
940         const token = localStorage.getItem('jwt_token');
941         return token ? {'Authorization':`Bearer ${token}`} : {};
942     }
943
944     async function apiFetch(path, opts={}){
945         const token = localStorage.getItem('jwt_token');
946         const authType = localStorage.getItem('auth_type');
947         const res = await fetch(`${API}${path}`, {
948             ...opts,
949             headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},
950             credentials: (!token && authType==='session') ? 'include' : 'omit'
951         });
952         if(res.status===401){
953             localStorage.removeItem('jwt_token'); localStorage.removeItem('auth_type');
954             showAlert('Session expired. Please login again.','error');
955             setTimeout(()=>window.location.href='/google.html',1500);
956             throw new Error('Unauthorized');
957         }
958         if(res.status===403) throw new Error('ACCESS_DENIED');
959         if(!res.ok){
960             const errText = await res.text();
961             throw new Error(errText || 'Request failed');
962         }
963         const text = await res.text();
964         try { return text ? JSON.parse(text) : null; }
965         catch { return text; }
966     }
967
968     function asArray(value){ return Array.isArray(value) ? value : []; }
969
970     function daysBetween(start,end){
971         if(!start||!end) return '-';
972         const d=(new Date(end)-new Date(start))/86400000+1;

```



```

970         return d>0?d:'-';
971     }
972
973     async function initUser(){
974         try {
975             let email='', name='';
976             const token = localStorage.getItem('jwt_token');
977             if(token){
978                 try{ const p=JSON.parse(atob(token.split('.')[1])); email=p.sub||p.email||''; }catch(e){}
979             }
980             try{
981                 const me = await apiFetch('/employee/me');
982                 if(me){
983                     name=me.name||email; email=me.email||email;
984                     currentEmployeeGender = me.gender; // store gender
985                     currentEmployeeId = me.employeeId;
986                     document.getElementById('p-name').textContent=me.name||'-';
987                     document.getElementById('p-email').textContent=me.email||'-';
988                     document.getElementById('p-dept').textContent=me.dept||'-';
989                     document.getElementById('p-role').textContent=me.role||'EMPLOYEE';
990                     document.getElementById('p-status').textContent=me.status||'-';
991                     document.getElementById('p-tz').textContent=me.timezone||'-';
992
993                     const maternityBtn = document.getElementById('maternity-apply-btn');
994                     if(maternityBtn) {
995                         maternityBtn.style.display = me.gender === 'F' ? 'block' : 'none';
996                     }
997                 }
998             }catch(e){ console.warn('Could not load profile'); }
999             const initials=(name||email||'?').charAt(0).toUpperCase();
1000             document.getElementById('avatar-initials').textContent=initials;
1001             document.getElementById('sidebar-name').textContent=name||email||'Employee';
1002             document.getElementById('sidebar-email').textContent=email;
1003             document.getElementById('topbar-email').textContent=email;
1004         }catch(e){ console.error(e); }
1005         loadDashboard();
1006     }
1007
1008     /* =====
1009     VIEW SWITCHING
1010     ===== */
1011     function showView(name, el){
1012         document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
1013         document.getElementById(`${name}-view`).classList.add('active');
1014         document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
1015         if(el) el.classList.add('active');
1016         document.getElementById('pageTitle').textContent=
1017             name.split('-').map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
1018
1019         const loaders={
1020             dashboard:loadDashboard,
1021             'my-leaves':loadMyLeaves,
1022             'apply-leave':()=>{},
1023             'my-calendar':loadMyCalendar,
1024             restaurants:loadRestaurants,
1025             subscriptions:loadSubscriptions,
1026             'add-subscription':()=>{},
1027             profile:()=>{},
1028             'my-devices':loadMyAssignedDevices,
1029             'service-requests':loadServiceRequests,
1030             'two-factor':loadTotpStatus
1031         };
1032         if(loaders[name]) loaders[name]();
1033         if(name==='add-subscription') loadRestaurantChoices();
1034     }
1035
1036     /* =====
1037     DASHBOARD
1038     ===== */
1039     async function loadDashboard(){
1040         const [leavesResult, dashboardResult] = await Promise.allSettled([
1041             apiFetch('/leave_requests/employee'),
1042             apiFetch('/api/employees/me/leave-dashboard')
1043         ]);
1044         const leavesPayload = leavesResult.status==='fulfilled' ? leavesResult.value : null;
1045         const dashboard = dashboardResult.status==='fulfilled' ? dashboardResult.value : null;

```

```

1046     const arr = asArray(leavesPayload?.requests || leavesPayload);
1047     const pending = arr.filter(l=>l.status==='PENDING').length;
1048     const approved = arr.filter(l=>l.status==='APPROVED').length;
1049     const rejected = arr.filter(l=>l.status==='REJECTED').length;
1050     document.getElementById('stat-pending').textContent=pending;
1051     document.getElementById('stat-approved').textContent=approved;
1052     document.getElementById('stat-rejected').textContent=rejected;
1053     const balances = dashboard?.leaveBalances ? Object.values(dashboard.leaveBalances) : [];
1054     const available = balances.reduce((t,b)=>t+Number(b.available||0),0);
1055     document.getElementById('stat-balance').textContent=available;
1056     const recent=arr.slice(-5).reverse();
1057     document.getElementById('recent-leaves').innerHTML=recent.length
1058         ? recent.map(l=><tr>
1059             <td><span class="chip
1060 ${{l.leaveType||''}.toLowerCase()}>${l.leaveType||'N/A'}</span></td>
1061             <td>${l.startDate||'-'}</td><td>${l.endDate||'-'}</td>
1062             <td>${(l.reason||'-').substring(0,30)}</td>
1063             <td><span class="chip ${l.status||''}.toLowerCase()}>${l.status||'-'}</span></td>
1064             </tr>` ).join('')
1065         : leavesResult.status==='rejected'
1066         ? '<tr><td colspan="5" class="empty-state">Could not load leave data</td></tr>'
1067         : '<tr><td colspan="5" class="empty-state">No leave requests yet</td></tr>';
1068     try{
1069         const subs = await apiFetch('/subscription/user');
1070         const arr2 = subs || [];
1071         document.getElementById('dashboard-sub').innerHTML = arr2.length
1072             ? arr2.filter(s=>s.status==='ACTIVE').slice(0,4).map(s=>`
1073                 <div class="sub-item">
1074                     <div class="meal-name">${s.restaurantName||'Restaurant'}</div>
1075                     <div class="meal-meta">${s.mealSlot||''} · ${s.scheduleType||''}${s.dayOfWeek?' '
1076 +s.dayOfWeek:''}</div>
1077                     <div class="meal-meta" style="margin-top:4px;color:var(--accent)">Next:
1078 ${s.nextDeliveryTime?new Date(s.nextDeliveryTime).toLocaleString():'N/A'}</div>
1079                     </div>` ).join('')
1080             : '<div style="padding:16px;color:var(--text3);font-style:italic">No active
1081 subscriptions</div>';
1082     }catch(e){
1083         document.getElementById('dashboard-sub').innerHTML='<div style="padding:16px;color:var(--
1084 text3)">Could not load subscriptions</div>';
1085     }
1086 }
1087
1088 /* =====
1089 MY LEAVES
1090 ===== */
1091 async function loadMyLeaves(){
1092     try{
1093         const data=await apiFetch('/leave_requests/employee');
1094         const leaves=data?.requests||data||[];
1095         document.getElementById('my-leaves-list').innerHTML=leaves.length
1096             ? leaves.map(l=><tr>
1097                 <td><span class="chip
1098 ${{l.leaveType||''}.toLowerCase()}>${l.leaveType||'N/A'}</span></td>
1099                 <td>${l.startDate||'-'}</td><td>${l.endDate||'-'}</td>
1100                 <td>${l.numberOfDays||daysBetween(l.startDate,l.endDate)}</td>
1101                 <td>${(l.reason||'-').substring(0,30)}</td>
1102                 <td><span class="chip ${l.status||''}.toLowerCase()}>${l.status}</span></td>
1103                 <td>${l.status==='PENDING'?`<button class="btn btn-danger btn-sm"
1104 onclick="cancelLeave(${l.leaveRequestId})">Cancel</button>`:'-'}</td>
1105                 </tr>` ).join('')
1106             : '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
1107     }catch(e){
1108         document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
1109 state">Could not load leave requests</td></tr>';
1110     }
1111 }
1112
1113 async function applyLeave(){
1114     const leaveType = document.getElementById('leave-type').value;
1115     const startDate = document.getElementById('leave-start').value;
1116     const endDate = document.getElementById('leave-end').value;
1117     const reason = document.getElementById('leave-reason').value.trim();
1118     if(!startDate || !endDate){ showAlert('Please fill all required fields','error'); return; }
1119     try{

```

```

1114     const data = await apiFetch('/leave_requests', {
1115         method: 'POST',
1116         body: JSON.stringify({ leaveType, startDate, endDate, reason })
1117     });
1118     if(data && data.warning){
1119         showAlert(data.warning, 'warning');
1120     } else {
1121         showAlert('Leave request submitted successfully!', 'success');
1122     }
1123     clearLeaveForm();
1124     loadDashboard();
1125     loadMyLeaves();
1126 } catch(e) {
1127     // Try to parse the error message as JSON to extract the "error" field
1128     let msg = e.message;
1129     try {
1130         const parsed = JSON.parse(msg);
1131         if(parsed.error) msg = parsed.error;
1132     } catch(_) { /* not JSON, use raw message */ }
1133     showAlert(msg, 'error');
1134 }
1135 }
1136
1137 document.getElementById('leave-type').addEventListener('change', async function() {
1138     document.getElementById('sick-leave-hint').style.display =
1139         this.value === 'SICK' ? 'block' : 'none';
1140
1141     const casualHint = document.getElementById('casual-leave-hint');
1142     if(this.value === 'CASUAL') {
1143         casualHint.style.display = 'block';
1144         document.getElementById('casual-balance-text').textContent = 'loading...';
1145         try {
1146             const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1147             const balances = dashboard?.leaveBalances || {};
1148             const cas = balances['CASUAL'] || balances['casual'] || {};
1149             const available = Number(cas.available ?? cas.availableBalance ?? 0);
1150             document.getElementById('casual-balance-text').textContent =
1151                 available > 0 ? `${available}` : '0 (no balance remaining)';
1152         } catch(e) {
1153             document.getElementById('casual-balance-text').textContent = 'unavailable';
1154         }
1155     } else {
1156         casualHint.style.display = 'none';
1157     }
1158 });
1159
1160 async function cancelLeave(id){
1161     if(!confirm('Cancel this leave request?')) return;
1162     try{
1163         await apiFetch(`/leave_requests/cancel/${id}`, {method: 'PUT'});
1164         showAlert('Request cancelled successfully', 'success');
1165         loadMyLeaves();
1166     }catch(e){
1167         if(e.message==='ACCESS_DENIED') showAlert('You do not have permission to cancel this
leave', 'error');
1168         else showAlert('Failed to cancel: '+e.message, 'error');
1169     }
1170 }
1171
1172 function clearLeaveForm(){
1173     ['leave-start', 'leave-end', 'leave-reason'].forEach(id=>document.getElementById(id).value='');
1174 }
1175
1176 /* =====
1177 MY CALENDAR (NEW)
1178 ===== */
1179 async function loadMyCalendar(){
1180     // 1. Load balance cards
1181     await loadCalendarBalanceCards();
1182     // 2. Build or refresh FullCalendar
1183     if(!calendarInitialized){
1184         initFullCalendar();
1185     } else {
1186         await refreshCalendarEvents();
1187     }
1188 }

```

```

1189
1190     async function loadCalendarBalanceCards() {
1191     try {
1192         const [dashboard, sickMonthData] = await Promise.all([
1193             apiFetch('/api/employees/me/leave-dashboard'),
1194             apiFetch('/leave_requests/sick/used-this-month')
1195         ]);
1196
1197         /* SICK */
1198         const sickBadge = document.getElementById('cal-sick-month-badge');
1199         const sick      = dashboard?.sick;
1200         const sickAvail = Number(sick?.available ?? 0);
1201         const usedThisMonth = sickMonthData?.usedThisMonth === true;
1202
1203         if (usedThisMonth) {
1204             // Explicitly used this month
1205             document.getElementById('cal-sick-remaining').textContent = '0';
1206             sickBadge.textContent      = ' Used this month';
1207             sickBadge.style.background = 'var(--red-dim)';
1208             sickBadge.style.color      = 'var(--red)';
1209             sickBadge.style.border     = '1px solid rgba(248,113,113,0.25)';
1210         } else if (sickAvail === 0) {
1211             // New employee – accrual hasn't run yet
1212             document.getElementById('cal-sick-remaining').textContent = '0';
1213             sickBadge.textContent      = ' Accrual pending';
1214             sickBadge.style.background = 'var(--amber-dim)';
1215             sickBadge.style.color      = 'var(--amber)';
1216             sickBadge.style.border     = '1px solid rgba(251,191,36,0.25)';
1217         } else {
1218             // Has balance and not used
1219             document.getElementById('cal-sick-remaining').textContent = sickAvail;
1220             sickBadge.textContent      = ' Available this month';
1221             sickBadge.style.background = 'var(--green-dim)';
1222             sickBadge.style.color      = 'var(--green)';
1223             sickBadge.style.border     = '1px solid rgba(52,211,153,0.25)';
1224         }
1225
1226         /* CASUAL */
1227         const cas      = dashboard?.casual;
1228         const casAvail = Number(cas?.available ?? 0);
1229         const casTotal = Number(cas?.opening   ?? 0) + Number(cas?.accrued ?? 0);
1230         const casUsed  = Number(cas?.used     ?? 0);
1231
1232         document.getElementById('cal-casual-remaining').textContent = casAvail;
1233         document.getElementById('cal-casual-sub').textContent =
1234             `${casUsed} used · ${casTotal} total this year`;
1235
1236     } catch(e) {
1237         console.error('Could not load leave balances:', e);
1238         document.getElementById('cal-sick-remaining').textContent = '--';
1239         document.getElementById('cal-casual-remaining').textContent = '--';
1240     }
1241 }
1242
1243     async function buildCalendarEvents(){
1244         const events = [];
1245         try{
1246             // – Employee leaves –
1247             const data = await apiFetch('/leave_requests/employee');
1248             const leaves = data?.requests || data || [];
1249             leaves.forEach(l=>{
1250                 const s = (l.status||'PENDING').toUpperCase();
1251                 const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
1252                 const typeLabel = TYPE_LABEL[l.leaveType] || l.leaveType || 'Leave';
1253
1254                 // FullCalendar end date is exclusive, so add 1 day for multi-day events
1255                 let endDate = l.endDate;
1256                 if(endDate){
1257                     const d = new Date(endDate);
1258                     d.setDate(d.getDate()+1);
1259                     endDate = d.toISOString().split('T')[0];
1260                 }
1261
1262                 events.push({
1263                     id      : `leave-${l.leaveRequestId}`,
1264                     title   : `${typeLabel} (${s.charAt(0)+s.slice(1).toLowerCase()})`,

```

```

1265         start   : l.startDate,
1266         end     : endDate,
1267         allDay   : true,
1268         backgroundColor : c.bg,
1269         borderColor : c.border,
1270         textColor : c.text,
1271         extendedProps : {
1272             type : 'leave',
1273             status : s,
1274             reason : l.reason || '-',
1275             leaveType: l.leaveType
1276         }
1277     });
1278 });
1279 }catch(e){ console.warn('Could not load leaves for calendar:', e); }
1280
1281 try{
1282     // - Public holidays (admin-added) -
1283     const holidays = await apiFetch('/holidays');
1284     (holidays||[]).forEach(h=>{
1285         const c = STATUS_COLORS.HOLIDAY;
1286         events.push({
1287             id : `holiday-${h.id||h.holidayId||Math.random()}`,
1288             title : ` ${h.name||h.holidayName||'Holiday'}`,
1289             start : h.date||h.holidayDate,
1290             allDay : true,
1291             backgroundColor : c.bg,
1292             borderColor : c.border,
1293             textColor : c.text,
1294             extendedProps : { type:'holiday', description: h.description||'' }
1295         });
1296     });
1297 }catch(e){ console.warn('Could not load holidays for calendar:', e); }
1298
1299 return events;
1300 }
1301
1302 function initFullCalendar(){
1303     const calendarEl = document.getElementById('employee-calendar');
1304
1305     employeeCalendar = new FullCalendar.Calendar(calendarEl, {
1306         initialView: 'dayGridMonth',
1307         height : 'auto',
1308         headerToolbar:{
1309             left : 'prev,next today',
1310             center: 'title',
1311             right : 'dayGridMonth,dayGridWeek,listMonth'
1312         },
1313         buttonText:{
1314             today : 'Today',
1315             month : 'Month',
1316             week : 'Week',
1317             listMonth: 'List'
1318         },
1319         dayMaxEvents: 3, // "+N more" link if overflowing
1320         navLinks : true, // click day number day view
1321         events : [], // start empty, populate below
1322
1323         // Event hover tooltip
1324         eventMouseEnter: function(info){
1325             const ep = info.event.extendedProps;
1326             const tooltip = document.getElementById('cal-tooltip');
1327             let html = `<div class="tt-title">${info.event.title}</div>`;
1328             if(ep.type==='leave'){
1329                 html += `<div class="tt-row"> ${info.event.startStr}
1330 ${info.event.endStr||info.event.startStr}</div>`;
1331                 if(ep.reason && ep.reason!=='-')
1332                     html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
1333                 html += `<div class="tt-row">Status: <strong
1334 style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
1335             } else {
1336                 if(ep.description) html += `<div class="tt-row">${ep.description}</div>`;
1337             }
1338             tooltip.innerHTML = html;
1339             tooltip.style.display = 'block';
1340         },

```

```

1339         eventMouseLeave: function(){
1340             document.getElementById('cal-tooltip').style.display='none';
1341         },
1342         eventClick: function(info){
1343             // Click navigates to leave history
1344             const ep = info.event.extendedProps;
1345             if(ep.type==='leave'){
1346                 showView('my-leaves', document.querySelector('.nav-item:nth-child(4)'));
1347             }
1348         },
1349     });
1350
1351     employeeCalendar.render();
1352     calendarInitialized = true;
1353
1354     // Load events asynchronously after render
1355     refreshCalendarEvents();
1356 }
1357
1358 async function refreshCalendarEvents(){
1359     if(!employeeCalendar) return;
1360     const events = await buildCalendarEvents();
1361     // Remove all existing events and re-add
1362     employeeCalendar.removeAllEvents();
1363     events.forEach(e=>employeeCalendar.addEvent(e));
1364 }
1365
1366 async function refreshCalendar(){
1367     await loadCalendarBalanceCards();
1368     await refreshCalendarEvents();
1369     showAlert('Calendar refreshed!', 'success');
1370 }
1371
1372 // Move tooltip with mouse
1373 document.addEventListener('mousemove', function(e){
1374     const tt = document.getElementById('cal-tooltip');
1375     if(tt.style.display==='block'){
1376         tt.style.left = (e.clientX+14)+'px';
1377         tt.style.top = (e.clientY+14)+'px';
1378         // Keep tooltip on screen
1379         const rect = tt.getBoundingClientRect();
1380         if(rect.right > window.innerWidth)
1381             tt.style.left = (e.clientX - rect.width - 10)+'px';
1382         if(rect.bottom > window.innerHeight)
1383             tt.style.top = (e.clientY - rect.height - 10)+'px';
1384     }
1385 });
1386
1387 /* =====
1388     RESTAURANTS
1389     ===== */
1390 async function loadRestaurants(){
1391     try{
1392         const data=await apiFetch('/restaurants');
1393         availableRestaurants=data||[];
1394         document.getElementById('restaurants-list').innerHTML=data?.length
1395             ? data.map(r=><tr>
1396                 <td>${r.name||'-'}</td>
1397                 <td>${r.supportedMealSlots?.join(', ')}||'-'}</td>
1398                 <td><span class="chip
1399 ${r.active?'active':'rejected'}>${r.active?'Active':'Inactive'}</span></td>
1400                 <td><button class="btn btn-secondary btn-sm"
1401 onclick="openSubscriptionForRestaurant(${r.restaurantId||r.id},${JSON.stringify(r.supportedMealSlots||[])}>
1402                 Subscribe</button></td>
1403                 </tr>`).join('')
1404             : '<tr><td colspan="4" class="empty-state">No restaurants available</td></tr>';
1405     }catch(e){
1406         document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-
1407 state">Could not load restaurants</td></tr>';
1408     }
1409 }
1410
1411 async function loadRestaurantChoices(){
1412     const select=document.getElementById('sub-restaurant-id');
1413     try{

```

```

1412         const data=await apiFetch('/restaurants');
1413         availableRestaurants=data||[];
1414         select.innerHTML=availableRestaurants.length
1415             ? '<option value="">Select a restaurant</option>'+
1416             availableRestaurants.map(r=>`<option
value="${r.restaurantId}|${r.id}">${r.name}</option>`).join('')
1417             : '<option value="">No restaurants available</option>';
1418         }catch(e){ select.innerHTML='<option value="">Could not load restaurants</option>'; }
1419     }
1420
1421     function openSubscriptionForRestaurant(restaurantId, supportedMealSlots){
1422         showView('add-subscription',null);
1423         if(supportedMealSlots?.length) document.getElementById('sub-meal-
slot').value=supportedMealSlots[0];
1424         loadRestaurantChoices().then(()=>{ document.getElementById('sub-restaurant-
id').value=String(restaurantId); });
1425     }
1426
1427     /* =====
1428     SUBSCRIPTIONS
1429     ===== */
1430     async function loadSubscriptions(){
1431         try{
1432             const data=await apiFetch('/subscription/user');
1433             const arr=data||[];
1434             document.getElementById('subscriptions-list').innerHTML=arr.length
1435                 ? arr.map(s=>`<tr>
1436                     <td>${s.restaurantName}|${s.restaurantId}|'-'}</td>
1437                     <td>${s.mealSlot}|'-'}</td>
1438                     <td>${s.scheduleType}|'-'}${s.dayOfWeek?' · '+s.dayOfWeek:''}</td>
1439                     <td>${s.nextDeliveryTime?new Date(s.nextDeliveryTime).toLocaleString():'-'}</td>
1440                     <td><span class="chip
1441                     ${s.status||''}.toLowerCase())">${s.status||'ACTIVE'}</span></td>
1442                     <td><button class="btn btn-danger btn-sm"
onlick="cancelSub(${s.subscriptionId})">Cancel</button></td>
1443                     </tr>`).join('')
1444                     : '<tr><td colspan="6" class="empty-state">No subscriptions found</td></tr>';
1445             document.getElementById('subscriptions-list').innerHTML='<tr><td colspan="6" class="empty-
state">Could not load subscriptions</td></tr>';
1446         }
1447     }
1448
1449     document.getElementById('sub-schedule-type').addEventListener('change',function(){
1450         document.getElementById('day-of-week-group').style.display=this.value==='WEEKLY'?'block':'none';
1451     });
1452     document.getElementById('sub-meal-slot').addEventListener('change',loadRestaurantChoices);
1453
1454     async function addSubscription(){
1455         const restaurantId=parseInt(document.getElementById('sub-restaurant-id').value);
1456         const mealSlot =document.getElementById('sub-meal-slot').value;
1457         const scheduleType=document.getElementById('sub-schedule-type').value;
1458         const dayOfWeek =document.getElementById('sub-day-of-week')?.value;
1459         if(!restaurantId||!mealSlot){showAlert('Please fill all fields','error');return;}
1460         const body={scheduleType,mealSlots:[mealSlot],slotOrders:[{mealSlot,restaurantId}]};
1461         if(scheduleType==='WEEKLY'&&dayOfWeek) body.dayOfWeek=dayOfWeek;
1462         try{
1463             await apiFetch('/subscription',{method:'POST',body:JSON.stringify(body)});
1464             showAlert('Subscribed successfully!','success');
1465             loadSubscriptions(); loadDashboard();
1466         }catch(e){ showAlert('Subscription failed: '+e.message,'error'); }
1467     }
1468
1469     async function cancelSub(id){
1470         if(!confirm('Cancel this subscription?')) return;
1471         try{
1472             await apiFetch(`/subscription/${id}/expire`,{method:'PUT'});
1473             showAlert('Subscription cancelled','success');
1474             loadSubscriptions();
1475         }catch(e){ showAlert('Failed to cancel: '+e.message,'error'); }
1476     }
1477
1478     /* =====
1479     MY DEVICES
1480     ===== */
1481     async function loadMyAssignedDevices(){

```

```

1482         try{
1483             const devices=await apiFetch('/devices/my');
1484             const tbody=document.getElementById('my-devices-list');
1485             if(!devices||!devices.length){
1486                 tbody.innerHTML=<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
1487                 return;
1488             }
1489             const today=new Date();
1490             tbody.innerHTML=devices.map(d=>{
1491                 const we=d.warrantyExpiryDate?new Date(d.warrantyExpiryDate):null;
1492                 const wd=we?(we>today
1493                     ?`<span style="color:var(--green);font-size:11px"> ${d.warrantyExpiryDate}</span>`
1494                     :`<span style="color:var(--red);font-size:11px"> Expired
${d.warrantyExpiryDate}</span>`)
1495                     :`<span style="color:var(--text3);font-size:11px">--</span>';
1496                 return `<tr>
1497                     <td>#${d.id}</td>
1498                     <td><strong>${d.deviceName||'-'}</strong></td>
1499                     <td>${d.brand||'-'}</td>
1500                     <td><span class="chip">${(d.deviceType||'').toLowerCase()}</span></td>
1501                     <td>${wd}</td>
1502                     <td style="font-size:12px;color:var(--text2)">${d.assignedDate||'-'}</td>
1503                     <td><span class="chip
${(d.deviceStatus||'').toLowerCase()}>${d.deviceStatus||'-'}</span></td>
1504                     </tr>`;
1505             }).join('');
1506         }catch(e){ console.error(e); }
1507     }
1508
1509     /* =====
1510     SERVICE REQUESTS
1511     ===== */
1512     async function loadServiceRequests(){
1513         await Promise.all([loadServiceRequestsList(),loadMyDevicesForSR()]);
1514     }
1515
1516     async function loadServiceRequestsList(){
1517         try{
1518             const requests=await apiFetch('/service-requests/my');
1519             const tbody=document.getElementById('service-requests-list');
1520             if(!requests||!requests.length){
1521                 tbody.innerHTML=<tr><td colspan="7" class="empty-state">No service requests
yet</td></tr>';
1522                 return;
1523             }
1524             tbody.innerHTML=requests.map(r=>`<tr>
1525                 <td style="color:var(--text3);font-family:var(--mono)">#${r.id}</td>
1526                 <td>${r.deviceName||'-'}</td>
1527                 <td><span class="chip pending">${r.requestType}</span></td>
1528                 <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap">${r.issueDescription||'-'}</td>
1529                 <td>${r.urgent?'<span class="chip rejected">YES</span>':`<span style="color:var(--
text3)">No</span>`}</td>
1530                 <td><span class="chip ${((r.status||'').toLowerCase())}>${r.status||'-'}</span></td>
1531                 <td style="font-size:12px;color:var(--text3)">${r.raisedAt?new
Date(r.raisedAt).toLocaleDateString():'-'}</td>
1532                 </tr>`.join('');
1533             }catch(e){ console.error(e); }
1534         }
1535
1536     async function loadMyDevicesForSR(){
1537         try{
1538             const devices=await apiFetch('/devices/my');
1539             const select=document.getElementById('sr-device-select');
1540             select.innerHTML=<option value="">-- Select your device --</option>';
1541             (devices||[]).forEach(d=>{
1542                 const opt=document.createElement('option');
1543                 opt.value=d.id;
1544                 opt.textContent=`${d.deviceName} (${d.brand||d.deviceType||'Device'})`;
1545                 select.appendChild(opt);
1546             });
1547         }catch(e){ console.error('Failed to load devices for SR',e); }
1548     }
1549
1550     async function submitServiceRequest(){

```



```

1551     const deviceId      =parseInt(document.getElementById('sr-device-select').value);
1552     const requestType   =document.getElementById('sr-type').value;
1553     const issueDescription=document.getElementById('sr-description').value.trim();
1554     const urgent        =document.getElementById('sr-urgent').checked;
1555     if(!deviceId){showAlert('Please select a device','error');return;}
1556     if(!issueDescription){showAlert('Please describe the issue','error');return;}
1557     try{
1558         await apiFetch('/service-
requests',{method:'POST',body:JSON.stringify({deviceId,requestType,issueDescription,urgent})});
1559         showAlert('Service request submitted!','success');
1560         clearServiceRequestForm();
1561         loadServiceRequestsList();
1562     }catch(e){ showAlert('Failed: '+e.message,'error'); }
1563 }
1564
1565 function clearServiceRequestForm(){
1566     document.getElementById('sr-device-select').value='';
1567     document.getElementById('sr-type').value='REPAIR';
1568     document.getElementById('sr-description').value='';
1569     document.getElementById('sr-urgent').checked=false;
1570 }
1571
1572 /* =====
1573     LOGOUT + ALERT
1574 ===== */
1575 function logout(){
1576     localStorage.removeItem('jwt_token');
1577     localStorage.removeItem('refresh_token');
1578     localStorage.removeItem('auth_type');
1579     localStorage.removeItem('last_role');
1580     fetch(`${API}/session/logout`,{method:'POST',credentials:'include'}).finally(()=>{
1581         document.cookie='JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
1582         window.location.href='/google.html';
1583     });
1584 }
1585 async function loadNotifCount() {
1586     try {
1587         const data = await apiFetch('/notifications/unread-count');
1588         const badge = document.getElementById('notif-badge');
1589         if (data && data.count > 0) {
1590             badge.textContent = data.count;
1591             badge.style.display = 'flex';
1592         } else {
1593             badge.style.display = 'none';
1594         }
1595     } catch(e) {
1596         console.warn('Could not load notification count:', e);
1597     }
1598 }
1599
1600 async function toggleNotifPanel() {
1601     const panel = document.getElementById('notif-panel');
1602     if (panel.style.display === 'none') {
1603         panel.style.display = 'block';
1604         try {
1605             await loadNotifications();
1606             await apiFetch('/notifications/mark-read', { method: 'POST' });
1607             document.getElementById('notif-badge').style.display = 'none';
1608         } catch(e) {
1609             console.warn('Notification panel error:', e);
1610         }
1611     } else {
1612         panel.style.display = 'none';
1613     }
1614 }
1615
1616 async function loadNotifications() {
1617     try {
1618         const data = await apiFetch('/notifications');
1619         const list = document.getElementById('notif-list');
1620         if (!data || data.length === 0) {
1621             list.innerHTML = '<div class="notif-item">No notifications yet.</div>';
1622             return;
1623         }
1624         list.innerHTML = data.map(n => `
1625         <div class="notif-item ${n.read ? '' : 'unread'}">

```

```

1626         <div>${n.message}</div>
1627         <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
1628     </div>
1629     `).join('');
1630     } catch(e) {
1631         document.getElementById('notif-list').innerHTML =
1632             '<div class="notif-item">Could not load notifications.</div>';
1633         console.warn('loadNotifications failed:', e);
1634     }
1635 }
1636
1637 // Load badge count on page load
1638 loadNotifCount();
1639 // Refresh count every 30 seconds
1640 setInterval(loadNotifCount, 30000);
1641
1642 document.getElementById('mat-start-date').addEventListener('change', function(){
1643     if(this.value) {
1644         const start = new Date(this.value);
1645         const end = new Date(start);
1646         end.setDate(end.getDate() + 179); // 180 days inclusive
1647         document.getElementById('mat-end-date').value =
1648             end.toISOString().split('T')[0];
1649     } else {
1650         document.getElementById('mat-end-date').value = '';
1651     }
1652 });
1653
1654 async function applyMaternityLeave() {
1655     const startDate = document.getElementById('mat-start-date').value;
1656     const reason = document.getElementById('mat-reason').value.trim();
1657
1658     if(!startDate) {
1659         showAlert('Please select a start date', 'error');
1660         return;
1661     }
1662
1663     try {
1664         await apiFetch('/leave_requests/maternity/apply', {
1665             method: 'POST',
1666             body: JSON.stringify({ startDate, reason })
1667         });
1668         showAlert('Maternity leave request submitted successfully!', 'success');
1669         // Clear form
1670         document.getElementById('mat-start-date').value = '';
1671         document.getElementById('mat-end-date').value = '';
1672         document.getElementById('mat-reason').value = '';
1673         // Refresh dashboard
1674         loadDashboard();
1675     } catch(e) {
1676         showAlert('Failed to submit: ' + e.message, 'error');
1677     }
1678 }
1679
1680 // Close panel when clicking outside
1681 document.addEventListener('click', e => {
1682     const panel = document.getElementById('notif-panel');
1683     const wrapper = document.querySelector('.notif-wrapper');
1684     if (!panel.contains(e.target) && !wrapper.contains(e.target)) {
1685         panel.style.display = 'none';
1686     }
1687 });
1688
1689 function showAlert(msg, type){
1690     const el=document.createElement('div');
1691     el.className=`alert alert-${type}`;
1692     el.textContent=msg;
1693     document.body.appendChild(el);
1694     setTimeout(()=>el.remove(),3000);
1695 }
1696 // ===== TWO-FACTOR AUTH =====
1697 async function loadTotpStatus() {
1698     const chip = document.getElementById('totp-status-chip');
1699     chip.textContent = 'Checking...';
1700     chip.className = 'chip';
1701     document.getElementById('qr-area').style.display = 'none';

```

```

1702     document.getElementById('totp-verify-input').value = '';
1703     document.getElementById('totp-verify-error').style.display = 'none';
1704     try {
1705         const data = await apiFetch('/totp/status');
1706         if (data.totpEnabled) {
1707             chip.textContent = 'ENABLED';
1708             chip.className = 'chip approved';
1709             document.getElementById('totp-setup-section').style.display = 'none';
1710             document.getElementById('totp-disable-section').style.display = 'block';
1711         } else {
1712             chip.textContent = 'DISABLED';
1713             chip.className = 'chip rejected';
1714             document.getElementById('totp-setup-section').style.display = 'block';
1715             document.getElementById('totp-disable-section').style.display = 'none';
1716         }
1717     } catch(e) {
1718         chip.textContent = 'Error';
1719         chip.className = 'chip pending';
1720     }
1721 }
1722
1723 async function generateTotpQr() {
1724     const btn = document.getElementById('generate-qr-btn');
1725     btn.textContent = 'Generating...';
1726     btn.disabled = true;
1727     try {
1728         const data = await apiFetch('/totp/setup', { method: 'POST' });
1729         document.getElementById('totp-secret-display').textContent = data.secret || '';
1730         const qrUri = encodeURIComponent(data.qrCodeUri || '');
1731         document.getElementById('qr-img').src =
1732             `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
1733         document.getElementById('qr-area').style.display = 'block';
1734     } catch(e) {
1735         showAlert('Failed to generate QR: ' + e.message, 'error');
1736     } finally {
1737         btn.textContent = 'Regenerate QR Code';
1738         btn.disabled = false;
1739     }
1740 }
1741
1742 async function verifyAndActivateTotp() {
1743     const code = document.getElementById('totp-verify-input').value.trim();
1744     const errEl = document.getElementById('totp-verify-error');
1745     errEl.style.display = 'none';
1746     if (code.length !== 6) {
1747         errEl.textContent = 'Please enter a 6-digit code.';
1748         errEl.style.display = 'block';
1749         return;
1750     }
1751     try {
1752         await apiFetch('/totp/verify-setup', {
1753             method: 'POST',
1754             body: JSON.stringify({ code })
1755         });
1756         showAlert('2FA activated! You will need your authenticator on next login.', 'success');
1757         loadTotpStatus();
1758     } catch(e) {
1759         errEl.textContent = 'Invalid code. Try again.';
1760         errEl.style.display = 'block';
1761     }
1762 }
1763
1764 async function disableTotp() {
1765     if (!confirm('Are you sure you want to disable two-factor authentication?')) return;
1766     try {
1767         await apiFetch('/totp/disable', { method: 'DELETE' });
1768         showAlert('2FA has been disabled.', 'success');
1769         loadTotpStatus();
1770     } catch(e) {
1771         showAlert('Failed to disable 2FA: ' + e.message, 'error');
1772     }
1773 }
1774 </script>
1775 </body>
1776 </html>

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Sign In</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#070910;
12             --panel:#0e1118;
13             --panel2:#131821;
14             --border:rgba(255,255,255,0.07);
15             --border-lit:rgba(255,255,255,0.14);
16             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
17             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
18             --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.12);
19             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
20             --orange:#f59e0b;--orange-dim:rgba(245,158,11,0.12);
21             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
22             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
23         }
24
25         body{
26             font-family:var(--font);
27             background:var(--bg);
28             color:var(--text);
29             min-height:100vh;
30             display:flex;
31             overflow-y:auto;
32         }
33
34         /* ---- BACKGROUND ---- */
35         .bg-layer{
36             position:fixed;inset:0;z-index:0;
37             background:
38                 radial-gradient(ellipse 60% 50% at 20% 50%, rgba(79,140,255,0.06) 0%, transparent 70%),
39                 radial-gradient(ellipse 40% 60% at 80% 20%, rgba(167,139,250,0.05) 0%, transparent 60%),
40                 radial-gradient(ellipse 50% 40% at 70% 80%, rgba(52,211,153,0.04) 0%, transparent 60%);
41         }
42         .grid-overlay{
43             position:fixed;inset:0;z-index:0;
44             background-image:
45                 linear-gradient(rgba(255,255,255,0.025) 1px, transparent 1px),
46                 linear-gradient(90deg, rgba(255,255,255,0.025) 1px, transparent 1px);
47             background-size:40px 40px;
48             mask-image:radial-gradient(ellipse 80% 80% at center, black 20%, transparent 80%);
49         }
50
51         #oauth-loading {
52             display: none;
53             position: fixed; inset: 0;
54             background: linear-gradient(135deg, #0d0f14 0%, #131720 100%);
55             z-index: 9999;
56             justify-content: center;
57             align-items: center;
58             flex-direction: column;
59             gap: 20px;
60         }
61         #oauth-loading.visible { display: flex; }
62         .spinner {
63             width: 48px; height: 48px;
64             border: 3px solid #2a2f3f;
65             border-top-color: #4f8cff;
66             border-radius: 50%;
67             animation: spin 0.9s linear infinite;
68         }
69         @keyframes spin { to { transform: rotate(360deg); } }
70

```

```

71      /* ---- SPLIT LAYOUT ---- */
72      .left-panel{
73          flex:1;display:flex;flex-direction:column;justify-content:center;
74          padding:60px;position:relative;z-index:1;
75          animation:fadeLeft 0.8s ease both;
76      }
77      @keyframes fadeLeft{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
78      .brand{margin-bottom:48px}
79      .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:20px}
80      .brand-icon{
81          width:44px;height:44px;background:linear-gradient(135deg,#4f8cff,#6366f1);
82          border-radius:12px;display:flex;align-items:center;justify-content:center;
83          font-weight:700;font-size:18px;color:#fff;
84          box-shadow:0 0 30px rgba(79,140,255,0.35);
85          flex-shrink:0;
86      }
87      .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
88      .brand-tagline{font-size:13px;color:var(--text2)}
89      .hero-text h1{
90          font-size:clamp(36px,4vw,52px);font-weight:700;
91          line-height:1.1;letter-spacing:-1px;color:var(--text);margin-bottom:16px;
92      }
93      .hero-text h1 .hl-blue{color:var(--blue)}
94      .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:380px}
95      .feature-pills{display:flex;flex-wrap:wrap;gap:8px;margin-top:32px}
96      .pill{
97          display:inline-flex;align-items:center;gap:6px;
98          padding:6px 14px;border-radius:20px;font-size:12px;font-weight:500;
99          border:1px solid var(--border);background:var(--panel);color:var(--text2);
100      }
101      .pill .dot{width:6px;height:6px;border-radius:50%;flex-shrink:0}
102      .pill.blue .dot{background:var(--blue)} .pill.blue{border-
color:rgba(79,140,255,0.2);color:rgba(79,140,255,0.8)}
103      .pill.violet .dot{background:var(--violet)} .pill.violet{border-
color:rgba(167,139,250,0.2);color:rgba(167,139,250,0.8)}
104      .pill.orange .dot{background:var(--orange)} .pill.orange{border-
color:rgba(245,158,11,0.2);color:rgba(245,158,11,0.8)}
105
106      /* ---- RIGHT PANEL (FORM) ---- */
107      .right-panel{
108          width:460px;flex-shrink:0;
109          display:flex;align-items:flex-start;justify-content:center;
110          overflow-y:auto;
111          padding:32px 40px;position:relative;z-index:1;
112          border-left:1px solid var(--border);
113          background:rgba(14,17,24,0.8);
114          backdrop-filter:blur(20px);
115          animation:fadeRight 0.8s ease both;
116      }
117      @keyframes
fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
118      .form-box{width:100%}
119
120      /* ---- AUTH TABS ---- */
121      .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-
bottom:24px;border:1px solid var(--border)}
122      .auth-tab{
123          flex:1;padding:9px;border:none;background:transparent;
124          color:var(--text2);font-family:var(--font);font-weight:600;
125          font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;
126      }
127      .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}
128
129      /* ---- FORM SECTION HEADER ---- */
130      .section-header{
131          margin-bottom:20px;
132      }
133      .section-title{
134          font-size:18px;font-weight:700;color:var(--text);letter-spacing:-0.3px;margin-bottom:4px;
135      }
136      .section-sub{
137          font-size:12px;color:var(--text3);
138      }
139
140      /* ---- FORM ELEMENTS ---- */
141      .form-group{margin-bottom:16px}

```

```

142     .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-
transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
143     .form-input{
144         width:100%;padding:11px 14px;
145         background:var(--bg);border:1px solid var(--border);
146         border-radius:9px;color:var(--text);
147         font-family:var(--font);font-size:14px;
148         transition:all 0.2s;
149     }
150     .form-input:focus{outline:none;border-color:var(--blue);box-shadow:0 0 3px rgba(79,140,255,0.08)}
151     .form-input::placeholder{color:var(--text3)}
152
153     /* ---- BUTTONS ---- */
154     .btn-login{
155         width:100%;padding:12px;border:none;border-radius:9px;
156         font-family:var(--font);font-size:14px;font-weight:600;
157         cursor:pointer;transition:all 0.2s;margin-top:4px;
158         background:var(--blue);color:#fff;
159     }
160     .btn-login:hover{background:#6ba0ff;box-shadow:0 6px 20px
161     rgba(79,140,255,0.35);transform:translateY(-1px)}
162     .btn-login:active{transform:translateY(0)}
163     .btn-login.loading{opacity:0.7;pointer-events:none}
164
165     .divider{display:flex;align-items:center;gap:12px;margin:20px 0;color:var(--text3);font-
166     size:11px;font-family:var(--mono)}
167     .divider::before,.divider::after{content:'';flex:1;height:1px;background:var(--border)}
168
169     .btn-google{
170         width:100%;padding:11px;background:transparent;
171         border:1px solid var(--border-lit);border-radius:9px;
172         color:var(--text2);font-family:var(--font);font-size:13px;font-weight:500;
173         cursor:pointer;transition:all 0.2s;
174         display:flex;align-items:center;justify-content:center;gap:10px;
175     }
176     .btn-google:hover{background:rgba(255,255,255,0.04);border-color:rgba(255,255,255,0.2);color:var(--
177     text)}
178
179     /* ---- VENDOR LINK ---- */
180     .vendor-link-row{
181         margin-top:18px;padding-top:16px;
182         border-top:1px solid var(--border);
183         display:flex;align-items:center;justify-content:space-between;
184     }
185     .vendor-link-row p{font-size:12px;color:var(--text3);}
186     .btn-vendor-link{
187         padding:7px 16px;border-radius:8px;
188         border:1px solid rgba(248,113,113,0.3);
189         background:var(--red-dim);color:var(--red);
190         font-family:var(--font);font-size:12px;font-weight:600;
191         cursor:pointer;text-decoration:none;transition:all 0.2s;
192         white-space:nowrap;
193     }
194     .btn-vendor-link:hover{background:rgba(248,113,113,0.2);border-color:var(--red);}
195
196     /* ---- ERROR / STATUS ---- */
197     .status-box{
198         border-radius:9px;padding:10px 14px;margin-top:14px;
199         font-size:13px;font-weight:500;display:none;
200         animation:fadeIn 0.3s ease;
201     }
202     @keyframes fadeIn{from{opacity:0;transform:translateY(-4px)}to{opacity:1;transform:translateY(0)}}
203     .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid
204     rgba(248,113,113,0.2)}
205     .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid
206     rgba(52,211,153,0.2)}
207     .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid
208     rgba(79,140,255,0.2)}
209
210     /* ---- FOOTER ---- */
211     .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
212     .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
213     .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
214     .tz-select{
215         flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);
216         border-radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;

```

```

211     }
212     .tz-select:focus{outline:none;border-color:var(--border-lit)}
213     .tz-select option{background:var(--panel)}
214     .tz-time{font-size:10px;color:var(--blue);font-family:var(--mono)}
215
216     /* ---- RESPONSIVE ---- */
217     @media(max-width:900px){
218         .left-panel{display:none}
219         .right-panel{width:100%;border-left:none;background:var(--bg)}
220     }
221 </style>
222 </head>
223 <body>
224 <div id="oauth-loading">
225     <div class="spinner"></div>
226     <div style="text-align: center;">
227         <p style="color: #e0e0e0; font-size: 16px; font-weight: 600; margin: 0 0 6px; font-family:
'Outfit', sans-serif;">Signing you in...</p>
228         <p style="color: #8b92a8; font-size: 13px; margin: 0; font-family: 'Outfit', sans-serif;">Verifying
your Google account</p>
229     </div>
230 </div>
231 <div class="bg-layer"></div>
232 <div class="grid-overlay"></div>
233
234 <!-- LEFT HERO -->
235 <div class="left-panel">
236     <div class="brand">
237         <div class="brand-logo">
238             <div class="brand-icon">E</div>
239             <div>
240                 <div class="brand-name">EMS Portal</div>
241                 <div class="brand-tagline">Employee Management System</div>
242             </div>
243         </div>
244     </div>
245     <div class="hero-text">
246         <h1>Your workspace,<br><span class="hl-blue">one login</span><br>away.</h1>
247         <p>Employees, Managers, and Admins each get their own tailored experience. Sign in to access your
personalized dashboard.</p>
248     </div>
249     <div class="feature-pills">
250         <div class="pill blue"><span class="dot"></span>Employee Portal</div>
251         <div class="pill violet"><span class="dot"></span>Manager Console</div>
252         <div class="pill orange"><span class="dot"></span>Admin Control Center</div>
253         <div class="pill blue"><span class="dot"></span>Leave Management</div>
254         <div class="pill violet"><span class="dot"></span>Meal Subscriptions</div>
255         <div class="pill orange"><span class="dot"></span>Delivery Tracking</div>
256     </div>
257 </div>
258
259 <!-- RIGHT FORM -->
260 <div class="right-panel">
261     <div class="form-box">
262
263         <!-- SECTION HEADER -->
264         <div class="section-header">
265             <div class="section-title">Welcome back</div>
266             <div class="section-sub">Sign in – your dashboard will load automatically based on your
account.</div>
267         </div>
268
269         <!-- AUTH METHOD TABS -->
270         <div class="auth-tabs">
271             <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
272             <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
273         </div>
274
275         <!-- JWT FORM -->
276         <div id="jwt-form">
277             <div class="form-group">
278                 <label class="form-label">Email</label>
279                 <input class="form-input" id="jwt-username" type="text"
placeholder="your.email@company.com" autocomplete="off">
280             </div>
281             <div class="form-group">

```

```

282         <label class="form-label">Password</label>
283         <input class="form-input" id="jwt-password" type="password" placeholder="*****"
autocomplete="current-password">
284     </div>
285     <button class="btn-login" id="jwt-btn" onclick="jwtLogin()">Sign In</button>
286 </div>
287
288 <!-- SESSION FORM -->
289 <div id="session-form" style="display:none">
290     <div class="form-group">
291         <label class="form-label">Email</label>
292         <input class="form-input" id="session-username" type="text"
placeholder="your.email@company.com">
293     </div>
294     <div class="form-group">
295         <label class="form-label">Password</label>
296         <input class="form-input" id="session-password" type="password" placeholder="*****">
297     </div>
298     <button class="btn-login" id="session-btn" onclick="sessionLogin()">Sign In with
Session</button>
299 </div>
300
301 <!-- GOOGLE OAUTH – no role selector needed -->
302 <div class="divider">or continue with</div>
303 <button class="btn-google" onclick="googleLogin()">
304     <svg width="18" height="18" viewBox="0 0 24 24">
305         <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04
2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
306         <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71
1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
307         <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-
2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>
308         <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1
7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
309     </svg>
310     Continue with Google
311 </button>
312
313 <div class="status-box" id="status-box"></div>
314
315 <!-- Vendor Login Link -->
316 <div class="vendor-link-row">
317     <p>Are you a vendor?</p>
318     <a class="btn-vendor-link" href="/vendor-login.html">Vendor Login </a>
319 </div>
320
321 <!-- FOOTER -->
322 <div class="form-footer">
323     <div class="tz-row">
324         <span class="tz-label">TZ</span>
325         <select class="tz-select" id="tz-select" onchange="updateTz()">
326             <option value="Asia/Kolkata">India – IST UTC+5:30</option>
327             <option value="UTC">UTC</option>
328             <option value="America/New_York">New York – EST UTC-5</option>
329             <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
330             <option value="Europe/London">London – GMT UTC+0</option>
331             <option value="Europe/Paris">Paris – CET UTC+1</option>
332             <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
333             <option value="Asia/Dubai">Dubai – GST UTC+4</option>
334             <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
335             <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
336         </select>
337         <span class="tz-time" id="tz-time">--:--:--</span>
338     </div>
339 </div>
340
341 </div>
342
343 <!-- TOTP MODAL OVERLAY -->
344 <div id="totp-overlay" style="display:none;position:fixed;inset:0;z-
index:999;background:rgba(7,9,16,0.85);backdrop-filter:blur(8px);align-items:center;justify-content:center;">
345     <div style="background:var(--panel);border:1px solid var(--border-lit);border-
radius:16px;padding:36px;width:340px;text-align:center;animation:fadeIn 0.25s ease;">
346         <div style="font-size:28px;margin-bottom:8px;"></div>
347         <h3 style="font-size:17px;font-weight:700;color:var(--text);margin-bottom:6px;">Two-Factor
Verification</h3>

```



```

348         <p style="font-size:13px;color:var(--text2);margin-bottom:24px;">Enter the 6-digit code from
your authenticator app.</p>
349         <input id="totp-code" class="form-input" type="text" inputmode="numeric" maxlength="6"
350             placeholder="000000"
351             style="text-align:center;font-size:22px;font-family:var(--mono);letter-
spacing:8px;padding:14px;"
352             oninput="this.value=this.value.replace(/\D/g, '')">
353         <div id="totp-status" style="display:none;margin-top:10px;font-size:13px;color:var(--
red);"></div>
354         <button id="totp-verify-btn" class="btn-login" style="margin-top:16px;"
onclick="submitTotp()">Verify</button>
355         <button onclick="cancelTotp()" style="margin-
top:10px;width:100%;padding:10px;border:none;background:transparent;color:var(--text2);font-family:var(--
font);font-size:13px;cursor:pointer...
356     </div>
357 </div>
358 </div>
359
360 <script>
361     const API = 'http://localhost:8080';
362
363     // Role dashboard mapping
364     const DASHBOARDS = {
365         EMPLOYEE: '/employee-dashboard.html',
366         MANAGER: '/manager-dashboard.html',
367         ADMIN: '/dashboard.html',
368         USER: '/pending.html'
369     };
370
371     // ---- Auth tab switching ----
372     function switchTab(type) {
373         document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
374         document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
375         document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
376         document.getElementById('tab-session').classList.toggle('active', type === 'session');
377         hideStatus();
378     }
379
380     // ---- Status display ----
381     function showStatus(type, msg) {
382         const box = document.getElementById('status-box');
383         box.className = `status-box ${type}`;
384         box.textContent = msg;
385         box.style.display = 'block';
386     }
387     function hideStatus() {
388         document.getElementById('status-box').style.display = 'none';
389     }
390
391     // ---- Parse role from JWT ----
392     function getRoleFromJWT(token) {
393         try {
394             const payload = JSON.parse(atob(token.split('.')[1]));
395             console.log('JWT payload:', payload);
396             const role = payload.role || 'USER';
397             return typeof role === 'string' ? role.replace(/^ROLE_/, '') : 'USER';
398         } catch (e) {
399             console.error('Failed to parse JWT:', e);
400             return 'USER';
401         }
402     }
403
404     // ---- Redirect by role from token ----
405     async function redirectByRole(token, authType = localStorage.getItem('auth_type')) {
406         let role;
407
408         if (token) {
409             role = getRoleFromJWT(token);
410         } else {
411             try {
412                 const res = await fetch(`${API}/employee/me`, { credentials: 'include' });
413                 if (res.ok) {
414                     const emp = await res.json();
415                     role = emp.role || 'USER';
416                 } else {
417                     role = 'USER';

```

```

418     }
419   } catch(e) {
420     role = localStorage.getItem('last_role') || 'USER';
421   }
422 }
423
424 console.log('Final role for redirect:', role);
425 localStorage.setItem('last_role', role);
426
427 // Block vendor types from using the main login page
428 if (role === 'FOOD_VENDOR' || role === 'TECH_VENDOR') {
429   localStorage.removeItem('jwt_token');
430   localStorage.removeItem('auth_type');
431   localStorage.removeItem('last_role');
432   const jwtBtn = document.getElementById('jwt-btn');
433   const sesBtn = document.getElementById('session-btn');
434   if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
435   if (sesBtn) { sesBtn.classList.remove('loading'); sesBtn.textContent = 'Sign In with Session'; }
436 }
437 showStatus('error', 'Vendors must use the Vendor Login page.');
```

```

438   return;
439 }
440 const destination = DASHBOARDS[role];
441 if (!destination) {
442   localStorage.removeItem('jwt_token');
443   localStorage.removeItem('auth_type');
444   showStatus('error', 'Unknown role: ' + role + '. Please contact your administrator.');
```

```

445   const jwtBtn = document.getElementById('jwt-btn');
446   if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
447   return;
448 }
449
450 console.log('Redirecting to:', destination);
451 window.location.href = destination;
452 }
453
454 // ---- JWT Login ----
455 let _preAuthToken = null;
456
457 async function jwtLogin() {
458   const username = document.getElementById('jwt-username').value.trim();
459   const password = document.getElementById('jwt-password').value;
460   if (!username || !password) {
461     showStatus('error', 'Please enter your email and password');
```

```

462     return;
463   }
464
465   const btn = document.getElementById('jwt-btn');
466   btn.classList.add('loading'); btn.textContent = 'Signing in...';
467   showStatus('loading', 'Authenticating...');
```

```

468
469   try {
470     const res = await fetch(`${API}/Authenticate`, {
471       method: 'POST',
472       headers: { 'Content-Type': 'application/json' },
473       body: JSON.stringify({ username, password })
474     });
475
476     if (res.ok) {
477       const data = await res.json();
478
479       // TOTP required
480       if (data.requiresTotp) {
481         _preAuthToken = data.preAuthToken;
482         btn.classList.remove('loading'); btn.textContent = 'Sign In';
483         hideStatus();
484         openTotpModal();
485         return;
486       }
487
488       localStorage.setItem('jwt_token', data.accessToken);
489       localStorage.setItem('refresh_token', data.refreshToken || '');
490       localStorage.setItem('auth_type', 'jwt');
491       localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
492       showStatus('success', 'Login successful! Redirecting...');
```

```

493         setTimeout(() => redirectByRole(data.accessToken, 'jwt'), 600);
494
495     } else {
496         const err = await res.text();
497         showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
498         btn.classList.remove('loading'); btn.textContent = 'Sign In';
499     }
500 } catch (e) {
501     showStatus('error', 'Connection error: ' + e.message);
502     btn.classList.remove('loading'); btn.textContent = 'Sign In';
503 }
504 }
505
506 // ---- TOTP ----
507 function openTotpModal() {
508     const overlay = document.getElementById('totp-overlay');
509     overlay.style.display = 'flex';
510     document.getElementById('totp-code').value = '';
511     document.getElementById('totp-status').style.display = 'none';
512     setTimeout(() => document.getElementById('totp-code').focus(), 100);
513 }
514
515 function cancelTotp() {
516     _preAuthToken = null;
517     document.getElementById('totp-overlay').style.display = 'none';
518 }
519
520 async function submitTotp() {
521     const code = document.getElementById('totp-code').value.trim();
522     if (code.length !== 6) {
523         document.getElementById('totp-status').textContent = 'Please enter a 6-digit code.';
524         document.getElementById('totp-status').style.display = 'block';
525         return;
526     }
527
528     const verifyBtn = document.getElementById('totp-verify-btn');
529     verifyBtn.classList.add('loading'); verifyBtn.textContent = 'Verifying...';
530     document.getElementById('totp-status').style.display = 'none';
531
532     try {
533         const res = await fetch(`${API}/Authenticate/totp`, {
534             method: 'POST',
535             headers: { 'Content-Type': 'application/json' },
536             body: JSON.stringify({ preAuthToken: _preAuthToken, code })
537         });
538
539         if (res.ok) {
540             const data = await res.json();
541             _preAuthToken = null;
542             document.getElementById('totp-overlay').style.display = 'none';
543             localStorage.setItem('jwt_token', data.accessToken);
544             localStorage.setItem('refresh_token', data.refreshToken || '');
545             localStorage.setItem('auth_type', 'jwt');
546             localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
547             showStatus('success', 'Verified! Redirecting...');
548             setTimeout(() => redirectByRole(data.accessToken, 'jwt'), 600);
549         } else {
550             const err = await res.json().catch(() => ({}));
551             document.getElementById('totp-status').textContent = err.error || 'Invalid code. Try
again.';
552             document.getElementById('totp-status').style.display = 'block';
553             document.getElementById('totp-code').value = '';
554             document.getElementById('totp-code').focus();
555         }
556     } catch (e) {
557         document.getElementById('totp-status').textContent = 'Connection error: ' + e.message;
558         document.getElementById('totp-status').style.display = 'block';
559     } finally {
560         verifyBtn.classList.remove('loading'); verifyBtn.textContent = 'Verify';
561     }
562 }
563
564 // ---- Session Login ----
565 async function sessionLogin() {
566     const username = document.getElementById('session-username').value.trim();
567     const password = document.getElementById('session-password').value;

```

```

568         if (!username || !password) { showStatus('error', 'Please enter your email and password'); return;
569     }
570     const btn = document.getElementById('session-btn');
571     btn.classList.add('loading'); btn.textContent = 'Signing in...';
572     showStatus('loading', 'Authenticating...');
573
574     try {
575         const res = await fetch(`${API}/session/login`, {
576             method: 'POST',
577             headers: { 'Content-Type': 'application/json' },
578             body: JSON.stringify({ username, password }),
579             credentials: 'include'
580         });
581         if (res.ok) {
582             localStorage.setItem('auth_type', 'session');
583             localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
584             showStatus('success', 'Login successful! Redirecting...');
585             setTimeout(() => redirectByRole(null, 'session'), 600);
586         } else {
587             const err = await res.json().catch(() => ({}));
588             showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
589             btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
590         }
591     } catch(e) {
592         showStatus('error', 'Connection error: ' + e.message);
593         btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
594     }
595 }
596
597 // ---- Google OAuth – no role param needed, backend determines role from account ----
598 function googleLogin() {
599     const timezone = document.getElementById('tz-select').value;
600     window.location.href =
601         `/auth/google/init` +
602         `?timezone=${encodeURIComponent(timezone)}`;
603 }
604
605 // ---- Handle OAuth callback (token in query params) ----
606 async function handleOAuthCallback() {
607     const params = new URLSearchParams(window.location.search);
608     const accessToken = params.get('accessToken');
609     const refreshToken = params.get('refreshToken');
610
611     if (!accessToken) return;
612
613     window.history.replaceState({}, document.title, window.location.pathname);
614
615     localStorage.setItem('jwt_token', accessToken);
616     localStorage.setItem('refresh_token', refreshToken || '');
617     localStorage.setItem('auth_type', 'jwt');
618
619     const role = getRoleFromJWT(accessToken);
620     console.log('OAuth role from token:', role);
621     localStorage.setItem('last_role', role);
622
623     showStatus('success', 'Signed in! Redirecting...');
624     setTimeout(() => {
625         window.location.href = DASHBOARDS[role] || '/pending.html';
626     }, 800);
627 }
628
629 // ---- Timezone ----
630 function updateTz() {
631     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
632     refreshTzClock();
633 }
634 function refreshTzClock() {
635     const tz = document.getElementById('tz-select').value;
636     document.getElementById('tz-time').textContent =
637         new Date().toLocaleTimeString('en-GB', { timeZone: tz, hour12: false });
638 }
639
640 // ---- Enter key support ----
641 document.addEventListener('keydown', e => {
642     if (e.key !== 'Enter') return;

```

```
643     if (document.getElementById('totp-overlay').style.display === 'flex') {
644         submitTotp();
645         return;
646     }
647     if (document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
648     else sessionLogin();
649 });
650
651 // ---- Init ----
652 window.onload = () => {
653     const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
654     document.getElementById('tz-select').value = savedTz;
655     refreshTzClock();
656     setInterval(refreshTzClock, 1000);
657
658     const params      = new URLSearchParams(window.location.search);
659     const accessToken = params.get('accessToken');
660     if (accessToken) {
661         document.getElementById('oauth-loading').classList.add('visible');
662         document.querySelector('.right-panel').style.display = 'none';
663         document.querySelector('.left-panel').style.display = 'none';
664         handleOAuthCallback();
665     }
666 };
667 </script>
668 </body>
669 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS - Leave Management</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
8      <style>
9          * { margin: 0; padding: 0; box-sizing: border-box; }
10         :root {
11             --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
12             --panel: #1e2436; --panel2: #252c40;
13             --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
14             --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
15             --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
16             --green: #36d399; --green-dim: rgba(54,211,153,0.12);
17             --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
18             --red: #f87171; --red-dim: rgba(248,113,113,0.12);
19             --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
20             --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
21         }
22         body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-
height: 1.5; overflow: hidden; }
23         .app { display: flex; height: 100vh; }
24
25         /* Sidebar */
26         .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display:
flex; flex-direction: column; overflow-y: auto; }
27         .logo { padding: 20px; border-bottom: 1px solid var(--border); }
28         .logo h2 { font-size: 20px; color: var(--text); }
29         .logo span { color: var(--accent); }
30         .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
31         .nav-section { padding: 12px 20px 4px; font-size: 10px; font-weight: 600; color: var(--text2); text-
transform: uppercase; letter-spacing: 0.08em; }
32         .nav-item { padding: 10px 20px; margin: 2px 12px; border-radius: 8px; cursor: pointer; transition:
all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; font-size: 1...
33         .nav-item svg { opacity: 0.6; flex-shrink: 0; }
34         .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
35         .nav-item:hover svg, .nav-item.active svg { opacity: 1; }
36         .nav-item .badge-count { margin-left: auto; background: var(--amber-dim); color: var(--amber); font-
size: 10px; padding: 2px 6px; border-radius: 10px; font-weight: 600; }
37
38         /* Main */
39         .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
40         .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border); }
41         .page-title { font-size: 20px; font-weight: 600; }
42         .topbar-right { display: flex; align-items: center; gap: 16px; }
43         .user-pill { display: flex; align-items: center; gap: 8px; background: var(--bg3); border: 1px solid
var(--border2); border-radius: 20px; padding: 6px 12px; }
44         .user-avatar { width: 24px; height: 24px; border-radius: 50%; background: var(--accent-dim);
display: flex; align-items: center; justify-content: center; font-size: 10px; font-weight: 600; ...
45         .user-name { font-size: 12px; color: var(--text2); }
46         .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid
rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; font-family: var(--font); font-
...
47         .logout-btn:hover { background: rgba(248,113,113,0.2); }
48         .content { flex: 1; overflow-y: auto; padding: 24px; }
49
50         /* Views */
51         .view { display: none; }
52         .view.active { display: block; }
53
54         /* Stats */
55         .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px;
}
56         .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--
border); position: relative; overflow: hidden; }
57         .stat-card::before { content: ''; position: absolute; top: 0; left: 0; right: 0; height: 2px; }
58         .stat-card.blue::before { background: var(--accent); }

```

```

59     .stat-card.green::before { background: var(--green); }
60     .stat-card.amber::before { background: var(--amber); }
61     .stat-card.red::before { background: var(--red); }
62     .stat-label { font-size: 11px; color: var(--text2); text-transform: uppercase; letter-spacing:
0.05em; margin-bottom: 8px; }
63     .stat-value { font-size: 28px; font-weight: 600; }
64     .stat-value.blue { color: var(--accent); }
65     .stat-value.green { color: var(--green); }
66     .stat-value.amber { color: var(--amber); }
67     .stat-value.red { color: var(--red); }
68
69     /* Section header */
70     .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom:
16px; flex-wrap: wrap; gap: 10px; }
71     .section-title { font-size: 16px; font-weight: 600; }
72     .section-actions { display: flex; gap: 8px; align-items: center; }
73
74     /* Table */
75     .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
overflow: hidden; }
76     table { width: 100%; border-collapse: collapse; }
77     th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text2); font-size:
11px; font-weight: 600; text-transform: uppercase; letter-spacing: 0.04em; border-bottom: ...
78     td { padding: 12px 16px; border-top: 1px solid var(--border); font-size: 13px; }
79     tr:hover td { background: rgba(255,255,255,0.02); }
80     .empty-state { text-align: center; padding: 48px; color: var(--text2); font-size: 13px; }
81
82     /* Chips */
83     .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-
block; }
84     .chip.pending, .chip.PENDING { background: var(--amber-dim); color: var(--amber); }
85     .chip.approved, .chip.APPROVED { background: var(--green-dim); color: var(--green); }
86     .chip.rejected, .chip.REJECTED { background: var(--red-dim); color: var(--red); }
87     .chip.cancelled, .chip.CANCELLED { background: rgba(139,146,168,0.12); color: var(--text2); }
88     .chip.sick, .chip.SICK { background: var(--red-dim); color: var(--red); }
89     .chip.casual, .chip.CASUAL { background: var(--accent-dim); color: var(--accent); }
90     .chip.maternity, .chip.MATERNITY { background: var(--violet-dim); color: var(--violet); }
91
92     /* Buttons */
93     .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size:
13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
94     .btn-primary { background: var(--accent); color: white; }
95     .btn-primary:hover { background: #6ba0ff; }
96     .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
97     .btn-secondary:hover { color: var(--text); }
98     .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
99     .btn-danger:hover { background: rgba(248,113,113,0.2); }
100    .btn-success { background: var(--green-dim); color: var(--green); border: 1px solid
rgba(54,211,153,0.3); }
101    .btn-success:hover { background: rgba(54,211,153,0.2); }
102    .btn-sm { padding: 4px 10px; font-size: 11px; }
103    .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
104
105    /* Form */
106    .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 24px; max-width: 640px; }
107    .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
108    .form-group { display: flex; flex-direction: column; gap: 6px; }
109    .form-group.full { grid-column: 1 / -1; }
110    label { font-size: 11px; font-weight: 600; color: var(--text2); text-transform: uppercase; letter-
spacing: 0.04em; }
111    input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--
border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; tr...
112    input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
113    input[readonly] { opacity: 0.5; cursor: not-allowed; }
114    select option { background: var(--bg3); }
115
116    /* Balance cards */
117    .balance-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap:
16px; margin-bottom: 24px; }
118    .balance-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 20px; }
119    .balance-name { font-size: 12px; color: var(--text2); text-transform: uppercase; letter-spacing:
0.05em; margin-bottom: 4px; }
120    .balance-row { display: flex; align-items: baseline; gap: 6px; margin-bottom: 10px; }

```

```

121     .balance-big { font-size: 28px; font-weight: 600; }
122     .balance-of { font-size: 13px; color: var(--text2); }
123     .balance-bar { height: 4px; background: var(--border2); border-radius: 2px; overflow: hidden; }
124     .balance-fill { height: 100%; border-radius: 2px; transition: width 0.4s ease; }
125
126     /* Tabs */
127     .tab-bar { display: flex; gap: 2px; background: var(--bg3); border-radius: 8px; padding: 4px;
margin-bottom: 20px; width: fit-content; }
128     .tab-btn { padding: 7px 16px; border-radius: 6px; font-size: 12px; font-weight: 500; cursor:
pointer; color: var(--text2); border: none; background: transparent; font-family: var(--font); t...
129     .tab-btn.active { background: var(--panel); color: var(--text); }
130     .tab-btn:hover:not(.active) { color: var(--text); }
131
132     /* Modal */
133     .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
134     .modal-overlay.open { display: flex; }
135     .modal { background: var(--panel); border-radius: 12px; border: 1px solid var(--border2); padding:
24px; width: 460px; max-width: 90%; }
136     .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
137     .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
138
139     /* Alert */
140     .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index:
1100; animation: slideIn 0.3s ease; font-size: 13px; font-weight: 500; }
141     .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid
rgba(54,211,153,0.3); }
142     .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
143     @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
144
145     /* Pending count indicator */
146     .pending-dot { width: 7px; height: 7px; border-radius: 50%; background: var(--amber); display:
inline-block; margin-left: 4px; animation: pulse 2s infinite; }
147     @keyframes pulse { 0%,100% { opacity: 1; } 50% { opacity: 0.4; } }
148 </style>
149 </head>
150 <body>
151 <div class="app">
152
153     <!-- Sidebar -->
154     <div class="sidebar">
155         <div class="logo">
156             <h2>E<span>M</span>S</h2>
157             <p>Leave Management</p>
158         </div>
159         <div style="flex:1; padding: 8px 0;">
160             <div class="nav-section">Overview</div>
161             <div class="nav-item active" onclick="showView('dashboard',this)">
162                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><rect x="3" y="3" width="7" height="7"/><rect x="14" y="3" width="7" height="7"/><...
163                 Dashboard
164             </div>
165             <div class="nav-section">My Leave</div>
166             <div class="nav-item" onclick="showView('apply',this)">
167                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><circle cx="12" cy="12" r="10"/><line x1="12" y1="8" x2="12" y2="16"/><line x1="8"...
168                 Apply for Leave
169             </div>
170             <div class="nav-item" onclick="showView('my-leaves',this)">
171                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><path d="M14 2H6a2 2 0 0 0 2 2V16a2 2 0 0 0 2 2h12a2 2 0 0 0 2 2V8z"/><polyline po...
172                 My Requests
173             </div>
174             <div class="nav-item" onclick="showView('balance',this)">
175                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><line x1="18" y1="20" x2="18" y2="10"/><line x1="12" y1="20" x2="12" y2="4"/><line...
176                 Leave Balance
177             </div>
178             <div class="nav-section">Admin</div>
179             <div class="nav-item" id="nav-pending" onclick="showView('pending',this)">
180                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="9 11 12 14 22 4"/><path d="M21 12v7a2 2 0 0 1 2 2H5a2 2 0 0 1 2-...
181                 Pending Approvals
182                 <span class="badge-count" id="pending-count" style="display:none">0</span>

```



```

183         </div>
184         <div class="nav-item" onclick="showView('all-leaves',this)">
185             <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/>...
186             All Leave Requests
187         </div>
188     </div>
189     <div style="padding: 16px; border-top: 1px solid var(--border);">
190         <a href="/dashboard.html" style="color:var(--text2);font-size:12px;text-
decoration:none;display:flex;align-items:center;gap:8px;">
191             <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="15 18 9 12 15 6"/></svg>
192             Back to Admin Console
193         </a>
194     </div>
195 </div>
196
197 <!-- Main -->
198 <div class="main-content">
199     <div class="topbar">
200         <div class="page-title" id="pageTitle">Dashboard</div>
201         <div class="topbar-right">
202             <div class="user-pill">
203                 <div class="user-avatar" id="user-initials">--</div>
204                 <span class="user-name" id="username-display">Loading...</span>
205             </div>
206             <button class="logout-btn" onclick="logout()">Logout</button>
207         </div>
208     </div>
209
210     <div class="content">
211
212         <!-- DASHBOARD VIEW -->
213         <div id="dashboard-view" class="view active">
214             <div class="stats-grid">
215                 <div class="stat-card blue"><div class="stat-label">Total Requests</div><div
class="stat-value blue" id="s-total">--</div></div>
216                 <div class="stat-card amber"><div class="stat-label">Pending</div><div class="stat-
value amber" id="s-pending">--</div></div>
217                 <div class="stat-card green"><div class="stat-label">Approved</div><div class="stat-
value green" id="s-approved">--</div></div>
218                 <div class="stat-card red"><div class="stat-label">Rejected</div><div class="stat-value
red" id="s-rejected">--</div></div>
219             </div>
220             <div class="section-head">
221                 <div class="section-title">Recent Leave Requests</div>
222                 <button class="btn btn-secondary btn-sm" onclick="loadDashboard()"> Refresh</button>
223             </div>
224             <div class="table-wrap">
225                 <table>
226                     <thead><tr><th>Employee ID</th><th>Leave Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th></tr></thead>
227                     <tbody id="recent-leaves-body"></tbody>
228                 </table>
229             </div>
230         </div>
231
232         <!-- APPLY LEAVE VIEW -->
233         <div id="apply-view" class="view">
234             <div class="section-title" style="margin-bottom:20px;">Apply for Leave</div>
235             <div class="form-card">
236                 <div class="form-grid">
237                     <div class="form-group full">
238                         <label>Your Email</label>
239                         <input type="email" id="leave-email" readonly
placeholder="your.email@company.com">
240                     </div>
241                     <div class="form-group">
242                         <label>Leave Type</label>
243                         <select id="leave-type">
244                             <option value="SICK">Sick Leave</option>
245                             <option value="CASUAL">Casual Leave</option>
246                         </select>
247                         <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(249,199,79,0.2);border-r...
248                         Sick leave is limited to <strong>1 day per month</strong>. If already used

```

this month, please select Unpaid Leave.

```
249             </div>
250             <div id="casual-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--accent);background:var(--accent-dim);border:1px solid rgba(79,140,255,0.2);bord...
251                 You have <strong id="casual-balance-text">...</strong> casual leave days
available.
252             </div>
253         </div>
254         <div class="form-group">
255             <label>Priority</label>
256             <select id="leave-priority" style="color:var(--text2);">
257                 <option value="">— Optional —</option>
258                 <option value="NORMAL">Normal</option>
259                 <option value="URGENT">Urgent</option>
260             </select>
261         </div>
262         <div class="form-group">
263             <label>Start Date</label>
264             <input type="date" id="leave-start">
265         </div>
266         <div class="form-group">
267             <label>End Date</label>
268             <input type="date" id="leave-end">
269         </div>
270         <div class="form-group full">
271             <label>Reason</label>
272             <textarea id="leave-reason" rows="3" placeholder="Please provide a brief reason
for your leave request..."></textarea>
273         </div>
274     </div>
275     <div style="display:flex;gap:12px;margin-top:20px;">
276         <button class="btn btn-primary" onclick="applyLeave()">Submit Leave
Request</button>
277         <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
278     </div>
279 </div>
280 </div>
281
282 <!-- MY LEAVES VIEW -->
283 <div id="my-leaves-view" class="view">
284     <div class="section-head">
285         <div class="section-title">My Leave Requests</div>
286         <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
287     </div>
288     <div class="table-wrap">
289         <table>
290             <thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead>
291             <tbody id="my-leaves-list"></tbody>
292         </table>
293     </div>
294 </div>
295
296 <!-- LEAVE BALANCE VIEW -->
297 <div id="balance-view" class="view">
298     <div class="section-head">
299         <div class="section-title">Leave Balance</div>
300         <button class="btn btn-secondary btn-sm" onclick="loadBalance()"> Refresh</button>
301     </div>
302     <div class="balance-grid" id="balance-grid">
303         <div class="balance-card" style="color:var(--text2);text-
align:center;padding:40px;">Loading balance...</div>
304     </div>
305     <div class="section-head" style="margin-top:8px;">
306         <div class="section-title">Leave History</div>
307     </div>
308     <div class="table-wrap">
309         <table>
310             <thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th><th>Remarks</th></tr></thead>
311             <tbody id="balance-history-list"></tbody>
312         </table>
313     </div>
314 </div>
315
316 <!-- PENDING APPROVALS VIEW -->
```

```

317         <div id="pending-view" class="view">
318             <div class="section-head">
319                 <div class="section-title">Pending Approvals</div>
320                 <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">
Refresh</button>
321             </div>
322             <div class="table-wrap">
323                 <table>
324                     <thead><tr><th>ID</th><th>Employee
ID</th><th>Type</th><th>Start</th><th>End</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
325                     <tbody id="pending-leaves-list"></tbody>
326                 </table>
327             </div>
328         </div>
329
330         <!-- ALL LEAVES VIEW -->
331         <div id="all-leaves-view" class="view">
332             <div class="section-head">
333                 <div class="section-title">All Leave Requests</div>
334                 <div class="section-actions">
335                     <select id="status-filter" onchange="filterAllLeaves()" style="padding:6px
10px;font-size:12px;border-radius:6px;">
336                         <option value="">All Statuses</option>
337                         <option value="PENDING">Pending</option>
338                         <option value="APPROVED">Approved</option>
339                         <option value="REJECTED">Rejected</option>
340                         <option value="CANCELLED">Cancelled</option>
341                     </select>
342                     <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">
Refresh</button>
343                 </div>
344             </div>
345             <div class="table-wrap">
346                 <table>
347                     <thead><tr><th>ID</th><th>Employee
ID</th><th>Type</th><th>Start</th><th>End</th><th>Days</th><th>Status</th></tr></thead>
348                     <tbody id="all-leaves-list"></tbody>
349                 </table>
350             </div>
351         </div>
352
353     </div><!-- /content -->
354 </div><!-- /main-content -->
355 </div><!-- /app -->
356
357 <!-- Approval Modal -->
358 <div class="modal-overlay" id="action-modal">
359     <div class="modal">
360         <div class="modal-title" id="action-modal-title">Review Leave Request</div>
361         <input type="hidden" id="action-leave-id">
362         <input type="hidden" id="action-type">
363         <div class="form-group" style="margin-bottom:12px;">
364             <label>Remarks (optional)</label>
365             <textarea id="action-remarks" rows="3" placeholder="Add remarks for the
employee..."></textarea>
366         </div>
367         <div class="modal-actions">
368             <button class="btn btn-secondary" onclick="closeModal('action-modal')">Cancel</button>
369             <button class="btn btn-primary" id="action-submit-btn"
onclick="submitApproval()">Approve</button>
370         </div>
371     </div>
372 </div>
373
374 <!-- Cancel Modal -->
375 <div class="modal-overlay" id="cancel-modal">
376     <div class="modal">
377         <div class="modal-title">Cancel Leave Request</div>
378         <p style="color:var(--text2);font-size:13px;margin-bottom:16px;">Are you sure you want to cancel
this leave request? This action cannot be undone.</p>
379         <input type="hidden" id="cancel-leave-id">
380         <div class="form-group" style="margin-bottom:12px;">
381             <label>Reason for cancellation (optional)</label>
382             <textarea id="cancel-remarks" rows="2" placeholder="Reason..."></textarea>
383         </div>
384         <div class="modal-actions">

```

```

385         <button class="btn btn-secondary" onclick="closeModal('cancel-modal')">Keep Request</button>
386         <button class="btn btn-danger" onclick="confirmCancel()">Yes, Cancel</button>
387     </div>
388 </div>
389 </div>
390
391 <script>
392     const API = '';
393     let currentUser = null;
394     let allLeavesCache = [];
395
396     // Auth
397     function getAuthHeader() {
398         const token = localStorage.getItem('jwt_token');
399         return token ? { 'Authorization': `Bearer ${token}` } : {};
400     }
401
402     async function apiFetch(path, opts = {}) {
403         const response = await fetch(`${API}${path}`, {
404             ...opts,
405             headers: { 'Content-Type': 'application/json', ...getAuthHeader(), ...opts.headers },
406             credentials: 'include'
407         });
408         if (response.status === 401) {
409             localStorage.removeItem('jwt_token');
410             localStorage.removeItem('auth_type');
411             showAlert('Session expired. Please login again.', 'error');
412             setTimeout(() => window.location.href = '/google.html', 1500);
413             throw new Error('Unauthorized');
414         }
415         if (!response.ok) {
416             const text = await response.text();
417             let msg = `Error ${response.status}`;
418             try {
419                 const body = JSON.parse(text);
420                 // handles: {error:...}, {message:...}, Spring's default {error:..., message:...}
421                 msg = body.error || body.message || body.detail || text || msg;
422             } catch(_) {
423                 msg = text || msg;
424             }
425             throw new Error(msg);
426         }
427         const text = await response.text();
428         return text ? JSON.parse(text) : {};
429     }
430
431     function logout() {
432         localStorage.removeItem('jwt_token');
433         localStorage.removeItem('auth_type');
434         window.location.href = '/google.html';
435     }
436
437     (async function initAuth() {
438         const token = localStorage.getItem('jwt_token');
439         const authType = localStorage.getItem('auth_type');
440         if (!token && authType !== 'session') { window.location.href = '/google.html'; return; }
441         try {
442             const employee = await apiFetch('/employee/me');
443             currentUser = employee;
444             const initials = (employee.name || employee.email || 'U').split(' ').map(w =>
w[0]).join('').substring(0,2).toUpperCase();
445             document.getElementById('user-initials').textContent = initials;
446             document.getElementById('username-display').textContent = employee.name || employee.email ||
'User';
447             document.getElementById('leave-email').value = employee.email || '';
448             loadDashboard();
449             loadPendingCount();
450         } catch(e) {
451             if (e.message !== 'Unauthorized') window.location.href = '/google.html';
452         }
453     })();
454
455     // Navigation
456     const viewTitles = {
457         dashboard: 'Leave Dashboard',
458         apply: 'Apply for Leave',

```

```

459     'my-leaves': 'My Leave Requests',
460     balance: 'Leave Balance',
461     pending: 'Pending Approvals',
462     'all-leaves': 'All Leave Requests'
463   };
464   const viewLoaders = {
465     dashboard: loadDashboard,
466     apply: () => {},
467     'my-leaves': loadMyLeaves,
468     balance: loadBalance,
469     pending: loadPendingLeaves,
470     'all-leaves': loadAllLeaves
471   };
472
473   function showView(name, el) {
474     document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
475     document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
476     document.getElementById(name + '-view').classList.add('active');
477     if (el) el.classList.add('active');
478     document.getElementById('pageTitle').textContent = viewTitles[name] || name;
479     if (viewLoaders[name]) viewLoaders[name]();
480   }
481
482   // Helpers
483   function daysBetween(start, end) {
484     if (!start || !end) return '-';
485     const d = (new Date(end) - new Date(start)) / 86400000 + 1;
486     return d > 0 ? d : '-';
487   }
488
489   function chip(val) {
490     if (!val) return '-';
491     const cls = val.toLowerCase();
492     return `${val}</span>`;
493   }
494
495   function showAlert(msg, type = 'success') {
496     const el = document.createElement('div');
497     el.className = `alert alert-${type}`;
498     el.textContent = msg;
499     document.body.appendChild(el);
500     setTimeout(() => el.remove(), 3000);
501   }
502
503   function closeModal(id) {
504     document.getElementById(id).classList.remove('open');
505   }
506
507   document.querySelectorAll('.modal-overlay').forEach(m => {
508     m.addEventListener('click', e => { if (e.target === m) m.classList.remove('open'); });
509   });
510
511   // Dashboard
512   async function loadDashboard() {
513     try {
514       const [all, mine] = await Promise.all([
515         apiFetch('/leave_requests').catch(() => []),
516         apiFetch('/leave_requests/employee').catch(() => ({ requests: [] }))
517       ]);
518       const leaves = Array.isArray(all) ? all : [];
519       document.getElementById('s-total').textContent = leaves.length;
520       document.getElementById('s-pending').textContent = leaves.filter(l => l.status ===
521 'PENDING').length;
521       document.getElementById('s-approved').textContent = leaves.filter(l => l.status ===
522 'APPROVED').length;
522       document.getElementById('s-rejected').textContent = leaves.filter(l => l.status ===
523 'REJECTED').length;
523       const recent = leaves.slice(-8).reverse();
524       const tbody = document.getElementById('recent-leaves-body');
525       tbody.innerHTML = recent.length
526         ? recent.map(l => `<tr>
527 <td>${l.employeeName || '-'}</td>
528 <td>${chip(l.leaveType)}</td>
529 <td>${l.startDate || '-'}</td>
530 <td>${l.endDate || '-'}</td>
531 <td>${daysBetween(l.startDate, l.endDate)}</td>

```

```

532         <td>${chip(l.status)}</td>
533     </tr>`).join('')
534     : `<tr><td colspan="6" class="empty-state">No leave requests yet</td></tr>`;
535 } catch(e) { showAlert('Failed to load dashboard: ' + e.message, 'error'); }
536 }
537
538 async function loadPendingCount() {
539     try {
540         const leaves = await apiFetch('/admin/leaves/pending').catch(() => []);
541         const count = Array.isArray(leaves) ? leaves.filter(l => l.status === 'PENDING').length : 0;
542         const el = document.getElementById('pending-count');
543         if (count > 0) { el.textContent = count; el.style.display = 'inline'; }
544         else el.style.display = 'none';
545     } catch(e) {}
546 }
547
548 // Apply Leave
549 async function applyLeave() {
550     const leaveType = document.getElementById('leave-type').value;
551     const startDate = document.getElementById('leave-start').value;
552     const endDate = document.getElementById('leave-end').value;
553     const reason = document.getElementById('leave-reason').value.trim();
554     if (!startDate || !endDate) { showAlert('Please select both start and end dates.', 'error'); return; }
555     if (new Date(endDate) < new Date(startDate)) { showAlert('End date cannot be before start date.',
556 'error'); return; }
557     try {
558         const data = await apiFetch('/leave_requests', {
559             method: 'POST',
560             body: JSON.stringify({ leaveType, startDate, endDate, reason })
561         });
562         if(data && data.warning) {
563             showAlert(data.warning, 'warning');
564         } else {
565             showAlert('Leave request submitted successfully!', 'success');
566             clearLeaveForm();
567             loadPendingCount();
568         } catch(e) {
569             let msg = e.message;
570             try {
571                 const parsed = JSON.parse(msg);
572                 if(parsed.error) msg = parsed.error;
573             } catch(_) {}
574             showAlert(msg, 'error');
575         }
576     }
577
578     function clearLeaveForm() {
579         document.getElementById('leave-start').value = '';
580         document.getElementById('leave-end').value = '';
581         document.getElementById('leave-reason').value = '';
582     }
583
584 // My Leaves
585 async function loadMyLeaves() {
586     try {
587         const data = await apiFetch('/leave_requests/employee');
588         const leaves = data.requests || (Array.isArray(data) ? data : []);
589         const tbody = document.getElementById('my-leaves-list');
590         tbody.innerHTML = leaves.length
591             ? leaves.map(l => `<tr>
592 <td>${l.leaveRequestId}</td>
593 <td>${chip(l.leaveType)}</td>
594 <td>${l.startDate || '-'}</td>
595 <td>${l.endDate || '-'}</td>
596 <td>${daysBetween(l.startDate, l.endDate)}</td>
597 <td>${chip(l.status)}</td>
598 <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;"
599 title="${l.reason|| ''}">${l.reason || '-'}</td>
600 <td>
601     ${l.status === 'PENDING'
602         ? `<button class="btn btn-danger btn-sm"
603 onclick="openCancelModal(${l.leaveRequestId})">Cancel</button>`
604         : '-'}
605     </td>
606 </tr>`).join('')

```

```

605         : `<tr><td colspan="8" class="empty-state">No leave requests found</td></tr>`;
606     } catch(e) { showAlert('Failed to load your leaves: ' + e.message, 'error'); }
607 }
608
609 function openCancelModal(id) {
610     document.getElementById('cancel-leave-id').value = id;
611     document.getElementById('cancel-remarks').value = '';
612     document.getElementById('cancel-modal').classList.add('open');
613 }
614
615 async function confirmCancel() {
616     const id = document.getElementById('cancel-leave-id').value;
617     try {
618         await apiFetch(`/leave_requests/cancel/${id}`, { method: 'PUT' });
619         showAlert('Leave request cancelled.', 'success');
620         closeModal('cancel-modal');
621         loadMyLeaves();
622         loadPendingCount();
623     } catch(e) { showAlert('Failed to cancel: ' + e.message, 'error'); }
624 }
625
626 // Leave Balance
627 const balanceColors = {
628     SICK: '#f87171', CASUAL: '#4f8cff', MATERNITY: '#a78bfa', UNPAID: '#8b92a8'
629 };
630
631 function normalizeDashboardBalances(data) {
632     if (!data) return [];
633     if (data.leaveBalances) {
634         return Object.values(data.leaveBalances).map(balance => ({
635             type: balance.leaveType,
636             opening: Number(balance.opening ?? 0),
637             accrued: Number(balance.accrued ?? 0),
638             used: Number(balance.used ?? 0),
639             available: Number(balance.available ?? 0)
640         }));
641     }
642     return ['sick', 'casual', 'maternity']
643         .map(key => data[key])
644         .filter(Boolean)
645         .map(balance => ({
646             type: balance.leaveType,
647             opening: Number(balance.opening ?? 0),
648             accrued: Number(balance.accrued ?? 0),
649             used: Number(balance.used ?? 0),
650             available: Number(balance.available ?? 0)
651         }));
652 }
653
654 async function loadBalance() {
655     try {
656         const data = await apiFetch('/api/employees/me/leave-dashboard');
657         const grid = document.getElementById('balance-grid');
658         const balances = normalizeDashboardBalances(data);
659
660         if (balances.length) {
661             grid.innerHTML = balances.map(info => {
662                 const type = info.type;
663                 const used = info.used;
664                 const total = info.opening + info.accrued;
665                 const left = info.available;
666                 const pct = total ? Math.min(100, Math.round((used / total) * 100)) : 0;
667                 const color = balanceColors[type] || '#4f8cff';
668                 return `<div class="balance-card">
669                     <div class="balance-name">${type.replace(/_/g, ' ')}</div>
670                     <div class="balance-row">
671                         <span class="balance-big" style="color:${color}">${left}</span>
672                         <span class="balance-of"> / ${total} days left</span>
673                     </div>
674                     <div class="balance-bar"><div class="balance-fill"
675 style="width:${pct}%;background:${color}"></div></div>
676                 </div>`;
677             }).join('');
678         } else {
679             grid.innerHTML = `<div class="balance-card" style="color:var(--text2);grid-column:1/-
1;text-align:center;padding:32px;">

```

```

679         Leave balance data not available from the API. Check <code>/api/employees/me/leave-dashboard</code>
endpoint.
680     </div>`;
681     }
682
683     const myData = await apiFetch('/leave_requests/employee');
684     const myLeaves = myData.requests || (Array.isArray(myData) ? myData : []);
685     const histTbody = document.getElementById('balance-history-list');
686     histTbody.innerHTML = myLeaves.length
687         ? myLeaves.map(l => `<tr>
688             <td>${l.leaveRequestId}</td>
689             <td>${chip(l.leaveType)}</td>
690             <td>${l.startDate || '-'}</td>
691             <td>${l.endDate || '-'}</td>
692             <td>${daysBetween(l.startDate, l.endDate)}</td>
693             <td>${chip(l.status)}</td>
694             <td style="font-size:12px;color:var(--text2)">${l.managerRemarks || l.remarks || '-'}</td>
695         </tr>`).join('')
696         : `<tr><td colspan="7" class="empty-state">No leave history</td></tr>`;
697     } catch(e) { showAlert('Failed to load balance: ' + e.message, 'error'); }
698 }
699
700 // Pending Approvals
701 async function loadPendingLeaves() {
702     try {
703         const leaves = await apiFetch('/admin/leaves/pending');
704         const arr = Array.isArray(leaves) ? leaves : [];
705         const count = arr.filter(l => l.status === 'PENDING').length;
706         const el = document.getElementById('pending-count');
707         if (count > 0) { el.textContent = count; el.style.display = 'inline'; }
708         else el.style.display = 'none';
709         const tbody = document.getElementById('pending-leaves-list');
710         tbody.innerHTML = arr.length
711             ? arr.map(l => `<tr>
712                 <td>${l.leaveRequestId}</td>
713                 <td>${l.employeeName || '-'}</td>
714                 <td>${chip(l.leaveType)}</td>
715                 <td>${l.startDate || '-'}</td>
716                 <td>${l.endDate || '-'}</td>
717                 <td>${daysBetween(l.startDate, l.endDate)}</td>
718                 <td style="max-width:140px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;"
719                 title="${l.reason||''}">${l.reason || '-'}</td>
720                 <td class="action-row">
721                     <button class="btn btn-success btn-sm"
722                     onclick="openActionModal(${l.leaveRequestId},'APPROVED')>Approve</button>
723                     <button class="btn btn-danger btn-sm"
724                     onclick="openActionModal(${l.leaveRequestId},'REJECTED')>Reject</button>
725                 </td>
726             </tr>`).join('')
727             : `<tr><td colspan="8" class="empty-state">No pending leave requests</td></tr>`;
728     } catch(e) { showAlert('Failed to load pending: ' + e.message, 'error'); }
729 }
730
731 function openActionModal(id, action) {
732     document.getElementById('action-leave-id').value = id;
733     document.getElementById('action-type').value = action;
734     document.getElementById('action-remarks').value = '';
735     document.getElementById('action-modal-title').textContent = `${action === 'APPROVED' ? 'Approve' :
'Reject'} Leave Request #${id}`;
736     const btn = document.getElementById('action-submit-btn');
737     btn.textContent = action === 'APPROVED' ?
738     'Approve' : 'Reject';
739     btn.className = action === 'APPROVED' ? 'btn btn-success' : 'btn btn-danger';
740     document.getElementById('action-modal').classList.add('open');
741 }
742
743 async function submitApproval() {
744     const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
745     const action = document.getElementById('action-type').value;
746     const remarks = document.getElementById('action-remarks').value;
747     try {
748         await apiFetch('/leave_requests/approval', {
749             method: 'PUT',
750             body: JSON.stringify({ leaveRequestId, action, remarks })
751         });
752     }

```



```

750         showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
751         closeModal('action-modal');
752         loadPendingLeaves();
753         loadDashboard();
754     } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
755 }
756
757 // All Leaves
758 async function loadAllLeaves() {
759     try {
760         const leaves = await apiFetch('/leave_requests');
761         allLeavesCache = Array.isArray(leaves) ? leaves : [];
762         renderAllLeaves(allLeavesCache);
763     } catch(e) { showAlert('Failed to load all leaves: ' + e.message, 'error'); }
764 }
765
766 function filterAllLeaves() {
767     const status = document.getElementById('status-filter').value;
768     const filtered = status ? allLeavesCache.filter(l => l.status === status) : allLeavesCache;
769     renderAllLeaves(filtered);
770 }
771
772 function renderAllLeaves(leaves) {
773     const tbody = document.getElementById('all-leaves-list');
774     tbody.innerHTML = leaves.length
775         ? leaves.map(l => `<tr>
776             <td>${l.leaveRequestId}</td>
777             <td>${l.employeeName || '-'}</td>
778             <td>${chip(l.leaveType)}</td>
779             <td>${l.startDate || '-'}</td>
780             <td>${l.endDate || '-'}</td>
781             <td>${daysBetween(l.startDate, l.endDate)}</td>
782             <td>${chip(l.status)}</td>
783         </tr>`).join('')
784       : `<tr><td colspan="8" class="empty-state">No leave requests found</td></tr>`;
785 }
786
787 document.getElementById('leave-type').addEventListener('change', async function() {
788     document.getElementById('sick-leave-hint').style.display =
789         this.value === 'SICK' ? 'block' : 'none';
790
791     const casualHint = document.getElementById('casual-leave-hint');
792     if(this.value === 'CASUAL') {
793         casualHint.style.display = 'block';
794         document.getElementById('casual-balance-text').textContent = 'loading...';
795         try {
796             const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
797             const balances = dashboard?.leaveBalances || {};
798             const cas = balances['CASUAL'] || balances['casual'] || {};
799             const available = Number(cas.available ?? cas.availableBalance ?? 0);
800             document.getElementById('casual-balance-text').textContent =
801                 available > 0 ? `${available}` : '0 (no balance remaining)';
802         } catch(e) {
803             document.getElementById('casual-balance-text').textContent = 'unavailable';
804         }
805     } else {
806         casualHint.style.display = 'none';
807     }
808 });
809 </script>
810 </body>
811 </html>
812

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Manager Console</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700&family=IBM+Plex+Mono:w
ght@400;500&display=swap" rel="stylesheet">
8      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
9      <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
10     <style>
11         *{margin:0;padding:0;box-sizing:border-box}
12         :root{
13             --bg:#0a0b0f;--bg2:#0f1117;--bg3:#14171f;
14             --panel:#171c28;--panel2:#1d2336;
15             --border:rgba(167,139,250,0.08);--border2:rgba(167,139,250,0.18);
16             --text:#e8e0f5;--text2:#8b82a8;--text3:#504d68;
17             --accent:#a78bfa;--accent-dim:rgba(167,139,250,0.1);
18             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
19             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
20             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
21             --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
22             --font:'Plus Jakarta Sans',sans-serif;--mono:'IBM Plex Mono',monospace;
23             --sidebar:250px;
24         }
25         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
26         .app{display:flex;height:100vh}
27
28         /* SIDEBAR */
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;position:relative;overflow:hidden}
30         .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:200px;background:linear-
gradient(to top,rgba(167,139,250,0.04),transparent);pointer-events:none}
31         .logo{padding:22px 20px;border-bottom:1px solid var(--border);position:relative}
32         .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
33         .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,var(--accent),#7c3aed);border-
radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
34         .logo h2{font-size:16px;font-weight:700;color:var(--text)}
35         .logo p{font-size:11px;color:var(--text3);font-family:var(--mono)}
36         .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:linear-
gradient(90deg,rgba(167,139,250,0.15),rgba(124,58,237,0.08));border:1px solid rgba(167,139,250,...
37         .role-dot{width:6px;height:6px;background:var(--accent);border-radius:50%;animation:glow 2s
infinite}
38         @keyframes glow{0%,100%{box-shadow:0 0 4px var(--accent)}50%{box-shadow:0 0 10px var(--accent)},0 0
20px rgba(167,139,250,0.4)}}
39         .nav{flex:1;padding:10px 0;overflow-y:auto}
40         .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2px;text-transform:uppercase}
41         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-
radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
42         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
43         .nav-item.active{background:linear-gradient(90deg,var(--accent-
dim),rgba(124,58,237,0.06));color:var(--accent);border:1px solid rgba(167,139,250,0.2)}
44         .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:10px;font-
weight:700;padding:2px 7px;border-radius:10px;min-width:18px;text-align:center}
45         .nav-badge.hidden{display:none}
46         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
47         .user-card{display:flex;align-items:center;gap:10px}
48         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#7c3aed,var(--accent));border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
49         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
50         .user-info-side .role-text{font-size:10px;color:var(--accent);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px}
51
52         /* MAIN */
53         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
54         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
55         .page-title{font-size:16px;font-weight:700;color:var(--text)}
56         .topbar-right{display:flex;align-items:center;gap:12px}

```

```

57     .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
58     .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
59     .content{flex:1;overflow-y:auto;padding:24px}
60     .view{display:none}
61     .view.active{display:block;animation:fadeIn 0.3s ease}
62     @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
63     .notif-wrapper { position: relative; cursor: pointer; }
64
65     .notif-badge {
66         position: absolute; top: -5px; right: -6px;
67         background: #e53e3e; color: white;
68         font-size: 10px; font-weight: 600;
69         min-width: 16px; height: 16px;
70         border-radius: 50%; display: flex;
71         align-items: center; justify-content: center;
72         padding: 0 3px;
73     }
74
75     .notif-panel {
76         position: absolute; top: 48px; right: 16px;
77         width: 320px; max-height: 400px; overflow-y: auto;
78         background: var(--panel); border: 1px solid #e2e8f0;
79         border-radius: 10px; box-shadow: 0 4px 16px rgba(0,0,0,0.1);
80         z-index: 999;
81     }
82
83     .notif-header {
84         padding: 12px 16px; font-weight: 600;
85         border-bottom: 1px solid #e2e8f0;
86     }
87
88     .notif-item {
89         padding: 12px 16px; font-size: 14px;
90         border-bottom: 1px solid #f1f5f9;
91     }
92
93     .notif-item.unread { background: #eff6ff; }
94     .notif-time { font-size: 11px; color: #94a3b8; margin-top: 4px; }
95
96     /* STATS */
97     .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
98     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:14px;padding:20px;position:relative;overflow:hidden;transition:all 0.2s;cursor:default}
99     .stat-card:hover{border-color:var(--border2);transform:translateY(-1px)}
100     .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:3px}
101     .stat-card.purple::before{background:linear-gradient(90deg,#7c3aed,var(--accent))}
102     .stat-card.green::before{background:var(--green)}
103     .stat-card.amber::before{background:var(--amber)}
104     .stat-card.blue::before{background:var(--blue)}
105     .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;margin-
bottom:10px;font-weight:600}
106     .stat-value{font-size:30px;font-weight:700;color:var(--text);font-family:var(--mono);line-height:1}
107     .stat-sub{font-size:11px;color:var(--text3);margin-top:6px}
108
109     /* SECTION */
110     .section{background:var(--panel);border:1px solid var(--border);border-radius:14px;margin-
bottom:20px;overflow:hidden}
111     .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
112     .section-title{font-size:14px;font-weight:700;color:var(--text)}
113     .section-sub{font-size:12px;color:var(--text3);margin-top:2px}
114
115     /* TABLE */
116     .tablewrap{overflow-x:auto}
117     table{width:100%;border-collapse:collapse}
118     th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-
spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border)}
119     td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
120     tr:last-child td{border-bottom:none}
121     tr:hover td{background:rgba(167,139,250,0.03);color:var(--text)}
122
123     /* CHIPS */
124     .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600}

```

```

125     .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
126     .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
127     .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
128     .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
129     .chip.open { background: var(--accent-dim); color: var(--accent); }
130     .chip.sick{background:var(--red-dim);color:var(--red)}
131     .chip.casual{background:var(--amber-dim);color:var(--amber)}
132     .chip.maternity{background:var(--accent-dim);color:var(--accent)}
133     .chip.employee{background:var(--blue-dim);color:var(--blue)}
134     .chip.manager{background:var(--accent-dim);color:var(--accent)}
135     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
136     .chip.unpaid { background: rgba(139,92,246,0.15); color: #a78bfa; border: 1px solid
rgba(139,92,246,0.2); }
137     .chip.assigned { background: var(--green-dim); color: var(--green);}
138     .chip.condemned { background: var(--red-dim); color: var(--red);}
139
140     /* BUTTONS */
141     .btn{padding:8px 16px;border-radius:8px;border:none;font-family:var(--font);font-size:12px;font-
weight:600;cursor:pointer;transition:all 0.2s}
142     .btn-primary{background:var(--accent);color:#fff}
143     .btn-primary:hover{background:#7c3aed;box-shadow:0 4px 12px rgba(167,139,250,0.3)}
144     .btn-approve{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
145     .btn-approve:hover{background:var(--green);color:#fff}
146     .btn-reject{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
147     .btn-reject:hover{background:var(--red);color:#fff}
148     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
149     .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
150     .btn-sm{padding:5px 12px;font-size:11px}
151     .action-row{display:flex;gap:6px}
152
153     /* FORM */
154     .form-card{padding:24px}
155     .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
156     .form-group{display:flex;flex-direction:column;gap:6px}
157     .form-group.full{grid-column:1/-1}
158     label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
159     input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
160     input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
161     input::placeholder{color:var(--text3)}
162     select option{background:var(--bg2)}
163     .form-actions{display:flex;gap:10px;margin-top:20px}
164
165     /* ALERT */
166     .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-
index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
167     .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.3)}
168     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
169     .alert-warning{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(251,191,36,0.3)}
170     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
171     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
172
173     /* GRID DASHBOARD */
174     .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:20px;margin-bottom:0}
175
176     /* SEARCH */
177     .search-bar{display:flex;align-items:center;gap:10px;margin-bottom:16px}
178     .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:9px 14px;color:var(--text);font-family:var(--font);font-size:13px}
179     .search-input:focus{outline:none;border-color:var(--accent)}
180     .search-input::placeholder{color:var(--text3)}
181
182     /* ===== */
183     /* CALENDAR VIEW STYLES */
184     /* ===== */
185
186     /* Leave balance summary cards above the calendar */
187     .leave-balance-grid {
188         display: grid;
189         grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
190         gap: 14px;
191         padding: 20px;

```

```

192         border-bottom: 1px solid var(--border);
193     }
194     .leave-bal-card {
195         background: var(--bg3);
196         border: 1px solid var(--border);
197         border-radius: 10px;
198         padding: 16px 18px;
199         display: flex;
200         flex-direction: column;
201         gap: 6px;
202         position: relative;
203         overflow: hidden;
204         transition: border-color 0.2s;
205     }
206     .leave-bal-card:hover { border-color: var(--border2); }
207     .leave-bal-card::before {
208         content: '';
209         position: absolute;
210         left: 0; top: 0; bottom: 0;
211         width: 3px;
212         border-radius: 3px 0 0 3px;
213     }
214     .leave-bal-card.sick::before { background: var(--red); }
215     .leave-bal-card.casual::before { background: var(--amber); }
216     .leave-bal-card.mat::before { background: var(--accent); }
217     .leave-bal-card.total::before { background: var(--green); }
218     .lbc-icon { font-size: 18px; }
219     .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }
220     .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-
height: 1; }
221     .lbc-sub { font-size: 11px; color: var(--text3); }
222
223     /* Legend row */
224     .cal-legend {
225         display: flex;
226         align-items: center;
227         gap: 20px;
228         padding: 12px 20px;
229         border-bottom: 1px solid var(--border);
230         flex-wrap: wrap;
231     }
232     .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase;
letter-spacing: 0.8px; margin-right: 4px; }
233     .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2);
}
234     .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
235
236     /* FullCalendar dark-theme overrides */
237     #employee-calendar {
238         padding: 16px 20px 20px;
239     }
240     /* Toolbar */
241     #employee-calendar .fc .fc-toolbar-title {
242         font-family: var(--font);
243         font-size: 15px;
244         font-weight: 600;
245         color: var(--text);
246     }
247     #employee-calendar .fc .fc-button {
248         background: var(--panel2);
249         border: 1px solid var(--border2);
250         color: var(--text2);
251         font-family: var(--font);
252         font-size: 12px;
253         font-weight: 600;
254         border-radius: 6px;
255         padding: 5px 12px;
256         transition: all 0.2s;
257         box-shadow: none;
258         text-transform: capitalize;
259     }
260     #employee-calendar .fc .fc-button:hover,
261     #employee-calendar .fc .fc-button-active,
262     #employee-calendar .fc .fc-button:not(:disabled):active {
263         background: var(--accent-dim);

```

```

264         border-color: var(--accent);
265         color: var(--accent);
266         box-shadow: none;
267     }
268     #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
269
270     /* Grid */
271     #employee-calendar .fc-theme-standard td,
272     #employee-calendar .fc-theme-standard th,
273     #employee-calendar .fc-theme-standard .fc-scrollgrid {
274         border-color: var(--border);
275     }
276     #employee-calendar .fc .fc-col-header-cell-cushion {
277         font-family: var(--font);
278         font-size: 11px;
279         font-weight: 600;
280         color: var(--text3);
281         text-transform: uppercase;
282         letter-spacing: 0.8px;
283         padding: 8px 4px;
284     }
285     #employee-calendar .fc .fc-daygrid-day-number {
286         font-family: var(--mono);
287         font-size: 12px;
288         color: var(--text3);
289         padding: 6px 8px;
290     }
291     #employee-calendar .fc .fc-daygrid-day.fc-day-today {
292         background: var(--accent-dim) !important;
293     }
294     #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number {
295         color: var(--accent);
296         font-weight: 700;
297     }
298     #employee-calendar .fc .fc-daygrid-day:hover {
299         background: rgba(56,189,248,0.03);
300     }
301     #employee-calendar .fc-daygrid-day-frame {
302         background: var(--bg3);
303     }
304     #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame {
305         background: var(--bg2);
306     }
307
308     /* Events */
309     #employee-calendar .fc-event {
310         border: none !important;
311         border-radius: 4px;
312         font-family: var(--font);
313         font-size: 11px;
314         font-weight: 600;
315         padding: 2px 6px;
316         cursor: pointer;
317     }
318     #employee-calendar .fc-event-title { font-weight: 600; }
319
320     /* Event tooltip */
321     .cal-tooltip {
322         position: fixed;
323         background: var(--panel2);
324         border: 1px solid var(--border2);
325         border-radius: 8px;
326         padding: 10px 14px;
327         font-size: 12px;
328         color: var(--text);
329         z-index: 9999;
330         pointer-events: none;
331         max-width: 200px;
332         box-shadow: 0 8px 24px rgba(0,0,0,0.4);
333     }
334     .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
335     .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
336
337     /* balance loading state */
338     .lbc-count.loading { color: var(--text3); font-size: 16px; }
339 </style>

```

```

340 </head>
341 <body>
342 <div class="app">
343     <!-- SIDEBAR -->
344     <div class="sidebar">
345         <div class="logo">
346             <div class="logo-row">
347                 <div class="logo-icon">M</div>
348                 <h2>EMS Console</h2>
349             </div>
350             <p>Management Portal</p>
351             <div class="role-badge"><span class="role-dot"></span>Manager</div>
352         </div>
353         <nav class="nav">
354             <div class="nav-section">Overview</div>
355             <div class="nav-item active" onclick="showView('dashboard',this)">
356                 <span></span> Dashboard
357             </div>
358
359             <div class="nav-section">Team</div>
360             <div class="nav-item" onclick="showView('employees',this)">
361                 <span></span> All Employees
362             </div>
363             <div class="nav-item" onclick="showView('add-employee',this)">
364                 <span></span> Add Employee
365             </div>
366             <div class="nav-item" onclick="showView('promote-users',this)">
367                 <span></span> Promote Users
368                 <span class="nav-badge" id="users-badge">0</span>
369             </div>
370             <div class="nav-item" onclick="showView('search-employee',this)">
371                 <span></span> Search Employee
372             </div>
373
374             <div class="nav-section">Leave Management</div>
375             <div class="nav-item" onclick="showView('pending-leaves',this)">
376                 <span></span> Pending Approvals
377                 <span class="nav-badge" id="pending-badge">0</span>
378             </div>
379             <div class="nav-item" onclick="showView('all-leaves',this)">
380                 <span></span> All Leave Requests
381             </div>
382
383             <div class="nav-section">Personal</div>
384             <div class="nav-item" onclick="showView('profile',this)">
385                 <span></span> My Profile
386             </div>
387             <div class="nav-item" onclick="showView('apply-leave',this)">
388                 <span></span> Apply Leave
389             </div>
390             <div class="nav-item" onclick="showView('my-leaves',this)">
391                 <span></span> My Leaves
392             </div>
393             <div class="nav-item" onclick="showView('my-calendar',this)">
394                 <span></span> My Calendar
395             </div>
396
397             <div class="nav-section">Devices</div>
398             <div class="nav-item" onclick="showView('my-devices',this)">
399                 <span></span> My Devices
400             </div>
401             <div class="nav-item" onclick="showView('service-requests', this)">
402                 <span></span> Service Requests
403             </div>
404
405             <div class="nav-section">Meal Services</div>
406             <div class="nav-item" onclick="showView('my-subscriptions',this)">
407                 <span></span> My Subscriptions
408             </div>
409             <div class="nav-item" onclick="showView('add-subscription',this)">
410                 <span></span> Subscribe to Meal
411             </div>
412             <div class="nav-item" onclick="showView('restaurants',this)">
413                 <span></span> Restaurants
414             </div>
415

```

```

416         <div class="nav-section">Security</div>
417         <div class="nav-item" onclick="showView('two-factor',this)">
418             <span></span> Two-Factor Auth
419         </div>
420     </nav>
421     <div class="sidebar-footer">
422         <div class="user-card">
423             <div class="avatar" id="avatar-initials">M</div>
424             <div class="user-info-side">
425                 <div class="name" id="sidebar-name">Manager</div>
426                 <div class="role-text">Manager</div>
427             </div>
428         </div>
429     </div>
430 </div>
431
432 <!-- MAIN -->
433 <div class="main">
434     <div class="topbar">
435         <div class="page-title" id="pageTitle">Dashboard</div>
436         <!-- Bell icon with badge -->
437         <div class="notif-wrapper" onclick="toggleNotifPanel()">
438             <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
439                 stroke="currentColor" stroke-width="2" stroke-linecap="round">
440                 <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
441                 <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
442             </svg>
443             <span id="notif-badge" class="notif-badge" style="display:none"></span>
444         </div>
445
446         <!-- Dropdown panel -->
447         <div id="notif-panel" class="notif-panel" style="display:none">
448             <div class="notif-header">Notifications</div>
449             <div id="notif-list"></div>
450         </div>
451         <div class="topbar-right">
452             <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
453             <button class="logout-btn" onclick="logout()">Sign Out</button>
454         </div>
455     </div>
456     <div class="content">
457
458         <!-- DASHBOARD -->
459         <div id="dashboard-view" class="view active">
460             <div class="stats-grid">
461                 <div class="stat-card purple">
462                     <div class="stat-label">Total Employees</div>
463                     <div class="stat-value" id="stat-total">--</div>
464                     <div class="stat-sub">in your organization</div>
465                 </div>
466                 <div class="stat-card green">
467                     <div class="stat-label">Active Now</div>
468                     <div class="stat-value" id="stat-active">--</div>
469                     <div class="stat-sub">currently working</div>
470                 </div>
471                 <div class="stat-card amber">
472                     <div class="stat-label">Pending Approvals</div>
473                     <div class="stat-value" id="stat-pending">--</div>
474                     <div class="stat-sub">need your action</div>
475                 </div>
476                 <div class="stat-card blue">
477                     <div class="stat-label">On Leave Today</div>
478                     <div class="stat-value" id="stat-on-leave">--</div>
479                     <div class="stat-sub">employees absent</div>
480                 </div>
481             </div>
482             <div class="dash-grid">
483                 <div class="section">
484                     <div class="section-head">
485                         <div><div class="section-title"> Needs Attention</div><div class="section-
sub">Pending leave requests</div></div>
486                         <button class="btn btn-secondary btn-sm" onclick="showView('pending-
leaves',null)">View All </button>
487                     </div>
488                 <div class="tablewrap">

```



```

489         <table>
490
491         <thead><tr><th>Employee</th><th>Type</th><th>Dates</th><th>Action</th></tr></thead>
492         <tbody id="dash-pending"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
493     </table>
494 </div>
495 <div class="section">
496     <div class="section-head">
497         <div><div class="section-title"> Team Overview</div><div class="section-
sub">Recent employees</div></div>
498         <button class="btn btn-secondary btn-sm"
onclick="showView('employees',null)">View All </button>
499     </div>
500     <div class="tablewrap">
501         <table>
502             <thead><tr><th>Name</th><th>Dept</th><th>Status</th></tr></thead>
503             <tbody id="dash-employees"><tr><td colspan="3" class="empty-
state">Loading...</td></tr></tbody>
504         </table>
505     </div>
506 </div>
507 </div>
508 </div>
509
510 <!-- ALL EMPLOYEES -->
511 <div id="employees-view" class="view">
512     <div class="section">
513         <div class="section-head">
514             <div><div class="section-title">All Employees</div><div class="section-sub">Manage
your team</div></div>
515             <button class="btn btn-secondary btn-sm" onclick="loadEmployees()">
Refresh</button>
516         </div>
517         <div style="padding:16px 20px;border-bottom:1px solid var(--border)">
518             <div class="search-bar">
519                 <input class="search-input" id="emp-search" placeholder="Search by name, email
or department..." oninput="filterEmployees()">
520             </div>
521         </div>
522         <div class="tablewrap">
523             <table>
524
525             <thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>Actions</th>
</tr></thead>
526             <tbody id="employees-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
527         </table>
528     </div>
529 </div>
530
531 <!-- ADD EMPLOYEE -->
532 <div id="add-employee-view" class="view">
533     <div class="section">
534         <div class="section-head"><div class="section-title">Add New Employee</div></div>
535         <div class="form-card">
536             <div class="form-grid">
537                 <div class="form-group"><label>Full Name</label><input id="emp-name"
placeholder="John Doe"></div>
538                 <div class="form-group"><label>Email</label><input type="email" id="emp-email"
placeholder="john@company.com"></div>
539                 <div class="form-group"><label>Password</label><input type="password" id="emp-
pass" placeholder="Temporary password"></div>
540                 <div class="form-group"><label>Gender</label>
541                 <select id="emp-gender">
542                     <option value="M">Male</option>
543                     <option value="F">Female</option>
544                     <option value="O">Other</option>
545                 </select>
546             </div>
547             <div class="form-group"><label>Timezone</label>
548             <select id="emp-timezone">
549                 <option value="UTC">UTC</option>
550                 <option value="Asia/Kolkata">Asia/Kolkata (IST)</option>

```

```

551             <option value="America/New_York">America/New_York (EST)</option>
552             <option value="America/Los_Angeles">America/Los_Angeles (PST)</option>
553             <option value="Europe/London">Europe/London (GMT)</option>
554             <option value="Asia/Singapore">Asia/Singapore (SGT)</option>
555         </select>
556     </div>
557 </div>
558     <div class="form-actions">
559         <button class="btn btn-primary" onclick="addEmployee()">Add Employee</button>
560         <button class="btn btn-secondary" onclick="clearEmpForm()">Clear</button>
561     </div>
562 </div>
563 </div>
564 </div>
565
566 <!-- SEARCH EMPLOYEE -->
567 <div id="search-employee-view" class="view">
568     <div class="section">
569         <div class="section-head">
570             <div><div class="section-title"> Search Employee</div><div class="section-sub">Look
up any employee across the organisation</div></div>
571         </div>
572         <div style="padding:20px;border-bottom:1px solid var(--border)">
573             <div style="display:flex;gap:10px">
574                 <input class="search-input" id="org-search-input" placeholder="Enter exact
employee name..."
575                     style="flex:1" onkeydown="if(event.key==='Enter')searchEmployee()">
576                 <button class="btn btn-primary" onclick="searchEmployee()">Search</button>
577             </div>
578             <div id="org-search-error" style="font-size:12px;color:var(--red);margin-
top:8px;display:none"></div>
579         </div>
580         <div id="org-search-results" style="padding:20px;display:none">
581
582         <!-- Employee Info Card -->
583         <div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-
bottom:20px">
584             <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px">
585                 <div style="font-size:10px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:12px">Employee Info</div>
586                 <div style="display:flex;flex-direction:column;gap:8px">
587                     <div style="display:flex;justify-content:space-between">
588                         <span style="font-size:12px;color:var(--text3)">Name</span>
589                         <span style="font-size:13px;font-weight:600;color:var(--text)"
id="sr-name">—</span>
590                     </div>
591                     <div style="display:flex;justify-content:space-between">
592                         <span style="font-size:12px;color:var(--text3)">ID</span>
593                         <span style="font-size:12px;font-family:var(--mono);color:var(--
accent)" id="sr-id">—</span>
594                     </div>
595                     <div style="display:flex;justify-content:space-between">
596                         <span style="font-size:12px;color:var(--text3)">Email</span>
597                         <span style="font-size:12px;font-family:var(--mono);color:var(--
text2)" id="sr-email">—</span>
598                     </div>
599                     <div style="display:flex;justify-content:space-between">
600                         <span style="font-size:12px;color:var(--text3)">Department</span>
601                         <span style="font-size:13px;color:var(--text)" id="sr-
dept">—</span>
602                     </div>
603                 </div>
604             </div>
605             <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px">
606                 <div style="font-size:10px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:12px">Status</div>
607                 <div style="display:flex;flex-direction:column;gap:8px">
608                     <div style="display:flex;justify-content:space-between;align-
items:center">
609                         <span style="font-size:12px;color:var(--text3)">Work Status</span>
610                         <span id="sr-status">—</span>
611                     </div>
612                     <div style="display:flex;justify-content:space-between;align-
items:center">

```

```

613                 <span style="font-size:12px;color:var(--text3)">Attendance</span>
614                 <span style="font-size:13px;font-weight:600;color:var(--text)"
id="sr-attendance"></span>
615             </div>
616         </div>
617     </div>
618 </div>
619
620     <!-- Devices -->
621     <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;overflow:hidden">
622         <div style="padding:12px 16px;border-bottom:1px solid var(--border)">
623             <span style="font-size:12px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px">Assigned Devices</span>
624         </div>
625         <table style="width:100%;border-collapse:collapse">
626             <thead>
627                 <tr>
628                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
629                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
630                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
631                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
632                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
633                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
634                 </tr>
635             </thead>
636             <tbody id="sr-devices"></tbody>
637         </table>
638     </div>
639 </div>
640
641     <!-- Empty / initial state -->
642     <div id="org-search-empty" style="padding:40px;text-align:center;color:var(--
text3);font-style:italic">
643         Enter a name above to look up an employee
644     </div>
645 </div>
646 </div>
647
648     <!-- PROMOTE USERS -->
649     <div id="promote-users-view" class="view">
650         <div class="section">
651             <div class="section-head">
652                 <div><div class="section-title"> Promote Users</div><div class="section-
sub">Onboard pending user registrations to your department</div></div>
653                 <button class="btn btn-secondary btn-sm" onclick="loadUsers()"> Refresh</button>
654             </div>
655             <div class="tablewrap">
656                 <table>
657                     <thead><tr><th>Name</th><th>Email</th><th>Timezone</th><th>Temp
Password</th><th>Action</th></tr></thead>
658                     <tbody id="users-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
659                 </table>
660             </div>
661         </div>
662     </div>
663
664     <!-- PENDING LEAVE APPROVALS -->
665     <div id="pending-leaves-view" class="view">
666         <div class="section">
667             <div class="section-head">
668                 <div><div class="section-title"> Pending Leave Approvals</div><div class="section-
sub">Review and take action</div></div>
669                 <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">
Refresh</button>
670             </div>
671             <div class="tablewrap">
672                 <table>
673

```

```

<thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
674         <tbody id="pending-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
675     </table>
676 </div>
677 </div>
678 </div>
679
680
681
682     <!-- ALL LEAVES -->
683     <div id="all-leaves-view" class="view">
684         <div class="section">
685             <div class="section-head">
686                 <div><div class="section-title">All Leave Requests</div></div>
687                 <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">
Refresh</button>
688             </div>
689             <div class="tablewrap">
690                 <table>
691 <thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
692         <tbody id="all-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
693     </table>
694 </div>
695 </div>
696 </div>
697
698     <!-- APPLY LEAVE -->
699     <div id="apply-leave-view" class="view">
700         <div class="section">
701             <div class="section-head">
702                 <div>
703                     <div class="section-title"> Apply for Leave</div>
704                     <div class="section-sub">Submit a new leave request</div>
705                 </div>
706             </div>
707             <div class="form-card">
708                 <div class="form-grid">
709                     <div class="form-group">
710                         <label>Leave Type</label>
711                         <select id="leave-type">
712                             <option value="SICK">Sick Leave</option>
713                             <option value="CASUAL">Casual Leave</option>
714                             <option value="UNPAID">Unpaid Leave</option>
715                         </select>
716                         <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(251,191,36,0.2);bord...
717                             Sick leave is limited to <strong>1 day per month</strong>. If already
used this month, please select Unpaid Leave.
718                         </div>
719                         <div id="casual-leave-hint" style="display:none; margin-top:6px; font-
size:11px; color:var(--accent); background:var(--accent-dim); border:1px solid rgba(167,139,250...
720                             You have <strong id="casual-balance-text">...</strong> casual leave
days available.
721                         </div>
722                     </div>
723                     <div class="form-group">
724                         <label>Reason</label>
725                         <input id="leave-reason" placeholder="Brief reason for leave">
726                     </div>
727                     <div class="form-group">
728                         <label>Start Date</label>
729                         <input type="date" id="leave-start">
730                     </div>
731                     <div class="form-group">
732                         <label>End Date</label>
733                         <input type="date" id="leave-end">
734                     </div>
735                 </div>
736                 <div class="form-actions">
737                     <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
738                     <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
739                 </div>

```

```

740         </div>
741     </div>
742 </div>
743
744 <!-- MY LEAVES -->
745 <div id="my-leaves-view" class="view">
746     <div class="section">
747         <div class="section-head">
748             <div class="section-title">My Leave Requests</div>
749             <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
750         </div>
751         <div class="tablewrap">
752             <table>
753 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Status</th><th>Action</th></tr></thead>
754             <tbody id="my-leaves-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
755             </table>
756         </div>
757     </div>
758 </div>
759
760 <!-- ===== -->
761 <!-- MY CALENDAR VIEW (NEW) -->
762 <!-- ===== -->
763 <div id="my-calendar-view" class="view">
764     <div class="section">
765         <div class="section-head">
766             <div>
767                 <div class="section-title">My Leave Calendar</div>
768                 <div class="section-sub">All approved leaves and public holidays</div>
769             </div>
770             <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
Refresh</button>
771         </div>
772     </div>
773 <!-- Leave Balance Summary Cards -->
774 <div class="leave-balance-grid" style="grid-template-columns: 1fr 1fr;">
775
776     <!-- Sick Leave Card -->
777     <div class="leave-bal-card sick">
778         <span class="lbc-icon"></span>
779         <span class="lbc-label">Sick Leave</span>
780         <span class="lbc-count" id="cal-sick-remaining">--</span>
781         <span class="lbc-sub">day available this month</span>
782         <div id="cal-sick-month-badge" style="
783             display:inline-flex; align-items:center; gap:5px;
784             margin-top:8px; padding:4px 10px;
785             border-radius:20px; font-size:10px; font-weight:700;
786             letter-spacing:0.8px; text-transform:uppercase;
787             ">...</div>
788         <span class="lbc-sub" id="cal-sick-accrued-sub" style="margin-top:4px"></span>
789     </div>
790
791     <!-- Casual Leave Card -->
792     <div class="leave-bal-card casual">
793         <span class="lbc-icon"></span>
794         <span class="lbc-label">Casual Leave</span>
795         <span class="lbc-count" id="cal-casual-remaining">--</span>
796         <span class="lbc-sub" id="cal-casual-sub">loading...</span>
797         <div style="margin-top:10px;display:flex;flex-direction:column;gap:3px">
798             <span class="lbc-sub" id="cal-casual-opening"></span>
799             <span class="lbc-sub" id="cal-casual-accrued"></span>
800             <span class="lbc-sub" id="cal-casual-used"></span>
801         </div>
802     </div>
803
804 </div>
805
806 <!-- Legend -->
807 <div class="cal-legend">
808     <span class="legend-label">Legend:</span>
809     <div class="legend-item">
810         <div class="legend-dot" style="background:#34d399"></div>
811         Approved Leave
812     </div>

```

```

813         <div class="legend-item">
814             <div class="legend-dot" style="background:#fbbf24"></div>
815             Pending Leave
816         </div>
817         <div class="legend-item">
818             <div class="legend-dot" style="background:#f87171"></div>
819             Rejected Leave
820         </div>
821         <div class="legend-item">
822             <div class="legend-dot" style="background:#38bdf8"></div>
823             Public Holiday
824         </div>
825     </div>
826
827     <!-- FullCalendar Mount Point -->
828     <div id="employee-calendar"></div>
829 </div>
830
831 <!-- END CALENDAR VIEW -->
832
833 <!-- MY SUBSCRIPTIONS -->
834 <div id="my-subscriptions-view" class="view">
835     <div class="section">
836         <div class="section-head">
837             <div><div class="section-title"> My Subscriptions</div><div class="section-
838 sub">Your active meal subscriptions</div></div>
839             <button class="btn btn-secondary btn-sm" onclick="loadMySubscriptions()">
Refresh</button>
840         </div>
841         <div class="tablewrap">
842             <table>
843 <thead><tr><th>Slot</th><th>Restaurant</th><th>Address</th><th>Schedule</th><th>Day</th><th>Status</th><th>Next
Delivery</th></tr></thead>
844             <tbody id="my-subscriptions-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
845             </table>
846         </div>
847     </div>
848
849 <!-- ADD SUBSCRIPTION -->
850 <div id="add-subscription-view" class="view">
851     <div class="section">
852         <div class="section-head">
853             <div><div class="section-title"> Subscribe to Meal</div><div class="section-
854 sub">Add a new meal slot subscription</div></div>
855             <div class="form-card">
856                 <div class="form-grid">
857                     <div class="form-group">
858                         <label>Schedule Type</label>
859                         <select id="sub-schedule" onchange="toggleDayOfWeek()">
860                             <option value="DAILY">Daily</option>
861                             <option value="WEEKLY">Weekly</option>
862                         </select>
863                     </div>
864                     <div class="form-group" id="day-of-week-group" style="display:none">
865                         <label>Day of Week</label>
866                         <select id="sub-day">
867                             <option value="MONDAY">Monday</option>
868                             <option value="TUESDAY">Tuesday</option>
869                             <option value="WEDNESDAY">Wednesday</option>
870                             <option value="THURSDAY">Thursday</option>
871                             <option value="FRIDAY">Friday</option>
872                             <option value="SATURDAY">Saturday</option>
873                             <option value="SUNDAY">Sunday</option>
874                         </select>
875                     </div>
876                 </div>
877             </div>
878
879 <!-- Slot Orders -->
880 <div style="margin-top:20px">
881     <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:12px">

```

```
882                 <label style="font-size:11px;font-weight:700;color:var(--text3);letter-
spacing:0.8px;text-transform:uppercase">Meal Slots & Restaurants</label>
883                 <button class="btn btn-secondary btn-sm" onclick="addSlotRow()"> Add
Slot</button>
884             </div>
885             <div id="slot-rows"></div>
886         </div>
887
888         <div class="form-actions">
889             <button class="btn btn-primary"
onclick="submitSubscription()">Subscribe</button>
890             <button class="btn btn-secondary"
onclick="clearSubscriptionForm()">Clear</button>
891         </div>
892     </div>
893 </div>
894 </div>
895 <!-- RESTAURANTS -->
896 <div id="restaurants-view" class="view">
897     <div class="section">
898         <div class="section-head">
899             <div><div class="section-title"> Restaurants</div><div class="section-sub">All
available restaurants</div></div>
900             <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">
Refresh</button>
901         </div>
902         <div class="tablewrap">
903             <table>
904                 <thead><tr><th>ID</th><th>Name</th><th>Address</th><th>Status</th></tr></thead>
905                 <tbody id="restaurants-list"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
906             </table>
907         </div>
908     </div>
909 </div>
910
911 <!-- MY DEVICES -->
912 <div id="my-devices-view" class="view">
913     <div class="section">
914         <div class="section-head">
915             <div>
916                 <div class="section-title"> My Assigned Devices</div>
917                 <div class="section-sub">Devices currently assigned to you</div>
918             </div>
919             <button class="btn btn-secondary btn-sm" onclick="loadMyAssignedDevices()">
Refresh</button>
920         </div>
921         <div class="tablewrap">
922             <table>
923                 <thead>
924                     <tr>
925                         <th>ID</th>
926                         <th>Device Name</th>
927                         <th>Brand</th>
928                         <th>Type</th>
929                         <th>Warranty</th>
930                         <th>Assigned On</th>
931                         <th>Status</th>
932                     </tr>
933                 </thead>
934                 <tbody id="my-devices-list">
935                     <tr><td colspan="7" class="empty-state">Loading...</td></tr>
936                 </tbody>
937             </table>
938         </div>
939     </div>
940 </div>
941
942 <!-- SERVICE REQUESTS -->
943 <div id="service-requests-view" class="view">
944
945     <!-- List -->
946     <div class="section" style="margin-bottom:20px">
947         <div class="section-head">
948             <div>
949                 <div class="section-title">My Service Requests</div>
```

```

950         <div class="section-sub">All requests you have raised</div>
951     </div>
952     <button class="btn btn-secondary btn-sm" onclick="loadServiceRequestsList()">
Refresh</button>
953 </div>
954 <div class="tablewrap">
955     <table>
956         <thead>
957             <tr>
958                 <th>ID</th>
959                 <th>Device</th>
960                 <th>Type</th>
961                 <th>Description</th>
962                 <th>Urgent</th>
963                 <th>Status</th>
964                 <th>Raised At</th>
965             </tr>
966         </thead>
967         <tbody id="service-requests-list">
968             <tr><td colspan="7" class="empty-state">Loading...</td></tr>
969         </tbody>
970     </table>
971 </div>
972 </div>
973
974 <!-- Raise form -->
975 <div class="section">
976     <div class="section-head"><div class="section-title">Raise a New Request</div></div>
977     <div class="form-card">
978         <div class="form-grid">
979             <div class="form-group full">
980                 <label>Device</label>
981                 <select id="sr-device-select">
982                     <option value="">-- Select your device --</option>
983                 </select>
984             </div>
985             <div class="form-group">
986                 <label>Request Type</label>
987                 <select id="sr-type">
988                     <option value="REPAIR">Repair</option>
989                     <option value="REPLACEMENT">Replacement</option>
990                     <option value="RETURN">Return</option>
991                 </select>
992             </div>
993             <div class="form-group" style="display:flex;align-
items:center;gap:10px;padding-top:22px">
994                 <input type="checkbox" id="sr-urgent"
style="width:16px;height:16px;padding:0;flex-shrink:0">
995                 <label for="sr-urgent" style="text-transform:none;font-size:13px;font-
weight:500;color:var(--text);cursor:pointer;letter-spacing:0">
996                     Mark as Urgent
997                 </label>
998             </div>
999             <div class="form-group full">
1000                 <label>Issue Description</label>
1001                 <textarea id="sr-description" rows="3" placeholder="Describe the issue in
detail..."></textarea>
1002             </div>
1003         </div>
1004         <div class="form-actions">
1005             <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
Request</button>
1006             <button class="btn btn-secondary"
onclick="clearServiceRequestForm()">Clear</button>
1007         </div>
1008     </div>
1009 </div>
1010 </div>
1011
1012 <!-- TWO-FACTOR AUTH -->
1013 <div id="two-factor-view" class="view">
1014     <div class="section">
1015         <div class="section-head">
1016             <div>
1017                 <div class="section-title"> Two-Factor Authentication</div>
1018                 <div class="section-sub">Protect your account with an authenticator app</div>

```



```

1019         </div>
1020         <span id="totp-status-chip" class="chip">Checking...</span>
1021     </div>
1022
1023     <div style="padding:24px;max-width:480px">
1024
1025         <!-- DISABLED state – setup flow -->
1026         <div id="totp-setup-section">
1027             <p style="font-size:13px;color:var(--text2);margin-bottom:16px;line-
height:1.6">
1028                 Use Google Authenticator, Authy, or any TOTP app.
1029                 Scan the QR code below, then enter the 6-digit code to activate.
1030             </p>
1031             <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-
btn">
1032                 Generate QR Code
1033             </button>
1034
1035             <div id="qr-area" style="display:none;margin-top:24px;text-align:center">
1036                 <img id="qr-img" src="" alt="QR Code"
1037                     style="width:180px;height:180px;border-radius:10px;
1038                     border:1px solid var(--border2);display:block;margin:0 auto">
1039                 <div style="margin-top:12px;font-size:11px;color:var(--text3)">
1040                     Can't scan? Enter this key manually:
1041                 </div>
1042                 <div id="totp-secret-display"
1043                     style="font-family:var(--mono);font-size:13px;color:var(--accent);
1044                     background:var(--bg3);border:1px solid var(--border2);
1045                     border-radius:6px;padding:8px 12px;margin-top:6px;
1046                     word-break:break-all"></div>
1047
1048                 <div style="margin-top:20px;text-align:left">
1049                     <label style="font-size:11px;color:var(--text3);font-weight:600;
1050                     text-transform:uppercase;letter-spacing:0.8px;
1051                     display:block;margin-bottom:8px">
1052                         Enter 6-digit code to activate
1053                     </label>
1054                     <input id="totp-verify-input" type="text" inputmode="numeric"
1055                         maxlength="6" placeholder="000000"
1056                         style="text-align:center;font-size:20px;
1057                         font-family:var(--mono);letter-spacing:8px;padding:12px"
1058                         oninput="this.value=this.value.replace(/\D/g, '')">
1059                     <button class="btn btn-primary"
1060                         onclick="verifyAndActivateTotp()"
1061                         style="width:100%;margin-top:10px">
1062                         Activate 2FA
1063                     </button>
1064                     <div id="totp-verify-error"
1065                         style="display:none;margin-top:8px;font-size:12px;
1066                         color:var(--red)"></div>
1067                 </div>
1068             </div>
1069         </div>
1070
1071         <!-- ENABLED state -->
1072         <div id="totp-disable-section" style="display:none">
1073             <div style="background:var(--green-dim);border:1px solid rgba(52,211,153,0.2);
1074             border-radius:8px;padding:12px 16px;font-size:13px;
1075             color:var(--green);margin-bottom:16px">
1076                 Two-factor authentication is active on your account.
1077             </div>
1078             <button class="btn btn-danger" onclick="disableTotp()">
1079                 Disable 2FA
1080             </button>
1081         </div>
1082     </div>
1083 </div>
1084 </div>
1085
1086 <!-- PROFILE -->
1087 <div id="profile-view" class="view">
1088     <div class="section">
1089         <div class="section-head">
1090             <div class="section-title">My Profile</div>
1091         </div>
1092         <div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;padding:24px">

```

```

1093         <div style="display:flex;flex-direction:column;gap:4px">
1094             <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1095                 letter-spacing:0.8px;font-weight:700">Name</div>
1096             <div style="font-size:14px;color:var(--text);padding:10px 14px;
1097                 background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1098                 font-family:var(--mono)" id="p-name"></div>
1099         </div>
1100         <div style="display:flex;flex-direction:column;gap:4px">
1101             <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1102                 letter-spacing:0.8px;font-weight:700">Email</div>
1103             <div style="font-size:14px;color:var(--text);padding:10px 14px;
1104                 background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1105                 font-family:var(--mono)" id="p-email"></div>
1106         </div>
1107         <div style="display:flex;flex-direction:column;gap:4px">
1108             <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1109                 letter-spacing:0.8px;font-weight:700">Department</div>
1110             <div style="font-size:14px;color:var(--text);padding:10px 14px;
1111                 background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1112                 font-family:var(--mono)" id="p-dept"></div>
1113         </div>
1114         <div style="display:flex;flex-direction:column;gap:4px">
1115             <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1116                 letter-spacing:0.8px;font-weight:700">Role</div>
1117             <div style="font-size:14px;color:var(--text);padding:10px 14px;
1118                 background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1119                 font-family:var(--mono)" id="p-role"></div>
1120         </div>
1121         <div style="display:flex;flex-direction:column;gap:4px">
1122             <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1123                 letter-spacing:0.8px;font-weight:700">Status</div>
1124             <div style="font-size:14px;color:var(--text);padding:10px 14px;
1125                 background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1126                 font-family:var(--mono)" id="p-status"></div>
1127         </div>
1128         <div style="display:flex;flex-direction:column;gap:4px">
1129             <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1130                 letter-spacing:0.8px;font-weight:700">Timezone</div>
1131             <div style="font-size:14px;color:var(--text);padding:10px 14px;
1132                 background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1133                 font-family:var(--mono)" id="p-tz"></div>
1134         </div>
1135     </div>
1136 </div>
1137
1138 <!-- MATERNITY SECTION – only visible for female managers -->
1139 <div class="section" id="maternity-apply-btn" style="display:none;margin-top:0">
1140     <div class="section-head">
1141         <div>
1142             <div class="section-title"> Maternity Leave</div>
1143             <div class="section-sub">Apply for 180 days maternity leave</div>
1144         </div>
1145     </div>
1146     <div class="form-card">
1147         <div class="form-grid">
1148             <div class="form-group">
1149                 <label>Start Date</label>
1150                 <input type="date" id="mat-start-date">
1151             </div>
1152             <div class="form-group">
1153                 <label>End Date (Auto)</label>
1154                 <input type="text" id="mat-end-date"
1155                     placeholder="Auto: start + 180 days"
1156                     disabled
1157                     style="color:var(--text3);cursor:not-allowed">
1158             </div>
1159             <div class="form-group full">
1160                 <label>Reason / Notes</label>
1161                 <textarea id="mat-reason" rows="3"
1162                     placeholder="Optional notes..."></textarea>
1163             </div>
1164             <div class="form-group full">
1165                 <div style="background:var(--accent-dim);
1166                     border:1px solid rgba(167,139,250,0.2);
1167                     border-radius:8px;padding:12px 16px;
1168                     font-size:12px;color:var(--text2)">

```

```

1169             Maternity leave is fixed at
1170             <strong style="color:var(--accent)">180 days</strong>.
1171             End date is calculated automatically.
1172             You can apply <strong style="color:var(--accent)">once per
year</strong>.
1173         </div>
1174     </div>
1175 </div>
1176     <div class="form-actions">
1177         <button class="btn btn-primary" onclick="applyMaternityLeave()">
1178             Submit Maternity Leave
1179         </button>
1180     </div>
1181 </div>
1182 </div>
1183 </div>
1184
1185 </div>
1186 </div>
1187 </div>
1188 <!-- Tooltip for calendar events -->
1189 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
1190 <script>
1191     const API='http://localhost:8080';
1192     let allEmployeesCache=[];
1193
1194     // Add these near the top of your <script>, right after: let allEmployeesCache=[];
1195
1196     let employeeCalendar = null;
1197     let calendarInitialized = false;
1198
1199     const STATUS_COLORS = {
1200         APPROVED : { bg:'rgba(52,211,153,0.25)', border:'#34d399', text:'#34d399' },
1201         PENDING  : { bg:'rgba(251,191,36,0.25)', border:'#fbbf24', text:'#fbbf24' },
1202         REJECTED : { bg:'rgba(248,113,113,0.25)', border:'#f87171', text:'#f87171' },
1203         HOLIDAY  : { bg:'rgba(167,139,250,0.25)', border:'#a78bfa', text:'#a78bfa' },
1204     };
1205
1206     const TYPE_LABEL = { SICK:' Sick', CASUAL:' Casual', MATERNITY:' Maternity', UNPAID:' Unpaid'
1207     };
1208
1209     (function(){
1210         const token=localStorage.getItem('jwt_token');
1211         const authType=localStorage.getItem('auth_type');
1212         if(!token&&authType!=='session'){window.location.href='/login.html';return;}
1213         if(token){
1214             try{
1215                 const p=JSON.parse(atob(token.split('.')[1]));
1216                 const rawRole = p.role || p.roles || p.authorities || '';
1217                 const normalizedRoles = Array.isArray(rawRole)
1218                     ? rawRole.map(v => String(v).replace(/ROLE_/, ''))
1219                     : [String(rawRole).replace(/ROLE_/, '')];
1220                 if(normalizedRoles.includes('MANAGER')) { initUser(); return; }
1221                 if(normalizedRoles.includes('ADMIN')){window.location.href='/dashboard.html';return;}
1222                 if(normalizedRoles.includes('EMPLOYEE')){window.location.href='/employee-
dashboard.html';return;}
1223                 if(normalizedRoles.includes('VENDOR')){window.location.href='/vendor-
dashboard.html';return;}
1224             }catch(e){}
1225         }
1226         initUser();
1227     })();
1228
1229     function getAuthHeader(){
1230         const t=localStorage.getItem('jwt_token');
1231         return t?{'Authorization': `Bearer ${t}`}:{};
1232     }
1233     async function apiFetch(path, opts={}){
1234         const res = await fetch(`${API}${path}`, {
1235             ...opts,
1236             headers: {'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},
1237             credentials: 'include'
1238         });
1239         if(res.status===401){
1240             localStorage.removeItem('jwt_token');
1241             localStorage.removeItem('auth_type');

```

```

1242     showAlert('Session expired','error');
1243     setTimeout(()=>window.location.href='/google.html',1500);
1244     throw new Error('Unauthorized');
1245 }
1246 const text = await res.text();
1247 if(!res.ok){
1248     let message = res.statusText || `Error ${res.status}`;
1249     try {
1250         const body = text ? JSON.parse(text) : null;
1251         message = body?.message || body?.error || message;
1252     } catch {
1253         message = text || message;
1254     }
1255     throw new Error(message);
1256 }
1257 if(!text) return null;
1258 try { return JSON.parse(text); } // try JSON first
1259 catch { return text; } // fall back to plain string
1260 }
1261
1262 function asArray(value){
1263     return Array.isArray(value) ? value : [];
1264 }
1265
1266 async function initUser(){
1267     const token=localStorage.getItem('jwt_token');
1268     let email='';
1269     if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
1270     document.getElementById('avatar-initials').textContent=(email||'M').charAt(0).toUpperCase();
1271     document.getElementById('sidebar-name').textContent=email||'Manager';
1272     document.getElementById('topbar-email').textContent=email;
1273     loadDashboard();
1274     loadUsers();
1275 }
1276
1277 function showView(name,el){
1278     document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
1279     document.getElementById(`${name}-view`).classList.add('active');
1280     document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
1281     if(el)el.classList.add('active');
1282     document.getElementById('pageTitle').textContent=name.split('-')
1283     .map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
1284     const loaders = {
1285         dashboard: loadDashboard,
1286         employees: loadEmployees,
1287         'pending-leaves': loadPendingLeaves,
1288         'all-leaves': loadAllLeaves,
1289         'my-leaves': loadMyLeaves,
1290         'my-calendar': loadMyCalendar,
1291         'add-employee': () => {},
1292         'promote-users': loadUsers,
1293         'my-subscriptions': loadMySubscriptions,
1294         'add-subscription': () => {},
1295         restaurants: loadRestaurants,
1296         'my-devices': loadMyAssignedDevices,
1297         'service-requests': loadServiceRequests,
1298         'two-factor':loadTotpStatus,
1299         'search-employee': () => {},
1300         'profile': loadProfile, // ADD THIS
1301     };
1302     if(loaders[name])loaders[name]();
1303 }
1304
1305 async function loadDashboard(){
1306     const [empsResult, leavesResult] = await Promise.allSettled([
1307         apiFetch('/manager/employees'),
1308         apiFetch('/manager/pending')
1309     ]);
1310     const emps = asArray(empsResult.status === 'fulfilled' ? empsResult.value : []);
1311     const leaves = asArray(leavesResult.status === 'fulfilled' ? leavesResult.value : []);
1312
1313     if (empsResult.status === 'rejected') {
1314         console.error('Failed to load manager employees:', empsResult.reason);
1315         document.getElementById('dash-employees').innerHTML =
1316             '<tr><td colspan="3" class="empty-state">Could not load employees</td></tr>';

```

```

1317     }
1318     if (leavesResult.status === 'rejected') {
1319         console.error('Failed to load pending leaves:', leavesResult.reason);
1320         document.getElementById('dash-pending').innerHTML =
1321             '<tr><td colspan="4" class="empty-state">Could not load pending requests</td></tr>';
1322     }
1323
1324     // Stats
1325     document.getElementById('stat-total').textContent = emps.length;
1326     document.getElementById('stat-active').textContent = emps.filter(e => e.status ===
'ACTIVE').length;
1327     document.getElementById('stat-on-leave').textContent = emps.filter(e => e.status ===
'ON_LEAVE').length;
1328     const pending = leaves.filter(l => l.status === 'PENDING');
1329     document.getElementById('stat-pending').textContent = pending.length;
1330     document.getElementById('pending-badge').textContent = pending.length;
1331     document.getElementById('pending-badge').classList.toggle('hidden', pending.length === 0);
1332
1333     // Build employeeId name lookup from employees list
1334     const empMap = {};
1335     emps.forEach(e => { empMap[e.employeeId] = e.name || '-'; });
1336
1337     // Dash pending
1338     if (leavesResult.status === 'fulfilled') {
1339         document.getElementById('dash-pending').innerHTML = pending.length
1340             ? pending.slice(0, 5).map(l => {
1341                 const leaveId = l.leaveRequestId || 0;
1342                 const empName = empMap[l.employeeName] || `#${l.employeeName}`;
1343                 return `<tr>
1344                     <td>${empName}</td>
1345                     <td><span class="chip ${l.leaveType || ''}.toLowerCase()>${l.leaveType}</span></td>
1346                     <td>${l.startDate} ${l.endDate}</td>
1347                     <td><div class="action-row">
1348                         <button class="btn btn-approve btn-sm"
onclick="approveLeave(${leaveId})"></button>
1349                         <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})"></button>
1350                     </div></td>
1351                 </tr>`;
1352             }).join('')
1353             : '<tr><td colspan="4" class="empty-state">No pending requests </td></tr>';
1354     }
1355
1356     // Dash employees
1357     if (empsResult.status === 'fulfilled') {
1358         document.getElementById('dash-employees').innerHTML = emps.length
1359             ? emps.slice(0, 6).map(e => `<tr>
1360                 <td>${e.name || '-'}</td>
1361                 <td>${e.dept || '-'}</td>
1362                 <td><span class="chip ${(e.status || '').toLowerCase()}>${e.status || '-'}</span></td>
1363             </tr>`).join('')
1364             : '<tr><td colspan="3" class="empty-state">No employees</td></tr>';
1365     }
1366 }
1367
1368 async function loadEmployees(){
1369     try{
1370         const emps=await apiFetch('/manager/employees');
1371         allEmployeesCache=emps||[];
1372         renderEmployees(allEmployeesCache);
1373     }catch(e){document.getElementById('employees-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load employees</td></tr>';}
1374 }
1375
1376 function renderEmployees(emps){
1377     document.getElementById('employees-list').innerHTML=emps.length
1378         ? emps.map(e=>`<tr><td style="font-family:var(--mono);font-
size:12px">#${e.employeeId}</td><td>${e.name||'-'}</td><td style="font-family:var(--mono);font-
size:12px">${e.email||'-'}</td>...
1379         : '<tr><td colspan="7" class="empty-state">No employees found</td></tr>';
1380 }
1381
1382 function filterEmployees(){
1383     const q=document.getElementById('emp-search').value.toLowerCase();
1384     renderEmployees(allEmployeesCache.filter(e=>(e.name||'').toLowerCase().includes(q)||
(e.email||'').toLowerCase().includes(q)||
(e.dept||'').toLowerCase().includes(q)));

```

```

1385     }
1386
1387     async function addEmployee(){
1388         const name      = document.getElementById('emp-name').value.trim();
1389         const email      = document.getElementById('emp-email').value.trim();
1390         const password    = document.getElementById('emp-pass').value;
1391         const gender      = document.getElementById('emp-gender').value;
1392         const timezone    = document.getElementById('emp-timezone').value;
1393         if(!name || !email || !password){
1394             showAlert('Please fill all required fields','error');
1395             return;
1396         }
1397         try{
1398             await apiFetch('/manager/employees', {
1399                 method: 'POST',
1400                 body: JSON.stringify({ name, email, password, timezone, gender })
1401             });
1402             showAlert('Employee added successfully!','success');
1403             clearEmpForm();
1404             loadEmployees();
1405         }catch(e){ showAlert('Failed: '+e.message,'error'); }
1406     }
1407
1408     function clearEmpForm(){
1409         ['emp-name','emp-email','emp-pass'].forEach(id=>document.getElementById(id).value='');
1410         document.getElementById('emp-timezone').value = 'UTC';
1411         document.getElementById('emp-gender').selectedIndex = 0;
1412     }
1413     async function deleteEmployee(id){
1414         if(!confirm('Remove this employee?'))return;
1415         try{await apiFetch(`/employee/${id}`, {method: 'DELETE'});showAlert('Employee
removed','success');loadEmployees();}
1416         catch(e){
1417             console.log(e);
1418             showAlert('Failed to remove employee','error');
1419         }
1420     }
1421
1422     async function loadPendingLeaves(){
1423         try{
1424             const leaves = await apiFetch('/manager/pending');
1425             console.log('Leave sample:', leaves?.[0]); // remove after confirming
1426             const pending=(leaves||[]).filter(l=>l.status==='PENDING');
1427             document.getElementById('pending-leaves-list').innerHTML=pending.length
1428                 ? pending.map(l=>{
1429                 // Try all possible ID field names your DTO might use
1430                 const leaveId = l.leaveRequestId || l.leaveId || l.id || 0;
1431                 return `<tr>
1432                     <td>${l.employeeName||l.employeeId||'-'}</td>
1433                     <td><span class="chip ${l.leaveType||''}.toLowerCase()>${l.leaveType}</span></td>
1434                     <td>${l.startDate}</td>
1435                     <td>${l.endDate}</td>
1436                     <td>${l.numberOfDays||'-'}</td>
1437                     <td>${(l.reason||'-').substring(0,40)}</td>
1438                     <td><div class="action-row">
1439                         <button class="btn btn-approve btn-sm" onclick="approveLeave(${leaveId})">
Approve</button>
1440                         <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})">
Reject</button>
1441                     </div></td>
1442                 </tr>`;
1443             }).join('')
1444             : '<tr><td colspan="7" class="empty-state">No pending approvals </td></tr>';
1445         }catch(e){
1446             document.getElementById('pending-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';
1447         }
1448     }
1449     async function approveLeave(id) {
1450         console.log('Approving leaveRequestId:', id); // verify this is not 0
1451         try {
1452             const res = await fetch(`${API}/leave_requests/approval`, {
1453                 method: 'PUT',
1454                 headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
1455                 credentials: 'include',
1456                 body: JSON.stringify({

```

```

1457         leaveRequestId: id, // matches ActionDTO exactly
1458         action: 'APPROVED', // check if your service expects uppercase or lowercase
1459         remarks: ''
1460     });
1461 });
1462     const text = await res.text();
1463     if (res.ok) {
1464         showAlert('Leave approved ', 'success');
1465         loadPendingLeaves(); loadDashboard(); loadAllLeaves();
1466     } else {
1467         showAlert('Error: ' + text, 'error');
1468     }
1469 } catch(e) { showAlert(e.message, 'error'); }
1470 }
1471
1472 async function rejectLeave(id) {
1473     console.log('Rejecting leaveRequestId:', id); // verify this is not 0
1474     try {
1475         const res = await fetch(`${API}/leave_requests/approval`, {
1476             method: 'PUT',
1477             headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
1478             credentials: 'include',
1479             body: JSON.stringify({
1480                 leaveRequestId: id, // matches ActionDTO exactly
1481                 action: 'REJECTED', // check if your service expects uppercase or lowercase
1482                 remarks: ''
1483             })
1484         });
1485         const text = await res.text();
1486         if (res.ok) {
1487             showAlert('Leave rejected', 'success');
1488             loadPendingLeaves(); loadDashboard(); loadAllLeaves();
1489         } else {
1490             showAlert('Error: ' + text, 'error');
1491         }
1492     } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
1493 }
1494
1495 async function loadAllLeaves(){
1496     try{
1497         const leaves=await apiFetch('/manager/requests');
1498         document.getElementById('all-leaves-list').innerHTML=(leaves||[]).length
1499             ? (leaves).map(l=><tr>
1500 <td>${l.employeeName||'-'}</td>
1501 <td><span class="chip ${l.leaveType||''}.toLowerCase()">${l.leaveType||'-'}</span></td>
1502 <td>${l.startDate||'-'}</td>
1503 <td>${l.endDate||'-'}</td>
1504 <td>${l.reason||'-'}.substring(0,50}</td>
1505 <td><span class="chip ${l.status||''}.toLowerCase()">${l.status||'-'}</span></td>
1506 </tr>`)
1507 : '<tr><td colspan="7" class="empty-state">No leave requests</td></tr>';
1508     }catch(e){document.getElementById('all-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';}
1509 }
1510
1511 async function applyLeave() {
1512     const leaveType = document.getElementById('leave-type').value;
1513     const reason = document.getElementById('leave-reason').value.trim();
1514     const startDate = document.getElementById('leave-start').value;
1515     const endDate = document.getElementById('leave-end').value;
1516
1517     if (!reason || !startDate || !endDate) {
1518         showAlert('Please fill all fields', 'error');
1519         return;
1520     }
1521     if (endDate < startDate) {
1522         showAlert('End date cannot be before start date', 'error');
1523         return;
1524     }
1525
1526     try {
1527         const data = await apiFetch('/leave_requests', {
1528             method: 'POST',
1529             body: JSON.stringify({ leaveType, reason, startDate, endDate })
1530         });
1531         if (data && data.warning) {

```

```

1532         showAlert(data.warning, 'warning');
1533     } else {
1534         showAlert('Leave request submitted successfully!', 'success');
1535     }
1536     clearLeaveForm();
1537     loadMyLeaves();
1538 } catch(e) {
1539     let msg = e.message;
1540     try {
1541         const parsed = JSON.parse(msg);
1542         if(parsed.error) msg = parsed.error;
1543     } catch(_) {}
1544     showAlert(msg, 'error');
1545 }
1546 }
1547 document.getElementById('leave-type').addEventListener('change', async function() {
1548     document.getElementById('sick-leave-hint').style.display =
1549         this.value === 'SICK' ? 'block' : 'none';
1550
1551     const casualHint = document.getElementById('casual-leave-hint');
1552     if(this.value === 'CASUAL') {
1553         casualHint.style.display = 'block';
1554         document.getElementById('casual-balance-text').textContent = 'loading...';
1555         try {
1556             const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1557             const balances = dashboard?.leaveBalances || {};
1558             const cas = balances['CASUAL'] || balances['casual'] || {};
1559             const available = Number(cas.available ?? cas.availableBalance ?? 0);
1560             document.getElementById('casual-balance-text').textContent =
1561                 available > 0 ? `${available}` : '0 (no balance remaining)';
1562         } catch(e) {
1563             document.getElementById('casual-balance-text').textContent = 'unavailable';
1564         }
1565     } else {
1566         casualHint.style.display = 'none';
1567     }
1568 });
1569
1570 function clearLeaveForm() {
1571     document.getElementById('leave-type').value = 'SICK';
1572     document.getElementById('leave-reason').value = '';
1573     document.getElementById('leave-start').value = '';
1574     document.getElementById('leave-end').value = '';
1575 }
1576
1577 async function loadMyLeaves(){
1578     try{
1579         const data = await apiFetch('/leave_requests/employee');
1580         const leaves = data?.requests || data || [];
1581         const today = new Date().toISOString().split('T')[0];
1582
1583         document.getElementById('my-leaves-list').innerHTML = leaves.length
1584             ? leaves.map(l => {
1585                 const canCancel = l.status === 'PENDING' || l.status === 'APPROVED';
1586                 const notStarted = l.startDate > today;
1587                 const showCancel = canCancel && notStarted;
1588
1589                 return `<tr>
1590                     <td><span class="chip ${l.leaveType||''}.toLowerCase()">${l.leaveType}</span></td>
1591                     <td>${l.startDate}</td>
1592                     <td>${l.endDate}</td>
1593                     <td>${l.numberOfDays || '-'}</td>
1594                     <td><span class="chip ${l.status||''}.toLowerCase()">${l.status}</span></td>
1595                     <td>${showCancel
1596                         ? `<button class="btn btn-reject btn-sm"
1597 onclick="cancelMyLeave(${l.leaveRequestId})"> Cancel</button>`
1598                         : `<span style="color:var(--text3);font-size:12px"></span>`}
1599                     </td>
1600                 </tr>`;
1601             }).join('')
1602             : `<tr><td colspan="6" class="empty-state">No leave requests</td></tr>`;
1603     } catch(e) {
1604         document.getElementById('my-leaves-list').innerHTML =
1605             `<tr><td colspan="6" class="empty-state">Could not load</td></tr>`;
1606     }
1607 }

```



```

1607
1608 async function cancelMyLeave(leaveId) {
1609     if (!confirm('Are you sure you want to cancel this leave request?')) return;
1610     try {
1611         await apiFetch(`/leave_requests/cancel/${leaveId}`, { method: 'PUT' });
1612         showAlert('Leave request cancelled successfully', 'success');
1613         loadMyLeaves();
1614         loadDashboard(); // refresh pending count
1615     } catch(e) {
1616         showAlert('Failed to cancel: ' + e.message, 'error');
1617     }
1618 }
1619
1620 async function loadUsers(){
1621     try{
1622         const users = await apiFetch('/manager/Users');
1623         const badge = document.getElementById('users-badge');
1624         if(badge){ badge.textContent = (users||[]).length;
badge.classList.toggle('hidden', (users||[]).length===0); }
1625         document.getElementById('users-list').innerHTML = (users||[]).length
1626             ? users.map(u=><tr>
1627                 <td>${u.name||'-'}</td>
1628                 <td style="font-family:var(--mono);font-size:12px">${u.email||'-'}</td>
1629                 <td style="font-family:var(--mono);font-size:12px">${u.timezone||'-'}</td>
1630                 <td><input type="password" placeholder="Set password" id="pwd-${CSS.escape(u.email)}"
1631                     style="padding:6px 10px;font-size:12px;width:160px"></td>
1632                 <td><button class="btn btn-approve btn-sm" onclick="promoteUser('${u.email}')">
Promote</button></td>
1633                 </tr>`).join('')
1634                 : '<tr><td colspan="5" class="empty-state">No pending user registrations</td></tr>';
1635     }catch(e){
1636         document.getElementById('users-list').innerHTML='<tr><td colspan="5" class="empty-state">Could not
load</td></tr>';
1637     }
1638 }
1639
1640 async function promoteUser(email){
1641     const pwdInput = document.getElementById('pwd-'+CSS.escape(email));
1642     const password = pwdInput?.value?.trim();
1643     if(!password){ showAlert('Please set a password before promoting','error'); return; }
1644     try{
1645         const res = await fetch(`${API}/manager/promote`,{
1646             method:'PUT',
1647             headers:{'Content-Type':'application/json',...getAuthHeader()},
1648             credentials:'include',
1649             body: JSON.stringify({ email, password })
1650         });
1651         const text = await res.text();
1652         if(res.ok){
1653             showAlert(`${email} promoted to Employee `, 'success');
1654             loadUsers();
1655         } else {
1656             showAlert('Error: '+text, 'error');
1657         }
1658     }catch(e){ showAlert('Failed: '+e.message, 'error'); }
1659 }
1660
1661 async function loadRestaurants(){
1662     try{
1663         const data = await apiFetch('/restaurants');
1664         document.getElementById('restaurants-list').innerHTML = (data||[]).length
1665             ? data.map(r=><tr>
1666                 <td style="font-family:var(--mono);font-size:12px">#${r.restaurantId||r.id}</td>
1667                 <td>${r.name||'-'}</td>
1668                 <td style="font-size:12px;color:var(--text3)">${r.address||'-'}</td>
1669                 <td><span class="chip active">${r.status||'ACTIVE'}</span></td>
1670                 </tr>`).join('')
1671                 : '<tr><td colspan="4" class="empty-state">No restaurants found</td></tr>';
1672     }catch(e){
1673         document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-
state">Could not load</td></tr>';
1674     }
1675 }
1676 // SUBSCRIPTIONS
1677
1678 async function loadMySubscriptions(){

```

```

1679     try{
1680         const subs = await apiFetch('/subscription/user');
1681         document.getElementById('my-subscriptions-list').innerHTML = (subs||[]).length
1682             ? subs.map(s=>`<tr>
1683                 <td><span class="chip pending">${s.mealSlot||'-'}</span></td>
1684                 <td>${s.restaurantName||'-'}</td>
1685                 <td style="font-size:12px;color:var(--text3)">${s.restaurantAddress||'-'}</td>
1686                 <td>${s.scheduleType||'-'}</td>
1687                 <td>${s.dayOfWeek||'-'}</td>
1688                 <td><span class="chip ${((s.status||'')).toLowerCase()}>${s.status||'-'}</span></td>
1689                 <td style="font-family:var(--mono);font-size:11px">${s.nextDeliveryTime ? new
Date(s.nextDeliveryTime).toLocaleString() : '-'}</td>
1690             </tr>`).join('')
1691             : '<tr><td colspan="7" class="empty-state">No subscriptions yet</td></tr>';
1692     }catch(e){
1693         document.getElementById('my-subscriptions-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';
1694     }
1695 }
1696
1697 let slotRowCount = 0;
1698 async function addSlotRow(){
1699     slotRowCount++;
1700     const id = slotRowCount;
1701     const div = document.createElement('div');
1702     div.id = `slot-row-${id}`;
1703     div.style.cssText = 'display:grid;grid-template-columns:1fr 1fr auto;gap:12px;margin-
bottom:10px;align-items:end';
1704
1705     // Fetch restaurants for the dropdown
1706     let options = '<option value="">Loading...</option>';
1707     try{
1708         const data = await apiFetch('/restaurants');
1709         options = data.map(r => `<option
value="${r.restaurantId||r.id}">${r.name}</option>`).join('');
1710     }catch(e){ options = '<option value="">Could not load</option>'; }
1711
1712     div.innerHTML = `
1713     <div class="form-group" style="margin:0">
1714         <label>Meal Slot</label>
1715         <select id="slot-meal-${id}">
1716             <option value="BREAKFAST">Breakfast</option>
1717             <option value="LUNCH">Lunch</option>
1718             <option value="DINNER">Dinner</option>
1719         </select>
1720     </div>
1721     <div class="form-group" style="margin:0">
1722         <label>Restaurant</label>
1723         <select id="slot-restaurant-${id}">${options}</select>
1724     </div>
1725     <button class="btn btn-reject btn-sm" onclick="removeSlotRow(${id})" style="margin-
bottom:1px"></button>
1726     `;
1727     document.getElementById('slot-rows').appendChild(div);
1728 }
1729
1730 function removeSlotRow(id){
1731     document.getElementById(`slot-row-${id}`)?.remove();
1732 }
1733
1734 function toggleDayOfWeek(){
1735     const isWeekly = document.getElementById('sub-schedule').value === 'WEEKLY';
1736     document.getElementById('day-of-week-group').style.display = isWeekly ? 'flex' : 'none';
1737 }
1738
1739 async function submitSubscription(){
1740     const scheduleType = document.getElementById('sub-schedule').value;
1741     const dayOfWeek = scheduleType === 'WEEKLY' ? document.getElementById('sub-day').value : null;
1742
1743     const slotOrders = [];
1744     let valid = true;
1745
1746     document.querySelectorAll('[id^="slot-row-"]').forEach(row => {
1747         const idNum = row.id.split('-')[2];
1748         const mealSlot = document.getElementById(`slot-meal-${idNum}`)?.value;
1749         const restaurantVal = document.getElementById(`slot-restaurant-${idNum}`)?.value;

```

```

1750
1751         if(!mealSlot || !restaurantVal){ valid = false; return; }
1752
1753         const restaurantId = parseInt(restaurantVal);
1754         if(isNaN(restaurantId)){ valid = false; return; }
1755
1756         slotOrders.push({ mealSlot, restaurantId });
1757     });
1758
1759     if(!valid || slotOrders.length === 0){
1760         showAlert('Please add at least one valid slot','error');
1761         return;
1762     }
1763
1764     // Derive mealSlots from slotOrders to satisfy @NotEmpty
1765     const mealSlots = slotOrders.map(s => s.mealSlot);
1766
1767     try{
1768         const res = await fetch(`${API}/subscription`, {
1769             method: 'POST',
1770             headers: {'Content-Type':'application/json', ...getAuthHeader()},
1771             credentials: 'include',
1772             body: JSON.stringify({ scheduleType, dayOfWeek, mealSlots, slotOrders })
1773         });
1774         const text = await res.text();
1775         if(res.ok){
1776             showAlert('Subscription added successfully! ','success');
1777             clearSubscriptionForm();
1778             loadMySubscriptions();
1779         } else {
1780             showAlert('Error: '+text,'error');
1781         }
1782     }catch(e){ showAlert('Failed: '+e.message,'error'); }
1783 }
1784
1785 function clearSubscriptionForm(){
1786     document.getElementById('sub-schedule').value = 'DAILY';
1787     document.getElementById('day-of-week-group').style.display = 'none';
1788     document.getElementById('slot-rows').innerHTML = '';
1789     slotRowCount = 0;
1790 }
1791
1792 /* =====
1793 MY CALENDAR (NEW)
1794 ===== */
1795 async function loadMyCalendar(){
1796     // 1. Load balance cards
1797     await loadCalendarBalanceCards();
1798     // 2. Build or refresh FullCalendar
1799     if(!calendarInitialized){
1800         initFullCalendar();
1801     } else {
1802         await refreshCalendarEvents();
1803     }
1804 }
1805
1806 async function loadCalendarBalanceCards() {
1807     try {
1808         const [dashboard, sickMonthData] = await Promise.all([
1809             apiFetch('/api/employees/me/leave-dashboard'),
1810             apiFetch('/leave_requests/sick/used-this-month')
1811         ]);
1812
1813         /* SICK */
1814         const sickBadge = document.getElementById('cal-sick-month-badge');
1815         const sick      = dashboard?.sick;
1816         const sickAvail = Number(sick?.available ?? 0);
1817         const usedThisMonth = sickMonthData?.usedThisMonth === true;
1818
1819         if (usedThisMonth) {
1820             // Explicitly used this month
1821             document.getElementById('cal-sick-remaining').textContent = '0';
1822             sickBadge.textContent      = ' Used this month';
1823             sickBadge.style.background = 'var(--red-dim)';
1824             sickBadge.style.color      = 'var(--red)';
1825             sickBadge.style.border     = '1px solid rgba(248,113,113,0.25)';

```

```

1826     } else if (sickAvail === 0) {
1827         // New employee – accrual hasn't run yet
1828         document.getElementById('cal-sick-remaining').textContent = '0';
1829         sickBadge.textContent = ' Accrual pending';
1830         sickBadge.style.background = 'var(--amber-dim)';
1831         sickBadge.style.color = 'var(--amber)';
1832         sickBadge.style.border = '1px solid rgba(251,191,36,0.25)';
1833     } else {
1834         // Has balance and not used
1835         document.getElementById('cal-sick-remaining').textContent = sickAvail;
1836         sickBadge.textContent = ' Available this month';
1837         sickBadge.style.background = 'var(--green-dim)';
1838         sickBadge.style.color = 'var(--green)';
1839         sickBadge.style.border = '1px solid rgba(52,211,153,0.25)';
1840     }
1841
1842     /* CASUAL */
1843     const cas = dashboard?.casual;
1844     const casAvail = Number(cas?.available ?? 0);
1845     const casTotal = Number(cas?.opening ?? 0) + Number(cas?.accrued ?? 0);
1846     const casUsed = Number(cas?.used ?? 0);
1847
1848     document.getElementById('cal-casual-remaining').textContent = casAvail;
1849     document.getElementById('cal-casual-sub').textContent =
1850         `${casUsed} used · ${casTotal} total this year`;
1851
1852     } catch(e) {
1853         console.error('Could not load leave balances:', e);
1854         document.getElementById('cal-sick-remaining').textContent = '--';
1855         document.getElementById('cal-casual-remaining').textContent = '--';
1856     }
1857 }
1858
1859 async function buildCalendarEvents(){
1860     const events = [];
1861     try{
1862         // – Employee leaves –
1863         const data = await apiFetch('/leave_requests/employee');
1864         const leaves = data?.requests || data || [];
1865         leaves.forEach(l=>{
1866             const s = (l.status||'PENDING').toUpperCase();
1867             const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
1868             const typeLabel = TYPE_LABEL[l.leaveType] || l.leaveType || 'Leave';
1869
1870             // FullCalendar end date is exclusive, so add 1 day for multi-day events
1871             let endDate = l.endDate;
1872             if(endDate){
1873                 const d = new Date(endDate);
1874                 d.setDate(d.getDate()+1);
1875                 endDate = d.toISOString().split('T')[0];
1876             }
1877
1878             events.push({
1879                 id : `leave-${l.leaveRequestId}`,
1880                 title : `${typeLabel} (${s.charAt(0)+s.slice(1).toLowerCase()}`,
1881                 start : l.startDate,
1882                 end : endDate,
1883                 allDay : true,
1884                 backgroundColor : c.bg,
1885                 borderColor : c.border,
1886                 textColor : c.text,
1887                 extendedProps : {
1888                     type : 'leave',
1889                     status : s,
1890                     reason : l.reason || '-',
1891                     leaveType: l.leaveType
1892                 }
1893             });
1894         });
1895     } catch(e){ console.warn('Could not load leaves for calendar:', e); }
1896
1897     try{
1898         // – Public holidays (admin-added) –
1899         const holidays = await apiFetch('/holidays');
1900         (holidays||[]).forEach(h=>{
1901             const c = STATUS_COLORS.HOLIDAY;
1902             events.push({

```

```

1902         id      : `holiday-${h.id||h.holidayId||Math.random()}`,
1903         title   : ` ${h.name||h.holidayName||'Holiday'}`,
1904         start    : h.date||h.holidayDate,
1905         allDay   : true,
1906         backgroundColor : c.bg,
1907         borderColor   : c.border,
1908         textColor     : c.text,
1909         extendedProps : { type:'holiday', description: h.description||'' }
1910     });
1911 });
1912 }catch(e){ console.warn('Could not load holidays for calendar:', e); }
1913
1914     return events;
1915 }
1916
1917 function initFullCalendar(){
1918     const calendarEl = document.getElementById('employee-calendar');
1919
1920     employeeCalendar = new FullCalendar.Calendar(calendarEl, {
1921         initialView: 'dayGridMonth',
1922         height      : 'auto',
1923         headerToolbar:{
1924             left  : 'prev,next today',
1925             center: 'title',
1926             right : 'dayGridMonth,dayGridWeek,listMonth'
1927         },
1928         buttonText:{
1929             today    : 'Today',
1930             month    : 'Month',
1931             week     : 'Week',
1932             listMonth: 'List'
1933         },
1934         dayMaxEvents: 3,      // "+N more" link if overflowing
1935         navLinks     : true,  // click day number day view
1936         events       : [],    // start empty, populate below
1937
1938         // Event hover tooltip
1939         eventMouseEnter: function(info){
1940             const ep = info.event.extendedProps;
1941             const tooltip = document.getElementById('cal-tooltip');
1942             let html = `<div class="tt-title">${info.event.title}</div>`;
1943             if(ep.type==='leave'){
1944                 html += `<div class="tt-row"> ${info.event.startStr}
1945 ${info.event.endStr||info.event.startStr}</div>`;
1946                 if(ep.reason && ep.reason!=='-'){
1947                     html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
1948                     html += `<div class="tt-row">Status: <strong
1949 style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
1950                 } else {
1951                     if(ep.description) html += `<div class="tt-row">${ep.description}</div>`;
1952                 }
1953                 tooltip.innerHTML = html;
1954                 tooltip.style.display = 'block';
1955             },
1956             eventMouseLeave: function(){
1957                 document.getElementById('cal-tooltip').style.display='none';
1958             },
1959             eventClick: function(info){
1960                 // Click navigates to leave history
1961                 const ep = info.event.extendedProps;
1962                 if(ep.type==='leave'){
1963                     showView('my-leaves', document.querySelector('.nav-item:nth-child(4)'));
1964                 }
1965             },
1966         });
1967
1968     employeeCalendar.render();
1969     calendarInitialized = true;
1970
1971     // Load events asynchronously after render
1972     refreshCalendarEvents();
1973 }
1974
1975 async function refreshCalendarEvents(){
1976     if(!employeeCalendar) return;
1977     const events = await buildCalendarEvents();

```

```

1976         // Remove all existing events and re-add
1977         employeeCalendar.removeAllEvents();
1978         events.forEach(e=>employeeCalendar.addEvent(e));
1979     }
1980
1981     async function refreshCalendar(){
1982         await loadCalendarBalanceCards();
1983         await refreshCalendarEvents();
1984         showAlert('Calendar refreshed!', 'success');
1985     }
1986
1987     // Move tooltip with mouse
1988     document.addEventListener('mousemove', function(e){
1989         const tt = document.getElementById('cal-tooltip');
1990         if(tt.style.display==='block'){
1991             tt.style.left = (e.clientX+14)+'px';
1992             tt.style.top = (e.clientY+14)+'px';
1993             // Keep tooltip on screen
1994             const rect = tt.getBoundingClientRect();
1995             if(rect.right > window.innerWidth)
1996                 tt.style.left = (e.clientX - rect.width - 10)+'px';
1997             if(rect.bottom > window.innerHeight)
1998                 tt.style.top = (e.clientY - rect.height - 10)+'px';
1999         }
2000     });
2001
2002     // ===== MY DEVICES =====
2003     async function loadMyAssignedDevices() {
2004         try {
2005             const devices = await apiFetch('/devices/my');
2006             const tbody = document.getElementById('my-devices-list');
2007             if (!devices || !devices.length) {
2008                 tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
2009                 return;
2010             }
2011             const today = new Date();
2012             tbody.innerHTML = devices.map(d => {
2013                 const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
2014                 const warrantyDisplay = warrantyExpiry
2015                     ? (warrantyExpiry > today
2016                       ? `<span style="color:var(--green);font-size:11px"> ${d.warrantyExpiryDate}</span>`
2017                       : `<span style="color:var(--red);font-size:11px"> Expired
${d.warrantyExpiryDate}</span>`)
2018                     : `<span style="color:var(--text3);font-size:11px">--</span>`;
2019                 return `<tr>
2020                     <td>#${d.id}</td>
2021                     <td><strong>${d.deviceName || '-'}</strong></td>
2022                     <td>${d.brand || '-'}</td>
2023                     <td><span class="chip">${(d.deviceType || '').toLowerCase()}</span></td>
2024                     <td>${warrantyDisplay}</td>
2025                     <td style="font-size:12px;color:var(--text2)">${d.assignedDate || '-'}</td>
2026                     <td><span class="chip">${(d.deviceStatus || '').toLowerCase()}>${d.deviceStatus ||
'-'}</span></td>
2027                 </tr>`;
2028             }).join('');
2029         } catch (e) {
2030             document.getElementById('my-devices-list').innerHTML =
2031                 '<tr><td colspan="7" class="empty-state">Could not load devices</td></tr>';
2032         }
2033     }
2034
2035     // ===== SERVICE REQUESTS =====
2036     async function loadServiceRequests() {
2037         await Promise.all([loadServiceRequestsList(), loadMyDevicesForSR()]);
2038     }
2039
2040     async function loadServiceRequestsList() {
2041         try {
2042             const requests = await apiFetch('/service-requests/my');
2043             const tbody = document.getElementById('service-requests-list');
2044             if (!requests || !requests.length) {
2045                 tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No service requests yet</td></tr>';
2046                 return;
2047             }
2048             tbody.innerHTML = requests.map(r => `<tr>

```

```

2049         <td style="color:var(--text3);font-family:var(--mono)">#{r.id}</td>
2050         <td>#{r.deviceName || '-'}</td>
2051         <td><span class="chip pending">#{r.requestType}</span></td>
2052         <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap">
2053             #{r.issueDescription || '-'}
2054         </td>
2055         <td>#{r.urgent
2056             ? '<span class="chip rejected">YES</span>'
2057             : '<span style="color:var(--text3)">No</span>'}
2058         </td>
2059         <td><span class="chip #{(r.status || '').toLowerCase()}">#{r.status || '-'}</span></td>
2060         <td style="font-size:12px;color:var(--text3)">
2061             #{r.raisedAt ? new Date(r.raisedAt).toLocaleDateString() : '-'}
2062         </td>
2063     </tr>` ).join('');
2064     } catch (e) { console.error(e); }
2065 }
2066
2067 async function loadMyDevicesForSR() {
2068     try {
2069         const devices = await apiFetch('/devices/my');
2070         const select = document.getElementById('sr-device-select');
2071         select.innerHTML = '<option value="">-- Select your device --</option>';
2072         (devices || []).forEach(d => {
2073             const opt = document.createElement('option');
2074             opt.value = d.id;
2075             opt.textContent = `${d.deviceName} (${d.brand || d.deviceType || 'Device'})`;
2076             select.appendChild(opt);
2077         });
2078     } catch (e) { console.error('Failed to load devices for SR', e); }
2079 }
2080
2081 async function submitServiceRequest() {
2082     const deviceId = parseInt(document.getElementById('sr-device-select').value);
2083     const requestType = document.getElementById('sr-type').value;
2084     const issueDescription = document.getElementById('sr-description').value.trim();
2085     const urgent = document.getElementById('sr-urgent').checked;
2086
2087     if (!deviceId) { showAlert('Please select a device', 'error'); return; }
2088     if (!issueDescription) { showAlert('Please describe the issue', 'error'); return; }
2089
2090     try {
2091         await apiFetch('/service-requests', {
2092             method: 'POST',
2093             body: JSON.stringify({ deviceId, requestType, issueDescription, urgent })
2094         });
2095         showAlert('Service request submitted!', 'success');
2096         clearServiceRequestForm();
2097         loadServiceRequestsList();
2098     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
2099 }
2100
2101 function clearServiceRequestForm() {
2102     document.getElementById('sr-device-select').value = '';
2103     document.getElementById('sr-type').value = 'REPAIR';
2104     document.getElementById('sr-description').value = '';
2105     document.getElementById('sr-urgent').checked = false;
2106 }
2107
2108 async function searchEmployee() {
2109     const name = document.getElementById('org-search-input').value.trim();
2110     const errorEl = document.getElementById('org-search-error');
2111     const resultsEl = document.getElementById('org-search-results');
2112     const emptyEl = document.getElementById('org-search-empty');
2113
2114     errorEl.style.display = 'none';
2115     resultsEl.style.display = 'none';
2116     emptyEl.style.display = 'block';
2117
2118     if (!name) {
2119         errorEl.textContent = 'Please enter a name to search.';
2120         errorEl.style.display = 'block';
2121         return;
2122     }
2123
2124     try {

```

```

2125     const emp = await apiFetch(`/manager/employees/search?name=${encodeURIComponent(name)}`);
2126
2127     // Populate info card
2128     document.getElementById('sr-name').textContent = emp.name || '-';
2129     document.getElementById('sr-id').textContent = `#${emp.employeeId}`;
2130     document.getElementById('sr-email').textContent = emp.email || '-';
2131     document.getElementById('sr-dept').textContent = emp.dept || '-';
2132     document.getElementById('sr-attendance').textContent = emp.attendance || '-';
2133
2134     // Status chip
2135     const statusMap = { ACTIVE: 'approved', ON_LEAVE: 'pending', INACTIVE: 'rejected' };
2136     const statusKey = (emp.status || '').toString();
2137     document.getElementById('sr-status').innerHTML =
2138         `<span class="chip ${statusMap[statusKey] || ''}">${statusKey || '-'}</span>`;
2139
2140     // Devices table
2141     const today = new Date();
2142     const devices = emp.devices || [];
2143     document.getElementById('sr-devices').innerHTML = devices.length
2144         ? devices.map(d => {
2145             const exp = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
2146             const warranty = exp
2147                 ? (exp > today
2148                     ? `<span style="font-size:11px;color:var(--green)">
2149 $${d.warrantyExpiryDate}</span>`
2150                     : `<span style="font-size:11px;color:var(--red)"> Expired</span>`)
2151                 : `<span style="font-size:11px;color:var(--text3)">-</span>`;
2152             const statusCss = { ASSIGNED: 'assigned', UNDER_REPAIR: 'under_repair', CONDEMNED:
2153 'condemned' };
2154             const ds = (d.deviceStatus || '').toString();
2155             return `<tr>
2156                 <td style="padding:10px 16px;border-top:1px solid var(--border);font-
2157 size:13px;color:var(--text)"><strong>${d.deviceName || '-'}</strong></td>
2158                 <td style="padding:10px 16px;border-top:1px solid var(--border);font-
2159 size:13px;color:var(--text2)">${d.brand || '-'}</td>
2160                 <td style="padding:10px 16px;border-top:1px solid var(--border)"><span class="chip
2161 pending">${(d.deviceType || '').toLowerCase()}</span></td>
2162                 <td style="padding:10px 16px;border-top:1px solid var(--border)"><span class="chip
2163 ${statusCss[ds] || ''}">${ds || '-'}</span></td>
2164                 <td style="padding:10px 16px;border-top:1px solid var(--border)">${warranty}</td>
2165                 <td style="padding:10px 16px;border-top:1px solid var(--border);font-
2166 size:12px;color:var(--text3)">${d.assignedDate || '-'}</td>
2167             </tr>`;
2168         }).join('')
2169         : `<tr><td colspan="6" style="padding:20px;text-align:center;color:var(--text3);font-
2170 style:italic">No devices assigned</td></tr>`;
2171
2172     emptyEl.style.display = 'none';
2173     resultsEl.style.display = 'block';
2174
2175 } catch (e) {
2176     errorEl.textContent = e.message.includes('not found') || e.message.includes('404')
2177         ? `No employee found with name "${name}"`
2178         : `Search failed: ` + e.message;
2179     errorEl.style.display = 'block';
2180     emptyEl.style.display = 'block';
2181 }
2182
2183 function logout(){
2184     // Clear all localStorage
2185     localStorage.removeItem('jwt_token');
2186     localStorage.removeItem('refresh_token');
2187     localStorage.removeItem('auth_type');
2188     localStorage.removeItem('last_role');
2189
2190     // Call server logout to invalidate session
2191     fetch(`${API}/session/logout`, {
2192         method: 'POST',
2193         credentials: 'include'
2194     }).finally(() => {
2195         // Force expire the session cookie on client side
2196         document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
2197         window.location.href = '/google.html';
2198     });
2199 }

```



```

2193     async function loadNotifCount() {
2194         try {
2195             const data = await apiFetch('/notifications/unread-count');
2196             const badge = document.getElementById('notif-badge');
2197             if (data.count > 0) {
2198                 badge.textContent = data.count;
2199                 badge.style.display = 'flex';
2200             } else {
2201                 badge.style.display = 'none';
2202             }
2203         } catch(e) { console.warn('Could not load notif count', e); }
2204     }
2205
2206     async function toggleNotifPanel() {
2207         const panel = document.getElementById('notif-panel');
2208         if (panel.style.display === 'none') {
2209             panel.style.display = 'block';
2210             await loadNotifications();
2211             try { await apiFetch('/notifications/mark-read', { method: 'POST' }); } catch(e) {}
2212             document.getElementById('notif-badge').style.display = 'none';
2213         } else {
2214             panel.style.display = 'none';
2215         }
2216     }
2217
2218     async function loadNotifications() {
2219         try {
2220             const data = await apiFetch('/notifications');
2221             const list = document.getElementById('notif-list');
2222             list.innerHTML = data.length
2223                 ? data.map(n => `
2224                     <div class="notif-item ${n.read ? '' : 'unread'}">
2225                         <div>${n.message}</div>
2226                         <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
2227                     </div>`).join('')
2228                 : '<div class="notif-item">No notifications yet.</div>';
2229         } catch(e) { console.warn('Could not load notifications', e); }
2230     }
2231
2232     loadNotifCount();
2233     setInterval(loadNotifCount, 30000);
2234
2235     document.addEventListener('click', e => {
2236         const panel = document.getElementById('notif-panel');
2237         const wrapper = document.querySelector('.notif-wrapper');
2238         if (panel && wrapper && !panel.contains(e.target) && !wrapper.contains(e.target)) {
2239             panel.style.display = 'none';
2240         }
2241     });
2242     function showAlert(msg,type){
2243         const el=document.createElement('div');
2244         el.className=`alert alert-${type}`;el.textContent=msg;
2245         document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
2246     }
2247
2248     // Gender store
2249     let currentManagerGender = null;
2250
2251     // Load profile data
2252     async function loadProfile() {
2253         try {
2254             const me = await apiFetch('/employee/me');
2255             if(me) {
2256                 currentManagerGender = me.gender;
2257
2258                 document.getElementById('p-name').textContent = me.name || '-';
2259                 document.getElementById('p-email').textContent = me.email || '-';
2260                 document.getElementById('p-dept').textContent = me.dept || '-';
2261                 document.getElementById('p-role').textContent = me.role || 'MANAGER';
2262                 document.getElementById('p-status').textContent = me.status || '-';
2263                 document.getElementById('p-tz').textContent = me.timezone || '-';
2264
2265                 // Show maternity section only for female managers
2266                 const maternityBtn = document.getElementById('maternity-apply-btn');
2267                 if(maternityBtn) {
2268                     maternityBtn.style.display = me.gender === 'F' ? 'block' : 'none';

```

```

2269     }
2270
2271     // Also update sidebar name with real name
2272     document.getElementById('sidebar-name').textContent = me.name || '-';
2273     document.getElementById('avatar-initials').textContent =
2274         (me.name || me.email || 'M').charAt(0).toUpperCase();
2275     }
2276 } catch(e) {
2277     console.error('Could not load profile:', e);
2278 }
2279 }
2280
2281 // Auto calculate maternity end date
2282 document.getElementById('mat-start-date').addEventListener('change', function(){
2283     if(this.value) {
2284         const start = new Date(this.value);
2285         const end = new Date(start);
2286         end.setDate(end.getDate() + 179);
2287         document.getElementById('mat-end-date').value =
2288             end.toISOString().split('T')[0];
2289     } else {
2290         document.getElementById('mat-end-date').value = '';
2291     }
2292 });
2293
2294 // Submit maternity leave
2295 async function applyMaternityLeave() {
2296     const startDate = document.getElementById('mat-start-date').value;
2297     const reason = document.getElementById('mat-reason').value.trim();
2298
2299     if(!startDate) {
2300         showAlert('Please select a start date', 'error');
2301         return;
2302     }
2303
2304     try {
2305         await apiFetch('/leave_requests/maternity/apply', {
2306             method: 'POST',
2307             body: JSON.stringify({ startDate, reason })
2308         });
2309         showAlert('Maternity leave submitted successfully!', 'success');
2310         document.getElementById('mat-start-date').value = '';
2311         document.getElementById('mat-end-date').value = '';
2312         document.getElementById('mat-reason').value = '';
2313         loadMyLeaves();
2314     } catch(e) {
2315         showAlert('Failed: ' + e.message, 'error');
2316     }
2317 }
2318
2319 // ===== TWO-FACTOR AUTH =====
2320 async function loadTotpStatus() {
2321     const chip = document.getElementById('totp-status-chip');
2322     chip.textContent = 'Checking...';
2323     chip.className = 'chip';
2324     document.getElementById('qr-area').style.display = 'none';
2325     document.getElementById('totp-verify-input').value = '';
2326     document.getElementById('totp-verify-error').style.display = 'none';
2327
2328     try {
2329         const data = await apiFetch('/totp/status');
2330         if (data.totpEnabled) {
2331             chip.textContent = 'ENABLED';
2332             chip.className = 'chip approved';
2333             document.getElementById('totp-setup-section').style.display = 'none';
2334             document.getElementById('totp-disable-section').style.display = 'block';
2335         } else {
2336             chip.textContent = 'DISABLED';
2337             chip.className = 'chip rejected';
2338             document.getElementById('totp-setup-section').style.display = 'block';
2339             document.getElementById('totp-disable-section').style.display = 'none';
2340         }
2341     } catch(e) {
2342         chip.textContent = 'Error';
2343         chip.className = 'chip pending';
2344     }
2345 }

```

```

2345     async function generateTotpQr() {
2346         const btn = document.getElementById('generate-qr-btn');
2347         btn.textContent = 'Generating...';
2348         btn.disabled = true;
2349         try {
2350             const data = await apiFetch('/totp/setup', { method: 'POST' });
2351             document.getElementById('totp-secret-display').textContent = data.secret || '';
2352             const qrUri = encodeURIComponent(data.qrCodeUri || '');
2353             document.getElementById('qr-img').src =
2354                 `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
2355             document.getElementById('qr-area').style.display = 'block';
2356         } catch(e) {
2357             showAlert('Failed to generate QR: ' + e.message, 'error');
2358         } finally {
2359             btn.textContent = 'Regenerate QR Code';
2360             btn.disabled = false;
2361         }
2362     }
2363
2364     async function verifyAndActivateTotp() {
2365         const code = document.getElementById('totp-verify-input').value.trim();
2366         const errEl = document.getElementById('totp-verify-error');
2367         errEl.style.display = 'none';
2368         if (code.length !== 6) {
2369             errEl.textContent = 'Please enter a 6-digit code.';
2370             errEl.style.display = 'block';
2371             return;
2372         }
2373         try {
2374             await apiFetch('/totp/verify-setup', {
2375                 method: 'POST',
2376                 body: JSON.stringify({ code })
2377             });
2378             showAlert(' 2FA activated! You will need your authenticator on next login.', 'success');
2379             loadTotpStatus();
2380         } catch(e) {
2381             errEl.textContent = 'Invalid code. Try again.';
2382             errEl.style.display = 'block';
2383         }
2384     }
2385
2386     async function disableTotp() {
2387         if (!confirm('Are you sure you want to disable two-factor authentication?')) return;
2388         try {
2389             await apiFetch('/totp/disable', { method: 'DELETE' });
2390             showAlert('2FA has been disabled.', 'success');
2391             loadTotpStatus();
2392         } catch(e) {
2393             showAlert('Failed to disable 2FA: ' + e.message, 'error');
2394         }
2395     }
2396 </script>
2397 </body>
2398 </html>
2399

```

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Account Pending</title>
6   <style>
7     * { margin: 0; padding: 0; box-sizing: border-box; }
8     body {
9       font-family: 'Outfit', sans-serif;
10      background: #070910;
11      color: #dde4f0;
12      min-height: 100vh;
13      display: flex;
14      align-items: center;
15      justify-content: center;
16    }
17    .card {
18      background: #0e1118;
19      border: 1px solid rgba(255,255,255,0.07);
20      border-radius: 16px;
21      padding: 48px;
22      max-width: 420px;
23      text-align: center;
24    }
25    .icon { font-size: 48px; margin-bottom: 20px; }
26    h1 { font-size: 22px; font-weight: 700; margin-bottom: 12px; }
27    p { font-size: 14px; color: #6b7591; line-height: 1.6; margin-bottom: 24px; }
28    .email {
29      background: #131821;
30      border: 1px solid rgba(79,140,255,0.2);
31      border-radius: 8px;
32      padding: 10px 16px;
33      font-size: 13px;
34      color: #4f8cff;
35      margin-bottom: 28px;
36      font-family: monospace;
37    }
38    .btn {
39      padding: 10px 24px;
40      background: transparent;
41      border: 1px solid rgba(255,255,255,0.1);
42      border-radius: 8px;
43      color: #6b7591;
44      font-size: 13px;
45      cursor: pointer;
46    }
47    .btn:hover { border-color: rgba(255,255,255,0.2); color: #dde4f0; }
48  </style>
49 </head>
50 <body>
51 <div class="card">
52   <div class="icon"></div>
53   <h1>Account Pending Approval</h1>
54   <p>You've successfully signed in with Google. Your account is currently awaiting approval from an admin
or manager.</p>
55   <div class="email" id="user-email">Loading...</div>
56   <p>Once approved, you'll have full access to the portal. Please contact your manager or admin.</p>
57   <button class="btn" onclick="logout()">Sign out</button>
58 </div>
59
60 <script>
61   // Show the user their email from JWT
62   const token = localStorage.getItem('jwt_token');
63   if (token) {
64     try {
65       const payload = JSON.parse(atob(token.split('.')[1]));
66       document.getElementById('user-email').textContent = payload.sub || 'Unknown';
67     } catch(e) {}
68   }
69
70   async function logout() {
71     try {

```

```
72         // Destroy the server-side OAuth2 session
73         await fetch('/session/logout', {
74             method: 'POST',
75             credentials: 'include'
76         });
77     } catch(e) {}
78
79     localStorage.clear();
80     sessionStorage.clear();
81     window.location.href = '/google.html';
82 }
83 </script>
84 </body>
85 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>TechVendor Hub</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Syne:wght@400;500;600;700;800&family=DM+Mono:wght@400;500&display
=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#080b10;--bg2:#0c1018;--bg3:#111620;--bg4:#161d2a;
12             --panel:#131a26;--panel2:#1a2235;
13             --border:rgba(0,210,255,0.08);--border2:rgba(0,210,255,0.18);--border3:rgba(0,210,255,0.35);
14             --text:#d4e8f5;--text2:#6b8aaa;--text3:#3a5270;
15             --cyan:#00d2ff;--cyan-dim:rgba(0,210,255,0.08);--cyan-glow:rgba(0,210,255,0.15);
16             --amber:#f59e0b;--amber-dim:rgba(245,158,11,0.1);
17             --green:#10d48e;--green-dim:rgba(16,212,142,0.1);
18             --red:#f05454;--red-dim:rgba(240,84,84,0.1);
19             --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.1);
20             --orange:#fb923c;--orange-dim:rgba(251,146,60,0.1);
21             --font:'Syne',sans-serif;--mono:'DM Mono',monospace;
22             --sidebar:260px;
23         }
24         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
25         .app{display:flex;height:100vh}
26
27         body::before{content:'';position:fixed;inset:0;background:repeating-linear-
gradient(0deg,transparent,transparent 2px,rgba(0,0,0,0.03) 2px,rgba(0,0,0,0.03) 4px);pointer-events:none;z-
index:9...
28
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;overflow:hidden;position:relative}
30         .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:300px;background:linear-
gradient(to top,rgba(0,210,255,0.03),transparent);pointer-events:none}
31
32         .logo{padding:24px 20px 20px;border-bottom:1px solid var(--border)}
33         .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
34         .logo-hex{width:36px;height:36px;background:linear-gradient(135deg,#00d2ff,#0066ff);clip-
path:polygon(50% 0%,100% 25%,100% 75%,50% 100%,0% 75%,0% 25%);display:flex;align-items:center;justif...
35         .logo-name{font-size:17px;font-weight:800;color:var(--text);letter-spacing:-0.5px}
36         .logo-sub{font-size:10px;color:var(--text3);font-family:var(--mono);letter-spacing:1px;text-
transform:uppercase}
37         .vendor-badge{display:inline-flex;align-items:center;gap:6px;margin-top:10px;padding:5px
12px;background:var(--cyan-dim);border:1px solid var(--border2);border-radius:4px;font-size:10px;font...
38         .badge-dot{width:5px;height:5px;background:var(--cyan);border-radius:50%;animation:pulse 2s
infinite}
39         @keyframes pulse{0%,100%{opacity:1;box-shadow:0 0 0 0 var(--cyan-glow)}50%{opacity:0.7;box-shadow:0
0 0 4px transparent}}
40
41         .nav{flex:1;padding:12px 0;overflow-y:auto}
42         .nav::-webkit-scrollbar{width:3px}
43         .nav::-webkit-scrollbar-thumb{background:var(--border2);border-radius:2px}
44         .nav-section{padding:14px 20px 5px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2.5px;text-transform:uppercase;font-family:var(--mono)}
45         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-
radius:6px;cursor:pointer;transition:all 0.15s;color:var(--text2);font-size:13px;font-weight:500;p...
46         .nav-item:hover{background:var(--cyan-dim);color:var(--cyan)}
47         .nav-item.active{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
48         .nav-item.active::before{content:'';position:absolute;left:-8px;top:50%;transform:translateY(-
50%);width:3px;height:60%;background:var(--cyan);border-radius:0 2px 2px 0}
49         .nav-icon{width:16px;height:16px;opacity:0.7;flex-shrink:0}
50         .nav-item.active .nav-icon,.nav-item:hover .nav-icon{opacity:1}
51         .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:9px;font-
weight:700;padding:2px 6px;border-radius:3px;font-family:var(--mono)}
52         .nav-badge.amber{background:var(--amber)}
53
54         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
55         .user-row{display:flex;align-items:center;gap:10px}
56         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#00d2ff22,#0066ff22);border:1px

```

```

solid var(--border2);border-radius:6px;display:flex;align-items:center;justify-content:cente...
57 .user-name{font-size:12px;font-weight:600;color:var(--text)}
58 .user-role{font-size:10px;color:var(--cyan);font-family:var(--mono);text-transform:uppercase;letter-
spacing:0.8px}
59
60 .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
61 .topbar{background:var(--bg2);padding:0 24px;height:56px;display:flex;justify-content:space-
between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
62 .page-title{font-size:15px;font-weight:700;color:var(--text);display:flex;align-
items:center;gap:8px}
63 .page-title-prefix{color:var(--cyan);font-family:var(--mono);font-size:12px;font-weight:400}
64 .topbar-right{display:flex;align-items:center;gap:10px}
65 .topbar-email{font-size:11px;color:var(--text3);font-family:var(--mono)}
66 .signout-btn{padding:6px 14px;background:transparent;border:1px solid var(--border2);border-
radius:5px;color:var(--text2);font-family:var(--font);font-size:11px;font-weight:600;cursor:point...
67 .signout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
68
69 .content{flex:1;overflow-y:auto;padding:24px}
70 .content::-webkit-scrollbar{width:4px}
71 .content::-webkit-scrollbar-thumb{background:var(--border2)}
72 .view{display:none}
73 .view.active{display:block;animation:viewIn 0.25s ease}
74 @keyframes viewIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}
75
76 .stats-row{display:grid;grid-template-columns:repeat(4,1fr);gap:14px;margin-bottom:22px}
77 .stat{background:var(--panel);border:1px solid var(--border);border-radius:8px;padding:18px
20px;position:relative;overflow:hidden;cursor:default;transition:border-color 0.2s}
78 .stat:hover{border-color:var(--border2)}
79 .stat::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
80 .stat.c{--c:var(--cyan)} .stat.g{--c:var(--green)} .stat.a{--c:var(--amber)} .stat.r{--c:var(--red)}
.stat.v{--c:var(--violet)} .stat.o{--c:var(--orange)}
81 .stat::after{background:var(--c)}
82 .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;font-
weight:600;margin-bottom:8px;font-family:var(--mono)}
83 .stat-val{font-size:28px;font-weight:800;color:var(--text);line-height:1;font-family:var(--mono)}
84 .stat-sub{font-size:11px;color:var(--text3);margin-top:5px}
85 .stat-icon{position:absolute;right:16px;top:50%;transform:translateY(-50%);font-
size:24px;opacity:0.12}
86
87 .card{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-
bottom:18px;overflow:hidden}
88 .card-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
89 .card-title{font-size:13px;font-weight:700;color:var(--text);letter-spacing:0.2px}
90 .card-sub{font-size:11px;color:var(--text3);margin-top:2px;font-family:var(--mono)}
91 .card-body{padding:20px}
92 .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:18px}
93
94 .tablewrap{overflow-x:auto}
95 table{width:100%;border-collapse:collapse}
96 th{padding:9px 16px;text-align:left;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
97 td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:12px;color:var(--text2);font-
family:var(--mono)}
98 tr:last-child td{border-bottom:none}
99 tr:hover td{background:rgba(0,210,255,0.02);color:var(--text)}
100 td.main-cell{font-family:var(--font);font-size:13px;color:var(--text);font-weight:500}
101
102 .chip{display:inline-flex;align-items:center;gap:5px;padding:3px 9px;border-radius:3px;font-
size:10px;font-weight:700;letter-spacing:0.8px;text-transform:uppercase;font-family:var(--mono)}
103 .chip::before{content:'';width:4px;height:4px;border-radius:50%;flex-shrink:0}
104 .chip.available{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)} .chip.available::before{background:var(--green)}
105 .chip.assigned{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.assigned::before{background:var(--cyan)}
106 .chip.under_repair{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(245,158,11,0.2)} .chip.under_repair::before{background:var(--amber)}
107 .chip.condemned{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
.chip.condemned::before{background:var(--red)}
108 .chip.returned_to_vendor{background:var(--violet-dim);color:var(--violet);border:1px solid
rgba(167,139,250,0.2)} .chip.returned_to_vendor::before{background:var(--violet)}
109 .chip.open{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
.chip.open::before{background:var(--amber)}
110 .chip.acknowledged{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.acknowledged::before{background:var(--cyan)}
111 .chip.sent_for_repair{background:var(--orange-dim);color:var(--orange);border:1px solid

```

```

111 rgba(251,146,60,0.2)} .chip.sent_for_repair::before{background:var(--orange)}
112 .chip.repair_done{background:var(--green-dim);color:var(--green);border:1px solid
113 rgba(16,212,142,0.2)} .chip.repair_done::before{background:var(--green)}
114 .chip.resolved{background:var(--green-dim);color:var(--green);border:1px solid
115 rgba(16,212,142,0.2)} .chip.resolved::before{background:var(--green)}
116 .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
117 .chip.rejected::before{background:var(--red)}
118 .chip.sent{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
119 .chip.sent::before{background:var(--amber)}
120 .chip.in_progress{background:var(--orange-dim);color:var(--orange);border:1px solid
121 rgba(251,146,60,0.2)} .chip.in_progress::before{background:var(--orange)}
122 .chip.fixed{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
123 .chip.fixed::before{background:var(--green)}
124 .chip.irreparable{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
125 .chip.irreparable::before{background:var(--red)}
126 .chip.laptop{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
127 .chip.laptop::before{background:var(--cyan)}
128 .chip.phone{background:var(--violet-dim);color:var(--violet);border:1px solid
129 rgba(167,139,250,0.2)} .chip.phone::before{background:var(--violet)}
130 .chip.tablet{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
131 .chip.tablet::before{background:var(--green)}
132 .chip.monitor,.chip.other,.chip.desktop,.chip.keyboard,.chip.mouse,.chip.headset{background:var(--
133 amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)} .chip.monitor::before,...
134 .chip.active_assign{background:var(--green-dim);color:var(--green);border:1px solid
135 rgba(16,212,142,0.2)} .chip.active_assign::before{background:var(--green)}
136 .chip.returned{background:var(--text3);color:var(--text2);border:1px solid var(--border)}
137 .chip.returned::before{background:var(--text2)}
138 .chip.not_fixable{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
139 .chip.not_fixable::before{background:var(--red)}
140 .chip.pending{background:var(--bg4);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
141 .chip.pending::before{background:var(--amber)}
142 .chip.negotiating{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
143 .chip.negotiating::before{background:var(--cyan)}
144 .chip.agreed,.chip.paid{background:var(--green-dim);color:var(--green);border:1px solid
145 rgba(16,212,142,0.2)} .chip.agreed::before,.chip.paid::before{background:var(--green)}
146 .chip.closed{background:var(--bg4);color:var(--text2);border:1px solid var(--border)}
147 .chip.closed::before{background:var(--text2)}
148
149 .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--font);font-size:12px;font-
150 weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.3px}
151 .btn-primary{background:var(--cyan);color:#080b10}
152 .btn-primary:hover{background:#33dbff;box-shadow:0 0 20px var(--cyan-glow)}
153 .btn-ghost{background:transparent;color:var(--text2);border:1px solid var(--border2)}
154 .btn-ghost:hover{border-color:var(--border3);color:var(--text)}
155 .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
156 .btn-danger:hover{background:var(--red);color:#fff}
157 .btn-warn{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
158 .btn-warn:hover{background:var(--amber);color:#080b10}
159 .btn-success{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
160 .btn-success:hover{background:var(--green);color:#080b10}
161 .btn-sm{padding:5px 10px;font-size:11px}
162 .btn-xs{padding:3px 8px;font-size:10px}
163 .action-row{display:flex;gap:6px;flex-wrap:wrap}
164
165 .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
166 .form-group{display:flex;flex-direction:column;gap:6px}
167 .form-group.full{grid-column:1/-1}
168 .form-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
169 spacing:1.5px;font-family:var(--mono)}
170 .form-input{background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:10px
171 13px;color:var(--text);font-family:var(--mono);font-size:12px;transition:border-color 0.2s;wi...
172 .form-input:focus{outline:none;border-color:var(--cyan);box-shadow:0 0 3px var(--cyan-dim)}
173 .form-input::placeholder{color:var(--text3)}
174 select.form-input option{background:var(--bg2)}
175 .form-actions{display:flex;gap:10px;margin-top:18px}
176
177 .empty{text-align:center;padding:40px 20px;color:var(--text3);font-family:var(--mono);font-
178 size:12px}
179 .empty-icon{font-size:32px;margin-bottom:10px;opacity:0.4}
180
181 .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:6px;z-
182 index:2000;animation:alertIn 0.25s ease;font-size:12px;font-weight:600;font-family:var(--mono);display:flex;a...
183 .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
184 rgba(16,212,142,0.3)}
185 .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.3)}
186 @keyframes alertIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}

```



```

163
164     .search-row{display:flex;gap:10px;padding:14px 16px;border-bottom:1px solid var(--border)}
165     .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-
radius:6px;padding:8px 13px;color:var(--text);font-family:var(--mono);font-size:12px}
166     .search-input:focus{outline:none;border-color:var(--cyan)}
167     .search-input::placeholder{color:var(--text3)}
168
169     .timeline{padding:20px;display:flex;flex-direction:column;gap:0}
170     .tl-item{display:flex;gap:14px;padding-bottom:20px;position:relative}
171     .tl-item:last-child{padding-bottom:0}
172     .tl-item:not(:last-
child)::after{content:'';position:absolute;left:14px;top:28px;bottom:0;width:1px;background:var(--border)}
173     .tl-dot{width:28px;height:28px;border-radius:50%;border:2px solid var(--border2);background:var(--
bg3);display:flex;align-items:center;justify-content:center;flex-shrink:0;font-size:11px;z-...
174     .tl-dot.cyan{border-color:var(--cyan);background:var(--cyan-dim)}
175     .tl-dot.amber{border-color:var(--amber);background:var(--amber-dim)}
176     .tl-dot.green{border-color:var(--green);background:var(--green-dim)}
177     .tl-dot.red{border-color:var(--red);background:var(--red-dim)}
178     .tl-content{flex:1;padding-top:4px}
179     .tl-title{font-size:12px;font-weight:600;color:var(--text);margin-bottom:2px}
180     .tl-meta{font-size:10px;color:var(--text3);font-family:var(--mono)}
181
182     /* Outcome hint box */
183     .outcome-hint{margin:12px 0 0;padding:10px 14px;border-radius:6px;font-size:11px;font-family:var(--
mono);transition:all 0.2s}
184     .outcome-hint.success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)}
185     .outcome-hint.danger{background:var(--red-dim);color:var(--red);border:1px solid
rgba(240,84,84,0.2)}
186
187     /* Inline repair panel */
188     .repair-inline{background:var(--bg3);border:1px solid var(--border2);border-radius:8px;padding:18px
20px;margin:0 16px 16px;animation:viewIn 0.2s ease}
189     .negotiation-grid{display:grid;grid-template-columns:minmax(0,1fr) 360px;gap:18px;align-
items:start}
190     .message-list{display:flex;flex-direction:column;gap:10px;max-height:360px;overflow-
y:auto;margin:14px 0}
191     .message-item{background:var(--bg3);border:1px solid var(--border);border-radius:8px;padding:10px
12px}
192     .message-meta{font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;margin-bottom:4px}
193
194     .modal-overlay{display:none;position:fixed;inset:0;background:rgba(8,11,16,0.85);z-
index:1500;align-items:center;justify-content:center;backdrop-filter:blur(4px)}
195     .modal-overlay.open{display:flex}
196     .modal{background:var(--panel2);border:1px solid var(--border2);border-radius:12px;width:680px;max-
width:95%;max-height:85vh;overflow-y:auto;animation:modalIn 0.2s ease}
197     @keyframes modalIn{from{opacity:0;transform:scale(0.96)}to{opacity:1;transform:scale(1)}}
198     .modal-head{padding:20px 24px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
199     .modal-title{font-size:16px;font-weight:700;color:var(--text)}
200     .modal-close{background:none;border:none;color:var(--text3);font-size:20px;cursor:pointer;padding:0
4px;transition:color 0.2s}
201     .modal-close:hover{color:var(--text)}
202     .modal-body{padding:24px}
203     .detail-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-bottom:20px}
204     .detail-item{display:flex;flex-direction:column;gap:4px}
205     .detail-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:1.5px;font-family:var(--mono)}
206     .detail-value{font-size:13px;color:var(--text);font-family:var(--mono)}
207
208     /* Repair log card inside modal */
209     .repair-log-card{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:16px;margin-bottom:10px;transition:border-color 0.2s}
210     .repair-log-card:hover{border-color:var(--border2)}
211     .repair-log-card:last-child{margin-bottom:0}
212     .repair-log-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:12px 20px;margin-bottom:10px}
213     .repair-log-remarks{padding:8px 12px;background:var(--panel);border:1px solid var(--border);border-
radius:5px;font-size:11px;font-family:var(--mono);color:var(--text2);line-height:1.6;margi...
214     .repair-log-header{display:flex;align-items:center;justify-content:space-between;margin-
bottom:12px;padding-bottom:10px;border-bottom:1px solid var(--border)}
215     .repair-log-id{font-size:10px;font-family:var(--mono);color:var(--text3)}
216
217     /* INVOICE CARDS */
218     .invoice-card-grid {
219         display: grid;

```

```
220     grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
221     gap: 16px;
222 }
223
224 .invoice-card {
225     background: var(--panel);
226     border: 1px solid var(--inv-border, var(--border2));
227     border-radius: 10px;
228     overflow: hidden;
229     transition: border-color 0.2s, transform 0.15s;
230     position: relative;
231 }
232
233 .invoice-card:hover {
234     border-color: var(--inv-accent);
235     transform: translateY(-2px);
236 }
237
238 .invoice-card-strip {
239     height: 3px;
240     background: var(--inv-accent);
241     width: 100%;
242 }
243
244 .invoice-card-inner {
245     padding: 18px 20px 20px;
246 }
247
248 .invoice-card-header {
249     display: flex;
250     justify-content: space-between;
251     align-items: flex-start;
252     margin-bottom: 16px;
253 }
254
255 .invoice-type-icon {
256     width: 38px;
257     height: 38px;
258     border-radius: 8px;
259     background: var(--inv-dim);
260     border: 1px solid var(--inv-border);
261     display: flex;
262     align-items: center;
263     justify-content: center;
264     font-size: 18px;
265     flex-shrink: 0;
266 }
267
268 .invoice-card-id {
269     font-size: 10px;
270     font-family: var(--mono);
271     color: var(--inv-accent);
272     font-weight: 700;
273     letter-spacing: 1.5px;
274     margin-bottom: 2px;
275 }
276
277 .invoice-card-type {
278     font-size: 13px;
279     font-weight: 700;
280     color: var(--text);
281 }
282
283 .invoice-amount-row {
284     background: var(--inv-dim);
285     border: 1px solid var(--inv-border);
286     border-radius: 8px;
287     padding: 12px 16px;
288     display: flex;
289     align-items: center;
290     justify-content: space-between;
291 }
292
293 .invoice-amount-label {
294     font-size: 10px;
295     font-weight: 700;
```

```

296     color: var(--text3);
297     text-transform: uppercase;
298     letter-spacing: 1px;
299     font-family: var(--mono);
300 }
301
302 .invoice-amount {
303     font-size: 24px;
304     font-weight: 800;
305     color: var(--inv-accent);
306     font-family: var(--mono);
307     letter-spacing: -0.5px;
308 }
309
310 .invoice-meta-grid {
311     display: grid;
312     grid-template-columns: 1fr 1fr;
313     gap: 12px;
314 }
315
316 .invoice-meta-item {
317     display: flex;
318     flex-direction: column;
319     gap: 3px;
320 }
321
322 .invoice-meta-label {
323     font-size: 9px;
324     font-weight: 700;
325     color: var(--text3);
326     text-transform: uppercase;
327     letter-spacing: 1.5px;
328     font-family: var(--mono);
329 }
330
331 .invoice-meta-value {
332     font-size: 12px;
333     color: var(--text2);
334     font-family: var(--mono);
335 }
336
337 .invoice-card-footer {
338     display: flex;
339     justify-content: space-between;
340     align-items: center;
341     gap: 12px;
342 }
343 </style>
344 </head>
345 <body>
346 <div class="app">
347
348     <!-- SIDEBAR -->
349     <div class="sidebar">
350         <div class="logo">
351             <div class="logo-mark">
352                 <div class="logo-hex">TV</div>
353                 <div>
354                     <div class="logo-name">TechVendor</div>
355                     <div class="logo-sub">Asset Hub</div>
356                 </div>
357             </div>
358             <div class="vendor-badge"><span class="badge-dot"></span>Tech Vendor</div>
359         </div>
360
361         <nav class="nav">
362             <div class="nav-section">Overview</div>
363             <div class="nav-item active" onclick="showView('dashboard',this)">
364                 <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="1" width="6"
height="6" rx="1" stroke="currentColor" stroke-width="1.5"/><rect x="9" y="1" width="6" height="6" ...
365                 Dashboard
366             </div>
367
368             <div class="nav-section">Devices</div>
369             <div class="nav-item" onclick="showView('my-devices',this)">
370                 <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="3" width="14"

```

```
height="10" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M5 13v2M11 13v2M3 15h10" str...
371         My Devices
372     </div>
373     <div class="nav-item" onclick="showView('add-device',this)">
374         <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="8" r="7"
375         stroke="currentColor" stroke-width="1.5"/><path d="M8 5v6M5 8h6" stroke="currentColor" stroke-width...
376     </div>
377
378     <div class="nav-section">Service</div>
379     <div class="nav-item" onclick="showView('service-requests',this)">
380         <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><path d="M8 11l1.5 4.5H14l-3.5 2.5 1.5
381         4.5L8 10l-4 2.5 1.5-4.5L2 5.5h4.5L8 12" stroke="currentColor" stroke-width="1.5" stroke-l...
382         Service Requests
383         <span class="nav-badge amber" id="sr-badge" style="display:none">0</span>
384     </div>
385     <div class="nav-item" onclick="showView('negotiations',this)">
386         <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><path d="M2 4.5h12M2 8h12M2 11.5h7"
387         stroke="currentColor" stroke-width="1.5" stroke-linecap="round"/><circle cx="12" cy="11.5" ...
388     </div>
389     <div class="nav-item" onclick="showView('invoices',this)">
390         <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="2" width="14"
391         height="12" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M4 6h8M4 9h5" stroke="curren...
392         Invoices
393         <span class="nav-badge amber" id="invoice-badge" style="display:none">0</span>
394     </div>
395
396     <div class="nav-section">Account</div>
397     <div class="nav-item" onclick="showView('two-factor',this)">
398         <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="7" width="14"
399         height="8" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M4 7V5a4 4 0 1 8 0v2" strok...
400         Two-Factor Auth
401     </div>
402     <div class="nav-item" onclick="showView('profile',this)">
403         <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="5" r="3"
404         stroke="currentColor" stroke-width="1.5"/><path d="M2 14c0-3.3 2.7-6 6-6s6 2.7 6 6" stroke="current...
405         My Profile
406     </div>
407 </nav>
408
409 <div class="sidebar-footer">
410     <div class="user-row">
411         <div class="avatar" id="sidebar-avatar">TV</div>
412         <div>
413             <div class="user-name" id="sidebar-name">TechVendor</div>
414             <div class="user-role">Tech Vendor</div>
415         </div>
416     </div>
417 </div>
418
419 <!-- MAIN -->
420 <div class="main">
421     <div class="topbar">
422         <div class="page-title">
423             <span class="page-title-prefix">// </span>
424             <span id="pageTitle">dashboard</span>
425         </div>
426         <div class="topbar-right">
427             <span class="topbar-email" id="topbar-email"></span>
428             <button class="signout-btn" onclick="logout()">Sign Out</button>
429         </div>
430     </div>
431
432     <div class="content">
433
434         <!-- DASHBOARD -->
435         <div id="dashboard-view" class="view active">
436             <div class="stats-row">
437                 <div class="stat-c">
438                     <div class="stat-label">Total Devices</div>
439                     <div class="stat-val" id="stat-total">--</div>
440                     <div class="stat-sub">in catalog</div>
441                     <div class="stat-icon"></div>
442                 </div>
443             </div>
444         </div>
445     </div>
446 </div>
```

```

440         <div class="stat g">
441             <div class="stat-label">Available</div>
442             <div class="stat-val" id="stat-avail">--</div>
443             <div class="stat-sub">ready to assign</div>
444             <div class="stat-icon"></div>
445         </div>
446         <div class="stat a">
447             <div class="stat-label">Pending Repairs</div>
448             <div class="stat-val" id="stat-repair">--</div>
449             <div class="stat-sub">awaiting your update</div>
450             <div class="stat-icon"></div>
451         </div>
452         <div class="stat r">
453             <div class="stat-label">Open Requests</div>
454             <div class="stat-val" id="stat-open">--</div>
455             <div class="stat-sub">sent for repair</div>
456             <div class="stat-icon">!</div>
457         </div>
458     </div>
459
460     <div class="dash-grid">
461         <div class="card">
462             <div class="card-head">
463                 <div>
464                     <div class="card-title">Repair Queue</div>
465                     <div class="card-sub">devices sent to you for repair</div>
466                 </div>
467                 <button class="btn btn-ghost btn-sm" onclick="showView('service-
requests',null)">View all </button>
468             </div>
469             <div class="tablewrap">
470                 <table>
471                     <thead><tr><th>Device</th><th>Issue</th><th>Status</th></tr></thead>
472                     <tbody id="dash-sr"></tbody>
473                 </table>
474             </div>
475         </div>
476
477         <div class="card">
478             <div class="card-head">
479                 <div>
480                     <div class="card-title">Active Repairs</div>
481                     <div class="card-sub">currently being serviced</div>
482                 </div>
483                 <button class="btn btn-ghost btn-sm" onclick="showView('service-
requests',null)">View all </button>
484             </div>
485             <div class="tablewrap">
486                 <table>
487                     <thead><tr><th>Device</th><th>Sent</th><th>Status</th></tr></thead>
488                     <tbody id="dash-repairs"></tbody>
489                 </table>
490             </div>
491         </div>
492     </div>
493
494     <div class="card">
495         <div class="card-head">
496             <div class="card-title">Device Status Overview</div>
497         </div>
498         <div class="tablewrap">
499             <table>
500                 <thead><tr><th>Device</th><th>Type</th><th>Status</th><th>Warranty</th><th>Rent/Month</th></tr></thead>
501                 <tbody id="dash-devices"></tbody>
502             </table>
503         </div>
504     </div>
505 </div>
506
507 <!-- MY DEVICES -->
508 <div id="my-devices-view" class="view">
509     <div class="card">
510         <div class="card-head">
511             <div>
512                 <div class="card-title">My Device Catalog</div>

```

```

513         <div class="card-sub">all devices supplied by you</div>
514     </div>
515     <div class="action-row">
516         <select class="form-input btn-sm" id="device-filter" onchange="filterDevices()"
517         style="width:140px;padding:5px 10px;font-size:11px">
518             <option value="">All Status</option>
519             <option value="AVAILABLE">Available</option>
520             <option value="ASSIGNED">Assigned</option>
521             <option value="UNDER_REPAIR">Under Repair</option>
522             <option value="CONDEMNED">Condemned</option>
523             <option value="RETURNED_TO_VENDOR">Returned</option>
524         </select>
525         <button class="btn btn-ghost btn-sm" onclick="loadMyDevices()">
Refresh</button>
526     </div>
527 </div>
528 <div class="search-row">
529     <input class="search-input" id="device-search" placeholder="Search by name, serial,
brand..." oninput="filterDevices()">
530 </div>
531 <div class="tablewrap">
532     <table>
533         <thead><tr>
534             <th>ID</th>
535             <th>Device Name</th>
536             <th>Brand</th>
537             <th>Type</th>
538             <th>Warranty</th>
539             <th>Rent/Month</th>
540             <th>Status</th>
541             <th>Actions</th>
542         </tr></thead>
543         <tbody id="devices-list"></tbody>
544     </table>
545 </div>
546 </div>
547
548 <!-- ADD DEVICE -->
549 <div id="add-device-view" class="view">
550     <div class="card" style="max-width:700px">
551         <div class="card-head">
552             <div>
553                 <div class="card-title">Add New Device</div>
554                 <div class="card-sub">register a device to your catalog</div>
555             </div>
556         </div>
557         <div class="card-body">
558             <div class="form-grid">
559                 <div class="form-group">
560                     <label class="form-label">Device Name</label>
561                     <input class="form-input" id="d-name" placeholder="Dell XPS 15 9520">
562                 </div>
563                 <div class="form-group">
564                     <label class="form-label">Brand</label>
565                     <input class="form-input" id="d-brand" placeholder="Dell">
566                 </div>
567                 <div class="form-group">
568                     <label class="form-label">Device Type</label>
569                     <select class="form-input" id="d-type">
570                         <option value="LAPTOP">Laptop</option>
571                         <option value="DESKTOP">Desktop</option>
572                         <option value="PHONE">Phone</option>
573                         <option value="TABLET">Tablet</option>
574                         <option value="MONITOR">Monitor</option>
575                         <option value="KEYBOARD">Keyboard</option>
576                         <option value="MOUSE">Mouse</option>
577                         <option value="HEADSET">Headset</option>
578                         <option value="OTHER">Other</option>
579                     </select>
580                 </div>
581                 <div class="form-group">
582                     <label class="form-label">Warranty Expiry</label>
583                     <input class="form-input" type="date" id="d-warranty">
584                 </div>
585                 <div class="form-group">

```

```

586             <label class="form-label">Rent Per Month (</label>
587             <input class="form-input" type="number" id="d-rent" placeholder="5000"
step="0.01" min="0">
588         </div>
589     </div>
590     <div class="form-actions">
591         <button class="btn btn-primary" onclick="addDevice()">Register Device</button>
592         <button class="btn btn-ghost" onclick="clearDeviceForm()">Clear</button>
593     </div>
594 </div>
595 </div>
596 </div>
597
598 <!-- SERVICE REQUESTS -->
599 <div id="service-requests-view" class="view">
600     <div class="card">
601         <div class="card-head">
602             <div>
603                 <div class="card-title">Service Requests</div>
604                 <div class="card-sub">all requests for your devices – expand to see repair
log</div>
605             </div>
606             <div class="action-row">
607                 <select class="form-input btn-sm" id="sr-filter"
onchange="filterServiceRequests()" style="width:160px;padding:5px 10px;font-size:11px">
608                     <option value="">All Status</option>
609                     <option value="SENT_FOR_REPAIR">Sent for Repair</option>
610                     <option value="REPAIR_DONE">Repair Done</option>
611                     <option value="IRREPARABLE">Irreparable</option>
612                     <option value="OPEN">Open</option>
613                     <option value="ACKNOWLEDGED">Acknowledged</option>
614                     <option value="RESOLVED">Resolved</option>
615                     <option value="REJECTED">Rejected</option>
616                 </select>
617                 <button class="btn btn-ghost btn-sm" onclick="loadServiceRequests()">
Refresh</button>
618             </div>
619         </div>
620         <div class="search-row">
621             <input class="search-input" id="sr-search" placeholder="Search by device, type,
description..." oninput="filterServiceRequests()">
622         </div>
623         <div class="tablewrap">
624             <table>
625                 <thead>
626                     <tr>
627                         <th></th>
628                         <th>ID</th>
629                         <th>Device</th>
630                         <th>Type</th>
631                         <th>Description</th>
632                         <th>Urgent</th>
633                         <th>Status</th>
634                         <th>Raised</th>
635                         <th>Actions</th>
636                     </tr>
637                 </thead>
638                 <tbody id="sr-list"></tbody>
639             </table>
640         </div>
641     </div>
642
643 <!-- INLINE REPAIR UPDATE PANEL -->
644 <div class="card" id="vendor-repair-panel" style="max-width:640px;display:none">
645     <div class="card-head">
646         <div>
647             <div class="card-title">Update Repair Result</div>
648             <div class="card-sub" id="vr-panel-subtitle">Request #-</div>
649         </div>
650         <button class="btn btn-ghost btn-sm" onclick="closeRepairPanel()"> Cancel</button>
651     </div>
652     <div class="card-body">
653         <input type="hidden" id="vr-request-id">
654         <div class="form-grid">
655             <div class="form-group full">
656                 <label class="form-label">Repair Outcome</label>

```

```

657         <select class="form-input" id="vr-status" onchange="onOutcomeChange()">
658             <option value="REPAIR_DONE"> Repair Done – device returns to
employee</option>
659             <option value="IRREPARABLE"> Could Not Repair – device will be
condemned</option>
660         </select>
661     </div>
662     <div class="form-group">
663         <label class="form-label">Repair Cost ( ) <span style="color:var(--
text3);font-weight:400">optional</span></label>
664         <input class="form-input" id="vr-repair-cost" type="number" min="0"
step="0.01" placeholder="e.g. 2500">
665     </div>
666     <div class="form-group">
667         <label class="form-label">Resolution Notes</label>
668         <input class="form-input" id="vr-resolution" placeholder="What was
repaired, parts replaced, or why it couldn't be fixed...">
669     </div>
670     <div class="form-group">
671         <label class="form-label">Damaged Component <span style="color:var(--
text3);font-weight:400">optional</span></label>
672         <input class="form-input" id="vr-damaged-component" placeholder="e.g.
Motherboard, Screen...">
673     </div>
674     <div class="form-group">
675         <label class="form-label">Replaced Component <span style="color:var(--
text3);font-weight:400">optional</span></label>
676         <input class="form-input" id="vr-replaced-component" placeholder="e.g. RAM,
Battery...">
677     </div>
678 </div>
679 <div id="vr-outcome-hint" class="outcome-hint success">
680     Device status <strong>ASSIGNED</strong> – returned to the employee
681 </div>
682 <div class="form-actions">
683     <button class="btn btn-primary" onclick="submitRepairUpdate()">Submit
Update</button>
684     <button class="btn btn-ghost" onclick="closeRepairPanel()">Cancel</button>
685 </div>
686 </div>
687 </div>
688 </div>
689
690 <!-- NEGOTIATIONS -->
691 <div id="negotiations-view" class="view">
692     <div class="negotiation-grid">
693         <div>
694             <div class="card">
695                 <div class="card-head">
696                     <div>
697                         <div class="card-title">Negotiations</div>
698                         <div class="card-sub">messages and payment discussions with admin</div>
699                     </div>
700                     <button class="btn btn-ghost btn-sm" onclick="loadNegotiations()">
Refresh</button>
701                 </div>
702                 <div class="tablewrap">
703                     <table>
704 <thead><tr><th>ID</th><th>Device</th><th>Subject</th><th>Amount</th><th>Status</th><th>Updated</th><th>Actions</
th></tr></thead>
705 <tbody id="negotiations-list"><tr><td colspan="7"><div
class="empty">Loading negotiations...</div></td></tr></tbody>
706 </table>
707 </div>
708 </div>
709 <div class="card">
710     <div class="card-head">
711         <div>
712             <div class="card-title">Start Negotiation</div>
713             <div class="card-sub">raise a new repair, pricing, or billing
discussion</div>
714         </div>
715     </div>
716     <div class="card-body">
717         <div class="form-grid">

```



```

718             <div class="form-group">
719                 <label class="form-label">Device</label>
720                 <select class="form-input" id="vendor-neg-device"><option
value="">No device</option></select>
721             </div>
722             <div class="form-group">
723                 <label class="form-label">Proposed Amount</label>
724                 <input class="form-input" id="vendor-neg-amount" type="number"
min="0" step="0.01" placeholder="e.g. 2500">
725             </div>
726             <div class="form-group full">
727                 <label class="form-label">Subject</label>
728                 <input class="form-input" id="vendor-neg-subject"
placeholder="Repair estimate or billing discussion">
729             </div>
730             <div class="form-group full">
731                 <label class="form-label">Description</label>
732                 <textarea class="form-input" id="vendor-neg-description" rows="3"
placeholder="Give the admin the context for this negotiation"></textarea>
733             </div>
734             <div class="form-group full">
735                 <label class="form-label">Initial Message</label>
736                 <textarea class="form-input" id="vendor-neg-message" rows="3"
placeholder="Write your opening message"></textarea>
737             </div>
738         </div>
739         <div class="form-actions">
740             <button class="btn btn-primary" onclick="createNegotiation()">Create
Negotiation</button>
741             <button class="btn btn-ghost"
onclick="clearNegotiationForm()">Clear</button>
742         </div>
743     </div>
744 </div>
745 </div>
746 <div class="card">
747     <div class="card-head">
748         <div>
749             <div class="card-title" id="neg-detail-title">Negotiation Detail</div>
750             <div class="card-sub">select a row to reply</div>
751         </div>
752     </div>
753     <div class="card-body" id="neg-detail-body">
754         <div class="empty">No negotiation selected</div>
755     </div>
756 </div>
757 </div>
758 </div>
759
760
761 <!-- INVOICES -->
762 <div id="invoices-view" class="view">
763     <div class="card">
764         <div class="card-head">
765             <div>
766                 <div class="card-title">My Invoices</div>
767                 <div class="card-sub">payment invoices sent by admin – acknowledge to confirm
receipt</div>
768             </div>
769             <button class="btn btn-ghost btn-sm" onclick="loadInvoices()"> Refresh</button>
770         </div>
771         <div style="padding:20px">
772             <div id="invoices-container" class="invoice-card-grid"></div>
773         </div>
774     </div>
775 </div>
776
777 <!-- TWO-FACTOR AUTH -->
778 <div id="two-factor-view" class="view">
779     <div class="card" style="max-width:560px">
780         <div class="card-head">
781             <div>
782                 <div class="card-title">Two-Factor Authentication</div>
783                 <div class="card-sub">secure your account with an authenticator app</div>
784             </div>
785             <span id="totp-status-chip" class="chip">checking...</span>

```

```

786         </div>
787         <div class="card-body">
788             <div id="totp-setup-section">
789                 <p style="font-size:12px;color:var(--text2);margin-bottom:16px;line-
height:1.7;font-family:var(--mono)">
790                     Use Google Authenticator, Authy, or any TOTP app.<br>
791                     Scan the QR code, then enter the 6-digit code to activate.
792                 </p>
793                 <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-
btn">Generate QR Code</button>
794                 <div id="qr-area" style="display:none;margin-top:24px;text-align:center">
795                     <img id="qr-img" src="" alt="QR Code"
style="width:180px;height:180px;border-radius:8px;border:1px solid var(--border2);display:block;margin:0 auto">
796                     <div style="margin-top:12px;font-size:10px;color:var(--text3);font-
family:var(--mono);letter-spacing:0.5px">CAN'T SCAN? ENTER KEY MANUALLY:</div>
797                     <div id="totp-secret-display" style="font-family:var(--mono);font-
size:12px;color:var(--cyan);background:var(--bg3);border:1px solid var(--border2);border-radius:6px...
798                     <div style="margin-top:20px;text-align:left">
799                         <label class="form-label" style="display:block;margin-bottom:8px">Enter
6-digit code to activate</label>
800                         <input id="totp-verify-input" type="text" inputmode="numeric"
maxlength="6" placeholder="000000" class="form-input" style="text-align:center;font-size:20px;lette...
801                         <button class="btn btn-primary" onclick="verifyAndActivateTotp()"
style="width:100%;margin-top:10px">Activate 2FA</button>
802                         <div id="totp-verify-error" style="display:none;margin-top:8px;font-
size:11px;color:var(--red);font-family:var(--mono)"></div>
803                     </div>
804                 </div>
805             </div>
806             <div id="totp-disable-section" style="display:none">
807                 <div style="background:var(--green-dim);border:1px solid
rgba(16,212,142,0.2);border-radius:6px;padding:12px 16px;font-size:12px;color:var(--green);font-family:var(--
mon...
808                     TWO-FACTOR AUTHENTICATION IS ACTIVE
809                 </div>
810                 <button class="btn btn-danger" onclick="disableTotp()">Disable 2FA</button>
811             </div>
812         </div>
813     </div>
814 </div>
815
816 <!-- PROFILE -->
817 <div id="profile-view" class="view">
818     <div class="card" style="max-width:600px">
819         <div class="card-head">
820             <div class="card-title">My Profile</div>
821         </div>
822         <div class="card-body">
823             <div style="display:flex;align-items:center;gap:16px;margin-bottom:24px;padding-
bottom:20px;border-bottom:1px solid var(--border)">
824                 <div style="width:56px;height:56px;background:var(--cyan-dim);border:2px solid
var(--border2);border-radius:10px;display:flex;align-items:center;justify-content:center;f...
825                 <div>
826                     <div style="font-size:18px;font-weight:700;color:var(--text)" id="profile-
name">--</div>
827                     <div style="font-size:11px;color:var(--cyan);font-family:var(--
mono);margin-top:3px">TECH_VENDOR</div>
828                 </div>
829             </div>
830             <div class="detail-grid">
831                 <div class="detail-item"><div class="detail-label">Email</div><div
class="detail-value" id="profile-email">--</div></div>
832                 <div class="detail-item"><div class="detail-label">Phone</div><div
class="detail-value" id="profile-phone">--</div></div>
833                 <div class="detail-item"><div class="detail-label">Registered</div><div
class="detail-value" id="profile-reg">--</div></div>
834                 <div class="detail-item"><div class="detail-label">Status</div><div
class="detail-value"><span class="chip active_assign">ACTIVE</span></div></div>
835             </div>
836         </div>
837     </div>
838 </div>
839
840 </div>
841 </div>
842 </div>

```

```

843
844 <!-- DEVICE DETAIL MODAL -->
845 <div class="modal-overlay" id="device-modal">
846   <div class="modal">
847     <div class="modal-head">
848       <div>
849         <div class="modal-title" id="modal-device-name">Device Details</div>
850         <div style="font-size:11px;color:var(--text3);font-family:var(--mono);margin-top:2px"
id="modal-device-id"></div>
851       </div>
852       <button class="modal-close" onclick="closeModal()"></button>
853     </div>
854     <div class="modal-body">
855       <div class="detail-grid" id="modal-detail-grid"></div>
856
857       <!-- REPAIR HISTORY SECTION -->
858       <div style="margin-top:16px">
859         <div style="display:flex;align-items:center;justify-content:space-between;margin-
bottom:14px">
860           <div style="font-size:10px;font-weight:700;color:var(--text3);font-family:var(--
mono);text-transform:uppercase;letter-spacing:1.5px">
861             Repair History
862           </div>
863           <button class="btn btn-ghost btn-xs" id="modal-refresh-logs-btn"
onclick="refreshModalRepairLogs()"> Refresh</button>
864         </div>
865         <div id="modal-repair-history">
866           <div class="empty" style="padding:20px">
867             <div class="empty-icon"></div>
868             Loading repair history...
869           </div>
870         </div>
871       </div>
872     </div>
873   </div>
874 </div>
875
876 <script>
877   const API = 'http://localhost:8080';
878   let devicesCache = [], srCache = [], srAllCache = [], assignmentsCache = [], negotiationsCache = [];
879   let underRepairCache = [];
880   let selectedNegotiationId = null;
881   let currentModalDeviceId = null; // track which device is open in modal
882
883   // AUTH
884   (function(){
885     const token = localStorage.getItem('jwt_token');
886     if(!token){ window.location.href='/login.html'; return; }
887     try{
888       const p = JSON.parse(atob(token.split('.')[1]));
889       const role = (p.role||'').replace(/^ROLE_/, '');
890       if(role !== 'TECH_VENDOR'){ window.location.href='/google.html'; return; }
891       const email = p.sub||p.email||'TechVendor';
892       document.getElementById('topbar-email').textContent = email;
893       document.getElementById('sidebar-name').textContent = email.split('@')[0];
894       document.getElementById('sidebar-avatar').textContent = email.charAt(0).toUpperCase();
895       document.getElementById('profile-avatar').textContent = email.charAt(0).toUpperCase();
896       document.getElementById('profile-email').textContent = email;
897     }catch(e){ window.location.href='/login.html'; }
898     initDashboard();
899   })();
900
901   function getAuthHeader(){
902     const t = localStorage.getItem('jwt_token');
903     return t ? {'Authorization': `Bearer ${t}`} : {};
904   }
905
906   async function apiFetch(path, opts={}){
907     const res = await fetch(`${API}${path}`, {
908       ...opts,
909       headers: {'Content-Type': 'application/json', ...getAuthHeader(), ...(opts.headers||{})},
910       credentials: 'include'
911     });
912     if(res.status===401){
913       localStorage.removeItem('jwt_token');
914       window.location.href='/login.html';

```

```

915     throw new Error('Unauthorized');
916 }
917 const text = await res.text();
918 if(!res.ok) throw new Error(text || 'Request failed');
919 if(!text) return null;
920 try { return JSON.parse(text); } catch { return text; } // falls back to raw string
921 }
922
923 // NAV
924 function showView(name, el){
925     document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
926     document.getElementById(`${name}-view`).classList.add('active');
927     document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
928     if(el) el.classList.add('active');
929     document.getElementById('pageTitle').textContent = name.replace(/-/g, ' ');
930     const loaders = {
931         'dashboard': loadDashboard,
932         'my-devices': loadMyDevices,
933         'service-requests': loadServiceRequests,
934         'negotiations': loadNegotiations,
935         'assignments': loadAssignments,
936         'profile': loadProfile,
937         'two-factor': loadTotpStatus,
938         'add-device': ()=>{},
939         'invoices': loadInvoices
940     };
941     if(loaders[name]) loaders[name]();
942 }
943
944 // INIT
945 async function initDashboard(){
946     await Promise.all([loadMyDevices(), loadServiceRequests(), loadAssignments(),
947     loadNegotiations(),loadInvoices()]);
948     loadDashboard();
949 }
950
951 // DASHBOARD
952 function loadDashboard() {
953     document.getElementById('stat-total').textContent = devicesCache.length || 0;
954     document.getElementById('stat-avail').textContent = devicesCache.filter(d => d.deviceStatus ===
955     'AVAILABLE').length || 0;
956     document.getElementById('stat-repair').textContent = underRepairCache.length || 0; // devices actively
957     under repair
958     document.getElementById('stat-open').textContent = srCache.length || 0; // awaiting receipt
959
960     // Repair Queue = awaiting receipt
961     document.getElementById('dash-sr').innerHTML = srCache.slice(0,5).map(s => `
962     <tr>
963         <td class="main-cell">${s.deviceName || 'Device #' + s.deviceId}</td>
964         <td>${(s.requestType||'').toLowerCase().replace('_', ' ')}</td>
965         <td>${chip('SENT_FOR_REPAIR')}</td>
966     </tr>`).join('') || `<tr><td colspan="3" class="empty">No pending receipts</td></tr>`;
967
968     // Active Repairs = under repair devices
969     document.getElementById('dash-repairs').innerHTML = underRepairCache.slice(0,5).map(d => `
970     <tr>
971         <td class="main-cell">${d.deviceName || '-'}</td>
972         <td>${d.brand || '-'}</td>
973         <td>${chip(d.deviceStatus)}</td>
974     </tr>`).join('') || `<tr><td colspan="3" class="empty">No active repairs</td></tr>`;
975
976     // Device overview unchanged
977     document.getElementById('dash-devices').innerHTML = devicesCache.slice(0,8).map(d => `
978     <tr>
979         <td class="main-cell">${d.deviceName || '-'}</td>
980         <td>${chip(d.deviceType)}</td>
981         <td>${chip(d.deviceStatus)}</td>
982         <td>${d.warrantyExpiryDate || '-'}</td>
983         <td>${d.rentPerMonth || '-'}</td>
984     </tr>`).join('') || `<tr><td colspan="5" class="empty">No devices</td></tr>`;
985 }
986
987 // DEVICES
988 async function loadMyDevices(){
989     try{
990         const data = await apiFetch('/devices/vendor/me');

```

```

988     devicesCache = data || [];
989     renderDevices(devicesCache);
990 }catch(e){ renderDevices([]); }
991 }
992
993 function renderDevices(list){
994     document.getElementById('devices-list').innerHTML = list.length
995     ? list.map(d=><tr>
996         <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${d.id}</td>
997         <td class="main-cell">${d.deviceName||'-'}</td>
998         <td>${d.brand||'-'}</td>
999         <td>${chip(d.deviceType)}</td>
1000         <td style="font-size:11px;font-family:var(--mono)">
1001             ${d.warrantyExpiryDate
1002             ? (new Date(d.warrantyExpiryDate) > new Date()
1003             ? `<span style="color:var(--green)"> ${d.warrantyExpiryDate}</span>`
1004             : `<span style="color:var(--red)"> ${d.warrantyExpiryDate}</span>`)
1005             : `<span style="color:var(--text3)">—</span>`}
1006         </td>
1007         <td style="font-family:var(--mono);color:var(--cyan)">
1008             ${d.rentPerMonth ? `'+d.rentPerMonth : `<span style="color:var(--text3)">—</span>`}
1009         </td>
1010         <td>${chip(d.deviceStatus)}</td>
1011         <td><div class="action-row">
1012             <button class="btn btn-ghost btn-xs" onclick="viewDevice(${d.id})">Details</button>
1013             ${d.deviceStatus === 'RETURNED_TO_VENDOR'
1014             ? `<button class="btn btn-danger btn-xs" onclick="removeDevice(${d.id},
1015             '${(d.deviceName||'Device').replace(/'/g, "\\')}')"> Remove</button>`
1016             : ``}
1017         </div></td>
1018         <tr><td colspan="8"><div class="empty"><div class="empty-icon"></div>No devices registered
yet</div></td></tr>`;
1019     }
1020
1021     function filterDevices(){
1022         const q = document.getElementById('device-search').value.toLowerCase();
1023         const f = document.getElementById('device-filter').value;
1024         renderDevices(devicesCache.filter(d=>{
1025             const matchQ = !q ||
1026             (d.deviceName||'').toLowerCase().includes(q)||
1027             (d.serialNumber||'').toLowerCase().includes(q)||
1028             (d.brand||'').toLowerCase().includes(q);
1029             const matchF = !f || d.deviceStatus === f;
1030             return matchQ && matchF;
1031         }));
1032     }
1033
1034     async function addDevice(){
1035         const body = {
1036             deviceName: document.getElementById('d-name').value.trim(),
1037             brand: document.getElementById('d-brand').value.trim(),
1038             deviceType: document.getElementById('d-type').value,
1039             warrantyExpiryDate: document.getElementById('d-warranty').value || null,
1040             rentPerMonth: parseFloat(document.getElementById('d-rent').value) || null
1041         };
1042         if(!body.deviceName || !body.brand || !body.deviceType){
1043             showAlert('Device name, brand and type are required','error'); return;
1044         }
1045         try{
1046             await apiFetch('/devices', {method:'POST', body:JSON.stringify(body)});
1047             showAlert('Device registered successfully!','success');
1048             clearDeviceForm();
1049             loadMyDevices();
1050         }catch(e){ showAlert('Failed: '+e.message,'error'); }
1051     }
1052
1053     function clearDeviceForm(){
1054         ['d-name','d-brand','d-warranty','d-rent'].forEach(id=>document.getElementById(id).value='');
1055         document.getElementById('d-type').value='LAPTOP';
1056     }
1057
1058     // SERVICE REQUESTS
1059     let expandedLogRow = null;
1060     async function loadServiceRequests() {
1061         try {
1062             const [sentData, underRepairData] = await Promise.all([

```

```

1060         apiFetch('/service-requests/vendor/me'),
1061         apiFetch('/service-requests/devices/repair')
1062     ]]);
1063
1064     srCache = sentData || [];
1065     srAllCache = srCache;           // keep in sync
1066     underRepairCache = underRepairData || [];
1067
1068     console.log('srCache sample:', srCache[0]);
1069     console.log('underRepairCache sample:', underRepairCache[0]);
1070
1071     const totalBadge = srCache.length;
1072     const badge = document.getElementById('sr-badge');
1073     if (totalBadge) { badge.textContent = totalBadge; badge.style.display = ''; }
1074     else badge.style.display = 'none';
1075
1076     filterServiceRequests();
1077 } catch(e) {
1078     console.error('loadServiceRequests failed:', e);
1079     renderServiceRequests([], []);
1080 }
1081 }
1082
1083 function filterServiceRequests() {
1084     const q = (document.getElementById('sr-search')?.value || '').toLowerCase();
1085     const f = document.getElementById('sr-filter')?.value || '';
1086
1087     const filteredSent = srCache.filter(s => {
1088         const matchQ = !q || (s.deviceName||'').toLowerCase().includes(q) ||
1089         (s.issueDescription||'').toLowerCase().includes(q);
1090         const matchF = !f || f === 'SENT_FOR_REPAIR';
1091         return matchQ && matchF;
1092     });
1093
1094     const filteredRepair = underRepairCache.filter(d => {
1095         const matchQ = !q || (d.deviceName||'').toLowerCase().includes(q) ||
1096         (d.brand||'').toLowerCase().includes(q);
1097         const matchF = !f || f === 'UNDER_REPAIR';
1098         return matchQ && matchF;
1099     });
1100
1101     renderServiceRequests(filteredSent, filteredRepair);
1102 }
1103
1104 function renderServiceRequests(sentList, repairList) {
1105     expandedLogRow = null;
1106
1107     // TABLE 1: Awaiting receipt (SENT_FOR_REPAIR)
1108     const sentRows = sentList.length
1109     ? sentList.map(s => `
1110         <tr id="sr-row-${s.id}">
1111             <td style="width:28px;padding:8px 6px 8px 12px">
1112                 <button class="btn btn-ghost btn-xs" style="padding:2px 6px;font-size:13px"
1113                     onclick="toggleRepairLog(${s.id}, this)"></button>
1114             </td>
1115             <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${s.id}</td>
1116             <td class="main-cell">${s.deviceName || 'Device #' + s.deviceId}</td>
1117             <td>${(s.requestType||'').toLowerCase().replace(/_/g, ' ')}</td>
1118             <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-
1119 space:nowrap;color:var(--text2)">${s.issueDescription||'-'}</td>
1120             <td>${s.urgent
1121 ? '<span style="color:var(--red);font-family:var(--mono);font-size:10px;font-
1122 weight:700"> URGENT</span>'
1123 : '<span style="color:var(--text3);font-family:var(--mono);font-
1124 size:10px">--</span>'}</td>
1125             <td>${chip(s.status)}</td>
1126             <td style="font-size:10px;font-family:var(--mono);color:var(--text3)">${s.raisedAt ?
1127 s.raisedAt.split('T')[0] : '-'}</td>
1128             <td>
1129                 <button class="btn btn-warn btn-xs"
1130                     onclick="markReceivedByVendor(${s.id},
1131 '${(s.deviceName||'').replace(/'/g, '\\')}')">
1132                     Mark Received
1133                 </button>
1134             </td>
1135         </tr>
1136     `
1137     <tr id="sr-log-${s.id}" style="display:none">

```

```

1129         <td colspan="9" style="padding:0;background:var(--bg3)">
1130             <div id="sr-log-content-${s.id}" style="padding:16px 20px">
1131                 <div style="color:var(--text3);font-family:var(--mono);font-size:11px">Loading
repair log...</div>
1132             </div>
1133         </td>
1134     </tr>`).join('')
1135     : `<tr><td colspan="9"><div class="empty">No devices awaiting receipt</div></td></tr>`;
1136
1137     // TABLE 2: Under Repair (RECEIVED_BY_VENDOR final update)
1138     const repairRows = repairList.length
1139     ? repairList.map(d => `
1140         <tr>
1141             <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${d.id}</td>
1142             <td class="main-cell">${d.deviceName || '-'}</td>
1143             <td>${d.requestType || '-'}</td>
1144             <td>${chip(d.status)}</td>
1145             <td style="font-size:10px;font-family:var(--mono);color:var(--text3)">${d.raisedAt ?
d.raisedAt.split('T')[0] : '-'}</td>
1146             <td>
1147                 <button class="btn btn-success btn-xs"
1148                     onclick="openRepairPanel(${d.id}, '${(d.deviceName||'').replace(/'/g, '\\\'')}')">
1149                     Submit Result
1150                 </button>
1151             </td>
1152         </tr>`).join('')
1153     : `<tr><td colspan="6"><div class="empty">No devices currently under repair</div></td></tr>`;
1154
1155     document.getElementById('sr-list').innerHTML = `
1156     <!-- SECTION 1 -->
1157     <tr>
1158         <td colspan="9" style="padding:10px 16px 6px;background:var(--bg2)">
1159             <span style="font-size:10px;font-weight:700;color:var(--amber);font-family:var(--mono);
1160                 text-transform:uppercase;letter-spacing:1.5px">
1161                 Awaiting Receipt – ${sentList.length} request(s)
1162             </span>
1163         </td>
1164     </tr>
1165     ${sentRows}
1166
1167     <!-- SECTION 2 HEADER (different columns) -->
1168     <tr>
1169         <td colspan="9" style="padding:16px 16px 0;background:var(--bg2)">
1170             <span style="font-size:10px;font-weight:700;color:var(--cyan);font-family:var(--mono);
1171                 text-transform:uppercase;letter-spacing:1.5px">
1172                 Under Repair – ${repairList.length} device(s)
1173             </span>
1174         </td>
1175     </tr>
1176     <tr style="background:var(--bg3)">
1177         <th style="padding:9px 16px;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;font-family:var(--mono)">ID</th>
1178         <th style="padding:9px 16px;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;font-family:var(--mono)">Device</th>
1179         <th style="padding:9px 16px;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;font-family:var(--mono)">Request Type</th>
1180         <th style="padding:9px 16px;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;font-family:var(--mono)">Status</th>
1181         <th style="padding:9px 16px;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;font-family:var(--mono)">Raised</th>
1182         <th style="padding:9px 16px;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;font-family:var(--mono)">Actions</th>
1183     </tr>
1184     ${repairRows}`;
1185 }
1186
1187 async function markReceivedByVendor(serviceRequestId, deviceName) {
1188     if (!confirm(`Confirm receipt of "${deviceName}"? Device will be marked as Under Repair.`)) return;
1189     try {
1190         await apiFetch(`/service-requests/vendor/${serviceRequestId}`, { method: 'PUT' });
1191         showAlert(`${deviceName} marked as received – now Under Repair`, 'success');
1192         await loadServiceRequests();
1193         loadDashboard();
1194     } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
1195 }
1196

```

```

1197     async function toggleRepairLog(srId, btn){
1198         const logRow = document.getElementById(`sr-log-${srId}`);
1199         const isOpen = logRow.style.display !== 'none';
1200
1201         if(expandedLogRow && expandedLogRow !== logRow){
1202             expandedLogRow.style.display = 'none';
1203             const prevId = expandedLogRow.id.replace('sr-log-', '');
1204             const prevBtn = document.querySelector(`#sr-row-${prevId} button`);
1205             if(prevBtn) prevBtn.textContent = '';
1206         }
1207
1208         if(isOpen){
1209             logRow.style.display = 'none';
1210             btn.textContent = '';
1211             expandedLogRow = null;
1212             return;
1213         }
1214
1215         logRow.style.display = '';
1216         btn.textContent = '';
1217         expandedLogRow = logRow;
1218
1219         // Find the deviceId for this service request
1220         const sr = srAllCache.find(s => s.id === srId);
1221         if(!sr || !sr.deviceId){
1222             document.getElementById(`sr-log-content-${srId}`).innerHTML =
1223             `

```



```

1262         <div class="detail-label" style="font-size:9px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:1.5px;font-family:var(--mono);margin-bottom:4px">Action Taken</...
1263         <div style="font-family:var(--mono);font-size:12px;color:var(--text)">${r.repairAction ||
'-'}</div>
1264     </div>
1265 </div>
1266     ${r.remarks ? `<div style="margin-top:10px;padding:8px 12px;background:var(--panel);border:1px
solid var(--border);border-radius:6px;font-size:11px;font-family:var(--mono);color:var(--text2...
1267     `).join('<hr style="border:none;border-top:1px solid var(--border);margin:12px 0">`);
1268     }
1269
1270 // REPAIR PANEL
1271 function openRepairPanel(requestId, deviceName){
1272     document.getElementById('vr-request-id').value = requestId;
1273     document.getElementById('vr-panel-subtitle').textContent = `Request #${requestId} – ${deviceName}`;
1274     document.getElementById('vr-status').value = 'REPAIR_DONE';
1275     document.getElementById('vr-resolution').value = '';
1276     document.getElementById('vr-repair-cost').value = '';
1277     document.getElementById('vr-damaged-component').value = '';
1278     document.getElementById('vr-replaced-component').value = '';
1279     onOutcomeChange();
1280     const panel = document.getElementById('vendor-repair-panel');
1281     panel.style.display = 'block';
1282     panel.scrollIntoView({behavior:'smooth', block:'start'});
1283 }
1284
1285 function closeRepairPanel(){
1286     document.getElementById('vendor-repair-panel').style.display = 'none';
1287     document.getElementById('vr-repair-cost').value = '';
1288     document.getElementById('vr-damaged-component').value = '';
1289     document.getElementById('vr-replaced-component').value = '';
1290 }
1291
1292 function onOutcomeChange(){
1293     const status = document.getElementById('vr-status').value;
1294     const hint = document.getElementById('vr-outcome-hint');
1295     if(status === 'REPAIR_DONE'){
1296         hint.className = 'outcome-hint success';
1297         hint.innerHTML = ' Device status <strong>ASSIGNED</strong> – returned to the employee';
1298     } else {
1299         hint.className = 'outcome-hint danger';
1300         hint.innerHTML = ' Device status <strong>CONDEMNED</strong> – device will be marked unusable';
1301     }
1302 }
1303
1304 async function submitRepairUpdate(){
1305     const requestId      = document.getElementById('vr-request-id').value;
1306     const status         = document.getElementById('vr-status').value;
1307     const resolution     = document.getElementById('vr-resolution').value.trim();
1308     const repairCost     = parseFloat(document.getElementById('vr-repair-cost').value) || null;
1309     const damagedComponent = document.getElementById('vr-damaged-component').value.trim() || null;
1310     const replacedComponent = document.getElementById('vr-replaced-component').value.trim() || null;
1311
1312     if(!resolution){
1313         showAlert('Please enter resolution notes','error'); return;
1314     }
1315
1316     const body = {
1317         requestId: parseInt(requestId),
1318         resolution,
1319         status,
1320         repairCost,
1321         damagedComponent,
1322         replacedComponent
1323     };
1324
1325     try{
1326         await apiFetch('/service-requests/vendor/finalUpdate', {method:'PUT', body:JSON.stringify(body)});
1327         const label = status === 'REPAIR_DONE'
1328             ? 'Repair marked done – device returned to employee'
1329             : 'Device condemned – marked irreparable';
1330         showAlert(label, 'success');
1331         closeRepairPanel();
1332         await Promise.all([loadServiceRequests(), loadMyDevices()]);
1333         loadDashboard();
1334     }catch(e){ showAlert('Failed: '+e.message,'error'); }

```

```

1335 }
1336
1337 // NEGOTIATIONS
1338 function escHtml(value){
1339     const div = document.createElement('div');
1340     div.textContent = value == null ? '' : String(value);
1341     return div.innerHTML;
1342 }
1343
1344 function money(value){
1345     return value == null || value === '' ? '-' : `${Number(value).toLocaleString('en-IN')}`;
1346 }
1347
1348 function populateNegotiationDevices(){
1349     const select = document.getElementById('vendor-neg-device');
1350     if(!select) return;
1351     select.innerHTML = '<option value="">No device</option>' + (devicesCache || []).map(d =>
1352         `<option value="${d.id}">${escHtml(d.deviceName || 'Device #${d.id}')}${d.deviceStatus ? ` `
1353         ${escHtml(d.deviceStatus)}` : ''}</option>`
1354     ).join('');
1355 }
1356
1357 async function loadNegotiations(){
1358     try{
1359         if(!devicesCache.length){
1360             const data = await apiFetch('/devices/vendor/me').catch(()=>[]);
1361             devicesCache = data || [];
1362         }
1363         populateNegotiationDevices();
1364         negotiationsCache = await apiFetch('/vendor-negotiations') || [];
1365         const tbody = document.getElementById('negotiations-list');
1366         if(!tbody) return;
1367         tbody.innerHTML = negotiationsCache.length ? negotiationsCache.map(n => `
1368             <tr>
1369                 <td>#${n.id}</td>
1370                 <td class="main-cell">${escHtml(n.deviceName || n.deviceId || '-')}</td>
1371                 <td>${escHtml(n.subject || '-')}</td>
1372                 <td>${money(n.proposedAmount)}</td>
1373                 <td>${chip(n.status || 'PENDING')}</td>
1374                 <td>${n.updatedAt ? new Date(n.updatedAt).toLocaleString() : '-'}</td>
1375                 <td><button class="btn btn-ghost btn-xs" onclick="openNegotiation(${n.id})">Open</button></td>
1376             </tr>`.join('') : `<tr><td colspan="7"><div class="empty">No negotiations yet</div></td></tr>`;
1377         if(selectedNegotiationId && negotiationsCache.some(n => n.id === selectedNegotiationId)){
1378             await openNegotiation(selectedNegotiationId);
1379         }
1380     }catch(e){
1381         const tbody = document.getElementById('negotiations-list');
1382         if(tbody) tbody.innerHTML = `<tr><td colspan="7"><div class="empty">Could not load
negotiations</div></td></tr>`;
1383     }
1384 }
1385
1386 async function openNegotiation(id){
1387     selectedNegotiationId = id;
1388     try{
1389         const n = await apiFetch(`/vendor-negotiations/${id}`);
1390         document.getElementById('neg-detail-title').textContent = `Negotiation #${n.id}`;
1391         document.getElementById('neg-detail-body').innerHTML = `
1392             <div class="detail-grid" style="grid-template-columns:1fr">
1393                 <div class="detail-item"><div class="detail-label">Subject</div><div class="detail-
1394                 value">${escHtml(n.subject || '-')}</div></div>
1395                 <div class="detail-item"><div class="detail-label">Device</div><div class="detail-
1396                 value">${escHtml(n.deviceName || n.deviceId || '-')}</div></div>
1397                 <div class="detail-item"><div class="detail-label">Amount</div><div class="detail-
1398                 value">${money(n.proposedAmount)}</div></div>
1399                 <div class="detail-item"><div class="detail-label">Status</div><div class="detail-
1400                 value">${chip(n.status || 'PENDING')}</div></div>
1401                 <div class="detail-item"><div class="detail-label">Description</div><div class="detail-
1402                 value">${escHtml(n.description || '-')}</div></div>
1403             </div>
1404             <div class="message-list">
1405                 ${n.messages || [].length ? n.messages.map(m => `
1406                     <div class="message-item">
1407                         <div class="message-meta">${escHtml(m.senderType || '-')} · ${escHtml(m.senderEmail ||
1408                         '')} · ${m.createdAt ? new Date(m.createdAt).toLocaleString() : ''}</div>
1409                         <div>${escHtml(m.message || '')}</div>

```

```

1403         </div>`).join('') : '<div class="empty" style="padding:16px">No messages yet</div>'}
1404     </div>
1405     <div class="form-group">
1406         <label class="form-label">Reply</label>
1407         <textarea class="form-input" id="vendor-neg-reply" rows="3" placeholder="Write a reply to
admin"></textarea>
1408     </div>
1409     <div class="form-actions">
1410         <button class="btn btn-primary" onclick="replyNegotiation(${n.id})">Send Reply</button>
1411         <select class="form-input" id="vendor-neg-status" style="max-width:170px">
1412             ${['PENDING', 'NEGOTIATING', 'AGREED', 'REJECTED', 'CLOSED'].map(s => `<option value="${s}" ${s
=== n.status ? 'selected' : ''}>${s}</option>`).join('')}
1413         </select>
1414         <button class="btn btn-ghost" onclick="updateNegotiationStatus(${n.id})">Update
Status</button>
1415     </div>`;
1416     }catch(e){ showAlert('Failed: '+e.message, 'error'); }
1417 }
1418
1419 async function createNegotiation(){
1420     const body = {
1421         deviceId: Number(document.getElementById('vendor-neg-device').value) || null,
1422         proposedAmount: document.getElementById('vendor-neg-amount').value || null,
1423         subject: document.getElementById('vendor-neg-subject').value.trim(),
1424         description: document.getElementById('vendor-neg-description').value.trim(),
1425         initialMessage: document.getElementById('vendor-neg-message').value.trim()
1426     };
1427     if(!body.subject || !body.description){
1428         showAlert('Subject and description are required', 'error');
1429         return;
1430     }
1431     try{
1432         const created = await apiFetch('/vendor-negotiations', {method:'POST',
body:JSON.stringify(body)});
1433         showAlert('Negotiation created', 'success');
1434         clearNegotiationForm();
1435         selectedNegotiationId = created.id;
1436         await loadNegotiations();
1437     }catch(e){ showAlert('Failed: '+e.message, 'error'); }
1438 }
1439
1440 async function replyNegotiation(id){
1441     const message = document.getElementById('vendor-neg-reply').value.trim();
1442     if(!message){ showAlert('Reply message is required', 'error'); return; }
1443     try{
1444         await apiFetch(`/vendor-negotiations/${id}/messages`, {method:'POST',
body:JSON.stringify({message})});
1445         showAlert('Reply sent', 'success');
1446         await loadNegotiations();
1447     }catch(e){ showAlert('Failed: '+e.message, 'error'); }
1448 }
1449
1450 async function updateNegotiationStatus(id){
1451     const status = document.getElementById('vendor-neg-status').value;
1452     try{
1453         await apiFetch(`/vendor-negotiations/${id}/status`, {method:'PUT',
body:JSON.stringify({status})});
1454         showAlert('Status updated', 'success');
1455         await loadNegotiations();
1456     }catch(e){ showAlert('Failed: '+e.message, 'error'); }
1457 }
1458
1459 function clearNegotiationForm(){
1460     ['vendor-neg-amount', 'vendor-neg-subject', 'vendor-neg-description', 'vendor-neg-message'].forEach(id
=> document.getElementById(id).value = '');
1461     document.getElementById('vendor-neg-device').value = '';
1462 }
1463
1464 // ASSIGNMENTS
1465 async function loadAssignments(){
1466     try{
1467         const data = await apiFetch('/devices/assignments');
1468         assignmentsCache = data || [];
1469         document.getElementById('assignments-list').innerHTML = assignmentsCache.length
? assignmentsCache.map(a=>`<tr>
1471             <td class="main-cell">${a.deviceName||'-'}</td>

```

```

1472         <td>${chip(a.deviceType)}</td>
1473         <td>${a.employeeName||'-'}</td>
1474         <td style="font-size:11px;font-family:var(--mono)">${a.assignedDate||'-'}</td>
1475         <td>${chip(a.status==='ACTIVE'? 'active_assign':a.status?.toLowerCase())}</td>
1476     </tr>` ).join('')
1477     : `<tr><td colspan="5"><div class="empty"><div class="empty-icon"></div>No assignments
found</div></td></tr>`;
1478     }catch(e){}
1479 }
1480
1481 // PROFILE
1482 async function loadProfile(){
1483     try{
1484         const data = await apiFetch('/vendors/me');
1485         document.getElementById('profile-name').textContent = data.name || '--';
1486         document.getElementById('profile-email').textContent = data.email || '--';
1487         document.getElementById('profile-phone').textContent = data.phone || '--';
1488         document.getElementById('profile-reg').textContent = data.registeredAt || '--';
1489         document.getElementById('profile-avatar').textContent = (data.name||'T').charAt(0).toUpperCase();
1490         document.getElementById('sidebar-name').textContent = data.name || 'TechVendor';
1491     }catch(e){}
1492 }
1493
1494 // DEVICE DETAIL MODAL
1495 // Calls GET /vendor/{deviceId}/repair-logs to fetch repair history
1496 async function viewDevice(id){
1497     const d = devicesCache.find(x=>x.id===id);
1498     if(!d) return;
1499
1500     currentModalDeviceId = id;
1501     document.getElementById('modal-device-name').textContent = d.deviceName;
1502     document.getElementById('modal-device-id').textContent = `#${d.id} · ${d.brand||''}`;
1503
1504     const today = new Date();
1505     const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1506     const warrantyDisplay = warrantyExpiry
1507         ? (warrantyExpiry > today
1508             ? `<span style="color:var(--green);font-family:var(--mono)"> Valid until
1509               ${d.warrantyExpiryDate}</span>`
1510             : `<span style="color:var(--red);font-family:var(--mono)"> Expired
1511               ${d.warrantyExpiryDate}</span>`)
1512         : `<span style="color:var(--text3);font-family:var(--mono)"></span>`;
1513
1514     document.getElementById('modal-detail-grid').innerHTML = `
1515         <div class="detail-item"><div class="detail-label">Brand</div><div class="detail-
1516         value">${d.brand||'-'}</div></div>
1517         <div class="detail-item"><div class="detail-label">Type</div><div class="detail-
1518         value">${chip(d.deviceType)}</div></div>
1519         <div class="detail-item"><div class="detail-label">Status</div><div class="detail-
1520         value">${chip(d.deviceStatus)}</div></div>
1521         <div class="detail-item"><div class="detail-label">Warranty</div><div class="detail-
1522         value">${warrantyDisplay}</div></div>
1523         <div class="detail-item"><div class="detail-label">Rent/Month</div><div class="detail-
1524         value">${d.rentPerMonth ? `<span style="color:var(--cyan);font-family:var(--mono)">'+d.rentPerMonth+'<...
1525         <div class="detail-item"><div class="detail-label">Added On</div><div class="detail-
1526         value">${d.createdAt ? d.createdAt.split('T')[0] : '-'}</div></div>
1527     `;
1528
1529     // Show loading state immediately, then fetch
1530     document.getElementById('modal-repair-history').innerHTML = `
1531         <div class="empty" style="padding:20px">
1532             <div class="empty-icon" style="font-size:20px;animation:spin 1s linear infinite"></div>
1533             Loading repair history...
1534         </div>`;
1535     document.getElementById('device-modal').classList.add('open');
1536
1537     await loadModalRepairLogs(id);
1538 }
1539
1540 // Fetches and renders repair logs for the currently open modal device
1541 async function loadModalRepairLogs(deviceId){
1542     const container = document.getElementById('modal-repair-history');
1543     try{
1544         const logs = await apiFetch(`/devices/vendor/${deviceId}/repair-logs`);
1545         renderModalRepairLogs(logs);
1546     }catch(e){

```

```

1539     container.innerHTML = `
1540         <div style="padding:12px 0;font-size:11px;font-family:var(--mono);color:var(--red)">
1541             Failed to load repair logs: ${e.message}
1542         </div>`;
1543     }
1544 }
1545
1546 // Re-fetches logs for currently open modal (used by refresh button)
1547 async function refreshModalRepairLogs(){
1548     if(!currentModalDeviceId) return;
1549     const btn = document.getElementById('modal-refresh-logs-btn');
1550     btn.textContent = ' Loading...';
1551     btn.disabled = true;
1552     await loadModalRepairLogs(currentModalDeviceId);
1553     btn.textContent = ' Refresh';
1554     btn.disabled = false;
1555 }
1556
1557 // Renders RepairLogResponseDTO list into the modal repair history section
1558 function renderModalRepairLogs(logs){
1559     const container = document.getElementById('modal-repair-history');
1560     if(!logs || !logs.length){
1561         container.innerHTML = `<div class="empty" style="padding:16px"><div class="empty-icon"></div>No
repair history for this device</div>`;
1562         return;
1563     }
1564
1565     container.innerHTML = logs.map(r => {
1566         // Determine dot color from repairStatus or serviceRequestStatus
1567         const statusVal = (r.repairStatus || r.serviceRequestStatus || '').toUpperCase();
1568         const dotColor = statusVal.includes('DONE') || statusVal.includes('FIXED') ? 'green'
1569             : statusVal.includes('IRREPARABLE') || statusVal.includes('NOT_FIXABLE') ? 'red'
1570             : statusVal.includes('IN_PROGRESS') || statusVal.includes('SENT') ? 'amber'
1571             : 'cyan';
1572
1573         return `
1574         <div class="repair-log-card">
1575             <div class="repair-log-header">
1576                 <div style="display:flex;align-items:center;gap:10px">
1577                     <div class="tl-dot ${dotColor}" style="width:24px;height:24px;font-size:10px"></div>
1578                     <div>
1579                         <div style="font-size:12px;font-weight:600;color:var(--text)">
1580                             ${r.repairStatus ? 'Repair Log' : 'Service Request'}
1581                         </div>
1582                         <div>
1583                             ${r.serviceRequestId ? `<div class="repair-log-id">SR #${r.serviceRequestId} · ${r.serviceRequestStatus ?
chip(r.serviceRequestStatus) : ''}</div>` : ''}
1584                             : ''
1585                         </div>
1586                     </div>
1587                     <div style="display:flex;align-items:center;gap:8px">
1588                         <div>
1589                             <span style="font-size:10px;font-family:var(--mono);color:var(--text3)">#${r.id}</span>
1590                         </div>
1591                     </div>
1592                 </div>
1593                 <div class="repair-log-grid">
1594                     <div>
1595                         <div class="detail-label">Repaired By</div>
1596                         <div style="font-family:var(--mono);font-size:12px;color:var(--text);margin-
top:3px">${r.repairedBy || '-'}</div>
1597                     </div>
1598                     <div>
1599                         <div class="detail-label">Repair Date</div>
1600                         <div style="font-family:var(--mono);font-size:12px;color:var(--text);margin-
top:3px">${r.repairDate || '-'}</div>
1601                     </div>
1602                     <div>
1603                         <div class="detail-label">Action Taken</div>
1604                         <div style="font-family:var(--mono);font-size:12px;color:var(--text);margin-
top:3px">${r.repairAction || '-'}</div>
1605                     </div>
1606                     <div>
1607                         <div class="detail-label">Damaged Component</div>
1608                         <div style="font-family:var(--mono);font-size:12px;color:var(--text);margin-
top:3px">${r.damagedComponent || 'NONE'}</div>

```

```

1609         </div>
1610         <div>
1611             <div class="detail-label">Replaced Component</div>
1612             <div style="font-family:var(--mono);font-size:12px;color:var(--text);margin-
top:3px">${r.replacedComponent || 'NONE'}</div>
1613         </div>
1614         <div>
1615             <div class="detail-label">Device</div>
1616             <div style="font-family:var(--mono);font-size:12px;color:var(--text);margin-
top:3px">${r.deviceName || '-'}</div>
1617         </div>
1618     </div>
1619
1620     ${r.remarks ? `<div class="repair-log-remarks"> ${r.remarks}</div>` : ''}
1621     </div>`;
1622     }).join('');
1623 }
1624
1625 function closeModal(){
1626     document.getElementById('device-modal').classList.remove('open');
1627     currentModalDeviceId = null;
1628 }
1629
1630 // HELPERS
1631 function chip(val){
1632     if(!val) return '-';
1633     const cls = val.toLowerCase().replace(/ /g, '_');
1634     return `<span class="chip ${cls}">${val.replace(/_/g, ' ')}</span>`;
1635 }
1636
1637 function showAlert(msg, type){
1638     const el = document.createElement('div');
1639     el.className = `alert alert-${type}`;
1640     el.innerHTML = `<span>${type==='success'?':':''}</span>${msg}`;
1641     document.body.appendChild(el);
1642     setTimeout(()=>el.remove(), 3500);
1643 }
1644
1645 function logout(){
1646     localStorage.removeItem('jwt_token');
1647     localStorage.removeItem('auth_type');
1648     fetch(`${API}/session/logout`, {method: 'POST'}).finally(()=>window.location.href='/vendor-
login.html');
1649 }
1650
1651 // TWO-FACTOR AUTH
1652 async function loadTotpStatus(){
1653     const chipEl = document.getElementById('totp-status-chip');
1654     chipEl.textContent = 'checking...';
1655     chipEl.className = 'chip';
1656     document.getElementById('qr-area').style.display = 'none';
1657     document.getElementById('totp-verify-input').value = '';
1658     document.getElementById('totp-verify-error').style.display = 'none';
1659     try{
1660         const data = await apiFetch('/totp/status');
1661         if(data.totpEnabled){
1662             chipEl.textContent = 'enabled';
1663             chipEl.className = 'chip repair_done';
1664             document.getElementById('totp-setup-section').style.display = 'none';
1665             document.getElementById('totp-disable-section').style.display = 'block';
1666         } else {
1667             chipEl.textContent = 'disabled';
1668             chipEl.className = 'chip rejected';
1669             document.getElementById('totp-setup-section').style.display = 'block';
1670             document.getElementById('totp-disable-section').style.display = 'none';
1671         }
1672     }catch(e){
1673         chipEl.textContent = 'error';
1674         chipEl.className = 'chip open';
1675     }
1676 }
1677
1678 async function generateTotpQr(){
1679     const btn = document.getElementById('generate-qr-btn');
1680     btn.textContent = 'Generating...';
1681     btn.disabled = true;

```

```

1682     try{
1683         const data = await apiFetch('/totp/setup', { method: 'POST' });
1684         document.getElementById('totp-secret-display').textContent = data.secret || '';
1685         const qrUri = encodeURIComponent(data.qrCodeUri || '');
1686         document.getElementById('qr-img').src = `https://api.qrserver.com/v1/create-qr-
code/?size=180x180&data=${qrUri}`;
1687         document.getElementById('qr-area').style.display = 'block';
1688     }catch(e){
1689         showAlert('Failed to generate QR: ' + e.message, 'error');
1690     }finally{
1691         btn.textContent = 'Regenerate QR Code';
1692         btn.disabled = false;
1693     }
1694 }
1695
1696 async function verifyAndActivateTotp(){
1697     const code = document.getElementById('totp-verify-input').value.trim();
1698     const errEl = document.getElementById('totp-verify-error');
1699     errEl.style.display = 'none';
1700     if(code.length !== 6){
1701         errEl.textContent = 'Please enter a 6-digit code.';
1702         errEl.style.display = 'block';
1703         return;
1704     }
1705     try{
1706         await apiFetch('/totp/verify-setup', { method: 'POST', body: JSON.stringify({ code }) });
1707         showAlert('2FA activated – required on next login', 'success');
1708         loadTotpStatus();
1709     }catch(e){
1710         errEl.textContent = 'Invalid code. Try again.';
1711         errEl.style.display = 'block';
1712     }
1713 }
1714
1715 async function disableTotp(){
1716     if(!confirm('Disable two-factor authentication?')) return;
1717     try{
1718         await apiFetch('/totp/disable', { method: 'DELETE' });
1719         showAlert('2FA has been disabled', 'success');
1720         loadTotpStatus();
1721     }catch(e){
1722         showAlert('Failed: ' + e.message, 'error');
1723     }
1724 }
1725
1726 // INVOICES
1727 async function loadInvoices() {
1728     try {
1729         const invoices = await apiFetch('/invoices/vendor/me');
1730
1731         // Update badge
1732         const badge = document.getElementById('invoice-badge');
1733         if (invoices && invoices.length) {
1734             badge.textContent = invoices.length;
1735             badge.style.display = '';
1736         } else {
1737             badge.style.display = 'none';
1738         }
1739
1740         const container = document.getElementById('invoices-container');
1741
1742         if (!invoices || !invoices.length) {
1743             container.innerHTML = `
1744                 <div class="empty">
1745                     <div class="empty-icon"></div>
1746                     No pending invoices – all caught up!
1747                 </div>`;
1748             return;
1749         }
1750
1751         container.innerHTML = invoices.map(inv => {
1752             const isRepair = (inv.invoiceType || '').toUpperCase() === 'REPAIR';
1753             const accentColor = isRepair ? 'var(--orange)' : 'var(--cyan)';
1754             const accentDim = isRepair ? 'var(--orange-dim)' : 'var(--cyan-dim)';
1755             const accentBorder = isRepair
1756                 ? 'rgba(251,146,60,0.25)'

```

```

1757         : 'rgba(0,210,255,0.25)';
1758         const typeIcon = isRepair ? '' : '';
1759
1760         return `
1761         <div class="invoice-card" id="invoice-row-${inv.id}"
1762             style="--inv-accent:${accentColor};--inv-dim:${accentDim};--inv-border:${accentBorder}">
1763
1764             <!-- TOP STRIP -->
1765             <div class="invoice-card-strip"></div>
1766
1767             <div class="invoice-card-inner">
1768
1769                 <!-- HEADER ROW -->
1770                 <div class="invoice-card-header">
1771                     <div style="display:flex;align-items:center;gap:10px">
1772                         <div class="invoice-type-icon">${typeIcon}</div>
1773                         <div>
1774                             <div class="invoice-card-id">INVOICE
1775                             ${String(inv.id).padStart(4, '0')}</div>
1776                             <div class="invoice-card-type">${(inv.invoiceType || '-').replace(/_/g, '
1777                             ')} Invoice</div>
1778                         </div>
1779                         <div style="text-align:right">
1780                             ${chip(inv.status)}
1781                             <div style="font-size:10px;color:var(--text3);font-family:var(--mono);margin-
1782                             top:4px">
1783                                 Issued ${inv.issuedDate || '-'}
1784                             </div>
1785                         </div>
1786                     </div>
1787
1788                     <!-- AMOUNT -->
1789                     <div class="invoice-amount-row">
1790                         <div class="invoice-amount-label">Amount Due</div>
1791                         <div class="invoice-amount">
1792                             ${inv.amount
1793                             ? '' + Number(inv.amount).toLocaleString('en-IN', {minimumFractionDigits:
1794                             2})
1795                             : '-'}
1796                         </div>
1797                     </div>
1798
1799                     <!-- DIVIDER -->
1800                     <div style="height:1px;background:var(--border);margin:14px 0"></div>
1801
1802                     <!-- META GRID -->
1803                     <div class="invoice-meta-grid">
1804                         <div class="invoice-meta-item">
1805                             <div class="invoice-meta-label">Vendor</div>
1806                             <div class="invoice-meta-value">${inv.vendorName || '-'}</div>
1807                         </div>
1808                         <div class="invoice-meta-item">
1809                             <div class="invoice-meta-label">Invoice Type</div>
1810                             <div class="invoice-meta-value">${(inv.invoiceType || '-').replace(/_/g, '
1811                             ')}</div>
1812                         </div>
1813                         <div class="invoice-meta-item">
1814                             <div class="invoice-meta-label">Issued On</div>
1815                             <div class="invoice-meta-value">${inv.issuedDate || '-'}</div>
1816                         </div>
1817                         <div class="invoice-meta-item">
1818                             <div class="invoice-meta-label">Acknowledged On</div>
1819                             <div class="invoice-meta-value" style="color:${inv.seenAt ? 'var(--green)' :
1820                             'var(--text3)'}">
1821                                 ${inv.seenAt || 'Pending'}
1822                             </div>
1823                         </div>
1824                     </div>
1825
1826                     <!-- DIVIDER -->
1827                     <div style="height:1px;background:var(--border);margin:14px 0"></div>
1828
1829                     <!-- FOOTER -->
1830                     <div class="invoice-card-footer">
1831                         <div style="font-size:11px;color:var(--text3);font-family:var(--mono);line-

```



```

height:1.6">
1827         By acknowledging, you confirm<br>receipt of this payment from admin.
1828     </div>
1829     <button
1830         class="btn btn-success"
1831         id="ack-btn-${inv.id}"
1832         style="min-width:150px"
1833         onclick="acknowledgeInvoice(${inv.id})">
1834         Acknowledge
1835     </button>
1836 </div>
1837
1838 </div>
1839 </div>;
1840 }).join('');
1841
1842 } catch(e) {
1843     const container = document.getElementById('invoices-container');
1844     if (container) container.innerHTML = `
1845         <div class="empty">
1846             <div class="empty-icon"></div>
1847             Could not load invoices: ${e.message}
1848         </div>`;
1849     }
1850 }
1851
1852 async function acknowledgeInvoice(invoiceId) {
1853     if (!confirm('Acknowledge this invoice? This confirms you have received the payment.')) return;
1854     const btn = document.getElementById(`ack-btn-${invoiceId}`);
1855     btn.disabled = true;
1856     btn.textContent = 'Processing...';
1857     try {
1858         await apiFetch(`/invoices/${invoiceId}/acknowledge`, { method: 'PUT' });
1859         showAlert('Invoice acknowledged successfully!', 'success');
1860
1861         // Remove the card immediately
1862         const row = document.getElementById(`invoice-row-${invoiceId}`);
1863         if (row) row.remove();
1864
1865         // Count remaining cards
1866         const remaining = document.querySelectorAll('[id^="invoice-row-"]').length;
1867         const badge = document.getElementById('invoice-badge');
1868
1869         if (remaining === 0) {
1870             badge.style.display = 'none';
1871             // Fix: target invoices-container, not invoices-list
1872             document.getElementById('invoices-container').innerHTML = `
1873                 <div class="empty">
1874                     <div class="empty-icon"></div>
1875                     No pending invoices – all caught up!
1876                 </div>`;
1877         } else {
1878             badge.textContent = remaining;
1879         }
1880     } catch(e) {
1881         showAlert('Failed: ' + e.message, 'error');
1882         btn.disabled = false;
1883         btn.textContent = ' Acknowledge';
1884     }
1885 }
1886
1887 async function removeDevice(deviceId, deviceName) {
1888     if (!confirm(`Remove "${deviceName}" from your catalog? This cannot be undone.`)) return;
1889     try {
1890         await apiFetch(`/devices/remove/${deviceId}`, { method: 'DELETE' });
1891         showAlert(`${deviceName} removed from catalog`, 'success');
1892         devicesCache = devicesCache.filter(d => d.id !== deviceId);
1893         filterDevices(); // re-render list without hitting the server again
1894         loadDashboard(); // update stats
1895     } catch(e) {
1896         showAlert('Failed: ' + e.message, 'error');
1897     }
1898 }
1899 </script>
1900 </body>
1901 </html>

```



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Two-Factor Authentication Setup</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
8      <!-- qrcode.js for QR rendering -->
9      <script src="https://cdnjs.cloudflare.com/ajax/libs/qrcodejs/1.0.0/qrcode.min.js"></script>
10     <style>
11         *{margin:0;padding:0;box-sizing:border-box}
12         :root{
13             --bg:#070910;--panel:#0e1118;--panel2:#131821;
14             --border:rgba(255,255,255,0.07);--border-lit:rgba(255,255,255,0.16);
15             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
16             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.13);
17             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
18             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
19             --amber:#f9c74f;
20             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
21         }
22         body{font-family:var(--font);background:var(--bg);color:var(--text);min-height:100vh;}
23
24         .bg-layer{position:fixed;inset:0;z-index:0;
25             background:
26                 radial-gradient(ellipse 50% 50% at 20% 30%, rgba(79,140,255,0.06) 0%,transparent 70%),
27                 radial-gradient(ellipse 40% 40% at 80% 70%, rgba(52,211,153,0.04) 0%,transparent 60%);
28         }
29         .grid-overlay{position:fixed;inset:0;z-index:0;
30             background-image:linear-gradient(rgba(255,255,255,0.015) 1px,transparent 1px),linear-
gradient(90deg,rgba(255,255,255,0.015) 1px,transparent 1px);
31             background-size:40px 40px;
32         }
33
34         /* Nav */
35         .nav{position:relative;z-index:10;display:flex;align-items:center;justify-content:space-
between;padding:18px 40px;border-bottom:1px solid var(--border);}
36         .nav-brand{display:flex;align-items:center;gap:10px;}
37         .nav-icon{width:36px;height:36px;background:linear-gradient(135deg,#4f8cff,#818cf8);border-
radius:9px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size:14px;c...
38         .nav-name{font-size:16px;font-weight:700;}
39         .nav-back{font-size:13px;color:var(--text2);text-decoration:none;display:flex;align-
items:center;gap:6px;transition:color .2s;}
40         .nav-back:hover{color:var(--blue);}
41
42         /* Page */
43         .page{position:relative;z-index:1;max-width:680px;margin:0 auto;padding:48px 24px 80px;}
44
45         .page-header{margin-bottom:36px;}
46         .page-header h1{font-size:28px;font-weight:700;letter-spacing:-.4px;margin-bottom:6px;}
47         .page-header p{font-size:14px;color:var(--text2);line-height:1.6;}
48
49         /* Status badge */
50         .status-bar{display:flex;align-items:center;justify-content:space-between;background:var(--
panel);border:1px solid var(--border);border-radius:14px;padding:18px 22px;margin-bottom:28px;}
51         .status-left{display:flex;align-items:center;gap:14px;}
52         .status-icon{width:44px;height:44px;border-radius:12px;display:flex;align-items:center;justify-
content:center;font-size:22px;flex-shrink:0;}
53         .status-icon.on{background:var(--green-dim);}
54         .status-icon.off{background:var(--border);}
55         .status-title{font-size:15px;font-weight:600;margin-bottom:2px;}
56         .status-desc{font-size:12px;color:var(--text2);}
57         .badge{font-size:11px;font-weight:700;padding:4px 10px;border-radius:20px;letter-spacing:.04em;text-
transform:uppercase;}
58         .badge-on{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.25);}
59         .badge-off{background:var(--border);color:var(--text2);border:1px solid var(--border-lit);}
60
61         /* Cards */
62         .card{background:var(--panel);border:1px solid var(--border);border-radius:14px;padding:24px;margin-
bottom:20px;}

```

```

63     .card-title{font-size:15px;font-weight:600;margin-bottom:4px;display:flex;align-
items:center;gap:8px;}
64     .card-subtitle{font-size:13px;color:var(--text2);margin-bottom:20px;}
65
66     /* Steps list */
67     .steps-list{display:flex;flex-direction:column;gap:16px;margin-bottom:24px;}
68     .step-row{display:flex;gap:14px;align-items:flex-start;}
69     .step-num{width:28px;height:28px;border-radius:50%;background:var(--blue-dim);border:1px solid
rgba(79,140,255,0.3);color:var(--blue);font-size:13px;font-weight:700;display:flex;align-items...
70     .step-text strong{font-size:13px;color:var(--text);display:block;margin-bottom:2px;}
71     .step-text span{font-size:12px;color:var(--text2);line-height:1.5;}
72
73     /* QR area */
74     .qr-area{display:none;flex-direction:column;align-
items:center;gap:20px;padding:24px;background:var(--panel2);border:1px solid var(--border);border-
radius:12px;margin-bottom:20px;}
75     .qr-area.show{display:flex;}
76     #qr-container{background:#fff;padding:14px;border-radius:10px;display:inline-block;}
77     .qr-container canvas, .qr-container img{display:block;}
78     .secret-box{width:100%;background:var(--bg);border:1px solid var(--border);border-
radius:9px;padding:12px 16px;text-align:center;}
79     .secret-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-
spacing:.06em;margin-bottom:6px;}
80     .secret-val{font-family:var(--mono);font-size:14px;color:var(--amber);letter-spacing:.12em;word-
break:break-all;}
81     .copy-btn{font-size:11px;color:var(--blue);background:none;border:none;cursor:pointer;font-
family:var(--font);padding:0;margin-top:4px;}
82     .copy-btn:hover{text-decoration:underline;}
83
84     /* OTP row */
85     .otp-row{display:flex;gap:10px;justify-content:center;margin:0 0 8px;}
86     .otp-digit{width:48px;height:58px;text-align:center;font-size:24px;font-family:var(--mono);font-
weight:500;background:var(--panel2);border:1px solid var(--border);border-radius:10px;color:v...
87     .otp-digit:focus{border-color:var(--blue);box-shadow:0 0 3px var(--blue-dim);}
88     .otp-digit.filled{border-color:rgba(79,140,255,0.4);}
89
90     /* Alerts */
91     .alert{border-radius:9px;padding:11px 14px;font-size:13px;margin-bottom:14px;display:none;align-
items:flex-start;gap:9px;}
92     .alert.show{display:flex;}
93     .alert-err{background:var(--red-dim);border:1px solid rgba(248,113,113,0.25);color:var(--red);}
94     .alert-ok{background:var(--green-dim);border:1px solid rgba(52,211,153,0.25);color:var(--green);}
95     .alert-icon{font-size:15px;flex-shrink:0;}
96
97     /* Buttons */
98     .btn{padding:12px 22px;border-radius:10px;border:none;font-family:var(--font);font-size:14px;font-
weight:600;cursor:pointer;transition:all .2s;}
99     .btn-primary{background:var(--blue);color:#fff;}
100    .btn-primary:hover{background:#6ba1ff;box-shadow:0 4px 16px rgba(79,140,255,0.3);}
101    .btn-primary.disabled{opacity:.5;cursor:not-allowed;box-shadow:none;}
102    .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.25);}
103    .btn-danger:hover{background:rgba(248,113,113,0.2);}
104    .btn-ghost{background:transparent;color:var(--text2);border:1px solid var(--border);}
105    .btn-ghost:hover{border-color:var(--border-lit);color:var(--text);}
106
107    .btn-row{display:flex;gap:10px;flex-wrap:wrap;}
108
109    .spinner{display:inline-block;width:14px;height:14px;border:2px solid rgba(255,255,255,0.2);border-
top-color:#fff;border-radius:50%;animation:spin .6s linear infinite;vertical-align:middle;...
110    @keyframes spin{to{transform:rotate(360deg)}}
111
112    /* Danger zone */
113    .danger-card{background:var(--red-dim);border:1px solid rgba(248,113,113,0.2);border-
radius:14px;padding:20px 24px;display:flex;align-items:center;justify-content:space-between;gap:16px;}
114    .danger-text strong{font-size:14px;color:var(--red);display:block;margin-bottom:3px;}
115    .danger-text span{font-size:12px;color:rgba(248,113,113,0.7);}
116
117    .loading{text-align:center;padding:60px 0;color:var(--text2);}
118    .loading .spinner{border-top-color:var(--blue);border-color:var(--border-
lit);width:24px;height:24px;}
119    </style>
120 </head>
121 <body>
122 <div class="bg-layer"></div>
123 <div class="grid-overlay"></div>
124

```

```

125 <nav class="nav">
126     <div class="nav-brand">
127         <div class="nav-icon">EMS</div>
128         <div class="nav-name">EmployeeMS</div>
129     </div>
130     <a class="nav-back" href="javascript:history.back()"> Back to dashboard</a>
131 </nav>
132
133 <div class="page">
134     <div class="page-header">
135         <h1>Two-Factor Authentication</h1>
136         <p>Add an extra layer of security to your account. Once enabled, you'll need to provide a 6-digit
code from your authenticator app every time you sign in.</p>
137     </div>
138
139     <!-- Loading state -->
140     <div class="loading" id="loading">
141         <span class="spinner"></span>
142     </div>
143
144     <!-- Main content (shown after status loads) -->
145     <div id="main-content" style="display:none;">
146
147         <!-- Status bar -->
148         <div class="status-bar">
149             <div class="status-left">
150                 <div class="status-icon" id="status-icon"></div>
151                 <div>
152                     <div class="status-title">Two-Factor Authentication</div>
153                     <div class="status-desc" id="status-desc">Loading...</div>
154                 </div>
155             </div>
156             <span class="badge" id="status-badge"></span>
157         </div>
158
159         <!-- ===== SETUP SECTION (shown when disabled) ===== -->
160         <div id="setup-section">
161             <div class="card">
162                 <div class="card-title"> Enable 2FA</div>
163                 <div class="card-subtitle">Follow these steps to add two-factor authentication to your
account.</div>
164
165                 <div class="steps-list">
166                     <div class="step-row">
167                         <div class="step-num">1</div>
168                         <div class="step-text">
169                             <strong>Install an authenticator app</strong>
170                             <span>Download <a href="https://support.google.com/accounts/answer/1066447"
target="_blank" style="color:var(--blue)">Google Authenticator</a> or <a href="https://authy...
171                         </div>
172                     </div>
173                     <div class="step-row">
174                         <div class="step-num">2</div>
175                         <div class="step-text">
176                             <strong>Scan the QR code</strong>
177                             <span>Click "Generate QR Code" below, then scan it with your authenticator app.
You can also enter the secret key manually.</span>
178                         </div>
179                     </div>
180                     <div class="step-row">
181                         <div class="step-num">3</div>
182                         <div class="step-text">
183                             <strong>Verify the setup</strong>
184                             <span>Enter the 6-digit code shown in your app to confirm everything is working
correctly.</span>
185                         </div>
186                     </div>
187                 </div>
188
189                 <!-- QR area -->
190                 <div class="qr-area" id="qr-area">
191                     <div id="qr-container"></div>
192                     <div class="secret-box">
193                         <div class="secret-label">Manual entry key (backup)</div>
194                         <div class="secret-val" id="secret-display"></div>
195                         <button class="copy-btn" onclick="copySecret()">Copy key</button>

```

```

196         </div>
197     </div>
198
199     <!-- Alerts -->
200     <div class="alert alert-err" id="setup-err"><span class="alert-icon"></span><span
id="setup-err-msg"></span></div>
201     <div class="alert alert-ok" id="setup-ok"><span class="alert-icon"></span><span id="setup-
ok-msg"></span></div>
202
203     <!-- Verify code input (shown after QR generated) -->
204     <div id="verify-area" style="display:none;">
205         <p style="font-size:13px;color:var(--text2);margin-bottom:16px;">Enter the code shown
in your authenticator app to activate 2FA:</p>
206         <div class="otp-row" id="setup-otp-row">
207             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s0">
208             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s1">
209             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s2">
210             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s3">
211             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s4">
212             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s5">
213         </div>
214     </div>
215
216     <div class="btn-row" style="margin-top:8px;">
217         <button class="btn btn-ghost" id="btn-generate" onclick="generateQr()"> Generate QR
Code</button>
218         <button class="btn btn-primary" id="btn-verify" style="display:none;" disabled
onclick="verifySetup()">Activate 2FA</button>
219     </div>
220 </div>
221 </div>
222
223     <!-- ===== ACTIVE SECTION (shown when enabled) ===== -->
224     <div id="active-section" style="display:none;">
225         <div class="card">
226             <div class="card-title"> 2FA is active</div>
227             <div class="card-subtitle">Your account is protected. Every sign-in requires your
authenticator app code.</div>
228             <p style="font-size:13px;color:var(--text2);line-height:1.6;">
229                 If you lose access to your authenticator app, contact your admin to reset 2FA. We
recommend also saving backup codes.
230             </p>
231         </div>
232
233         <div class="danger-card">
234             <div class="danger-text">
235                 <strong>Disable Two-Factor Authentication</strong>
236                 <span>This will remove the extra protection from your account.</span>
237             </div>
238             <button class="btn btn-danger" onclick="disableTotp()">Disable 2FA</button>
239         </div>
240     </div>
241
242 </div>
243 </div>
244
245 <script>
246     const token = () => localStorage.getItem('jwt_token');
247     const authHeader = () => ({ 'Authorization': 'Bearer ' + token(), 'Content-Type': 'application/json'
});
248
249     // ---- Load status on page open ----
250     async function loadStatus() {
251         try {
252             const res = await fetch('/totp/status', { headers: authHeader() });
253             if (res.status === 401) { window.location.href = '/google.html'; return; }
254             const data = await res.json();
255             renderStatus(data.totpEnabled);
256         } catch (e) {
257             console.error(e);
258         } finally {
259             document.getElementById('loading').style.display = 'none';
260             document.getElementById('main-content').style.display = 'block';
261         }
262     }
263

```

```

264     function renderStatus(enabled) {
265         const icon = document.getElementById('status-icon');
266         const desc = document.getElementById('status-desc');
267         const badge = document.getElementById('status-badge');
268         const setup = document.getElementById('setup-section');
269         const active = document.getElementById('active-section');
270
271         if (enabled) {
272             icon.textContent = '';
273             icon.className = 'status-icon on';
274             desc.textContent = 'Your account requires a 6-digit code on every sign-in.';
275             badge.textContent = 'Enabled';
276             badge.className = 'badge badge-on';
277             setup.style.display = 'none';
278             active.style.display = 'block';
279         } else {
280             icon.textContent = '';
281             icon.className = 'status-icon off';
282             desc.textContent = 'Not enabled – only your password protects your account.';
283             badge.textContent = 'Disabled';
284             badge.className = 'badge badge-off';
285             setup.style.display = 'block';
286             active.style.display = 'none';
287         }
288     }
289
290     // ---- Generate QR ----
291     let currentSecret = null;
292
293     async function generateQr() {
294         const btn = document.getElementById('btn-generate');
295         btn.disabled = true;
296         btn.innerHTML = '<span class="spinner"></span>Generating...';
297         clearAlerts();
298
299         try {
300             const res = await fetch('/totp/setup', { method: 'POST', headers: authHeader() });
301             const data = await res.json();
302             if (!res.ok) { showAlert('setup-err', data.error || 'Failed to generate QR code'); return; }
303
304             currentSecret = data.secret;
305
306             // Render QR code
307             const qrContainer = document.getElementById('qr-container');
308             qrContainer.innerHTML = '';
309             new QRCode(qrContainer, {
310                 text: data.qrCodeUri,
311                 width: 180,
312                 height: 180,
313                 colorDark: '#000000',
314                 colorLight: '#ffffff',
315                 correctLevel: QRCode.CorrectLevel.M
316             });
317
318             document.getElementById('secret-display').textContent = formatSecret(data.secret);
319             document.getElementById('qr-area').classList.add('show');
320             document.getElementById('verify-area').style.display = 'block';
321             document.getElementById('btn-verify').style.display = 'inline-block';
322             btn.textContent = 'Regenerate';
323
324             // Focus first digit
325             setTimeout(() => document.getElementById('s0').focus(), 200);
326         } catch (e) {
327             showAlert('setup-err', 'Network error. Please try again.');
```

```

340     navigator.clipboard.writeText(currentSecret || '');
341     const btn = document.querySelector('.copy-btn');
342     btn.textContent = 'Copied!';
343     setTimeout(() => btn.textContent = 'Copy key', 1500);
344 }
345
346 // ---- OTP inputs for setup ----
347 const setupDigits = Array.from({ length: 6 }, (_, i) => document.getElementById('s' + i));
348 const btnVerify = document.getElementById('btn-verify');
349
350 setupDigits.forEach((d, i) => {
351     d.addEventListener('input', () => {
352         const val = d.value.replace(/\D/g, '');
353         d.value = val;
354         d.classList.toggle('filled', val.length > 0);
355         if (val && i < 5) setupDigits[i + 1].focus();
356         btnVerify.disabled = setupDigits.map(x => x.value).join('').length < 6;
357     });
358     d.addEventListener('keydown', e => {
359         if (e.key === 'Backspace' && !d.value && i > 0) {
360             setupDigits[i - 1].focus();
361             setupDigits[i - 1].value = '';
362             setupDigits[i - 1].classList.remove('filled');
363             btnVerify.disabled = true;
364         }
365         if (e.key === 'Enter' && !btnVerify.disabled) verifySetup();
366     });
367     d.addEventListener('paste', e => {
368         const pasted = (e.clipboardData || window.clipboardData).getData('text').replace(/\D/g,
369             '').slice(0, 6);
370         if (pasted.length === 6) {
371             pasted.split('').forEach((ch, idx) => { setupDigits[idx].value = ch;
372             setupDigits[idx].classList.add('filled'); });
373             setupDigits[5].focus();
374             btnVerify.disabled = false;
375             e.preventDefault();
376         }
377     });
378 });
379
380 // ---- Verify setup ----
381 async function verifySetup() {
382     clearAlerts();
383     const code = setupDigits.map(d => d.value).join('');
384     if (code.length < 6) return;
385
386     btnVerify.disabled = true;
387     btnVerify.innerHTML = '<span class="spinner"></span>Activating...';
388
389     try {
390         const res = await fetch('/totp/verify-setup', {
391             method: 'POST',
392             headers: authHeader(),
393             body: JSON.stringify({ code })
394         });
395         const data = await res.json();
396         if (!res.ok) {
397             showAlert('setup-err', data.error || 'Invalid code. Try again.');
```



```
account less secure.')) return;
414
415     try {
416         const res = await fetch('/totp/disable', { method: 'DELETE', headers: authHeader() });
417         if (!res.ok) { alert('Failed to disable 2FA.');
```

return; }

```
418         renderStatus(false);
419     } catch (e) {
420         alert('Network error.');
```

}

```
421     }
422 }
423
424 // ---- Alerts ----
425 function showAlert(id, msg) {
426     const el = document.getElementById(id);
427     el.classList.add('show');
```

document.getElementById(id + '-msg').textContent = msg;

```
429 }
430 function clearAlerts() {
431     ['setup-err', 'setup-ok'].forEach(id => document.getElementById(id).classList.remove('show'));
432 }
433
434 // ---- Boot ----
435 if (!localStorage.getItem('jwt_token')) {
436     window.location.href = '/google.html';
437 } else {
438     loadStatus();
439 }
440 </script>
441 </body>
442 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Vendor Hub</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700&family=Space+Mono:wght@400;700&
isplay=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#08100a;--bg2:#0c160e;--bg3:#101c12;
12             --panel:#131f15;--panel2:#182219;
13             --border:rgba(52,211,153,0.08);--border2:rgba(52,211,153,0.18);
14             --text:#e8f0ea;--text2:#8aaa92;--text3:#3d5e44;
15             --accent:#34d399;--accent-dim:rgba(52,211,153,0.1);
16             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
17             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
18             --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
19             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
20             --cyan:#22d3ee;
21             --font:'DM Sans',sans-serif;--mono:'Space Mono',monospace;
22             --sidebar:245px;
23         }
24         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
25         .app{display:flex;height:100vh}
26
27         /* SIDEBAR */
28         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;position:relative}
29         .sidebar::before{content:'';position:absolute;left:0;top:0;bottom:0;width:2px;background:linear-
gradient(to bottom,transparent,var(--accent),transparent)}
30         .logo{padding:22px 20px;border-bottom:1px solid var(--border)}
31         .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
32         .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
33         .logo h2{font-size:16px;font-weight:700;color:var(--text)}
34         .logo p{font-size:10px;color:var(--text3);font-family:var(--mono)}
35         .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:var(--accent-dim);border:1px solid rgba(52,211,153,0.2);border-radius:4px;font-size:10px;font-weight:7...
36         .nav{flex:1;padding:10px 0;overflow-y:auto}
37         .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2px;text-transform:uppercase;font-family:var(--mono)}
38         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:6px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
39         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
40         .nav-item.active{background:var(--accent-dim);color:var(--accent);border-left:2px solid var(--
accent);margin-left:6px}
41         .nav-icon{width:16px;height:16px;flex-shrink:0}
42         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
43         .avatar-card{display:flex;align-items:center;gap:10px}
44         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:6px;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
45         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
46         .user-info-side .role-text{font-size:9px;color:var(--accent);font-family:var(--mono);text-
transform:uppercase;letter-spacing:1px}
47
48         /* MAIN */
49         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
50         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
51         .page-title{font-size:16px;font-weight:700;color:var(--text);font-family:var(--mono)}
52         .topbar-right{display:flex;align-items:center;gap:12px}
53         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:4px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;transition:all...
54         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
55         .content{flex:1;overflow-y:auto;padding:24px}
56         .view{display:none}
57         .view.active{display:block;animation:fadeIn 0.25s ease}
58         @keyframes fadeIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}

```

```

59
60     /* STATS */
61     .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:14px;margin-
bottom:24px}
62     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:10px;padding:18px;position:relative;transition:border-color 0.2s}
63     .stat-card:hover{border-color:var(--border2)}
64     .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:2px;border-radius:10px
10px 0 0}
65     .stat-card.green::before{background:var(--accent)}
66     .stat-card.amber::before{background:var(--amber)}
67     .stat-card.blue::before{background:var(--blue)}
68     .stat-card.red::before{background:var(--red)}
69     .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1.2px;margin-
bottom:10px;font-weight:600;font-family:var(--mono)}
70     .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
71     .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
72
73     /* SECTION */
74     .section{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-
bottom:18px;overflow:hidden}
75     .section-head{padding:14px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
76     .section-title{font-size:13px;font-weight:700;color:var(--text);font-family:var(--mono)}
77     .section-sub{font-size:11px;color:var(--text3);margin-top:2px}
78
79     /* TABLE */
80     .tablewrap{overflow-x:auto}
81     table{width:100%;border-collapse:collapse}
82     th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-
spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
83     td{padding:11px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
84     tr:last-child td{border-bottom:none}
85     tr:hover td{background:rgba(52,211,153,0.03);color:var(--text)}
86
87     /* CHIPS */
88     .chip{display:inline-flex;align-items:center;padding:3px 8px;border-radius:4px;font-size:10px;font-
weight:700;font-family:var(--mono);letter-spacing:0.5px}
89     .chip.active{background:var(--accent-dim);color:var(--accent);border:1px solid rgba(52,211,153,0.2)}
90     .chip.inactive{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
91     .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
92     .chip.delivered{background:var(--accent-dim);color:var(--accent);border:1px solid
rgba(52,211,153,0.2)}
93     .chip.processing{background:var(--blue-dim);color:var(--blue);border:1px solid rgba(96,165,250,0.2)}
94     .chip.breakfast{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(251,191,36,0.2)}
95     .chip.lunch{background:var(--accent-dim);color:var(--accent);border:1px solid rgba(52,211,153,0.2)}
96     .chip.dinner{background:var(--blue-dim);color:var(--blue);border:1px solid rgba(96,165,250,0.2)}
97
98     /* BUTTONS */
99     .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--mono);font-size:11px;font-
weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.5px;text-transform:uppercase}
100     .btn-primary{background:var(--accent);color:#08100a}
101     .btn-primary:hover{background:#22c77a;box-shadow:0 4px 12px rgba(52,211,153,0.3)}
102     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
103     .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
104     .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
105     .btn-danger:hover{background:var(--red);color:#fff}
106     .btn-warning{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
107     .btn-warning:hover{background:var(--amber);color:#000}
108     .btn-sm{padding:5px 10px;font-size:10px}
109     .action-row{display:flex;gap:6px}
110
111     /* FORM */
112     .form-card{padding:22px}
113     .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
114     .form-group{display:flex;flex-direction:column;gap:5px}
115     .form-group.full{grid-column:1/-1}
116     label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px;font-family:var(--mono)}
117     input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:6px;padding:9px 12px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-colo...
118     input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
119     input::placeholder,textarea::placeholder{color:var(--text3)}
120     select option{background:var(--bg2);color:var(--text)}
121     .form-actions{display:flex;gap:10px;margin-top:18px}

```

```

122
123     /* ALERT */
124     .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:8px;z-
index:1100;animation:slideIn 0.3s ease;font-size:12px;font-weight:700;font-family:var(--mono)}
125     .alert-success{background:var(--accent-dim);color:var(--accent);border:1px solid
rgba(52,211,153,0.3)}
126     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
127     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
128     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:28px;font-family:var(--
mono);font-size:12px}
129
130     /* RESTAURANT CARDS */
131     .restaurant-grid{display:grid;grid-template-columns:repeat(auto-
fill,minmax(240px,1fr));gap:14px;padding:20px}
132     .restaurant-card{background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:18px;transition:all 0.2s;position:relative;overflow:hidden}
133     .restaurant-
card::before{content:'';position:absolute;top:0;left:0;bottom:0;width:3px;background:var(--accent)}
134     .restaurant-card:hover{border-color:var(--border2);transform:translateX(2px)}
135     .restaurant-card.inactive::before{background:var(--red)}
136     .rest-name{font-weight:700;color:var(--text);margin-bottom:6px;font-size:14px}
137     .rest-meta{font-size:11px;color:var(--text3);font-family:var(--mono);margin-bottom:12px}
138     .rest-actions{display:flex;gap:8px}
139
140     /* CARD */
141     .card{background:var(--panel);border:1px solid var(--border);border-radius:10px;overflow:hidden}
142     .card-head{padding:14px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
143     .card-title{font-size:13px;font-weight:700;color:var(--text);font-family:var(--mono)}
144     .card-sub{font-size:11px;color:var(--text3);margin-top:2px}
145     .card-body{padding:20px}
146     .form-label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px;font-family:var(--mono)}
147     .form-input{background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:9px
12px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-color 0.2s;wid...
148     .form-input:focus{outline:none;border-color:var(--accent)}
149
150     /* SCROLLBAR */
151     ::-webkit-scrollbar{width:6px}
152     ::-webkit-scrollbar-track{background:var(--bg)}
153     ::-webkit-scrollbar-thumb{background:var(--text3);border-radius:3px}
154     ::-webkit-scrollbar-thumb:hover{background:var(--text2)}
155 </style>
156 </head>
157 <body>
158 <div class="app">
159   <!-- SIDEBAR -->
160   <div class="sidebar">
161     <div class="logo">
162       <div class="logo-row">
163         <div class="logo-icon">V</div>
164         <h2>Vendor Hub</h2>
165       </div>
166       <p>EMS Vendor Portal</p>
167       <div class="role-badge">// FOOD VENDOR</div>
168     </div>
169     <nav class="nav">
170       <div class="nav-section">// overview</div>
171       <div class="nav-item active" onclick="showView('dashboard',this)"><span></span> Dashboard</div>
172       <div class="nav-section">// restaurants</div>
173       <div class="nav-item" onclick="showView('my-restaurants',this)"><span></span> My
Restaurants</div>
174       <div class="nav-item" onclick="showView('add-restaurant',this)"><span></span> Add
Restaurant</div>
175       <div class="nav-section">// operations</div>
176       <div class="nav-item" onclick="showView('subscriptions',this)"><span></span>
Subscriptions</div>
177       <div class="nav-item" onclick="showView('deliveries',this)"><span></span> Deliveries</div>
178       <div class="nav-section">// account</div>
179       <div class="nav-item" onclick="showView('profile',this)"><span></span> My Profile</div>
180       <div class="nav-item" onclick="showView('two-factor',this)">
181         <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="7" width="14"
height="8" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M4 7V5a4 4 0 0 1 8 0v2" strok...
182         Two-Factor Auth
183       </div>
184     </nav>

```

```

185     <div class="sidebar-footer">
186         <div class="user-card">
187             <div class="avatar" id="avatar-initials">V</div>
188             <div class="user-info-side">
189                 <div class="name" id="sidebar-name">Vendor</div>
190                 <div class="role-text">// vendor</div>
191             </div>
192         </div>
193     </div>
194 </div>
195
196 <!-- MAIN -->
197 <div class="main">
198     <div class="topbar">
199         <div class="page-title" id="pageTitle">// dashboard</div>
200         <div class="topbar-right">
201             <span style="font-size:11px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
202             <button class="logout-btn" onclick="logout()">SIGN OUT</button>
203         </div>
204     </div>
205     <div class="content">
206
207         <!-- DASHBOARD -->
208         <div id="dashboard-view" class="view active">
209             <div class="stats-grid">
210                 <div class="stat-card green">
211                     <div class="stat-label">Restaurants</div>
212                     <div class="stat-value" id="stat-restaurants">--</div>
213                     <div class="stat-sub">active listings</div>
214                 </div>
215                 <div class="stat-card amber">
216                     <div class="stat-label">Subscriptions</div>
217                     <div class="stat-value" id="stat-subs">--</div>
218                     <div class="stat-sub">active customers</div>
219                 </div>
220                 <div class="stat-card blue">
221                     <div class="stat-label">Deliveries</div>
222                     <div class="stat-value" id="stat-deliveries">--</div>
223                     <div class="stat-sub">total today</div>
224                 </div>
225                 <div class="stat-card red">
226                     <div class="stat-label">Pending</div>
227                     <div class="stat-value" id="stat-pending">--</div>
228                     <div class="stat-sub">deliveries pending</div>
229                 </div>
230             </div>
231             <div style="display:grid;grid-template-columns:1fr 1fr;gap:18px">
232                 <div class="section">
233                     <div class="section-head">
234                         <div><div class="section-title">// my_restaurants</div></div>
235                         <button class="btn btn-secondary btn-sm" onclick="showView('my-
restaurants',document.querySelectorAll('.nav-item')[1])">view </button>
236                     </div>
237                     <div id="dash-restaurants" style="padding:4px 0"></div>
238                 </div>
239                 <div class="section">
240                     <div class="section-head">
241                         <div><div class="section-title">// recent_deliveries</div></div>
242                         <button class="btn btn-secondary btn-sm"
onclick="showView('deliveries',document.querySelectorAll('.nav-item')[4])">view </button>
243                     </div>
244                     <div class="tablewrap">
245                         <table>
246                             <thead><tr><th>Meal</th><th>Slot</th><th>Status</th></tr></thead>
247                             <tbody id="dash-deliveries"><tr><td colspan="3" class="empty-
state">loading...</td></tr></tbody>
248                         </table>
249                     </div>
250                 </div>
251             </div>
252         </div>
253
254         <!-- MY RESTAURANTS -->
255         <div id="my-restaurants-view" class="view">
256             <div class="section">

```

```

257         <div class="section-head">
258             <div><div class="section-title">// my_restaurants</div><div class="section-
sub">Manage your listings</div></div>
259                 <button class="btn btn-secondary btn-sm" onclick="loadMyRestaurants()">
refresh</button>
260             </div>
261         <div class="restaurant-grid" id="restaurants-grid">
262             <div class="empty-state">loading...</div>
263         </div>
264     </div>
265 </div>
266
267 <!-- ADD RESTAURANT -->
268 <div id="add-restaurant-view" class="view">
269     <div class="section">
270         <div class="section-head"><div class="section-title">// add_restaurant</div></div>
271         <div class="form-card">
272             <div class="form-grid">
273                 <div class="form-group full">
274                     <label>Restaurant Name</label>
275                     <input id="rest-name" placeholder="My Restaurant Name">
276                 </div>
277                 <div class="form-group full">
278                     <label>Address</label>
279                     <input id="rest-address" placeholder="Street, area, city">
280                 </div>
281                 <div class="form-group full">
282                     <label>Meal Slots</label>
283                     <div style="display:flex;gap:14px;flex-wrap:wrap;padding:10px
12px;background:var(--bg3);border:1px solid var(--border);border-radius:6px">
284                         <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px;cursor:pointer">
285                             <input type="checkbox" class="rest-slot-check" value="BREAKFAST"
style="width:auto;cursor:pointer"> Breakfast
286                         </label>
287                         <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px;cursor:pointer">
288                             <input type="checkbox" class="rest-slot-check" value="LUNCH"
style="width:auto;cursor:pointer"> Lunch
289                         </label>
290                         <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px;cursor:pointer">
291                             <input type="checkbox" class="rest-slot-check" value="DINNER"
style="width:auto;cursor:pointer"> Dinner
292                         </label>
293                     </div>
294                 </div>
295                 <div class="form-group full">
296                     <label>Description (optional)</label>
297                     <textarea id="rest-desc" rows="2" placeholder="Brief description of your
restaurant..."></textarea>
298                 </div>
299             </div>
300             <div class="form-actions">
301                 <button class="btn btn-primary" onclick="addRestaurant()">CREATE
RESTAURANT</button>
302                 <button class="btn btn-secondary" onclick="clearRestForm()">CLEAR</button>
303             </div>
304         </div>
305     </div>
306 </div>
307
308 <!-- SUBSCRIPTIONS -->
309 <div id="subscriptions-view" class="view">
310     <div class="section">
311         <div class="section-head">
312             <div><div class="section-title">// subscriptions</div><div class="section-
sub">Customers subscribed to your restaurants</div></div>
313             <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">
refresh</button>
314         </div>
315         <div class="tablewrap">
316             <table>
317                 <thead><tr><th>Employee</th><th>Restaurant</th><th>Meal
Slot</th><th>From</th><th>To</th><th>Status</th></tr></thead>
318                 <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-

```

```

state">loading...</td></tr></tbody>
319         </table>
320     </div>
321 </div>
322 </div>
323
324     <!-- DELIVERIES -->
325     <div id="deliveries-view" class="view">
326         <div class="section">
327             <div class="section-head">
328                 <div><div class="section-title">// deliveries</div></div>
329                 <button class="btn btn-secondary btn-sm" onclick="loadDeliveries()">
refresh</button>
330             </div>
331             <div class="tablewrap">
332                 <table>
333                     <thead><tr><th>ID</th><th>Restaurant</th><th>Meal
Slot</th><th>Date</th><th>Status</th><th>Action</th></tr></thead>
334                     <tbody id="deliveries-list"><tr><td colspan="6" class="empty-
state">loading...</td></tr></tbody>
335                     </table>
336                 </div>
337             </div>
338         </div>
339
340     <!-- TWO-FACTOR -->
341     <div id="two-factor-view" class="view">
342         <div class="card" style="max-width:560px">
343             <div class="card-head">
344                 <div>
345                     <div class="card-title">Two-Factor Authentication</div>
346                     <div class="card-sub">Secure your account with an authenticator app</div>
347                 </div>
348                 <span id="totp-status-chip" class="chip">checking...</span>
349             </div>
350             <div class="card-body">
351                 <!-- SETUP FLOW -->
352                 <div id="totp-setup-section">
353                     <p style="font-size:12px;color:var(--text2);margin-bottom:16px;line-
height:1.7;font-family:var(--mono)">
354                         Use Google Authenticator, Authy, or any TOTP app.<br>
355                         Scan the QR code, then enter the 6-digit code to activate.
356                     </p>
357                     <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-
btn">
358                         Generate QR Code
359                     </button>
360                     <div id="qr-area" style="display:none;margin-top:24px;text-align:center">
361                         <img id="qr-img" src="" alt="QR Code"
style="width:180px;height:180px;border-radius:8px;border:1px solid var(--border2);display:block;margin:0 auto">
362                         <div style="margin-top:12px;font-size:10px;color:var(--text3);font-
family:var(--mono);letter-spacing:0.5px">
363                             CAN'T SCAN? ENTER KEY MANUALLY:
364                         </div>
365                         <div id="totp-secret-display" style="font-family:var(--mono);font-
size:12px;color:var(--cyan);background:var(--bg3);border:1px solid var(--border2);border-radius:6px...
366                         <div style="margin-top:20px;text-align:left">
367                             <label class="form-label" style="display:block;margin-bottom:8px">Enter
6-digit code to activate</label>
368                             <input id="totp-verify-input" type="text" inputmode="numeric"
maxlength="6" placeholder="000000" class="form-input" style="text-align:center;font-size:20px;lette...
369                             <button class="btn btn-primary" onclick="verifyAndActivateTotp()"
style="width:100%;margin-top:10px">Activate 2FA</button>
370                             <div id="totp-verify-error" style="display:none;margin-top:8px;font-
size:11px;color:var(--red);font-family:var(--mono)"></div>
371                         </div>
372                     </div>
373                 </div>
374                 <!-- DISABLE OPTION -->
375                 <div id="totp-disable-section" style="display:none">
376                     <div style="background:var(--green-dim);border:1px solid
rgba(52,211,153,0.2);border-radius:6px;padding:12px 16px;font-size:12px;color:var(--green);font-family:var(--
mon...
377                     TWO-FACTOR AUTHENTICATION IS ACTIVE
378                 </div>
379                 <button class="btn btn-danger" onclick="disableTotp()">Disable 2FA</button>

```

```

380         </div>
381     </div>
382 </div>
383 </div>
384
385 <!-- PROFILE -->
386 <div id="profile-view" class="view">
387     <div class="section">
388         <div class="section-head"><div class="section-title">// my_profile</div></div>
389         <div style="padding:24px;display:grid;grid-template-columns:1fr 1fr;gap:16px"
id="vendor-profile">
390             <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">NAME</div><div style="font-family...
391             <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">EMAIL</div><div style="font-famil...
392             <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">VENDOR ID</div><div style="font-f...
393             <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">ROLE</div><div style="font-family...
394         </div>
395     </div>
396 </div>
397
398 </div>
399 </div>
400 </div>
401
402 <script>
403     const API = 'http://localhost:8080';
404     let vendorId = null;
405
406     // Auth check
407     (function() {
408         const token = localStorage.getItem('jwt_token');
409         const authType = localStorage.getItem('auth_type');
410         if (!token && authType !== 'session') {
411             window.location.href = '/google.html';
412             return;
413         }
414         if (token) {
415             try {
416                 const p = JSON.parse(atob(token.split('.')[1]));
417                 const roles = p.roles || p.authorities || '';
418                 if (typeof roles === 'string') {
419                     if (roles.includes('ROLE_EMPLOYEE')) { window.location.href = '/employee-
dashboard.html'; return; }
420                     if (roles.includes('ROLE_MANAGER')) { window.location.href = '/manager-dashboard.html';
return; }
421                     if (roles.includes('ROLE_ADMIN')) { window.location.href = '/dashboard.html'; return; }
422                 } else if (Array.isArray(roles)) {
423                     const roleStr = roles.join(',');
424                     if (roleStr.includes('ROLE_EMPLOYEE')) { window.location.href = '/employee-
dashboard.html'; return; }
425                     if (roleStr.includes('ROLE_MANAGER')) { window.location.href = '/manager-
dashboard.html'; return; }
426                     if (roleStr.includes('ROLE_ADMIN')) { window.location.href = '/dashboard.html'; return;
}
427                 }
428             } catch(e) {
429                 console.error('Token parse error', e);
430             }
431         }
432         initUser();
433     })();
434
435     function getAuthHeader() {
436         const t = localStorage.getItem('jwt_token');
437         return t ? { 'Authorization': `Bearer ${t}` } : {};
438     }
439
440     // FIX 1: apiFetch now throws on any non-2xx response, surfacing real errors
441     async function apiFetch(path, opts = {}) {
442         const res = await fetch(`${API}${path}`, {
443             ...opts,
444             headers: {
445                 'Content-Type': 'application/json',

```



```

446         ...getAuthHeader(),
447         ...(opts.headers || {})
448     },
449     credentials: 'include'
450 });
451
452 if (res.status === 401) {
453     localStorage.removeItem('jwt_token');
454     localStorage.removeItem('auth_type');
455     showAlert('SESSION EXPIRED', 'error');
456     setTimeout(() => window.location.href = '/google.html', 1500);
457     throw new Error('Unauthorized');
458 }
459
460 const text = await res.text();
461
462 // FIX 1: Throw for all other non-OK statuses with the response body as the message
463 if (!res.ok) {
464     let errMsg = `HTTP ${res.status}`;
465     try {
466         const errBody = JSON.parse(text);
467         errMsg += ': ' + (errBody.message || errBody.error || JSON.stringify(errBody));
468     } catch {
469         if (text) errMsg += ': ' + text;
470     }
471     throw new Error(errMsg);
472 }
473
474 try {
475     return text ? JSON.parse(text) : null;
476 } catch {
477     // Response was OK but not valid JSON – return raw text
478     return text || null;
479 }
480 }
481
482 async function initUser() {
483     const token = localStorage.getItem('jwt_token');
484     let email = '';
485     if (token) {
486         try {
487             const p = JSON.parse(atob(token.split('.')[1]));
488             email = p.sub || p.email || '';
489         } catch (e) {}
490     }
491
492     const displayName = email ? email.split('@')[0] : 'Vendor';
493     document.getElementById('avatar-initials').textContent = (displayName ||
494 'V').charAt(0).toUpperCase();
495     document.getElementById('sidebar-name').textContent = displayName;
496     document.getElementById('topbar-email').textContent = email;
497     document.getElementById('p-email').textContent = email || '-';
498
499     try {
500         const me = await apiFetch('/vendors/me');
501         if (me) {
502             vendorId = me.id || me.vendorId || null;
503             document.getElementById('p-name').textContent = me.name || displayName || '-';
504             document.getElementById('p-id').textContent = vendorId || '-';
505             if (me.name) {
506                 document.getElementById('sidebar-name').textContent = me.name;
507                 document.getElementById('avatar-initials').textContent =
508 me.name.charAt(0).toUpperCase();
509             }
510         }
511     } catch (e) {
512         console.error('Could not load vendor profile', e);
513     }
514
515     loadDashboard();
516 }
517
518 // View loader mapping
519 const viewLoaders = {
520     'dashboard': loadDashboard,
521     'my-restaurants': loadMyRestaurants,

```

```

520     'add-restaurant': () => {},
521     'subscriptions': loadSubscriptions,
522     'deliveries': loadDeliveries,
523     'profile': () => {},
524     'two-factor': loadTotpStatus
525   };
526
527   function showView(name, el) {
528     document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
529     const targetView = document.getElementById(`${name}-view`);
530     if (targetView) targetView.classList.add('active');
531
532     document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
533     if (el) el.classList.add('active');
534
535     document.getElementById('pageTitle').textContent = '// ' + name.replace(/-/g, '_');
536
537     if (viewLoaders[name]) viewLoaders[name]();
538   }
539
540   async function loadDashboard() {
541     try {
542       const [rests, subs, deliveries] = await Promise.all([
543         apiFetch('/restaurants/vendor').catch(() => []),
544         apiFetch('/subscriptions').catch(() => []),
545         apiFetch('/deliveries').catch(() => [])
546       ]);
547
548       const restArr = rests || [];
549       const subArr = subs || [];
550       const delArr = deliveries || [];
551
552       document.getElementById('stat-restaurants').textContent = restArr.filter(r => r.active !==
false).length;
553       document.getElementById('stat-subs').textContent = subArr.length;
554       document.getElementById('stat-deliveries').textContent = delArr.length;
555       document.getElementById('stat-pending').textContent = delArr.filter(d => d.status ===
'PENDING').length;
556
557       document.getElementById('dash-restaurants').innerHTML = restArr.length
? restArr.slice(0, 4).map(r => `
558         <div style="padding:12px 20px;border-bottom:1px solid var(--
border);display:flex;justify-content:space-between;align-items:center">
559           <div>
560             <div style="font-weight:600;color:var(--text);font-size:13px">${escHtml(r.name
|| '-')}</div>
561             <div style="font-size:11px;color:var(--text3);font-family:var(--mono);margin-
top:2px">${escHtml((r.supportedMealSlots || []).join(', '))}</div>
562           </div>
563           <span class="chip ${r.active === true ? 'active' : 'inactive'}">${r.active === true
? 'LIVE' : 'OFF'}</span>
564         </div>`)).join('')
: '<div class="empty-state">no restaurants yet</div>';
567
568       document.getElementById('dash-deliveries').innerHTML = delArr.length
? delArr.slice(0, 5).map(d => `
569         <tr>
570           <td>${escHtml(d.restaurantName || d.restaurantId || '-')}</td>
571           <td><span class="chip ${d.mealSlot || ''}.toLowerCase()}>${escHtml(d.mealSlot ||
'-'')}</span></td>
572           <td><span class="chip ${d.status || 'pending'}.toLowerCase()}>${escHtml(d.status
|| '-')}</span></td>
573         </tr>`)).join('')
: '<tr><td colspan="3" class="empty-state">no deliveries</td></tr>';
576     } catch(e) {
577       console.error('Dashboard load error', e);
578     }
579   }
580
581   async function loadMyRestaurants() {
582     try {
583       const rests = await apiFetch('/restaurants/vendor');
584       const arr = rests || [];
585       document.getElementById('restaurants-grid').innerHTML = arr.length
? arr.map(r => {
586         const rid = r.id || r.restaurantId;

```

```

588         return `
589         <div class="restaurant-card ${r.active === false ? 'inactive' : ''}">
590             <div class="rest-name">${escHtml(r.name || '-')}</div>
591             <div class="rest-meta">${escHtml((r.supportedMealSlots || []).join(', '))} · ID:
592             ${rid || '-'}</div>
593             <div style="font-size:12px;color:var(--text2);margin-
bottom:8px">${escHtml(r.address || '-')}</div>
594             <div style="font-size:11px;color:var(--text3);margin-
bottom:12px">${escHtml(r.description || '')}</div>
595             <div class="rest-actions">
596                 ${r.active !== false
? `<button class="btn btn-warning btn-sm"
onclick="deactivateRestaurant('${rid}')">DEACTIVATE</button>`
: `<button class="btn btn-primary btn-sm"
onclick="activateRestaurant('${rid}')">ACTIVATE</button>`}
597             </div>
598         </div>
599     </div>`;
600     }).join('');
601     : `<div class="empty-state">no restaurants found. create one!</div>`;
602 } catch(e) {
603     document.getElementById('restaurants-grid').innerHTML = `<div class="empty-state">could not
load restaurants</div>`;
604 }
605 }
606 }
607
608 async function deactivateRestaurant(id) {
609     try {
610         await apiFetch(`/restaurants/${id}/deactivate`, { method: 'PUT' });
611         showAlert('RESTAURANT DEACTIVATED', 'success');
612         loadMyRestaurants();
613         loadDashboard();
614     } catch(e) {
615         showAlert('FAILED: ' + e.message, 'error');
616     }
617 }
618
619 async function activateRestaurant(id) {
620     try {
621         await apiFetch(`/restaurants/${id}/activate`, { method: 'PUT' });
622         showAlert('RESTAURANT ACTIVATED', 'success');
623         loadMyRestaurants();
624         loadDashboard();
625     } catch(e) {
626         showAlert('FAILED: ' + e.message, 'error');
627     }
628 }
629
630 async function addRestaurant() {
631     const name = document.getElementById('rest-name').value.trim();
632     const address = document.getElementById('rest-address').value.trim();
633     const description = document.getElementById('rest-desc').value.trim();
634     const supportedMealSlots = Array.from(document.querySelectorAll('.rest-slot-check:checked')).map(el
=> el.value);
635
636     if (!name || !address || supportedMealSlots.length === 0) {
637         showAlert('NAME, ADDRESS AND AT LEAST ONE MEAL SLOT REQUIRED', 'error');
638         return;
639     }
640
641     try {
642         await apiFetch('/restaurants', {
643             method: 'POST',
644             body: JSON.stringify({ name, address, description, supportedMealSlots })
645         });
646         showAlert('RESTAURANT CREATED!', 'success');
647         clearRestForm();
648         showView('my-restaurants', document.querySelectorAll('.nav-item')[1]);
649     } catch(e) {
650         showAlert('FAILED: ' + e.message, 'error');
651     }
652 }
653
654 function clearRestForm() {
655     ['rest-name', 'rest-address', 'rest-desc'].forEach(id => document.getElementById(id).value = '');
656     document.querySelectorAll('.rest-slot-check').forEach(el => el.checked = false);

```

```

657     }
658
659     async function loadSubscriptions() {
660         try {
661             const data = await apiFetch('/subscriptions');
662             document.getElementById('subscriptions-list').innerHTML = (data || []).length
663                 ? data.map(s => `
664                     <tr>
665                         <td style="font-family:var(--mono);font-size:12px">${escHtml(s.employeeEmail ||
666 s.employeeId || '-')}</td>
667                         <td>${escHtml(s.restaurantName || s.restaurantId || '-')}</td>
668                         <td><span class="chip ${s.mealSlot || ''}.toLowerCase()">${escHtml(s.mealSlot ||
669 '-')}</span></td>
670                         <td>${escHtml(s.startDate || '-')}</td>
671                         <td>${escHtml(s.endDate || '-')}</td>
672                         <td><span class="chip active">${escHtml(s.status || 'ACTIVE')}</span></td>
673                     </tr>`
674                     ).join('')
675                 : `<tr><td colspan="6" class="empty-state">no subscriptions found</td></tr>`;
676         } catch(e) {
677             document.getElementById('subscriptions-list').innerHTML = `<tr><td colspan="6" class="empty-
678 state">could not load subscriptions</td></tr>`;
679         }
680     }
681
682     async function loadDeliveries() {
683         try {
684             const data = await apiFetch('/deliveries');
685             document.getElementById('deliveries-list').innerHTML = (data || []).length
686                 ? data.map(d => {
687                     const did = d.id || d.deliveryId;
688                     return `
689                         <tr>
690                             <td style="font-family:var(--mono);font-size:11px">#${did || '-'}</td>
691                             <td>${escHtml(d.restaurantName || d.restaurantId || '-')}</td>
692                             <td><span class="chip ${d.mealSlot || ''}.toLowerCase()">${escHtml(d.mealSlot ||
693 '-')}</span></td>
694                             <td>${escHtml(d.deliveryDate || '-')}</td>
695                             <td><span class="chip ${d.status || 'pending'}.toLowerCase()">${escHtml(d.status
696 || '-')}</span></td>
697                             <td>${d.status === 'PENDING' ? `<button class="btn btn-primary btn-sm"
698 onclick="markDelivered('${did}')">MARK DELIVERED</button>` : '-'}</td>
699                         </tr>`;
700                     }).join('')
701                 : `<tr><td colspan="6" class="empty-state">no deliveries found</td></tr>`;
702         } catch(e) {
703             document.getElementById('deliveries-list').innerHTML = `<tr><td colspan="6" class="empty-
704 state">could not load deliveries</td></tr>`;
705         }
706     }
707
708     async function markDelivered(id) {
709         try {
710             await apiFetch(`/deliveries/${id}/delivered`, { method: 'PUT' });
711             showAlert('MARKED AS DELIVERED', 'success');
712             loadDeliveries();
713             loadDashboard();
714         } catch(e) {
715             showAlert('FAILED', 'error');
716         }
717     }
718
719     function logout() {
720         localStorage.removeItem('jwt_token');
721         localStorage.removeItem('refresh_token');
722         localStorage.removeItem('auth_type');
723         localStorage.removeItem('last_role');
724
725         fetch(`${API}/session/logout`, {
726             method: 'POST',
727             credentials: 'include'
728         }).finally(() => {
729             document.cookie = `JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;`;
730             window.location.href = '/google.html';
731         });
732     }

```

```

726     function showAlert(msg, type) {
727         const el = document.createElement('div');
728         el.className = `alert alert-${type}`;
729         el.textContent = msg;
730         document.body.appendChild(el);
731         setTimeout(() => el.remove(), 3000);
732     }
733
734     // TOTP FUNCTIONS
735
736     async function loadTotpStatus() {
737         const chipEl = document.getElementById('totp-status-chip');
738         chipEl.textContent = 'checking...';
739         chipEl.className = 'chip';
740         // Reset QR area on every load
741         document.getElementById('qr-area').style.display = 'none';
742         document.getElementById('totp-verify-input').value = '';
743         document.getElementById('totp-verify-error').style.display = 'none';
744         document.getElementById('generate-qr-btn').textContent = 'Generate QR Code';
745         document.getElementById('generate-qr-btn').disabled = false;
746         try {
747             const data = await apiFetch('/totp/status');
748             if (data.totpEnabled) {
749                 chipEl.textContent = 'ACTIVE';
750                 chipEl.className = 'chip active';
751                 document.getElementById('totp-setup-section').style.display = 'none';
752                 document.getElementById('totp-disable-section').style.display = 'block';
753             } else {
754                 chipEl.textContent = 'DISABLED';
755                 chipEl.className = 'chip inactive';
756                 document.getElementById('totp-setup-section').style.display = 'block';
757                 document.getElementById('totp-disable-section').style.display = 'none';
758             }
759         } catch(e) {
760             chipEl.textContent = 'ERROR';
761             chipEl.className = 'chip inactive';
762         }
763     }
764
765     async function generateTotpQr() {
766         const btn = document.getElementById('generate-qr-btn');
767         btn.textContent = 'Generating...';
768         btn.disabled = true;
769         try {
770             const data = await apiFetch('/totp/setup', { method: 'POST' });
771             document.getElementById('totp-secret-display').textContent = data.secret || '';
772             const qrUri = encodeURIComponent(data.qrCodeUri || '');
773             document.getElementById('qr-img').src =
774                 `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
775             document.getElementById('qr-area').style.display = 'block';
776         } catch(e) {
777             showAlert('Failed to generate QR: ' + e.message, 'error');
778         } finally {
779             btn.textContent = 'Regenerate QR Code';
780             btn.disabled = false;
781         }
782     }
783
784     async function verifyAndActivateTotp() {
785         const code = document.getElementById('totp-verify-input').value.trim();
786         const errEl = document.getElementById('totp-verify-error');
787         errEl.style.display = 'none';
788         if (code.length !== 6) {
789             errEl.textContent = 'Please enter a 6-digit code.';
790             errEl.style.display = 'block';
791             return;
792         }
793         try {
794             await apiFetch('/totp/verify-setup', {
795                 method: 'POST',
796                 body: JSON.stringify({ code })
797             });
798             showAlert('2FA activated – required on next login', 'success');
799             loadTotpStatus();
800         } catch(e) {
801             errEl.textContent = 'Invalid code. Try again.';

```

```
802         errEl.style.display = 'block';
803     }
804 }
805
806 async function disableTotp() {
807     if (!confirm('Are you sure you want to disable two-factor authentication? Your account will be less
secure.')) {
808         return;
809     }
810     try {
811         await apiFetch('/totp/disable', { method: 'DELETE' });
812         showAlert('2FA has been disabled', 'success');
813         loadTotpStatus();
814     } catch(e) {
815         showAlert('Failed to disable 2FA: ' + e.message, 'error');
816     }
817 }
818
819 // HTML escape utility to prevent XSS
820 function escHtml(str) {
821     if (!str) return '';
822     const div = document.createElement('div');
823     div.textContent = str;
824     return div.innerHTML;
825 }
826 </script>
827 </body>
828 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Vendor Login</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#070910;
12             --panel:#0e1118;
13             --panel2:#131821;
14             --border:rgba(255,255,255,0.07);
15             --border-lit:rgba(255,255,255,0.14);
16             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
17             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
18             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
19             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
20             --amber:#f9c74f;--amber-dim:rgba(249,199,79,0.12);
21             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
22         }
23         body{font-family:var(--font);background:var(--bg);color:var(--text);min-
height:100vh;display:flex;overflow:auto;}
24
25         .bg-layer{position:fixed;inset:0;z-index:0;
26             background:
27                 radial-gradient(ellipse 60% 50% at 20% 50%, rgba(248,113,113,0.06) 0%,transparent 70%),
28                 radial-gradient(ellipse 40% 60% at 80% 20%, rgba(249,199,79,0.05) 0%,transparent 60%),
29                 radial-gradient(ellipse 50% 40% at 70% 80%, rgba(248,113,113,0.04) 0%,transparent 60%);
30         }
31         .grid-overlay{position:fixed;inset:0;z-index:0;background-image:linear-
gradient(rgba(255,255,255,0.02) 1px,transparent 1px),linear-gradient(90deg,rgba(255,255,255,0.02)
1px,transparent 1px)...
32
33         /* LEFT */
34         .left-panel{flex:1;display:flex;flex-direction:column;justify-
content:center;padding:60px;position:relative;z-index:1;animation:fadeLeft 0.8s ease both;}
35         @keyframes fadeLeft{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
36         .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:32px}
37         .brand-icon{width:44px;height:44px;background:linear-gradient(135deg,#f87171,#fb923c);border-
radius:12px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size:18p...
38         .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
39         .brand-tagline{font-size:13px;color:var(--text2)}
40         .hero-text h1{font-size:clamp(36px,4vw,50px);font-weight:700;line-height:1.1;letter-spacing:-
1px;color:var(--text);margin-bottom:16px;}
41         .hl-red{color:var(--red)}
42         .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:360px}
43
44         .info-cards{display:flex;flex-direction:column;gap:12px;margin-top:40px;max-width:360px;}
45         .info-card{background:var(--panel);border:1px solid var(--border);border-radius:10px;padding:14px
18px;display:flex;align-items:flex-start;gap:12px;}
46         .info-card-icon{font-size:18px;flex-shrink:0;margin-top:1px}
47         .info-card-text strong{font-size:13px;color:var(--text);display:block;margin-bottom:2px}
48         .info-card-text span{font-size:12px;color:var(--text2)}
49
50         .back-link{display:inline-flex;align-items:center;gap:6px;font-size:12px;color:var(--text3);text-
decoration:none;margin-top:36px;transition:color 0.2s;}
51         .back-link:hover{color:var(--text2)}
52
53         /* RIGHT */
54         .right-panel{width:460px;flex-shrink:0;display:flex;align-items:flex-start;overflow-y:auto;justify-
content:center;padding:32px 40px;position:relative;z-index:1;border-left:1px solid var(--b...
55         @keyframes
fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
56         .form-box{width:100%}
57
58         .vendor-badge{display:inline-flex;align-items:center;gap:8px;background:var(--red-dim);border:1px
solid rgba(248,113,113,0.25);border-radius:20px;padding:6px 14px;font-size:12px;font-weight...
59         .vendor-badge .dot{width:7px;height:7px;border-radius:50%;background:var(--red);animation:pulse 1.4s

```

```

infinite;}}
60     @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
61
62     .form-title{font-size:22px;font-weight:700;color:var(--text);margin-bottom:6px;}
63     .form-subtitle{font-size:13px;color:var(--text2);margin-bottom:28px;}
64
65     /* AUTH TABS */
66     .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-
bottom:24px;border:1px solid var(--border)}
67     .auth-tab{flex:1;padding:9px;border:none;background:transparent;color:var(--text2);font-family:var(-
font);font-weight:600;font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;}
68     .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}
69
70     /* FORM ELEMENTS */
71     .form-group{margin-bottom:16px}
72     .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-
transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
73     .form-input{width:100%;padding:11px 14px;background:var(--bg);border:1px solid var(--border);border-
radius:9px;color:var(--text);font-family:var(--font);font-size:14px;transition:all 0.2s;}
74     .form-input:focus{outline:none;border-color:var(--red);box-shadow:0 0 3px rgba(248,113,113,0.08)}
75     .form-input::placeholder{color:var(--text3)}
76
77     /* BUTTONS */
78     .btn-login{width:100%;padding:12px;border:none;border-radius:9px;font-family:var(--font);font-
size:14px;font-weight:600;cursor:pointer;transition:all 0.2s;margin-top:4px;background:var(--re...
79     .btn-login:hover{background:#f98a8a;box-shadow:0 6px 20px
rgba(248,113,113,0.35);transform:translateY(-1px)}
80     .btn-login:active{transform:translateY(0)}
81     .btn-login.loading{opacity:0.7;pointer-events:none}
82
83     /* STATUS */
84     .status-box{border-radius:9px;padding:10px 14px;margin-top:14px;font-size:13px;font-
weight:500;display:none;animation:fadeIn 0.3s ease;}
85     @keyframes fadeIn{from{opacity:0;transform:translateY(4px)}to{opacity:1;transform:translateY(0)}}
86     .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
87     .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
88     .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid
rgba(79,140,255,0.2)}
89
90     .notice{background:rgba(249,199,79,0.06);border:1px solid rgba(249,199,79,0.15);border-
radius:9px;padding:12px 14px;margin-top:20px;font-size:12px;color:var(--text2);line-height:1.5;}
91     .notice strong{color:var(--amber)}
92
93     /* FOOTER */
94     .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
95     .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
96     .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
97     .tz-select{flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);border-
radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;}
98     .tz-select:focus{outline:none;border-color:var(--border-lit)}
99     .tz-time{font-size:10px;color:var(--red);font-family:var(--mono)}
100
101     /* ---- TOTP OVERLAY ---- */
102     #totp-overlay{
103         display:none;
104         position:fixed;inset:0;z-index:999;
105         background:rgba(7,9,16,0.88);
106         backdrop-filter:blur(10px);
107         align-items:center;justify-content:center;
108     }
109     #totp-overlay.open{ display:flex; }
110
111     .totp-card{
112         background:var(--panel);
113         border:1px solid var(--border-lit);
114         border-radius:20px;
115         padding:40px 36px 32px;
116         width:360px;
117         text-align:center;
118         animation:totpSlideUp 0.3s cubic-bezier(0.34,1.56,0.64,1) both;
119         position:relative;
120     }
121     @keyframes totpSlideUp{from{opacity:0;transform:translateY(24px)
scale(0.96)}to{opacity:1;transform:translateY(0) scale(1)}}
122

```



```

123     .totp-shield{
124         width:56px;height:56px;
125         background:linear-gradient(135deg, rgba(248,113,113,0.2), rgba(251,146,60,0.15));
126         border:1.5px solid rgba(248,113,113,0.3);
127         border-radius:16px;
128         display:flex;align-items:center;justify-content:center;
129         font-size:26px;
130         margin:0 auto 20px;
131     }
132     .totp-title{font-size:18px;font-weight:700;color:var(--text);margin-bottom:8px;}
133     .totp-subtitle{font-size:13px;color:var(--text2);margin-bottom:28px;line-height:1.5;}
134
135     /* Segmented digit boxes – matching google.html aesthetic */
136     .totp-digits{
137         display:flex;gap:8px;justify-content:center;margin-bottom:20px;
138     }
139     .totp-digit{
140         width:44px;height:54px;
141         background:var(--bg);
142         border:1.5px solid var(--border);
143         border-radius:10px;
144         font-size:24px;font-weight:700;
145         color:var(--text);
146         font-family:var(--mono);
147         text-align:center;
148         caret-color:var(--red);
149         transition:border-color 0.15s, box-shadow 0.15s;
150         outline:none;
151     }
152     .totp-digit:focus{border-color:var(--red);box-shadow:0 0 0 3px rgba(248,113,113,0.12);}
153     .totp-digit.filled{border-color:rgba(248,113,113,0.4);}
154
155     /* Progress bar under digits */
156     .totp-progress-wrap{height:3px;background:rgba(255,255,255,0.05);border-radius:2px;margin-
bottom:20px;overflow:hidden;}
157     .totp-progress-bar{height:100%;background:linear-gradient(90deg,var(--red),#fb923c);border-
radius:2px;width:100%;transition:width 1s linear;}
158
159     .totp-error{display:none;margin-bottom:14px;font-size:12px;color:var(--red);font-family:var(--
mono);background:var(--red-dim);border:1px solid rgba(248,113,113,0.2);border-radius:8px;padding...
160
161     .totp-verify-btn{
162         width:100%;padding:12px;border:none;border-radius:10px;
163         background:var(--red);color:#fff;
164         font-family:var(--font);font-size:14px;font-weight:600;
165         cursor:pointer;transition:all 0.2s;
166     }
167     .totp-verify-btn:hover{background:#f98a8a;box-shadow:0 6px 20px
rgba(248,113,113,0.3);transform:translateY(-1px)}
168     .totp-verify-btn:active{transform:translateY(0)}
169     .totp-verify-btn.loading{opacity:0.7;pointer-events:none}
170
171     .totp-cancel{
172         margin-top:10px;width:100%;padding:10px;border:none;
173         background:transparent;color:var(--text2);
174         font-family:var(--font);font-size:13px;cursor:pointer;
175         border-radius:8px;transition:color 0.2s;
176     }
177     .totp-cancel:hover{color:var(--text)}
178
179     /* Divider between digit groups (visual) */
180     .totp-sep{
181         width:12px;height:2px;background:var(--text3);border-radius:1px;align-self:center;flex-
shrink:0;
182     }
183
184     @media(max-width:900px){
185         .left-panel{display:none}
186         .right-panel{width:100%;border-left:none;background:var(--bg)}
187     }
188 </style>
189 </head>
190 <body>
191 <div class="bg-layer"></div>
192 <div class="grid-overlay"></div>
193

```

```

194 <!-- LEFT -->
195 <div class="left-panel">
196     <div class="brand-logo">
197         <div class="brand-icon">V</div>
198         <div>
199             <div class="brand-name">Vendor Hub</div>
200             <div class="brand-tagline">Employee Management System</div>
201         </div>
202     </div>
203     <div class="hero-text">
204         <h1>Manage your<br><span class="hl-red">vendor account</span><br>securely.</h1>
205         <p>Access your deliveries, restaurants, and subscriptions. Your account is created and managed by
the system admin.</p>
206     </div>
207     <div class="info-cards">
208         <div class="info-card">
209             <span class="info-card-icon"></span>
210             <div class="info-card-text">
211                 <strong>Delivery Tracking</strong>
212                 <span>Monitor your active and scheduled meal deliveries in real-time.</span>
213             </div>
214         </div>
215         <div class="info-card">
216             <span class="info-card-icon"></span>
217             <div class="info-card-text">
218                 <strong>Restaurant Management</strong>
219                 <span>View and update restaurant details and meal offerings.</span>
220             </div>
221         </div>
222         <div class="info-card">
223             <span class="info-card-icon"></span>
224             <div class="info-card-text">
225                 <strong>Two-Factor Auth (TOTP)</strong>
226                 <span>Accounts protected with TOTP authenticator for enhanced security.</span>
227             </div>
228         </div>
229     </div>
230     <a class="back-link" href="/google.html"> Back to main login</a>
231 </div>
232
233 <!-- RIGHT FORM -->
234 <div class="right-panel">
235     <div class="form-box">
236
237         <div class="vendor-badge"><span class="dot"></span>Vendor Portal</div>
238         <div class="form-title">Vendor Sign In</div>
239         <div class="form-subtitle">Enter the credentials provided by your system admin.</div>
240
241         <!-- AUTH TABS -->
242         <div class="auth-tabs">
243             <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
244             <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
245         </div>
246
247         <!-- JWT FORM -->
248         <div id="jwt-form">
249             <div class="form-group">
250                 <label class="form-label">Email / Username</label>
251                 <input class="form-input" id="jwt-username" type="text" placeholder="vendor@company.com"
autocomplete="off">
252             </div>
253             <div class="form-group">
254                 <label class="form-label">Password</label>
255                 <input class="form-input" id="jwt-password" type="password" placeholder="....."
autocomplete="current-password">
256             </div>
257             <button class="btn-login" id="jwt-btn" onclick="jwtLogin()">Sign In as Vendor</button>
258         </div>
259
260         <!-- SESSION FORM -->
261         <div id="session-form" style="display:none">
262             <div class="form-group">
263                 <label class="form-label">Email / Username</label>
264                 <input class="form-input" id="session-username" type="text"
placeholder="vendor@company.com">
265             </div>

```

```

266         <div class="form-group">
267             <label class="form-label">Password</label>
268             <input class="form-input" id="session-password" type="password" placeholder=".....">
269         </div>
270         <button class="btn-login" id="session-btn" onclick="sessionLogin()">Sign In with
Session</button>
271     </div>
272
273     <div class="status-box" id="status-box"></div>
274
275     <div class="notice">
276         <strong>Don't have an account?</strong> Vendor accounts are created exclusively by the system
admin. Please contact your administrator to set up access.
277     </div>
278
279     <!-- FOOTER -->
280     <div class="form-footer">
281         <div class="tz-row">
282             <span class="tz-label">TZ</span>
283             <select class="tz-select" id="tz-select" onchange="updateTz()">
284                 <option value="Asia/Kolkata">India – IST UTC+5:30</option>
285                 <option value="UTC">UTC</option>
286                 <option value="America/New_York">New York – EST UTC-5</option>
287                 <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
288                 <option value="Europe/London">London – GMT UTC+0</option>
289                 <option value="Europe/Paris">Paris – CET UTC+1</option>
290                 <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
291                 <option value="Asia/Dubai">Dubai – GST UTC+4</option>
292                 <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
293                 <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
294             </select>
295             <span class="tz-time" id="tz-time">--:--:--</span>
296         </div>
297     </div>
298
299 </div>
300 </div>
301
302 <!-- ===== TOTP MODAL ===== -->
303 <div id="totp-overlay">
304     <div class="totp-card">
305
306         <div class="totp-shield"></div>
307         <div class="totp-title">Two-Factor Verification</div>
308         <div class="totp-subtitle">Enter the 6-digit code from your<br>authenticator app (e.g. Google
Authenticator).</div>
309
310         <!-- Segmented digit inputs -->
311         <div class="totp-digits" id="totp-digits">
312             <input class="totp-digit" id="d0" type="text" inputmode="numeric" maxlength="1"
autocomplete="one-time-code">
313             <input class="totp-digit" id="d1" type="text" inputmode="numeric" maxlength="1">
314             <input class="totp-digit" id="d2" type="text" inputmode="numeric" maxlength="1">
315             <div class="totp-sep"></div>
316             <input class="totp-digit" id="d3" type="text" inputmode="numeric" maxlength="1">
317             <input class="totp-digit" id="d4" type="text" inputmode="numeric" maxlength="1">
318             <input class="totp-digit" id="d5" type="text" inputmode="numeric" maxlength="1">
319         </div>
320
321         <!-- 30s TOTP timer progress bar -->
322         <div class="totp-progress-wrap">
323             <div class="totp-progress-bar" id="totp-progress"></div>
324         </div>
325
326         <div class="totp-error" id="totp-error"></div>
327
328         <button class="totp-verify-btn" id="totp-verify-btn" onclick="submitTotp()">Verify Code</button>
329         <button class="totp-cancel" onclick="cancelTotp()">Back to login</button>
330
331     </div>
332 </div>
333
334 <script>
335     const API = 'http://localhost:8080';
336
337     const DASHBOARDS = {

```

```

338     FOOD_VENDOR: '/vendor-dashboard.html',
339     TECH_VENDOR: '/tech-vendor-dashboard.html'
340 };
341
342 // ---- Already logged in? ----
343 (function(){
344     const token = localStorage.getItem('jwt_token');
345     if(!token) return;
346     try{
347         const role = getRoleFromJWT(token);
348         if(DASHBOARDS[role]) window.location.href = DASHBOARDS[role];
349     }catch(e){}
350 })();
351
352 // ---- Helpers ----
353 function getRoleFromJWT(token){
354     const payload = JSON.parse(atob(token.split('.')[1]));
355     const raw = payload.role || payload.roles || payload.authorities || payload.scope || '';
356     const role = Array.isArray(raw) ? raw[0] : raw;
357     return String(role).replace(/^ROLE_/, '').toUpperCase().trim();
358 }
359
360 function showStatus(type, msg){
361     const box = document.getElementById('status-box');
362     box.className = `status-box ${type}`;
363     box.textContent = msg;
364     box.style.display = 'block';
365 }
366 function hideStatus(){
367     document.getElementById('status-box').style.display = 'none';
368 }
369
370 function switchTab(type){
371     document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
372     document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
373     document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
374     document.getElementById('tab-session').classList.toggle('active', type === 'session');
375     hideStatus();
376 }
377
378 function redirectByRole(role){
379     const dest = DASHBOARDS[role];
380     if(!dest){
381         showStatus('error', 'This portal is for vendors only. Please use the main login page.');
```

382 return false;

383 }

384 showStatus('success', `Login successful! Redirecting to \${role === 'TECH\_VENDOR' ? 'Tech Vendor' : 'Food Vendor'} dashboard...`);

385 setTimeout(() => window.location.href = dest, 700);

386 return true;

387 }

388

389 // ---- JWT LOGIN ----

390 async function jwtLogin(){

391 const username = document.getElementById('jwt-username').value.trim();

392 const password = document.getElementById('jwt-password').value;

393 if(!username || !password){ showStatus('error', 'Please enter your email and password'); return; }

394

395 const btn = document.getElementById('jwt-btn');

396 btn.classList.add('loading'); btn.textContent = 'Signing in...';

397 showStatus('loading', 'Authenticating...');

398

399 try{

400 const res = await fetch(`\${API}/Authenticate`, {

401 method: 'POST',

402 headers: {'Content-Type': 'application/json'},

403 body: JSON.stringify({username, password})

404 });

405

406 if(res.ok){

407 const data = await res.json();

408

409 // TOTP required – open modal

410 if(data.requiresTotp){

411 _preAuthToken = data.preAuthToken;

412 _activeLoginType = 'jwt';

```

413         btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
414         hideStatus();
415         openTotpModal();
416         return;
417     }
418
419     const token = data.accessToken || data.token;
420     const role = getRoleFromJWT(token);
421
422     if(!DASHBOARDS[role]){
423         showStatus('error', 'This portal is for vendors only. Please use the main login
page.');
```

```

424         btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
425         return;
426     }
427     localStorage.setItem('jwt_token', token);
428     localStorage.setItem('refresh_token', data.refreshToken || '');
429     localStorage.setItem('auth_type', 'jwt');
430     localStorage.setItem('last_role', role);
431     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
432     btn.textContent = 'Signed In ';
433     redirectByRole(role);
434 } else {
435     const err = await res.text();
436     showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
437     btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
438 }
439 } catch(e){
440     showStatus('error', 'Connection error: ' + e.message);
441     btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
442 }
443 }
444
445 // ---- SESSION LOGIN ----
446 async function sessionLogin(){
447     const username = document.getElementById('session-username').value.trim();
448     const password = document.getElementById('session-password').value;
449     if(!username || !password){ showStatus('error', 'Please enter your email and password'); return; }
450
451     const btn = document.getElementById('session-btn');
452     btn.classList.add('loading'); btn.textContent = 'Signing in...';
453     showStatus('loading', 'Authenticating...');
454
455     try{
456         const res = await fetch(`${API}/session/login`, {
457             method: 'POST',
458             headers: {'Content-Type': 'application/json'},
459             body: JSON.stringify({username, password}),
460             credentials: 'include'
461         });
462
463         if(res.ok){
464             const data = await res.json().catch(() => ({}));
465
466             // TOTP required for session too
467             if(data.requiresTotp){
468                 _preAuthToken = data.preAuthToken;
469                 _activeLoginType = 'session';
470                 btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
471                 hideStatus();
472                 openTotpModal();
473                 return;
474             }
475
476             // No TOTP – confirm identity via /vendors/me
477             try{
478                 const meRes = await fetch(`${API}/vendors/me`, {credentials: 'include'});
479                 if(meRes.ok){
480                     const me = await meRes.json();
481                     const role = String(me.role || '').replace(/^ROLE_/, '').toUpperCase();
482                     localStorage.setItem('auth_type', 'session');
483                     localStorage.setItem('last_role', role);
484                     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
485                     btn.textContent = 'Signed In ';
486                     redirectByRole(role);
487                     return;

```

```

488     }
489     } catch(e){}
490
491     showStatus('error', 'Could not determine vendor type. Please use JWT login.');
```

492 btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';

493 } else {

494 const err = await res.json().catch(() => ({}));

495 showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));

496 btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';

497 }

498 } catch(e){

499 showStatus('error', 'Connection error: ' + e.message);

500 btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';

501 }

502 }

503

504 // =====

505 // TOTP MODAL

506 // =====

507 let _preAuthToken = null;

508 let _activeLoginType = 'jwt'; // 'jwt' | 'session'

509 let _totpTimer = null;

510

511 function openTotpModal(){

512 // Reset digit boxes

513 for(let i = 0; i < 6; i++){

514 const d = document.getElementById('d' + i);

515 d.value = '';

516 d.classList.remove('filled');

517 }

518 document.getElementById('totp-error').style.display = 'none';

519 document.getElementById('totp-verify-btn').classList.remove('loading');

520 document.getElementById('totp-verify-btn').textContent = 'Verify Code';

521

522 // Start countdown progress bar

523 startTotpProgress();

524

525 document.getElementById('totp-overlay').classList.add('open');

526 setTimeout(() => document.getElementById('d0').focus(), 120);

527 }

528

529 function cancelTotp(){

530 _preAuthToken = null;

531 _activeLoginType = 'jwt';

532 clearInterval(_totpTimer);

533 document.getElementById('totp-overlay').classList.remove('open');

534 }

535

536 // 30-second TOTP cycle progress bar

537 function startTotpProgress(){

538 clearInterval(_totpTimer);

539 const bar = document.getElementById('totp-progress');

540 function tick(){

541 const now = Math.floor(Date.now() / 1000);

542 const secs = 30 - (now % 30);

543 const pct = (secs / 30) * 100;

544 bar.style.width = pct + '%';

545 // Warn when < 7 seconds remaining

546 if(secs <= 7){

547 bar.style.background = 'linear-gradient(90deg,#f87171,#fca5a5)';

548 } else {

549 bar.style.background = 'linear-gradient(90deg,var(--red),#fb923c)';

550 }

551 }

552 tick();

553 _totpTimer = setInterval(tick, 1000);

554 }

555

556 // ---- Digit input navigation ----

557 function initDigitInputs(){

558 const digits = Array.from({length: 6}, (_, i) => document.getElementById('d' + i));

559

560 digits.forEach((inp, idx) => {

561 inp.addEventListener('input', e => {

562 const val = inp.value.replace(/\D/g, '');

563 inp.value = val ? val[val.length - 1] : '';

```

564         inp.classList.toggle('filled', !!inp.value);
565         if(inp.value && idx < 5) digits[idx + 1].focus();
566         // Auto-submit when all filled
567         if(digits.every(d => d.value)) submitTotp();
568     });
569
570     inp.addEventListener('keydown', e => {
571         if(e.key === 'Backspace'){
572             if(!inp.value && idx > 0){
573                 digits[idx - 1].value = '';
574                 digits[idx - 1].classList.remove('filled');
575                 digits[idx - 1].focus();
576             } else {
577                 inp.value = '';
578                 inp.classList.remove('filled');
579             }
580         } else if(e.key === 'ArrowLeft' && idx > 0){
581             digits[idx - 1].focus();
582         } else if(e.key === 'ArrowRight' && idx < 5){
583             digits[idx + 1].focus();
584         } else if(e.key === 'Enter'){
585             submitTotp();
586         }
587     });
588
589     // Handle paste of 6-digit code
590     inp.addEventListener('paste', e => {
591         e.preventDefault();
592         const pasted = (e.clipboardData || window.clipboardData).getData('text').replace(/\D/g,
593         '');
594         if(pasted.length >= 6){
595             for(let i = 0; i < 6; i++){
596                 digits[i].value = pasted[i] || '';
597                 digits[i].classList.toggle('filled', !!digits[i].value);
598             }
599             digits[5].focus();
600             if(pasted.length >= 6) submitTotp();
601         }
602     });
603 }
604
605 function getTotpCode(){
606     return Array.from({length: 6}, (_, i) => document.getElementById('d' + i).value).join('');
607 }
608
609 async function submitTotp(){
610     const code = getTotpCode();
611     const errEl = document.getElementById('totp-error');
612     errEl.style.display = 'none';
613
614     if(code.length !== 6){
615         errEl.textContent = 'Please enter all 6 digits.';
616         errEl.style.display = 'block';
617         return;
618     }
619
620     const btn = document.getElementById('totp-verify-btn');
621     btn.classList.add('loading'); btn.textContent = 'Verifying...';
622
623     try{
624         // JWT TOTP endpoint
625         if(_activeLoginType === 'jwt'){
626             const res = await fetch(`${API}/Authenticate/totp`, {
627                 method: 'POST',
628                 headers: {'Content-Type': 'application/json'},
629                 body: JSON.stringify({ preAuthToken: _preAuthToken, code })
630             });
631             if(res.ok){
632                 const data = await res.json();
633                 const token = data.accessToken || data.token;
634                 const role = getRoleFromJWT(token);
635                 clearInterval(_totpTimer);
636                 document.getElementById('totp-overlay').classList.remove('open');
637                 _preAuthToken = null;
638                 localStorage.setItem('jwt_token', token);

```

```

639         localStorage.setItem('refresh_token', data.refreshToken || '');
640         localStorage.setItem('auth_type', 'jwt');
641         localStorage.setItem('last_role', role);
642         localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
643         showStatus('success', 'Verified! Redirecting...');
644         setTimeout(() => redirectByRole(role), 600);
645     } else {
646         const err = await res.json().catch(() => ({}));
647         errEl.textContent = err.error || 'Invalid code. Try again.';
648         errEl.style.display = 'block';
649         btn.classList.remove('loading'); btn.textContent = 'Verify Code';
650     }
651
652     } else {
653         // Session TOTP endpoint
654         const res = await fetch(`${API}/session/totp`, {
655             method: 'POST',
656             headers: {'Content-Type': 'application/json'},
657             body: JSON.stringify({ preAuthToken: _preAuthToken, code }),
658             credentials: 'include'
659         });
660         if(res.ok){
661             clearInterval(_totpTimer);
662             document.getElementById('totp-overlay').classList.remove('open');
663             _preAuthToken = null;
664             // Re-confirm via /vendors/me
665             try{
666                 const meRes = await fetch(`${API}/vendors/me`, {credentials: 'include'});
667                 if(meRes.ok){
668                     const me = await meRes.json();
669                     const role = String(me.role || '').replace(/^ROLE_/, '').toUpperCase();
670                     localStorage.setItem('auth_type', 'session');
671                     localStorage.setItem('last_role', role);
672                     localStorage.setItem('user_timezone', document.getElementById('tz-
select').value);
673                     showStatus('success', 'Verified! Redirecting...');
674                     setTimeout(() => redirectByRole(role), 600);
675                     return;
676                 }
677             } catch(e){}
678             showStatus('error', 'Verified but could not determine vendor type.');
```

```

679         } else {
680             const err = await res.json().catch(() => ({}));
681             errEl.textContent = err.error || 'Invalid code. Try again.';
682             errEl.style.display = 'block';
683             btn.classList.remove('loading'); btn.textContent = 'Verify Code';
684         }
685     }
686     } catch(e){
687         errEl.textContent = 'Connection error. Try again.';
688         errEl.style.display = 'block';
689         btn.classList.remove('loading'); btn.textContent = 'Verify Code';
690     }
691 }
692
693 // ---- Timezone ----
694 function updateTz(){
695     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
696     refreshTzClock();
697 }
698 function refreshTzClock(){
699     const tz = document.getElementById('tz-select').value;
700     document.getElementById('tz-time').textContent =
701         new Date().toLocaleTimeString('en-GB', {timeZone: tz, hour12: false});
702 }
703
704 // ---- Enter key support ----
705 document.addEventListener('keydown', e => {
706     if(e.key !== 'Enter') return;
707     if(document.getElementById('totp-overlay').classList.contains('open')){
708         submitTotp(); return;
709     }
710     if(document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
711     else sessionLogin();
712 });
713

```



```
714     // ---- Init ----
715     window.onload = () => {
716         const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
717         document.getElementById('tz-select').value = savedTz;
718         refreshTzClock();
719         setInterval(refreshTzClock, 1000);
720         initDigitInputs();
721     };
722 </script>
723 </body>
724 </html>
725
```

217 Current:

src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
1 package com.example.EmployeeManagementSystem;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class EmployeeManagementSystemApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
14
```

```

1  package com.example.EmployeeManagementSystem.service;
2
3
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6  import com.example.EmployeeManagementSystem.Entity.LeaveType;
7  import com.example.EmployeeManagementSystem.Enum.Gender;
8  import com.example.EmployeeManagementSystem.Enum.Role;
9  import com.example.EmployeeManagementSystem.Enum.Status;
10 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
11 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
12 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
13 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
14 import org.junit.jupiter.api.BeforeEach;
15 import org.junit.jupiter.api.Test;
16 import org.mockito.*;
17
18 import java.math.BigDecimal;
19 import java.time.LocalDate;
20 import java.util.List;
21 import java.util.Optional;
22
23 import static org.assertj.core.api.Assertions.assertThat;
24 import static org.mockito.ArgumentMatchers.any;
25 import static org.mockito.Mockito.*;
26
27 class LeaveAccrualServiceTest {
28
29     @Mock
30     EmployeeRepo employeeRepo;
31     @Mock
32     LeaveTypeRepository leaveTypeRepo;
33     @Mock
34     LeaveEntitlementRepository entitlementRepo;
35
36     @InjectMocks
37     LeaveAccrualService service;
38
39     @BeforeEach void setup() { MockitoAnnotations.openMocks(this); }
40
41     @Test
42     void maleEmployee_doesNotAccrueMaternity() {
43         Employee emp = employee(Gender.M);
44         LeaveType maternity = leaveType("MATERNITY", true, Gender.F);
45
46         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
47         when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));
48
49         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
50
51         verify(entitlementRepo, never()).save(any());
52     }
53
54     @Test
55     void femaleEmployee_doesNotAccrueMaternity() {
56         Employee emp = employee(Gender.F);
57         LeaveType maternity = leaveType("MATERNITY", true, Gender.F);
58
59         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
60         when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));
61
62         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
63
64         verify(entitlementRepo, never()).save(any());
65     }
66
67     @Test
68     void employeeJoiningAfterAccrualDate_isSkipped() {
69         Employee emp = employee(Gender.M);
70         emp.setJoined_at(LocalDate.of(2025, 6, 15)); // joins mid-month
71         LeaveType casual = leaveType("CASUAL", false, null);

```

```

72
73     when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
74     when(leaveTypeRepo.findAll()).thenReturn(List.of(casual));
75
76     service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
77
78     verify(entitlementRepo, never()).save(any());
79 }
80
81 @Test
82 void newEmployeeJoiningBeforeMonthEnd_getsInitialSickAndCasualLeave() {
83     Employee emp = employee(Gender.M);
84     emp.setJoined_at(LocalDate.of(2025, 5, 8));
85     LeaveType sick = leaveType("SICK", false, null);
86     sick.setId(1);
87     LeaveType casual = leaveType("CASUAL", false, null);
88     casual.setId(2);
89
90     when(leaveTypeRepo.findAll()).thenReturn(List.of(sick, casual));
91     when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), any()))
92         .thenReturn(Optional.empty());
93     when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
94
95     service.grantInitialMonthlyAccrual(emp, LocalDate.of(2025, 5, 8));
96
97     ArgumentCaptor<LeaveEntitlement> captor = ArgumentCaptor.forClass(LeaveEntitlement.class);
98     verify(entitlementRepo, atLeast(2)).save(captor.capture());
99
100     List<LeaveEntitlement> saved = captor.getAllValues();
101     assertThat(saved).anySatisfy(e -> {
102         assertThat(e.getLeaveType().getName()).isEqualTo("SICK");
103         assertThat(e.getAccruedThisYear()).isEqualToByComparingTo(BigDecimal.ONE);
104     });
105     assertThat(saved).anySatisfy(e -> {
106         assertThat(e.getLeaveType().getName()).isEqualTo("CASUAL");
107         assertThat(e.getAccruedThisYear()).isEqualToByComparingTo(BigDecimal.ONE);
108     });
109 }
110
111 @Test
112 void newEmployeeJoiningInLastWeek_waitsForNextMonthAccrual() {
113     Employee emp = employee(Gender.M);
114     emp.setJoined_at(LocalDate.of(2025, 5, 25));
115
116     service.grantInitialMonthlyAccrual(emp, LocalDate.of(2025, 5, 25));
117
118     verifyNoInteractions(leaveTypeRepo);
119     verify(entitlementRepo, never()).save(any());
120 }
121
122 @Test
123 void newEmployeeJoiningBeforeLastWeek_getsInitialLeave() {
124     Employee emp = employee(Gender.M);
125     emp.setJoined_at(LocalDate.of(2025, 5, 24));
126     LeaveType sick = leaveType("SICK", false, null);
127
128     when(leaveTypeRepo.findAll()).thenReturn(List.of(sick));
129     when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), any()))
130         .thenReturn(Optional.empty());
131     when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
132
133     service.grantInitialMonthlyAccrual(emp, LocalDate.of(2025, 5, 24));
134
135     verify(entitlementRepo, atLeastOnce()).save(any());
136 }
137
138 @Test
139 void activeAdminReceivesMonthlySickAndCasualAccrual() {
140     Employee admin = employee(Gender.M);
141     admin.setRole(Role.ADMIN);
142     LeaveType sick = leaveType("SICK", false, null);
143     sick.setId(1);
144     LeaveType casual = leaveType("CASUAL", false, null);
145     casual.setId(2);
146
147     when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(admin));

```

```

148         when(leaveTypeRepo.findAll()).thenReturn(List.of(sick, casual));
149         when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), any()))
150             .thenReturn(Optional.empty());
151         when(entitlementRepo.save(any())).thenReturn(inv -> inv.getArgument(0));
152
153         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
154
155         ArgumentCaptor<LeaveEntitlement> captor = ArgumentCaptor.forClass(LeaveEntitlement.class);
156         verify(entitlementRepo, atLeast(2)).save(captor.capture());
157
158         assertThat(captor.getAllValues()).anySatisfy(e -> {
159             assertThat(e.getLeaveType().getName()).isEqualTo("SICK");
160             assertThat(e.getAccruedThisYear()).isEqualToByComparingTo(BigDecimal.ONE);
161         });
162         assertThat(captor.getAllValues()).anySatisfy(e -> {
163             assertThat(e.getLeaveType().getName()).isEqualTo("CASUAL");
164             assertThat(e.getAccruedThisYear()).isEqualToByComparingTo(BigDecimal.ONE);
165         });
166     }
167
168     // --- helpers ---
169     private Employee employee(Gender gender) {
170         Employee e = new Employee();
171         e.setEmployeeId(1L);
172         e.setGender(gender);
173         e.setStatus(Status.ACTIVE);
174         e.setRole(Role.EMPLOYEE);
175         e.setJoined_at(LocalDate.of(2023, 1, 1));
176         return e;
177     }
178
179     private LeaveType leaveType(String name, boolean genderRestricted, Gender restriction) {
180         LeaveType lt = new LeaveType();
181         lt.setId(1);
182         lt.setName(name);
183         lt.setGenderRestricted(genderRestricted);
184         lt.setGenderRestriction(restriction);
185         lt.setCarriesForward(!name.equals("SICK"));
186         lt.setMonthlyReset(name.equals("SICK"));
187         return lt;
188     }
189 }
190

```

```

1  package com.example.EmployeeManagementSystem.service;
2
3  import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
7  import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
8  import com.example.EmployeeManagementSystem.Enum.Role;
9  import com.example.EmployeeManagementSystem.Enum.Status;
10 import com.example.EmployeeManagementSystem.Enum.LeaveType;
11 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
12 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
13 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
14 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
15 import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
16 import com.example.EmployeeManagementSystem.Service.NotificationService;
17 import org.junit.jupiter.api.AfterEach;
18 import org.junit.jupiter.api.BeforeEach;
19 import org.junit.jupiter.api.Test;
20 import org.mockito.MockedStatic;
21 import org.mockito.InjectMocks;
22 import org.mockito.Mock;
23 import org.mockito.MockitoAnnotations;
24 import org.springframework.http.HttpStatus;
25 import org.springframework.http.ResponseEntity;
26 import org.springframework.security.core.Authentication;
27
28 import com.example.EmployeeManagementSystem.Util.AuthUtil;
29
30 import java.math.BigDecimal;
31 import java.time.LocalDate;
32 import java.util.List;
33 import java.util.Map;
34 import java.util.Optional;
35
36 import static org.assertj.core.api.Assertions.assertThat;
37 import static org.assertj.core.api.Assertions.assertThatThrownBy;
38 import static org.mockito.ArgumentMatchers.any;
39 import static org.mockito.Mockito.doNothing;
40 import static org.mockito.Mockito.mock;
41 import static org.mockito.Mockito.when;
42
43 class LeaveRequestServiceTest {
44
45     @Mock
46     LeaveRequestRepo leaveRequestRepo;
47     @Mock
48     EmployeeRepo employeeRepo;
49     @Mock
50     LeaveTypeRepository leaveTypeRepository;
51     @Mock
52     LeaveEntitlementRepository leaveEntitlementRepository;
53     @Mock
54     NotificationService notificationService;
55
56     @InjectMocks
57     LeaveRequestService service;
58
59     private MockedStatic<AuthUtil> authUtilMock;
60
61     @BeforeEach
62     void setup() {
63         MockitoAnnotations.openMocks(this);
64         authUtilMock = org.mockito.Mockito.mockStatic(AuthUtil.class);
65     }
66
67     @AfterEach
68     void tearDown() {
69         authUtilMock.close();
70     }
71

```

```

72     @Test
73     void approveLeave_updatesUsedEntitlementDays() {
74         Employee employee = new Employee();
75         employee.setEmployeeId(1L);
76
77         LeaveRequest request = new LeaveRequest();
78         request.setId(10L);
79         request.setEmployee(employee);
80         request.setLeaveType(LeaveType.CASUAL);
81         request.setStartDate(LocalDate.of(2026, 5, 4));
82         request.setEndDate(LocalDate.of(2026, 5, 6));
83         request.setStatus(LeaveStatus.PENDING);
84
85         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
86             new com.example.EmployeeManagementSystem.Entity.LeaveType();
87         casual.setId(2);
88         casual.setName("CASUAL");
89         casual.setCarriesForward(true);
90
91         LeaveEntitlement entitlement = new LeaveEntitlement();
92         entitlement.setEmployee(employee);
93         entitlement.setLeaveType(casual);
94         entitlement.setYear(2026);
95         entitlement.setOpeningBalance(BigDecimal.ZERO);
96         entitlement.setAccruedThisYear(BigDecimal.TEN);
97         entitlement.setUsedThisYear(BigDecimal.ONE);
98         entitlement.setClosingBalance(BigDecimal.ZERO);
99
100        when(leaveRequestRepo.findById(10L)).thenReturn(Optional.of(request));
101        when(leaveRequestRepo.save(any())).thenReturn(inv -> inv.getArgument(0));
102        when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));
103        when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(1L, 2, 2026))
104            .thenReturn(Optional.of(entitlement));
105        when(leaveEntitlementRepository.save(any())).thenReturn(inv -> inv.getArgument(0));
106        doNothing().when(notificationService).notify(any(), any(), any());
107
108        service.approveLeave(10L);
109
110        assertThat(entitlement.getUsedThisYear()).isEqualToByComparingTo("4");
111    }
112
113     @Test
114     void createRequest_allowsCasualLeaveEqualToAvailableBalance() {
115         Authentication authentication = mock(Authentication.class);
116         authUtilMock.when(() -> AuthUtil.extractEmail(authentication)).thenReturn("emp@test.com");
117
118         Employee employee = new Employee();
119         employee.setEmployeeId(1L);
120         employee.setEmail("emp@test.com");
121         employee.setStatus(Status.ACTIVE);
122         employee.setRole(Role.EMPLOYEE);
123         employee.setTimezone("UTC");
124         employee.setName("Emp");
125
126         LeaveRequestDTO dto = new LeaveRequestDTO();
127         dto.setLeaveType(LeaveType.CASUAL);
128         dto.setStartDate(LocalDate.now().plusDays(1));
129         dto.setEndDate(LocalDate.now().plusDays(2));
130         dto.setReason("Personal work");
131
132         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
133             new com.example.EmployeeManagementSystem.Entity.LeaveType();
134         casual.setId(2);
135         casual.setName("CASUAL");
136         casual.setCarriesForward(true);
137
138         LeaveEntitlement entitlement = new LeaveEntitlement();
139         entitlement.setEmployee(employee);
140         entitlement.setLeaveType(casual);
141         entitlement.setYear(dto.getStartDate().getYear());
142         entitlement.setOpeningBalance(BigDecimal.ONE);
143         entitlement.setAccruedThisYear(BigDecimal.valueOf(3));
144         entitlement.setUsedThisYear(BigDecimal.valueOf(2));
145
146         when(employeeRepo.findByEmail("emp@test.com")).thenReturn(Optional.of(employee));
147         when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));

```

```

148         when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
149             1L, 2, dto.getStartDate().getYear()).thenReturn(Optional.of(entitlement)));
150         when(leaveRequestRepo.checkDuplicate(1L, dto.getStartDate(), dto.getEndDate(),
LeaveStatus.PENDING.name()))
151             .thenReturn(0L);
152         when(leaveRequestRepo.countOverlappingLeave(1L, dto.getStartDate(), dto.getEndDate()))
153             .thenReturn(0L);
154         when(leaveRequestRepo.save(any())).thenReturn(inv -> inv.getArgument(0));
155         when(employeeRepo.findByRole(Role.ADMIN)).thenReturn(List.of());
156
157         ResponseEntity<?> response = service.createRequest(authentication, dto);
158
159         assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
160     }
161
162     @Test
163     void createRequest_rejectsCasualLeaveAboveAvailableBalance() {
164         Authentication authentication = mock(Authentication.class);
165         authUtilMock.when(() -> AuthUtil.extractEmail(authentication)).thenReturn("emp@test.com");
166
167         Employee employee = new Employee();
168         employee.setEmployeeId(1L);
169         employee.setEmail("emp@test.com");
170         employee.setStatus(Status.ACTIVE);
171         employee.setRole(Role.EMPLOYEE);
172         employee.setTimezone("UTC");
173
174         LeaveRequestDTO dto = new LeaveRequestDTO();
175         dto.setLeaveType(LeaveType.CASUAL);
176         dto.setStartDate(LocalDate.now().plusDays(1));
177         dto.setEndDate(LocalDate.now().plusDays(3));
178
179         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
180             new com.example.EmployeeManagementSystem.Entity.LeaveType();
181         casual.setId(2);
182         casual.setName("CASUAL");
183         casual.setCarriesForward(true);
184
185         LeaveEntitlement entitlement = new LeaveEntitlement();
186         entitlement.setEmployee(employee);
187         entitlement.setLeaveType(casual);
188         entitlement.setYear(dto.getStartDate().getYear());
189         entitlement.setOpeningBalance(BigDecimal.ONE);
190         entitlement.setAccruedThisYear(BigDecimal.ONE);
191         entitlement.setUsedThisYear(BigDecimal.ONE);
192
193         when(employeeRepo.findByEmail("emp@test.com")).thenReturn(Optional.of(employee));
194         when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));
195         when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
196             1L, 2, dto.getStartDate().getYear()).thenReturn(Optional.of(entitlement)));
197
198         ResponseEntity<?> response = service.createRequest(authentication, dto);
199
200         assertThat(response.getStatusCode()).isEqualTo(HttpStatus.BAD_REQUEST);
201         assertThat(response.getBody()).isInstanceOf(Map.class);
202         assertThat(((Map<?, ?>) response.getBody()).get("error").toString())
203             .contains("exceeds available balance");
204     }
205
206     @Test
207     void approveLeave_rejectsWhenPendingRequestWouldOverdrawBalance() {
208         Employee employee = new Employee();
209         employee.setEmployeeId(1L);
210
211         LeaveRequest request = new LeaveRequest();
212         request.setId(10L);
213         request.setEmployee(employee);
214         request.setLeaveType(LeaveType.CASUAL);
215         request.setStartDate(LocalDate.of(2026, 5, 4));
216         request.setEndDate(LocalDate.of(2026, 5, 6));
217         request.setStatus(LeaveStatus.PENDING);
218
219         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
220             new com.example.EmployeeManagementSystem.Entity.LeaveType();
221         casual.setId(2);
222         casual.setName("CASUAL");

```



```
223     casual.setCarriesForward(true);
224
225     LeaveEntitlement entitlement = new LeaveEntitlement();
226     entitlement.setEmployee(employee);
227     entitlement.setLeaveType(casual);
228     entitlement.setYear(2026);
229     entitlement.setOpeningBalance(BigDecimal.ONE);
230     entitlement.setAccruedThisYear(BigDecimal.ONE);
231     entitlement.setUsedThisYear(BigDecimal.ONE); // only 1 day left
232
233     when(leaveRequestRepo.findById(10L)).thenReturn(Optional.of(request));
234     when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));
235     when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(1L, 2, 2026))
236         .thenReturn(Optional.of(entitlement));
237
238     assertThatThrownBy(() -> service.approveLeave(10L))
239         .assertInstanceOf(IllegalStateException.class)
240         .hasMessageContaining("Insufficient casual leave balance");
241 }
242 }
243
```

```

1 package com.example.EmployeeManagementSystem.service;
2
3 import com.example.EmployeeManagementSystem.DTO.EmployeeDetails;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Enum.Gender;
6 import com.example.EmployeeManagementSystem.Enum.Role;
7 import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
10 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
11 import com.example.EmployeeManagementSystem.Service.ManagerService;
12 import org.junit.jupiter.api.BeforeEach;
13 import org.junit.jupiter.api.Test;
14 import org.mockito.Mock;
15 import org.mockito.MockitoAnnotations;
16 import org.springframework.security.crypto.password.PasswordEncoder;
17
18 import java.time.LocalDate;
19
20 import static org.assertj.core.api.Assertions.assertThat;
21 import static org.mockito.ArgumentMatchers.any;
22 import static org.mockito.Mockito.verify;
23 import static org.mockito.Mockito.when;
24
25 class ManagerServiceTest {
26
27     @Mock
28     EmployeeRepo employeeRepo;
29     @Mock
30     PasswordEncoder passwordEncoder;
31     @Mock
32     LeaveRequestRepo leaveRequestRepo;
33     @Mock
34     DeviceAssignmentRepo deviceAssignmentRepo;
35     @Mock
36     LeaveAccrualService leaveAccrualService;
37
38     ManagerService service;
39
40     @BeforeEach
41     void setup() {
42         MockitoAnnotations.openMocks(this);
43         service = new ManagerService(
44             employeeRepo,
45             passwordEncoder,
46             leaveRequestRepo,
47             deviceAssignmentRepo,
48             leaveAccrualService
49         );
50     }
51
52     @Test
53     void createAdminGrantsInitialSickAndCasualAccrual() {
54         EmployeeDetails details = new EmployeeDetails();
55         details.setName("Project Admin");
56         details.setEmail("admin@example.com");
57         details.setPassword("secret");
58         details.setGender(Gender.F);
59         details.setTimezone("Asia/Kolkata");
60
61         when(passwordEncoder.encode("secret")).thenReturn("encoded-secret");
62         when(employeeRepo.save(any(Employee.class))).thenAnswer(invocation -> {
63             Employee employee = invocation.getArgument(0);
64             employee.setEmployeeId(42L);
65             employee.setJoined_at(LocalDate.of(2026, 5, 13));
66             return employee;
67         });
68
69         Employee saved = service.createAdmin(details);
70
71         assertThat(saved.getRole()).isEqualTo(Role.ADMIN);

```

```
72         assertThat(saved.getGender()).isEqualTo(Gender.F);
73         verify(leaveAccrualService).grantInitialMonthlyAccrual(saved, LocalDate.of(2026, 5, 13));
74     }
75 }
76
```

```

1 package com.example.EmployeeManagementSystem.service;
2
3 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
4 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
5 import org.junit.jupiter.api.BeforeEach;
6 import org.junit.jupiter.api.Test;
7 import org.mockito.*;
8 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
9 import com.example.EmployeeManagementSystem.Entity.LeaveType;
10 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
11 import com.example.EmployeeManagementSystem.Entity.Employee;
12
13 import java.math.BigDecimal;
14 import java.time.LocalDate;
15 import java.time.YearMonth;
16 import java.util.List;
17
18 import static org.assertj.core.api.Assertions.assertThat;
19 import static org.mockito.Mockito.*;
20
21 class SickLeaveResetServiceTest {
22
23     @Mock
24     LeaveTypeRepository leaveTypeRepo;
25     @Mock
26     LeaveEntitlementRepository entitlementRepo;
27     @InjectMocks
28     SickLeaveResetService service;
29
30     @BeforeEach
31     void setup() {
32         MockitoAnnotations.openMocks(this);
33     }
34
35     @Test
36     void unusedSickDays_areZeroedAtMonthEnd() {
37         LeaveType sick = sickLeaveType();
38         LeaveEntitlement entitlement = entitlement(sick, BigDecimal.valueOf(3), BigDecimal.ONE);
39         // accrued=3, used=1, available=2 should be wiped
40
41         when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
42         when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
43             .thenReturn(List.of(entitlement));
44         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
45
46         service.runMonthlyReset(LocalDate.of(2025, 6, 30));
47
48         // After reset: accrued == used, available == 0
49         assertThat(entitlement.getAccruedThisYear())
50             .isEqualToComparingTo(entitlement.getUsedThisYear());
51         assertThat(entitlement.getAvailableBalance())
52             .isEqualToComparingTo(BigDecimal.ZERO);
53     }
54
55     @Test
56     void fullyUsedSickLeave_noChangeNeeded() {
57         LeaveType sick = sickLeaveType();
58         LeaveEntitlement entitlement = entitlement(sick, BigDecimal.ONE, BigDecimal.ONE);
59         // accrued=1, used=1, available=0 nothing to wipe
60
61         when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
62         when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
63             .thenReturn(List.of(entitlement));
64         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
65
66         service.runMonthlyReset(LocalDate.of(2025, 6, 30));
67
68         // accrued unchanged – still equals used
69         assertThat(entitlement.getAccruedThisYear()).isEqualToComparingTo("1");
70     }
71 }

```

```

72 // FIX: New test – ensures reset does NOT alter a negative balance (over-used edge case)
73 @Test
74 void negativeBalance_isNotTouched() {
75     LeaveType sick = sickLeaveType();
76     // accrued=0, used=1, available=-1 (negative – already over-consumed)
77     LeaveEntitlement entitlement = entitlement(sick, BigDecimal.ZERO, BigDecimal.ONE);
78
79     when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
80     when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
81         .thenReturn(List.of(entitlement));
82     when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
83
84     service.runMonthlyReset(LocalDate.of(2025, 6, 30));
85
86     // available is <= 0 so no reset should happen; accrued stays 0
87     assertThat(entitlement.getAccruedThisYear()).isEqualByComparingTo(BigDecimal.ZERO);
88 }
89
90 // FIX: New test – verifies correct year/month is used when resetDate is end-of-month
91 // (guards against the midnight-fire fragility in the scheduler)
92 @Test
93 void resetDate_usesCorrectYearFromPassedDate_notFromSystemClock() {
94     LeaveType sick = sickLeaveType();
95     LeaveEntitlement entitlement = entitlement(sick, BigDecimal.valueOf(2), BigDecimal.ZERO);
96
97     LocalDate endOfJune2025 = YearMonth.of(2025, 6).atEndOfMonth(); // 2025-06-30
98
99     when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
100    when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
101        .thenReturn(List.of(entitlement));
102    when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
103
104    service.runMonthlyReset(endOfJune2025);
105
106    // Should query year=2025, not system year – verified by mock setup using 2025 only
107    verify(entitlementRepo).findByLeaveTypeIdAndYear(sick.getId(), 2025);
108    assertThat(entitlement.getAvailableBalance()).isEqualByComparingTo(BigDecimal.ZERO);
109 }
110
111 // --- helpers ---
112
113 private LeaveType sickLeaveType() {
114     LeaveType lt = new LeaveType();
115     lt.setId(1);
116     lt.setName("SICK");
117     lt.setMonthlyReset(true);
118     lt.setCarriesForward(false);
119     return lt;
120 }
121
122 private LeaveEntitlement entitlement(LeaveType type, BigDecimal accrued, BigDecimal used) {
123     Employee emp = new Employee();
124     emp.setEmployeeId(1L);
125     LeaveEntitlement e = new LeaveEntitlement();
126     e.setEmployee(emp);
127     e.setLeaveType(type);
128     e.setYear(2025);
129     e.setOpeningBalance(BigDecimal.ZERO);
130     e.setAccruedThisYear(accrued);
131     e.setUsedThisYear(used);
132     e.setClosingBalance(BigDecimal.ZERO);
133     return e;
134 }
135 }

```

```

1 package com.example.EmployeeManagementSystem.service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveType;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
9 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
10 import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
11 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
12
13 import org.junit.jupiter.api.BeforeEach;
14 import org.junit.jupiter.api.Test;
15
16 import org.mockito.InjectMocks;
17 import org.mockito.Mock;
18 import org.mockito.MockitoAnnotations;
19
20 import java.math.BigDecimal;
21 import java.util.List;
22 import java.util.Optional;
23
24 import static org.mockito.ArgumentMatchers.any;
25 import static org.mockito.ArgumentMatchers.eq;
26
27 import static org.mockito.Mockito.when;
28 import static org.mockito.Mockito.verify;
29 import static org.mockito.Mockito.never;
30 import static org.mockito.Mockito.atLeastOnce;
31
32 import static org.assertj.core.api.Assertions.assertThat;
33 class YearEndCarryForwardServiceTest {
34
35     @Mock
36     EmployeeRepo employeeRepo;
37     @Mock
38     LeaveEntitlementRepository entitlementRepo;
39     @Mock
40     YearEndCarryForwardLogRepository logRepo;
41     @Mock
42     LeaveWarningRepository warningRepo;
43     @InjectMocks
44     YearEndCarryForwardService service;
45
46     @BeforeEach
47     void setup() { MockitoAnnotations.openMocks(this); }
48
49     @Test
50     void whenTotalUnderCap_noCapApplied() {
51         Employee emp = employee();
52         LeaveType casual = carryForwardType("CASUAL");
53
54         // casual: opening=0, accrued=8, used=2 closing=6
55         LeaveEntitlement ent = entitlement(emp, casual, 0, 8, 2);
56
57         when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
58         when(employeeRepo.findById(Status.ACTIVE)).thenReturn(List.of(emp));
59         when(entitlementRepo.findByIdEmployeeEmployeeIdAndYear(emp.getEmployeeId(), 2025))
60             .thenReturn(List.of(ent));
61         when(entitlementRepo.findByIdEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
62             .thenReturn(Optional.empty());
63         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
64         when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
65
66         service.runYearEnd(2025);
67
68         // closing balance = 6, no cap needed, nothing wasted
69         assertThat(ent.getClosingBalance()).isEqualToByComparingTo("6");
70         verify(warningRepo, never()).save(any());
71     }

```

72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137

```
@Test
void whenTotalExceedsCap_capAppliedProportionally() {
    Employee emp = employee();
    LeaveType casual = carryForwardType("CASUAL");
    LeaveType maternity = carryForwardType("MATERNITY");

    // casual: closing=20, maternity: closing=16 total=36, over cap by 6
    LeaveEntitlement casualEnt = entitlement(emp, casual, 0, 20, 0);
    LeaveEntitlement maternityEnt = entitlement(emp, maternity, 0, 16, 0);

    when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
    when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
    when(entitlementRepo.findByEmployeeEmployeeIdAndYear(emp.getEmployeeId(), 2025))
        .thenReturn(List.of(casualEnt, maternityEnt));
    when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
        .thenReturn(Optional.empty());
    when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
    when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

    service.runYearEnd(2025);

    // casual: 20/36 * 30 = 16.67, maternity: 16/36 * 30 = 13.33, total = 30
    BigDecimal casualCapped = casualEnt.getClosingBalance();
    BigDecimal maternityCapped = maternityEnt.getClosingBalance();
    BigDecimal total = casualCapped.add(maternityCapped);

    assertThat(total).isEqualByComparingTo("30.00");
    verify(warningRepo, atLeastOnce()).save(any()); // wasted warning issued
}

@Test
void idempotencyGuard_preventsDoubleRun() {
    when(logRepo.existsByProcessedYear(2025)).thenReturn(true);

    service.runYearEnd(2025);

    verify(employeeRepo, never()).findByStatus(any());
}

// --- helpers ---
private Employee employee() {
    Employee e = new Employee(); e.setEmployeeId(1L); return e;
}

private LeaveType carryForwardType(String name) {
    LeaveType lt = new LeaveType();
    lt.setId(name.equals("CASUAL") ? 2 : 3);
    lt.setName(name);
    lt.setCarriesForward(true);
    lt.setMonthlyReset(false);
    return lt;
}

private LeaveEntitlement entitlement(Employee emp, LeaveType type,
    int opening, int accrued, int used) {
    LeaveEntitlement e = new LeaveEntitlement();
    e.setEmployee(emp); e.setLeaveType(type); e.setYear(2025);
    e.setOpeningBalance(BigDecimal.valueOf(opening));
    e.setAccruedThisYear(BigDecimal.valueOf(accrued));
    e.setUsedThisYear(BigDecimal.valueOf(used));
    e.setClosingBalance(BigDecimal.ZERO);
    return e;
}
}
```